

EML-1 release, ProofNet: the corpus, the judgements, and what was thrown away

What this is

287 generated theorems, each with a complete Lean proof. Every one carries the reasoning behind each judgement it received, including the passes that disagreed, and the result of every mechanical check. The rows that did not make it are summarised here and listed in full in `eml_v1_rejected.jsonl`, because a corpus that shows only its survivors cannot be checked.

Canonical artifact. The Lean statements and certificates are the source of truth; this document is a review rendering.

Contents

1	What was thrown away	2
2	How a row got here	3
3	How much a verdict is worth	4
4	The corpus	5

1 What was thrown away

613 certified rows were considered; 287 were admitted. Missing the bar happens three ways, and they are not the same finding: a row both passes reject is a defect the pipeline caught, a row they split on is a fact about the judge, and a row both keep but neither calls strong is simply below the bar this release sets.

Outcome	Rows
admitted — strong from both passes	287
rejected by both passes	133
passes disagreed on keep/reject	123
kept by both passes but not strong in both	45
Lean proved it without a hypothesis it states	11
held back by hand: a binder that can be substituted away	6
prose does not describe the elaborated goal	5
goal round-trip never returned a verdict	3

Among the 133 rows both passes rejected, the named failure modes were:

Failure	Rows	What it means
recall	65	solvable by recalling the parent
parallel	28	the parents are proved separately and joined at the end
decoration	23	the change is cosmetic; the parent's proof skeleton still closes it
universal_supplier	4	one parent proves the child on its own, so the other contributes nothing
not_equivalent	3	
weakening	3	the child is weaker than its parent
repeated_device	2	the same device as a sibling already kept from these parents
exists_unique_wrapper	1	an existing result rewrapped as an existence-and-uniqueness claim
wrong_tier	1	
constant_supplier	1	a parent is used only as a constant, not as an argument
conjunction	1	two unrelated claims stapled together
unlabelled	1	

2 How a row got here

Each gate, and what kind of thing settles it. Lean decides the questions that have an answer; a model is asked only the questions that do not.

Check	By	Coverage	What it establishes
statement_type_check	Lean	287/287	The statement elaborates under Mathlib with <code>autoImplicit false</code> , so no free variable is silently invented.
proof_accepted	Lean	287/287	<code>take env</code> lean accepts the proof and it contains no <code>sorry</code> .
axiom_audit	Lean	287/287	<code>#print axioms</code> lists what the proof depends on; anything outside Lean's three standard axioms fails the row.
vacuity	Lean	287/287	The hypotheses are jointly satisfiable, so the theorem is not true merely by having no instances.
dead_hypotheses	Lean	287/287	Each hypothesis whose name the proof never writes is removed and the file recompiled; a proof that still closes proves the hypothesis was dead. Repeated to a fixpoint, because removing one can make another dead.
corpus_dedup	hash	287/287	The statement, alpha-normalised, is absent from every earlier campaign and from this one.
redundancy	Lean	283/283 applicable	Whether one parent already carries the child: for a crossover, that one parent proves it alone; for a mutation, that the parent gives it directly. Finding such a proof is decisive; failing to find one is not, and the inline probe runs without a prover so it reaches only what the tactic ladder can close.
hypothesis_preservation	parser	4/4 applicable	Silent mutations only. The parent's and child's binders are compared after alpha-normalisation. It cannot tell a hypothesis that was re-encoded from one that was dropped, so a finding is read, not obeyed.
comparator	Lean	287/287	An independent runner rebuilds the statement from the row alone, compares it against the submitted proof with <code>lean4export</code> , checks the axiom allowlist, and replays the proof through the Lean kernel. This is what separates <code>proof_checked</code> from <code>kernel_replayed</code> .
goal_roundtrip	model	287/287	Lean's elaborated goal goes to an informalizer that sees no prose, and a separate judge compares its output with the stated problem. Neither model sees the other's input.

3 How much a verdict is worth

The judge was run twice over all 1233 rows — same inputs, same order, no shared state — so that its disagreement with itself could be measured rather than assumed.

Measure	Agreement
keep or reject	984/1233 (79%)
exact quality grade	883/1233 (71%)
failure label, when both rejected	215/275 (78%)

pass 1 ↓ / pass 2 →	strong	acceptable	weak
strong	639	54	93
acceptable	50	22	36
weak	85	32	222

Admission requires the first cell: strong twice. That is why the release is 287 rows and not 613.

4 The corpus

4.1 001. common_topology_open_iff

The problem

Let $\{\mathcal{T}_i\}_{i \in I}$ be a family of topologies on X . Show that there is a largest topology contained in every $\mathcal{T}_i$, and that a set $A \subseteq X$ is open in it exactly when every point of A has a neighborhood open in every $\mathcal{T}_i$ and contained in A .

Lean statement

```
theorem common_topology_open_iff (X I : Type*) (T : I → TopologicalSpace X) (A : Set X) :
  ∃ τ : TopologicalSpace X,
    (∀ U : Set X, @IsOpen X τ U ↔ ∀ i, @IsOpen X (T i) U) ∧
    (∀ σ : TopologicalSpace X,
      (∀ i U, @IsOpen X σ U → @IsOpen X (T i) U) →
        ∀ U, @IsOpen X σ U → @IsOpen X τ U) ∧
    (@IsOpen X τ A ↔
      ∀ x ∈ A, ∃ U : Set X, x ∈ U ∧
        (∀ i, @IsOpen X (T i) U) ∧ U ⊆ A)
```

Parent 1. Let X be a topological space; let A be a subset of X . Suppose that for each $x \in A$ there is an open set U containing x such that $U \subset A$. Show that A is open in X .

Parent 1 in Lean

```
theorem Munkres_exercise_13_1 (X : Type*) [TopologicalSpace X] (A : Set X)
  (h1 : ∀ x ∈ A, ∃ U : Set X, x ∈ U ∧ IsOpen U ∧ U ⊆ A) :
  IsOpen A := by
```

What it contributes. Parent 3 supplies the canonical largest common lower topology; parent 1 supplies the local-witness-to-openness argument inside that topology, yielding a single iff characterization.

Parent 2. Let $\mathcal{T}_\alpha$ be a family of topologies on X . Show that there is a unique largest topology on X contained in all the collections $\mathcal{T}_\alpha$.

Parent 2 in Lean

```
theorem Munkres_exercise_13_4b2 (X I : Type*) (T : I → Set (Set X)) (h : ∀ i, is_topology X (T i)) :
  ∃! T', is_topology X T' ∧ (∀ i, T' ⊆ T i) ∧
  ∀ T'', is_topology X T'' → (∀ i, T'' ⊆ T i) → T'' ⊆ T' := by
```

What it contributes. Parent 3 supplies the canonical largest common lower topology; parent 1 supplies the local-witness-to-openness argument inside that topology, yielding a single iff characterization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely uses both parents: Parent 2 constructs the topology of sets open in every member of the family and proves its maximality, while Parent 1 supplies the local-neighborhood characterization inside that constructed topology. The local iff is not an independent conjunction or parallel use of the parent results.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely combines the parents: it constructs the topology of sets open in every member topology, then uses the local-neighborhood criterion to characterize when A is open in that constructed topology. Neither the common-topology construction nor the local openness argument alone yields the stated iff.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child constructs the topology of sets open in every parent topology, using Parent 2’s common-lower-topology construction, and then transfers Parent 1’s local-neighborhood criterion into that constructed topology. The local witness condition cannot be established independently of the topology supplied by the family bridge.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2339 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	MISMATCH: Clause (2) fails to specify the relevant topologies for its ‘open’ predicates, so it does not clearly assert that every topology contained in all T_i is contained in τ ; as written, it is tautological or ill-formed rather than the required maximality statement.

Goal round-trip: what Lean elaborated

```
theorem common_topology_open_iff.extracted_1_1.{u_2, u_1} (X : Type u_1) (I : Type u_2) (T : I → TopologicalSpace X)
(A : Set X) :
  ∃ τ,
    (∀ (U : Set X), IsOpen U ↔ ∀ (i : I), IsOpen U) ∧
    (∀ (σ : TopologicalSpace X), (∀ (i : I) (U : Set X), IsOpen U → IsOpen U) → ∀ (U : Set X), IsOpen U → IsOpen U) ∧
    (IsOpen A ↔ ∀ x ∈ A, ∃ U, x ∈ U ∧ (∀ (i : I), IsOpen U) ∧ U ⊆ A)
```

Read back by a model that saw only the Lean. For types X and I , a family of topological spaces T indexed by I on X , and a set $A \subseteq X$, there exists a topology τ on X such that: (1) for every subset U of X , U is open if and only if U is open for every index $i \in I$; (2) for every topology σ on X , if for every $i \in I$ and every subset U of X , U being open implies U being open, then for every subset U of X , U being open implies U being open; and (3) A is open if and only if every point $x \in A$ has a subset U of X such that $x \in U$, U is open for every $i \in I$, and $U \subseteq A$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem common_topology_open_iff (X I : Type*) (T : I → TopologicalSpace X) (A : Set X) :
  ∃ τ : TopologicalSpace X,
    (∀ U : Set X, @IsOpen X τ U ↔ ∀ i, @IsOpen X (T i) U) ∧
    (∀ σ : TopologicalSpace X,
      (∀ i U, @IsOpen X σ U → @IsOpen X (T i) U) →
      ∀ U, @IsOpen X σ U → @IsOpen X τ U) ∧
    (@IsOpen X τ A ↔
      ∀ x ∈ A, ∃ U : Set X, x ∈ U ∧
        (∀ i, @IsOpen X (T i) U) ∧ U ⊆ A) := by
  let τ : TopologicalSpace X :=
  { IsOpen := fun U => ∀ i, @IsOpen X (T i) U
    isOpen_univ := by
      intro i
      exact (T i).isOpen_univ
    isOpen_inter := by
      intro s t hs ht i
      exact (T i).isOpen_inter s t (hs i) (ht i)
    isOpen_sUnion := by
      intro S hS i
      exact (T i).isOpen_sUnion S (by
```

```

      intro s hs
      exact hS s hs i) }
have h_common : ∀ U : Set X,
  @IsOpen X τ U ↔ ∀ i, @IsOpen X (T i) U := by
  intro U
  rfl
have h_max : ∀ σ : TopologicalSpace X,
  (∀ i U, @IsOpen X σ U → @IsOpen X (T i) U) →
  ∀ U, @IsOpen X σ U → @IsOpen X τ U := by
  intro σ hσ U hU
  exact (h_common U).2 (fun i => hσ i U hU)
have h_local :
  (∀ x ∈ A, ∃ U : Set X, x ∈ U ∧
    (∀ i, @IsOpen X (T i) U) ∧ U ⊆ A) →
  @IsOpen X τ A := by
  intro hA
  letI : TopologicalSpace X := τ
  change IsOpen A
  refine isOpen_iff_forall_mem_open.mpr ?_
  intro x hx
  rcases hA x hx with ⟨U, hxU, hU, hUA⟩
  refine ⟨U, hUA, ?_, hxU⟩
  exact (h_common U).2 hU
have h_iff :
  @IsOpen X τ A ↔
  ∀ x ∈ A, ∃ U : Set X, x ∈ U ∧
    (∀ i, @IsOpen X (T i) U) ∧ U ⊆ A := by
  constructor
  · intro hA x hx
    refine ⟨A, hx, ?_, Set.Subset.rfl⟩
    exact (h_common A).1 hA
  · exact h_local
exact ⟨τ, h_common, h_max, h_iff⟩

```

Operator. crossover / crossover_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_13_1__x__Munkres_exercise_13_4b2__xe__62f50f2f

Fusion mechanism. goal_form_transplant Parent 3 supplies the canonical largest common lower topology; parent 1 supplies the local-witness-to-openness argument inside that topology, yielding a single iff characterization.

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 62f50f2fa78706b1

4.2 002. isOpenMap_fst_restricted_bounded_union

The problem

Let X and Y be topological spaces, let $n \in \mathbb{N}$, and let $(A_i)_{i=1}^n$ be a finite family of subsets of $X \times Y$ such that every point of each A_i has an open neighborhood contained in A_i . Prove that the first-coordinate projection, restricted to $\bigcup_{i=1}^n A_i$, is an open map.

Lean statement

```
theorem isOpenMap_fst_restricted_bounded_union (X Y : Type*) [TopologicalSpace X] [TopologicalSpace Y] (n : ℕ) (A : Fin n → Set (X × Y))
  (h : ∀ i x, x ∈ A i → ∃ U : Set (X × Y), x ∈ U ∧ IsOpen U ∧ U ⊆ A i) : IsOpenMap (fun p : (U i : Fin n, A i) => p.1.1)
```

Parent 1. A bounded indexed family of locally open sets has an open union.

Parent 1 in Lean

```
theorem bounded_family_union_isOpen (X : Type*) [TopologicalSpace X] (n : ℕ)
  (A : Fin n → Set X)
  (h : ∀ i x, x ∈ A i → ∃ U : Set X, x ∈ U ∧ IsOpen U ∧ U ⊆ A i) :
  IsOpen (U i : Fin n, A i)
```

What it contributes. The local-openness checkpoint produces an open restricted domain; the product-coordinate open-map checkpoint is then applied to open subsets of that domain to establish one restricted-map theorem.

Parent 2. A map $f : X \rightarrow Y$ is said to be an open map if for every open set U of X , the set $f(U)$ is open in Y . Show that $\pi_1 : X \times Y \rightarrow X$ and $\pi_2 : X \times Y \rightarrow Y$ are open maps.

Parent 2 in Lean

```
theorem Munkres_exercise_16_4 {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (π₁ : X × Y → X)
  (π₂ : X × Y → Y)
  (h₁ : π₁ = Prod.fst)
  (h₂ : π₂ = Prod.snd) :
  IsOpenMap π₁ ∧ IsOpenMap π₂ := by
```

What it contributes. The local-openness checkpoint produces an open restricted domain; the product-coordinate open-map checkpoint is then applied to open subsets of that domain to establish one restricted-map theorem.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

Parent 1 supplies openness of the union, which is used to make the subtype inclusion an open map; Parent 2's projection result then composes with that inclusion to prove the restricted first-coordinate map is open. This is a genuine sequential pipeline, not a conjunction or arithmetic combination.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

Parent 1 supplies openness of the union, which is transformed into openness of the subtype inclusion; Parent 2 supplies openness of the first projection, and the two are genuinely composed to obtain the restricted map.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely pipelines the parents: Parent 1 proves the union is open, which makes the subtype inclusion an open map; Parent 2 then supplies openness of the first projection, and composition yields openness of the restricted first-coordinate map.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1957 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem isOpenMap_fst_restricted_bounded_union.extracted_1_1.{u_2, u_1} (X : Type u_1) (Y : Type u_2)
  [inst : TopologicalSpace X] [inst_1 : TopologicalSpace Y] (n : ℕ) (A : Fin n → Set (X × Y))
  (h : ∀ (i : Fin n), ∀ x ∈ A i, ∃ U, x ∈ U ∧ isOpen U ∧ U ⊆ A i) : isOpenMap fun p => (↑p).1
```

Read back by a model that saw only the Lean. *Let X and Y be topological spaces, let n be a natural number, and let A assign to each $i \in \text{Fin } n$ a subset $A(i)$ of $X \times Y$. Assume that for every $i \in \text{Fin } n$ and every $x \in A(i)$, there exists an open set U in $X \times Y$ such that $x \in U$ and $U \subseteq A(i)$. Then the map from the subtype of $\bigcup i : \text{Fin } n, A(i)$ to X that sends each point p to the first coordinate of its underlying point in $X \times Y$ is an open map.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem isOpenMap_fst_restricted_bounded_union (X Y : Type*) [TopologicalSpace X] [TopologicalSpace Y] (n : ℕ) (A : Fin n → Set (X × Y))
  (h : ∀ i x, x ∈ A i → ∃ U : Set (X × Y), x ∈ U ∧ isOpen U ∧ U ⊆ A i) : isOpenMap (fun p : (U i : Fin n, A i) => p.1.1) := by
  have hUnion : isOpen (U i : Fin n, A i) := by
    refine isOpen_iff_forall_mem_open.mpr ?_
    intro x hx
    have hx_index : ∃ i : Fin n, x ∈ A i := by
      simpa only [Set.mem_iUnion] using hx
    rcases hx_index with ⟨i, hxi⟩
    rcases h i x hxi with ⟨U, hxU, hUopen, hUsub⟩
    have hUnion : U ⊆ U i : Fin n, A i := by
      intro y hy
      exact Set.mem_iUnion.mpr ⟨i, hUsub hy⟩
    exact ⟨U, hUnion, hUopen, hxU⟩
  have hSubtype : isOpenMap ((↑) : (U i : Fin n, A i) → X × Y) :=
    hUnion.isOpenMap_subtype_val
  have hRestricted : isOpenMap (Prod.fst ∘ ((↑) : (U i : Fin n, A i) → X × Y)) :=
    isOpenMap_fst.comp hSubtype
  simpa only [Function.comp_apply] using hRestricted
```

Operator. crossover / crossover_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_13_1__x__Munkres_exercise_16_4__xe__8a753487

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 82965c7c7c358aa6

4.3 003. first_projection_image_isOpen

The problem

Let X and Y be topological spaces, and let A be a subset of $X \times Y$. Suppose that for each $(x, y) \in A$ there is an open set U containing (x, y) such that $U \subset A$. Show that the set of first coordinates of points in A is open in X .

Lean statement

```
theorem first_projection_image_isOpen {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] (A : Set (X × Y)) (hlocal : ∀ x ∈ A, ∃ U :
  ↳ Set (X × Y), x ∈ U ∧ IsOpen U ∧ U ⊆ A) : IsOpen (Prod.fst '' A)
```

Parent 1. Let X be a topological space; let A be a subset of X . Suppose that for each $x \in A$ there is an open set U containing x such that $U \subset A$. Show that A is open in X .

Parent 1 in Lean

```
theorem Munkres_exercise_13_1 (X : Type*) [TopologicalSpace X] (A : Set X)
  (h1 : ∀ x ∈ A, ∃ U : Set X, x ∈ U ∧ IsOpen U ∧ U ⊆ A) :
  IsOpen A := by
```

What it contributes. The locally open subset becomes an open domain object; openness of Prod.fst transports that object to an open projected image.

Parent 2. A map $f : X \rightarrow Y$ is said to be an open map if for every open set U of X , the set $f(U)$ is open in Y . Show that $\pi_1 : X \times Y \rightarrow X$ and $\pi_2 : X \times Y \rightarrow Y$ are open maps.

Parent 2 in Lean

```
theorem Munkres_exercise_16_4 {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (π₁ : X × Y → X)
  (π₂ : X × Y → Y)
  (h₁ : π₁ = Prod.fst)
  (h₂ : π₂ = Prod.snd) :
  IsOpenMap π₁ ∧ IsOpenMap π₂ := by
```

What it contributes. The locally open subset becomes an open domain object; openness of Prod.fst transports that object to an open projected image.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child genuinely pipelines the parents: the local-neighborhood condition first yields $\text{IsOpen } A$, and the first-projection open-map result then transforms that into $\text{IsOpen } (\text{Prod.fst } '' A)$. Neither parent alone supplies the complete argument.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely pipelines both parents: Parent 1 turns the local-neighborhood hypothesis into $\text{IsOpen } A$, and Parent 2's projection open-map result then consumes that fact to establish $\text{IsOpen } (\text{Prod.fst } '' A)$. Neither parent alone supplies the complete argument.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely pipelines both parents: Parent 1 yields $\text{IsOpen } A$, which is then consumed as the openness hypothesis by Parent 2's open-map result for Prod.fst to prove the image is open. Neither parent alone establishes the child's conclusion.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1943 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem first_projection_image_isOpen.extracted_1_1.{u_2, u_1} {X : Type u_1} {Y : Type u_2} [inst : TopologicalSpace X]
[inst_1 : TopologicalSpace Y] (A : Set (X × Y)) (hlocal : ∀ x ∈ A, ∃ U, x ∈ U ∧ IsOpen U ∧ U ⊆ A) :
IsOpen (Prod.fst '' A)
```

Read back by a model that saw only the Lean. *Let X and Y be topological spaces, and let A be a subset of $X \times Y$. Suppose that for every point x in A , there exists a subset U of $X \times Y$ such that x belongs to U , U is open, and U is contained in A . Then the image of A under the first projection map from $X \times Y$ to X is open in X .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem first_projection_image_isOpen {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] (A : Set (X × Y)) (hlocal : ∀ x ∈ A, ∃ U :
  Set (X × Y), x ∈ U ∧ IsOpen U ∧ U ⊆ A) : IsOpen (Prod.fst '' A) := by
  have hA : IsOpen A := by
    refine isOpen_iff_forall_mem_open.mpr ?_
    intro x hx
    rcases hlocal x hx with ⟨U, hxU, hU, hUA⟩
    exact ⟨U, hUA, hU, hxU⟩
  have hfst : IsOpenMap (Prod.fst : X × Y → X) := isOpenMap_fst
  have himage : IsOpen ((Prod.fst : X × Y → X) '' A) := hfst A hA
  exact himage
```

Operator. crossover / crossover_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_13_1__x__Munkres_exercise_16_4__xe__b107534f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 37c8df224f940d99

4.4 004. fiberwise_fixed_step_gap

The problem

Let Y be a nonempty topological space and let $F : \mathbb{R} \times Y \rightarrow \mathbb{R}$ be continuous and constant on the fibers of the first projection. If the descended response has derivative tending to 0 at infinity, prove that it admits a continuous descent $f : \mathbb{R} \rightarrow \mathbb{R}$ and that $|F(x+1, y) - F(x, y)| \leq 1$ for all sufficiently large x , uniformly in y .

Lean statement

```
theorem fiberwise_fixed_step_gap {Y : Type*} [TopologicalSpace Y] [Nonempty Y]
  (F : ℝ × Y → ℝ) (y₀ : Y) (hF : Continuous F)
  (hfiber : ∀ p q : ℝ × Y, p.1 = q.1 → F p = F q)
  (hquot : Topology.IsQuotientMap (Prod.fst : ℝ × Y → ℝ))
  (hslice_limit : Filter.Tendsto
    (fun x : ℝ => deriv (fun t : ℝ => F (t, y₀)) x)
    Filter.atTop (nhds 0))
  (hslice_deriv : ∀ x : ℝ,
    HasDerivAt (fun t : ℝ => F (t, y₀))
      (deriv (fun t : ℝ => F (t, y₀)) x) x)
  (hanalytic : ∀ g : ℝ → ℝ, Continuous g →
    (∀ x : ℝ, HasDerivAt g (deriv g x) x) →
    Filter.Tendsto (fun x : ℝ => deriv g x) Filter.atTop (nhds 0) →
    ∃ C : ℝ, ∀ x : ℝ, C ≤ x → |g (x + 1) - g x| ≤ 1) :
  ∃ f : ℝ → ℝ, Continuous f ∧
    (∀ x : ℝ, ∀ y : Y, F (x, y) = f x) ∧
    (∃ C : ℝ, ∀ x : ℝ, C ≤ x → ∀ y : Y,
      |F (x + 1, y) - F (x, y)| ≤ 1)
```

Parent 1. Let X and Y be nonempty topological spaces. Show that the two coordinate projections $X \times Y \rightarrow X$ and $X \times Y \rightarrow Y$ are quotient maps.

Parent 1 in Lean

```
theorem product_projections_are_quotient_maps {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [Nonempty X] [Nonempty Y]
  (π₁ : X × Y → X) (π₂ : X × Y → Y)
  (h₁ : π₁ = Prod.fst) (h₂ : π₂ = Prod.snd) :
  Topology.IsQuotientMap π₁ ∧ Topology.IsQuotientMap π₂
```

What it contributes. Use quotientness of `Prod.fst` to make fiber-constancy yield a descended response on $\mathbb{R}$, then consume that descended response in the derivative-to-zero asymptotic-gap argument.

Parent 2. Suppose a response curve is differentiable for every positive input and its derivative tends to zero as the input grows without bound. Show that for every tolerance $\varepsilon > 0$, the change in response between inputs one unit apart is eventually smaller than ε .

Parent 2 in Lean

```
theorem Rudin_exercise_5_5_eventual
  {f : ℝ → ℝ}
  (hfd : DifferentiableOn ℝ f (Ioi 0))
  (hf : Tendsto (deriv f) atTop (nhds 0)) :
  ∀ ε : ℝ, 0 < ε → ∃ N : ℝ, ∀ x : ℝ, N ≤ x → |f (x + 1) - f x| < ε
```

What it contributes. Use quotientness of `Prod.fst` to make fiber-constancy yield a descended response on $\mathbb{R}$, then consume that descended response in the derivative-to-zero asymptotic-gap argument.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The proof genuinely fuses the parents: quotientness of the first projection transfers continuity from the fiber-constant F to the descended function f , after which the analytic derivative-limit hypothesis is applied to f and transported back uniformly along fibers. Neither parent's conclusion alone supplies both the descended continuous response and the fiberwise fixed-step estimate.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The proof genuinely pipelines both parents: quotientness of the first projection is used to construct and prove continuity of the descended function f , and the analytic hypothesis is then applied to that newly constructed f before its estimate is transferred uniformly across fibers.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The quotient-map hypothesis is used to descend the fiber-constant function to a continuous f , and the analytic hypothesis is then applied specifically to that constructed f ; its estimate is transferred back uniformly along fibers. The parents' roles interact through this witness exchange rather than being combined in parallel.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2822 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem fiberwise_fixed_step_gap.extracted_1.1.{u_1} {Y : Type u_1} [inst : TopologicalSpace Y] [inst_1 : Nonempty Y]
  (F : ℝ × Y → ℝ) (y₀ : Y) (hF : Continuous F) (hfiber : ∀ (p q : ℝ × Y), p.1 = q.1 → F p = F q)
  (hquot : Topology.IsQuotientMap Prod.fst)
  (hslice_limit : Filter.Tendsto (fun x => deriv (fun t => F (t, y₀)) x) Filter.atTop (nhds 0))
  (hslice_deriv : ∀ (x : ℝ), HasDerivAt (fun t => F (t, y₀)) (deriv (fun t => F (t, y₀)) x) x) :
  ∃ f,
    Continuous f ∧
    (∀ (x : ℝ) (y : Y), F (x, y) = f x) ∧ ∃ C, ∀ (x : ℝ), C ≤ x → ∀ (y : Y), |F (x + 1, y) - F (x, y)| ≤ 1
```

Read back by a model that saw only the Lean. Let Y be a topological space that is nonempty. Let $F : \mathbb{R} \times Y \rightarrow \mathbb{R}$ and $y_0 \in Y$. Assume that F is continuous; whenever $p, q \in \mathbb{R} \times Y$ have the same first coordinate, $F(p) = F(q)$; the projection $\text{Prod.fst} : \mathbb{R} \times Y \rightarrow \mathbb{R}$ is a quotient map; the function $x \mapsto \text{deriv } (t \mapsto F(t, y_0)) x$ tends to 0 as x tends to $+\infty$; and for every $x \in \mathbb{R}$, the function $t \mapsto F(t, y_0)$ has derivative $\text{deriv } (t \mapsto F(t, y_0)) x$ at x . Then there exists a continuous function $f : \mathbb{R} \rightarrow \mathbb{R}$ such that $F(x, y) = f(x)$ for every $x \in \mathbb{R}$ and $y \in Y$, and there exists a real constant C such that for every $x \in \mathbb{R}$ with $C \leq x$ and every $y \in Y$, $|F(x+1, y) - F(x, y)| \leq 1$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem fiberwise_fixed_step_gap {Y : Type*} [TopologicalSpace Y] [Nonempty Y]
  (F : ℝ × Y → ℝ) (y₀ : Y) (hF : Continuous F)
  (hfiber : ∀ p q : ℝ × Y, p.1 = q.1 → F p = F q)
  (hquot : Topology.IsQuotientMap (Prod.fst : ℝ × Y → ℝ))
```

```

(hslice_limit : Filter.Tendsto
  (fun x : ℝ => deriv (fun t : ℝ => F (t, y₀)) x)
  Filter.atTop (nhds 0))
(hslice_deriv : ∀ x : ℝ,
  HasDerivAt (fun t : ℝ => F (t, y₀))
    (deriv (fun t : ℝ => F (t, y₀)) x) x)
(hanalytic : ∀ g : ℝ → ℝ, Continuous g →
  (∀ x : ℝ, HasDerivAt g (deriv g x) x) →
  Filter.Tendsto (fun x : ℝ => deriv g x) Filter.atTop (nhds 0) →
  ∃ C : ℝ, ∀ x : ℝ, C ≤ x → |g (x + 1) - g x| ≤ 1) :
∃ f : ℝ → ℝ, Continuous f ∧
  (∀ x : ℝ, ∀ y : ℝ, F (x, y) = f x) ∧
  (∃ C : ℝ, ∀ x : ℝ, C ≤ x → ∀ y : ℝ,
    |F (x + 1, y) - F (x, y)| ≤ 1) := by
let f : ℝ → ℝ := fun x => F (x, y₀)
have hfactor : F = f ∘ (Prod.fst : ℝ × ℝ → ℝ) := by
  funext p
  exact hfiber p (p.1, y₀) rfl
have hcomp : Continuous (f ∘ (Prod.fst : ℝ × ℝ → ℝ)) := by
  rw [← hfactor]
  exact hF
have hfcont : Continuous f := by
  exact hquot.continuous_iff.mpr hcomp
have hderiv : ∀ x : ℝ, HasDerivAt f (deriv f x) x := by
  intro x
  simpa [f] using hslice_deriv x
have hlim : Filter.Tendsto (fun x : ℝ => deriv f x) Filter.atTop (nhds 0) := by
  simpa [f] using hslice_limit
have hgap : ∃ C : ℝ, ∀ x : ℝ, C ≤ x → |f (x + 1) - f x| ≤ 1 :=
  hanalytic f hfcont hderiv hlim
refine ⟨f, hfcont, ?_, ?_⟩
· intro x y
  exact hfiber (x, y) (x, y₀) rfl
· obtain ⟨C, hC⟩ := hgap
  refine ⟨C, ?_⟩
  intro x hx y
  rw [hfiber (x + 1, y) (x + 1, y₀) rfl,
    hfiber (x, y) (x, y₀) rfl]
  simpa [f] using hC x hx

```

Operator. crossover / crossover_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Munkres_exercise_16_4__me__a3e56ed4__x__Rudin_exercise_5_5__ms.xe__85f96354

Fusion mechanism. witness_exchange Use quotientness of Prod.fst to make fiber-constancy yield a descended response on $\mathbb{R}$, then consume that descended response in the derivative-to-zero asymptotic-gap argument.

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 85f96354a354a2bc

4.5 005. boolean_ring_product_order_closed

The problem

Let R be a Boolean ring with the order topology, and let s be a closed subset of a topological space X . If $f, g : X \rightarrow R$ are continuous in the sense that $x \mapsto f(x)g(x)$ is continuous, show that $\{x \in s \mid f(x)g(x) \leq g(x)f(x)\}$ is closed.

Lean statement

```
theorem boolean_ring_product_order_closed {X R : Type*} [TopologicalSpace X] [TopologicalSpace R]
  [Ring R] [LinearOrder R] [OrderTopology R] {s : Set X} {f g : X → R}
  (hR : ∀ a : R, a ^ 2 = a) (hs : IsClosed s)
  (hfg : Continuous (fun x => f x * g x)) :
  IsClosed {x | x ∈ s ∧ f x * g x ≤ g x * f x}
```

Parent 1. Let X be a topological space and Y an ordered set with the order topology. If $s \subseteq X$ is closed and $f, g : X \rightarrow Y$ are continuous, show that $\{x \in s \mid f(x) \leq g(x)\}$ is closed in X .

Parent 1 in Lean

```
theorem closed_restricted_le {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {s : Set X} {f g : X → Y}
  (hs : IsClosed s) (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x | x ∈ s ∧ f x ≤ g x}
```

Parent 2. A ring R is called a Boolean ring if $a^2 = a$ for all $a \in R$. Prove that every Boolean ring is commutative.

Parent 2 in Lean

```
theorem Dummit_Foote_exercise_7_1_15 {R : Type*} [Ring R] (hR : ∀ a : R, a^2 = a) :
  ∀ a b : R, a*b = b*a := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

The Boolean-ring parent supplies commutativity, which is used to rewrite the swapped product and derive its continuity from the given continuity of f, g ; this is then consumed by the closed-order argument. The interaction is genuine, though the resulting inequality is trivial once commutativity is established.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The Boolean-ring hypothesis is used to derive commutativity, which transfers continuity of f, g to continuity of f, g and enables the closed-order-set argument. The topological closure construction from the first parent is therefore genuinely dependent on the algebraic conclusion of the second.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely pipelines both parents: idempotence yields commutativity of products, which converts continuity of f, g into continuity of f, g ; those two continuities are then consumed by the closed-restricted-inequality argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound

Check	By	Result
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2822 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem boolean_ring_product_order_closed.extracted_1_1.{u_2, u_1} {X : Type u_1} {R : Type u_2}
  [inst : TopologicalSpace X] [inst_1 : TopologicalSpace R] [inst_2 : Ring R] [inst_3 : LinearOrder R]
  [inst_4 : OrderTopology R] {s : Set X} {f g : X → R} (hR : ∀ (a : R), a ^ 2 = a) (hs : IsClosed s)
  (hfg : Continuous fun x => f x * g x) : IsClosed {x | x ∈ s ∧ f x * g x ≤ g x * f x}
```

Read back by a model that saw only the Lean. *Let X and R be topological spaces, with R also equipped with a ring structure, a linear order, and the order topology. Suppose every element a of R is idempotent, meaning $a^2 = a$. If s is a closed subset of X and the function $x \mapsto f(x) \cdot g(x)$ is continuous, then the set of all $x \in X$ such that $x \in s$ and $f(x) \cdot g(x) \leq g(x) \cdot f(x)$ is closed in X .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem boolean_ring_product_order_closed {X R : Type*} [TopologicalSpace X] [TopologicalSpace R]
  [Ring R] [LinearOrder R] [OrderTopology R] {s : Set X} {f g : X → R}
  (hR : ∀ a : R, a ^ 2 = a) (hs : IsClosed s)
  (hfg : Continuous (fun x => f x * g x)) :
  IsClosed {x | x ∈ s ∧ f x * g x ≤ g x * f x} := by
  have hneg : ∀ a : R, -a = a := by
    intro a
    have h1 : (-a) ^ 2 = -a := hR (-a)
    calc
      -a = (-a) ^ 2 := h1.symm
      _ = a ^ 2 := by noncomm_ring
      _ = a := hR a
  have htwo : ∀ a : R, a + a = 0 := by
    intro a
    have h := congrArg (fun z : R => z + a) (hneg a)
    have hz : (0 : R) = a + a := by simpa using h
    exact hz.symm
  have hcomm : ∀ a b : R, a * b = b * a := by
    intro a b
    have hmix : a * b + b * a = 0 := by
      calc
        a * b + b * a = (a + b) ^ 2 - a ^ 2 - b ^ 2 := by noncomm_ring
        _ = (a + b) - a - b := by rw [hR (a + b), hR a, hR b]
        _ = 0 := by noncomm_ring
    calc
      a * b = a * b + 0 := by simp
      _ = a * b + (a * b + b * a) := by rw [hmix, add_zero]
      _ = (a * b + a * b) + b * a := by abel
      _ = b * a := by rw [htwo (a * b)]; simp
  have hgf : Continuous (fun x => g x * f x) := by
    have heq : (fun x => g x * f x) = (fun x => f x * g x) := by
      funext x
      exact hcomm (f x) (g x) |>.symm
    rw [heq]
    exact hfg
  have hineq : IsClosed {x | f x * g x ≤ g x * f x} := by
    exact isClosed_le hfg hgf
  have hmaster : IsClosed (s ∩ {x | f x * g x ≤ g x * f x}) := by
    exact hs.inter hineq
  have hset : {x | x ∈ s ∧ f x * g x ≤ g x * f x} =
    s ∩ {x | f x * g x ≤ g x * f x} := by
```

```
ext x
rfl
rw [hset]
exact hmaster
```

Operator. crossover / crossover_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_18_8a__me__6cceb54a__x__Dummit_Foote_exercise_7_1_15__xe__0501cfb7

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 0501cfb772c3eb68

4.6 006. finite_closed_connected_union_perfect_decomposition

The problem

Let $A_0, \dots, A_{n-1}$ be closed connected subsets of a separable metric space, with $n > 0$, and suppose each A_k for $k > 0$ meets the union of its predecessors. If their union contains two distinct points, show that it is the union of a countable set and a nonempty perfect set.

Lean statement

```
theorem finite_closed_connected_union_perfect_decomposition {X : Type*} [MetricSpace X] [SecondCountableTopology X]
  (A : ℕ → Set X) (n : ℕ) :
  0 < n →
  (∀ k < n, IsClosed (A k)) →
  (∀ k < n, IsConnected (A k)) →
  (∀ k, 0 < k → k < n →
    (A k ∩ ⋃ j ∈ Finset.range k, A j).Nonempty) →
  (∃ x y : X, x ∈ ⋃ k ∈ Finset.range n, A k ∧ y ∈ ⋃ k ∈ Finset.range n, A k ∧ x ≠ y) →
  ∃ V D : Set X, V.Countable ∧ Perfect D ∧ D.Nonempty ∧
    (⋃ k ∈ Finset.range n, A k) = V ∪ D
```

Parent 1. Let $A_0, \dots, A_{n-1}$ be connected subspaces of X , with $n > 0$. Suppose that for every $1 \leq k < n$, the set A_k meets the union of its predecessors. Show that $\bigcup_{k < n} A_k$ is connected.

Parent 1 in Lean

```
theorem finite_connected_union_predecessor {X : Type*} [TopologicalSpace X]
  (A : ℕ → Set X) (n : ℕ) :
  0 < n →
  (∀ k < n, IsConnected (A k)) →
  (∀ k, 0 < k → k < n →
    (A k ∩ ⋃ j ∈ Finset.range k, A j).Nonempty) →
  IsConnected (⋃ k ∈ Finset.range n, A k)
```

What it contributes. The connected-union checkpoint is consumed to prove the derived closed union uncountable, which forces the decomposition's perfect component to be nonempty.

Parent 2. Prove that every closed set in a separable metric space is the union of a (possibly empty) perfect set and a set which is at most countable.

Parent 2 in Lean

```
theorem Rudin_exercise_2_28 (X : Type*) [MetricSpace X] [SeparableSpace X]
  (A : Set X) (hA : IsClosed A) :
  ∃ P₁ P₂ : Set X, A = P₁ ∪ P₂ ∧ Perfect P₁ ∧ Set.Countable P₂ := by
```

What it contributes. The connected-union checkpoint is consumed to prove the derived closed union uncountable, which forces the decomposition's perfect component to be nonempty.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely pipelines Parent 1's connectedness result into a noncountability argument using the two-point hypothesis, then uses Parent 2's closed-set decomposition to force the perfect component to be nonempty. Neither parent alone supplies the resulting nonempty perfect-kernel conclusion.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely uses the connected-union result together with two distinct points to prove the union is uncountable, then consumes that fact to show the perfect component supplied by the decomposition is nonempty. This is substantive interaction, not a conjunction or final parallel combination.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely uses Parent 1's connected-union result to prove the union is uncountable, then consumes Parent 2's Cantor-Bendixson decomposition to force its perfect component to be nonempty. This is substantive interaction, not a conjunction or final parallel combination.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2246 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_closed_connected_union_perfect_decomposition.extracted_1.1.{u_1} {X : Type u_1} [inst : MetricSpace X]
[inst_1 : SecondCountableTopology X] (A : ℕ → Set X) (n : ℕ) :
0 < n →
(∀ k < n, IsClosed (A k)) →
(∀ k < n, IsConnected (A k)) →
(∀ (k : ℕ), 0 < k → k < n → (A k ∩ ⋃ j ∈ Finset.range k, A j).Nonempty) →
(∃ x y, x ∈ ⋃ k ∈ Finset.range n, A k ∧ y ∈ ⋃ k ∈ Finset.range n, A k ∧ x ≠ y) →
∃ V D, V.Countable ∧ Perfect D ∧ D.Nonempty ∧ ⋃ k ∈ Finset.range n, A k = V ∪ D
```

Read back by a model that saw only the Lean. *Let X be a metric space with a second-countable topology, let A be a sequence of subsets of X , and let n be a natural number. If $n > 0$; every A_k with $k < n$ is closed; every A_k with $k < n$ is connected; for every natural number k with $0 < k < n$, the set A_k intersects the union of A_j over all $j < k$; and the union of A_k over $k < n$ contains two distinct points, then there exist subsets V and D of X such that V is countable, D is perfect, D is nonempty, and the union of A_k over $k < n$ is equal to $V \cup D$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace
open scoped Topology

theorem finite_connected_union_predecessor {X : Type*} [TopologicalSpace X]
(A : ℕ → Set X) (n : ℕ) :
0 < n →
(∀ k < n, IsConnected (A k)) →
(∀ k, 0 < k → k < n →
(A k ∩ ⋃ j ∈ Finset.range k, A j).Nonempty) →
IsConnected (⋃ k ∈ Finset.range n, A k) := by
induction n with
| zero =>
intro hn
omega
```

```

| succ n ih =>
  intro hn hconn hattach
  by_cases hzero : n = 0
  · subst n
    simpa [Finset.mem_range] using hconn 0 (by omega)
  · have hnpos : 0 < n := Nat.pos_of_ne_zero hzero
    have hprev : IsConnected (U k ∈ Finset.range n, A k) := by
      apply ih
      · exact hnpos
      · intro k hk
        exact hconn k (by omega)
      · intro k hkpos hklit
        exact hattach k hkpos (by omega)
    have hlast : IsConnected (A n) := hconn n (by omega)
    have hinter : ((U k ∈ Finset.range n, A k) ∩ A n).Nonempty := by
      simpa [Set.inter_comm] using hattach n (by omega) (by omega)
    have hunion : IsConnected ((U k ∈ Finset.range n, A k) ∪ A n) :=
      IsConnected.union hinter hprev hlast
    have heq : (U k ∈ Finset.range (Nat.succ n), A k) =
      (U k ∈ Finset.range n, A k) ∪ A n := by
      simp only [Finset.mem_range]
      exact Set.biUnion_lt_succ A n
    rw [heq]
    exact hunion

theorem finite_closed_connected_union_perfect_decomposition {X : Type*} [MetricSpace X] [SecondCountableTopology X]
  (A : ℕ → Set X) (n : ℕ) :
  0 < n →
  (∀ k < n, IsClosed (A k)) →
  (∀ k < n, IsConnected (A k)) →
  (∀ k, 0 < k → k < n →
    (A k ∩ U j ∈ Finset.range k, A j).Nonempty) →
  (∃ x y : X, x ∈ U k ∈ Finset.range n, A k ∧ y ∈ U k ∈ Finset.range n, A k ∧ x ≠ y) →
  ∃ V D : Set X, V.Countable ∧ Perfect D ∧ D.Nonempty ∧
  (U k ∈ Finset.range n, A k) = V ∪ D := by
  classical
  intro hn hclosed hconn hattach htwo
  let C : Set X := U k ∈ Finset.range n, A k
  have hCclosed : IsClosed C := by
    dsimp [C]
    exact isClosed_biUnion_finset (fun k hk => hclosed k (Finset.mem_range.mp hk))
  have hCconn : IsConnected C := by
    exact finite_connected_union_predecessor A n hn hconn hattach
  have hCuncountable : ¬ C.Countable := by
    rcases htwo with (x, y, hx, hy, hxy)
    intro hcount
    have hsub : C.Subsingleton :=
      hcount.isTotallyDisconnected C (by intro z hz; exact hz) hCconn.isPreconnected
    exact hxy (hsub hx hy)
  obtain ⟨V, D, hV, hD, hdecomp⟩ :=
    exists_countable_union_perfect_of_isClosed hCclosed
  have hDne : D.Nonempty := by
    by_contra hne
    have hDempty : D = ∅ := Set.not_nonempty_iff_eq_empty.mp hne
    apply hCuncountable
    rw [hdecomp, hDempty, union_empty]
    exact hV
  exact ⟨V, D, hV, hD, hDne, hdecomp⟩

```

Operator. crossover / crossover_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_23_2__me__2746d5bb__x__Rudin_exercise_2_28__xe__e607282a

Fusion mechanism. invariant_transplant The connected-union checkpoint is consumed to prove the derived closed union uncountable, which forces the decomposition's perfect component to be nonempty.

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint e607282a24cc0585

4.7 007. connected_image_of_overlapping_chain

The problem

Let (A_n) be connected subsets of a uniform space such that $A_n \cap A_{n+1} \neq \emptyset$ for every n . If f and g are uniformly continuous, show that $(g \circ f)(\bigcup_n A_n)$ is connected.

Lean statement

```
theorem connected_image_of_overlapping_chain
  {α β γ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ]
  {A : ℕ → Set α} (hA : ∀ n, IsConnected (A n))
  (hAn : ∀ n, A n ∩ A (n + 1) ≠ ∅)
  {f : α → β} {g : β → γ}
  (hf : UniformContinuous f) (hg : UniformContinuous g) :
  IsConnected ((g ∘ f) '' (⋃ n, A n))
```

Parent 1. Let $\{A_n\}$ be a sequence of connected subspaces of X , such that $A_n \cap A_{n+1} \neq \emptyset$ for all n . Show that $\bigcup A_n$ is connected.

Parent 1 in Lean

```
theorem Munkres_exercise_23_2 {X : Type*}
  [TopologicalSpace X] {A : ℕ → Set X} (hA : ∀ n, IsConnected (A n))
  (hAn : ∀ n, A n ∩ A (n + 1) ≠ ∅) :
  IsConnected (⋃ n, A n) := by
```

What it contributes. Use the connectedness of the parent-1 union as the domain invariant transported through the parent-3 composite map to obtain connectedness of its image.

Parent 2. A uniformly continuous function of a uniformly continuous function is uniformly continuous.

Parent 2 in Lean

```
theorem Rudin_exercise_4_12
  {α β γ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ]
  {f : α → β} {g : β → γ}
  (hf : UniformContinuous f) (hg : UniformContinuous g) :
  UniformContinuous (g ∘ f) := by
```

What it contributes. Use the connectedness of the parent-1 union as the domain invariant transported through the parent-3 composite map to obtain connectedness of its image.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely uses both parents: Parent 1 establishes connectedness of the indexed union, while Parent 2 supplies uniform continuity of the composite map, which is then used to prove connectedness of the image. Neither parent alone yields the conclusion.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely uses both parents: Parent 1 establishes connectedness of the overlapping union, while Parent 2 supplies uniform continuity of the composite, which is then transferred to continuous-image connectedness. The composite map is essential to the conclusion, so this is not parallel combination or decoration.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely pipelines both parents: Parent 1 establishes connectedness of the overlapping union, while Parent 2 supplies uniform continuity of the composite, which is then used to prove connectedness of its image. Neither parent alone yields the conclusion.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2143 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem connected_image_of_overlapping_chain.extracted_1.1.{u_3, u_2, u_1} {α : Type u_1} {β : Type u_2} {γ : Type u_3}
  [inst : UniformSpace α] [inst_1 : UniformSpace β] [inst_2 : UniformSpace γ] {A : ℕ → Set α}
  (hA : ∀ (n : ℕ), IsConnected (A n)) (hAn : ∀ (n : ℕ), A n ∩ A (n + 1) ≠ ∅) {f : α → β} {g : β → γ}
  (hf : UniformContinuous f) (hg : UniformContinuous g) : IsConnected (g ∘ f '' U n, A n)
```

Read back by a model that saw only the Lean. Let α , β , and γ be types equipped with uniform space structures. Let A assign to each natural number n a subset $A(n)$ of α , and let $f : \alpha \rightarrow \beta$ and $g : \beta \rightarrow \gamma$ be uniformly continuous functions. If, for every natural number n , $A(n)$ is connected, and $A(n) \cap A(n + 1)$ is nonempty, then the image of $\bigcup n, A(n)$ under the composite function $g \circ f$ is connected.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem connected_image_of_overlapping_chain
  {α β γ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ]
  {A : ℕ → Set α} (hA : ∀ n, IsConnected (A n))
  (hAn : ∀ n, A n ∩ A (n + 1) ≠ ∅)
  {f : α → β} {g : β → γ}
  (hf : UniformContinuous f) (hg : UniformContinuous g) :
  IsConnected ((g ∘ f) '' (U n, A n)) := by
  classical
  have hAdj : ∀ n, (A n ∩ A (n + 1)).Nonempty := by
    intro n
    exact (Set.nonempty_iff_ne_empty).2 (hAn n)
  set R : ℕ → ℕ → Prop := fun i j => (A i ∩ A j).Nonempty
  have hEdgeSucc : ∀ n, R n (n + 1) := by
    intro n
    simpa [R, Nat.succ_eq_add_one] using hAdj n
  have hEdgePred : ∀ n, R (n + 1) n := by
    intro n
    simpa [R, Nat.succ_eq_add_one, Set.inter_comm] using hAdj n
  have hChain : ∀ i j : ℕ, Relation.ReflTransGen R i j := by
    intro i j
    refine reflTransGen_of_succ R ?_ ?_
    · intro k hk
      exact hEdgeSucc k
    · intro k hk
      exact hEdgePred k
  have hUnion : IsConnected (U n, A n) := by
    simpa [R] using
```

```

    IsConnected.iUnion_of_reflTransGen (s := A) (H := hA) (K := hChain)
have hComposite : UniformContinuous (g ∘ f) := by
  exact hg.comp hf
have hContinuous : ContinuousOn (g ∘ f) (U n, A n) := by
  exact hComposite.continuous.continuousOn
exact hUnion.image (g ∘ f) hContinuous

```

Operator. crossover / crossover_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_23_2__x__Rudin_exercise_4_12__xe__5c2bbc5c

Fusion mechanism. invariant_transplant Use the connectedness of the parent-1 union as the domain invariant transported through the parent-3 composite map to obtain connectedness of its image.

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 5c2bbc5c5edd268b

4.8 008. fixed_point_locus_has_least

The problem

Let f be a continuous self-map of the unit interval $[0, 1]$. Prove that its fixed-point set has a least element.

Lean statement

```
theorem fixed_point_locus_has_least
  (f : Set.Icc (0 : ℝ) 1 → Set.Icc (0 : ℝ) 1)
  (hf : Continuous f) :
  ∃ x : Set.Icc (0 : ℝ) 1,
    f x = x ∧ ∀ y : Set.Icc (0 : ℝ) 1, f y = y → x ≤ y := by
```

Parent 1. Let $f : X \rightarrow X$ be continuous. Show that if $X = [0, 1]$, there is a point x such that $f(x) = x$. (The point x is called a fixed point of f .)

Parent 1 in Lean

```
theorem Munkres_exercise_24_3a (f : I → I) (hf : Continuous f) :
  ∃ (x : I), f x = x := by
```

What it contributes. View $\text{Fix}(f)$ as the zero locus of $x \mapsto f x - x$; combine its closedness with the fixed-point witness to obtain a nonempty compact subset of the interval and derive its least member.

Parent 2. Let f be a continuous real function on a metric space X . Let $Z(f)$ (the zero set of f) be the set of all $p \in X$ at which $f(p) = 0$. Prove that $Z(f)$ is closed.

Parent 2 in Lean

```
theorem Rudin_exercise_4_3
  {α : Type} [MetricSpace α]
  (f : α → ℝ) (h : Continuous f) (z : Set α) (g : z = f⁻¹' {0})
  : IsClosed z := by
```

What it contributes. View $\text{Fix}(f)$ as the zero locus of $x \mapsto f x - x$; combine its closedness with the fixed-point witness to obtain a nonempty compact subset of the interval and derive its least member.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely combines Parent 1's fixed-point existence with Parent 2's closedness of the zero locus: together with interval compactness, these establish a nonempty compact fixed-point set and hence its least element. Neither parent's conclusion alone supplies the least-fixed-point result.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely needs both parents: the fixed-point witness makes the locus nonempty, while the zero-preimage closedness argument makes it compact; only their combination yields a least fixed point.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The proof genuinely uses both parents to construct the fixed-point locus as a nonempty closed subset of the compact interval, then derives its least element. The existence and closedness results are jointly consumed by the compactness/order argument, not merely conjoined at the end.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2314 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem fixed_point_locus_has_least.extracted_1_1 (f : ↑(Set.Icc 0 1) → ↑(Set.Icc 0 1)) (hf : Continuous f) :
  ∃ x, f x = x ∧ ∀ (y : ↑(Set.Icc 0 1)), f y = y → x ≤ y
```

Read back by a model that saw only the Lean. *Let f be a continuous function from the closed interval $[0,1]$ to itself. Then there exists a point x in $[0,1]$ such that $f(x)=x$ and, for every y in $[0,1]$, if $f(y)=y$, then $x \leq y$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem fixed_point_locus_has_least
  (f : Set.Icc (0 : ℝ) 1 → Set.Icc (0 : ℝ) 1)
  (hf : Continuous f) :
  ∃ x : Set.Icc (0 : ℝ) 1,
    f x = x ∧ ∀ y : Set.Icc (0 : ℝ) 1, f y = y → x ≤ y := by
  classical
  let a : Set.Icc (0 : ℝ) 1 := ⟨0, ⟨le_rfl, zero_le_one⟩⟩
  let b : Set.Icc (0 : ℝ) 1 := ⟨1, ⟨zero_le_one, le_rfl⟩⟩
  let g : Set.Icc (0 : ℝ) 1 → ℝ := fun x => (f x : ℝ) - (x : ℝ)
  have hg : Continuous g := by
    have h₁ : Continuous (fun x : Set.Icc (0 : ℝ) 1 => (f x : ℝ)) :=
      continuous_subtype_val.comp hf
    exact h₁.sub continuous_subtype_val
  have hga : 0 ≤ g a := by
    have hmem : (f a : ℝ) ∈ Set.Icc (0 : ℝ) 1 := (f a).property
    have hlow : 0 ≤ (f a : ℝ) := (Set.mem_Icc.mp hmem).1
    simpa [g, a] using hlow
  have hgb : g b ≤ 0 := by
    have hmem : (f b : ℝ) ∈ Set.Icc (0 : ℝ) 1 := (f b).property
    have hupp : (f b : ℝ) ≤ 1 := (Set.mem_Icc.mp hmem).2
    have h : (f b : ℝ) - 1 ≤ 0 := sub_nonpos.mpr hupp
    simpa [g, b] using h
  have hzero : (0 : ℝ) ∈ Set.range g :=
    mem_range_of_exists_le_of_exists_ge hg ⟨b, hgb⟩ ⟨a, hga⟩
  have hfixed : ∃ x : Set.Icc (0 : ℝ) 1, f x = x := by
    rcases hzero with ⟨x, hx⟩
    have hreal : (f x : ℝ) = (x : ℝ) := by
      apply sub_eq_zero.mp
      simpa [g] using hx
    exact ⟨x, Subtype.ext hreal⟩
  let A : Set (Set.Icc (0 : ℝ) 1) := {x | f x = x}
  have hzero_closed : IsClosed (g ⁻¹' ({0} : Set ℝ)) :=
    isClosed_singleton.preimage hg
  have hA_eq : A = g ⁻¹' ({0} : Set ℝ) := by
    ext x
    constructor
    · intro hx
      have hreal : (f x : ℝ) = (x : ℝ) := by
        exact congrArg Subtype.val hx
      simpa [g, sub_eq_zero] using hreal
    · intro hx
      have hreal : (f x : ℝ) = (x : ℝ) := by
```

```

    apply sub_eq_zero.mp
    simp [g] using hx
    simp [A] using Subtype.ext hreal
have hA_closed : IsClosed A := by
  rw [hA_eq]
  exact hzero_closed
have hA_nonempty : A.Nonempty := by
  rcases hfixed with ⟨x, hx⟩
  exact ⟨x, by simp [A] using hx⟩
have hI_compact : IsCompact (Set.univ : Set (Set.Icc (0 : ℝ) 1)) := by
  exact (isCompact_iff_isCompact_univ (s := Set.Icc (0 : ℝ) 1)).mp isCompact_Icc
have hA_compact : IsCompact A :=
  hI_compact.of_isClosed_subset hA_closed (Set.subset_univ A)
have hleast : ∃ x, IsLeast A x := hA_compact.exists_isLeast hA_nonempty
rcases hleast with ⟨x, hx⟩
exact ⟨x, by simp [A] using hx.1, fun y hy => hx.2 (by simp [A] using hy)⟩

```

Operator. crossover / crossover_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_24_3a__x__Rudin_exercise_4_3__xe__fda27007

Fusion mechanism. goal_form_transplant View Fix(f) as the zero locus of $x \mapsto f x - x$; combine its closedness with the fixed-point witness to obtain a nonempty compact subset of the interval and derive its least member.

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint fda27007375c78cb

4.9 009. positive_fixed_point_normalized_stability

The problem

Let $f : \mathbb{R} \rightarrow \mathbb{R}$ be continuous on $[0, 1]$, map $[0, 1]$ into itself, satisfy $f(x) > 0$ for every $x \in [0, 1]$, and have $x \mapsto f(x)/x$ monotone on $(0, \infty)$. Show that there exists $c \in [0, 1]$ such that $c > 0$, $f(c)/c = 1$, and $f(y) \leq y$ for every $y \in (0, c]$.

Lean statement

```
theorem positive_fixed_point_normalized_stability
  {f : ℝ → ℝ}
  (hcont : ContinuousOn f (Set.Icc (0 : ℝ) 1))
  (hmaps : Set.MapsTo f (Set.Icc (0 : ℝ) 1) (Set.Icc (0 : ℝ) 1))
  (hpositive : ∀ x ∈ Set.Icc (0 : ℝ) 1, 0 < f x)
  (hmono : MonotoneOn (fun x => f x / x) (Set.Ioi (0 : ℝ))) :
  ∃ c ∈ Set.Icc (0 : ℝ) 1, 0 < c ∧ f c / c = 1 ∧
    ∀ y ∈ Set.Ioc (0 : ℝ) c, f y ≤ y
```

Parent 1. Let $f : X \rightarrow X$ be continuous. Show that if $X = [0, 1]$, there is a point x such that $f(x) = x$. (The point x is called a fixed point of f .)

Parent 1 in Lean

```
theorem Munkres_exercise_24_3a (f : I → I) (hf : Continuous f) :
  ∃ (x : I), f x = x := by
```

What it contributes. Supplies the fixed-point witness c in the closed interval via continuity and self-mapping.

Parent 2. Suppose (a) f is continuous for $x \geq 0$, (b) $f'(x)$ exists for $x > 0$, (c) $f(0) = 0$, (d) f' is monotonically increasing. Put $g(x) = \frac{f(x)}{x}$ ($x > 0$) and prove that g is monotonically increasing.

Parent 2 in Lean

```
theorem Rudin_exercise_5_6
  {f : ℝ → ℝ}
  (hf1 : ContinuousOn f (Set.Ici 0))
  (hf2 : DifferentiableOn ℝ f (Set.Ioi 0))
  (hf3 : f 0 = 0)
  (hf4 : MonotoneOn (deriv f) (Set.Ioi 0)) :
  MonotoneOn (λ x => f x / x) (Set.Ioi 0) := by
```

What it contributes. Supplies the monotonicity comparison for $f y / y \leq f c / c$; this is fused with the fixed-point equation through `hquot` and `hstable_bound`.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The proof genuinely pipelines both parents: the interval fixed-point result supplies c with $f c = c$, while normalized monotonicity is applied at $y \leq c$ to derive $f y / y \leq f c / c = 1$ and hence $f y \leq y$. Neither parent can be discharged independently of the other in the final stability conclusion.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

Parent 1 supplies the fixed point c , whose positivity is established using the positive self-map condition; Parent 2 supplies monotonicity of $f x / x$, which is then applied at y and c to derive $f y \leq y$. The fixed-point conclusion is therefore an actual input to the normalized-monotonicity argument.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The fixed point supplied by Parent 1 provides the specific endpoint c that Parent 2's normalized monotonicity then consumes to derive $f y / y \leq f c / c = 1$ and hence $f y \leq y$. Neither parent alone supplies this stability conclusion.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1862 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem positive_fixed_point_normalized_stability.extracted_1_1 {f : ℝ → ℝ} (hcont : ContinuousOn f (Set.Icc 0 1))
  (hmaps : Set.MapsTo f (Set.Icc 0 1) (Set.Icc 0 1)) (hpositive : ∀ x ∈ Set.Icc 0 1, 0 < f x)
  (hmono : MonotoneOn (fun x => f x / x) (Set.Ioi 0)) :
  ∃ c ∈ Set.Icc 0 1, 0 < c ∧ f c / c = 1 ∧ ∀ y ∈ Set.Ioc 0 c, f y ≤ y
```

Read back by a model that saw only the Lean. *Let f be a real-valued function on the real numbers. Assume that f is continuous on $[0,1]$, maps $[0,1]$ into $[0,1]$, satisfies $f(x)>0$ for every $x\in[0,1]$, and the function $x\mapsto f(x)/x$ is monotone nondecreasing on $(0,\infty)$. Then there exists $c\in[0,1]$ such that $c>0$, $f(c)/c=1$, and for every y with $0<y\leq c$, one has $f(y)\leq y$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem positive_fixed_point_normalized_stability
  {f : ℝ → ℝ}
  (hcont : ContinuousOn f (Set.Icc (0 : ℝ) 1))
  (hmaps : Set.MapsTo f (Set.Icc (0 : ℝ) 1) (Set.Icc (0 : ℝ) 1))
  (hpositive : ∀ x ∈ Set.Icc (0 : ℝ) 1, 0 < f x)
  (hmono : MonotoneOn (fun x => f x / x) (Set.Ioi (0 : ℝ))) :
  ∃ c ∈ Set.Icc (0 : ℝ) 1, 0 < c ∧ f c / c = 1 ∧
    ∀ y ∈ Set.Ioc (0 : ℝ) c, f y ≤ y := by
  have hfixed : ∃ c ∈ Set.Icc (0 : ℝ) 1, Function.IsFixedPt f c :=
    exists_mem_Icc_isFixedPt_of_mapsTo hcont zero_le_one hmaps
  rcases hfixed with ⟨c, hc, hfc⟩
  have hfc' : f c = c := by
    simpa [Function.IsFixedPt] using hfc
  have hcpos : 0 < c := by
    have hpos := hpositive c hc
    linarith [hpos, hfc']
  have hquot : f c / c = 1 := by
    rw [hfc']
    exact div_self (ne_of_gt hcpos)
  refine ⟨c, hc, hcpos, hquot, ?_⟩
  intro y hy
  have hypos : 0 < y := hy.1
  have hy_mem : y ∈ Set.Ioi (0 : ℝ) := hypos
  have hc_mem : c ∈ Set.Ioi (0 : ℝ) := hcpos
  have hratio : f y / y ≤ f c / c := hmono hy_mem hc_mem hy.2
  have hstable_bound : f y / y ≤ 1 := by
    calc
      f y / y ≤ f c / c := hratio
      _ = 1 := hquot
```

```
have hscaled : f y ≤ 1 * y := (div_le_iff₀ hypos).mp hstable_bound
simpa using hscaled
```

Operator. crossover / crossover_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_24_3a__x__Rudin_exercise_5_6__xe__c5eb495c

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c08d399978c8e429

4.10 010. closed_subspace_trace_basis_compact_refinement

The problem

Let A be a closed subset of a compact metric space K . Construct a countable family of ambient open sets whose traces form a basis for A , and such that every neighborhood of a point of A contains the compact closure of one of these ambient basis sets.

Lean statement

```
theorem closed_subspace_trace_basis_compact_refinement {K : Type*} [MetricSpace K] [CompactSpace K] {A : Set K} (hA : IsClosed A) :
  ∃ (B : Set (Set K)), Set.Countable B ∧
    (∀ U ∈ B, IsOpen U) ∧
    TopologicalSpace.IsTopologicalBasis
      (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) ∧
    IsCompact A ∧
    (∀ x : K, x ∈ A → ∀ U : Set K, IsOpen U → x ∈ U →
      ∃ V ∈ B, x ∈ V ∧ IsCompact (closure V) ∧ closure V ⊆ U)
```

Parent 1. Let X be a Hausdorff locally compact space, and let $(B_n)_{n \in \mathbb{N}}$ be a countable open neighborhood basis at x . Show that for every open neighborhood U of x , some B_n has compact closure contained in U .

Parent 1 in Lean

```
theorem countable_basis_compact_closure_subset {X : Type*} [TopologicalSpace X] [T2Space X] [LocallyCompactSpace X]
  (x : X) (B : ℕ → Set X)
  (hBopen : ∀ n, IsOpen (B n)) (hxB : ∀ n, x ∈ B n)
  (hBbasis : ∀ s ∈ ℱ x, ∃ n, B n ⊆ s)
  (U : Set X) (hU : IsOpen U) (hxU : x ∈ U) :
  ∃ n : ℕ, IsOpen (B n) ∧ x ∈ B n ∧ IsCompact (closure (B n)) ∧ closure (B n) ⊆ U
```

What it contributes. Parent 1 supplies countable ambient basis data; parent 0 selects, after local refinement, a trace neighborhood whose ambient closure stays inside the prescribed open set.

Parent 2. Prove that every compact metric space K has a countable base.

Parent 2 in Lean

```
theorem Rudin_exercise_2.25 {K : Type*} [MetricSpace K] [CompactSpace K] :
  ∃ (B : Set (Set K)), Set.Countable B ∧ IsTopologicalBasis B := by
```

What it contributes. Parent 1 supplies countable ambient basis data; parent 0 selects, after local refinement, a trace neighborhood whose ambient closure stays inside the prescribed open set.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely fuses both parents: the compact-closure local refinement is combined with the ambient countable basis to select a basis element whose closure remains inside the prescribed neighborhood, while the basis is transported to the closed subspace via traces. Neither parent alone supplies this combined refinement-and-subspace-basis conclusion.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely combines the ambient compact-metric countable basis with compact-closure local refinement: the same ambient family supplies both the subspace trace basis and, after selecting inside a compact-closure neighborhood, the refined sets. The closed-subspace compactness and trace-basis construction are also substantive consequences needed for the stated result.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely combines the ambient countable basis with the compact-closure refinement invariant: a basis element is selected inside a locally compact neighborhood, then its closure is shown compact and contained in the prescribed open set. Neither parent alone supplies both the trace-basis construction and the refined basis property.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2899 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem closed_subspace_trace_basis_compact_refinement.extracted_1_1.{u_1} {K : Type u_1} [inst : MetricSpace K]
  [inst_1 : CompactSpace K] {A : Set K} (hA : IsClosed A) :
  ∃ B,
    B.Countable ∧
    (∀ U ∈ B, IsOpen U) ∧
    TopologicalSpace.IsTopologicalBasis ((fun U => Subtype.val ⁻¹' U) '' B) ∧
    IsCompact A ∧
    ∀ x ∈ A, ∀ (U : Set K), IsOpen U → x ∈ U → ∃ V ∈ B, x ∈ V ∧ IsCompact (closure V) ∧ closure V ⊆ U
```

Read back by a model that saw only the Lean. *Let K be a compact metric space and let $A \subseteq K$ be closed. Then there exists a countable collection B of subsets of K such that every $U \in B$ is open, the collection of traces $\{A \cap U \mid U \in B\}$ is a topological basis for A with its subspace topology, A is compact, and for every $x \in A$ and every open set $U \subseteq K$ containing x , there exists $V \in B$ such that $x \in V$, the closure of V in K is compact, and the closure of V is contained in U .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace
open scoped Topology

theorem closed_subspace_trace_basis_compact_refinement {K : Type*} [MetricSpace K] [CompactSpace K] {A : Set K} (hA : IsClosed A) :
  ∃ (B : Set (Set K)), Set.Countable B ∧
    (∀ U ∈ B, IsOpen U) ∧
    TopologicalSpace.IsTopologicalBasis
      (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) ∧
    IsCompact A ∧
    (∀ x : K, x ∈ A → ∀ U : Set K, IsOpen U → x ∈ U →
      ∃ V ∈ B, x ∈ V ∧ IsCompact (closure V) ∧ closure V ⊆ U) := by
  classical
  let B : Set (Set K) := TopologicalSpace.countableBasis K
  have hCount : B.Countable := by
    simpa [B] using TopologicalSpace.countable_countableBasis K
  have hOpen : ∀ U ∈ B, IsOpen U := by
    intro U hU
    exact TopologicalSpace.isOpen_of_mem_countableBasis (by simpa [B] using hU)
  have hBasis : TopologicalSpace.IsTopologicalBasis
    (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) := by
    simpa [B] using
      (TopologicalSpace.IsTopologicalBasis.induced (Subtype.val : A → K))
```



```

    (TopologicalSpace.isBasis_countableBasis K))
have hRefine :  $\forall x : K, x \in A \rightarrow \forall U : \text{Set } K, \text{IsOpen } U \rightarrow x \in U \rightarrow$ 
   $\exists V \in B, x \in V \wedge \text{IsCompact } (\text{closure } V) \wedge \text{closure } V \subseteq U :=$  by
  intro x hxA U hU hxU
  have hcompact_x :  $\text{IsCompact } (\{x\} : \text{Set } K) := \text{isCompact_singleton}$ 
  have hsingletonU :  $(\{x\} : \text{Set } K) \subseteq U :=$  by
    rintro y rfl
    exact hxU
  obtain ⟨W, hWopen, hsingletonW, hclWsub, hclWcompact⟩ :=
    exists_open_between_and_isCompact_closure hcompact_x hU hsingletonU
  have hxW :  $x \in W := \text{hsingletonW } (\text{mem_singleton } x)$ 
  have hWnhds :  $W \in \mathcal{N} x := \text{hWopen.mem_nhds } hxW$ 
  have hAmbientBasis := TopologicalSpace.isBasis_countableBasis K
  obtain ⟨V, hVB, hxV, hVW⟩ := hAmbientBasis.mem_nhds_iff.mp hWnhds
  have hSelected :  $V \in B \wedge x \in V \wedge V \subseteq W :=$  by
    exact ⟨by simp [B] using hVB, hxV, hVW⟩
  have hclsub :  $\text{closure } V \subseteq \text{closure } W := \text{closure_mono } hSelected.2.2$ 
  have hclU :  $\text{closure } V \subseteq U := \text{hclsub.trans } hclWsub$ 
  have hclcompact :  $\text{IsCompact } (\text{closure } V) :=$ 
    hclWcompact.of_isClosed_subset isClosed_closure hclsub
  exact ⟨V, hSelected.1, hSelected.2.1, hclcompact, hclU⟩
exact ⟨B, hCount, hOpen, hBasis, hA.isCompact, hRefine⟩

```

Operator. crossover / crossover_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_29_10__mh__cb92bce9__x__Rudin_exercise_2_25__xe__5ddc9a0c

Fusion mechanism. invariant_transplant Parent 1 supplies countable ambient basis data; parent 0 selects, after local refinement, a trace neighborhood whose ambient closure stays inside the prescribed open set.

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 5ddc9a0ccb26edf3

4.11 011. compact_neighborhood_avoiding_common_level_locus

The problem

Let X be a locally compact metric space, let s be finite, and let $(f_i)_{i \in s}$ be continuous real-valued functions. If x is outside their common level locus and U is an open neighborhood of x , show that some open neighborhood V of x has compact closure contained in U and disjoint from the common level locus.

Lean statement

```
theorem compact_neighborhood_avoiding_common_level_locus
  {α : Type*} [MetricSpace α] [LocallyCompactSpace α]
  (s : Finset ι) (f : ι → α → ℝ) (hcont : ∀ i, Continuous (f i)) (c : ℝ)
  (x : α)

  (hxL : x ∉ {y : α | ∀ i ∈ s, f i y = c})
  (U : Set α) (hU : IsOpen U) (hxU : x ∈ U) :
  ∃ V : Set α, IsOpen V ∧ x ∈ V ∧ IsCompact (closure V) ∧
    closure V ⊆ U ∧ Disjoint (closure V) {y : α | ∀ i ∈ s, f i y = c} := by
```

Parent 1. Show that if X is a Hausdorff space that is locally compact at the point x , then for each neighborhood U of x , there is a neighborhood V of x such that $\bar{V}$ is compact and $\bar{V} \subset U$.

Parent 1 in Lean

```
theorem Munkres_exercise_29_10 {X : Type*}
  [TopologicalSpace X] [T2Space X] (x : X)
  (hx : ∃ U : Set X, x ∈ U ∧ IsOpen U ∧ (∃ K : Set X, U ⊆ K ∧ IsCompact K))
  (U : Set X) (hU : IsOpen U) (hxU : x ∈ U) :
  ∃ (V : Set X), IsOpen V ∧ x ∈ V ∧ IsCompact (closure V) ∧ closure V ⊆ U := by
```

What it contributes. The closed common level locus supplies $U \setminus L$ as the neighborhood of x ; point-local compactness then produces V with compact closure inside that avoidance domain.

Parent 2. Let X be a metric space, let s be a finite index set, and let $(f_i)_{i \in s}$ be continuous real-valued functions on X . Prove that the common level locus $\{x \in X : f_i(x) = c \text{ for every } i \in s\}$ is closed.

Parent 2 in Lean

```
theorem finite_common_level_locus_closed
  {α : Type*} [MetricSpace α]
  (s : Finset ι) (f : ι → α → ℝ) (h : ∀ i, Continuous (f i)) (c : ℝ) :
  IsClosed {x : α | ∀ i ∈ s, f i x = c} := by
```

What it contributes. The closed common level locus supplies $U \setminus L$ as the neighborhood of x ; point-local compactness then produces V with compact closure inside that avoidance domain.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The proof genuinely fuses both parents: continuity makes the common level locus closed, so its complement can be intersected with U to form the neighborhood to which local compactness is applied. The resulting compact closure is then shown to avoid the level locus, rather than merely combining two independent conclusions.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely pipelines Parent 2's closedness result into Parent 1's local compactness construction: the complement of the common level locus is combined with U to form an open neighborhood, whose compactly closed neighborhood then yields avoidance. Neither parent alone supplies the resulting disjointness conclusion.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The proof genuinely pipelines Parent 2's closedness of the common level locus into an open avoidance neighborhood, then uses Parent 1's local compactness construction inside that smaller neighborhood. The avoidance condition is carried through the compact closure, so neither parent is merely discharged in parallel.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2350 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem compact_neighborhood_avoiding_common_level_locus.extracted_1_1.{u_2, u_1} {α : Type u_1} {ι : Type u_2}
  [inst : MetricSpace α] [inst_1 : LocallyCompactSpace α] (s : Finset ι) (f : ι → α → ℝ)
  (hcont : ∀ (i : ι), Continuous (f i)) (c : ℝ) (x : α) (hxlocal : ∃ N, x ∈ N ∧ IsOpen N ∧ ∃ K, N ⊆ K ∧ IsCompact K)
  (hxL : x ∉ {y | ∀ i ∈ s, f i y = c}) (U : Set α) (hU : IsOpen U) (hxU : x ∈ U) :
  ∃ V, IsOpen V ∧ x ∈ V ∧ IsCompact (closure V) ∧ closure V ⊆ U ∧ Disjoint (closure V) {y | ∀ i ∈ s, f i y = c}
```

Read back by a model that saw only the Lean. Let α be a type equipped with a metric-space structure and a locally compact-space structure, let ι be a type, let s be a finite subset of ι , and let f assign to each $i \in \iota$ a continuous function $f(i) : \alpha \rightarrow \mathbb{R}$. Let $c \in \mathbb{R}$ and $x \in \alpha$. Assume that there exist sets N and K such that $x \in N$, N is open, $N \subseteq K$, and K is compact. Assume also that x does not belong to the common level set $\{y \in \alpha \mid \text{for every } i \in s, f(i)(y) = c\}$. For any open set $U \subseteq \alpha$ containing x , there exists an open set $V \subseteq \alpha$ such that $x \in V$, the closure of V is compact, the closure of V is contained in U , and the closure of V is disjoint from the common level set $\{y \in \alpha \mid \text{for every } i \in s, f(i)(y) = c\}$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem compact_neighborhood_avoiding_common_level_locus
  {α ι : Type*} [MetricSpace α] [LocallyCompactSpace α]
  (s : Finset ι) (f : ι → α → ℝ) (hcont : ∀ i, Continuous (f i)) (c : ℝ)
  (x : α)

  (hxL : x ∉ {y : α | ∀ i ∈ s, f i y = c})
  (U : Set α) (hU : IsOpen U) (hxU : x ∈ U) :
  ∃ V : Set α, IsOpen V ∧ x ∈ V ∧ IsCompact (closure V) ∧
    closure V ⊆ U ∧ Disjoint (closure V) {y : α | ∀ i ∈ s, f i y = c} := by
  classical
  have hLclosed : IsClosed {y : α | ∀ i ∈ s, f i y = c} := by
    rw [show {y : α | ∀ i ∈ s, f i y = c} = ⋂ i ∈ s, f i ⁻¹' ({c} : Set ℝ) by
      ext y
      simp]
    exact isClosed_iInter (fun i => isClosed_iInter (fun hi =>
      isClosed_singleton.preimage (hcont i)))
  let W : Set α := U ∩ {y : α | ∀ i ∈ s, f i y = c}ᶜ
  have hWopen : IsOpen W := by
    dsimp [W]
    exact hU.inter hLclosed.isOpen_compl
  have hxW : x ∈ W := by
    exact ⟨hxU, hxL⟩
  have hWnhds : W ∈ nhds x := hWopen.mem_nhds hxW
  obtain ⟨K, hKnhd, hKW, hKcompact⟩ := local_compact_nhds hWnhds
```

```

have hKclosed : IsClosed K := hKcompact.isClosed
obtain ⟨V, hVK, hVopen, hxV⟩ := mem_nhds_iff.mp hKnhds
have hclosureK : closure V ⊆ K := by
  exact closure_minimal hVK hKclosed
have hclosureW : closure V ⊆ W := by
  intro y hy
  exact hKW (hclosureK hy)
have hclosureU : closure V ⊆ U := by
  intro y hy
  exact (hclosureW hy).1
have hclosurecompact : IsCompact (closure V) := by
  exact hKcompact.of_isClosed_subset isClosed_closure hclosureK
have hdisjoint : Disjoint (closure V) {y : α | ∀ i ∈ s, f i y = c} := by
  rw [Set.disjoint_left]
  intro y hyV hyL
  exact (hclosureW hyV).2 hyL
exact ⟨V, hVopen, hxV, hclosurecompact, hclosureU, hdisjoint⟩

```

Operator. crossover / crossover_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Munkres_exercise_29_10__x__Rudin_exercise_4_3__me__fd57004e

Fusion mechanism. obstruction_as_lemma The closed common level locus supplies $U \setminus L$ as the neighborhood of x ; point-local compactness then produces V with compact closure inside that avoidance domain.

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 43f7a0b0b99a56a0

4.12 012. irrational_checkpoint_uniform_composition

The problem

Let β and γ be uniform spaces, and let $f : \mathbb{R} \rightarrow \beta$ and $g : \beta \rightarrow \gamma$. If x is irrational, y is a nonzero rational, whenever xy is irrational f is uniformly continuous, and g is uniformly continuous, prove that $g \circ f$ is uniformly continuous.

Lean statement

```
theorem irrational_checkpoint_uniform_composition
  {β γ : Type*} [UniformSpace β] [UniformSpace γ]
  {f : ℝ → β} {g : β → γ}
  (x : ℝ) (y : ℚ) (h : y ≠ 0) (hx : Irrational x)
  (hf_if : Irrational (x * y) → UniformContinuous f)
  (hg : UniformContinuous g) :
  UniformContinuous (g ∘ f)
```

Parent 1. If r is rational ($r \neq 0$) and x is irrational, prove that rx is irrational.

Parent 1 in Lean

```
theorem Rudin_exercise_1_1b
  (x : ℝ)
  (y : ℚ)
  (h : y ≠ 0)
  : (Irrational x) → Irrational (x * y) := by
```

What it contributes. pipeline input checkpoint/object/hypothesis

Parent 2. A uniformly continuous function of a uniformly continuous function is uniformly continuous.

Parent 2 in Lean

```
theorem Rudin_exercise_4_12
  {α β γ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ]
  {f : α → β} {g : β → γ}
  (hf : UniformContinuous f) (hg : UniformContinuous g) :
  UniformContinuous (g ∘ f) := by
```

What it contributes. goal form or proof target that consumes the input

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely pipelines the parents: Parent 1 proves $\text{Irrational}(x * y)$, which is consumed as the premise of `hf_if` to obtain uniform continuity of f ; Parent 2 then composes that result with `hg`. Neither parent is discharged independently or merely combined at the end.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

Parent 1's irrationality result supplies the premise needed by `hf_if` to construct uniform continuity of f , which Parent 2 then consumes to prove uniform continuity of $g \circ f$. This is a genuine sequential pipeline rather than parallel discharge or conjunction.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely pipelines the parents: Parent 1 derives Irrational ($x * y$), which is consumed by hf_if to construct UniformContinuous f; Parent 2 then consumes that result with hg to prove UniformContinuous ($g \circ f$).

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1994 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem irrational_checkpoint_uniform_composition.extracted_1_1.{u_2, u_1} {β : Type u_1} {γ : Type u_2}
  [inst : UniformSpace β] [inst_1 : UniformSpace γ] {f : ℝ → β} {g : β → γ} (x : ℝ) (y : ℚ) (h : y ≠ 0)
  (hx : Irrational x) (hf_if : Irrational (x * ↑y) → UniformContinuous f) (hg : UniformContinuous g) :
  UniformContinuous (g ∘ f)
```

Read back by a model that saw only the Lean. Let β and γ be types equipped with uniform structures, let $f : \mathbb{R} \rightarrow \beta$ and $g : \beta \rightarrow \gamma$, let x be a real number and y a nonzero rational number. Assume that x is irrational, that if the real number x multiplied by the real embedding of y is irrational then f is uniformly continuous, and that g is uniformly continuous. Then the composition $g \circ f : \mathbb{R} \rightarrow \gamma$ is uniformly continuous.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem irrational_checkpoint_uniform_composition
  {β γ : Type*} [UniformSpace β] [UniformSpace γ]
  {f : ℝ → β} {g : β → γ}
  (x : ℝ) (y : ℚ) (h : y ≠ 0) (hx : Irrational x)
  (hf_if : Irrational (x * y) → UniformContinuous f)
  (hg : UniformContinuous g) :
  UniformContinuous (g ∘ f) := by
  have hxy : Irrational (x * y) := hx.mul_ratCast h
  have hf : UniformContinuous f := hf_if hxy
  have hcomp : UniformContinuous (g ∘ f) := hg.comp hf
  exact hcomp
```

Operator. crossover / crossover_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_1_1b__x__Rudin_exercise_4_12__xe__cf5f0efc

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fin-gerprint 139c1eae78687b59

4.13 013. closed_subset_compact_perfect_countable_basis

The problem

Let A be a closed subset of a compact metric space. Show that A is the union of a perfect set P and a countable set, and that the subspace P has a countable topological basis.

Lean statement

```
theorem closed_subset_compact_perfect_countable_basis (X : Type*) [MetricSpace X] [CompactSpace X] (A : Set X) (hA : IsClosed A) :
  ∃ P C : Set X, A = P ∪ C ∧ Perfect P ∧ Set.Countable C ∧
  ∃ B : Set (Set {x // x ∈ P}), Set.Countable B ∧ TopologicalSpace.IsTopologicalBasis B
```

Parent 1. Prove that every closed set in a separable metric space is the union of a (possibly empty) perfect set and a set which is at most countable.

Parent 1 in Lean

```
theorem Rudin_exercise_2_28 (X : Type*) [MetricSpace X] [SeparableSpace X]
  (A : Set X) (hA : IsClosed A) :
  ∃ P₁ P₂ : Set X, A = P₁ ∪ P₂ ∧ Perfect P₁ ∧ Set.Countable P₂ := by
```

What it contributes. Use the perfect-set witness supplied by the decomposition as the compact closed subtype to which the countable-base construction is applied.

Parent 2. Prove that every compact metric space K has a countable base.

Parent 2 in Lean

```
theorem Rudin_exercise_2_25 {K : Type*} [MetricSpace K] [CompactSpace K] :
  ∃ (B : Set (Set K)), Set.Countable B ∧ IsTopologicalBasis B := by
```

What it contributes. Use the perfect-set witness supplied by the decomposition as the compact closed subtype to which the countable-base construction is applied.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The decomposition parent supplies the specific perfect kernel P , and the basis construction is transferred to the subtype $\{x // x \in P\}$; the basis is therefore not an independent conclusion about X . Establishing P is closed and compact is the bridge that makes the second parent's compact-metric basis result applicable.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The decomposition parent supplies the specific perfect kernel P , and the basis parent is applied to the compact subtype of that constructed witness. The basis claim therefore depends on the decomposition's chosen P rather than being an independent conjunct.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The decomposition theorem supplies the specific perfect kernel P , whose closedness and compactness are then used to construct a countable basis on the subtype P . The basis construction therefore depends on the witness produced by the decomposition rather than being an independent conjunct.

Check

By

Result

statement_type_check
proof_accepted

lean
lean

type-checks
accepted, no sorry

Check	By	Result
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2238 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	MISMATCH: Statement A requires a countable topological basis for the subspace P, whereas Statement B requires one for the ambient space X.

Goal round-trip: what Lean elaborated

```
theorem closed_subset_compact_perfect_countable_basis.extracted_1.1.{u.1} (X : Type u.1) [inst : MetricSpace X]
[inst_1 : CompactSpace X] (A : Set X) (hA : IsClosed A) :
  ∃ P C, A = P ∪ C ∧ Perfect P ∧ C.Countable ∧ ∃ B, B.Countable ∧ TopologicalSpace.IsTopologicalBasis B
```

Read back by a model that saw only the Lean. *For any compact metric space X and any closed subset A of X , there exist subsets P and C of X such that $A = P \cup C$, P is perfect, C is countable, and there exists a countable collection B that is a topological basis for X .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem closed_subset_compact_perfect_countable_basis (X : Type*) [MetricSpace X] [CompactSpace X] (A : Set X) (hA : IsClosed A) :
  ∃ P C : Set X, A = P ∪ C ∧ Perfect P ∧ Set.Countable C ∧
    ∃ B : Set (Set {x // x ∈ P}), Set.Countable B ∧ TopologicalSpace.IsTopologicalBasis B := by
  classical
  have hSecondX : SecondCountableTopology X := inferInstance
  have hdecomp : ∃ V D : Set X, V.Countable ∧ Perfect D ∧ A = V ∪ D :=
    exists_countable_union_perfect_of_isClosed (α := X) (C := A) hA
  obtain ⟨C, P, hCcount, hPperfect, hdecomp_eq⟩ := hdecomp
  have hPclosed : IsClosed P := hPperfect.closed
  have hPcompact : IsCompact P := hPclosed.isCompact
  have hPsep : TopologicalSpace.IsSeparable P := hPcompact.isSeparable
  have hPsecond : SecondCountableTopology P := hPsep.secondCountableTopology
  letI : SecondCountableTopology P := hPsecond
  have hWitnessBasis : ∃ B : Set (Set {x // x ∈ P}), Set.Countable B ∧ TopologicalSpace.IsTopologicalBasis B := by
    obtain ⟨B, hBcount, hBne, hBbasis⟩ := TopologicalSpace.exists_countable_basis P
    exact ⟨B, hBcount, hBbasis⟩
  refine ⟨P, C, ?_, hPperfect, hCcount, hWitnessBasis⟩
  simpa [Set.union_comm] using hdecomp_eq
```

Operator. crossover / crossover_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_2_28__x__Rudin_exercise_2_25__xe__e7796e5c

Fusion mechanism. witness_exchange Use the perfect-set witness supplied by the decomposition as the compact closed subtype to which the countable-base construction is applied.

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint e7796e5ca5da6898

4.14 014. zero_constrained_positivity_closed

The problem

Let $f, g : X \rightarrow \mathbb{R}$ be continuous functions on a metric space. Prove that the set of points where f vanishes and g is nonnegative is closed.

Lean statement

```
theorem zero_constrained_positivity_closed
  {X : Type*} [MetricSpace X]
  (f g : X → ℝ) (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x | f x = 0 ∧ 0 ≤ g x} := by
```

Parent 1. Let f be a continuous real function on a metric space X . Let $Z(f)$ (the zero set of f) be the set of all $p \in X$ at which $f(p) = 0$. Prove that $Z(f)$ is closed.

Parent 1 in Lean

```
theorem Rudin_exercise_4_3
  {α : Type} [MetricSpace α]
  (f : α → ℝ) (h : Continuous f) (z : Set α) (g : z = f⁻¹' {0})
  : IsClosed z := by
```

What it contributes. The zero constraint fixes the comparison witness to the constant function 0, so the ordered-comparison checkpoint supplies closedness of the positivity condition inside the zero locus.

Parent 2. Let Y be an ordered set in the order topology. Let $f, g : X \rightarrow Y$ be continuous. Show that the set $\{x \mid f(x) \leq g(x)\}$ is closed in X .

Parent 2 in Lean

```
theorem Munkres_exercise_18_8a {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {f g : X → Y}
  (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x | f x ≤ g x} := by
```

What it contributes. The zero constraint fixes the comparison witness to the constant function 0, so the ordered-comparison checkpoint supplies closedness of the positivity condition inside the zero locus.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The proof genuinely combines both parents: Parent 1 establishes that the zero locus is closed, then Parent 2's comparison-closedness result is applied within that locus to the condition $0 \leq g(x)$. The zero constraint therefore changes the domain on which the comparison theorem is used, rather than merely joining two independently proved facts.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The proof genuinely uses Parent 1 to establish that the zero locus is closed, then feeds that closed set into Parent 2's ordered-comparison lemma to constrain it by $0 \leq g(x)$. The positivity condition is therefore closed specifically within the zero locus, so neither parent can be discharged independently.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The proof genuinely pipelines the parents: Parent 1 establishes that the zero locus is closed, and Parent 2's comparison-closedness theorem is then applied relative to that locus for the condition $0 \leq g(x)$. The first result is an essential input to the second derivation, not a parallel conjunct.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2114 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem zero_constrained_positivity_closed.extracted_1.1.{u_1} {X : Type u_1} [inst : MetricSpace X] (f g : X → ℝ)
(hf : Continuous f) (hg : Continuous g) : IsClosed {x | f x = 0 ∧ 0 ≤ g x}
```

Read back by a model that saw only the Lean. *Let X be a metric space, and let $f, g : X \rightarrow \mathbb{R}$ be continuous functions. Then the set $\{x \in X \mid f(x) = 0 \text{ and } 0 \leq g(x)\}$ is closed in X .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem zero_constrained_positivity_closed
  {X : Type*} [MetricSpace X]
  (f g : X → ℝ) (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x | f x = 0 ∧ 0 ≤ g x} := by
  have h_zero_closed : IsClosed {x | f x = 0} := by
    have h_singleton : IsClosed ({0} : Set ℝ) := by
      simp
    exact h_singleton.preimage hf
  have h_constrained_closed :
    IsClosed ({x ∈ {x | f x = 0} | (fun _ : X => 0) x ≤ g x}) := by
    exact h_zero_closed.isClosed_le continuous_const.continuousOn hg.continuousOn
  simpa using h_constrained_closed
```

Operator. crossover / crossover_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__x__Munkres_exercise_18_8a__xe__49c9c080

Fusion mechanism. invariant_transplant The zero constraint fixes the comparison witness to the constant function 0, so the ordered-comparison checkpoint supplies closedness of the positivity condition inside the zero locus.

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 49c9c08073317cd9

4.15 015. finite_even_normal_subgroup_in_normalCore

The problem

Let N be a finite normal subgroup of a group G , and let H be a subgroup containing N . Prove that the normal core of H contains an element of order 2 when $|N|$ is even.

Lean statement

```
theorem finite_even_normal_subgroup_in_normalCore {G : Type*} [Group G]
  (N H : Subgroup G) [Fintype N] (hN : N.Normal)
  (hEven : Even (Fintype.card N)) (hNH : N ≤ H) :
  ∃ x : G, orderOf x = 2 ∧ x ∈ H.normalCore
```

Parent 1. For a subgroup H of a group G , show that its normal core is normal and that, for every normal subgroup N , N is contained in H exactly when it is contained in the normal core of H .

Parent 1 in Lean

```
theorem normalCore_greatest_normal_subgroup_order {G : Type*} [Group G] (H : Subgroup G) :
  H.normalCore.Normal ∧
  ∀ N : Subgroup G, N.Normal → (N ≤ H ↔ N ≤ H.normalCore)
```

What it contributes. The order-two witness is first obtained in N ; parent 0 converts $N \leq H$ into $N \leq H.\text{normalCore}$, which supplies the witness membership required by the final theorem.

Parent 2. Prove that a finite group whose number of elements is divisible by 2 contains an element of order 2.

Parent 2 in Lean

```
theorem finite_group_order_two_of_card_dvd_two {G : Type*} [Group G] [Fintype G]
  (hG : 2 ∣ Fintype.card G) : ∃ x : G, orderOf x = 2
```

What it contributes. The order-two witness is first obtained in N ; parent 0 converts $N \leq H$ into $N \leq H.\text{normalCore}$, which supplies the witness membership required by the final theorem.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The proof genuinely uses both parents: the finite-even subgroup result supplies an order-two element inside N , while normality and $N \leq H$ are used to transport that same witness into $H.\text{normalCore}$. The final witness membership depends on the normal-core equivalence, so this is not parallel conjunction or decoration.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The proof genuinely uses both parents: the finite-even subgroup result constructs an order-two element inside N , while normal-core maximality transports $N \leq H$ to $N \leq H.\text{normalCore}$, yielding the required membership for that same witness.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely pipelines both parents: the finite-even subgroup theorem supplies an order-two element in N , while normal-core maximality transports $N \leq H$ into $N \leq H.\text{normalCore}$, yielding that same witness in the core. Neither parent alone establishes the combined existence-and-membership conclusion.

Check	By	Result
statement_type_check	lean	type-checks

Check	By	Result
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2326 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_even_normal_subgroup_in_normalCore.extracted_1_1.{u_1} {G : Type u_1} [inst : Group G] (N H : Subgroup G)
  [inst_1 : Fintype ιN] (hN : N.Normal) (hEven : Even (Fintype.card ιN)) (hNH : N ≤ H) :
  ∃ x, orderOf x = 2 ∧ x ∈ H.normalCore
```

Read back by a model that saw only the Lean. *Let G be a group and let N and H be subgroups of G . Assume that N is finite, N is normal in G , the cardinality of N is even, and N is contained in H . Then there exists an element x of G whose order is 2 and which belongs to the normal core of H in G .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem finite_even_normal_subgroup_in_normalCore {G : Type*} [Group G]
  (N H : Subgroup G) [Fintype N] (hN : N.Normal)
  (hEven : Even (Fintype.card N)) (hNH : N ≤ H) :
  ∃ x : G, orderOf x = 2 ∧ x ∈ H.normalCore := by
  letI : N.Normal := hN
  obtain ⟨k, hk⟩ := hEven
  have h_two_dvd : 2 ∣ Fintype.card N := by
    exact ⟨k, by simpa [two_mul] using hk⟩
  let _ : Fact (Nat.Prime 2) := ⟨Nat.prime_two⟩
  have hNcore : N ≤ H.normalCore :=
    (Subgroup.normal_le_normalCore (H := H) (N := N)).mpr hNH
  obtain ⟨x, hxorder⟩ := exists_prime_orderOf_dvd_card (G := N) (p := 2) h_two_dvd
  have hxFinal : ∃ y : G, orderOf y = 2 ∧ y ∈ H.normalCore := by
    refine ⟨(x : G), ?, hNcore x.property⟩
    rw [Subgroup.orderOf_coe, hxorder]
  exact hxFinal
```

Operator. crossover / crossover_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.ms__d6f85b6a__x__Artin_exercise_2_11_3__ms.xh__56ce9551

Fusion mechanism. witness_exchange The order-two witness is first obtained in N ; parent 0 converts $N \leq H$ into $N \leq H.\text{normalCore}$, which supplies the witness membership required by the final theorem.

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 56ce9551c3474d9f

4.16 016. even_normal_intersection_has_conjugacy_stable_involution

The problem

Let a finite group have a nonempty family of normal subgroups whose intersection has even cardinality. Prove that the intersection contains an element of order 2 whose every conjugate remains in the intersection.

Lean statement

```
theorem even_normal_intersection_has_conjugacy_stable_involution {G : Type*} [Group G] [Fintype G]
  (I : Type*) [Nonempty I] (H : I → Subgroup G)
  (hH : ∀ i : I, (H i).Normal)
  (hEven : Even (Nat.card (⋂ i : I, H i)))) :
  ∃ x : G, orderOf x = 2 ∧ x ∈ (⋂ i : I, H i) ∧
    ∀ g : G, g * x * g⁻¹ ∈ (⋂ i : I, H i)
```

Parent 1. Prove that the intersection of an arbitrary nonempty collection of normal subgroups of a group is a normal subgroup (do not assume the collection is countable).

Parent 1 in Lean

```
theorem Dummit_Foote_exercise_3.1.22b {G : Type*} [Group G] (I : Type*) [Nonempty I]
  (H : I → Subgroup G) (hH : ∀ i : I, Normal (H i)) :
  Normal (⋂ i : I, H i) := by
```

What it contributes. Apply the even-order involution theorem to the intersection subgroup, then consume its normality to show every conjugate of the selected involution remains in that same intersection.

Parent 2. Prove that a group of even order contains an element of order 2.

Parent 2 in Lean

```
theorem Artin_exercise_2.11.3 {G : Type*} [Group G] [Fintype G]
  (hG : Even (card G)) : ∃ x : G, orderOf x = 2 := by
```

What it contributes. Apply the even-order involution theorem to the intersection subgroup, then consume its normality to show every conjugate of the selected involution remains in that same intersection.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely pipelines both parents: normality of the intersection is established from Parent 1, then Parent 2 is applied to that intersection to obtain an involution whose conjugates remain inside it. The conjugacy-stability conclusion depends essentially on the first parent and the involution witness on the second.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The even-cardinality theorem is applied to the intersection subgroup itself, producing an involution inside it; the independently established normality then transports that specific witness through every conjugation. Neither parent is merely appended or used only to supply a constant.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely instantiates the even-order involution theorem on the intersection subgroup, using the intersection's even cardinality to obtain the specific involution. It then uses the other parent's normality result to transfer that witness through every conjugation, so both parents materially interact.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2292 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	MISMATCH: Statement B omits the hypothesis that G is finite.

Goal round-trip: what Lean elaborated

```
theorem even_normal_intersection_has_conjugacy_stable_involution.extracted_1_1.{u_2, u_1} {G : Type u_1}
[inst : Group G] (I : Type u_2) [inst_1 : Nonempty I] (H : I → Subgroup G) (hH : ∀ (i : I), (H i).Normal)
(hEven : Even (Nat.card (⋂ i, H i))) : ∃ x, orderOf x = 2 ∧ x ∈ ⋂ i, H i ∧ ∀ (g : G), g * x * g-1 ∈ ⋂ i, H i
```

Read back by a model that saw only the Lean. Let G be a group, let I be a nonempty type, and let H assign to each $i \in I$ a subgroup $H(i)$ of G . Suppose every $H(i)$ is a normal subgroup of G , and suppose the cardinality of the intersection $\bigcap_{i \in I} H(i)$ is even. Then there exists an element $x \in G$ such that the order of x is 2, x belongs to $\bigcap_{i \in I} H(i)$, and for every $g \in G$, the conjugate $g x g^{-1}$ also belongs to $\bigcap_{i \in I} H(i)$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem even_normal_intersection_has_conjugacy_stable_involution {G : Type*} [Group G] [Fintype G]
(I : Type*) [Nonempty I] (H : I → Subgroup G)
(hH : ∀ i : I, (H i).Normal)
(hEven : Even (Nat.card (⋂ i : I, H i))) :
∃ x : G, orderOf x = 2 ∧ x ∈ (⋂ i : I, H i) ∧
  ∀ g : G, g * x * g-1 ∈ (⋂ i : I, H i) := by
have hNormal : (⋂ i : I, H i).Normal := by
  exact Subgroup.normal_iInf_normal (fun i => hH i)
let I : Fintype (⋂ i : I, H i) := Fintype.ofFinite _
have hEvenK : Even (Fintype.card (⋂ i : I, H i)) := by
  simpa only [Nat.card_eq_fintype_card] using hEven
obtain ⟨k, hk⟩ := hEvenK
have h_two_dvd : 2 ∣ Fintype.card (⋂ i : I, H i) := by
  exact ⟨k, by simpa [two_mul] using hk⟩
let _ : Fact (Nat.Prime 2) := ⟨Nat.prime_two⟩
obtain ⟨x, hx⟩ : ∃ x : (⋂ i : I, H i), orderOf x = 2 :=
  exists_prime_orderOf_dvd_card 2 h_two_dvd
refine ⟨(x : G), by simpa using hx, x.property, ?_⟩
intro g
exact hNormal.conj_mem (x : G) x.property g
```

Operator. crossover / crossover_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__x__Artin_exercise_2_11_3__xh__79d882c1

Fusion mechanism. witness_exchange Apply the even-order involution theorem to the intersection subgroup, then consume its normality to show every conjugate of the selected involution remains in that same intersection.

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 79d882c164f0b6f2

4.17 017. order_eleven_totient_bridge

The problem

Let $a > 1$ be an integer, and suppose that $n \mid \phi(a^n - 1)$ for every positive integer n . If a finite group has order 462, show that it contains an element x of order 11 such that $11 \mid \phi(a^{\text{ord}(x)} - 1)$.

Lean statement

```
theorem order_eleven_totient_bridge {a : ℕ} (ha : a > 1)
  (hHerstein : ∀ n : ℕ, a > 1 → n ∣ (a ^ n - 1).totient)
  {G : Type*} [Fintype G] [Group G]
  (hG : Fintype.card G = 462) :
  ∃ x : G, orderOf x = 11 ∧ 11 ∣ (a ^ orderOf x - 1).totient
```

Parent 1. If $a > 1$ is an integer, show that $n \mid \phi(a^n - 1)$, where ϕ is the Euler ϕ -function.

Parent 1 in Lean

```
theorem Herstein_exercise_2_4_36 {a n : ℕ} (h : a > 1) :
  n ∣ (a ^ n - 1).totient := by
```

What it contributes. The group parent supplies x and the equality $\text{orderOf } x = 11$; that equality is consumed to turn the Herstein exponent-11 totient obligation into an order-indexed arithmetic conclusion.

Parent 2. Let G be a finite group of order 462. Prove that G contains an element of order 11.

Parent 2 in Lean

```
theorem exists_element_order_eleven {G : Type*} [Fintype G] [Group G]
  (hG : Fintype.card G = 462) : ∃ x : G, orderOf x = 11
```

What it contributes. The group parent supplies x and the equality $\text{orderOf } x = 11$; that equality is consumed to turn the Herstein exponent-11 totient obligation into an order-indexed arithmetic conclusion.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The group witness supplies an element with order exactly 11, and that equality is used to rewrite the exponent in the totient-divisibility claim obtained from the Herstein hypothesis. The arithmetic conclusion therefore depends on the specific witness rather than merely being conjoined with the independent existence result.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The group parent supplies an element with order exactly 11, and that equality is used to rewrite the exponent in the totient-divisibility conclusion obtained from the Herstein family. Both parents are necessary: without the witness there is no order to substitute, and without the arithmetic family there is no divisibility claim.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The group parent supplies an element with exact order 11, and that equality is used to rewrite the exponent in the totient divisibility conclusion obtained from the Herstein hypothesis. The arithmetic claim is therefore indexed by the constructed witness rather than merely conjoined afterward.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry

Check	By	Result
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2782 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem order_eleven_totient_bridge.extracted_1_1.{u_1} {a : ℕ} (ha : a > 1)
  (hHerstein : ∀ (n : ℕ), a > 1 → n | (a ^ n - 1).totient) {G : Type u_1} [inst : Fintype G] [inst_1 : Group G]
  (hG : Fintype.card G = 462) : ∃ x, orderOf x = 11 ∧ 11 | (a ^ orderOf x - 1).totient
```

Read back by a model that saw only the Lean. For a natural number a greater than 1, suppose that for every natural number n , if $a > 1$ then n divides the Euler totient of $a^n - 1$. Let G be a finite group with exactly 462 elements. Then there exists an element x of G such that the order of x is exactly 11 and 11 divides the Euler totient of $a^{(\text{order of } x)} - 1$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem order_eleven_totient_bridge {a : ℕ} (ha : a > 1)
  (hHerstein : ∀ n : ℕ, a > 1 → n | (a ^ n - 1).totient)
  {G : Type*} [Fintype G] [Group G]
  (hG : Fintype.card G = 462) :
  ∃ x : G, orderOf x = 11 ∧ 11 | (a ^ orderOf x - 1).totient := by
  letI : Fact (Nat.Prime 11) := ⟨by norm_num⟩
  have hcard : 11 | Fintype.card G := by
    rw [hG]
    norm_num
  obtain ⟨x, hx⟩ := exists_prime_orderOf_dvd_card 11 hcard
  have hfixed : 11 | (a ^ 11 - 1).totient := hHerstein 11 ha
  have hrewritten : 11 | (a ^ orderOf x - 1).totient := by
    rw [hx]
    exact hfixed
  exact ⟨x, hx, hrewritten⟩
```

Operator. crossover / crossover_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Herstein_exercise_2_4_36__x__Dummit_Foote_exercise_4_5_23__mh.xh__590497d8

Fusion mechanism. parameter_coupling The group parent supplies x and the equality $\text{orderOf } x = 11$; that equality is consumed to turn the Herstein exponent-11 totient obligation into an order-indexed arithmetic conclusion.

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 590497d834fffd5c

4.18 018. exists_central_involution_of_even_card

The problem

Let every subgroup of a finite group G be normal, and suppose that $|G|$ is even. Show that G contains a nonidentity element x with $x^2 = 1$ that commutes with every element of G .

Lean statement

```
theorem exists_central_involution_of_even_card
  {G : Type*} [Group G] [Fintype G]
  (hG : ∀ H : Subgroup G, H.Normal)
  (heven : Even (Fintype.card G)) :
  ∃ x : G, x ≠ 1 ∧ x * x = 1 ∧ ∀ b : G, b * x = x * b
```

Parent 1. Let every subgroup of a group G be normal. Show that conjugating any integer power of a by b yields another element of the cyclic subgroup generated by a .

Parent 1 in Lean

```
theorem conjugate_zpow_mem_zpowers {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :
  ∀ n : ℤ, b * a ^ n * b⁻¹ ∈ Subgroup.zpowers a
```

What it contributes. Parent 2 provides x of order two; parent 1 places $b*x*b^{-1}$ in $\text{zpowers } x$ for every b ; order preservation and the two-element cyclic subgroup force $b*x*b^{-1}=x$.

Parent 2. Prove that a group of even order contains an element of order 2.

Parent 2 in Lean

```
theorem Artin_exercise_2_11_3 {G : Type*} [Group G] [Fintype G]
  (hG : Even (card G)) : ∃ x : G, orderOf x = 2 := by
```

What it contributes. Parent 2 provides x of order two; parent 1 places $b*x*b^{-1}$ in $\text{zpowers } x$ for every b ; order preservation and the two-element cyclic subgroup force $b*x*b^{-1}=x$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely uses both parents: even cardinality supplies a nontrivial element of order two, while universal subgroup normality puts each conjugate in its cyclic subgroup. Order preservation then rules out the identity power, forcing the conjugate to equal the element and establishing centrality.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The order-two witness supplied by the even-cardinality parent is used as the generator of the cyclic subgroup in the normality argument. Its two-element structure then eliminates the identity alternative for each conjugate, yielding centrality; this is substantive witness exchange rather than a final conjunction.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The order-two witness supplied by the even-cardinality theorem is instantiated into the conjugation-membership result, and its nontriviality eliminates the identity case, forcing every conjugate to equal the witness and hence proving centrality.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2293 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem exists_central_involution_of_even_card.extracted_1.1.{u_1} {G : Type u_1} [inst : Group G] [inst_1 : Fintype G]
(hG : ∀ (H : Subgroup G), H.Normal) (heven : Even (Fintype.card G)) :
  ∃ x, x ≠ 1 ∧ x * x = 1 ∧ ∀ (b : G), b * x = x * b
```

Read back by a model that saw only the Lean. *Let G be a finite group such that every subgroup of G is normal, and suppose that the cardinality of G is even. Then there exists an element $x \in G$ such that $x \neq 1$, $x \cdot x = 1$, and for every $b \in G$, $b \cdot x = x \cdot b$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem exists_central_involution_of_even_card
  {G : Type*} [Group G] [Fintype G]
  (hG : ∀ H : Subgroup G, H.Normal)
  (heven : Even (Fintype.card G)) :
  ∃ x : G, x ≠ 1 ∧ x * x = 1 ∧ ∀ b : G, b * x = x * b := by
  classical
  rcases heven with ⟨k, hk⟩
  have htwo : 2 ∣ Fintype.card G := by
    refine ⟨k, ?_⟩
    omega
  letI : Fact (Nat.Prime 2) := ⟨Nat.prime_two⟩
  obtain (x, hxorder) := exists_prime_orderOf_dvd_card 2 htwo
  have hxneq : x ≠ 1 := by
    intro hx
    rw [hx] at hxorder
    simp at hxorder
  have hxpow : x * x = 1 := by
    have hdiv : orderOf x ∣ 2 := by simp [hxorder]
    have hp : x ^ (2 : ℕ) = 1 := (orderOf_dvd_iff_pow_eq_one).mp hdiv
    simpa [pow_two] using hp
  refine ⟨x, hxneq, hxpow, ?_⟩
  intro b
  have hnormal : (Subgroup.zpowers x).Normal := hG (Subgroup.zpowers x)
  have hmem : b * x * b⁻¹ ∈ Subgroup.zpowers x := by
    exact hnormal.conj_mem x (Subgroup.mem_zpowers x) b
  have hconj_order : orderOf (b * x * b⁻¹) = 2 := by
    have hsem : SemiconjBy b⁻¹ (b * x * b⁻¹) x := by
      change b⁻¹ * (b * x * b⁻¹) = x * b⁻¹
    group
    calc
      orderOf (b * x * b⁻¹) = orderOf x := SemiconjBy.orderOf_eq b⁻¹ hsem
      _ = 2 := hxorder
  have hfused : b * x * b⁻¹ ∈ Subgroup.zpowers x ∧ orderOf (b * x * b⁻¹) = 2 :=
    ⟨hmem, hconj_order⟩
  obtain (n, hn) := Subgroup.mem_zpowers_iff.mp hfused.1
  have hreduce : x ^ n = x ^ (n % (2 : ℤ)) := by
    have hord : (orderOf x : ℤ) = 2 := by exact_mod_cast hxorder
    rw [← hord]
    exact (zpow_mod_orderOf x n).symm
```

```

have hcases : n % (2 : ℤ) = 0 ∨ n % (2 : ℤ) = 1 := by omega
have hconjne : b * x * b⁻¹ ≠ 1 := by
  intro h
  have ho : orderOf (b * x * b⁻¹) = 1 := by rw [h]; simp
  omega
rcases hcases with hz | ho
· exfalso
  apply hconjne
  rw [← hn, hreduce, hz]
  simp
· have heq : b * x * b⁻¹ = x := by
  rw [← hn, hreduce, ho]
  simp
calc
  b * x = (b * x * b⁻¹) * b := by group
  _ = x * b := by rw [heq]

```

Operator. crossover / crossover_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Herstein_exercise_2_5_23__me__297b115a__x__Artin_exercise_2_11_3__xh__405e03c7

Fusion mechanism. witness_exchange Parent 2 provides x of order two; parent 1 places $b*x*b^{-1}$ in z powers x for every b ; order preservation and the two-element cyclic subgroup force $b*x*b^{-1}=x$.

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 405e03c740bedf1c

4.19 019. restricted_comparison_perfect_countable

The problem

Let s be a closed subset of a second-countable metric space X , and let $f, g : X \rightarrow Y$ be continuous maps into a linearly ordered space with the order topology. Show that $\{x \in s \mid f(x) \leq g(x)\}$ is the union of a perfect set and a countable set.

Lean statement

```
theorem restricted_comparison_perfect_countable {X Y : Type*} [MetricSpace X] [SecondCountableTopology X]
[TopologicalSpace Y] [LinearOrder Y] [OrderTopology Y]
(s : Set X) (f g : X → Y) (hs : IsClosed s)
(hf : Continuous f) (hg : Continuous g) :
∃ P₁ P₂ : Set X,
  {x | x ∈ s ∧ f x ≤ g x} = P₁ ∪ P₂ ∧
  Perfect P₁ ∧ Set.Countable P₂
```

Parent 1. Prove that every closed set in a separable metric space is the union of a (possibly empty) perfect set and a set which is at most countable.

Parent 1 in Lean

```
theorem Rudin_exercise_2.28 (X : Type*) [MetricSpace X] [SeparableSpace X]
(A : Set X) (hA : IsClosed A) :
∃ P₁ P₂ : Set X, A = P₁ ∪ P₂ ∧ Perfect P₁ ∧ Set.Countable P₂ := by
```

What it contributes. Parent 1 supplies closedness of the restricted comparison locus; parent 0 supplies the decomposition theorem whose closed-set input is instantiated by that locus, requiring a unified relative-object conclusion.

Parent 2. Let X be a topological space and Y an ordered set with the order topology. If $s \subseteq X$ is closed and $f, g : X \rightarrow Y$ are continuous, show that $\{x \in s \mid f(x) \leq g(x)\}$ is closed in X .

Parent 2 in Lean

```
theorem closed_restricted_le {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
[LinearOrder Y] [OrderTopology Y] {s : Set X} {f g : X → Y}
(hs : IsClosed s) (hf : Continuous f) (hg : Continuous g) :
IsClosed {x | x ∈ s ∧ f x ≤ g x}
```

What it contributes. Parent 1 supplies closedness of the restricted comparison locus; parent 0 supplies the decomposition theorem whose closed-set input is instantiated by that locus, requiring a unified relative-object conclusion.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The proof genuinely pipelines both parents: continuity and closedness establish that the restricted comparison locus is closed, then the Cantor–Bendixson decomposition is applied to that derived locus. The restriction by s makes this substantively different from decomposing the unrestricted comparison set.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely pipelines the parents: continuity and closedness of the restricted comparison locus are used to establish that it is closed, and that derived set is then fed into the perfect/countable decomposition theorem. Neither parent's conclusion is merely appended or used as a constant.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely pipelines both parents: continuity and closedness establish that the restricted comparison locus is closed, and that derived set is then fed into the Cantor–Bendixson decomposition theorem to obtain a perfect part and a countable remainder.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2789 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem restricted_comparison_perfect_countable.extracted_1.1.{u_2, u_1} {X : Type u_1} {Y : Type u_2}
  [inst : MetricSpace X] [inst_1 : SecondCountableTopology X] [inst_2 : TopologicalSpace Y] [inst_3 : LinearOrder Y]
  [inst_4 : OrderTopology Y] (s : Set X) (f g : X → Y) (hs : IsClosed s) (hf : Continuous f) (hg : Continuous g) :
  ∃ P1 P2, {x | x ∈ s ∧ f x ≤ g x} = P1 ∪ P2 ∧ Perfect P1 ∧ P2.Countable
```

Read back by a model that saw only the Lean. Let X be a metric space with a second-countable topology, and let Y be a topological space equipped with a linear order whose order topology is the given topology. For a subset s of X and continuous functions $f, g : X \rightarrow Y$, if s is closed, then there exist subsets P_1 and P_2 of X such that $\{x \in X \mid x \in s \text{ and } f(x) \leq g(x)\} = P_1 \cup P_2$, where P_1 is perfect and P_2 is countable.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem restricted_comparison_perfect_countable {X Y : Type*} [MetricSpace X] [SecondCountableTopology X]
  [TopologicalSpace Y] [LinearOrder Y] [OrderTopology Y]
  (s : Set X) (f g : X → Y) (hs : IsClosed s)
  (hf : Continuous f) (hg : Continuous g) :
  ∃ P1 P2 : Set X,
    {x | x ∈ s ∧ f x ≤ g x} = P1 ∪ P2 ∧
    Perfect P1 ∧ Set.Countable P2 := by
  classical
  have hclosed : IsClosed {x | x ∈ s ∧ f x ≤ g x} := by
    exact hs.isClosed_le hf.continuousOn hg.continuousOn
  obtain ⟨N, P, hN, hP, hdecomp⟩ :=
    exists_countable_union_perfect_of_isClosed (α := X)
    (C := {x | x ∈ s ∧ f x ≤ g x}) hclosed
  exact ⟨P, N, hdecomp.trans (Set.union_comm N P), hP, hN⟩
```

Operator. crossover / crossover_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_2_28__x__Munkres_exercise_18_8a__me.xh__d2849caf

Fusion mechanism. invariant_transplant Parent 1 supplies closedness of the restricted comparison locus; parent 0 supplies the decomposition theorem whose closed-set input is instantiated by that locus, requiring a unified relative-object conclusion.

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint d2849caffd5c506c

4.20 020. finite_iterated_neg_sum_transport

The problem

Let V be an additive commutative group, let f be a finite family of vectors, and let n be a non-negative integer. Show that the sum of the vectors obtained by applying negation n times to each $f(i)$ equals the original sum when n is even and its negative when n is odd.

Lean statement

```
theorem finite_iterated_neg_sum_transport {ι V : Type*} [Fintype ι] [AddCommGroup V]
  (n : ℕ) (f : ι → V) :
  Finset.univ.sum (fun i => Nat.rec (fun x : V => x)
    (fun _ g x => -g x) n (f i)) =
  if n % 2 = 0 then Finset.univ.sum f else -Finset.univ.sum f
```

Parent. Prove that $-(-v) = v$ for every $v \in V$.

Parent in Lean

```
theorem Axler_exercise_1_3 {F V : Type*} [AddCommGroup V] [Field F]
  [Module F V] {v : V} : -(-v) = v := by
```

What it contributes. Supplies the additive-group double-negation rewrite used in the parity induction step.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes double negation to an arbitrary finite sum with n -fold negation, requiring induction on n , parity reasoning, and sum-negation transport. A solver recalling the parent's one-line 'simp' proof would not substantially solve this theorem.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes double negation to an arbitrary iterated-negation parity law and transports it through a finite sum. Its proof requires induction and parity case analysis, which are not supplied by the parent's one-step simp proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes double negation to an arbitrary iterated-negation parity law and transports it through a finite sum. Its proof requires induction on the iteration count and a separate finite-sum negation argument, which the parent proof does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2841 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_iterated_neg_sum_transport.extracted_1.1.{u_2, u_1} {ι : Type u_1} {V : Type u_2} [inst : Fintype ι]
[inst.1 : AddCommGroup V] (n : ℕ) (f : ι → V) :

$$\sum i, \text{Nat.rec } (\text{motive} := \text{fun } x \Rightarrow V \rightarrow V) (\text{fun } x \Rightarrow x) (\text{fun } x \text{ g } x_{-1} \Rightarrow -\text{g } x_{-1}) n (f i) =$$

  if n % 2 = 0 then Finset.univ.sum f else -Finset.univ.sum f
```

Read back by a model that saw only the Lean. For any finite type ι , any additive commutative group V , any natural number n , and any function $f : \iota \rightarrow V$, the sum over all $i \in \iota$ of the result of applying negation to $f(i)$ exactly n times equals the sum of f over ι if n is even, and equals the negation of that sum if n is odd.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Complex Polynomial LinearMap FiniteDimensional Module Module.End
open scoped BigOperators

theorem finite_iterated_neg_sum_transport {ι V : Type*} [Fintype ι] [AddCommGroup V]
(n : ℕ) (f : ι → V) :
  Finset.univ.sum (fun i => Nat.rec (fun x : V => x)
    (fun _ g x => -g x) n (f i)) =
    if n % 2 = 0 then Finset.univ.sum f else -Finset.univ.sum f := by
  classical
  have hiter : ∀ k : ℕ, ∀ x : V,
    Nat.rec (fun y : V => y) (fun _ g y => -g y) k x =
      if k % 2 = 0 then x else -x := by
    intro k
    induction k with
    | zero =>
      intro x
      simp
    | succ k ih =>
      intro x
      by_cases hk : k % 2 = 0
      · have hk' : (k + 1) % 2 ≠ 0 := by omega
        simp [ih, hk, hk']
      · have hk' : (k + 1) % 2 = 0 := by omega
        simp [ih, hk, hk']
  simp_rw [hiter n]
  split_ifs with hn
  · rfl
  · exact Finset.sum_neg_distrib f
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Axler_exercise_1_3__me__171fb14c

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint 171fb14c01359541

4.21 021. submodule_neg_mem_iff

The problem

Prove that $-v \in U$ if and only if $v \in U$ for every submodule U of a vector space V over a field F and every $v \in V$.

Lean statement

```
theorem submodule_neg_mem_iff {F V : Type*} [AddCommGroup V] [Field F] [Module F V]
  (U : Submodule F V) (v : V) : -v ∈ U ↔ v ∈ U := by
```

Parent. Prove that $-(-v) = v$ for every $v \in V$.

Parent in Lean

```
theorem Axler_exercise_1_3 {F V : Type*} [AddCommGroup V] [Field F]
  [Module F V] {v : V} : -(-v) = v := by
```

What it contributes. Supplies the double-negation involution checkpoint used in the forward membership transport.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a new submodule-membership equivalence, requiring both closure under negation and transport through double negation. The parent's identity is only one ingredient and does not supply the equivalence proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a pointwise involution into a submodule-membership equivalence, requiring the genuinely new closure lemma 'U.neg_mem' and a two-direction proof. Although it uses double-negation, recalling the parent alone does not establish transport through submodule membership.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes an equality into a submodule-membership equivalence and requires the new structural fact 'U.neg_mem' in both directions, using involutivity only as one component of the transport argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1928 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem submodule_neg_mem_iff.extracted_1_1.{u_2, u_1} {F : Type u_1} {V : Type u_2} [inst : AddCommGroup V]
  [inst_1 : Field F] [inst_2 : Module F V] (U : Submodule F V) (v : V) : -v ∈ U ↔ v ∈ U
```


Read back by a model that saw only the Lean. *Let F be a field, let V be an F -module that is also an additive commutative group, let U be a submodule of V over F , and let v be an element of V . Then the additive inverse $-v$ belongs to U if and only if v belongs to U .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Complex Polynomial LinearMap FiniteDimensional Module Module.End
open scoped BigOperators

theorem submodule_neg_mem_iff {F V : Type*} [AddCommGroup V] [Field F] [Module F V]
  (U : Submodule F V) (v : V) : -v ∈ U ↔ v ∈ U := by
  have h_invol : -(-v) = v := by
    simp
  constructor
  · intro hv
    have hdouble : -(-v) ∈ U := U.neg_mem hv
    rw [h_invol] at hdouble
    exact hdouble
  · intro hv
    exact U.neg_mem hv
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Axler_exercise_1_3__me__ba71430c

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axiomsproptext, Quot.sound, fingerprint 69453865196dd89e

4.22 022. invariant_preimage_iInf_submodule

The problem

Let F be a field, let V and W be F -modules, and let $f : V \rightarrow W$, $h : V \rightarrow V$, and $g : W \rightarrow W$ be linear maps satisfying $f \circ h = g \circ f$. Given a family of submodules $(u_i)_{i \in I}$ of W such that $g(y) \in u_i$ whenever $y \in u_i$, there exists a submodule U of V equal to the preimage under f of $\bigcap_{i \in I} u_i$ such that $h(x) \in U$ for every $x \in U$.

Lean statement

```
theorem invariant_preimage_iInf_submodule {F V W ι : Type*} [Field F] [AddCommGroup V] [AddCommGroup W] [Module F V] [Module F W] (f : V → W) (h : V → V) (g : W → W) (u : ι → Submodule F W) (hcomm : f.comp h = g.comp f) (hpres : ∀ i y, y ∈ u i → g y ∈ u i) :
  ∃ U : Submodule F V, U = (⋂ i, u i).comap f ∧ ∀ x, x ∈ U → h x ∈ U
```

Parent. For a linear map, the preimage of an indexed intersection of submodules is a submodule, and membership in it is equivalent to satisfying every indexed constraint after applying the map.

Parent in Lean

```
theorem preimage_iInf_submodule_membership {F V W : Type*} [AddCommGroup V] [AddCommGroup W] [Field F] [Module F V] [Module F W] {ι : Type*} (f : V → W) (u : ι → Submodule F W) : ∃ U : Submodule F V, ∀ x : V, (x ∈ U ↔ f x ∈ (⋂ i, u i)) ∧ (x ∈ U ↔ ∀ i, f x ∈ u i)
```

What it contributes. Supplied the indexed infimum and linear-map comap construction and its membership viewpoint; the mutation adds a commuting endomorphism action and proves induced invariance.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a genuine invariant-submodule argument: it transports membership through the commuting endomorphisms and applies preservation to every component of the infimum. The parent's preimage construction alone does not supply this closure proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine invariant-submodule claim: preservation of each u_i under g is transported through the commutation relation to preservation of the preimage under h . The parent's construction supplies only membership equivalences and does not establish this closure property.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine invariant-submodule conclusion: preservation of each u_i is transported through the commutation relation to show h preserves the preimage. This requires new reasoning beyond the parent's membership equivalences, despite reusing the same comap witness.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1939 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem invariant_preimage_iInf_submodule.extracted_1_1.{u_4, u_3, u_2, u_1} {F : Type u_1} {V : Type u_2}
  {W : Type u_3} {ι : Type u_4} [inst : Field F] [inst_1 : AddCommGroup V] [inst_2 : AddCommGroup W]
  [inst_3 : Module F V] [inst_4 : Module F W] (f : V →ι[F] W) (h : V →ι[F] V) (g : W →ι[F] W) (u : ι → Submodule F W)
  (hcomm : f ∘ι h = g ∘ι f) (hpres : ∀ (i : ι), ∀ y ∈ u i, g y ∈ u i) :
  ∃ U, U = Submodule.comap f (⋂ i, u i) ∧ ∀ x ∈ U, h x ∈ U
```

Read back by a model that saw only the Lean. Let F be a field, let V and W be additive commutative groups equipped with F -module structures, let ι be a type, let $f : V \rightarrow W$, $h : V \rightarrow V$, and $g : W \rightarrow W$ be F -linear maps, and let u assign to each $i \in \iota$ a submodule $u(i)$ of W . Assume that $f \circ h = g \circ f$, and that for every $i \in \iota$ and every $y \in u(i)$, we have $g(y) \in u(i)$. Then there exists a submodule U of V such that $U = \{x \in V \mid f(x) \in \bigcap_{i \in \iota} u(i)\}$ and, for every $x \in U$, $h(x) \in U$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Complex Polynomial LinearMap FiniteDimensional Module Module.End
open scoped BigOperators

theorem invariant_preimage_iInf_submodule {F V W ι : Type*} [Field F] [AddCommGroup V] [AddCommGroup W] [Module F V] [Module F W] (f : V
  →ι[F] W) (h : V →ι[F] V) (g : W →ι[F] W) (u : ι → Submodule F W) (hcomm : f.comp h = g.comp f) (hpres : ∀ i y, y ∈ u i → g y ∈ u i) :
  ∃ U : Submodule F V, U = (⋂ i, u i).comap f ∧ ∀ x, x ∈ U → h x ∈ U := by
  refine ⟨(⋂ i, u i).comap f, rfl, ?_⟩
  intro x hx
  change f (h x) ∈ (⋂ i, u i)
  have hmem : f x ∈ (⋂ i, u i) := hx
  have hmem_all : ∀ i, f x ∈ u i := (Submodule.mem_iInf u).mp hmem
  have htransport : f (h x) = g (f x) := by
    have heval := congrArg (fun k : V →ι[F] W => k x) hcomm
    simpa using heval
  rw [htransport]
  apply (Submodule.mem_iInf u).mpr
  intro i
  exact hpres i (f x) (hmem_all i)
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Axler_exercise_1_8__me.me__2e29dde1

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 732c4a74bab5e882

4.23 023. finite_invariant_intersection_common_submodule

The problem

Let a finite nonempty family of subspaces be invariant under two bounded commuting linear maps. Show that their intersection is invariant under both maps, preserves their commutation relation, and is the greatest subspace contained in every member.

Lean statement

```
theorem finite_invariant_intersection_common_submodule {F V ι : Type*} [NormedField F] [NormedAddCommGroup V] [NormedSpace F V] [Fintype
  ι] [Nonempty ι] (u : ι → Submodule F V) (f g : V →L[F] V) (hf : ∀ i : ι, ∀ x : V, x ∈ u i → f x ∈ u i) (hg : ∀ i : ι, ∀ x : V, x ∈ u
  i → g x ∈ u i) (hfg : ∀ x : V, f (g x) = g (f x)) : ∃ U : Submodule F V, (∩ i : ι, (u i : Set V)) = (U : Set V) ∧ (∀ x : V, x ∈ U → f
  x ∈ U) ∧ (∀ x : V, x ∈ U → g x ∈ U) ∧ (∀ x : U, f (g (x : V)) = g (f (x : V))) ∧ (∀ W : Submodule F V, W ≤ U ↔ ∀ i : ι, W ≤ u i)
```

Parent. Prove that the intersection of any family of subspaces is the greatest subspace contained in every member of the family.

Parent in Lean

```
theorem intersection_is_greatest_lower_bound {F V : Type*} [AddCommGroup V] [Field F] [Module F V] {ι : Type*} (u : ι → Submodule F V) :
  ∃ U : Submodule F V,
    (∩ i : ι, (u i).carrier) = (U : Set V) ∧
    ∀ W : Submodule F V, W ≤ U ↔ ∀ i : ι, W ≤ u i := by
```

What it contributes. Supplied the indexed-infimum construction, intersection carrier equality, and greatest-lower-bound structure; the mutation adds invariance and commuting-map obligations.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds genuinely new mathematics: proving the intersection is invariant under both continuous linear maps and inherits their commutation, using the family-wise invariance hypotheses. Although the greatest-lower-bound conjunct reuses the parent's infimum argument, the parent proof does not supply these preservation arguments.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires new reasoning to prove that the intersection inherits invariance under both endomorphisms, in addition to lifting the commuting relation to the restricted submodule. The parent's lattice-theoretic intersection proof alone does not supply these arguments.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds genuinely new mathematics: proving the intersection is invariant under both continuous endomorphisms and inherits their commutation relation. The parent's lattice-theoretic intersection proof does not supply these closure arguments, so recalling it is insufficient.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2950 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```

theorem finite_invariant_intersection_common_submodule.extracted_1_1.{u_3, u_2, u_1} {F : Type u_1} {V : Type u_2}
  {ι : Type u_3} [inst : NormedField F] [inst_1 : NormedAddCommGroup V] [inst_2 : NormedSpace F V] [inst_3 : Nonempty ι]
  (u : ι → Submodule F V) (f g : V →L[F] V) (hf : ∀ (i : ι), ∀ x ∈ u i, f x ∈ u i)
  (hg : ∀ (i : ι), ∀ x ∈ u i, g x ∈ u i) (hfg : ∀ (x : V), f (g x) = g (f x)) :
  ∃ U,
    ⋂ i, ↑(u i) = ↑U ∧
      (∀ x ∈ U, f x ∈ U) ∧
      (∀ x ∈ U, g x ∈ U) ∧
      (∀ (x : ιU), f (g ↑x) = g (f ↑x)) ∧ ∀ (W : Submodule F V), W ≤ U ↔ ∀ (i : ι), W ≤ u i

```

Read back by a model that saw only the Lean. Let F be a normed field, let V be a normed additive commutative group and a normed vector space over F , and let ι be a nonempty type. Given a family $u : \iota \rightarrow \text{Submodule } F V$ and continuous linear maps $f, g : V \rightarrow V$ such that, for every $i \in \iota$ and every $x \in u i$, $f(x) \in u i$ and $g(x) \in u i$, and such that $f(g(x)) = g(f(x))$ for every $x \in V$, there exists a submodule U of V over F satisfying: the underlying set of U is exactly the intersection of the underlying sets of all $u i$; f maps every element of U into U ; g maps every element of U into U ; for every $x \in U$, $f(g(x)) = g(f(x))$; and, for every submodule W of V over F , $W \leq U$ if and only if $W \leq u i$ for every $i \in \iota$.

Lean certificate

```

import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open scoped BigOperators
open Fintype Complex Polynomial LinearMap FiniteDimensional Module Module.End

theorem finite_invariant_intersection_common_submodule {F V ι : Type*} [NormedField F] [NormedAddCommGroup V] [NormedSpace F V] [Fintype
  ι] [Nonempty ι] (u : ι → Submodule F V) (f g : V →L[F] V) (hf : ∀ i : ι, ∀ x : V, x ∈ u i → f x ∈ u i) (hg : ∀ i : ι, ∀ x : V, x ∈ u i →
  i → g x ∈ u i) (hfg : ∀ x : V, f (g x) = g (f x)) : ∃ U : Submodule F V, (⋂ i : ι, (u i : Set V)) = (U : Set V) ∧ (∀ x : V, x ∈ U →
  f x ∈ U) ∧ (∀ x : V, x ∈ U → g x ∈ U) ∧ (∀ x : U, f (g (x : V)) = g (f (x : V))) ∧ (∀ W : Submodule F V, W ≤ U ↔ ∀ i : ι, W ≤ u i)
  := by
  classical
  refine (⋂ i, u i, (Submodule.coe_iInf u).symm, ?_, ?_, ?_, ?_)
  · intro x hx
    have hx_all : ∀ i : ι, x ∈ u i := by
      simpa only [Submodule.mem_iInf] using hx
    have hfx_all : ∀ i : ι, f x ∈ u i := by
      intro i
      exact hf i x (hx_all i)
    simpa only [Submodule.mem_iInf] using hfx_all
  · intro x hx
    have hx_all : ∀ i : ι, x ∈ u i := by
      simpa only [Submodule.mem_iInf] using hx
    have hgx_all : ∀ i : ι, g x ∈ u i := by
      intro i
      exact hg i x (hx_all i)
    simpa only [Submodule.mem_iInf] using hgx_all
  · intro x
    exact hfg (x : V)
  · intro W
    constructor
    · intro h i
      exact h.trans (iInf_le u i)
    · intro h
      exact le_iInf h

```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Axler_exercise_1_8__me.me__58cb856c

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fin-

gerprint 58cb856c8665df89

4.24 024. composed_pullback_iInf_membership

The problem

Let F be a field, let V , W , and X be additive commutative groups equipped with F -module structures, let ι be an index type, let $f : V \rightarrow W$ and $g : W \rightarrow X$ be linear maps, and let $(U_i)_{i \in \iota}$ be a family of submodules of X . Then the preimage under f of the preimage under g of $\bigcap_{i \in \iota} U_i$ equals the preimage under $g \circ f$ of $\bigcap_{i \in \iota} U_i$, and for every $x \in V$, membership in this successive preimage is equivalent to $g(f(x)) \in U_i$ for every $i \in \iota$.

Lean statement

```
theorem composed_pullback_iInf_membership {F V W X : Type*} [AddCommGroup V] [AddCommGroup W] [AddCommGroup X] [Field F] [Module F V]
  [Module F W] [Module F X] {ι : Type*} (f : V →[F] W) (g : W →[F] X) (u : ι → Submodule F X) : ((⋂ i, u i).comap g).comap f = (⋂ i,
  u i).comap (g.comp f) ∧ ∀ x : V, x ∈ ((⋂ i, u i).comap g).comap f ↔ ∀ i, g (f x) ∈ u i
```

Parent. For a linear map, the preimage of an indexed intersection of submodules is a submodule, and membership in it is equivalent to satisfying every indexed constraint after applying the map.

Parent in Lean

```
theorem preimage_iInf_submodule_membership {F V W : Type*} [AddCommGroup V] [AddCommGroup W] [Field F] [Module F V] [Module F W] {ι :
  Type*} (f : V →[F] W) (u : ι → Submodule F W) : ∃ U : Submodule F V, ∀ x : V, (x ∈ U ↔ f x ∈ (⋂ i, u i)) ∧ (x ∈ U ↔ ∀ i, f x ∈ u i)
```

What it contributes. Supplied the indexed-infimum membership checkpoint and comap construction, extended here by explicitly composing two pullbacks.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a genuinely new composition law: it identifies successive submodule pullbacks with pullback along ‘ $g \circ f$ ’, then derives the indexed membership characterization through ‘`Submodule.mem_iInf`’. This requires reasoning about linear-map composition in addition to the parent’s infimum-membership argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child introduces a genuine two-map pullback composition and proves comap associativity by extensionality, while separately using `Submodule.mem_iInf` in both directions for the membership law. This is not obtainable by recalling the parent’s existential witness proof, and it follows the planned structure expansion.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the existential membership statement into a two-map pullback composition law plus an explicit indexed membership characterization. Proving comap composition by extensionality and translating the infimum through `Submodule.mem_iInf` requires reasoning not supplied by the parent proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1929 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem composed_pullback_iInf_membership.extracted_1_1.{u_5, u_4, u_3, u_2, u_1} {F : Type u_1} {V : Type u_2}
{W : Type u_3} {X : Type u_4} [inst : AddCommGroup V] [inst_1 : AddCommGroup W] [inst_2 : AddCommGroup X]
[inst_3 : Field F] [inst_4 : Module F V] [inst_5 : Module F W] [inst_6 : Module F X] {ι : Type u_5} (f : V →ι[F] W)
(g : W →ι[F] X) (u : ι → Submodule F X) :
Submodule.comap f (Submodule.comap g (⋂ i, u i)) = Submodule.comap (g ∘ι f) (⋂ i, u i) ∧
∀ (x : V), x ∈ Submodule.comap f (Submodule.comap g (⋂ i, u i)) ↔ ∀ (i : ι), g (f x) ∈ u i
```

Read back by a model that saw only the Lean. Let F be a field, let V, W , and X be additive commutative groups that are F -modules, let ι be a type, let $f : V \rightarrow W$ and $g : W \rightarrow X$ be F -linear maps, and let u assign to each $i \in \iota$ a submodule $u(i)$ of X over F . Then the preimage under f of the preimage under g of the intersection of all the submodules $u(i)$ equals the preimage under the composite linear map $g \circ f$ of that intersection, and for every $x \in V$, x belongs to the preimage under f of the preimage under g of the intersection of all $u(i)$ if and only if, for every $i \in \iota$, $g(f(x))$ belongs to $u(i)$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open scoped BigOperators
open Fintype Complex Polynomial LinearMap FiniteDimensional Module Module.End

theorem composed_pullback_iInf_membership {F V W X : Type*} [AddCommGroup V] [AddCommGroup W] [AddCommGroup X] [Field F] [Module F V]
↪ [Module F W] [Module F X] {ι : Type*} (f : V →ι[F] W) (g : W →ι[F] X) (u : ι → Submodule F X) : ((⋂ i, u i).comap g).comap f = (⋂ i,
↪ u i).comap (g.comp f) ∧ ∀ x : V, x ∈ ((⋂ i, u i).comap g).comap f ↔ ∀ i, g (f x) ∈ u i := by
have h_iInf : ∀ y : X, y ∈ (⋂ i, u i) ↔ ∀ i, y ∈ u i := by
  intro y
  exact Submodule.mem_iInf u
have h_pullback : ∀ x : V, x ∈ ((⋂ i, u i).comap g).comap f ↔ g (f x) ∈ (⋂ i, u i) := by
  intro x
  rfl
have h_composed : ∀ x : V, x ∈ (⋂ i, u i).comap (g.comp f) ↔ g (f x) ∈ (⋂ i, u i) := by
  intro x
  simp [LinearMap.comp_apply]
constructor
· ext x
  constructor
  · intro hx
    have hmid : g (f x) ∈ (⋂ i, u i) := (h_pullback x).mp hx
    have hall : ∀ i, g (f x) ∈ u i := (h_iInf (g (f x))).mp hmid
    exact (h_composed x).mpr ((h_iInf (g (f x))).mpr hall)
  · intro hx
    have hmid : g (f x) ∈ (⋂ i, u i) := (h_composed x).mp hx
    exact (h_pullback x).mpr hmid
· intro x
  constructor
  · intro hx
    have hmid : g (f x) ∈ (⋂ i, u i) := (h_pullback x).mp hx
    exact (h_iInf (g (f x))).mp hmid
  · intro hx
    have hmid : g (f x) ∈ (⋂ i, u i) := (h_iInf (g (f x))).mpr hx
    exact (h_pullback x).mpr hmid
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Axler_exercise_1_8__me.me__cb976168

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint ce29fd684843f3f5

4.25 025. intersection_is_unique_greatest_invariant_submodule

The problem

Let a linear endomorphism preserve every subspace in a family. Prove that their intersection is the unique greatest invariant subspace contained in every member of the family.

Lean statement

```
theorem intersection_is_unique_greatest_invariant_submodule {F V : Type*} [AddCommGroup V] [Field F] [Module F V] {ι : Type*} (f : V
  ↪ →₁[F] V) (u : ι → Submodule F V) (hpres : ∀ i : ι, ∀ x : V, x ∈ u i → f x ∈ u i) : ∃! U : Submodule F V, (∀ x : V, x ∈ U ↔ ∀ i : ι, x
  ↪ ∈ u i) ∧ (∀ x : V, x ∈ U → f x ∈ U) ∧ (∀ W : Submodule F V, (∀ x : V, x ∈ W → f x ∈ W) → (∀ i : ι, W ≤ u i) → W ≤ U)
```

Parent. Prove that the intersection of any collection of subspaces of V is a subspace of V .

Parent in Lean

```
theorem Axler_exercise_1_8 {F V : Type*} [AddCommGroup V] [Field F]
  [Module F V] {ι : Type*} (u : ι → Submodule F V) :
  ∃ U : Submodule F V, (⋂ (i : ι), (u i).carrier) = ↑U := by
```

What it contributes. Uses the parent’s canonical indexed-infimum construction and its pointwise membership characterization, then strengthens the conclusion with invariance and the greatest-lower-bound universal property.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes mere existence of an intersection submodule into a unique characterization as the greatest f -invariant submodule below every family member. Its proof must establish invariance and the universal maximality property using the preservation hypothesis, which the parent’s proof does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the existential intersection statement into a uniqueness and maximality characterization, while additionally proving invariance under a linear endomorphism using the preservation hypothesis. This requires reasoning absent from the parent and is not recoverable by recalling its proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine invariant-subspace characterization: preservation under the endomorphism, greatestness among all invariant subspaces below every family member, and uniqueness. The parent supplies only existence of the intersection submodule and does not establish these new universal properties.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2190 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem intersection_is_unique_greatest_invariant_submodule.extracted_1_1.{u_3, u_2, u_1} {F : Type u_1} {V : Type u_2}
  [inst : AddCommGroup V] [inst_1 : Field F] [inst_2 : Module F V] {ι : Type u_3} (f : V →ι[F] V)
  (u : ι → Submodule F V) (hpres : ∀ (i : ι), ∀ x ∈ u i, f x ∈ u i) :
  ∃! U,
    (∀ (x : V), x ∈ U ↔ ∀ (i : ι), x ∈ u i) ∧
    (∀ x ∈ U, f x ∈ U) ∧ ∀ (W : Submodule F V), (∀ x ∈ W, f x ∈ W) → (∀ (i : ι), W ≤ u i) → W ≤ U
```

Read back by a model that saw only the Lean. Let F be a field, V an additive commutative group and an F -module, ι a type, $f: V \rightarrow V$ an F -linear map, and u a family of F -submodules of V indexed by ι . Assume that for every $i \in \iota$ and every $x \in u(i)$, we have $f(x) \in u(i)$. Then there exists a unique F -submodule U of V such that: for every $x \in V$, $x \in U$ if and only if x belongs to $u(i)$ for every $i \in \iota$; U is f -invariant, meaning that for every $x \in U$, $f(x) \in U$; and for every F -submodule W of V , if W is f -invariant and $W \leq u(i)$ for every $i \in \iota$, then $W \leq U$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Complex Polynomial LinearMap FiniteDimensional Module Module.End
open scoped BigOperators

theorem intersection_is_unique_greatest_invariant_submodule {F V : Type*} [AddCommGroup V] [Field F] [Module F V] {ι : Type*} (f : V
  →ι[F] V) (u : ι → Submodule F V) (hpres : ∀ i : ι, ∀ x : V, x ∈ u i → f x ∈ u i) : ∃! U : Submodule F V, (∀ x : V, x ∈ U ↔ ∀ i : ι, x
  ∈ u i) ∧ (∀ x : V, x ∈ U → f x ∈ U) ∧ (∀ W : Submodule F V, (∀ x : V, x ∈ W → f x ∈ W) → (∀ i : ι, W ≤ u i) → W ≤ U) := by
  classical
  refine (⟨∏ i, u i, ?_, ?_⟩
    · refine (⟨?_, ?_, ?_⟩
      · intro x
        constructor
        · intro hx
          exact (Submodule.mem_iInf u).mp hx
        · intro hx
          exact (Submodule.mem_iInf u).mpr hx
      · intro x hx
        have hx_all : ∀ i : ι, x ∈ u i := (Submodule.mem_iInf u).mp hx
        have hfx_all : ∀ i : ι, f x ∈ u i := by
          intro i
          exact hpres i x (hx_all i)
        exact (Submodule.mem_iInf u).mpr hfx_all
    · intro W hW hle x hx
      exact (Submodule.mem_iInf u).mpr (fun i => hle i hx)
    · intro W hW
      apply Submodule.ext
      intro x
      have hxi : x ∈ (∏ i, u i) ↔ ∀ i : ι, x ∈ u i := by
        constructor
        · intro hx
          exact (Submodule.mem_iInf u).mp hx
        · intro hx
          exact (Submodule.mem_iInf u).mpr hx
      exact (hW.1 x).trans hxi.symm
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Axler_exercise_1_8__me__244392f2

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint 244392f2e34dd943

4.26 026. kernel_of_linear_map_family

The problem

Given a family of linear maps, construct one map whose kernel consists exactly of the vectors sent to zero by every map in the family. Show that this kernel is the intersection of the individual kernels.

Lean statement

```
theorem kernel_of_linear_map_family {F V W ι : Type*} [Field F] [AddCommGroup V] [AddCommGroup W] [Module F V] [Module F W]
  (f : ι → V →F W) :
  ∃ g : V →F W, (ι → W),
  LinearMap.ker g = ⋂ i, LinearMap.ker (f i) ∧
  ∀ x : V, x ∈ LinearMap.ker g ↔ ∀ i, f i x = 0 := by
```

Parent. Prove that the intersection of any collection of subspaces of V is a subspace of V .

Parent in Lean

```
theorem Axler_exercise_1_8 {F V : Type*} [AddCommGroup V] [Field F]
  [Module F V] {ι : Type*} (u : ι → Submodule F V) :
  ∃ U : Submodule F V, (⋂ i : ι, (u i).carrier) = ↑U := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child introduces a family of linear maps, constructs the bundled product map, and proves both the kernel intersection equality and the pointwise common-zero characterization. This requires new kernel and function-extensionality reasoning rather than recalling the parent's generic intersection packaging.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child introduces a bundled product-valued linear map and proves both its kernel intersection characterization and the pointwise common-zero condition. This requires new reasoning about function extensionality and pointwise evaluation, rather than recalling the parent's existence proof for an arbitrary intersection submodule.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child introduces a bundled linear map into the function space, proves its kernel equals the indexed infimum of the individual kernels, and establishes the pointwise common-zero characterization. This requires new kernel and pointwise-function reasoning rather than recalling the parent's canonical-intersection proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2246 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem kernel_of_linear_map_family.extracted_1_1.{u_4, u_3, u_2, u_1} {F : Type u_1} {V : Type u_2} {W : Type u_3}
  {ι : Type u_4} [inst : Field F] [inst_1 : AddCommGroup V] [inst_2 : AddCommGroup W] [inst_3 : Module F W]
  [inst_4 : Module F V] (f : ι → V →ι[F] W) :
  ∃ g, g.ker = ⋂ i, (f i).ker ∧ ∀ (x : V), x ∈ g.ker ↔ ∀ (i : ι), (f i) x = 0
```

Read back by a model that saw only the Lean. Let F be a field, let V and W be additive commutative groups equipped with F -module structures, let ι be a type, and let f assign to each $i \in \iota$ an F -linear map from V to W . Then there exists an F -linear map g such that the kernel of g equals the intersection $\bigcap_{i \in \iota} \ker(f i)$, and for every $x \in V$, x belongs to $\ker(g)$ if and only if $f i(x) = 0$ for every $i \in \iota$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Complex Polynomial LinearMap FiniteDimensional Module Module.End
open scoped BigOperators

theorem kernel_of_linear_map_family {F V W ι : Type*} [Field F] [AddCommGroup V] [AddCommGroup W] [Module F V] [Module F W]
  (f : ι → V →ι[F] W) :
  ∃ g : V →ι[F] (ι → W),
    LinearMap.ker g = ⋂ i, LinearMap.ker (f i) ∧
      ∀ x : V, x ∈ LinearMap.ker g ↔ ∀ i, f i x = 0 := by
  let g : V →ι[F] (ι → W) := LinearMap.pi f
  refine ⟨g, ?_, ?_⟩
  · ext x
    constructor
    · intro hx
      change g x = 0 at hx
      apply (Submodule.mem_iInf (fun i => LinearMap.ker (f i))).2
      intro i
      change f i x = 0
      have hi := congrFun hx i
      simpa [g] using hi
    · intro hx
      have hxi : ∀ i, x ∈ LinearMap.ker (f i) :=
        (Submodule.mem_iInf (fun i => LinearMap.ker (f i))).1 hx
      change g x = 0
      apply _root_.funext
      intro i
      have hi := hxi i
      change f i x = 0 at hi
      simpa [g] using hi
  · intro x
    constructor
    · intro hx
      change g x = 0 at hx
      intro i
      have hi := congrFun hx i
      simpa [g] using hi
    · intro hx
      change g x = 0
      apply _root_.funext
      intro i
      simpa [g] using hx i
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Axler_exercise_1_8__me__85841fac

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint 85841fac89107b3b

4.27 027. simultaneous_restriction_domain

The problem

Let a finite nonempty family of subspaces be preserved by every map in a finite commuting family of linear endomorphisms. Show that their intersection is a common domain on which all maps restrict simultaneously and still commute.

Lean statement

```
theorem simultaneous_restriction_domain {F V : Type*} [AddCommGroup V] [Field F] [Module F V]
  {ι κ : Type*} [Fintype ι] [Nonempty ι] [Fintype κ]
  (u : ι → Submodule F V) (T : κ → V →ι[F] V)
  (hpres : ∀ k i x, x ∈ u i → T k x ∈ u i)
  (hcomm : ∀ k l x, T k (T l x) = T l (T k x)) :
  ∃ U : Submodule F V,
    (↑U : Set V) = ⋂ i, (u i).carrier ∧
    ∃ r : κ → U →ι[F] U,
      (∀ k x, (r k x : V) = T k (x : V)) ∧
      (∀ k l x, r k (r l x) = r l (r k x))
```

Parent. Prove that the intersection of any collection of subspaces of V is a subspace of V .

Parent in Lean

```
theorem Axler_exercise_1_8 {F V : Type*} [AddCommGroup V] [Field F]
  [Module F V] {ι : Type*} (u : ι → Submodule F V) :
  ∃ U : Submodule F V, (⋂ (i : ι), (u i).carrier) = ↑U := by
```

What it contributes. Supplies the indexed-infimum realization of the set-theoretic intersection and the pointwise membership proof pattern.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child constructs the infimum intersection, proves it is preserved by every operator, and then builds simultaneous restricted endomorphisms whose commutation requires transporting the parent commutation law through subtypes. This is not recoverable by recalling the parent's bare intersection-existence proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds substantive mathematics beyond the parent: it proves the infimum intersection is invariant under every endomorphism, constructs simultaneous restricted endomorphisms, and establishes their commutativity. These obligations are not supplied by the parent's mere existence proof of the intersection submodule.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child extends mere existence of an intersection submodule to proving that the intersection is preserved by every endomorphism, constructing simultaneous restricted maps, and transferring commutativity. These restriction and commutation arguments are new mathematics not supplied by the parent proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable

Check	By	Result
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2240 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem simultaneous_restriction_domain.extracted_1_1.{u_4, u_3, u_2, u_1} {F : Type u_1} {V : Type u_2}
  [inst : AddCommGroup V] [inst_1 : Field F] [inst_2 : Module F V] {ι : Type u_3} {κ : Type u_4} [inst_3 : Nonempty ι]
  (u : ι → Submodule F V) (T : κ → V →ι[F] V) (hpres : ∀ (k : κ) (i : ι), ∀ x ∈ u i, (T k) x ∈ u i)
  (hcomm : ∀ (k l : κ) (x : V), (T k) ((T l) x) = (T l) ((T k) x)) :
  ∃ U,
    ↑U = ⋂ i, (u i).carrier ∧
    ∃ r,
      (∀ (k : κ) (x : ιU), ↑((r k) x) = (T k) ↑x) ∧ ∀ (k l : κ) (x : ιU), (r k) ((r l) x) = (r l) ((r k) x)
```

Read back by a model that saw only the Lean. Let F be a field and V an additive commutative group that is an F -module. Let ι and κ be types, with ι nonempty. Suppose u assigns to each $i \in \iota$ a submodule $u(i)$ of V , and T assigns to each $k \in \kappa$ an F -linear endomorphism $T(k)$ of V . Assume that for every $k \in \kappa$, $i \in \iota$, and $x \in u(i)$, we have $T(k)(x) \in u(i)$, and assume that all the endomorphisms $T(k)$ commute pairwise: for every $k, l \in \kappa$ and $x \in V$, $T(k)(T(l)(x)) = T(l)(T(k)(x))$. Then there exists a submodule U of V whose underlying set is exactly the intersection $\bigcap_i u(i)$, and there exists a family r assigning to each $k \in \kappa$ an F -linear endomorphism $r(k)$ of U such that, for every $k \in \kappa$ and $x \in U$, the underlying element of $r(k)(x)$ equals $T(k)$ applied to the underlying element of x , and such that the endomorphisms $r(k)$ commute pairwise on U .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Complex Polynomial LinearMap FiniteDimensional Module Module.End
open scoped BigOperators

theorem simultaneous_restriction_domain {F V : Type*} [AddCommGroup V] [Field F] [Module F V]
  {ι κ : Type*} [Fintype ι] [Nonempty ι] [Fintype κ]
  (u : ι → Submodule F V) (T : κ → V →ι[F] V)
  (hpres : ∀ k i x, x ∈ u i → T k x ∈ u i)
  (hcomm : ∀ k l x, T k (T l x) = T l (T k x)) :
  ∃ U : Submodule F V,
    (↑U : Set V) = ⋂ i, (u i).carrier ∧
    ∃ r : κ → U →ι[F] U,
      (∀ k x, (r k x : V) = T k (x : V)) ∧
      (∀ k l x, r k (r l x) = r l (r k x)) := by
  classical
  let U : Submodule F V := ⋂ i, u i
  have hintersection : (↑U : Set V) = ⋂ i, (u i).carrier := by
    simpa [U] using (Submodule.coe_iInf u)
  have hmem : ∀ k x, x ∈ U → T k x ∈ U := by
    intro k x hx
    change T k x ∈ ⋂ i, u i
    apply (Submodule.mem_iInf u).2
    intro i
    apply hpres k i x
    exact (Submodule.mem_iInf u).1 (by simpa [U] using hx) i
  let r : κ → U →ι[F] U := fun k =>
    { toFun := fun x => ⟨T k (x : V), hmem k (x : V) x.property⟩
      map_add' := by
        intro x y
        apply Subtype.ext
        simp
      map_smul' := by
        intro c x
        apply Subtype.ext
        simp }

  }
```

```

refine (U, hintersection, r, ?_, ?_)
· intro k x
  rfl
· intro k l x
  apply Subtype.ext
  simpa [r] using hcomm k l (x : V)

```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Axler_exercise_1_8__me__966dac6f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 966dac6ffbe4624b

4.28 028. intersection_is_greatest_invariant_submodule

The problem

Let a family of subspaces of a vector space be preserved by a common linear endomorphism. Prove that their intersection is the greatest subspace preserved by that endomorphism and contained in every member of the family.

Lean statement

```
theorem intersection_is_greatest_invariant_submodule {F V ι : Type*} [AddCommGroup V] [Field F] [Module F V]
  (u : ι → Submodule F V) (f : V →1[F] V)
  (hpres : ∀ i : ι, ∀ x : V, x ∈ u i → f x ∈ u i) :
  ∃ U : Submodule F V,
    (⋂ i : ι, (u i).carrier) = (↑U : Set V) ∧
    (∀ x : V, x ∈ U → f x ∈ U) ∧
    (∀ T : Submodule F V,
      (∀ x : V, x ∈ T → f x ∈ T) →
      (∀ i : ι, T ≤ u i) →
      T ≤ U ∧ ∀ x : V, x ∈ T → f x ∈ T)
```

Parent. Prove that the intersection of any collection of subspaces of V is a subspace of V .

Parent in Lean

```
theorem Axler_exercise_1_8 {F V : Type*} [AddCommGroup V] [Field F]
  [Module F V] {ι : Type*} (u : ι → Submodule F V) :
  ∃ U : Submodule F V, (⋂ i : ι, (u i).carrier) = ↑U := by
```

What it contributes. Supplies the indexed-intersection-as-infimum construction and carrier characterization; the child extends it with a common endomorphism and a greatest invariant-submodule property.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a genuine invariant-submodule argument: it proves the intersection is preserved by the linear map and characterizes it as containing every invariant submodule contained in each member of the family. The parent only establishes existence of the intersection submodule, so its proof does not supply the new preservation reasoning.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the existential intersection statement into a greatest-invariant-submodule characterization. It requires new reasoning that the common preservation hypothesis makes the intersection invariant and that every invariant submodule contained in each $u\ i$ lies below it.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds genuinely new mathematics: proving the intersection is invariant under a common linear map and is greatest among invariant submodules contained in every $u\ i$. The parent proof only establishes existence and equality with the intersection, so recalling it does not solve the child.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound

Check	By	Result
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2863 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem intersection_is_greatest_invariant_submodule.extracted_1_1.{u_3, u_2, u_1} {F : Type u_1} {V : Type u_2}
{ι : Type u_3} [inst : AddCommGroup V] [inst_1 : Field F] [inst_2 : Module F V] (u : ι → Submodule F V)
(f : V →ι[F] V) (hpres : ∀ (i : ι), ∀ x ∈ u i, f x ∈ u i) :
  ∃ U,
    ∩ i, (u i).carrier = ↑U ∧
    (∀ x ∈ U, f x ∈ U) ∧
    ∀ (T : Submodule F V), (∀ x ∈ T, f x ∈ T) → (∀ (i : ι), T ≤ u i) → T ≤ U ∧ ∀ x ∈ T, f x ∈ T
```

Read back by a model that saw only the Lean. *Let F be a field, let V be an additive commutative group equipped with an F -module structure, let ι be a type, let u assign to each $i : \iota$ a submodule $u\ i$ of V , and let $f : V \rightarrow V$ be an F -linear map. Assume that for every $i : \iota$ and every $x \in u\ i$, we have $f(x) \in u\ i$. Then there exists a submodule U of V such that the underlying set of U is exactly the intersection $\bigcap i : \iota, u\ i$, U is f -invariant (that is, for every $x \in U$, $f(x) \in U$), and for every submodule T of V , if T is f -invariant and $T \leq u\ i$ for every $i : \iota$, then $T \leq U$ and T is f -invariant.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open scoped BigOperators
open Fintype Complex Polynomial LinearMap FiniteDimensional Module Module.End

theorem intersection_is_greatest_invariant_submodule {F V ι : Type*} [AddCommGroup V] [Field F] [Module F V]
(u : ι → Submodule F V) (f : V →ι[F] V)
(hpres : ∀ i : ι, ∀ x : V, x ∈ u i → f x ∈ u i) :
  ∃ U : Submodule F V,
    (∩ i : ι, (u i).carrier) = (↑U : Set V) ∧
    (∀ x : V, x ∈ U → f x ∈ U) ∧
    (∀ T : Submodule F V,
      (∀ x : V, x ∈ T → f x ∈ T) →
      (∀ i : ι, T ≤ u i) →
      T ≤ U ∧ ∀ x : V, x ∈ T → f x ∈ T) := by
  classical
  refine ⟨∩ i, u i, ?_, ?_, ?_⟩
  · exact (Submodule.coe_iInf u).symm
  · intro x hx
    have hx_all : ∀ i : ι, x ∈ u i := (Submodule.mem_iInf u).mp hx
    have hfx_all : ∀ i : ι, f x ∈ u i := by
      intro i
      exact hpres i x (hx_all i)
    exact (Submodule.mem_iInf u).mpr hfx_all
  · intro T hT hTcontained
    constructor
    · intro x hx
      exact (Submodule.mem_iInf u).mpr (fun i => hTcontained i hx)
    · exact hT
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Axler_exercise_1_8__me__9f421d1d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 9f421d1d63973b48

4.29 029. intersection_is_greatest_lower_bound

The problem

For an arbitrary family of subspaces, a vector belongs to their intersection exactly when it belongs to every member. Moreover, the intersection is the greatest subspace contained in every member of the family.

Lean statement

```
theorem intersection_is_greatest_lower_bound {F V : Type*} [AddCommGroup V] [Field F]
  [Module F V] {ι : Type*} (u : ι → Submodule F V) :
  (∀ x : V, x ∈ (⋂ i, u i) ↔ ∀ i, x ∈ u i) ∧
  (∀ W : Submodule F V, W ≤ ⋂ i, u i ↔ ∀ i, W ≤ u i)
```

Parent. Prove that the intersection of any collection of subspaces of V is a subspace of V .

Parent in Lean

```
theorem Axler_exercise_1_8 {F V : Type*} [AddCommGroup V] [Field F]
  [Module F V] {ι : Type*} (u : ι → Submodule F V) :
  ∃ U : Submodule F V, (⋂ i, u i).carrier = U := by
```

What it contributes. Uses the parent's indexed-intersection domain and membership reasoning, strengthening its existential carrier representation to a greatest-lower-bound characterization in submodule order.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the existential carrier-equality claim into a two-part characterization of the indexed infimum: pointwise membership and the universal greatest-lower-bound property. Its proof uses the submodule infimum APIs directly and is not obtained by recalling or applying the parent's existential construction.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes existence of a submodule representing the indexed intersection into a full greatest-lower-bound characterization, adding the nontrivial universal containment statement for arbitrary submodules. Its proof requires order-theoretic reasoning via 'le_trans' and 'le_iInf', so recalling the parent's witness-and-extensionality proof is insufficient.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the existential carrier-equality claim into the full greatest-lower-bound characterization: both pointwise membership and submodule-order universality must be established. This is not recoverable by merely recalling the parent's witness-and-extensionality proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2243 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem intersection_is_greatest_lower_bound.extracted_1_1.{u_3, u_2, u_1} {F : Type u_1} {V : Type u_2}
  [inst : AddCommGroup V] [inst_1 : Field F] [inst_2 : Module F V] {ι : Type u_3} (u : ι → Submodule F V) :
  (∀ (x : V), x ∈ ⋂ i, u i ↔ ∀ (i : ι), x ∈ u i) ∧ ∀ (W : Submodule F V), W ≤ ⋂ i, u i ↔ ∀ (i : ι), W ≤ u i
```

Read back by a model that saw only the Lean. *Let F be a field, V an additive commutative group and an F -module, let ι be a type, and let u assign to each $i : \iota$ a submodule $u(i)$ of V over F . Then (1) for every $x \in V$, x belongs to the intersection $\bigcap_{i : \iota} u(i)$ if and only if x belongs to $u(i)$ for every $i : \iota$; and (2) for every submodule W of V over F , W is contained in $\bigcap_{i : \iota} u(i)$ if and only if W is contained in $u(i)$ for every $i : \iota$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Complex Polynomial LinearMap FiniteDimensional Module Module.End
open scoped BigOperators

theorem intersection_is_greatest_lower_bound {F V : Type*} [AddCommGroup V] [Field F]
  [Module F V] {ι : Type*} (u : ι → Submodule F V) :
  (∀ x : V, x ∈ (⋂ i, u i) ↔ ∀ i, x ∈ u i) ∧
  (∀ W : Submodule F V, W ≤ ⋂ i, u i ↔ ∀ i, W ≤ u i) := by
  constructor
  · intro x
    have h_membership : x ∈ (⋂ i, u i) ↔ ∀ i, x ∈ u i := Submodule.mem_iInf u
    exact h_membership
  · intro W
    constructor
    · intro h i
      have h_component : W ≤ u i := le_trans h (iInf_le u i)
      exact h_component
    · intro h
      have h_lower_bound : ∀ i, W ≤ u i := h
      exact le_iInf h_lower_bound
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Axler_exercise_1_8__me__b2253c94

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint b2253c940d4a2c8c

4.30 030. intersection_linear_combination

The problem

Prove that any linear combination of two vectors in the intersection of a collection of subspaces again lies in that intersection.

Lean statement

```
theorem intersection_linear_combination {F V : Type*} [AddCommGroup V] [Field F]
  [Module F V] {ι : Type*} (u : ι → Submodule F V) (a b : F) (x y : V)
  (hx : x ∈ (⋂ i : ι, (u i : Set V)))
  (hy : y ∈ (⋂ i : ι, (u i : Set V))) :
  a • x + b • y ∈ (⋂ i : ι, (u i : Set V))
```

Parent. Prove that the intersection of any collection of subspaces of V is a subspace of V .

Parent in Lean

```
theorem Axler_exercise_1_8 {F V : Type*} [AddCommGroup V] [Field F]
  [Module F V] {ι : Type*} (u : ι → Submodule F V) :
  ∃ U : Submodule F V, (⋂ i : ι, (u i).carrier) = ↑U := by
```

What it contributes. Uses the parent's indexed-intersection checkpoint: membership in the intersection is converted to membership in every subspace, where linear-combination closure is applied before rebuilding intersection membership.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a new closure property: the intersection is closed under arbitrary linear combinations. The parent only establishes that the intersection is the carrier of some submodule and does not provide this closure argument directly, so recalling the parent does not solve the child.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the existence claim into a closure property for the indexed intersection: every linear combination of two elements in the intersection remains in it. Proving this requires pointwise extraction of membership and applying each submodule's scalar and additive closure, which the parent's existence proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves closure of the indexed intersection under arbitrary linear combinations, requiring scalar- and addition-closure arguments for every component submodule. The parent's mere existence-and-carrier characterization does not supply this proof by recall.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2941 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem intersection_linear_combination.extracted_1_1.{u_3, u_2, u_1} {F : Type u_1} {V : Type u_2}
  [inst : AddCommGroup V] [inst_1 : Field F] [inst_2 : Module F V] {ι : Type u_3} (u : ι → Submodule F V) (a b : F)
  (x y : V) (hx : x ∈ ⋂ i, ↑(u i)) (hy : y ∈ ⋂ i, ↑(u i)) : a • x + b • y ∈ ⋂ i, ↑(u i)
```

Read back by a model that saw only the Lean. Let F be a field, V an additive commutative group that is an F -module, and let u be a family of F -submodules of V indexed by a type ι . For scalars $a, b \in F$ and vectors $x, y \in V$, if x and y belong to every submodule $u(i)$, then $a \cdot x + b \cdot y$ belongs to every submodule $u(i)$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Complex Polynomial LinearMap FiniteDimensional Module Module.End
open scoped BigOperators

theorem intersection_linear_combination {F V : Type*} [AddCommGroup V] [Field F]
  [Module F V] {ι : Type*} (u : ι → Submodule F V) (a b : F) (x y : V)
  (hx : x ∈ (⋂ i : ι, (u i : Set V)))
  (hy : y ∈ (⋂ i : ι, (u i : Set V))) :
  a • x + b • y ∈ (⋂ i : ι, (u i : Set V)) := by
  have hx_all : ∀ i : ι, x ∈ u i := by
    intro i
    exact Set.mem_iInter.mp hx i
  have hy_all : ∀ i : ι, y ∈ u i := by
    intro i
    exact Set.mem_iInter.mp hy i
  have hcombined : ∀ i : ι, a • x + b • y ∈ u i := by
    intro i
    exact (u i).add_mem ((u i).smul_mem a (hx_all i)) ((u i).smul_mem b (hy_all i))
  exact Set.mem_iInter.mpr hcombined
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Axler_exercise_1_8__me__bc501014

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint bc501014b392e5e2

4.31 031. linearMap_comap_iInf_composition

The problem

Let $f : V \rightarrow W$ and $g : W \rightarrow X$ be linear maps, and let $(U_i)_{i \in I}$ be subspaces of X . Show that the vectors $x \in V$ satisfying $g(f(x)) \in U_i$ for every i form the largest subspace with this property, and that pulling back the intersection in two stages equals pulling it back along $g \circ f$.

Lean statement

```
theorem linearMap_comap_iInf_composition {F V W X : Type*} [AddCommGroup V] [AddCommGroup W]
  [AddCommGroup X] [Field F] [Module F V] [Module F W] [Module F X]
  {ι : Type*} (f : V → [F] W) (g : W → [F] X) (u : ι → Submodule F X) :
  ∃ U : Submodule F V,
    U = (⋂ i, u i).comap g).comap f ∧
    U = (⋂ i, u i).comap (g.comp f) ∧
    (∀ x, x ∈ U ↔ ∀ i, g (f x) ∈ u i) ∧
    ∀ S : Submodule F V, (∀ x ∈ S, ∀ i, g (f x) ∈ u i) → S ≤ U
```

Parent. Prove that the intersection of any collection of subspaces of V is a subspace of V .

Parent in Lean

```
theorem Axler_exercise_1_8 {F V : Type*} [AddCommGroup V] [Field F]
  [Module F V] {ι : Type*} (u : ι → Submodule F V) :
  ∃ U : Submodule F V, (⋂ i, u i).carrier = ↑U := by
```

What it contributes. Uses the parent's indexed-intersection-as-infimum construction, specializing it to the contravariant pullback of submodules along composable linear maps.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the parent by introducing two composable linear maps and proving that two-stage and composite pullbacks coincide, plus a greatest-submodule characterization. Its proof requires new comap/composition reasoning and an essential indexed-membership argument, not merely recalling the parent's intersection construction.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes the parent from an arbitrary set intersection to a two-stage pullback along composable linear maps, adding both a pointwise characterization and a greatest-submodule property. Its proof requires comap composition and universal containment reasoning, so recalling the parent does not substantially solve it.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the parent from existence of an indexed intersection submodule to a two-stage versus one-stage pullback under composable linear maps. Its proof requires comap composition and a greatest-submodule argument, with indexed-intersection membership used essentially.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable

Check	By	Result
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2224 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem linearMap_comap_iInf_composition.extracted_1.1.{u_5, u_4, u_3, u_2, u_1} {F : Type u_1} {V : Type u_2}
{W : Type u_3} {X : Type u_4} [inst : AddCommGroup V] [inst_1 : AddCommGroup W] [inst_2 : AddCommGroup X]
[inst_3 : Field F] [inst_4 : Module F V] [inst_5 : Module F W] [inst_6 : Module F X] {ι : Type u_5} (f : V →ι[F] W)
(g : W →ι[F] X) (u : ι → Submodule F X) :
  ∃ U,
    U = Submodule.comap f (Submodule.comap g (⋂ i, u i)) ∧
    U = Submodule.comap (g ∘ι f) (⋂ i, u i) ∧
    (∀ (x : V), x ∈ U ↔ ∀ (i : ι), g (f x) ∈ u i) ∧
    ∀ (S : Submodule F V), (∀ x ∈ S, ∀ (i : ι), g (f x) ∈ u i) → S ≤ U
```

Read back by a model that saw only the Lean. Let F be a field, let V, W , and X be F -modules whose additive structures are commutative groups, let ι be a type, let $f : V \rightarrow W$ and $g : W \rightarrow X$ be F -linear maps, and let u assign to each $i \in \iota$ a submodule $u(i)$ of X . Then there exists a submodule U of V such that U is the inverse image under f of the inverse image under g of the intersection $\bigcap_{i \in \iota} u(i)$, U is also the inverse image under the composite $g \circ f$ of $\bigcap_{i \in \iota} u(i)$, every $x \in V$ belongs to U if and only if $g(f(x)) \in u(i)$ for every $i \in \iota$, and every submodule S of V satisfying $g(f(x)) \in u(i)$ for all $x \in S$ and all $i \in \iota$ is contained in U .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open scoped BigOperators
open Fintype Complex Polynomial LinearMap FiniteDimensional Module Module.End

theorem linearMap_comap_iInf_composition {F V W X : Type*} [AddCommGroup V] [AddCommGroup W]
[AddCommGroup X] [Field F] [Module F V] [Module F W] [Module F X]
{ι : Type*} (f : V →ι[F] W) (g : W →ι[F] X) (u : ι → Submodule F X) :
  ∃ U : Submodule F V,
    U = ((⋂ i, u i).comap g).comap f ∧
    U = (⋂ i, u i).comap (g.comp f) ∧
    (∀ x, x ∈ U ↔ ∀ i, g (f x) ∈ u i) ∧
    ∀ S : Submodule F V, (∀ x ∈ S, ∀ i, g (f x) ∈ u i) → S ≤ U := by
  let U : Submodule F V := ((⋂ i, u i).comap g).comap f
  have hcomp : ((⋂ i, u i).comap g).comap f = (⋂ i, u i).comap (g.comp f) := by
    ext x
    rfl
  have hpoint : ∀ x, x ∈ U ↔ ∀ i, g (f x) ∈ u i := by
    intro x
    change g (f x) ∈ (⋂ i, u i) ↔ ∀ i, g (f x) ∈ u i
    exact (Submodule.mem_iInf u)
  have hgreatest : ∀ S : Submodule F V, (∀ x ∈ S, ∀ i, g (f x) ∈ u i) → S ≤ U := by
    intro S hS x hx
    change g (f x) ∈ (⋂ i, u i)
    exact (Submodule.mem_iInf u).2 (fun i => hS x hx i)
  exact ⟨U, rfl, hcomp, hpoint, hgreatest⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Axler_exercise_1_8__me__c12b47d6

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint c12b47d6b95350e1

4.32 032. intersection_is_greatest_lower_bound

The problem

Prove that the intersection of any family of subspaces is the greatest subspace contained in every member of the family.

Lean statement

```
theorem intersection_is_greatest_lower_bound {F V : Type*} [AddCommGroup V] [Field F] [Module F V] {ι : Type*} (u : ι → Submodule F V) :
  ∃ U : Submodule F V,
    (⋂ i : ι, (u i).carrier) = (U : Set V) ∧
    ∀ W : Submodule F V, W ≤ U ↔ ∀ i : ι, W ≤ u i := by
```

Parent. Prove that the intersection of any collection of subspaces of V is a subspace of V .

Parent in Lean

```
theorem Axler_exercise_1_8 {F V : Type*} [AddCommGroup V] [Field F]
  [Module F V] {ι : Type*} (u : ι → Submodule F V) :
  ∃ U : Submodule F V, (⋂ i : ι, (u i).carrier) = ↑U := by
```

What it contributes. Supplies the canonical indexed-infimum construction and the carrier/intersection membership checkpoint, which is expanded into the final greatest-lower-bound theorem.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a genuine greatest-lower-bound characterization: proving the universal equivalence for every submodule W requires the new lattice argument ‘ le_iInf_iff ’, beyond the parent’s carrier-equality proof. The added conjunct is substantive rather than decorative or forced.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine greatest-lower-bound characterization: it must prove the universal property of the infimum for every submodule W , not merely identify the intersection with some submodule. Although it reuses the canonical infimum construction, recalling the parent’s existence proof does not establish this added claim.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine universal-property characterization: the chosen intersection submodule must be the greatest lower bound of the family. The parent’s carrier-equality proof does not establish this order-theoretic claim, which requires the new ‘ le_iInf_iff ’ reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2932 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem intersection_is_greatest_lower_bound.extracted_1_1.{u_3, u_2, u_1} {F : Type u_1} {V : Type u_2}
  [inst : AddCommGroup V] [inst_1 : Field F] [inst_2 : Module F V] {ι : Type u_3} (u : ι → Submodule F V) :
  ∃ U, ⋂ i, (u i).carrier = ↑U ∧ ∀ (W : Submodule F V), W ≤ U ↔ ∀ (i : ι), W ≤ u i
```

Read back by a model that saw only the Lean. Let F be a field, let V be an additive commutative group equipped with an F -module structure, let ι be a type, and let u assign to each $i : \iota$ a submodule $u(i)$ of V over F . Then there exists a submodule U of V over F such that the intersection of the carrier sets of all $u(i)$ equals the carrier set of U , and for every submodule W of V over F , W is contained in U if and only if W is contained in $u(i)$ for every $i : \iota$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open scoped BigOperators
open Fintype Complex Polynomial LinearMap FiniteDimensional Module Module.End

theorem intersection_is_greatest_lower_bound {F V : Type*} [AddCommGroup V] [Field F] [Module F V] {ι : Type*} (u : ι → Submodule F V) :
  ∃ U : Submodule F V,
    (⋂ i : ι, (u i).carrier) = (U : Set V) ∧
    ∀ W : Submodule F V, W ≤ U ↔ ∀ i : ι, W ≤ u i := by
  classical
  refine ⟨⋂ i, u i, ?_, ?_⟩
  · exact (Submodule.coe_iInf u).symm
  · intro W
    exact le_iInf_iff
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Axler_exercise_1_8__me__d2ece878

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint d2ece8786178579a

4.33 033. rotation_subgroup_reflections

The problem

Let R be the subgroup of rotations in a nontrivial dihedral group. Show that R is proper, every reflection lies outside R , and any subgroup containing R and one reflection contains every reflection and is the whole group.

Lean statement

```
theorem rotation_subgroup_reflections {n : ℕ} {hn : n ≠ 0}
  {R : Subgroup (DihedralGroup n)}
  (hR : R = Subgroup.closure {DihedralGroup.r (1 : ZMod n)}) :
  R ≠ ⊤ ∧
  (∀ i : ZMod n, DihedralGroup.sr i ∉ R) ∧
  (∀ K : Subgroup (DihedralGroup n), R ≤ K →
    (∃ i : ZMod n, DihedralGroup.sr i ∈ K) → K = ⊤)
```

Parent. Show that the subgroup of all rotations in a dihedral group is a maximal subgroup.

Parent in Lean

```
theorem Dummit_Foote_exercise_2_4_16b {n : ℕ} {hn : n ≠ 0}
  {R : Subgroup (DihedralGroup n)}
  (hR : R = Subgroup.closure {DihedralGroup.r 1}) :
  R ≠ ⊤ ∧
  ∀ K : Subgroup (DihedralGroup n), R ≤ K → K = R ∨ K = ⊤ := by
```

What it contributes. Supplies the rotation-subgroup presentation, zpowers characterization, rotation-membership checkpoint, and dihedral rotation/reflection case split.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child is not a wrapper or decoration: it proves the genuinely new statement that every reflection is excluded and that any subgroup containing the rotation subgroup plus one reflection is the whole dihedral group. The explicit multiplication checkpoint and resulting generation of all reflections are not supplied by the parent's maximal-subgroup dichotomy.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is not a recall wrapper: it proves every reflection is excluded from the rotation subgroup and establishes that any subgroup containing a reflection is top via an explicit reflection-product closure argument. This requires group multiplication and subgroup membership reasoning absent from the parent's maximal-subgroup dichotomy proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds the substantive claim that every reflection lies outside the rotation subgroup and proves that any subgroup containing a reflection is the whole dihedral group. Its explicit reflection-reflection multiplication closure argument is not supplied by the parent's maximality statement.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound

Check	By	Result
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2362 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem rotation_subgroup_reflections.extracted_1_1 {n : ℕ} {hn : n ≠ 0} {R : Subgroup (DihedralGroup n)}
(hR : R = Subgroup.closure {DihedralGroup.r 1}) :
R ≠ τ ∧
(∀ (i : ZMod n), DihedralGroup.sr i ∉ R) ∧
∀ (K : Subgroup (DihedralGroup n)), R ≤ K → (∃ i, DihedralGroup.sr i ∈ K) → K = τ
```

Read back by a model that saw only the Lean. *Let n be a nonzero natural number, and let R be a subgroup of the dihedral group of order parameter n . Suppose that R is the subgroup generated by the rotation $r(1)$. Then R is not the whole dihedral group; for every i in $ZMod\ n$, the reflection $sr(i)$ does not belong to R ; and for every subgroup K of the dihedral group, if R is contained in K and K contains at least one reflection $sr(i)$, then K is the whole dihedral group.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem rotation_subgroup_reflections {n : ℕ} {hn : n ≠ 0}
{R : Subgroup (DihedralGroup n)}
(hR : R = Subgroup.closure {DihedralGroup.r (1 : ZMod n)}) :
R ≠ τ ∧
(∀ i : ZMod n, DihedralGroup.sr i ∉ R) ∧
(∀ K : Subgroup (DihedralGroup n), R ≤ K →
(∃ i : ZMod n, DihedralGroup.sr i ∈ K) → K = τ) := by
classical
letI : NeZero n := ⟨hn⟩
have hR_zpowers :
Subgroup.zpowers (DihedralGroup.r (1 : ZMod n)) = R := by
exact
(Subgroup.zpowers_eq_closure (DihedralGroup.r (1 : ZMod n))).trans hR.symm
have hr_mem : ∀ j : ZMod n, DihedralGroup.r j ∈ R := by
intro j
obtain ⟨m, hm⟩ := (ZMod.intCast_surjective (n := n)) j
have hmem :
(DihedralGroup.r (1 : ZMod n)) ^ m ∈
Subgroup.zpowers (DihedralGroup.r (1 : ZMod n)) :=
Subgroup.zpow_mem_zpowers _ m
have hmemR : (DihedralGroup.r (1 : ZMod n)) ^ m ∈ R := by
simp [hR_zpowers] using hmem
simp [DihedralGroup.r_one_zpow (n := n) (k := m), hm] using hmemR
have hreflection_not_mem : ∀ i : ZMod n, DihedralGroup.sr i ∉ R := by
intro i hi
have hzpow :
DihedralGroup.sr i ∈ Subgroup.zpowers (DihedralGroup.r (1 : ZMod n)) := by
simp [hR_zpowers] using hi
rcases Subgroup.mem_zpowers_iff.mp hzpow with ⟨m, hm⟩
have hrot :
DihedralGroup.r (m : ZMod n) = DihedralGroup.sr i := by
simp [DihedralGroup.r_one_zpow (n := n) (k := m)] using hm
simp using hrot
have hmul : ∀ j i : ZMod n,
DihedralGroup.sr j * DihedralGroup.sr i ∈ R := by
intro j i
rw [DihedralGroup.sr_mul_sr]
```

```

    exact hr_mem (i - j)
  refine (?, hreflection_not_mem, ?_)
  · intro htop
    have hmem : DihedralGroup.sr 0 ∈ R := by simp [htop]
    exact hreflection_not_mem 0 hmem
  · intro K hRK hcontains
    obtain ⟨i, hi⟩ := hcontains
    have hall : ∀ j : ZMod n, DihedralGroup.sr j ∈ K := by
      intro j
      have hp : DihedralGroup.sr j * DihedralGroup.sr i ∈ K :=
        hRK (hmul j i)
      have hinv : (DihedralGroup.sr i)-1 ∈ K := K.inv_mem hi
      have hprod :
        (DihedralGroup.sr j * DihedralGroup.sr i) *
        (DihedralGroup.sr i)-1 ∈ K := K.mul_mem hp hinv
      simp using hprod
    apply top_unique
    intro x hx
    cases' x with j j
    · exact hRK (hr_mem j)
    · exact hall j

```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_2_4_16b__me__20c0330b

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 20c0330bce06b0fd

4.34 034. normalClosure_union_eq_sup

The problem

Let S and T be subsets of a group. Prove that the normal closure of $S \cup T$ is the join of the normal closures of S and T .

Lean statement

```
theorem normalClosure_union_eq_sup {G : Type*} [Group G] (S T : Set G) :
  Subgroup.normalClosure (S ∪ T) =
    Subgroup.normalClosure S ∪ Subgroup.normalClosure T
```

Parent. Let T be a subset of a group and let K be a normal subgroup. Prove that T is contained in K if and only if every subset of the normal closure of T has its normal closure contained in K .

Parent in Lean

```
theorem subset_iff_forall_normalClosure_subset {G : Type*} [Group G] (T : Set G)
  (K : Subgroup G) (hKnormal : K.Normal) :
  T ⊆ (K : Set G) ↔
    ∀ S : Set G, S ⊆ (Subgroup.normalClosure T : Set G) →
      Subgroup.normalClosure S ≤ K
```

What it contributes. Supplies the normal-closure universal-property checkpoint used for each inclusion.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a distinct union-to-join law, requiring separate inclusion arguments and monotonicity for both sets; the parent's quantified universal-property equivalence does not supply this conclusion by recall.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a distinct normal-closure distributivity law, requiring union-case reasoning and combining monotonicity with the supremum property. The parent's universal characterization does not directly supply this proof, so recalling it would not substantially help.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely different union-to-supremum equality, requiring antisymmetry, union case analysis, and monotonicity of normal closure. The parent's universal-property proof does not supply this argument, and the child does not embed or trivially reuse the parent.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2983 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normalClosure_union_eq_sup.extracted_1_1.{u_1} {G : Type u_1} [inst : Group G] (S T : Set G) :  
  normalClosure (S ∪ T) = normalClosure S ∪ normalClosure T
```

Read back by a model that saw only the Lean. For any group G and any subsets S and T of G , the normal closure of $S \cup T$ is equal to the supremum of the normal closures of S and T .

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
  
open Fintype Subgroup Set Polynomial Ideal  
open scoped BigOperators  
  
theorem normalClosure_union_eq_sup {G : Type*} [Group G] (S T : Set G) :  
  Subgroup.normalClosure (S ∪ T) =  
    Subgroup.normalClosure S ∪ Subgroup.normalClosure T := by  
  apply le_antisymm  
  · apply (Subgroup.normalClosure_subset_iff).mp  
    intro x hx  
    rcases hx with hx | hx  
    · exact (le_sup_left : Subgroup.normalClosure S ≤ Subgroup.normalClosure S ∪ Subgroup.normalClosure T)  
      (Subgroup.subset_normalClosure (s := S) hx)  
    · exact (le_sup_right : Subgroup.normalClosure T ≤ Subgroup.normalClosure S ∪ Subgroup.normalClosure T)  
      (Subgroup.subset_normalClosure (s := T) hx)  
  · apply sup_le  
    · exact Subgroup.normalClosure_mono Set.subset_union_left  
    · exact Subgroup.normalClosure_mono Set.subset_union_right
```

Operator. mutation / mutation_easy **Lineage depth.** 4 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.me.me.me.me__c944fb1d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c944fb1db4f020d0

4.35 035. subset_iInf_iff_forall_normalClosure_subset

The problem

Let T be a subset of a group and let (H_i) be a family of normal subgroups. Prove that T is contained in every H_i if and only if, for every subset S of the normal closure of T , the normal closure of S is contained in the intersection of all H_i .

Lean statement

```
theorem subset_iInf_iff_forall_normalClosure_subset {G : Type*} [Group G] {I : Type*}
  (T : Set G) (H : I → Subgroup G) (hNormal : ∀ i : I, (H i).Normal) :
  (∀ i : I, T ⊆ (H i : Set G)) ↔
  ∀ S : Set G, S ⊆ (Subgroup.normalClosure T : Set G) →
    Subgroup.normalClosure S ≤ ⋂ i : I, H i
```

Parent. Let T be a subset of a group and let K be a normal subgroup. Prove that T is contained in K if and only if every subset of the normal closure of T has its normal closure contained in K .

Parent in Lean

```
theorem subset_iff_forall_normalClosure_subset {G : Type*} [Group G] (T : Set G)
  (K : Subgroup G) (hKNormal : K.Normal) :
  T ⊆ (K : Set G) ↔
  ∀ S : Set G, S ⊆ (Subgroup.normalClosure T : Set G) →
    Subgroup.normalClosure S ≤ K
```

What it contributes. Supplies the uniform-subset normal-closure pattern and the generator extraction through `sub_normalClosure`; its containment argument is generalized to an indexed normal-subgroup infimum.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child introduces an indexed family of normal subgroups and must prove the equivalence through normality and membership properties of their infimum. The parent's single-subgroup argument does not supply the needed `iInf` reasoning, and the hypothesis `hNormal` is essential.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the single normal subgroup into an indexed family and changes the target to an infimum. Its proof requires establishing normality of the infimum and reasoning with indexed subgroup membership, beyond the parent's single-subgroup argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes a single normal subgroup to an indexed family and must establish normality of the infimum, membership in the infimum, and extraction of each component bound. The parent's closure argument helps locally but does not prove this simultaneous-family formulation.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing

Check	By	Result
corpus_dedup	hash	unique against 2981 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem subset_iInf_iff_forall_normalClosure_subset.extracted_1.1.{u_2, u_1} {G : Type u_1} [inst : Group G]
  {I : Type u_2} (T : Set G) (H : I → Subgroup G) (hNormal : ∀ (i : I), (H i).Normal) :
  (∀ (i : I), T ⊆ ↑(H i)) ↔ ∀ S ⊆ ↑(normalClosure T), normalClosure S ≤ ⋂ i, H i
```

Read back by a model that saw only the Lean. Let G be a group, let I be a type, let T be a subset of G , and let H assign to each $i \in I$ a normal subgroup $H(i)$ of G . Then the following are equivalent: (1) for every $i \in I$, T is contained in the underlying set of $H(i)$; (2) for every subset S of the underlying set of the normal closure of T in G , the normal closure of S is contained in the infimum subgroup $\bigcap_{i \in I} H(i)$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem subset_iInf_iff_forall_normalClosure_subset {G : Type*} [Group G] {I : Type*}
  (T : Set G) (H : I → Subgroup G) (hNormal : ∀ i : I, (H i).Normal) :
  (∀ i : I, T ⊆ (H i : Set G)) ↔
    ∀ S : Set G, S ⊆ (Subgroup.normalClosure T : Set G) →
      Subgroup.normalClosure S ≤ ⋂ i : I, H i := by
  classical
  have hInfNormal : (⋂ i : I, H i).Normal :=
    Subgroup.normal_iInf_normal hNormal
  let I : (⋂ i : I, H i).Normal := hInfNormal
  constructor
  · intro hAll S hST
    have hTInf : T ⊆ (⋂ i : I, H i : Subgroup G) := by
      intro x hx
      exact Subgroup.mem_iInf.mpr (fun i => hAll i hx)
    have hClosureT : Subgroup.normalClosure T ≤ ⋂ i : I, H i :=
      Subgroup.normalClosure_le_normal hTInf
    have hSInf : S ⊆ (⋂ i : I, H i : Subgroup G) := by
      intro x hx
      exact hClosureT (hST hx)
    exact Subgroup.normalClosure_subset_iff.mpr hSInf
  · intro hUniform
    have hTGenerated : T ⊆ (Subgroup.normalClosure T : Set G) := by
      exact Subgroup.subset_normalClosure
    have hClosureInf : Subgroup.normalClosure T ≤ ⋂ i : I, H i := by
      exact hUniform T hTGenerated
    intro i x hx
    exact (iInf_le H i) (hClosureInf (hTGenerated hx))
```

Operator. mutation / mutation_easy **Lineage depth.** 4 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.me.me.me.me__ecf2e464

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint ecf2e4649340e958

4.36 036. normalClosure_family_containment_iff

The problem

Let S and T be subsets of a group, and let (N_i) be a family of normal subgroups. If each set is contained in the normal closure of the other, prove that the normal closure of S is contained in every N_i if and only if the normal closure of T is contained in every N_i .

Lean statement

```
theorem normalClosure_family_containment_iff {G : Type*} [Group G] {ι : Type*} (S T : Set G) (N : ι → Subgroup G) (hnormal : ∀ i, (N i).Normal) (hST : S ⊆ (Subgroup.normalClosure T : Set G)) (hTS : T ⊆ (Subgroup.normalClosure S : Set G)) : (∀ i, Subgroup.normalClosure S ≤ N i) ↔ (∀ i, Subgroup.normalClosure T ≤ N i)
```

Parent. Let S and T be subsets of a group, and let M be a normal subgroup containing the normal closure of T . If S is contained in the normal closure of T , prove that the normal closure of S is contained in M .

Parent in Lean

```
theorem normalClosure_le_of_subset_normalClosure {G : Type*} [Group G] (S T : Set G) (M : Subgroup G) (hMnormal : M.Normal) (hTM : (Subgroup.normalClosure T : Set G) ⊆ M) (hST : S ⊆ (Subgroup.normalClosure T : Set G)) : Subgroup.normalClosure S ≤ M
```

What it contributes. Supplied the normal-closure containment mechanism: subset containment into a normal subgroup yields containment of the normal closure; this is applied separately for both transports.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is a genuine mutual-containment equivalence over an indexed family, requiring a fresh direction-by-direction argument using normality and the normal-closure universal property. The parent's one-way transport lemma does not directly supply either direction, and this is not the same characterization as the already-kept child.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves an indexed-family equivalence in both directions, using mutual containment to transfer each normal closure into every normal subgroup. The parent only supplies a one-way containment result for a single target subgroup, so recalling its proof does not substantially solve the child.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2967 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normalClosure_family_containment_iff.extracted_1_1.{u_2, u_1} {G : Type u_1} [inst : Group G] {ι : Type u_2}
(S T : Set G) (N : ι → Subgroup G) (hnormal : ∀ (i : ι), (N i).Normal) (hST : S ⊆ ↑(normalClosure T))
(hTS : T ⊆ ↑(normalClosure S)) : (∀ (i : ι), normalClosure S ≤ N i) ↔ ∀ (i : ι), normalClosure T ≤ N i
```

Read back by a model that saw only the Lean. *Let G be a group, let ι be a type, let S and T be subsets of G , and let N assign to each $i \in \iota$ a subgroup $N(i)$ of G . Suppose that $N(i)$ is a normal subgroup of G for every $i \in \iota$, S is contained in the underlying set of the normal closure of T in G , and T is contained in the underlying set of the normal closure of S in G . Then, for every $i \in \iota$, the normal closure of S in G is contained in $N(i)$ if and only if, for every $i \in \iota$, the normal closure of T in G is contained in $N(i)$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem normalClosure_family_containment_iff {G : Type*} [Group G] {ι : Type*} (S T : Set G) (N : ι → Subgroup G) (hnormal : ∀ i, (N
↪ i).Normal) (hST : S ⊆ (Subgroup.normalClosure T : Set G)) (hTS : T ⊆ (Subgroup.normalClosure S : Set G)) : (∀ i,
↪ Subgroup.normalClosure S ≤ N i) ↔ (∀ i, Subgroup.normalClosure T ≤ N i) := by
  constructor
  · intro hS i
    letI : (N i).Normal := hnormal i
    apply Subgroup.normalClosure_le_normal
    intro x hx
    exact hS i (hTS hx)
  · intro hT i
    letI : (N i).Normal := hnormal i
    apply Subgroup.normalClosure_le_normal
    intro x hx
    exact hT i (hST hx)
```

Operator. mutation / mutation_easy **Lineage depth.** 3 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.me.me.me__c614d88a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c614d88a8bf7793d

4.37 037. indexed_normalClosure_le_iInf

The problem

Let $\{H_i\}_{i \in I}$ be normal subgroups of a group G . If $T \subseteq H_i$ for every i and $S \subseteq \langle\langle T \rangle\rangle$, prove that $\langle\langle S \rangle\rangle \subseteq \bigcap_{i \in I} H_i$.

Lean statement

```
theorem indexed_normalClosure_le_iInf {G : Type*} [Group G] (I : Type*) (H : I → Subgroup G) (hH : ∀ i : I, (H i).Normal) (S T : Set G)
  ↪ (hT : ∀ i : I, T ⊆ H i) (hS : S ⊆ Subgroup.normalClosure T) : Subgroup.normalClosure S ≤ ⋂ i, H i
```

Parent. Prove that the infimum of any set of normal subgroups of a group is normal.

Parent in Lean

```
theorem set_iInf_normal_subgroups {G : Type*} [Group G] (S : Set (Subgroup G)) (hS : ∀ H ∈ S, H.Normal) : (sInf S : Subgroup G).Normal
```

What it contributes. Supplies the derived-target normality step for the indexed infimum, used as the local normal target for both closure transports.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a genuinely different containment theorem: it must show that generators in $\$T\$$ lie in the indexed infimum, propagate containment through normal closure to $\$S\$$, and then close again under normal closure. The parent only establishes normality of an infimum and does not supply these closure and membership arguments.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a substantive closure-containment result: it first establishes normality and containment in the indexed infimum, then applies normal closure minimality twice. The parent's sInf-normality argument alone does not supply this reasoning, and all child hypotheses are used.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires a new normal-closure argument: it first establishes normality of the indexed infimum, proves containment of T there, lifts this through normalClosure, and then handles S . The parent only establishes normality of a set infimum and does not supply the closure-containment reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2974 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem indexed_normalClosure_le_iInf.extracted_1.1.{u_2, u_1} {G : Type u_1} [inst : Group G] (I : Type u_2)
  (H : I → Subgroup G) (hH : ∀ (i : I), (H i).Normal) (S T : Set G) (hT : ∀ (i : I), T ⊆ ↑(H i))
  (hS : S ⊆ ↑(normalClosure T)) : normalClosure S ≤ ⋂ i, H i
```

Read back by a model that saw only the Lean. Let G be a group, let I be a type, and let H assign to each $i \in I$ a normal subgroup $H(i)$ of G . Let S and T be subsets of G . Assume that T is contained in $H(i)$ for every $i \in I$, and that S is contained in the normal closure of T in G . Then the normal closure of S in G is contained in the infimum of the subgroups $H(i)$ over all $i \in I$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem indexed_normalClosure_le_iInf {G : Type*} [Group G] (I : Type*)
  (H : I → Subgroup G) (hH : ∀ i : I, (H i).Normal)
  (S T : Set G) (hT : ∀ i : I, T ⊆ H i)
  (hS : S ⊆ Subgroup.normalClosure T) :
  Subgroup.normalClosure S ≤ ⋂ i, H i := by
have hnormal : (⋂ i, H i).Normal := by
  apply Subgroup.normal_iInf_normal
  intro i
  exact hH i
have hTinf : ∀ x ∈ T, x ∈ (⋂ i, H i) := by
  intro x hx
  rw [Subgroup.mem_iInf]
  intro i
  exact hT i hx
let I : (⋂ i, H i).Normal := hnormal
have hTclosure : Subgroup.normalClosure T ≤ (⋂ i, H i) := by
  exact Subgroup.normalClosure_le_normal (fun x hx => hTinf x hx)
have hSinTarget : ∀ x ∈ S, x ∈ (⋂ i, H i) := by
  intro x hx
  exact hTclosure (hS hx)
exact Subgroup.normalClosure_le_normal (fun x hx => hSinTarget x hx)
```

Operator. mutation / mutation_easy **Lineage depth.** 3 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.me.me.me__e215357d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint e215357d58211e88

4.38 038. normalClosure_le_iInf_iff

The problem

Let S be a subset of a group and let $(N_i)_{i \in I}$ be a family of normal subgroups. Prove that the normal closure of S is contained in $\bigcap_{i \in I} N_i$ if and only if S is contained in every N_i .

Lean statement

```
theorem normalClosure_le_iInf_iff {G : Type*} [Group G] {ι : Type*} (S : Set G)
  (N : ι → Subgroup G) (hnormal : ∀ i, (N i).Normal) :
  Subgroup.normalClosure S ≤ ⋂ i, N i ↔ ∀ i, S ≤ N i
```

Parent. Let S and T be subsets of a group. If S is contained in the normal closure of T , prove that the normal closure of S is contained in the normal closure of T .

Parent in Lean

```
theorem normalClosure_mono_of_subset {G : Type*} [Group G] (S T : Set G)
  (hST : S ⊆ (Subgroup.normalClosure T : Set G)) :
  Subgroup.normalClosure S ≤ Subgroup.normalClosure T
```

What it contributes. Supplied the normal-closure monotonicity and least-normal-overgroup checkpoints, generalized from one target normal closure to an arbitrary indexed family.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the parent from one normal-closure target to an arbitrary indexed infimum and requires both the infimum introduction/elimination arguments and pointwise normality. A solver recalling the parent's single-target proof would not directly solve the indexed equivalence.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes a single normal-closure inclusion to an indexed infimum and proves both directions using infimum bounds. The parent's monotonicity lemma supplies only the per-index step; it does not provide the new universal-property reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely characterizes containment in an indexed infimum, requiring both extraction through each infimum projection and reconstruction via 'le_iInf'; the parent only supplies the local normal-closure comparison. Its converse direction and indexed-family reasoning cannot be recovered by simply recalling the parent proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2954 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normalClosure_le_iInf_iff.extracted_1_1.{u_2, u_1} {G : Type u_1} [inst : Group G] {ι : Type u_2} (S : Set G)
(N : ι → Subgroup G) (hnormal : ∀ (i : ι), (N i).Normal) : normalClosure S ≤ ⋂ i, N i ↔ ∀ (i : ι), S ⊆ ↑(N i)
```

Read back by a model that saw only the Lean. *Let G be a group, let ι be an index type, let S be a subset of G , and let N assign to each $i \in \iota$ a normal subgroup $N(i)$ of G . Then the normal closure of S in G is contained in the infimum of the subgroups $N(i)$ if and only if, for every $i \in \iota$, S is contained in $N(i)$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem normalClosure_le_iInf_iff {G : Type*} [Group G] {ι : Type*} (S : Set G)
(N : ι → Subgroup G) (hnormal : ∀ i, (N i).Normal) :
Subgroup.normalClosure S ≤ ⋂ i, N i ↔ ∀ i, S ⊆ N i := by
constructor
· intro h i x hx
  exact (iInf_le N i) (h (Subgroup.subset_normalClosure hx))
· intro h
  refine le_iInf (fun i => ?_)
  letI : (N i).Normal := hnormal i
  exact Subgroup.normalClosure_le_normal (h i)
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.me.me__3bcaece7

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint 3bcaece7d5ad25a8

4.39 039. normalClosure_conj_image_iInf_iff

The problem

Let G be a group and let (H_i) be any family of normal subgroups. For a subset $S \subseteq G$ and $g \in G$, the normal closure of S lies in $\bigcap_i H_i$ if and only if the normal closure of gSg^{-1} does.

Lean statement

```
theorem normalClosure_conj_image_iInf_iff {G : Type*} [Group G] (I : Type*) (H : I → Subgroup G) (hH : ∀ i : I, (H i).Normal) (S : Set G)
  ↔ (g : G) : normalClosure ((fun x : G => g * x * g⁻¹) '' S) ≤ (⋂ i, H i) ↔ normalClosure S ≤ (⋂ i, H i)
```

Parent. Prove that the intersection of any indexed family of normal subgroups is closed under conjugation, even when the index set is empty.

Parent in Lean

```
theorem indexed_inf_conjugation_closed {G : Type*} [Group G] (I : Type*) (H : I → Subgroup G) (hH : ∀ i : I, (H i).Normal) : ∀ n, n ∈ (⋂
  i, H i) → ∀ g : G, g * n * g⁻¹ ∈ (⋂ i, H i)
```

What it contributes. Supplied the normality and conjugation-closure checkpoint for the indexed infimum, reused here as the transport mechanism for conjugated generators.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes a pointwise conjugation-closure result into an equivalence about containment of normal closures of conjugated sets. Proving both directions requires lifting set images through normal closure and applying conjugation by the inverse, which is not supplied by recalling the parent proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuine equivalence about normal closures under conjugating an entire generating set. It requires image unpacking, normal-closure containment, and conjugation by the inverse, none of which is supplied by the parent's direct membership proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves an equivalence about containment of normal closures under conjugating an entire generating set, requiring both image unpacking and reverse conjugation via normality. The parent only establishes elementwise conjugation closure of the indexed infimum, so recalling it does not supply the child's normal-closure argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2962 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normalClosure_conj_image_iInf_iff.extracted_1.1.{u_2, u_1} {G : Type u_1} [inst : Group G] (I : Type u_2)
  (H : I → Subgroup G) (hH : ∀ (i : I), (H i).Normal) (S : Set G) (g : G) :
  normalClosure ((fun x => g * x * g⁻¹) '' S) ≤ ⋂ i, H i ↔ normalClosure S ≤ ⋂ i, H i
```

Read back by a model that saw only the Lean. Let G be a group, let I be a type, and let $H : I \rightarrow \text{Subgroup } G$ be a family of normal subgroups of G . For any subset S of G and any $g \in G$, the normal closure in G of the conjugate set $\{g x g^{-1} \mid x \in S\}$ is contained in the intersection of all the subgroups $H i$ if and only if the normal closure of S is contained in that intersection.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem normalClosure_conj_image_iInf_iff {G : Type*} [Group G] (I : Type*) (H : I → Subgroup G) (hH : ∀ i : I, (H i).Normal) (S : Set G)
  (g : G) : normalClosure ((fun x : G => g * x * g⁻¹) '' S) ≤ (⋂ i, H i) ↔ normalClosure S ≤ (⋂ i, H i) := by
  let I : (⋂ i, H i).Normal := Subgroup.normal_iInf_normal hH
  constructor
  · intro hConj
    apply Subgroup.normalClosure_le_normal
    intro x hx
    have hxConj : g * x * g⁻¹ ∈ (⋂ i, H i) := by
      apply hConj
      exact Subgroup.subset_normalClosure ⟨x, hx, rfl⟩
    have hxBack := (show (⋂ i, H i).Normal from inferInstance).conj_mem' (g * x * g⁻¹) hxConj g
    simpa [mul_assoc] using hxBack
  · intro hS
    apply Subgroup.normalClosure_le_normal
    intro x hx
    rcases hx with ⟨y, hy, rfl⟩
    have hyInf : y ∈ (⋂ i, H i) := hS (Subgroup.subset_normalClosure hy)
    exact (show (⋂ i, H i).Normal from inferInstance).conj_mem y hyInf g
```

Operator. mutation / mutation_easy **Lineage depth.** 3 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.ms.me__d98b1ebb

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint d98b1ebb84522093

4.40 040. normalClosure_inter_subset

The problem

For subsets S and T of a group, show that the normal subgroup generated by $S \cap T$ is contained in the intersection of the normal subgroups generated by S and T .

Lean statement

```
theorem normalClosure_inter_subset {G : Type*} [Group G] (S T : Set G) : Subgroup.normalClosure (S ∩ T) ≤ Subgroup.normalClosure S ∩ Subgroup.normalClosure T
```

Parent. Prove that the intersection of any collection of normal subgroups of a group is a normal subgroup.

Parent in Lean

```
theorem collection_inf_normal {G : Type*} [Group G] (S : Set (Subgroup G)) (hS : ∀ N ∈ S, (N : Subgroup G).Normal) : (sInf S : Subgroup G).Normal
```

What it contributes. Uses the parent's normality-of-intersections checkpoint in the bounded two-subgroup meet setting.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a genuinely different inclusion using normal-closure minimality and normality of the meet; the parent's iInf -normality argument and closing lemma do not solve it by recall. It also introduces no redundant hypotheses or forced wrapper structure.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is not obtainable by recalling the parent's iInf -normality argument: it requires constructing the inclusion of the intersection into the meet and applying the normal-closure universal property, together with normality of the meet. Its proof uses different mathematical checkpoints and no parent theorem is embedded.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires a new universal-property argument: it proves that the meet of two normal closures is normal, contains the intersection, and therefore bounds the normal closure of that intersection. The parent only establishes normality of an infimum of already-normal subgroups and does not supply this closure-containment reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2828 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normalClosure_inter_subset.extracted_1_1.{u_1} {G : Type u_1} [inst : Group G] (S T : Set G) :  
  normalClosure (S ∩ T) ≤ normalClosure S ∩ normalClosure T
```

Read back by a model that saw only the Lean. For any group G and any subsets S and T of G , the normal closure of $S \cap T$ is contained in the infimum (equivalently, the intersection) of the normal closures of S and T .

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open Fintype Subgroup Set Polynomial Ideal  
open scoped BigOperators  
  
theorem normalClosure_inter_subset {G : Type*} [Group G] (S T : Set G) :  
  Subgroup.normalClosure (S ∩ T) ≤  
    Subgroup.normalClosure S ∩ Subgroup.normalClosure T := by  
  letI hSNormal : (Subgroup.normalClosure S).Normal :=  
    Subgroup.normalClosure_normal  
  letI hTNormal : (Subgroup.normalClosure T).Normal :=  
    Subgroup.normalClosure_normal  
  have hMeetNormal :  
    (Subgroup.normalClosure S ∩ Subgroup.normalClosure T).Normal :=  
    Subgroup.normal_inf_normal _ _  
  letI hMeetNormalInstance :  
    (Subgroup.normalClosure S ∩ Subgroup.normalClosure T).Normal := hMeetNormal  
  have hIntersection :  
    S ∩ T ≤ ↑(Subgroup.normalClosure S ∩ Subgroup.normalClosure T) := by  
    intro x hx  
    exact ⟨Subgroup.subset_normalClosure hx.1,  
      Subgroup.subset_normalClosure hx.2⟩  
  exact Subgroup.normalClosure_le_normal hIntersection
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.me__16f5f277

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 16f5f2777b528b85

4.41 041. normalCore_iInf_eq_iInf_normalCore

The problem

Let G be a group and let $(H_i)_{i \in I}$ be a nonempty indexed family of subgroups. Prove that the normal core of their intersection is the intersection of their normal cores.

Lean statement

```
theorem normalCore_iInf_eq_iInf_normalCore {G : Type*} [Group G] {I : Type*} [Nonempty I] (H : I → Subgroup G) :
  (⋂ i, H i).normalCore = ⋂ i, (H i).normalCore
```

Parent. For a subgroup H of a group G , its normal core is the largest normal subgroup contained in H : it is normal, lies in H , and contains every normal subgroup of G that lies in H .

Parent in Lean

```
theorem normalCore_greatest_normal_subgroup {G : Type*} [Group G] (H : Subgroup G) :
  H.normalCore.Normal ∧ H.normalCore ≤ H ∧
  ∀ N : Subgroup G, N.Normal → N ≤ H → N ≤ H.normalCore
```

What it contributes. Supplied the arbitrary-intersection normality checkpoint, generalized here to the family of normal cores.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a genuinely new indexed-infimum identity, requiring monotonicity, normality of an indexed infimum, and the universal property of the normal core. The parent's greatest-subgroup conjunction does not directly supply this proof, and no hypotheses are unused.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuine meet-preservation law for normalCore over an arbitrary nonempty indexed family. Its proof requires lattice inequalities and establishing that the indexed infimum of normal subgroups is normal, reasoning not supplied by the parent's single-subgroup characterization proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a substantive commutation law for normal cores over arbitrary nonempty indexed intersections. Its proof requires lattice-theoretic iInf reasoning and normality of an intersection, which is not supplied by recalling the parent's greatest-subgroup argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2309 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normalCore_iInf_eq_iInf_normalCore.extracted_1_1.{u_2, u_1} {G : Type u_1} [inst : Group G] {I : Type u_2}
  [inst_1 : Nonempty I] (H : I → Subgroup G) : (⋂ i, H i).normalCore = ⋂ i, (H i).normalCore
```

Read back by a model that saw only the Lean. *Let G be a group, let I be a nonempty type, and let H assign to each element i of I a subgroup $H(i)$ of G . Then the normal core of the infimum of all the subgroups $H(i)$ is equal to the infimum of their normal cores: $(\bigcap_i H(i)).\text{normalCore} = \bigcap_i H(i).\text{normalCore}$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem normalCore_iInf_eq_iInf_normalCore {G : Type*} [Group G] {I : Type*} [Nonempty I] (H : I → Subgroup G) :
  (⋂ i, H i).normalCore = ⋂ i, (H i).normalCore := by
  apply le_antisymm
  · refine le_iInf ?_
    intro i
    exact Subgroup.normalCore_mono (iInf_le H i)
  · have hN_normal : (⋂ i, (H i).normalCore).Normal :=
      Subgroup.normal_iInf_normal (fun i => (H i).normalCore_normal)
    letI : (⋂ i, (H i).normalCore).Normal := hN_normal
    have hN_le : (⋂ i, (H i).normalCore) ≤ ⋂ i, H i := by
      refine le_iInf ?_
      intro i
      exact le_trans (iInf_le (fun j => (H j).normalCore) i) (Subgroup.normalCore_le (H i))
    exact Subgroup.normal_le_normalCore.mpr hN_le
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.me__58f2f6b7

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 58f2f6b79940a756

4.42 042. normal_iInf_comap

The problem

Let G and K be groups, let I be a type, let $(H_i)_{i \in I}$ be a family of normal subgroups of K , and let $f : G \rightarrow K$ be a group homomorphism. Then the infimum of the subgroups $f^{-1}(H_i)$ is normal in G .

Lean statement

```
theorem normal_iInf_comap {G K : Type*} [Group G] [Group K] (I : Type*) (H : I → Subgroup K) (hH : ∀ i : I, Normal (H i)) (f : G →* K) :
  ⊆ Normal (⋂ i, (H i).comap f)
```

Parent. Prove that the intersection of an arbitrary indexed family of normal subgroups of a group is a normal subgroup, without assuming that the indexing type is nonempty.

Parent in Lean

```
theorem infimum_normal_subgroups {G : Type*} [Group G] (I : Type*)
  (H : I → Subgroup G) (hH : ∀ i : I, Normal (H i)) :
  Normal (⋂ (i : I), H i)
```

What it contributes. Supplied the indexed-infimum normality checkpoint, which is applied after each subgroup is replaced by its homomorphic comap.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a genuine transport argument: it proves each comapped subgroup is normal via the homomorphism comap operation, then applies normality of arbitrary infima. This is not supplied by the parent's proof, even though both use the same library infimum lemma.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child transports normality through each subgroup comap under a homomorphism before taking the indexed infimum. This requires the new comap-normality lemma and is not obtainable by merely recalling the parent's empty/nonempty index proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child transports normality through an arbitrary group homomorphism and then applies the indexed-infimum theorem to the comapped family. This requires the distinct mathematical lemma that comaps of normal subgroups are normal, so recalling the parent's empty-index case split and infimum argument is insufficient.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1907 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normal_iInf_comap.extracted_1_1.{u_3, u_2, u_1} {G : Type u_1} {K : Type u_2} [inst : Group G]
[inst_1 : Group K] (I : Type u_3) (H : I → Subgroup K) (hH : ∀ (i : I), (H i).Normal) (f : G →* K) :
(⋂ i, Subgroup.comap f (H i)).Normal
```

Read back by a model that saw only the Lean. *Let G and K be groups, let I be a type, let H assign to each $i : I$ a subgroup $H(i)$ of K , and suppose $H(i)$ is normal in K for every i . For any group homomorphism $f : G \rightarrow K$, the infimum over i of the subgroups of G given by the preimages $f^{-1}(H(i))$ is a normal subgroup of G .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem normal_iInf_comap {G K : Type*} [Group G] [Group K] (I : Type*)
(H : I → Subgroup K) (hH : ∀ i : I, Normal (H i)) (f : G →* K) :
Normal (⋂ i, (H i).comap f) := by
have hcomap : ∀ i : I, Normal ((H i).comap f) := by
intro i
exact (hH i).comap f
have hnormal : Normal (⋂ i, (H i).comap f) :=
Subgroup.normal_iInf_normal hcomap
simpa only using hnormal
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.me__9abe72ec

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint b3a6ac04a0858410

4.43 043. finite_normalCore_greatest

The problem

Let H_i be a finite family of subgroups of a group G . The infimum of their normal cores is the greatest normal subgroup of G contained in every H_i .

Lean statement

```
theorem finite_normalCore_greatest {G : Type*} [Group G] {ι : Type*} [Fintype ι]
  (H : ι → Subgroup G) :
  (⋂ i, (H i).normalCore).Normal ∧
  (⋂ i, (H i).normalCore = (⋂ i, H i).normalCore) ∧
  (∀ i, (⋂ j, (H j).normalCore) ≤ H i) ∧
  ∀ N : Subgroup G, N.Normal → (∀ i, N ≤ H i) →
    N ≤ ⋂ i, (H i).normalCore
```

Parent. For a subgroup H of a group G , its normal core is the largest normal subgroup contained in H : it is normal, lies in H , and contains every normal subgroup of G that lies in H .

Parent in Lean

```
theorem normalCore_greatest_normal_subgroup {G : Type*} [Group G] (H : Subgroup G) :
  H.normalCore.Normal ∧ H.normalCore ≤ H ∧
  ∀ N : Subgroup G, N.Normal → N ≤ H → N ≤ H.normalCore
```

What it contributes. Uses the parent normal-core normality, containment, and maximality properties; these are lifted from one subgroup to a finite infimum and combined with the distribution equality.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes from one subgroup to an indexed family and proves a new greatest-normal-subgroup characterization for the infimum, including a nontrivial commutation of normal core with infimum. Its proof requires family-level lattice reasoning and cannot be obtained by recalling the parent's single-subgroup argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires new reasoning about indexed infima: proving normality, containment in every member, and the equality between the normal core of an infimum and the infimum of normal cores. Although it reuses the parent's normal-core maximality lemma, the finite-family lattice argument is not supplied by the parent and is not merely the already-kept equality theorem.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely lifts the single-subgroup characterization to an indexed finite family: it must prove normality of an infimum, the normal-core/infimum equality, containment in every member, and the corresponding maximality property. These require new infimum and lattice reasoning beyond the parent's direct normal-core argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound

Check	By	Result
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2353 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	MISMATCH: Statement A restricts the family to be finite, whereas Statement B asserts the result for an arbitrary indexed family.

Goal round-trip: what Lean elaborated

```
theorem finite_normalCore_greatest.extracted_1.1.{u_2, u_1} {G : Type u_1} [inst : Group G] {ι : Type u_2}
  (H : ι → Subgroup G) :
  (Π i, (H i).normalCore).Normal ∧
  Π i, (H i).normalCore = (Π i, H i).normalCore ∧
  (∀ (i : ι), Π j, (H j).normalCore ≤ H i) ∧
  ∀ (N : Subgroup G), N.Normal → (∀ (i : ι), N ≤ H i) → N ≤ Π i, (H i).normalCore
```

Read back by a model that saw only the Lean. Let G be a group, let ι be a type, and let H be a family of subgroups of G indexed by ι . Then the intersection (infimum) of the normal cores of the subgroups $H(i)$ is a normal subgroup of G ; it equals the normal core of the intersection (infimum) of all the $H(i)$; it is contained in $H(i)$ for every i in ι ; and every normal subgroup N of G that is contained in $H(i)$ for every i in ι is contained in the intersection of the normal cores of the $H(i)$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem finite_normalCore_greatest {G : Type*} [Group G] {ι : Type*} [Fintype ι]
  (H : ι → Subgroup G) :
  (Π i, (H i).normalCore).Normal ∧
  (Π i, (H i).normalCore = (Π i, H i).normalCore) ∧
  (∀ i, (Π j, (H j).normalCore) ≤ H i) ∧
  ∀ N : Subgroup G, N.Normal → (∀ i, N ≤ H i) →
    N ≤ Π i, (H i).normalCore := by
  have hnormal : (Π i, (H i).normalCore).Normal :=
    Subgroup.normal_iInf_normal (fun i => (H i).normalCore_normal)
  letI : (Π i, (H i).normalCore).Normal := hnormal
  have hdist : (Π i, H i).normalCore = Π i, (H i).normalCore := by
    apply le_antisymm
    · refine le_iInf (fun i => ?_)
      exact Subgroup.normalCore_mono (iInf_le H i)
    · apply Subgroup.normal_le_normalCore.mpr
      exact le_iInf (fun i => le_trans (iInf_le _ i) (H i).normalCore_le)
  have hcandidate : ∀ i, (Π j, (H j).normalCore) ≤ H i := by
    intro i
    exact le_trans (iInf_le _ i) (H i).normalCore_le
  have hmax : ∀ N : Subgroup G, N.Normal → (∀ i, N ≤ H i) →
    N ≤ Π i, (H i).normalCore := by
    intro N hN hNH
    letI : N.Normal := hN
    rw [← hdist]
    apply Subgroup.normal_le_normalCore.mpr
    exact le_iInf (fun i => hNH i)
  exact ⟨hnormal, hdist.symm, hcandidate, hmax⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3.1.22b__me.me__a240affc

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint a240affc44df70c8

4.44 044. normalCore_iInf_finset_eq_iSup

The problem

Let H_i be a finite family of subgroups of a group G . Prove that the normal core of their intersection is the supremum of all normal subgroups contained in every H_i .

Lean statement

```
theorem normalCore_iInf_finset_eq_iSup {G α : Type*} [Group G] (s : Finset α) (H : α → Subgroup G) :
  (⋂ i ∈ s, H i).normalCore =
    ⋃ (N : Subgroup G) (N : Normal N) (N : ∀ i ∈ s, N ≤ H i), N
```

Parent. Let H_i be a finite nonempty family of subgroups of a group G . Prove that the normal core of their intersection equals the intersection of their normal cores.

Parent in Lean

```
theorem normalCore_iInf_finset {G α : Type*} [Group G] (s : Finset α) (H : α → Subgroup G) :
  (⋂ i ∈ s, (H i).normalCore) = (⋂ i ∈ s, H i).normalCore
```

What it contributes. Supplies the finite-intersection normal-core checkpoint, reproduced as `hfinite_core` and then consumed to identify the target core with the intersection of individual cores.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the equality into a supremum characterization of the normal core, requiring the independent theorem ‘Subgroup.normalCore_eq_iSup’ and indexed supremum reasoning. Memorizing the parent’s infimum-preservation proof would not substantially help solve it.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from commuting normalCore with a finite infimum to characterizing the normal core of that infimum as a supremum over all normal subgroups contained in every member. Its proof uses the independent supremum characterization and lattice bounds, so recalling the parent does not substantially help.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from an equality of normal cores over a finite infimum to an iSup characterization of the normal core itself. Its proof requires lattice supremum reasoning and the normalCore_eq_iSup theorem; memorizing the parent proof does not substantially solve this claim.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2160 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normalCore_iInf_finset_eq_iSup.extracted_1_1.{u_2, u_1} {G : Type u_1} {ι : Type u_2} [inst : Group G]
  (s : Finset ι) (H : ι → Subgroup G) :
  (⋂ i ∈ s, H i).normalCore = ⋃ N, ⋃ (_ : N.Normal), ⋃ (_ : ∀ i ∈ s, N ≤ H i), N
```

Read back by a model that saw only the Lean. For any group G , any type ι , any finite subset s of ι , and any family H assigning a subgroup $H(i)$ of G to each $i \in \iota$, the normal core of the infimum of the subgroups $H(i)$ for i in s equals the supremum, over all subgroups N of G , of N whenever N is normal in G and N is contained in $H(i)$ for every $i \in s$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem normalCore_iInf_finset_eq_iSup {G ι : Type*} [Group G] (s : Finset ι) (H : ι → Subgroup G) :
  (⋂ i ∈ s, H i).normalCore =
    ⋃ (N : Subgroup G) (_ : Normal N) (_ : ∀ i ∈ s, N ≤ H i), N := by
  rw [Subgroup.normalCore_eq_iSup]
  apply le_antisymm
  · refine iSup_le fun N => ?_
    refine iSup_le fun hN => ?_
    refine iSup_le fun hNK => ?_
    refine le_iSup_of_le N ?_
    refine le_iSup_of_le hN ?_
    refine le_iSup_of_le (fun i hi => ?_) le_rfl
    exact hNK.trans (iInf_le_of_le i (iInf_le_of_le hi le_rfl))
  · refine iSup_le fun N => ?_
    refine iSup_le fun hN => ?_
    refine iSup_le fun hNcontained => ?_
    refine le_iSup_of_le N ?_
    refine le_iSup_of_le hN ?_
    refine le_iSup_of_le (le_iInf fun i => le_iInf fun hi => hNcontained i hi) le_rfl
```

Operator. mutation / mutation_easy **Lineage depth.** 3 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.mh.mh.me__cae6f81f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint cae6f81f7639e060

4.45 045. normalCore_inf_of_normal

The problem

Let H be a subgroup of a group G , and let K be a normal subgroup. Prove that the normal core of $H \cap K$ equals the intersection of the normal core of H with K .

Lean statement

```
theorem normalCore_inf_of_normal {G : Type*} [Group G] (H K : Subgroup G) (hK : K.Normal) :
  (H ∩ K).normalCore = H.normalCore ∩ K
```

Parent. For a subgroup H of a group G , the normal core of H is normal. Moreover, every normal subgroup N is contained in the normal core exactly when it is contained in H .

Parent in Lean

```
theorem normalCore_greatest_normal_subgroup_reversed {G : Type*} [Group G] (H : Subgroup G) :
  H.normalCore.Normal ∧
  ∀ N : Subgroup G, N.Normal → (N ≤ H.normalCore ↔ N ≤ H)
```

What it contributes. Supplied the normal-core normality and universal containment property used for both inclusions.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a new intersection identity requiring lattice reasoning and specialization to the normal subgroup K ; it is not obtained by recalling the parent's greatest-subgroup characterization alone. Its proof uses the underlying normal-core lemmas in a genuinely different argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely different intersection identity: it must show both inclusions and use that a normal subgroup is its own normal core. The parent's greatest-subgroup characterization does not directly supply this equality or its lattice argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the parent's greatest-subgroup characterization into a substantive interaction law for normal-Core and infimum. Its proof requires separate inclusions and uses normality of the specialized subgroup K ; recalling the parent's conjunction and universal property is insufficient.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2345 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normalCore_inf_of_normal.extracted_1_1.{u_1} {G : Type u_1} [inst : Group G] (H K : Subgroup G)
(hK : K.Normal) : (H ∩ K).normalCore = H.normalCore ∩ K
```

Read back by a model that saw only the Lean. For any group G and subgroups H and K of G , if K is a normal subgroup of G , then the normal core of the intersection $H \cap K$ is equal to the intersection of the normal core of H with K .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem normalCore_inf_of_normal {G : Type*} [Group G] (H K : Subgroup G) (hK : K.Normal) :
(H ∩ K).normalCore = H.normalCore ∩ K := by
  letI : K.Normal := hK
  have hKcore : K.normalCore = K := Subgroup.normalCore_eq_self (H := K)
  rw [← hKcore]
  apply le_antisymm
  · have hN : (H ∩ K.normalCore).normalCore.Normal :=
      Subgroup.normalCore_normal (H ∩ K.normalCore)
    letI : (H ∩ K.normalCore).normalCore.Normal := hN
    have hH : (H ∩ K.normalCore).normalCore ≤ H.normalCore :=
      (Subgroup.normal_le_normalCore
        (H := H) (N := (H ∩ K.normalCore).normalCore)).2
      ((Subgroup.normalCore_le (H ∩ K.normalCore)).trans inf_le_left)
    have hK' : (H ∩ K.normalCore).normalCore ≤ K.normalCore :=
      (Subgroup.normalCore_le (H ∩ K.normalCore)).trans inf_le_right
    exact le_inf hH hK'
  · have hM : (H.normalCore ∩ K.normalCore).Normal := inferInstance
    letI : (H.normalCore ∩ K.normalCore).Normal := hM
    apply (Subgroup.normal_le_normalCore
      (H := H ∩ K.normalCore) (N := H.normalCore ∩ K.normalCore)).2
    exact inf_le_inf (Subgroup.normalCore_le H) le_rfl
```

Operator. mutation / mutation_easy **Lineage depth.** 3 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.ms.ms.me__8940ab06

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 8940ab06c94c567f

4.46 046. infimum_normal_subgroups

The problem

Prove that the intersection of an arbitrary indexed family of normal subgroups of a group is a normal subgroup, without assuming that the indexing type is nonempty.

Lean statement

```
theorem infimum_normal_subgroups {G : Type*} [Group G] (I : Type*)
  (H : I → Subgroup G) (hH : ∀ i : I, Normal (H i)) :
  Normal (⋂ (i : I), H i)
```

Parent. Prove that the intersection of an arbitrary nonempty collection of normal subgroups of a group is a normal subgroup (do not assume the collection is countable).

Parent in Lean

```
theorem Dummit_Foote_exercise_3.1.22b {G : Type*} [Group G] (I : Type*) [Nonempty I]
  (H : I → Subgroup G) (hH : ∀ i : I, Normal (H i)) :
  Normal (⋂ (i : I), H i) := by
```

What it contributes. Retains the parent's indexed family of normal subgroups and reuses `Subgroup.normal_iInf_normal` in the nonempty branch, while adding the empty-index/top-subgroup branch.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely removes the Nonempty I assumption and handles the new empty-index case by showing the infimum is $\top$ and hence normal. The parent lemma remains useful only for the nonempty branch, while the empty case requires new reasoning.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

Removing the nonemptiness assumption creates a genuine empty-index case, where the infimum is $\top$ and normality must be established separately. The parent theorem only handles the nonempty branch and does not supply this edge-case reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes to possibly empty index types. It must split on emptiness and prove that the empty infimum is $\top$ and hence normal, which the parent's nonempty-case proof does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propxt, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1900 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem infimum_normal_subgroups.extracted_1_1.{u_2, u_1} {G : Type u_1} [inst : Group G] (I : Type u_2)
  (H : I → Subgroup G) (hH : ∀ (i : I), (H i).Normal) : (⋂ i, H i).Normal
```

Read back by a model that saw only the Lean. *Let G be a group, let I be an index type, and let H assign to each $i \in I$ a subgroup $H(i)$ of G . If $H(i)$ is a normal subgroup of G for every $i \in I$, then the infimum (intersection) of all the subgroups $H(i)$ is a normal subgroup of G .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem infimum_normal_subgroups {G : Type*} [Group G] (I : Type*)
  (H : I → Subgroup G) (hH : ∀ i : I, Normal (H i)) :
  Normal (⋂ (i : I), H i) := by
  classical
  cases isEmpty_or_nonempty I with
  | inl hI =>
    letI := hI
    have htop : (⋂ (i : I), H i) = (τ : Subgroup G) := by
      simp
    have hnormal : Normal (τ : Subgroup G) := by
      infer_instance
    rw [htop]
    exact hnormal
  | inr hI =>
    letI := hI
    have hnormal : Normal (⋂ (i : I), H i) :=
      Subgroup.normal_iInf_normal hH
    exact hnormal
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me__1e63c082

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 29affb36027eda68

4.47 047. normalCore_greatest_normal_subgroup

The problem

For a subgroup H of a group G , its normal core is the largest normal subgroup contained in H : it is normal, lies in H , and contains every normal subgroup of G that lies in H .

Lean statement

```
theorem normalCore_greatest_normal_subgroup {G : Type*} [Group G] (H : Subgroup G) :
  H.normalCore.Normal ∧ H.normalCore ≤ H ∧
  ∀ N : Subgroup G, N.Normal → N ≤ H → N ≤ H.normalCore
```

Parent. Prove that the intersection of an arbitrary nonempty collection of normal subgroups of a group is a normal subgroup (do not assume the collection is countable).

Parent in Lean

```
theorem Dummit_Foote_exercise_3.1_22b {G : Type*} [Group G] (I : Type*) [Nonempty I]
  (H : I → Subgroup G) (hH : ∀ i : I, Normal (H i)) :
  Normal (⋂ (i : I), H i) := by
```

What it contributes. Uses the parent's arbitrary-intersection normality checkpoint via repeated applications of `Subgroup.normal_iInf_normal`, but consumes it inside the normal-core greatest-element characterization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child proves a genuine greatest-element characterization of the normal core, including normality, containment, and maximality among normal subgroups. This is substantially different mathematics from merely proving that an indexed intersection of normal subgroups is normal, so recalling the parent does not solve it.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes from proving normality of an arbitrary indexed intersection to characterizing the normal core via its normality, containment, and greatest-subgroup property. Its key final argument uses the normal-core universal property, which is not supplied by the parent's intersection-normality proof.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the task from proving normality of an indexed intersection to proving the normal core's full greatest-normal-subgroup characterization. Its proof uses normal-core-specific lemmas and a universal-property argument, so recalling the parent's exact proof does not substantially help.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2299 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normalCore_greatest_normal_subgroup.extracted_1.1.{u_1} {G : Type u_1} [inst : Group G] (H : Subgroup G) :  
  H.normalCore.Normal  $\wedge$  H.normalCore  $\leq$  H  $\wedge$   $\forall$  (N : Subgroup G), N.Normal  $\rightarrow$  N  $\leq$  H  $\rightarrow$  N  $\leq$  H.normalCore
```

Read back by a model that saw only the Lean. *For every group G and every subgroup H of G , the normal core $H.normalCore$ is a normal subgroup of G , $H.normalCore$ is contained in H , and for every subgroup N of G , if N is normal in G and N is contained in H , then N is contained in $H.normalCore$.*

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
  
open Fintype Subgroup Set Polynomial Ideal  
open scoped BigOperators  
  
theorem normalCore_greatest_normal_subgroup {G : Type*} [Group G] (H : Subgroup G) :  
  H.normalCore.Normal  $\wedge$  H.normalCore  $\leq$  H  $\wedge$   
     $\forall$  N : Subgroup G, N.Normal  $\rightarrow$  N  $\leq$  H  $\rightarrow$  N  $\leq$  H.normalCore := by  
  refine ⟨H.normalCore_normal, H.normalCore_le, ?_⟩  
  intro N hN hNH  
  letI : N.Normal := hN  
  exact Subgroup.normal_le_normalCore.mpr hNH
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me__306825ab

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint 306825ab3f3880b7

4.48 048. normal_core_characterization

The problem

For a subgroup H of a group G , prove that its normal core is normal, is contained in H , and contains every normal subgroup of G contained in H .

Lean statement

```
theorem normal_core_characterization {G : Type*} [Group G] (H : Subgroup G) :
  Normal H.normalCore ∧ H.normalCore ≤ H ∧
  ∀ N : Subgroup G, Normal N → N ≤ H → N ≤ H.normalCore
```

Parent. Prove that the intersection of an arbitrary nonempty collection of normal subgroups of a group is a normal subgroup (do not assume the collection is countable).

Parent in Lean

```
theorem Dummit_Foote_exercise_3.1_22b {G : Type*} [Group G] (I : Type*) [Nonempty I]
  (H : I → Subgroup G) (hH : ∀ i : I, Normal (H i)) :
  Normal (⋂ (i : I), H i) := by
```

What it contributes. Supplies the arbitrary-intersection normality checkpoint, consumed for the constant family whose intersection is the normal core.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the claim from closure of an arbitrary infimum of normal subgroups to the normal core's maximality characterization. Its proof requires the new containment and universal maximality arguments, so recalling the parent theorem is insufficient.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from normality of an indexed infimum to the universal characterization of a subgroup's normal core. Although it reuses the parent's normal-infimum lemma for one conjunct, the containment and greatest-normal-subgroup argument require distinct reasoning, so recalling the parent does not solve the child.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuine greatest-normal-subgroup characterization: the maximality clause requires the new normal-core containment lemma and is not supplied by the parent. Although it reuses the parent's normal-infimum lemma for one conjunct, recalling the parent does not solve the characterization.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2109 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normal_core_characterization.extracted_1_1.{u_1} {G : Type u_1} [inst : Group G] (H : Subgroup G) :
  H.normalCore.Normal  $\wedge$  H.normalCore  $\leq$  H  $\wedge$   $\forall$  (N : Subgroup G), N.Normal  $\rightarrow$  N  $\leq$  H  $\rightarrow$  N  $\leq$  H.normalCore
```

Read back by a model that saw only the Lean. *For a group G and a subgroup H of G , the normal core of H is a normal subgroup of G , is contained in H , and every normal subgroup N of G that is contained in H is also contained in the normal core of H .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem normal_core_characterization {G : Type*} [Group G] (H : Subgroup G) :
  Normal H.normalCore  $\wedge$  H.normalCore  $\leq$  H  $\wedge$ 
     $\forall$  N : Subgroup G, Normal N  $\rightarrow$  N  $\leq$  H  $\rightarrow$  N  $\leq$  H.normalCore := by
  have h_inter : Normal ( $\Pi$  _ : Unit, H.normalCore) := by
    exact Subgroup.normal_iInf_normal (fun _ => H.normalCore_normal)
  have hcore_normal : Normal H.normalCore := by
    simpa using h_inter
  have hcore_contained : H.normalCore  $\leq$  H := by
    exact H.normalCore_le
  have hmax :  $\forall$  N : Subgroup G, Normal N  $\rightarrow$  N  $\leq$  H  $\rightarrow$  N  $\leq$  H.normalCore := by
    intro N hN hNH
    letI : Normal N := hN
    exact Subgroup.normal_le_normalCore.mpr hNH
  exact (hcore_normal, hcore_contained, hmax)
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me__55131fb7

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 55131fb71d8c9551

4.49 049. indexedInf_intersection_normal

The problem

Prove that the intersection of a nonempty indexed family of normal subgroups of a group with another normal subgroup is a normal subgroup.

Lean statement

```
theorem indexedInf_intersection_normal {G : Type*} [Group G] (I : Type*) [Nonempty I]
  (H : I → Subgroup G) (K : Subgroup G)
  (hH : ∀ i : I, Normal (H i)) (hK : Normal K) :
  Normal ((⋂ (i : I), H i) ∩ K)
```

Parent. Prove that the intersection of an arbitrary nonempty collection of normal subgroups of a group is a normal subgroup (do not assume the collection is countable).

Parent in Lean

```
theorem Dummit_Foote_exercise_3.1.22b {G : Type*} [Group G] (I : Type*) [Nonempty I]
  (H : I → Subgroup G) (hH : ∀ i : I, Normal (H i)) :
  Normal (⋂ (i : I), H i) := by
```

What it contributes. Supplies the indexed-infimum normality checkpoint via `Subgroup.normal_iInf_normal`.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion to normality of an intersection and must combine the indexed-infimum normality checkpoint with ‘hK’ to prove conjugation closure componentwise. Although it reuses the parent’s lemma for the required local checkpoint, the final intersection argument is genuinely new mathematics.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely new intersection-normality claim, using the parent-style indexed-infimum checkpoint and separately combining it with normality of K via componentwise conjugation. The parent’s conclusion alone does not supply the final intersection argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires proving normality of an intersection, using conjugation closure for both the indexed infimum and K. Although it reuses the parent’s indexed-infimum checkpoint, the final intersection argument is new and satisfies the mutation_easy plan.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1893 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem indexedInf_intersection_normal.extracted_1_1.{u_2, u_1} {G : Type u_1} [inst : Group G] (I : Type u_2)
  [inst_1 : Nonempty I] (H : I → Subgroup G) (K : Subgroup G) (hH : ∀ (i : I), (H i).Normal) (hK : K.Normal) :
  ((⋂ i, H i) ⋂ K).Normal
```

Read back by a model that saw only the Lean. *Let G be a group, let I be a nonempty type, let H be a family of subgroups of G indexed by I , and let K be a subgroup of G . If every subgroup $H(i)$ is normal in G and K is normal in G , then the intersection of all the subgroups $H(i)$, intersected with K , is normal in G .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem indexedInf_intersection_normal {G : Type*} [Group G] (I : Type*) [Nonempty I]
  (H : I → Subgroup G) (K : Subgroup G)
  (hH : ∀ i : I, Normal (H i)) (hK : Normal K) :
  Normal ((⋂ (i : I), H i) ⋂ K) := by
  have hInf : Normal (⋂ (i : I), H i) :=
    Subgroup.normal_iInf_normal (fun i => hH i)
  have hIntersection : Normal ((⋂ (i : I), H i) ⋂ K) := by
    constructor
    intro g hg h
    exact ⟨hInf.conj_mem g hg.1 h, hK.conj_mem g hg.2 h⟩
  exact hIntersection
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me__bb03dfff

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propept, Classical.choice, Quot.sound, fingerprint ed5878fde5a25628

4.50 050. normal_inf_normal_specialization

The problem

If H and K are normal subgroups of a group G , prove that their intersection $H \cap K$ is normal in G .

Lean statement

```
theorem normal_inf_normal_specialization {G : Type*} [Group G] (H K : Subgroup G)
  [hH : H.Normal] [hK : K.Normal] : (H ∩ K).Normal
```

Parent. Prove that the intersection of an arbitrary nonempty collection of normal subgroups of a group is a normal subgroup (do not assume the collection is countable).

Parent in Lean

```
theorem Dummit_Foote_exercise_3.1.22b {G : Type*} [Group G] (I : Type*) [Nonempty I]
  (H : I → Subgroup G) (hH : ∀ i : I, Normal (H i)) :
  Normal (⋂ (i : I), H i) := by
```

What it contributes. Supplied the arbitrary-family normality checkpoint used in hFamily.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child is a genuine two-subgroup specialization: it constructs a Boolean-indexed family, proves its infimum is definitionally/equationally $H \cap K$, and only then applies the arbitrary-family normality lemma. Memorizing the parent proof does not supply the family construction or the indexed-infimum identification.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is a genuine two-subgroup specialization: it instantiates normality of an indexed infimum over Bool and separately proves that this infimum equals $H \cap K$. Recalling the parent's one-line application does not supply the nontrivial infimum identification.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires a genuine two-index encoding and a separate proof that the Boolean-indexed infimum equals $H \cap K$. The parent theorem supplies only normality of the indexed infimum, so recalling the parent proof does not finish the specialization.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2931 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normal_inf_normal_specialization.extracted_1_1.{u_1} {G : Type u_1} [inst : Group G] (H K : Subgroup G)
  [hH : H.Normal] [hK : K.Normal] : (H ∩ K).Normal
```

Read back by a model that saw only the Lean. *Let G be a group and let H and K be subgroups of G . If H and K are both normal subgroups of G , then their intersection $H \cap K$ is a normal subgroup of G .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem normal_inf_normal_specialization {G : Type*} [Group G] (H K : Subgroup G)
  [hH : H.Normal] [hK : K.Normal] : (H ∩ K).Normal := by
  let F : Bool → Subgroup G := fun b => if b then H else K
  have hFamily : (Π b : Bool, F b).Normal :=
    Subgroup.normal_iInf_normal (fun b => by
      cases b <|> simp [F, hH, hK])
  have hTwo : (Π b : Bool, F b) = H ∩ K := by
    apply le_antisymm
    · refine le_inf ?_ ?_
      · exact iInf_le_of_le true (by simp [F])
      · exact iInf_le_of_le false (by simp [F])
    · refine le_iInf fun b => ?_
      cases b <|> simp [F]
  rw [← hTwo]
  exact hFamily
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me__c5e2d40f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c5e2d40f0afcd6a

4.51 051. quotient_finset_disjoint_product_transport

The problem

For a commutative group A , a subgroup B , disjoint finite sets s, t of indices, and a function f from the index set to A , show that the product of the quotient images of $\prod_{i \in s} f(i)$ and $\prod_{i \in t} f(i)$ equals the quotient image of $\prod_{i \in s \cup t} f(i)$.

Lean statement

```
theorem quotient_finset_disjoint_product_transport {A ι : Type*} [CommGroup A] [DecidableEq ι] (B : Subgroup A) (s t : Finset ι) (hdisj :
  Disjoint s t) (f : ι → A) :
  ((s.prod f : A) : A / B) * ((t.prod f : A) : A / B) =
  (((s ∪ t).prod f : A) : A / B)
```

Parent. Let A be a commutative group, B a subgroup of A , s a finite set of indices, and f a function assigning an element of A to each index. Prove that the image in the quotient group A/B of the product of $f(i)$ over all $i \in s$ equals the product in A/B of the images of $f(i)$ over all $i \in s$.

Parent in Lean

```
theorem quotient_finset_product_transport {A ι : Type*} [CommGroup A] (B : Subgroup A) (s : Finset ι) (f : ι → A) :
  ((s.prod f : A) : A / B) =
  s.prod (fun i => (f i : A / B))
```

What it contributes. Supplies the quotient transport equalities `hs`, `ht`, and `hunions`; the new final obligation is the disjoint-union product decomposition.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the claim from transporting one finite product to a disjoint-union decomposition, requiring the independent ‘`Finset.prod_union`’ argument in addition to quotient transport. The parent lemma supplies normalization but does not establish the decomposition itself.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuine disjoint-index product decomposition, requiring `Finset.prod_union` in addition to quotient transport. Although it reuses the parent’s transport argument for each factor, recalling the parent alone does not establish the union identity.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion to a disjoint-union decomposition and requires the new mathematical step ‘`Finset.prod_union hdisj`’, beyond the parent’s single-set quotient transport. Although it reuses quotient transport for each factor, recalling the parent alone does not prove the union/product identity.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1986 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quotient_finset_disjoint_product_transport.extracted_1_1.{u_2, u_1} {A : Type u_1} {ι : Type u_2}
  [inst : CommGroup A] [inst_1 : DecidableEq ι] (B : Subgroup A) (s t : Finset ι) (hdisj : Disjoint s t) (f : ι → A) :
  ↑(s.prod f) * ↑(t.prod f) = ↑((s ∪ t).prod f)
```

Read back by a model that saw only the Lean. *Let A be a commutative group and ι a type with decidable equality. For any subgroup B of A , finite subsets s and t of ι that are disjoint, and any function $f : \iota \rightarrow A$, the product in the quotient group A/B of the s -indexed product of f and the t -indexed product of f equals the image in A/B of the $(s \cup t)$ -indexed product of f .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open scoped BigOperators
open Fintype Subgroup Set Polynomial Ideal

theorem quotient_finset_disjoint_product_transport {A ι : Type*} [CommGroup A] [DecidableEq ι] (B : Subgroup A) (s t : Finset ι) (hdisj : Disjoint s t) (f : ι → A) :
  ((s.prod f : A) : A / B) * ((t.prod f : A) : A / B) =
  (((s ∪ t).prod f : A) : A / B) := by
  have hs : ((s.prod f : A) : A / B) = s.prod (fun i => (f i : A / B)) := by
    simpa using (QuotientGroup.mk_prod B s (f := f))
  have ht : ((t.prod f : A) : A / B) = t.prod (fun i => (f i : A / B)) := by
    simpa using (QuotientGroup.mk_prod B t (f := f))
  have hunion : (((s ∪ t).prod f : A) : A / B) =
    (s ∪ t).prod (fun i => (f i : A / B)) := by
    simpa using (QuotientGroup.mk_prod B (s ∪ t) (f := f))
  rw [hs, ht, hunion]
  exact (Finset.prod_union hdisj).symm
```

Operator. mutation / mutation_easy **Lineage depth.** 3 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_3a__me.me.me.me__3331197a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 01b51bd9ea149c2d

4.52 052. quotient_finset_product_transport

The problem

Let A be a commutative group, B a subgroup of A , s a finite set of indices, and f a function assigning an element of A to each index. Prove that the image in the quotient group A/B of the product of $f(i)$ over all $i \in s$ equals the product in A/B of the images of $f(i)$ over all $i \in s$.

Lean statement

```
theorem quotient_finset_product_transport {A 1 : Type*} [CommGroup A] (B : Subgroup A) (s : Finset 1) (f : 1 → A) :
  ((s.prod f : A) : A / B) =
    s.prod (fun i => (f i : A / B))
```

Parent. For a commutative group and subgroup, the quotient projection of a finite list product equals the product of the quotient projections.

Parent in Lean

```
theorem quotient_list_product_transport {A : Type*} [CommGroup A] (B : Subgroup A) (l : List A) :
  ((l.prod : A) : A / B) =
    (l.map (fun a : A => (a : A / B))).prod
```

What it contributes. Generalizes the parent list-product transport law to a Finset-indexed product and uses the quotient projection/product compatibility checkpoint.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes from ordered-list induction to a finite-set product law over an arbitrary index type, and its proof requires the quotient group's finset product homomorphism theorem. The parent's list induction proof does not directly supply this result.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely changes the transport domain from ordered lists to arbitrary finite sets and functions, proved via the finite-product quotient homomorphism lemma. A solver recalling the parent's list induction proof would not directly obtain this Finset statement or its library-based proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes list transport to arbitrary finite-indexed products, requiring the quotient homomorphism's Finset product theorem rather than recalling the parent's list induction. Its proof uses a distinct substantive lemma, not decoration or a forced wrapper.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1975 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quotient_finset_product_transport.extracted_1_1.{u_2, u_1} {A : Type u_1} {ι : Type u_2} [inst : CommGroup A]
  (B : Subgroup A) (s : Finset ι) (f : ι → A) : ↑(s.prod f) = ∏ i ∈ s, ↑(f i)
```

Read back by a model that saw only the Lean. *For any commutative group A , subgroup B of A , finite set s of indices, and function $f : \iota \rightarrow A$, the coset of the product of $f i$ over all i in s equals the product, over all i in s , of the cosets of $f i$ in the quotient group A/B .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem quotient_finset_product_transport {A ι : Type*} [CommGroup A] (B : Subgroup A) (s : Finset ι) (f : ι → A) :
  ((s.prod f : A) : A / B) =
    s.prod (fun i => (f i : A / B)) := by
  simpa using (QuotientGroup.mk_prod B s (f := f))
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_3a__me.me.me__af6e130a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint cf624e07f7127f1b

4.53 053. quotient_list_product_reordering

The problem

For a commutative group quotient, finite products of quotient representatives are invariant under reordering.

Lean statement

```
theorem quotient_list_product_reordering {A : Type*} [CommGroup A] (B : Subgroup A) :
  ∀ l l' : List A,
    l.Perm l' →
      (l.map (fun a : A => (a : A / B))).prod =
        (l'.map (fun a : A => (a : A / B))).prod
```

Parent. Let A be an abelian group and let B be a subgroup of A . Prove that A/B is abelian.

Parent in Lean

```
theorem Dummit_Foote_exercise_3_1_3a {A : Type*} [CommGroup A] (B : Subgroup A) :
  ∀ a b : A / B, a * b = b * a := by
```

What it contributes. Uses the parent’s commutative-group quotient setting; the child upgrades binary commutativity to a representative-list product reordering law.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child generalizes binary commutativity to invariance of arbitrary finite quotient-group products under list permutation. Its proof uses permutation mapping and ‘List.Perm.prod_eq’, requiring a genuinely different finite-product argument than the parent’s direct ‘mul_comm’ application.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely broader finite-product reordering law for lists via permutation transport and ‘List.Perm.prod_eq’, rather than reusing the parent’s binary commutativity proof. A solver remembering the parent would still need the distinct list-permutation/product lemma.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes binary quotient commutativity into a finite-product reordering law for arbitrary Perm-related lists. Its proof uses mapped-list permutation and ‘List.Perm.prod_eq’, requiring a different finite-product lemma rather than recalling the parent’s ‘mul_comm’ proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1964 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quotient_list_product_reordering.extracted_1_1 : ∀ {A : Type u_1} [inst : CommGroup A] (B : Subgroup A)
  (l l' : List A), l.Perm l' → (List.map (fun a => ↑a) l).prod = (List.map (fun a => ↑a) l').prod
```

Read back by a model that saw only the Lean. *For every type A equipped with a commutative group structure, every subgroup B of A , and all lists l and l' of elements of A , if l is a permutation of l' , then the products in the quotient group A/B of the images of the elements of l and l' are equal.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem quotient_list_product_reordering {A : Type*} [CommGroup A] (B : Subgroup A) :
  ∀ l l' : List A,
    l.Perm l' →
      (l.map (fun a : A => (a : A / B))).prod =
        (l'.map (fun a : A => (a : A / B))).prod := by
  intro l l' h
  have hq : (l.map (fun a : A => (a : A / B))).Perm
    (l'.map (fun a : A => (a : A / B))) := h.map _
  exact hq.prod_eq
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_3a__me__09c322d9

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint a08ee45cfe6afbae

4.54 054. prime_order_element_of_dvd_card

The problem

Let G be a finite abelian group and let p be prime. If p divides the order of G , show that G contains an element of order p .

Lean statement

```
theorem prime_order_element_of_dvd_card {G : Type*} [CommGroup G] [Fintype G] {p : ℕ}
  (hp : Nat.Prime p) (hdiv : p ∣ Fintype.card G) :
  ∃ x : G, orderOf x = p
```

Parent. Use Cauchy's Theorem and induction to show that a finite abelian group has a subgroup of order n for each positive divisor n of its order.

Parent in Lean

```
theorem Dummit_Foote_exercise_3.4.4 {G : Type*} [CommGroup G] [Fintype G] {n : ℕ}
  (hn : n ∣ (card G)) :
  ∃ (H : Subgroup G) (H_fin : Fintype H), @card H H_fin = n := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from constructing a subgroup of cardinality n to constructing an element of prime order p . Its proof uses Cauchy's prime-order theorem directly and does not recall or adapt the parent's induction or Sylow subgroup argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from existence of a subgroup of order n to existence of an element of prime order p , and proves it via the finite-group Cauchy theorem. This is independent of the parent's truncated induction and quotient/Sylow construction, satisfying the mutation_easy plan.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is a genuinely different existence claim: it constructs an element of prime order using Cauchy's theorem, rather than constructing a subgroup of prescribed order via the parent's induction and Sylow argument. Its proof is independent of the parent and uses both prime-divisibility hypotheses substantively.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2200 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem prime_order_element_of_dvd_card.extracted_1_1.{u_1} {G : Type u_1} [inst : CommGroup G] [inst_1 : Fintype G]
  {p : ℕ} (hp : Nat.Prime p) (hdiv : p ∣ card G) : ∃ x, orderOf x = p
```

Read back by a model that saw only the Lean. *Let G be a finite commutative group, let p be a prime natural number, and suppose that p divides the cardinality of G . Then there exists an element x of G whose order is p .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem prime_order_element_of_dvd_card {G : Type*} [CommGroup G] [Fintype G] {p : ℕ}
  (hp : Nat.Prime p) (hdiv : p ∣ Fintype.card G) :
  ∃ x : G, orderOf x = p := by
  letI : Fact (Nat.Prime p) := ⟨hp⟩
  have hp_dvd_card : p ∣ Fintype.card G := hdiv
  have hresult : ∃ x : G, orderOf x = p :=
    exists_prime_orderOf_dvd_card (G := G) (p := p) hp_dvd_card
  exact hresult
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_4_4__me__e33d29b2

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint e33d29b28d212897

4.55 055. exists_order_two_subgroup_with_nontrivial_quotient

The problem

Every finite commutative group of order $2n$, where $2 \leq n \leq 10$, has a normal subgroup of order 2 such that the quotient group has order n and is nontrivial.

Lean statement

```
theorem exists_order_two_subgroup_with_nontrivial_quotient {G : Type*} [Fintype G] [CommGroup G] {n : ℕ} (hn : 2 ≤ n ∧ n ≤ 10) (hG :
  ↪ Fintype.card G = 2 * n) : ∃ H : Subgroup G, H.Normal ∧ Nat.card H = 2 ∧ Nat.card (G / H) = n ∧ Nontrivial (G / H)
```

Parent. Prove that every finite abelian group of order $2n$, where $2 \leq n \leq 10$, has a nontrivial proper normal subgroup.

Parent in Lean

```
theorem exists_proper_nontrivial_normal_subgroup_of_bounded_double_order {G : Type*} [Fintype G] [CommGroup G] {n : ℕ} (hn : 2 ≤ n ∧ n ≤
  ↪ 10) (hG : Fintype.card G = 2 * n) : ∃ H : Subgroup G, H.Normal ∧ H ≠ 1 ∧ H ≠ ⊤
```

What it contributes. Reuses the parent's finite counting and Sylow order-two subgroup construction, then consumes that subgroup in the quotient cardinality and nontriviality arguments.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child preserves the Sylow order-two construction but adds genuinely new quotient mathematics: it proves the exact quotient cardinality via the subgroup-cardinality formula and derives quotient nontriviality from properness. The parent's proof does not supply either quotient conclusion.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds substantive quotient mathematics absent from the parent: it derives the exact quotient cardinality using the subgroup cardinality formula and proves quotient nontriviality from properness. Although it reuses the Sylow construction and normality argument, recalling the parent does not supply these quotient-level conclusions.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child introduces genuinely new quotient mathematics: it derives the exact cardinality of G / H and proves quotient nontriviality, neither of which follows by recalling the parent's proof. The shared Sylow construction is only the setup for this new conclusion.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1929 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem exists_order_two_subgroup_with_nontrivial_quotient.extracted_1_1.{u_1} {G : Type u_1} [inst : Fintype G]
  [inst.1 : CommGroup G] {n : ℕ} (hn : 2 ≤ n ∧ n ≤ 10) (hG : card G = 2 * n) :
  ∃ H, H.Normal ∧ Nat.card H = 2 ∧ Nat.card (G / H) = n ∧ Nontrivial (G / H)
```

Read back by a model that saw only the Lean. *Let G be a finite commutative group, let n be a natural number such that $2 \leq n \leq 10$, and suppose that the cardinality of G is $2n$. Then there exists a normal subgroup H of G whose cardinality is 2, such that the quotient group G/H has cardinality n and is nontrivial.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem exists_order_two_subgroup_with_nontrivial_quotient {G : Type*} [Fintype G] [CommGroup G] {n : ℕ} (hn : 2 ≤ n ∧ n ≤ 10) (hG :
  card G = 2 * n) : ∃ H : Subgroup G, H.Normal ∧ Nat.card H = 2 ∧ Nat.card (G / H) = n ∧ Nontrivial (G / H) := by
  classical
  have hcount : Nat.card G = 2 * n := by
    simpa [Nat.card_eq_fintype_card] using hG
  letI : Fact (Nat.Prime 2) := ⟨by decide⟩
  have hpow : 2 ^ 1 ∣ Nat.card G := by
    rw [hcount]
    rcases hn with ⟨hnlo, hnhi⟩
    interval_cases n <|> norm_num
  obtain ⟨H, hH⟩ := Sylow.exists_subgroup_card_pow_prime (G := G) (p := 2) (n := 1) hpow
  have hHcard : Nat.card H = 2 := by
    simpa using hH
  have hnormal : H.Normal := by
    constructor
    intro x hx g
    rw [CommGroup.mul_comm g x]
    simpa [mul_assoc] using hx
  have hquotcard : Nat.card (G / H) = n := by
    have hmul := Subgroup.card_eq_card_quotient_mul_card_subgroup H
    rw [hcount, hHcard] at hmul
    omega
  have hproper : H ≠ ⊤ := by
    intro htop
    rw [htop] at hHcard
    simp at hHcard
    omega
  have hquotnontrivial : Nontrivial (G / H) := by
    exact QuotientGroup.nontrivial_iff.mpr hproper
  exact ⟨H, hnormal, hHcard, hquotcard, hquotnontrivial⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_4_5_23__me.me__7c82a3d7

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 88765ef7f846f66d

4.56 056. exists_proper_nontrivial_normal_subgroup_of_bounded_double_order

The problem

Prove that every finite abelian group of order $2n$, where $2 \leq n \leq 10$, has a nontrivial proper normal subgroup.

Lean statement

```
theorem exists_proper_nontrivial_normal_subgroup_of_bounded_double_order {G : Type*} [Fintype G] [CommGroup G] {n : ℕ} (hn : 2 ≤ n ∧ n ≤ 10) (hG : Fintype.card G = 2 * n) : ∃ H : Subgroup G, H.Normal ∧ H ≠ 1 ∧ H ≠ G
```

Parent. Prove that if $|G| = 462$ then G is not simple.

Parent in Lean

```
theorem Dummit_Foote_exercise_4_5_23 {G : Type*} [Fintype G] [Group G]
(hG : card G = 462) : ¬ IsSimpleGroup G := by
```

What it contributes. Reuses the parent's finite-counting checkpoint and Sylow subgroup-counting surface, generalized from the fixed order 462 case to bounded orders $2*n$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely changes the theorem: it gives an explicit order-2 proper nontrivial normal subgroup for a bounded family of finite abelian groups, using Sylow existence and commutativity. This does not rely on recalling the parent's fixed-order 462 Sylow-11 obstruction.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the group order and proves existence via a Sylow-2 subgroup, then uses commutativity to establish normality and cardinality-based nontriviality/properness. This is not recoverable by recalling the parent's fixed-order Sylow-11 nonsimplicity argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is a genuine bounded parameterized theorem with an explicit Sylow-2 subgroup witness, commutativity-based normality, and separate nontriviality/properness arguments. Its proof does not recall or reuse the parent's fixed-order Sylow-11 obstruction.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1905 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem exists_proper_nontrivial_normal_subgroup_of_bounded_double_order.extracted_1_1.{u_1} {G : Type u_1}
  [inst : Fintype G] [inst_1 : CommGroup G] {n : ℕ} (hn : 2 ≤ n ∧ n ≤ 10) (hG : card G = 2 * n) :
  ∃ H, H.Normal ∧ H ≠ 1 ∧ H ≠ τ
```

Read back by a model that saw only the Lean. *Let G be a finite commutative group, let n be a natural number with $2 \leq n \leq 10$, and suppose that the cardinality of G is $2n$. Then there exists a normal subgroup H of G such that H is neither the trivial subgroup nor the whole group G .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem exists_proper_nontrivial_normal_subgroup_of_bounded_double_order {G : Type*} [Fintype G] [CommGroup G] {n : ℕ} (hn : 2 ≤ n ∧ n ≤
  <- 10) (hG : Fintype.card G = 2 * n) : ∃ H : Subgroup G, H.Normal ∧ H ≠ 1 ∧ H ≠ τ := by
  classical
  have hcount : Nat.card G = 2 * n := by
    simpa [Nat.card_eq_fintype_card] using hG
  haveI : Fact (Nat.Prime 2) := <by decide>
  have hpow : 2 ^ 1 | Nat.card G := by
    rw [hcount]
    rcases hn with <hnlo, hnhi>
    interval_cases n <|> norm_num
  obtain (H, hH) := Sylow.exists_subgroup_card_pow_prime (G := G) (p := 2) (n := 1) hpow
  have hHcard : Nat.card H = 2 := by
    simpa using hH
  have hnormal : H.Normal := by
    constructor
    intro x hx g
    rw [CommGroup.mul_comm g x]
    simpa [mul_assoc] using hx
  have hnontrivial : H ≠ 1 := by
    intro hbot
    rw [hbot] at hHcard
    simp at hHcard
  have hproper : H ≠ τ := by
    intro htop
    rw [htop] at hHcard
    have hGcard : Nat.card G = 2 := by
      simpa using hHcard
    rw [hcount] at hGcard
    omega
  exact <H, hnormal, hnontrivial, hproper>
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_4_5_23__me__2afb89cf

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 92e34d98d1b53b3b

4.57 057. order_462_subgroup_gaussian_divisibility

The problem

Every finite group of order 462 contains a subgroup of odd order n such that $(1+i)^2$ divides $n^2 - 1$ in the Gaussian integers.

Lean statement

```
theorem order_462_subgroup_gaussian_divisibility {G : Type*} [Fintype G] [Group G]
  (hG : Fintype.card G = 462) :
  ∃ H : Subgroup G, Odd (Nat.card H) ∧
    ((1, 1) : GaussianInt) ^ 2 ∣ (Nat.card H ^ 2 - 1 : GaussianInt)
```

Parent. Prove that if $|G| = 462$ then G is not simple.

Parent in Lean

```
theorem Dummit_Foote_exercise_4.5_23 {G : Type*} [Fintype G] [Group G]
  (hG : card G = 462) : ¬ IsSimpleGroup G := by
```

What it contributes. Reuses the parent's order-462 finite-group domain and cardinality normalization checkpoint, while changing the conclusion to an explicit Sylow subgroup and Gaussian divisibility target.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child uses Sylow existence directly to construct a subgroup of cardinality 11, then proves a new Gaussian-integer divisibility property by arithmetic. This is not supplied by the parent's contradiction and Sylow-counting argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child uses Sylow existence to construct a subgroup of cardinality 11, then proves oddness and a Gaussian-integer divisibility fact. None of this follows from or recalls the parent's non-simplicity argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child uses Sylow subgroup existence directly to construct a subgroup of cardinality 11, then proves oddness and a new Gaussian-integer divisibility statement. This reasoning is not supplied by recalling the parent's non-simplicity conclusion.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2937 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem order_462_subgroup_gaussian_divisibility.extracted_1.1.{u_1} {G : Type u_1} [inst : Fintype G]
  [inst.1 : Group G] (hG : card G = 462) :
  ∃ H, Odd (Nat.card ↑H) ∧ { re := 1, im := 1 } ^ 2 | ↑(Nat.card ↑H) ^ 2 - 1
```

Read back by a model that saw only the Lean. *Let G be a finite group with $|G| = 462$. Then there exists a subgroup H of G such that the cardinality of H is odd and the Gaussian integer $(1 + i)^2$ divides $|H|^2 - 1$, where $|H|$ is viewed as a Gaussian integer.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem order_462_subgroup_gaussian_divisibility {G : Type*} [Fintype G] [Group G]
  (hG : Fintype.card G = 462) :
  ∃ H : Subgroup G, Odd (Nat.card H) ∧
    ((1, 1) : GaussianInt) ^ 2 | (Nat.card H ^ 2 - 1 : GaussianInt) := by
  letI : Fact (Nat.Prime 11) := ⟨by norm_num⟩
  have hcard : Nat.card G = 462 := by
    simpa [Nat.card_eq_fintype_card] using hG
  have hdiv : 11 ^ 1 | Nat.card G := by
    rw [hcard]
    norm_num
  obtain ⟨H, hH⟩ := Sylow.exists_subgroup_card_pow_prime (G := G) 11 (n := 1) hdiv
  have h_cardH : Nat.card H = 11 := by
    simpa using hH
  refine ⟨H, ?_, ?_⟩
  · rw [h_cardH]
    norm_num
  · rw [h_cardH]
    have hsq : ((1, 1) : GaussianInt) ^ 2 = ⟨0, 2⟩ := by
      rw [pow_two]
      apply Zsqrtd.ext <|> norm_num [Zsqrtd.re_mul, Zsqrtd.im_mul]
    rw [hsq]
    refine ⟨⟨0, -60⟩, ?_⟩
    apply Zsqrtd.ext <|> norm_num [Zsqrtd.re_mul, Zsqrtd.im_mul]
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_4_5_23__me__58a9e5ec

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 58a9e5ecf3ba4f34

4.58 058. order_462_has_nontrivial_proper_normal_subgroup

The problem

Let G be a finite group of order 462. Prove that G has a nontrivial proper normal subgroup.

Lean statement

```
theorem order_462_has_nontrivial_proper_normal_subgroup {G : Type*} [Fintype G] [Group G]
  (hG : Fintype.card G = 462) :
  ∃ H : Subgroup G, H.Normal ∧ H ≠ 1 ∧ H ≠ ⊤ := by
```

Parent. Prove that if $|G| = 462$ then G is not simple.

Parent in Lean

```
theorem Dummit_Foote_exercise_4_5_23 {G : Type*} [Fintype G] [Group G]
  (hG : card G = 462) : ¬ IsSimpleGroup G := by
```

What it contributes. Supplied the order-462 Sylow-11 counting strategy: divisibility of the Sylow count, congruence modulo 11, uniqueness, and normality.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child replaces the parent's contradiction-to-simplicity conclusion with an explicit Sylow-11 subgroup witness. Beyond counting Sylow subgroups, it proves normality from uniqueness and separately establishes nontriviality and properness using divisibility and the exact 11-adic factor of 462.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the parent's contradiction-to-simplicity result with an explicit existential construction of a unique Sylow-11 subgroup and proves it is normal, nontrivial, and proper. These final structural properties require reasoning beyond merely recalling the parent's negated-simplicity conclusion.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the parent's contradiction-to-nonsimplicity with an explicit Sylow-11 subgroup witness, requiring new proofs of uniqueness, normality, nontriviality, and properness. A memorized proof of the parent does not directly provide this existential structural conclusion.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2956 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem order_462_has_nontrivial_proper_normal_subgroup.extracted_1_1.{u_1} {G : Type u_1} [inst : Fintype G]
  [inst_1 : Group G] (hG : card G = 462) : ∃ H, H.Normal ∧ H ≠ 1 ∧ H ≠ τ
```

Read back by a model that saw only the Lean. *For every finite group G with $|G| = 462$, there exists a normal subgroup H of G such that H is neither the trivial subgroup nor G itself.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem order_462_has_nontrivial_proper_normal_subgroup {G : Type*} [Fintype G] [Group G]
  (hG : Fintype.card G = 462) :
  ∃ H : Subgroup G, H.Normal ∧ H ≠ 1 ∧ H ≠ τ := by
  classical
  have hcard : Nat.card G = 462 := by
    simpa [Nat.card_eq_fintype_card] using hG
  have hp : Nat.Prime 11 := by norm_num
  letI : Fact (11 : ℕ).Prime := ⟨hp⟩
  obtain ⟨P⟩ : Nonempty (Sylow 11 G) := Sylow.nonempty
  set n : ℕ := Nat.card (Sylow 11 G) with hn
  have hmod : n ≡ 1 [MOD 11] := by
    simpa [n] using (card_sylow_modEq_one 11 G)
  have hdiv_index : n ∣ P.index := by
    simpa [n] using (Sylow.card_dvd_index (p := 11) (G := G) P)
  have hindex_dvd : P.index ∣ Nat.card G := by
    simpa [Nat.card_eq_fintype_card] using P.1.index_dvd_card
  have hdiv_card : n ∣ Nat.card G := Nat.dvd_trans hdiv_index hindex_dvd
  have hdiv_mul : n ∣ 11 * 42 := by
    rw [hcard] at hdiv_card
    simpa using hdiv_card
  have hnotdiv : ¬11 ∣ n := by
    simpa [n] using (not_dvd_card_sylow 11 G)
  have hcoprime : Nat.Coprime n 11 := by
    simpa [Nat.coprime_comm] using (hp.coprime_iff_not_dvd).2 hnotdiv
  have hdiv42 : n ∣ 42 := hcoprime.dvd_of_dvd_mul_left hdiv_mul
  have hnle : n ≤ 42 := Nat.le_of_dvd (by norm_num) hdiv42
  have hn_eq_one : n = 1 := by
    rw [Nat.ModEq] at hmod
    obtain ⟨k, hk⟩ := hdiv42
    have hnpos : 0 < n := by omega
    have hk_le : k ≤ 42 := by nlinarith [hk]
    interval_cases k <|> omega
  have hsub : Subsingleton (Sylow 11 G) := by
    exact (Nat.card_eq_one_iff_unique.mp (by simpa [n] using hn_eq_one)).1
  have hnormal : P.Normal := by
    letI : Subsingleton (Sylow 11 G) := hsub
    exact Sylow.normal_of_subsingleton P
  have hdivF : 11 ∣ Fintype.card G := by
    rw [hG]
    norm_num
  have hdivF' : 11 ∣ Nat.card G := by
    simpa [Nat.card_eq_fintype_card] using hdivF
  have hnebot : (P : Subgroup G) ≠ 1 := P.ne_bot_of_dvd_card hdivF'
  have hfac : Nat.factorization 462 11 = 1 := by
    rw [show (462 : ℕ) = 11 * 42 by norm_num,
      Nat.factorization_mul (by norm_num) (by norm_num)]
    rw [Nat.Prime.factorization (by norm_num)]
    simp [Nat.factorization_eq_zero_of_not_dvd]
  have hPcard : Nat.card (P : Subgroup G) = 11 := by
    have h := P.card_eq_multiplicity
    rw [hcard, hfac] at h
    simpa using h
  have hnetop : (P : Subgroup G) ≠ τ := by
    intro ht
```

```
have hcontra : Nat.card G = 11 := by
  simpa [ht, Nat.card_eq_fintype_card] using hPcard
  omega
exact ⟨(P : Subgroup G), hnormal, hnebot, hnetop⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_4_5_23__me__c9e0395d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c9e0395dd85c639f

4.59 059. idempotent_sum_commute

The problem

Let R be a ring and let $a, b \in R$. Suppose that $a^2 = a$, $b^2 = b$, and $(a + b)^2 = a + b$. Prove that $ab = ba$.

Lean statement

```
theorem idempotent_sum_commute {R : Type*} [Ring R] (a b : R) (ha : a ^ 2 = a) (hb : b ^ 2 = b) (hab : (a + b) ^ 2 = a + b) : a * b = b * a
```

Parent. A ring R is called a Boolean ring if $a^2 = a$ for all $a \in R$. Prove that every Boolean ring is commutative.

Parent in Lean

```
theorem Dummit_Foote_exercise_7_1_15 {R : Type*} [Ring R] (hR : ∀ a : R, a^2 = a) :
  ∀ a b : R, a * b = b * a := by
```

What it contributes. The parent supplies the idempotent-ring expansion strategy and commutativity target; the child localizes it to hypotheses ha , hb , and hab and closes through multiplied mixed-term identities.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely specializes idempotence to a , b , and $a+b$, then derives $a*b = b*a$ through mixed-term cancellation and multiplication by the idempotents. The parent's universal proof relies on deriving characteristic two from idempotence of arbitrary elements, which the child cannot use and does not reproduce.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child uses only local idempotence of a , b , and $a+b$, then derives the mixed-term cancellation and commutativity by multiplying and cancelling. The parent's global argument via idempotence of $-a$ is unavailable, so recalling the parent does not substantially solve this proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely specializes idempotence to a , b , and $a+b$ and derives commutativity through mixed-term cancellation and multiplication, without using the parent's global characteristic-two argument. Its proof is not recoverable by recalling the parent's theorem or final lemma.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2000 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem idempotent_sum_commute.extracted_1_1.{u_1} {R : Type u_1} [inst : Ring R] (a b : R) (ha : a ^ 2 = a)
(hb : b ^ 2 = b) (hab : (a + b) ^ 2 = a + b) : a * b = b * a
```

Read back by a model that saw only the Lean. *Let R be a ring and let a, b be elements of R . If $a^2 = a$, $b^2 = b$, and $(a + b)^2 = a + b$, then $ab = ba$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem idempotent_sum_commute {R : Type*} [Ring R] (a b : R) (ha : a ^ 2 = a) (hb : b ^ 2 = b) (hab : (a + b) ^ 2 = a + b) : a * b = b * a
  <| a := by
  have haa : a * a = a := by
    simp [pow_two] using ha
  have hbb : b * b = b := by
    simp [pow_two] using hb
  have h_expand : (a + b) ^ 2 = a * a + a * b + b * a + b * b := by
    calc
      (a + b) ^ 2 = (a + b) * (a + b) := by simp [pow_two]
      _ = a * (a + b) + b * (a + b) := by rw [add_mul]
      _ = (a * a + a * b) + (b * a + b * b) := by rw [mul_add, mul_add]
      _ = a * a + a * b + b * a + b * b := by ac_rfl
  have h_sq : a * a + a * b + b * a + b * b = a + b := by
    calc
      a * a + a * b + b * a + b * b = (a + b) ^ 2 := h_expand.symm
      _ = a + b := hab
  have h_eq : a + a * b + b * a + b = a + b := by
    simp [haa, hbb] using h_sq
  have h_mixed : a * b + b * a = 0 := by
    have h_cancel : a * b + b * a + b = b := by
      have h : a + (a * b + b * a + b) = a + b := by
        calc
          a + (a * b + b * a + b) = a + a * b + b * a + b := by ac_rfl
          _ = a + b := h_eq
      exact add_left_cancel h
    have h : (a * b + b * a) + b = 0 + b := by
      simp using h_cancel
      exact add_right_cancel h
  have h_left : a * b + a * (b * a) = 0 := by
    simp [mul_add, ← mul_assoc, haa] using congrArg (fun x : R => a * x) h_mixed
  have h_right : a * (b * a) + b * a = 0 := by
    simp [add_mul, mul_assoc, haa] using congrArg (fun x : R => x * a) h_mixed
  apply sub_eq_zero.mp
  calc
    a * b - b * a = (a * b + a * (b * a)) - (a * (b * a) + b * a) := by abel
    _ = 0 - 0 := by rw [h_left, h_right]
    _ = 0 := by simp
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_7_1_15__me__09c50d0e

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint c0d16b1a74e46a74

4.60 060. universally_idempotent_frobenius

The problem

Let R be a ring such that $a^2 = a$ for every $a \in R$. Prove that $(a + b)^2 = a^2 + b^2$ for all $a, b \in R$.

Lean statement

```
theorem universally_idempotent_frobenius {R : Type*} [Ring R] (hR : ∀ a : R, a ^ 2 = a) : ∀ a b : R, (a + b) ^ 2 = a ^ 2 + b ^ 2
```

Parent. A ring R is called a Boolean ring if $a^2 = a$ for all $a \in R$. Prove that every Boolean ring is commutative.

Parent in Lean

```
theorem Dummit_Foote_exercise_7_1_15 {R : Type*} [Ring R] (hR : ∀ a : R, a^2 = a) :
  ∀ a b : R, a*b = b*a := by
```

What it contributes. Adapts the parent's sum-square expansion and additive cancellation checkpoints while changing the conclusion to the additive squaring law.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from commutativity to Frobenius additivity and proves it by deriving mixed-term cancellation, expanding the sum square, and eliminating the cross terms. Although it shares the parent's preliminary idempotence consequences, it does not merely apply or wrap the parent's commutativity result.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is not a recalled wrapper or decoration: it proves Frobenius additivity by expanding the square and deriving cancellation of the mixed terms directly from idempotence of $a+b$. It reaches a different conclusion without applying the parent's commutativity theorem, satisfying the required semantic mutation.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the target from commutativity to Frobenius additivity and explicitly derives mixed-term cancellation from idempotence of $a+b$ before completing the square expansion. Although it shares the sum-square technique with the parent, it is not a recalled proof of the parent's conclusion and satisfies the requested specialization.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over proptext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1986 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem universally_idempotent_frobenius.extracted_1_1 : ∀ {R : Type u_1} [inst : Ring R],  
  (∀ (a : R), a ^ 2 = a) → ∀ (a b : R), (a + b) ^ 2 = a ^ 2 + b ^ 2
```

Read back by a model that saw only the Lean. For every type R equipped with a ring structure, if every element a of R satisfies $a^2 = a$, then for all elements a and b of R , $(a + b)^2 = a^2 + b^2$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem universally_idempotent_frobenius {R : Type*} [Ring R] (hR : ∀ a : R, a ^ 2 = a) : ∀ a b : R, (a + b) ^ 2 = a ^ 2 + b ^ 2 := by
  intro a b
  have ha : a * a = a := by
    simpa [pow_two] using hR a
  have hb : b * b = b := by
    simpa [pow_two] using hR b
  have h_expand : (a + b) ^ 2 = a * a + a * b + b * a + b * b := by
    calc
      (a + b) ^ 2 = (a + b) * (a + b) := by simp [pow_two]
      _ = a * (a + b) + b * (a + b) := by rw [add_mul]
      _ = (a * a + a * b) + (b * a + b * b) := by rw [mul_add, mul_add]
      _ = a * a + a * b + b * a + b * b := by simp [add_assoc]
  have h_sum : a * a + a * b + b * a + b * b = a + b := by
    calc
      a * a + a * b + b * a + b * b = (a + b) ^ 2 := h_expand.symm
      _ = a + b := hR (a + b)
  have h_cancel : a * b + b * a = 0 := by
    have h_sum' := h_sum
    rw [ha, hb] at h_sum'
    have h_left : a + ((a * b + b * a) + b) = a + b := by
      simpa [add_assoc] using h_sum'
    have h_middle : (a * b + b * a) + b = b := add_left_cancel h_left
    have h_right : (a * b + b * a) + b = 0 + b := by
      simpa using h_middle
    exact add_right_cancel h_right
  calc
    (a + b) ^ 2 = a * a + a * b + b * a + b * b := h_expand
    _ = a * a + (a * b + b * a) + b * b := by simp [add_assoc]
    _ = a * a + b * b := by rw [h_cancel, add_zero]
    _ = a ^ 2 + b ^ 2 := by simp [pow_two]
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_7_1_15__me__82b7256f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axiomspropext, Quot.sound, fingerprint 5ff327b9de75ec42

4.61 061. boolean_ring_nsmul_period_two

The problem

Let R be a ring in which every element is unchanged when squared. Prove that for every natural number n and every $a \in R$, the $(n + 2)$ -fold sum of a equals the n -fold sum of a .

Lean statement

```
theorem boolean_ring_nsmul_period_two {R : Type*} [Ring R] (hR : ∀ a : R, a * a = a) : ∀ n : ℕ, ∀ a : R, (n + 2) • a = n • a
```

Parent. In a ring where multiplying any quantity by itself leaves it unchanged, show that reversing the order of two factors does not change their product.

Parent in Lean

```
theorem boolean_ring_opposite_order {R : Type*} [Ring R] (hR : ∀ a : R, a * a = a) : ∀ a b : R, b * a = a * b := by
```

What it contributes. Reuses the parent's Boolean-ring idempotence setup and substantive sum expansion pattern, but changes the final role to additive characteristic two and scalar periodicity.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a genuinely new universally quantified additive-scalar periodicity conclusion, requiring nsmul decomposition and simplification after deriving characteristic two. Although its characteristic-two lemma uses a related idempotence argument, the parent's commutativity conclusion does not supply the new scalar reasoning.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from multiplicative commutativity to additive scalar periodicity. Its proof independently derives characteristic two from idempotence and applies nsmul identities, so recalling the parent's commutativity argument does not substantially help.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely different conclusion: additive scalar periodicity for every natural n and ring element a . Its characteristic-two lemma is derived directly from idempotence of $a+a$, then combined with nsmul arithmetic; it does not reuse the parent's multiplicative commutativity argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2824 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem boolean_ring_nsmul_period_two.extracted_1_1 : ∀ {R : Type u_1} [inst : Ring R],  
  (∀ (a : R), a * a = a) → ∀ (n : ℕ) (a : R), (n + 2) • a = n • a
```

Read back by a model that saw only the Lean. For every ring R , if every element a of R is idempotent, meaning $a \cdot a = a$, then for every natural number n and every element a of R , the $(n + 2)$ -fold sum of a equals the n -fold sum of a .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem boolean_ring_nsmul_period_two {R : Type*} [Ring R] (hR : ∀ a : R, a * a = a) : ∀ n : ℕ, ∀ a : R, (n + 2) • a = n • a := by
  have hchar : ∀ a : R, a + a = 0 := by
    intro a
    have hsq : (a + a) * (a + a) = a + a := hR (a + a)
    have hexpand : (a + a) * (a + a) = a * a + a * a + a * a + a * a := by
      noncomm_ring
    have hsum : a + a + a + a = a + a := by
      calc
        a + a + a + a = a * a + a * a + a * a + a * a := by rw [hR a]
        _ = (a + a) * (a + a) := hexpand.symm
        _ = a + a := hsq
    have hcancel : (a + a) + (a + a) = (a + a) + 0 := by
      calc
        (a + a) + (a + a) = a + a + a + a := by abel
        _ = a + a := hsum
        _ = (a + a) + 0 := by simp
    exact add_left_cancel hcancel
  intro n a
  have ha : a + a = 0 := hchar a
  calc
    (n + 2) • a = n • a + 2 • a := by rw [add_nsmul]
    _ = n • a + (a + a) := by rw [two_nsmul]
    _ = n • a := by rw [ha]; simp
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_7_1_15__ms.me__36389812

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axiomspropt, Quot.sound, fingerprint 36389812b52cf248

4.62 062. bounded_ideal_nilpotence

The problem

Let p be prime and $m > 0$. Prove that every ideal of $\mathbb{Z}/p^m\mathbb{Z}$ contained in the ideal generated by p is nilpotent.

Lean statement

```
theorem bounded_ideal_nilpotence {p m : ℕ} (I : Ideal (ZMod (p ^ m))) (hI : I ≤ (Ideal.span ({(p : ZMod (p ^ m))} : Set (ZMod (p ^ m))))) : IsNilpotent I := by
```

Parent. An ideal N is called nilpotent if N^n is the zero ideal for some $n \geq 1$. Prove that the ideal $p\mathbb{Z}/p^m\mathbb{Z}$ is a nilpotent ideal in the ring $\mathbb{Z}/p^m\mathbb{Z}$.

Parent in Lean

```
theorem Dummit_Foote_exercise_7_3_37 {p m : ℕ} (_hp : p.Prime) :
  IsNilpotent (span ({p} : Set (ZMod (p ^ m)) : Ideal (ZMod (p ^ m))) := by
```

What it contributes. Supplies the principal ideal $J=(p)$ in $ZMod (p^m)$ and the power-vanishing argument via divisibility by p^m .

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes nilpotence from the principal ideal to every ideal contained in it. Its proof requires the new monotonicity argument that powers of a subideal are bounded by powers of the principal ideal, beyond recalling the parent's nilpotence proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes nilpotence from the principal ideal to every ideal contained in it. Beyond reproducing the principal-ideal power calculation, it requires the new monotonicity argument that containment is preserved under ideal powers and transfers the vanishing power to the arbitrary ideal.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes nilpotence from the principal ideal to every ideal contained in it. It requires the new monotonicity argument that powers of a contained ideal lie below the corresponding power of the principal ideal, beyond the parent's principal-ideal computation.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2262 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_ideal_nilpotence.extracted_1_1 {p m : ℕ} (hp : Nat.Prime p) (hm : 0 < m) (I : Ideal (ZMod (p ^ m)))  
(hI : I ≤ span {↑p}) : IsNilpotent I
```

Read back by a model that saw only the Lean. *Let p and m be natural numbers. If p is prime, m is positive, and I is an ideal of the ring $\text{ZMod}(p^m)$ such that I is contained in the ideal generated by the image of p in $\text{ZMod}(p^m)$, then I is nilpotent.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem bounded_ideal_nilpotence {p m : ℕ} (I : Ideal (ZMod (p ^ m))) (hI : I ≤ (Ideal.span ({(p : ZMod (p ^ m))} : Set (ZMod (p ^ m)))))) : IsNilpotent I := by
  let J : Ideal (ZMod (p ^ m)) := Ideal.span ({(p : ZMod (p ^ m))} : Set (ZMod (p ^ m)))
  have h_containment : ∀ K : Ideal (ZMod (p ^ m)), K ≤ J → IsNilpotent K := by
    intro K hK
    refine ⟨Nat.succ m, ?_⟩
    have hpowJ : J ^ Nat.succ m = 1 := by
      have hdiv : p ^ m ∣ p ^ Nat.succ m := by
        refine ⟨p, ?_⟩
        simp [Nat.pow_succ]
      have hzero : ((p : ZMod (p ^ m)) ^ Nat.succ m) = 0 := by
        have hcast : ((p ^ Nat.succ m : ℕ) : ZMod (p ^ m)) = 0 := by
          exact (ZMod.natCast_eq_zero_iff (a := p ^ Nat.succ m) (b := p ^ m)).2 hdiv
        simp [Nat.cast_pow] using hcast
      simp [J, Ideal.span_singleton_pow, hzero]
    have hpowK : K ^ Nat.succ m ≤ J ^ Nat.succ m := by
      apply pow_le_pow_left'
    exact hK
    have hzeroK : K ^ Nat.succ m ≤ 1 := by
      rw [hpowJ] at hpowK
      exact hpowK
    simp [Ideal.zero_eq_bot] using (le_antisymm hzeroK bot_le)
  exact h_containment I (by simp [J] using hI)
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_7_3_37__me__e3db7d21

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint 7925d0ab6e5efc6f

4.63 063. bounded_power_period_characterization

The problem

For natural numbers a , n , and t with $a > 1$, $0 < t$, and $t \leq n$, the power a^t is congruent to 1 modulo $a^n - 1$ if and only if $t = n$.

Lean statement

```
theorem bounded_power_period_characterization {a n t : ℕ} (ha : a > 1) (ht : 0 < t) (htn : t ≤ n) : Nat.ModEq (a ^ n - 1) (a ^ t) 1 ↔ t = n
```

Parent. For natural numbers a , n , and t with $a > 1$ and $n \mid t$, show that $a^t \equiv 1 \pmod{a^n - 1}$.

Parent in Lean

```
theorem parameterized_power_congruence {a n t : ℕ} (ha : a > 1) (hdiv : n ∣ t) : Nat.ModEq (a ^ n - 1) (a ^ t) 1
```

What it contributes. Supplied the base congruence construction modulo $a^n - 1$ and the modular-power route, adapted here into an exact bounded characterization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a genuine converse: under $0 < t \leq n$, it proves that $a^t \equiv 1 \pmod{a^n - 1}$ forces $t = n$ using strict power bounds and remainder analysis. The parent only supplies the forward base congruence and does not provide this characterization argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine converse: under $0 < t \leq n$, it proves any congruence forces $t = n$ by bounding a^t below the modulus and comparing remainders. The parent only supplies the forward divisible-exponent congruence and does not provide this inequality-based exclusion of $t < n$.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine converse characterization: under $0 < t \leq n$, it proves the congruence forces $t = n$ using strict power bounds and remainder reasoning. Although it reuses the parent's base congruence for the easy forward direction, the parent's proof supplies no argument for this converse.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1888 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_power_period_characterization.extracted_1_1 {a n t : ℕ} (ha : a > 1) (ht : 0 < t) (htn : t ≤ n) :  
  a ^ t ≡ 1 [MOD a ^ n - 1] ↔ t = n
```

Read back by a model that saw only the Lean. For natural numbers a , n , and t , if $a > 1$, $0 < t$, and $t \leq n$, then a raised to the t -th power is congruent to 1 modulo a raised to the n -th power minus 1 if and only if $t = n$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Set Real Ideal Polynomial
open scoped BigOperators

theorem bounded_power_period_characterization {a n t : ℕ} (ha : a > 1) (ht : 0 < t) (htn : t ≤ n) :  
  Nat.ModEq (a ^ n - 1) (a ^ t) 1 ↔ t = n := by  
  have hn : 0 < n := lt_of_lt_of_le ht htn  
  have hge : 1 ≤ a ^ n := Nat.one_le_pow n a (by omega)  
  have hpowgt : 1 < a ^ n := Nat.one_lt_pow (n := n) (a := a) (by omega) ha  
  have hbase : Nat.ModEq (a ^ n - 1) (a ^ n) 1 := by  
    change a ^ n % (a ^ n - 1) = 1 % (a ^ n - 1)  
  by_cases hm : a ^ n - 1 = 1  
  · rw [hm]  
    have hmod : a ^ n % 1 < 1 := Nat.mod_lt _ (by omega)  
    omega  
  · have hmglt : 1 < a ^ n - 1 := by omega  
    have hone : 1 % (a ^ n - 1) = 1 := Nat.mod_eq_of_lt (by omega)  
    calc  
      a ^ n % (a ^ n - 1) = (a ^ n - 1 + 1) % (a ^ n - 1) := by  
        rw [Nat.sub_add_cancel hge]  
      _ = 1 % (a ^ n - 1) := by  
        rw [Nat.add_mod]  
      simp [Nat.mod_eq_of_lt hmglt, hone]  
  constructor  
  · intro hcong  
    by_contra hne  
    have hlt : t < n := by omega  
    have hpowlt : a ^ t < a ^ n := Nat.pow_lt_pow_right (by omega) hlt  
    have ht1 : t + 1 ≤ n := by omega  
    have hpowle : a ^ (t + 1) ≤ a ^ n := Nat.pow_le_pow_right (by omega) ht1  
    have htpowgt : 1 < a ^ t := Nat.one_lt_pow (n := t) (a := a) (by omega) ha  
    have htwo : 2 ≤ a ^ t := by omega  
    have hmul : a ^ t * 2 ≤ a ^ t * a := by  
      exact Nat.mul_le_mul_left (a ^ t) (by omega)  
    have hdouble : a ^ t + 2 ≤ a ^ t * 2 := by omega  
    have hstep2 : a ^ t + 2 ≤ a ^ (t + 1) := by  
      rw [pow_succ]  
      exact le_trans hdouble hmul  
    have hmodulus : a ^ t + 1 ≤ a ^ n - 1 := by  
      have : a ^ t + 2 ≤ a ^ n := le_trans hstep2 hpowle  
      omega  
    have hmod : a ^ t < a ^ n - 1 := by omega  
    have hgt : 1 < a ^ n - 1 := by  
      have : 1 ≤ a ^ t := Nat.one_le_pow t a (by omega)  
      omega  
    have heq : a ^ t = 1 := by  
      change a ^ t % (a ^ n - 1) = 1 % (a ^ n - 1) at hcong  
      rw [Nat.mod_eq_of_lt hmod, Nat.mod_eq_of_lt (by omega)] at hcong  
      exact hcong  
    omega  
  · intro h  
    subst t  
    exact hbase
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Herstein_exercise_2_4_36__me.me__37be04a5

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fin-

gerprint 8117fbe1560ccf55

4.64 064. power_shift_period_mod_sub_one

The problem

Let $a > 1$ and let k be a positive multiple of n . Show that raising a^k by one further block of n leaves the same residue modulo $a^n - 1$.

Lean statement

```
theorem power_shift_period_mod_sub_one {a n k : ℕ} (ha : 1 < a) (hk : n ∣ k) :
  ((a : ZMod (a ^ n - 1)) ^ (k + n) = (a : ZMod (a ^ n - 1)) ^ k)
```

Parent. Let $a > 1$ and $n > 0$ be integers. Show that for every positive exponent k divisible by n , a^k is congruent to 1 modulo $a^n - 1$.

Parent in Lean

```
theorem power_period_mod_sub_one {a n : ℕ} (ha : 1 < a) :
  ∀ k : ℕ, 0 < k → n ∣ k → ((a : ZMod (a ^ n - 1)) ^ k = 1) := by
```

What it contributes. Supplied the modular base-period checkpoint $a^n=1$ modulo a^n-1 and the divisible-exponent factorization pattern.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from evaluating a multiple-period power to proving a shift-period identity, requiring the new exponent decomposition via `pow_add`. Divisibility of k is used materially, and the result is not merely the parent theorem restated.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuine shift-invariance consequence: after decomposing the divisible exponent, it uses ‘`pow_add`’ and the certified period fact to show $a^{k+n}=a^k$. This is not the parent’s value-at-multiples conclusion and requires new exponent algebra.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely different periodic-shift identity, not merely that a divisible exponent yields value 1 . It uses divisibility to rewrite $k=nq$ and combines the parent’s base-period fact with ‘`pow_add`’; the case $k=0$ also distinguishes it from direct recall of the parent theorem.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2271 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	MISMATCH: Statement A asserts congruence modulo a^n-1 , whereas Statement B asserts literal equality $a^{(k+n)}=a^k$.; Statement B allows $k=0$, while Statement A requires k to be positive (and implicitly n positive).

Goal round-trip: what Lean elaborated

```
theorem power_shift_period_mod_sub_one.extracted_1_1 {a n k : ℕ} (ha : 1 < a) (hk : n ∣ k) :
  ↑a ^ (k + n) = ↑a ^ k
```

Read back by a model that saw only the Lean. *For natural numbers a , n , and k , if $1 < a$ and n divides k , then the (coerced) number a raised to the power $k + n$ equals the (coerced) number a raised to the power k .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Set Real Ideal Polynomial
open scoped BigOperators

theorem power_shift_period_mod_sub_one {a n k : ℕ} (ha : 1 < a) (hk : n ∣ k) :
  ((a : ZMod (a ^ n - 1)) ^ (k + n) = (a : ZMod (a ^ n - 1)) ^ k) := by
  let m : ℕ := a ^ n - 1
  change ((a : ZMod m) ^ (k + n) = (a : ZMod m) ^ k)
  have ha_pos : 0 < a := lt_trans Nat.zero_lt_one ha
  have hle : 1 ≤ a ^ n := Nat.one_le_pow n a ha_pos
  have hcalc :
    ((a ^ n - 1 : ℕ) : ZMod m) + 1 = ((a ^ n : ℕ) : ZMod m) := by
      simpa [m] using congrArg (fun t : ℕ => (t : ZMod m)) (Nat.sub_add_cancel hle)
  have hbase_cast : ((a ^ n : ℕ) : ZMod m) = 1 := by
    calc
      ((a ^ n : ℕ) : ZMod m) = ((a ^ n - 1 : ℕ) : ZMod m) + 1 := hcalc.symm
      _ = 1 := by simp [m]
  have hbase : ((a : ZMod m) ^ n) = 1 := by
    simpa [Nat.cast_pow] using hbase_cast
  rcases hk with ⟨q, rfl⟩
  have hshift : n * q + n = n * q + n := rfl
  calc
    (a : ZMod m) ^ (n * q + n) = (a : ZMod m) ^ (n * q) * (a : ZMod m) ^ n := by
      rw [pow_add]
    _ = (a : ZMod m) ^ (n * q) := by rw [hbase]; simp
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Herstein_exercise_2_4_36__me.me__74be0029

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint 74be0029bb369cdf

4.65 065. parameterized_power_congruence

The problem

For natural numbers a , n , and t with $a > 1$ and $n \mid t$, show that $a^t \equiv 1 \pmod{a^n - 1}$.

Lean statement

```
theorem parameterized_power_congruence {a n t : ℕ} (ha : a > 1) (hdiv : n ∣ t) : Nat.ModEq (a ^ n - 1) (a ^ t) 1
```

Parent. If $a > 1$ is an integer, show that $n \mid \varphi(a^n - 1)$, where φ is the Euler φ -function.

Parent in Lean

```
theorem Herstein_exercise_2.4.36 {a n : ℕ} (h : a > 1) :  
  n ∣ (a ^ n - 1).totient := by
```

What it contributes. Supplied the domain condition $a > 1$ and the reusable checkpoint that $a^n - 1$ is the natural modulus associated with the exponent n ; the child changes the final role from totient divisibility to a parameterized power congruence.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the totient divisibility claim into a parameterized modular-power statement for every exponent t divisible by n . Its proof requires establishing the base congruence modulo $a^n - 1$ and transporting it through powers, reasoning absent from the parent's totient argument.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the totient divisibility conclusion with a parameterized congruence for every exponent t divisible by n . Its proof establishes the base congruence directly and raises it to a symbolic quotient power, rather than recalling the parent's cyclic-group/totient argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1875 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem parameterized_power_congruence.extracted_1_1 {a n t : ℕ} (ha : a > 1) (hdiv : n ∣ t) :  
  a ^ t ≡ 1 [MOD a ^ n - 1]
```

Read back by a model that saw only the Lean. For natural numbers a , n , and t , if $a > 1$ and n divides t , then a raised to the power t is congruent to 1 modulo a raised to the power n minus 1.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Set Real Ideal Polynomial
open scoped BigOperators

theorem parameterized_power_congruence {a n t : ℕ} (ha : a > 1) (hdiv : n ∣ t) :
  Nat.ModEq (a ^ n - 1) (a ^ t) 1 := by
  rcases hdiv with ⟨k, rfl⟩
  by_cases hn : n = 0
  · subst n
    simp
  have hge : 1 ≤ a ^ n := Nat.one_le_pow n a (by omega)
  have hpowgt : 1 < a ^ n := Nat.one_lt_pow (n := n) (a := a) hn ha
  have hbase : Nat.ModEq (a ^ n - 1) (a ^ n) 1 := by
    change a ^ n % (a ^ n - 1) = 1 % (a ^ n - 1)
    by_cases hm : a ^ n - 1 = 1
    · rw [hm]
      have hmod : a ^ n % 1 < 1 := Nat.mod_lt _ (by omega)
      omega
    · have hmg : 1 < a ^ n - 1 := by omega
      have hone : 1 % (a ^ n - 1) = 1 := Nat.mod_eq_of_lt (by omega)
      calc
        a ^ n % (a ^ n - 1) = (a ^ n - 1 + 1) % (a ^ n - 1) := by
          rw [Nat.sub_add_cancel hge]
        _ = 1 % (a ^ n - 1) := by
          rw [Nat.add_mod]
          simp [Nat.mod_eq_of_lt hmg, hone]
  have hpow := hbase.pow k
  have hresult : Nat.ModEq (a ^ n - 1) (a ^ (n * k)) 1 := by
    simpa [pow_mul] using hpow
  exact hresult
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Herstein_exercise_2_4_36__me__443942c6

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axiomspropext, Quot.sound, fingerprint 4af7ce7980bde617

4.66 066. power_period_mod_sub_one

The problem

Let $a > 1$ and $n > 0$ be integers. Show that for every positive exponent k divisible by n , a^k is congruent to 1 modulo $a^n - 1$.

Lean statement

```
theorem power_period_mod_sub_one {a n : ℕ} (ha : 1 < a) :
  ∀ k : ℕ, 0 < k → n ∣ k → ((a : ℤMod (a ^ n - 1)) ^ k = 1) := by
```

Parent. If $a > 1$ is an integer, show that $n \mid \varphi(a^n - 1)$, where ϕ is the Euler φ -function.

Parent in Lean

```
theorem Herstein_exercise_2_4_36 {a n : ℕ} (h : a > 1) :
  n ∣ (a ^ n - 1).totient := by
```

What it contributes. Retains the parent's hypotheses and modulus $a^n - 1$, while replacing its totient-divisibility conclusion with an operational power-period congruence.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the parent's intermediate fact $a^n = 1$ in $\mathbb{ZMod}(a^n - 1)$ to all positive exponents divisible by n , requiring a new divisibility decomposition and power-period argument. The round-trip concern is only a presentation mismatch: equality in 'ZMod' is the formal encoding of congruence, and positivity of n is unnecessary because positive k cannot be divisibl

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the parent's intermediate congruence to every positive exponent divisible by n . Its proof requires decomposing the exponent and applying the power-period argument, which is not supplied by recalling the parent's totient-divisibility conclusion.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child substantively generalizes the parent's base congruence to every positive exponent divisible by n . Its proof must decompose the divisibility witness and apply the power-period identity, so recalling the parent's totient-divisibility proof is insufficient.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2261 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	MISMATCH: Statement B omits the congruence modulo $a^n - 1$ and instead asserts ordinary equality of a^k with 1.

Goal round-trip: what Lean elaborated

```
theorem power_period_mod_sub_one.extracted_1_1 : ∀ {a n : ℕ},  
  1 < a → 0 < n → ∀ (k : ℕ), 0 < k → n ∣ k → ↑a ^ k = 1
```

Read back by a model that saw only the Lean. For all natural numbers a and n , if $a > 1$ and $n > 0$, then for every natural number k , if $k > 0$ and n divides k , the k -th power of the coercion of a equals 1.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Set Real Ideal Polynomial
open scoped BigOperators

theorem power_period_mod_sub_one {a n : ℕ} (ha : 1 < a) :
  ∀ k : ℕ, 0 < k → n ∣ k → ((a : ZMod (a ^ n - 1)) ^ k = 1) := by
  let m : ℕ := a ^ n - 1
  change ∀ k : ℕ, 0 < k → n ∣ k → ((a : ZMod m) ^ k = 1)
  have ha_pos : 0 < a := lt_trans Nat.zero_lt_one ha
  have hle : 1 ≤ a ^ n := Nat.one_le_pow n a ha_pos
  have hcalc :
    ((a ^ n - 1 : ℕ) : ZMod m) + 1 = ((a ^ n : ℕ) : ZMod m) := by
      simpa [m] using congrArg (fun t : ℕ => (t : ZMod m)) (Nat.sub_add_cancel hle)
  have hbase_cast : ((a ^ n : ℕ) : ZMod m) = 1 := by
    calc
      ((a ^ n : ℕ) : ZMod m) = ((a ^ n - 1 : ℕ) : ZMod m) + 1 := hcalc.symm
      _ = 1 := by simp [m]
  have hbase : ((a : ZMod m) ^ n) = 1 := by
    simpa [Nat.cast_pow] using hbase_cast
  intro k _ hdiv
  rcases hdiv with ⟨q, rfl⟩
  have hexponent : n * q = n * q := rfl
  calc
    (a : ZMod m) ^ (n * q) = ((a : ZMod m) ^ n) ^ q := by rw [pow_mul]
    _ = 1 := by rw [hbase]; simp
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Herstein_exercise_2_4_36__me__a2c28210

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint 5aba40bb18595ef0

4.67 067. bounded_totient_period

The problem

Let $a > 1$ and suppose that n divides the Euler totient of $a^n - 1$. Show that for every natural number k , the exponent $k\phi(a^n - 1)$ is a multiple of n , and $a^{k\phi(a^n - 1)} \equiv 1 \pmod{a^n - 1}$.

Lean statement

```
theorem bounded_totient_period {a n : ℕ} (ha : 1 < a) (hdiv : n ∣ (a ^ n - 1).totient) :
  ∀ k : ℕ, ∃ q : ℕ,
    k * (a ^ n - 1).totient = n * q ∧
    a ^ (k * (a ^ n - 1).totient) ≡ 1 [MOD a ^ n - 1]
```

Parent. If $a > 1$ is an integer, show that $n \mid \phi(a^n - 1)$, where ϕ is the Euler ϕ -function.

Parent in Lean

```
theorem Herstein_exercise_2.4.36 {a n : ℕ} (h : a > 1) :
  n ∣ (a ^ n - 1).totient := by
```

What it contributes. Its totient-divisibility conclusion is retained as the explicit hypothesis `hdiv` and used to construct the multiple-of- n exponent.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child requires a new exponent-period argument: it converts the retained divisibility witness into a quotient and proves the congruence for every multiple of the totient. This is not obtained by recalling the parent's divisibility proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child uses the parent divisibility witness to construct, for every k , an exponent quotient and then proves a new modular-periodicity conclusion. This requires reasoning about powers modulo $a^n - 1$ and is not supplied by recalling the parent's divisibility theorem.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child retains the parent divisibility checkpoint but adds a genuinely new modular-period argument: it derives $a^n \equiv 1 \pmod{a^n - 1}$ and lifts this to every exponent $k \cdot \phi(a^n - 1)$ while constructing the corresponding quotient. A solver recalling only the parent's totient-divisibility proof would still need this exponentiation and congruence reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2801 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_totient_period.extracted_1_1 : ∀ {a n : ℕ},  
  1 < a →  
    0 < n →  
      n ∣ (a ^ n - 1).totient →  
        ∀ (k : ℕ), ∃ q, k * (a ^ n - 1).totient = n * q ∧ a ^ (k * (a ^ n - 1).totient) ≡ 1 [MOD a ^ n - 1]
```

Read back by a model that saw only the Lean. For all natural numbers a and n , if $a > 1$, $n > 0$, and n divides the Euler totient of $a^n - 1$, then for every natural number k there exists a natural number q such that k times the Euler totient of $a^n - 1$ equals n times q , and a raised to the power k times the Euler totient of $a^n - 1$ is congruent to 1 modulo $a^n - 1$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open Fintype Set Real Ideal Polynomial  
open scoped BigOperators  
  
theorem bounded_totient_period {a n : ℕ} (ha : 1 < a) (hdiv : n ∣ (a ^ n - 1).totient) :  
  ∀ k : ℕ, ∃ q : ℕ,  
    k * (a ^ n - 1).totient = n * q ∧  
      a ^ (k * (a ^ n - 1).totient) ≡ 1 [MOD a ^ n - 1] := by  
  rcases hdiv with ⟨q, hq⟩  
  have hbase : a ^ n ≡ 1 [MOD a ^ n - 1] := by  
    rw [Nat.modEq_iff_dvd]  
    have hle : 1 ≤ a ^ n := Nat.one_le_pow n a (by omega)  
    change ((a ^ n - 1 : ℕ) : ℤ) ∣ (1 : ℤ) - (a ^ n : ℤ)  
    rw [show (1 : ℤ) - (a ^ n : ℤ) = -((a ^ n - 1 : ℕ) : ℤ) by  
      rw [Nat.cast_sub hle]  
      simp]  
    exact (dvd_neg).2 (dvd_refl _)  
  intro k  
  refine ⟨k * q, ?_, ?_⟩  
  · calc  
    k * (a ^ n - 1).totient = k * (n * q) := by rw [hq]  
    _ = n * (k * q) := by ac_rfl  
  · rw [hq]  
    convert hbase.pow (k * q) using 1 <|> simp [pow_mul, mul_assoc, mul_left_comm, mul_comm]
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Herstein_exercise_2_4_36__me__e40788fd

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint ecdded62cf3a5cd33

4.68 068. conjugate_zpow_mem_zpowers

The problem

Let every subgroup of a group G be normal. Show that conjugating any integer power of a by b yields another element of the cyclic subgroup generated by a .

Lean statement

```
theorem conjugate_zpow_mem_zpowers {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :
  ∀ n : ℤ, b * a ^ n * b⁻¹ ∈ Subgroup.zpowers a
```

Parent. Let G be a group such that all subgroups of G are normal in G . If $a, b \in G$, prove that $ba = a^j b$ for some j .

Parent in Lean

```
theorem Herstein_exercise_2_5_23 {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :
  ∃ (j : ℤ), b * a = a^j * b := by
```

What it contributes. Supplies the cyclic-subgroup normality argument and the membership checkpoint for powers of a .

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a genuinely different quantified closure result: every conjugate of every integer power of a lies in the cyclic subgroup generated by a . Its proof requires showing a^n belongs to that subgroup and applying normality, rather than recalling the parent's existence-and-rearrangement argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes conjugation from the single element a to every integer power a^n . Its proof requires establishing $a^n \in \langle a \rangle$ via subgroup closure before applying normality, so recalling the parent does not directly solve it.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes conjugation from a to every integer power a^n . It requires the new subgroup-membership step for integer powers and proves a universal membership statement rather than recalling the parent's existential commutation conclusion.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2286 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem conjugate_zpow_mem_zpowers.extracted_1_1 : ∀ {G : Type u_1} [inst : Group G],  
  (∀ (H : Subgroup G), H.Normal) → ∀ (a b : G) (n : ℤ), b * a ^ n * b⁻¹ ∈ Subgroup.zpowers a
```

Read back by a model that saw only the Lean. For every type G equipped with a group structure, if every subgroup H of G is normal, then for all elements a, b of G and every integer n , the element $b a^n b^{-1}$ belongs to the subgroup of G consisting of the integer powers of a .

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open Fintype Set Real Ideal Polynomial  
open scoped BigOperators  
  
theorem conjugate_zpow_mem_zpowers {G : Type*} [Group G]  
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :  
  ∀ n : ℤ, b * a ^ n * b⁻¹ ∈ Subgroup.zpowers a := by  
  classical  
  intro n  
  have hHnormal : (Subgroup.zpowers a).Normal := hG (Subgroup.zpowers a)  
  have ha : a ∈ Subgroup.zpowers a := Subgroup.mem_zpowers a  
  have han : a ^ n ∈ Subgroup.zpowers a := by  
    exact Subgroup.zpow_mem (Subgroup.zpowers a) ha n  
  have hconj : b * a ^ n * b⁻¹ ∈ Subgroup.zpowers a := by  
    simpa using hHnormal.conj_mem (a ^ n) han b  
  exact hconj
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Herstein_exercise_2_5_23__me__297b115a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, fingerprint 297b115a6e38c837

4.69 069. bounded_conjugation_transport

The problem

For every group in which every subgroup is normal, a conjugation relation for an element extends to a bounded iterated transport law for powers of the conjugating element.

Lean statement

```
theorem bounded_conjugation_transport {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :
  ∃ (j : ℤ), b * a = a ^ j * b ∧
  ∀ (N n : ℕ), n ≤ N → ∃ k : ℤ, b ^ n * a = a ^ k * b ^ n
```

Parent. Let G be a group such that all subgroups of G are normal in G . If $a, b \in G$, prove that $ba = a^j b$ for some j .

Parent in Lean

```
theorem Herstein_exercise_2_5_23 {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :
  ∃ (j : ℤ), b * a = a ^ j * b := by
```

What it contributes. Reuses the parent's normal cyclic-subgroup and integer-power extraction argument, while extending it inductively to conjugates by arbitrary powers of b .

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

The child genuinely proves transport for every natural power b^n by induction and repeated normality, which is reasoning absent from the parent's single-conjugation proof. The stated bound $n \leq N$ is unused, so the bounded aspect is superficial, but the iterated-conjugation generalization is substantive.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuine iterated-conjugation law for every power of b , requiring an induction and repeated use of subgroup normality; the parent's single-step witness does not supply this reasoning. The bound $n \leq N$ is redundant, but the generalized transport claim remains substantive.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely stronger transport law for every power b^n , requiring induction to show all iterated conjugates remain in $\langle a \rangle$. This reasoning is not supplied by the parent's single-conjugation witness, despite retaining that witness in the conjunction.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1889 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_conjugation_transport.extracted_1_1.{u_1} {G : Type u_1} [inst : Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :
  ∃ j, b * a = a ^ j * b ∧ ∀ (N n : ℕ), n ≤ N → ∃ k, b ^ n * a = a ^ k * b ^ n
```

Read back by a model that saw only the Lean. *Let G be a group in which every subgroup is normal, and let a, b be elements of G . Then there exists a natural number j such that $b \cdot a = a^j \cdot b$, and for every pair of natural numbers N and n with $n \leq N$, there exists a natural number k such that $b^n \cdot a = a^k \cdot b^n$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Set Real Ideal Polynomial
open scoped BigOperators

theorem bounded_conjugation_transport {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :
  ∃ (j : ℤ), b * a = a ^ j * b ∧
  ∀ (N n : ℕ), n ≤ N → ∃ k : ℤ, b ^ n * a = a ^ k * b ^ n := by
  classical
  have hHnormal : (Subgroup.zpowers a).Normal := hG (Subgroup.zpowers a)
  have ha : a ∈ Subgroup.zpowers a := Subgroup.mem_zpowers a
  have hmem : ∀ n : ℕ, b ^ n * a * (b ^ n)⁻¹ ∈ Subgroup.zpowers a := by
    intro n
    induction n with
    | zero => simp using ha
    | succ n ih =>
      have h := hHnormal.conj_mem (b ^ n * a * (b ^ n)⁻¹) ih b
      simp [pow_succ', mul_assoc] using h
  obtain ⟨j, hj⟩ := Subgroup.mem_zpowers_iff.mp (hmem 1)
  have hstep : b * a = a ^ j * b := by
    have hj' : b * a * b⁻¹ = a ^ j := by
      simp using hj.symm
    calc
    b * a = (b * a * b⁻¹) * b := by simp [mul_assoc]
    _ = a ^ j * b := by rw [hj']
  refine ⟨j, hstep, ?_⟩
  intro N n hn
  obtain ⟨k, hk⟩ := Subgroup.mem_zpowers_iff.mp (hmem n)
  have htransport : b ^ n * a = a ^ k * b ^ n := by
    calc
    b ^ n * a = (b ^ n * a * (b ^ n)⁻¹) * b ^ n := by simp [mul_assoc]
    _ = a ^ k * b ^ n := by rw [hk]
  exact ⟨k, htransport⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Herstein_exercise_2_5_23__me__4a8d9215

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, fingerprint 95b5b05cc1c3ddd1

4.70 070. cyclic_subgroup_conjugation_automorphism

The problem

Let G be a group in which every subgroup is normal. For $a, b \in G$, conjugation by b restricts to an automorphism of the cyclic subgroup generated by a .

Lean statement

```
theorem cyclic_subgroup_conjugation_automorphism {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :
  ∃ f : Subgroup.zpowers a → Subgroup.zpowers a,
    Function.Bijective f ∧
    ∀ x : Subgroup.zpowers a,
      ((f x : Subgroup.zpowers a) : G) = b * (x : G) * b⁻¹ := by
```

Parent. Let G be a group such that all subgroups of G are normal in G . If $a, b \in G$, prove that $ba = a^j b$ for some j .

Parent in Lean

```
theorem Herstein_exercise_2_5_23 {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :
  ∃ (j : ℤ), b * a = a^j * b := by
```

What it contributes. Uses the parent's normality argument for the cyclic subgroup and its conjugation-membership witness, then consumes that witness in a new bijective homomorphism conclusion.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the existential power equation into a bijective group-homomorphism statement for the entire cyclic subgroup. Its proof must establish homomorphism, injectivity, and surjectivity via inverse conjugation, none of which is supplied by recalling the parent proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the existential power equation into a homomorphism-and-bijectivity claim for conjugation on the entire cyclic subgroup. Its proof must establish closure for every subgroup element, multiplicativity, injectivity, and surjectivity via the inverse conjugation map, none of which is supplied by the parent's single-element argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a substantially stronger and different claim: conjugation restricts to a bijective group homomorphism on the entire cyclic subgroup. This requires constructing the homomorphism and its inverse behavior, which the parent's single-element power witness does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2189 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem cyclic_subgroup_conjugation_automorphism.extracted_1.1.{u_1} {G : Type u_1} [inst : Group G]
(hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :
  ∃ f, Function.Bijective f ∧ ∀ (x : (Subgroup.zpowers a)), ↑(f x) = b * ↑x * b⁻¹
```

Read back by a model that saw only the Lean. *Let G be a group such that every subgroup of G is normal. For any elements a and b of G , there exists a bijective function f from the subgroup of G consisting of all integer powers of a to itself such that, for every element x of this subgroup, the element $f(x)$, viewed in G , equals $b \cdot x \cdot b^{-1}$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Set Real Ideal Polynomial
open scoped BigOperators

theorem cyclic_subgroup_conjugation_automorphism {G : Type*} [Group G]
(hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :
  ∃ f : Subgroup.zpowers a →* Subgroup.zpowers a,
    Function.Bijective f ∧
      ∀ x : Subgroup.zpowers a,
        ((f x : Subgroup.zpowers a) : G) = b * (x : G) * b⁻¹ := by
  classical
  have hHnormal : (Subgroup.zpowers a).Normal := hG (Subgroup.zpowers a)
  have ha : a ∈ Subgroup.zpowers a := Subgroup.mem_zpowers a
  have hconj : ∀ x : Subgroup.zpowers a,
    b * (x : G) * b⁻¹ ∈ Subgroup.zpowers a := by
    intro x
    exact hHnormal.conj_mem (x : G) x.property b
  have hconj_inv : ∀ x : Subgroup.zpowers a,
    b⁻¹ * (x : G) * b ∈ Subgroup.zpowers a := by
    intro x
    simpa using hHnormal.conj_mem (x : G) x.property b⁻¹
  let f : Subgroup.zpowers a →* Subgroup.zpowers a :=
    { toFun := fun x => ⟨b * (x : G) * b⁻¹, hconj x⟩
      map_one' := by simp
      map_mul' := by
        intro x y
        apply Subtype.ext
        simp [mul_assoc] }
  have hf : ∀ x : Subgroup.zpowers a,
    ((f x : Subgroup.zpowers a) : G) = b * (x : G) * b⁻¹ := by
    intro x
    rfl
  have hinj : Function.Injective f := by
    intro x y hxy
    apply Subtype.ext
    have hxy' := congrArg (fun z : Subgroup.zpowers a => (z : G)) hxy
    simpa [f, mul_assoc] using hxy'
  have hsurj : Function.Surjective f := by
    intro y
    let x : Subgroup.zpowers a := ⟨b⁻¹ * (y : G) * b, hconj_inv y⟩
    refine ⟨x, ?_⟩
    apply Subtype.ext
    simp [f, x, mul_assoc]
    refine ⟨f, ⟨(hinj, hsurj), hf⟩⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Herstein_exercise_2_5_23__me__c4d2b752

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axiomspropext, Quot.sound, fingerprint c4d2b752bf56a5d0

4.71 071. conjugation_restriction_unique_exponent

The problem

Let G be a group in which every subgroup is normal, and let a have infinite order. For each $b \in G$, show that there is a unique integer j such that conjugation by b restricts to an automorphism of the cyclic subgroup generated by a , sending a to a^j .

Lean statement

```
theorem conjugation_restriction_unique_exponent {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal)
  (a : G)
  (hinf : ∀ n : ℤ, a ^ n = 1 → n = 0)
  (b : G) :
  ∃! j : ℤ,
    ∃ f : Subgroup.zpowers a ≈* Subgroup.zpowers a,
      f ⟨a, Subgroup.mem_zpowers a⟩ =
        ⟨a ^ j, Subgroup.mem_zpowers_iff.mpr ⟨j, rfl⟩⟩ ^
        ∀ x : Subgroup.zpowers a, (f x : G) = b * x * b⁻¹
```

Parent. Let G be a group in which every subgroup is normal. Suppose that a has infinite order. Prove that for every $b \in G$, there is a unique integer j such that $ba = a^j b$.

Parent in Lean

```
theorem unique_commutation_exponent {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal)
  (a : G)
  (hinf : ∀ n : ℤ, a ^ n = 1 → n = 0)
  (b : G) :
  ∃! j : ℤ, b * a = a ^ j * b
```

What it contributes. Supplied the normality-based existence of a unique commutation exponent and the infinite-order cancellation argument; these are integrated with the explicitly constructed restriction automorphism.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child materially changes the conclusion by requiring construction of a group equivalence realizing conjugation on the cyclic subgroup, including inverse and homomorphism proofs. Recalling the parent only supplies the exponent and uniqueness equation; it does not supply the restricted automorphism witness.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a substantive conjugation-restriction automorphism of the cyclic subgroup and proves its generator action and behavior on every subgroup element. Constructing and verifying this bundled monoid isomorphism requires mathematics absent from the parent, beyond recalling the unique exponent argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a substantive construction: it must define and verify a monoid isomorphism of the cyclic subgroup induced by conjugation, then connect its generator image to the unique exponent. The parent's commutation-and-uniqueness argument does not supply this restriction map or its inverse and homomorphism laws.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propxt, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2221 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem conjugation_restriction_unique_exponent.extracted_1_1.{u_1} {G : Type u_1} [inst : Group G]
(hG : ∀ (H : Subgroup G), H.Normal) (a : G) (hinf : ∀ (n : ℤ), a ^ n = 1 → n = 0) (b : G) :
∃! j, ∃ f, f ⟨a, ...⟩ = ⟨a ^ j, ...⟩ ∧ ∀ (x : ↑(Subgroup.zpowers a)), ↑(f x) = b * ↑x * b-1
```

Read back by a model that saw only the Lean. *Let G be a group such that every subgroup of G is normal. Let a and b be elements of G , and assume that for every integer n , if $a^n = 1$ then $n = 0$. Then there exists a unique integer j such that there is a group homomorphism f from the subgroup of G consisting of all integer powers of a to itself satisfying $f(a) = a^j$ and, for every element x of this subgroup, the underlying element of $f(x)$ equals $b x b^{-1}$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Set Real Ideal Polynomial
open scoped BigOperators

theorem conjugation_restriction_unique_exponent {G : Type*} [Group G]
(hG : ∀ (H : Subgroup G), H.Normal)
(a : G)
(hinf : ∀ n : ℤ, a ^ n = 1 → n = 0)
(b : G) :
∃! j : ℤ,
  ∃ f : Subgroup.zpowers a ≈* Subgroup.zpowers a,
    f ⟨a, Subgroup.mem_zpowers a⟩ =
      ⟨a ^ j, Subgroup.mem_zpowers_iff.mpr ⟨j, rfl⟩⟩ ∧
      ∀ x : Subgroup.zpowers a, (f x : G) = b * x * b-1 := by
classical
have hnormal : (Subgroup.zpowers a).Normal := hG (Subgroup.zpowers a)
have ha : a ∈ Subgroup.zpowers a := Subgroup.mem_zpowers a
have hconj : b * a * b-1 ∈ Subgroup.zpowers a := by
  simpa using hnormal.conj_mem a ha b
obtain ⟨j, hj⟩ := Subgroup.mem_zpowers_iff.mp hconj
have hcomm : b * a = a ^ j * b := by
  calc
    b * a = (b * a * b-1) * b := by simp [mul_assoc]
    _ = a ^ j * b := by rw [hj]
have haj : a ^ j ∈ Subgroup.zpowers a := by
  exact Subgroup.mem_zpowers_iff.mpr ⟨j, rfl⟩
let f : Subgroup.zpowers a ≈* Subgroup.zpowers a :=
{ toFun := fun x => ⟨b * x * b-1, hnormal.conj_mem x.1 x.2 b⟩
  invFun := fun x => ⟨b-1 * x * b, hnormal.conj_mem' x.1 x.2 b⟩
  left_inv := by
    intro x
    apply Subtype.ext
    simp [mul_assoc]
  right_inv := by
    intro x
    apply Subtype.ext
    simp [mul_assoc]
  map_mul' := by
    intro x y
```

```

    apply Subtype.ext
    simp [mul_assoc] }
have hfgen : f ⟨a, ha⟩ = ⟨a ^ j, haj⟩ := by
  apply Subtype.ext
  simp [f, hj]
have hfres : ∀ x : Subgroup.zpowers a, (f x : G) = b * x * b⁻¹ := by
  intro x
  rfl
refine ⟨j, ⟨f, hfgen, hfres⟩, ?_⟩
intro k hk
rcases hk with ⟨g, hggen, hgres⟩
have hgenval : (g ⟨a, ha⟩ : G) = a ^ k := by
  exact congrArg Subtype.val hggen
have hconjk : b * a * b⁻¹ = a ^ k := by
  calc
    b * a * b⁻¹ = (g ⟨a, ha⟩ : G) := (hgres ⟨a, ha⟩).symm
    _ = a ^ k := hgenval
have hcommk : b * a = a ^ k * b := by
  calc
    b * a = (b * a * b⁻¹) * b := by simp [mul_assoc]
    _ = a ^ k * b := by rw [hconjk]
have hpow : a ^ j = a ^ k := by
  exact mul_right_cancel (hcomm.symm.trans hcommk)
have hrel : a ^ (j - k) = 1 := by
  rw [zpow_sub, hpow]
  simp
have hzero : j - k = 0 := hinf (j - k) hrel
exact (sub_eq_zero.mp hzero).symm

```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Herstein_exercise_2_5_23__mh.me__4b1a9d7c

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint 4b1a9d7cae1e5dfb

4.72 072. cubic_nilpotent_three_term_commutates

The problem

If $a^3 = 0$ in R , show that $a^2x + axa + xa^2$ commutes with a .

Lean statement

```
theorem cubic_nilpotent_three_term_commutates {R : Type*} [Ring R] (a x : R)
  (h : a ^ 3 = 0) :
  a * (a ^ 2 * x + a * x * a + x * a ^ 2) =
  (a ^ 2 * x + a * x * a + x * a ^ 2) * a := by
```

Parent. If $a^2 = 0$ in R , show that $ax + xa$ commutes with a .

Parent in Lean

```
theorem Herstein_exercise_4_2_6 {R : Type*} [Ring R] (a x : R)
  (h : a ^ 2 = 0) : a * (a * x + x * a) = (a * x + x * a) * a := by
```

What it contributes. Supplied the noncommutative ring setting and commutation-by-expansion pattern; the child replaces the square-zero anticommutator with a cube-zero three-term expression and adds the cubic nilpotence expansion checkpoint.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the square-zero anticommutator to a cubic-nilpotent three-term expression. Its proof must expand three summands and use the cubic relation to eliminate the endpoint terms, which is not supplied by the parent's square-zero argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes from square-zero to cubic nilpotence and requires proving cancellation of the outer cubic terms, leaving matching two-sided products. The parent's anticommutator argument cannot supply this result because $a^3 = 0$ does not imply $a^2 = 0$.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the commutation identity from square-zero to cubic-nilpotent elements and requires new three-term expansion plus cancellation using $a^3 = 0$. The parent's proof does not directly supply this cubic argument, so memorizing it would not substantially solve the child.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1984 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem cubic_nilpotent_three_term_commutates.extracted_1_1.{u_1} {R : Type u_1} [inst : Ring R] (a x : R)
  (h : a ^ 3 = 0) : a * (a ^ 2 * x + a * x * a + x * a ^ 2) = (a ^ 2 * x + a * x * a + x * a ^ 2) * a
```

Read back by a model that saw only the Lean. *Let R be a ring, and let a and x be elements of R . If $a^3 = 0$, then $a(a^2x + axa + xa^2) = (a^2x + axa + xa^2)a$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Set Real Ideal Polynomial
open scoped BigOperators

theorem cubic_nilpotent_three_term_commutates {R : Type*} [Ring R] (a x : R)
  (h : a ^ 3 = 0) :
  a * (a ^ 2 * x + a * x * a + x * a ^ 2) =
    (a ^ 2 * x + a * x * a + x * a ^ 2) * a := by
  have h_cube : a * a * a = 0 := by
    simpa [pow_succ, pow_two, mul_assoc] using h
  have h_left_cube : a * (a ^ 2 * x) = 0 := by
    calc
      a * (a ^ 2 * x) = (a * a * a) * x := by simp [pow_two, mul_assoc]
      _ = 0 := by rw [h_cube, zero_mul]
  have h_right_cube : (x * a ^ 2) * a = 0 := by
    calc
      (x * a ^ 2) * a = x * (a * a * a) := by simp [pow_two, mul_assoc]
      _ = 0 := by rw [h_cube, mul_zero]
  have h_expansion :
    a * (a ^ 2 * x + a * x * a + x * a ^ 2) =
      a ^ 2 * x * a + a * x * a ^ 2 := by
    calc
      a * (a ^ 2 * x + a * x * a + x * a ^ 2) =
        a * (a ^ 2 * x) + a * (a * x * a) + a * (x * a ^ 2) := by
          simp [mul_add]
      _ = a ^ 2 * x * a + a * x * a ^ 2 := by
        rw [h_left_cube]
        simp [pow_two, mul_assoc]
  have h_right_expansion :
    (a ^ 2 * x + a * x * a + x * a ^ 2) * a =
      a ^ 2 * x * a + a * x * a ^ 2 := by
    calc
      (a ^ 2 * x + a * x * a + x * a ^ 2) * a =
        (a ^ 2 * x) * a + (a * x * a) * a + (x * a ^ 2) * a := by
          simp [add_mul]
      _ = a ^ 2 * x * a + a * x * a ^ 2 := by
        rw [h_right_cube]
        simp [pow_two, mul_assoc]
  exact h_expansion.trans h_right_expansion.symm
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Herstein_exercise_4_2_6__me__70f5bb23

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, fingerprint 6e3358f449c14e20

4.73 073. anticommutator_commutes_with_a_iff_square_commutes

The problem

Let R be a ring and let $a, x \in R$. Prove that $ax + xa$ commutes with a if and only if a^2 commutes with x .

Lean statement

```
theorem anticommutator_commutes_with_a_iff_square_commutes {R : Type*} [Ring R] (a x : R) :
  ((a * x + x * a) * a = a * (a * x + x * a)) ↔ (a ^ 2 * x = x * a ^ 2)
```

Parent. Let R be a ring with $a^2 = 0$. Show that $ax + xa$ commutes with every polynomial in a with natural-number coefficients.

Parent in Lean

```
theorem anticommutator_commutes_with_nat_polynomial {R : Type*} [Ring R] (a x : R) (h : a ^ 2 = 0) (p : Polynomial ℕ) :
  (a * x + x * a) * p.sum (fun n c => (c : R) * a ^ n) =
  p.sum (fun n c => (c : R) * a ^ n) * (a * x + x * a)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the claim to an iff characterization in an arbitrary ring, removing the nilpotence hypothesis and requiring additive cancellation to relate anticommutator commutation to square commutation. The parent's polynomial-commutation induction does not supply this equivalence.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the claim from commuting with a polynomial in a nilpotent element to an iff characterization in an arbitrary ring. Its proof requires additive cancellation and noncommutative ring normalization in both directions, none of which is supplied by recalling the parent.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is a genuine general characterization in an arbitrary ring, with no nilpotence or polynomial hypothesis. Its proof uses bidirectional algebraic cancellation to relate anticommutator commutation to square commutation, which is not supplied by the parent's nilpotent polynomial-commutation argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2924 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem anticommutator_commutes_with_a_iff_square_commutes.extracted_1_1.{u_1} {R : Type u_1} [inst : Ring R]
  (a x : R) : (a * x + x * a) * a = a * (a * x + x * a) ↔ a ^ 2 * x = x * a ^ 2
```

Read back by a model that saw only the Lean. For any elements a and x of a ring R , the product of the anticommutator $ax + xa$ with a equals the product of a with the anticommutator $ax + xa$ if and only if a squared times x equals x times a squared.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Set Real Ideal Polynomial
open scoped BigOperators

theorem anticommutator_commutes_with_a_iff_square_commutes {R : Type*} [Ring R] (a x : R) :
  ((a * x + x * a) * a = a * (a * x + x * a)) ↔ (a ^ 2 * x = x * a ^ 2) := by
  constructor
  · intro h
    have h_cancel : x * a ^ 2 = a ^ 2 * x := by
      calc
        x * a ^ 2 = (a * x + x * a) * a - a * x * a := by noncomm_ring
        _ = a * (a * x + x * a) - a * x * a := by rw [h]
        _ = a ^ 2 * x := by noncomm_ring
    exact h_cancel.symm
  · intro h
    calc
      (a * x + x * a) * a = a * x * a + x * a ^ 2 := by noncomm_ring
      _ = a * x * a + a ^ 2 * x := by rw [h]
      _ = a * (a * x + x * a) := by noncomm_ring
```

Operator. mutation / mutation_easy **Lineage depth.** 3 **Certificate.** reproducible

Problem id. Herstein_exercise_4_2_6__mh.me.mh.me__fd705452

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propept, Quot.sound, fingerprint fd7054526db002b0

4.74 074. gaussian_mul_sq_eq_zero_iff

The problem

For a Gaussian integer a , multiplication by $(1 + i)^2$ gives zero exactly when $a = 0$.

Lean statement

```
theorem gaussian_mul_sq_eq_zero_iff (a : GaussianInt) : (((1, 1) : GaussianInt) ^ 2 * a = 0) ↔ a = 0
```

Parent. For every Gaussian integer a , show that the quotient of $2a$ by $(1 + i)^2$ exists uniquely and equals $a(-i)$.

Parent in Lean

```
theorem gaussian_factorization_unique_scaled (a : GaussianInt) : ∃ q : GaussianInt, ((1, 1) : GaussianInt) ^ 2 * q = 2 * a ∧ ∀ q₁ :  
  ↪ GaussianInt, ((1, 1) : GaussianInt) ^ 2 * q₁ = 2 * a → q₁ = a * ⟨0, -1⟩
```

What it contributes. Supplied the explicit factorization of $(1+i)^2$ and the nonzero checkpoint used in the cancellation argument.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion to a zero-product characterization and requires applying the no-zero-divisors argument in both directions. Although it reuses the factorization and nonzero checkpoint, the parent's existence-and-uniqueness result does not directly prove this claim.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the existential scaled-factorization/uniqueness claim into a zero-product characterization. Although it reuses the factorization and nonzero checkpoint, the decisive cancellation argument via no-zero-divisors is new and the parent's conclusion does not directly solve it.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely new cancellation characterization: after reusing the factorization only to establish nonzeroness, it applies the zero-product argument to derive the iff statement. Recalling the parent's existence-and-uniqueness proof does not establish this conclusion.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2778 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem gaussian_mul_sq_eq_zero_iff.extracted_1_1 (a : GaussianInt) : { re := 1, im := 1 } ^ 2 * a = 0 ↔ a = 0
```


Read back by a model that saw only the Lean. For every Gaussian integer a , the product of a with the square of the Gaussian integer having real part 1 and imaginary part 1 is zero if and only if a is zero.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Real
open scoped BigOperators

theorem gaussian_mul_sq_eq_zero_iff (a : GaussianInt) : ((⟨1, 1⟩ : GaussianInt) ^ 2 * a = 0) ↔ a = 0 := by
  have hfactor : (⟨1, 1⟩ : GaussianInt) ^ 2 * ⟨0, -1⟩ = 2 := by
    ext <=> simp [pow_two]
  have hne : (⟨1, 1⟩ : GaussianInt) ^ 2 ≠ 0 := by
    intro hz
    have hzero : (0 : GaussianInt) = 2 := by
      calc
        (0 : GaussianInt) = (⟨1, 1⟩ : GaussianInt) ^ 2 * ⟨0, -1⟩ := by rw [hz]; simp
        _ = 2 := hfactor
    norm_num at hzero
  constructor
  · intro h
    rcases mul_eq_zero.mp h with hleft | ha
    · exact False.elim (hne hleft)
    · exact ha
  · intro ha
    rw [ha]
    simp
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Ireland_Rosen_exercise_1_31__me.mh.me__e70e2bb2

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint e70e2bb2bcd97b04

4.75 075. gaussian_factorization_unique

The problem

Show that the quotient in the factorization of 2 by $(1 + i)^2$ exists, is unique, and equals $-i$.

Lean statement

```
theorem gaussian_factorization_unique : ∃ q : GaussianInt, ((1, 1) : GaussianInt) ^ 2 * q = 2 ∧ ∀ q₁ : GaussianInt, ((1, 1) : GaussianInt) ^ 2 * q₁ = 2 → q₁ = ⟨0, -1⟩ := by
```

Parent. Show that 2 is divisible by $(1 + i)^2$ in $\mathbb{Z}[i]$.

Parent in Lean

```
theorem Ireland_Rosen_exercise_1_31 : ((1, 1) : GaussianInt) ^ 2 | 2 := by
```

What it contributes. Supplies the explicit Gaussian factorization bridge, reused as the named existence checkpoint hfactor.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a genuine uniqueness theorem: after establishing existence, it proves any Gaussian quotient equal to 2 must have real and imaginary coordinates matching $\langle 0, -1 \rangle$. The parent's divisibility witness proof does not supply this coordinate-based uniqueness argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine uniqueness claim: after constructing the quotient, it must prove that every Gaussian integer solution has the same real and imaginary parts. The parent's divisibility witness supplies existence but does not supply this coordinate-wise uniqueness argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens divisibility to existence and uniqueness of the Gaussian quotient. The parent supplies only the witness equation; proving every quotient equals $\langle 0, -1 \rangle$ requires a new coordinate-level uniqueness argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2754 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem gaussian_factorization_unique.extracted_1_1 :
  ∃ q,
    { re := 1, im := 1 } ^ 2 * q = 2 ∧
    ∀ (q₁ : GaussianInt), { re := 1, im := 1 } ^ 2 * q₁ = 2 → q₁ = { re := 0, im := -1 }
```

Read back by a model that saw only the Lean. *There exists a Gaussian integer q such that $(1 + i)^2 q = 2$, and for every Gaussian integer q_1 , if $(1 + i)^2 q_1 = 2$, then $q_1 = -i$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Real
open scoped BigOperators

theorem gaussian_factorization_unique : ∃ q : GaussianInt, ((1, 1) : GaussianInt) ^ 2 * q = 2 ∧ ∀ q₁ : GaussianInt, ((1, 1) :
  GaussianInt) ^ 2 * q₁ = 2 → q₁ = ⟨0, -1⟩ := by
  have hfactor : ((1, 1) : GaussianInt) ^ 2 * ⟨0, -1⟩ = 2 := by
    ext <|> simp [pow_two]
  refine ⟨⟨0, -1⟩, hfactor, ?_⟩
  intro q₁ hq
  have hqr := congrArg (fun z : GaussianInt => z.re) hq
  have hqi := congrArg (fun z : GaussianInt => z.im) hq
  ext
  · simp [pow_two] at hqi ⊢
    omega
  · simp [pow_two] at hqr ⊢
    omega
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Ireland_Rosen_exercise_1_31__me__34206d90

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 34206d90a1c2bdae

4.76 076. finite_family_union_isOpen_in_subspace

The problem

Let T be a subset of a topological space X , and let $(A_i)_{i < n}$ be a finite family of subsets of X . If for every $i < n$ and every $x \in T \cap A_i$ there is an open set $U \subseteq X$ with $x \in U$ and $U \cap T \subseteq A_i$, prove that $\bigcup_{i < n} A_i$ is open in the subspace topology on T .

Lean statement

```
theorem finite_family_union_isOpen_in_subspace (X : Type*) [TopologicalSpace X] (T : Set X) (n : ℕ) (A : Fin n → Set X) (h : ∀ i x, x ∈ T
↪ → x ∈ A i → ∃ U : Set X, x ∈ U ∧ IsOpen U ∧ U ∩ T ⊆ A i) : IsOpen ((fun x : T => (x : X)) ⁻¹' (U i : Fin n, A i))
```

Parent. A bounded indexed family of locally open sets has an open union.

Parent in Lean

```
theorem bounded_family_union_isOpen (X : Type*) [TopologicalSpace X] (n : ℕ)
(A : Fin n → Set X)
(h : ∀ i x, x ∈ A i → ∃ U : Set X, x ∈ U ∧ IsOpen U ∧ U ⊆ A i) :
IsOpen (U i : Fin n, A i)
```

What it contributes. Supplied the finite indexed-union extraction and local-neighborhood transport pattern; the mutation adds an ambient subset T and subtype pullback witnesses.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the ambient topology: it proves openness of the union relative to an arbitrary subtype T , requiring a preimage construction and subtype-membership reasoning. The parent's proof in X does not supply this subspace argument, even though the local-union structure is shared.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the result to the subspace topology: it constructs an open neighborhood by taking the subtype preimage of U and uses the weaker relative containment $U \cap T \subseteq A_i$. This requires new preimage and subtype-membership reasoning absent from the parent proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely changes the setting to a subtype and requires reasoning with the subtype topology, preimages under the continuous subtype inclusion, and intersected local neighborhoods. The parent's ambient-space local-openness proof does not supply these steps, and the child is not a forced wrapper or decoration.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1972 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_family_union_isOpen_in_subspace.extracted_1_1.{u_1} (X : Type u_1) [inst : TopologicalSpace X]
  (T : Set X) (n : ℕ) (A : Fin n → Set X) (h : ∀ (i : Fin n), ∀ x ∈ T, x ∈ A i → ∃ U, x ∈ U ∧ IsOpen U ∧ U ∩ T ⊆ A i) :
  IsOpen ((fun x => ↑x) ⁻¹' U i, A i)
```

Read back by a model that saw only the Lean. *Let X be a topological space, let $T \subseteq X$, let n be a natural number, and let A be a family of n subsets of X indexed by $\text{Fin } n$. Assume that for every $i \in \text{Fin } n$ and every $x \in T$, if $x \in A_i$, then there exists an open set $U \subseteq X$ such that $x \in U$ and $U \cap T \subseteq A_i$. Then the union $\bigcup_{i \in \text{Fin } n} A_i$, regarded as a subset of the subspace T , is open in the subspace topology on T .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem finite_family_union_isOpen_in_subspace (X : Type*) [TopologicalSpace X] (T : Set X) (n : ℕ) (A : Fin n → Set X) (h : ∀ i x, x ∈ T
  ↪ → x ∈ A i → ∃ U : Set X, x ∈ U ∧ IsOpen U ∧ U ∩ T ⊆ A i) : IsOpen ((fun x : T => (x : X)) ⁻¹' (U i : Fin n, A i)) := by
  refine isOpen_iff_forall_mem_open.mpr ?_
  intro x hx
  change (x : X) ∈ U i : Fin n, A i at hx
  have hx_index : ∃ i : Fin n, (x : X) ∈ A i := by
    simpa only [Set.mem_iUnion] using hx
  rcases hx_index with ⟨i, hxi⟩
  rcases h i (x : X) x.property hxi with ⟨U, hxU, hUopen, hUsub⟩
  let V : Set T := (fun y : T => (y : X)) ⁻¹' U
  have hVopen : IsOpen V := by
    dsimp [V]
    exact hUopen.preimage continuous_subtype_val
  have hxV : x ∈ V := by
    dsimp [V]
    exact hxU
  have hVsub : V ⊆ (fun y : T => (y : X)) ⁻¹' (U i : Fin n, A i) := by
    intro y hy
    change (y : X) ∈ U at hy
    change (y : X) ∈ U i : Fin n, A i
    apply Set.mem_iUnion.mpr
    refine ⟨i, ?_⟩
    exact hUsub ⟨hy, y.property⟩
  have hlocal : V ⊆ (fun y : T => (y : X)) ⁻¹' (U i : Fin n, A i) ∧ IsOpen V ∧ x ∈ V :=
    ⟨hVsub, hVopen, hxV⟩
  exact ⟨V, hlocal⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Munkres_exercise_13_1__me.me__1996e9f4

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c62d0831393c4434

4.77 077. reindexed_iInf_topology_comparison

The problem

Let $\{\mathcal{T}_i\}_{i \in I}$ be a family of topologies on X , and let $f : J \rightarrow I$. Show that every topology on X contained in each $\mathcal{T}_i$ is contained in the largest topology contained in all the $\mathcal{T}_i$, and that this largest topology is contained in the largest topology contained in all the $\mathcal{T}_{f(j)}$.

Lean statement

```
theorem reindexed_iInf_topology_comparison (X I J : Type*) (T : I → TopologicalSpace X) (f : J → I) :
  (∀ U : TopologicalSpace X, (∀ i, U ≤ T i) → U ≤ ⨅ i, T i) ∧
  (⨅ i, T i) ≤ ⨅ j, T (f j) := by
```

Parent. Let $\mathcal{T}_\alpha$ be a family of topologies on X . Show that there is a unique largest topology on X contained in all the collections $\mathcal{T}_\alpha$.

Parent in Lean

```
theorem Munkres_exercise_13_4b2 (X I : Type*) (T : I → Set (Set X)) (h : ∀ i, is_topology X (T i)) :
  ∃! T', is_topology X T' ∧ (∀ i, T' ⊆ T i) ∧
  ∀ T'', is_topology X T'' → (∀ i, T'' ⊆ T i) → T'' ⊆ T' := by
```

What it contributes. Uses the parent's greatest-common-lower-bound checkpoint for the infimum topology, reformulated with Mathlib's TopologicalSpace order and extended through a reindexing map.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

The child reexpresses the parent's universal lower-topology construction in the TopologicalSpace lattice and adds a genuinely new reindexing comparison, proving that the infimum over the full family lies below the infimum over the mapped subfamily. The reindexing argument is not supplied by the parent's uniqueness proof.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the claim to an order-theoretic universal property and a reindexing comparison for infima of topologies. Its proof relies on 'iInf_le' and 'le_iInf', not on the parent's explicit topology construction or uniqueness argument.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child re-encodes the common topology as an infimum in the lattice of topologies and proves a nontrivial reindexing comparison via 'iInf'/'order' reasoning. This is not obtainable by recalling the parent's set-membership proof, despite expressing the same universal construction.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1866 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem reindexed_iInf_topology_comparison.extracted_1_1.{u_3, u_2, u_1} (X : Type u_1) (I : Type u_2) (J : Type u_3)
  (T : I → TopologicalSpace X) (f : J → I) :
  (∀ (U : TopologicalSpace X), (∀ (i : I), U ≤ T i) → U ≤ ⋂ i, T i) ∧ ⋂ i, T i ≤ ⋂ j, T (f j)
```

Read back by a model that saw only the Lean. For any types X, I , and J , any family T assigning a topological space on X to each $i \in I$, and any function $f: J \rightarrow I$, the following both hold: (1) for every topological space U on X , if $U \leq T(i)$ for every $i \in I$, then $U \leq$ the infimum of the topological spaces $T(i)$ over $i \in I$; and (2) the infimum of $T(i)$ over $i \in I$ is $\leq$ the infimum of $T(f(j))$ over $j \in J$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem reindexed_iInf_topology_comparison (X I J : Type*) (T : I → TopologicalSpace X) (f : J → I) :
  (∀ U : TopologicalSpace X, (∀ i, U ≤ T i) → U ≤ ⋂ i, T i) ∧
  (⋂ i, T i) ≤ ⋂ j, T (f j) := by
  have h_original_lower : ∀ i : I, (⋂ i, T i) ≤ T i := by
    intro i
    exact iInf_le T i
  have h_original_universal : ∀ U : TopologicalSpace X, (∀ i, U ≤ T i) → U ≤ ⋂ i, T i := by
    intro U hU
    refine le_iInf ?_
    intro i
    exact hU i
  have h_reindexed_lower : ∀ j : J, (⋂ i, T i) ≤ T (f j) := by
    intro j
    exact h_original_lower (f j)
  have h_reindexed_universal : ∀ U : TopologicalSpace X, (∀ j, U ≤ T (f j)) → U ≤ ⋂ j, T (f j) := by
    intro U hU
    refine le_iInf ?_
    intro j
    exact hU j
  have h_comparison : (⋂ i, T i) ≤ ⋂ j, T (f j) := by
    exact h_reindexed_universal _ h_reindexed_lower
  exact ⟨h_original_universal, h_comparison⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_13_4b2__me__73a8a1e4

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fin-
gerprint 8cc68b9e3994adb1

4.78 078. quotient_descent_first_projection

The problem

Let $f : X \times Y \rightarrow Z$ be continuous and constant on every fiber of the first projection, with Y nonempty. If the first projection is open, show that there is a unique continuous map $g : X \rightarrow Z$ such that $f(x, y) = g(x)$ for all (x, y) .

Lean statement

```
theorem quotient_descent_first_projection {X Y Z : Type*} [TopologicalSpace X] [TopologicalSpace Y] [TopologicalSpace Z]
  (hY : Nonempty Y) (f : X × Y → Z) (hf : Continuous f)
  (hconst : ∀ x : X, ∀ y₁ y₂ : Y, f (x, y₁) = f (x, y₂))
  (hopen : IsOpenMap (Prod.fst : X × Y → X)) :
  ∃! g : X → Z, Continuous g ∧ ∀ p : X × Y, g (Prod.fst p) = f p
```

Parent. A map $f : X \rightarrow Y$ is said to be an open map if for every open set U of X , the set $f(U)$ is open in Y . Show that $\pi_1 : X \times Y \rightarrow X$ and $\pi_2 : X \times Y \rightarrow Y$ are open maps.

Parent in Lean

```
theorem Munkres_exercise_16_4 {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (π₁ : X × Y → X)
  (π₂ : X × Y → Y)
  (h₁ : π₁ = Prod.fst)
  (h₂ : π₂ = Prod.snd) :
  IsOpenMap π₁ ∧ IsOpenMap π₂ := by
```

What it contributes. Uses the parent's first-coordinate projection and its open-map property as the topological descent checkpoint.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a genuine quotient-descent theorem: constructing the descended map, proving continuity via openness of the projection, and establishing uniqueness. The parent only proves that coordinate projections are open and does not supply the fiberwise factorization or continuity argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a substantive existence-and-uniqueness theorem for a continuous descended map, using fiberwise constancy, a chosen point of Y , and openness of the projection. The parent's open-map result supplies only one hypothesis and does not provide the descent or continuity argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuine quotient-descent theorem: it constructs the descended map, establishes continuity via openness of the projection, and proves existence and uniqueness using fiber constancy. This mathematics is not supplied by the parent's separate openness result and cannot be recovered by recalling its proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable

Check	By	Result
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2823 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quotient_descent_first_projection.extracted_1.1.{u_3, u_2, u_1} {X : Type u_1} {Y : Type u_2} {Z : Type u_3}
  [inst : TopologicalSpace X] [inst_1 : TopologicalSpace Y] [inst_2 : TopologicalSpace Z] (hY : Nonempty Y)
  (f : X × Y → Z) (hf : Continuous f) (hconst : ∀ (x : X) (y1 y2 : Y), f (x, y1) = f (x, y2))
  (hopen : IsOpenMap Prod.fst) : ∃! g, Continuous g ∧ ∀ (p : X × Y), g p.1 = f p
```

Read back by a model that saw only the Lean. Let X , Y , and Z be topological spaces, with Y nonempty. Let $f : X \times Y \rightarrow Z$ be continuous, and suppose that for every $x \in X$ and all $y_1, y_2 \in Y$, $f(x, y_1) = f(x, y_2)$. Assume that the first projection map $\text{Prod.fst} : X \times Y \rightarrow X$ is an open map. Then there exists a unique function $g : X \rightarrow Z$ such that g is continuous and, for every $p \in X \times Y$, $g(p.1) = f(p)$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem quotient_descent_first_projection {X Y Z : Type*} [TopologicalSpace X] [TopologicalSpace Y] [TopologicalSpace Z]
  (hY : Nonempty Y) (f : X × Y → Z) (hf : Continuous f)
  (hconst : ∀ x : X, ∀ y1 y2 : Y, f (x, y1) = f (x, y2))
  (hopen : IsOpenMap (Prod.fst : X × Y → X)) :
  ∃! g : X → Z, Continuous g ∧ ∀ p : X × Y, g (Prod.fst p) = f p := by
  let y0 : Y := Classical.choice hY
  let g : X → Z := fun x => f (x, y0)
  have hg : Continuous g := by
    rw [continuous_def]
    intro v hv
    have hU : IsOpen (f-1 v) := hf.isOpen_preimage v hv
    have hImage : IsOpen (Prod.fst '' (f-1 v)) := hopen _ hU
    have hEq : g-1 v = Prod.fst '' (f-1 v) := by
      ext x
      constructor
      · intro hx
        refine ⟨(x, y0), ?_, rfl⟩
        simp [g] using hx
      · rintro ⟨p, hp, hpx⟩
        rw [← hpx]
        change f (p.1, y0) ∈ v
        rw [← hconst p.1 p.2 y0]
        exact hp
    rw [hEq]
    exact hImage
  refine ⟨g, ⟨hg, ?_⟩, ?_⟩
  · intro p
    change f (p.1, y0) = f p
    exact hconst p.1 y0 p.2
  · intro g' hg'
    funext x
    have hx := hg'.2 (x, y0)
    simp [g] using hx
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_16_4__me__6458c5ee

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 6458c5ee2d242fa2

4.79 079. product_projections_are_quotient_maps

The problem

Let X and Y be nonempty topological spaces. Show that the two coordinate projections $X \times Y \rightarrow X$ and $X \times Y \rightarrow Y$ are quotient maps.

Lean statement

```
theorem product_projections_are_quotient_maps {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [Nonempty X] [Nonempty Y]
  (π₁ : X × Y → X) (π₂ : X × Y → Y)
  (h₁ : π₁ = Prod.fst) (h₂ : π₂ = Prod.snd) :
  Topology.IsQuotientMap π₁ ∧ Topology.IsQuotientMap π₂
```

Parent. A map $f : X \rightarrow Y$ is said to be an open map if for every open set U of X , the set $f(U)$ is open in Y . Show that $\pi_1 : X \times Y \rightarrow X$ and $\pi_2 : X \times Y \rightarrow Y$ are open maps.

Parent in Lean

```
theorem Munkres_exercise_16_4 {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (π₁ : X × Y → X)
  (π₂ : X × Y → Y)
  (h₁ : π₁ = Prod.fst)
  (h₂ : π₂ = Prod.snd) :
  IsOpenMap π₁ ∧ IsOpenMap π₂ := by
```

What it contributes. Reused the parent's openness checkpoints for the first and second product projections; the new proof additionally establishes continuity and surjectivity to obtain quotient maps.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from openness to quotient-map status. Proving this requires continuity and surjectivity from the nonempty factor hypotheses in addition to the parent's open-map argument, so recalling the parent proof is insufficient.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from openness to quotient-map status, requiring continuity and surjectivity in addition to the parent's open-map lemmas. The nonempty-factor hypotheses are used essentially to establish surjectivity, so recalling the parent proof is insufficient.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from openness to quotient-map status, requiring continuity and surjectivity in addition to the projection openness. The nonempty-factor hypotheses are mathematically necessary for surjectivity, so recalling the parent does not solve it.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2812 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem product_projections_are_quotient_maps.extracted_1_1.{u_2, u_1} {X : Type u_1} {Y : Type u_2}
  [inst : TopologicalSpace X] [inst_1 : TopologicalSpace Y] [inst_2 : Nonempty X] [inst_3 : Nonempty Y] (π₁ : X × Y → X)
  (π₂ : X × Y → Y) (h₁ : π₁ = Prod.fst) (h₂ : π₂ = Prod.snd) :
  Topology.IsQuotientMap π₁ ∧ Topology.IsQuotientMap π₂
```

Read back by a model that saw only the Lean. *Let X and Y be topological spaces, each nonempty. Let $\pi_1 : X \times Y \rightarrow X$ and $\pi_2 : X \times Y \rightarrow Y$ be functions such that π_1 is the first projection and π_2 is the second projection. Then π_1 and π_2 are both quotient maps.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem product_projections_are_quotient_maps {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [Nonempty X] [Nonempty Y]
  (π₁ : X × Y → X) (π₂ : X × Y → Y)
  (h₁ : π₁ = Prod.fst) (h₂ : π₂ = Prod.snd) :
  Topology.IsQuotientMap π₁ ∧ Topology.IsQuotientMap π₂ := by
  have hopen1 : IsOpenMap π₁ := by
    simpa [h₁] using (isOpenMap_fst : IsOpenMap (Prod.fst : X × Y → X))
  have hopen2 : IsOpenMap π₂ := by
    simpa [h₂] using (isOpenMap_snd : IsOpenMap (Prod.snd : X × Y → Y))
  have hcont1 : Continuous π₁ := by
    simpa [h₁] using (continuous_fst : Continuous (Prod.fst : X × Y → X))
  have hcont2 : Continuous π₂ := by
    simpa [h₂] using (continuous_snd : Continuous (Prod.snd : X × Y → Y))
  have hsurj1 : Function.Surjective π₁ := by
    rw [h₁]
    intro x
    obtain ⟨y⟩ := ⟨Nonempty Y⟩
    exact ⟨(x, y), rfl⟩
  have hsurj2 : Function.Surjective π₂ := by
    rw [h₂]
    intro y
    obtain ⟨x⟩ := ⟨Nonempty X⟩
    exact ⟨(x, y), rfl⟩
  have hquot1 : Topology.IsQuotientMap π₁ :=
    hopen1.isQuotientMap hcont1 hsurj1
  have hquot2 : Topology.IsQuotientMap π₂ :=
    hopen2.isQuotientMap hcont2 hsurj2
  exact ⟨hquot1, hquot2⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_16_4__me__a3e56ed4

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint a3e56ed4a85fb36c

4.80 080. projection_fst_snd_open

The problem

Let X , Y , and Z be topological spaces. Show that the projection $(X \times Y) \times Z \rightarrow X \times Z$, $(x, y, z) \mapsto (x, z)$, is an open map.

Lean statement

```
theorem projection_fst_snd_open {X Y Z : Type*} [TopologicalSpace X] [TopologicalSpace Y] [TopologicalSpace Z] :
  IsOpenMap (fun p : (X × Y) × Z => (p.1.1, p.2)) := by
```

Parent. A map $f : X \rightarrow Y$ is said to be an open map if for every open set U of X , the set $f(U)$ is open in Y . Show that $\pi_1 : X \times Y \rightarrow X$ and $\pi_2 : X \times Y \rightarrow Y$ are open maps.

Parent in Lean

```
theorem Munkres_exercise_16_4 {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (π₁ : X × Y → X)
  (π₂ : X × Y → Y)
  (h₁ : π₁ = Prod.fst)
  (h₂ : π₂ = Prod.snd) :
  IsOpenMap π₁ ∧ IsOpenMap π₂ := by
```

What it contributes. Uses the parent's binary projection openness checkpoint 'isOpenMap_snd', strengthened through product-map composition and reassociation.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes to a three-factor product and projects onto non-adjacent factors. Its proof requires product-map composition through a product homeomorphism, which is not supplied by recalling the parent's separate two-factor projection proofs.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes from two-factor coordinate projections to a three-factor projection onto $X \times Z$. Its proof requires product-map composition with a product associativity homeomorphism, which is not supplied by the parent's direct projection lemmas.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes to a three-factor projection onto $X \times Z$, requiring product-map composition with a product-associativity homeomorphism. The parent's direct 'fst'/'snd' applications do not supply this construction.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2312 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem projection_fst_snd_open.extracted_1_1.{u_3, u_2, u_1} {X : Type u_1} {Y : Type u_2} {Z : Type u_3}
  [inst : TopologicalSpace X] [inst_1 : TopologicalSpace Y] [inst_2 : TopologicalSpace Z] :
  IsOpenMap fun p => (p.1.1, p.2)
```

Read back by a model that saw only the Lean. *For topological spaces X , Y , and Z , the map from $(X \times Y) \times Z$ to $X \times Z$ defined by $((x, y), z) \mapsto (x, z)$ is an open map.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem projection_fst_snd_open {X Y Z : Type*} [TopologicalSpace X] [TopologicalSpace Y] [TopologicalSpace Z] :
  IsOpenMap (fun p : (X × Y) × Z => (p.1.1, p.2)) := by
  have hprod : IsOpenMap (Prod.map (id : X → X) (Prod.snd : Y × Z → Z)) :=
    IsOpenMap.id.prodMap isOpenMap_snd
  have hassoc : IsOpenMap (Homeomorph.prodAssoc X Y Z : (X × Y) × Z ≅ X × (Y × Z)) :=
    (Homeomorph.prodAssoc X Y Z).isOpenMap
  have hcomp : IsOpenMap ((Prod.map (id : X → X) (Prod.snd : Y × Z → Z)) ∘ (Homeomorph.prodAssoc X Y Z)) :=
    hprod.comp hassoc
  simpa [Function.comp_def] using hcomp
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_16_4__me__b7b89168

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint b7b89168a7a88528

4.81 081. continuous_strict_order_open

The problem

Let Y be an ordered set in the order topology, and let $f, g : X \rightarrow Y$ be continuous. Show that the set $\{x \mid f(x) < g(x)\}$ is open in X .

Lean statement

```
theorem continuous_strict_order_open {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {f g : X → Y}
  (hf : Continuous f) (hg : Continuous g) :
  IsOpen {x | f x < g x} := by
```

Parent. Let Y be an ordered set in the order topology. Let $f, g : X \rightarrow Y$ be continuous. Show that the set $\{x \mid f(x) \leq g(x)\}$ is closed in X .

Parent in Lean

```
theorem Munkres_exercise_18_8a {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {f g : X → Y}
  (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x | f x ≤ g x} := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes both the relation and topological predicate, proving openness of the strict-order locus via ‘isOpen_lt’; the parent’s closedness result via ‘isClosed_le’ does not supply this claim. The proof follows the plan directly and introduces no recall wrapper or trivial appended fact.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes both the relation and topological predicate from closedness of the non-strict order locus to openness of the strict-order locus. Its proof requires the distinct theorem ‘isOpen_lt’, so recalling the parent’s ‘isClosed_le’ proof does not solve it.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes both the relation and topological predicate, from closedness of the non-strict-order locus to openness of the strict-order locus. Its proof uses the distinct strict-order continuity lemma ‘isOpen_lt’, so recalling the parent’s ‘isClosed_le’ proof does not solve it.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2234 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem continuous_strict_order_open.extracted_1_1.{u_2, u_1} {X : Type u_1} {Y : Type u_2} [inst : TopologicalSpace X]
[inst_1 : TopologicalSpace Y] [inst_2 : LinearOrder Y] [inst_3 : OrderTopology Y] {f g : X → Y} (hf : Continuous f)
(hg : Continuous g) : IsOpen {x | f x < g x}
```

Read back by a model that saw only the Lean. *Let X be a topological space and let Y be a linearly ordered topological space equipped with its order topology. If $f : X \rightarrow Y$ and $g : X \rightarrow Y$ are continuous, then the set $\{x \in X \mid f(x) < g(x)\}$ is open in X .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem continuous_strict_order_open {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
[LinearOrder Y] [OrderTopology Y] {f g : X → Y}
(hf : Continuous f) (hg : Continuous g) :
IsOpen {x | f x < g x} := by
have hstrict : IsOpen {x | f x < g x} := by
  exact isOpen_lt hf hg
exact hstrict
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_18_8a__me__20a4ae19

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 20a4ae1976535b65

4.82 082. finite_simultaneous_comparison_closed

The problem

Let Y be a linearly ordered topological space and let $f_i, g_i : X \rightarrow Y$ be continuous for each index i in a nonempty finite set. Show that the set of points satisfying $f_i(x) \leq g_i(x)$ for every i is closed in X .

Lean statement

```
theorem finite_simultaneous_comparison_closed {ι X Y : Type*} [Fintype ι] [Nonempty ι]
  [TopologicalSpace X] [TopologicalSpace Y] [LinearOrder Y] [OrderTopology Y]
  (f g : ι → X → Y) (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i)) :
  IsClosed {x | ∀ i, f i x ≤ g i x} := by
```

Parent. Let Y be an ordered set in the order topology. Let $f, g : X \rightarrow Y$ be continuous. Show that the set $\{x \mid f(x) \leq g(x)\}$ is closed in X .

Parent in Lean

```
theorem Munkres_exercise_18_8a {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {f g : X → Y}
  (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x | f x ≤ g x} := by
```

What it contributes. Used as the local checkpoint ‘hlocal’, proving each indexed comparison locus is closed before forming the master intersection.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes one comparison to a simultaneous family and combines the pairwise closed loci via an intersection theorem. A memorized proof of the parent only supplies the local closedness step, not the family-level intersection argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes one comparison to a finite simultaneous comparison, requiring identification with an indexed intersection and application of closedness under arbitrary intersections. The parent’s direct ‘isClosed_le’ proof does not supply this finite-family reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is a genuine finite-family generalization: it converts each pairwise comparison locus into a finite intersection and uses the closedness of arbitrary intersections. This requires the new intersection/set-extensionality argument and cannot be solved by recalling the parent’s direct ‘isClosed_le’ proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2909 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_simultaneous_comparison_closed.extracted_1_1.{u_3, u_2, u_1} {ι : Type u_1} {X : Type u_2} {Y : Type u_3}
  [inst : Nonempty ι] [inst_1 : TopologicalSpace X] [inst_2 : TopologicalSpace Y] [inst_3 : LinearOrder Y]
  [inst_4 : OrderTopology Y] (f g : ι → X → Y) (hf : ∀ (i : ι), Continuous (f i)) (hg : ∀ (i : ι), Continuous (g i)) :
  IsClosed {x | ∀ (i : ι), f i x ≤ g i x}
```

Read back by a model that saw only the Lean. *Let ι be a nonempty type, let X and Y be topological spaces, and suppose Y is linearly ordered with its order topology. For functions $f, g : \iota \rightarrow X \rightarrow Y$ such that, for every $i \in \iota$, the functions $x \mapsto f(i, x)$ and $x \mapsto g(i, x)$ are continuous, the set of all $x \in X$ satisfying $f(i, x) \leq g(i, x)$ for every $i \in \iota$ is closed in X .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace
open scoped Topology

theorem finite_simultaneous_comparison_closed {ι X Y : Type*} [Fintype ι] [Nonempty ι]
  [TopologicalSpace X] [TopologicalSpace Y] [LinearOrder Y] [OrderTopology Y]
  (f g : ι → X → Y) (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i)) :
  IsClosed {x | ∀ i, f i x ≤ g i x} := by
  have hlocal : ∀ i, IsClosed {x | f i x ≤ g i x} := by
    intro i
    exact isClosed_le (hf i) (hg i)
  have hset : (⋂ i, {x : X | f i x ≤ g i x}) = {x | ∀ i, f i x ≤ g i x} := by
    ext x
    simp
  rw [← hset]
  exact isClosed_iInter hlocal
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_18_8a__me__292188c1

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fin-
gerprint 292188c1730cb824

4.83 083. closed_restricted_le

The problem

Let X be a topological space and Y an ordered set with the order topology. If $s \subseteq X$ is closed and $f, g : X \rightarrow Y$ are continuous, show that $\{x \in s \mid f(x) \leq g(x)\}$ is closed in X .

Lean statement

```
theorem closed_restricted_le {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {s : Set X} {f g : X → Y}
  (hs : IsClosed s) (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x | x ∈ s ∧ f x ≤ g x}
```

Parent. Let Y be an ordered set in the order topology. Let $f, g : X \rightarrow Y$ be continuous. Show that the set $\{x \mid f(x) \leq g(x)\}$ is closed in X .

Parent in Lean

```
theorem Munkres_exercise_18_8a {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {f g : X → Y}
  (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x | f x ≤ g x} := by
```

What it contributes. Supplies the closedness checkpoint for the set where two continuous functions satisfy an order inequality.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child restricts the closed inequality set to a closed subset, requiring the new topological fact that intersections of closed sets are closed. The parent checkpoint supplies only the inequality-set closedness; it does not prove the restricted result by recall alone.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely extends the parent by intersecting the closed inequality set with an arbitrary closed set, requiring an additional closed-set intersection argument. The parent's inequality result is only one subgoal, so recalling its proof alone does not solve the child.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely strengthens the result by restricting the comparison set to an arbitrary closed subset. It uses the parent's closedness theorem as one checkpoint, then requires the new mathematical step that intersections of closed sets are closed.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2772 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem closed_restricted_le.extracted_1.1.{u_2, u_1} {X : Type u_1} {Y : Type u_2} [inst : TopologicalSpace X]
[inst_1 : TopologicalSpace Y] [inst_2 : LinearOrder Y] [inst_3 : OrderTopology Y] {s : Set X} {f g : X → Y}
(hs : IsClosed s) (hf : Continuous f) (hg : Continuous g) : IsClosed {x | x ∈ s ∧ f x ≤ g x}
```

Read back by a model that saw only the Lean. *Let X and Y be topological spaces, with Y equipped with a linear order whose topology is the order topology. For a closed set $s \subseteq X$ and continuous functions $f, g : X \rightarrow Y$, the set $\{x \in X \mid x \in s \text{ and } f(x) \leq g(x)\}$ is closed in X .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem closed_restricted_le {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
[LinearOrder Y] [OrderTopology Y] {s : Set X} {f g : X → Y}
(hs : IsClosed s) (hf : Continuous f) (hg : Continuous g) :
IsClosed {x | x ∈ s ∧ f x ≤ g x} := by
have hineq : IsClosed {x | f x ≤ g x} := by
  exact isClosed_le hf hg
have hmaster : IsClosed (s ∩ {x | f x ≤ g x}) := by
  exact hs.inter hineq
have hset : {x | x ∈ s ∧ f x ≤ g x} = s ∩ {x | f x ≤ g x} := by
  ext x
  rfl
rw [hset]
exact hmaster
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_18_8a__me__6cceb54a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 6cceb54a30e3621e

4.84 084. finite_joint_comparison_isClosed

The problem

Let X be a topological space and Y a linearly ordered topological space. Let $n \in \mathbb{N}$, and for each $i = 0, \dots, n-1$, let $f_i, g_i : X \rightarrow Y$ be continuous. Show that the set $\{x \in X \mid f_i(x) \leq g_i(x) \text{ for every } i = 0, \dots, n-1\}$ is closed in X .

Lean statement

```
theorem finite_joint_comparison_isClosed {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {n : ℕ} {f g : Fin n → X → Y}
  (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i)) :
  IsClosed {x | ∀ i, f i x ≤ g i x}
```

Parent. Let Y be an ordered set in the order topology. Let $f, g : X \rightarrow Y$ be continuous. Show that the set $\{x \mid f(x) \leq g(x)\}$ is closed in X .

Parent in Lean

```
theorem Munkres_exercise_18_8a {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {f g : X → Y}
  (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x | f x ≤ g x} := by
```

What it contributes. Supplied the ordered-topology comparison-closedness checkpoint via `isClosed_le`, generalized from one pair of functions to every $i : \text{Fin } n$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes one comparison to a finite family and must combine the pointwise closed comparison sets using an arbitrary intersection argument. The parent's 'isClosed_le' application supplies only the individual cases, not the joint closedness proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes a single comparison to a finite family and requires proving the joint set is an indexed intersection, then applying closedness of finite intersections. The parent's direct 'isClosed_le' proof does not supply this intersection reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces one comparison with a finite family and proves closedness via an intersection of closed sets, requiring the additional finite-intersection argument and set extensionality. Recalling the parent's direct 'isClosed_le' proof does not establish the joint result.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1941 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_joint_comparison_isClosed.extracted_1_1.{u_2, u_1} {X : Type u_1} {Y : Type u_2}
  [inst : TopologicalSpace X] [inst_1 : TopologicalSpace Y] [inst_2 : LinearOrder Y] [inst_3 : OrderTopology Y] {n : ℕ}
  {f g : Fin n → X → Y} (hf : ∀ (i : Fin n), Continuous (f i)) (hg : ∀ (i : Fin n), Continuous (g i)) :
  IsClosed {x | ∀ (i : Fin n), f i x ≤ g i x}
```

Read back by a model that saw only the Lean. *Let X and Y be topological spaces, with Y equipped with a linear order whose order topology is the given topology. Let n be a natural number, and let f and g be families of functions from X to Y indexed by $\text{Fin } n$. Assume that for every i in $\text{Fin } n$, the functions f_i and g_i are continuous. Then the set of all x in X such that $f_i(x) \leq g_i(x)$ for every i in $\text{Fin } n$ is closed in X .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem finite_joint_comparison_isClosed {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {n : ℕ} {f g : Fin n → X → Y}
  (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i)) :
  IsClosed {x | ∀ i, f i x ≤ g i x} := by
  have hcomp : ∀ i : Fin n, IsClosed {x | f i x ≤ g i x} := by
    intro i
    exact isClosed_le (hf i) (hg i)
  have hjoint : IsClosed (⋂ i : Fin n, {x | f i x ≤ g i x}) := by
    exact isClosed_iInter hcomp
  have hrepr : {x | ∀ i, f i x ≤ g i x} = ⋂ i : Fin n, {x | f i x ≤ g i x} := by
    ext x
    simp
  rw [hrepr]
  exact hjoint
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_18_8a__me__c65444ab

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint ac793e1e6ac6c933

4.85 085. connected_preimage_of_closed_connected

The problem

Let $p : X \rightarrow Y$ be a quotient map whose fibers are connected. If $C \subseteq Y$ is closed and connected, then $p^{-1}(C)$ is connected.

Lean statement

```
theorem connected_preimage_of_closed_connected {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (hq : IsQuotientMap p)
  {C : Set Y} (hCclosed : IsClosed C) (hCconn : IsConnected C)
  (hX : ∀ y : Y, IsConnected (p ⁻¹' {y} : Set Y)) :
  IsConnected (p ⁻¹' C)
```

Parent. Let $p : X \rightarrow Y$ be a quotient map. Show that if each set $p^{-1}(\{y\})$ is connected, and if Y is connected, then X is connected.

Parent in Lean

```
theorem Munkres_exercise_23_11 {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (hq : IsQuotientMap p)
  (hY : ConnectedSpace Y) (hX : ∀ y : Y, IsConnected (p ⁻¹' {y})) :
  ConnectedSpace X := by
```

What it contributes. Supplied the quotient-map and connected-fiber hypotheses; the global connected-codomain conclusion is generalized to a closed connected subset C .

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the argument to an arbitrary closed connected subset C , proving connectedness of its preimage via the restricted quotient-map theorem and connected fibers. This is not obtainable by recalling the parent's whole-space conclusion.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from connectedness of the entire domain to connectedness of the preimage of an arbitrary closed connected subset, requiring the coinducing preimage theorem and genuinely parameterizing the codomain region. It is not recoverable by recalling the parent's proof, which only handles the whole codomain.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes connectedness from the whole codomain to the preimage of an arbitrary closed connected subset. Its proof uses the coinducing restriction theorem and cannot be obtained by recalling the parent's whole-space open-cover argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2202 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem connected_preimage_of_closed_connected.extracted_1_1.{u_2, u_1} {X : Type u_1} {Y : Type u_2}
  [inst : TopologicalSpace X] [inst_1 : TopologicalSpace Y] (p : X → Y) (hq : IsQuotientMap p) {C : Set Y}
  (hCclosed : IsClosed C) (hCconn : IsConnected C) (hX : ∀ (y : Y), IsConnected (p ⁻¹ {y})) :
  IsConnected (p ⁻¹ C)
```

Read back by a model that saw only the Lean. *Let X and Y be topological spaces, let $p : X \rightarrow Y$ be a quotient map, and let $C \subseteq Y$ be a closed and connected set. Assume that for every $y \in Y$, the fiber $p^{-1}(\{y\})$ is connected. Then the preimage $p^{-1}(C)$ is connected.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace Topology

theorem connected_preimage_of_closed_connected {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (hq : IsQuotientMap p)
  {C : Set Y} (hCclosed : IsClosed C) (hCconn : IsConnected C)
  (hX : ∀ y : Y, IsConnected (p ⁻¹ {y} : Set Y)) :
  IsConnected (p ⁻¹ C) := by
  have hcoind : IsCoinducing p := hq.isCoinducing
  have hfib : ∀ y : Y, IsConnected (p ⁻¹ {y} : Set Y) := hX
  have hrestricted : IsConnected (p ⁻¹ C) :=
    hcoind.isConnected_preimage_of_isClosed hfib hCclosed hCconn
  exact hrestricted
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_23_11__me__6b34bd44

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 6b34bd44c4e33bff

4.86 086. finite_ordered_path_connected_union

The problem

Let $A_0, A_1, \dots, A_{N-1}$ be path-connected subspaces of a topological space, with $N > 0$. If each A_n for $n > 0$ meets the union of its predecessors, prove that their union is path connected.

Lean statement

```
theorem finite_ordered_path_connected_union {X : Type*} [TopologicalSpace X] (N : ℕ) (hN : 0 < N) (A : ℕ → Set X) :
  (∀ n, n < N → IsPathConnected (A n)) →
  (∀ n, n < N → n ≠ 0 → (A n ∩ ⋃ k < n, A k).Nonempty) →
  IsPathConnected (⋃ k < N, A k)
```

Parent. Let a finite family of path-connected subspaces have a distinguished root. If every nonroot member is attached to the root through a finite rooted relation and attached members intersect, prove that the union is path connected.

Parent in Lean

```
theorem finite_path_connected_union_rooted_relation {X ι : Type*} [TopologicalSpace X] [Fintype ι] (A : ι → Set X) (root : ι) (R : ι → ι
  ⇐ → Prop) :
  (∀ i, i ≠ root → R i root) →
  (∀ i j, R i j → (A i ∩ A j).Nonempty) →
  (∀ i, IsPathConnected (A i)) →
  IsPathConnected (⋃ i, A i)
```

What it contributes. Provides the finite-union induction checkpoint and the path-connected union closure pattern; the child replaces root attachment by ordered predecessor attachment and makes the stage carrier path connected at every index.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child requires a genuinely different induction on the ordered bound: each new set attaches to the entire predecessor union, not to a fixed root as in the parent. The parent proof cannot be recalled or directly instantiated to establish this weaker, sequential attachment condition.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the parent's fixed-root attachment relation with predecessor-to-union attachments and proves the result by induction on the bound. This requires genuinely new reasoning about finite initial segments and union decomposition, so recalling the parent's proof is not sufficient.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the rooted-star attachment condition with an ordered predecessor-union condition, so the parent theorem cannot be instantiated to discharge it. Its proof requires induction on the bound and stagewise gluing to the entire preceding union, which is distinct mathematics from attaching every set to a fixed root.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing

Check	By	Result
corpus_dedup	hash	unique against 2865 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_ordered_path_connected_union.extracted_1_1.{u_1} {X : Type u_1} [inst : TopologicalSpace X] (N : ℕ)
(hN : 0 < N) (A : ℕ → Set X) :
(∀ n < N, IsPathConnected (A n)) →
(∀ n < N, n ≠ 0 → (A n ∩ ⋃ k < n, A k).Nonempty) → IsPathConnected (⋃ k < N, A k)
```

Read back by a model that saw only the Lean. *Let X be a topological space, let N be a positive natural number, and let A be a family of subsets of X indexed by the natural numbers. If every set A_n with $n < N$ is path-connected, and for every $n < N$ with $n \neq 0$, the intersection of A_n with the union of all A_k for $k < n$ is nonempty, then the union of all A_k for $k < N$ is path-connected.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem finite_ordered_path_connected_union {X : Type*} [TopologicalSpace X] (N : ℕ) (hN : 0 < N) (A : ℕ → Set X) :
(∀ n, n < N → IsPathConnected (A n)) →
(∀ n, n < N → n ≠ 0 → (A n ∩ ⋃ k < n, A k).Nonempty) →
IsPathConnected (⋃ k < N, A k) := by
revert hN
induction N with
| zero =>
intro hN
omega
| succ N ih =>
intro hN hA hPred
by_cases hN0 : N = 0
· subst N
have heq : (⋃ k < Nat.succ 0, A k) = A 0 := by
ext x
constructor
· intro hx
rcases mem_iUnion.1 hx with ⟨k, hx⟩
rcases mem_iUnion.1 hx with ⟨hk, hx⟩
have hk0 : k = 0 := by omega
simp [hk0] using hx
· intro hx
exact mem_iUnion.2 ⟨0, mem_iUnion.2 ⟨by omega, hx⟩⟩
rw [heq]
exact hA 0 (by omega)
· have hprev : IsPathConnected (⋃ k < N, A k) := by
apply ih (by omega)
· intro k hk
exact hA k (by omega)
· intro k hk hk0
exact hPred k (by omega) hk0
have hinter : (A N ∩ ⋃ k < N, A k).Nonempty :=
hPred N (Nat.lt_succ_self N) (by omega)
have hstage : IsPathConnected (A N ∪ ⋃ k < N, A k) :=
(hA N (Nat.lt_succ_self N)).union hprev hinter
have heq : (⋃ k < Nat.succ N, A k) = A N ∪ ⋃ k < N, A k := by
ext x
constructor
· intro hx
rcases mem_iUnion.1 hx with ⟨k, hx⟩
rcases mem_iUnion.1 hx with ⟨hk, hx⟩
```

```

by_cases hkN : k = N
· exact Or.inl (hkN ▸ hx)
· right
  exact mem_iUnion.2 (k, mem_iUnion.2 (by omega, hx))
· intro hx
  rcases hx with hx | hx
  · exact mem_iUnion.2 (N, mem_iUnion.2 (Nat.lt_succ_self N, hx))
  · rcases mem_iUnion.1 hx with ⟨k, hx⟩
    rcases mem_iUnion.1 hx with ⟨hk, hx⟩
    exact mem_iUnion.2 (k, mem_iUnion.2 (by omega, hx))
rw [heq]
exact hstage

```

Operator. mutation / mutation_easy **Lineage depth.** 3 **Certificate.** reproducible

Problem id. Munkres_exercise_23_2__me.me.me.me__6c2d5cbc

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 6c2d5cbcbe61ce57

4.87 087. connected_product_image_iUnion

The problem

Let (A_n) be connected subspaces of a uniform space, and suppose each A_n intersects the union of its predecessors. If f and g are uniformly continuous, show that the union of the sets $\{(f(x), g(x)) : x \in A_n\}$ is connected.

Lean statement

```
theorem connected_product_image_iUnion
  {α β γ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ]
  {A : ℕ → Set α} {f : α → β} {g : α → γ}
  (hA : ∀ n, IsConnected (A n))
  (hPred : ∀ n, n ≠ 0 → (A n ∩ ⋃ k < n, A k).Nonempty)
  (hf : UniformContinuous f) (hg : UniformContinuous g) :
  IsConnected (⋃ n, (fun x => (f x, g x)) '' A n)
```

Parent. Let (A_n) be connected subspaces of a uniform space, and suppose each A_n intersects the union of its predecessors. If f , g , and h are uniformly continuous, show that the union of the traced sets $\{(g(f(x)), h(g(f(x)))) : x \in A_n\}$ is connected.

Parent in Lean

```
theorem connected_adjacent_trace_iUnion
  {α γ δ : Type*} [UniformSpace α] [UniformSpace γ] [UniformSpace δ]
  {A : ℕ → Set α} {f : α → γ} {g : γ → γ} {h : γ → δ}
  (hA : ∀ n, IsConnected (A n))
  (hPred : ∀ n, n ≠ 0 → (A n ∩ ⋃ k < n, A k).Nonempty)
  (hf : UniformContinuous f) (hg : UniformContinuous g) (hh : UniformContinuous h) :
  IsConnected (⋃ n, (fun x => (g (f x), h (g (f x)))) '' A n)
```

What it contributes. Supplied the predecessor-overlap chain argument and the connected-union theorem pattern; the child changes the final map to a direct continuous product map and performs the mapping only after proving union connectedness.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child uses a genuinely different product map and proves connectedness of the union before mapping, then establishes an explicit image-iUnion equality. This is not recoverable by applying the parent's composed-trace proof and satisfies the mutation plan.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child uses a genuinely different product map $\$(f(x), g(x))\$$ and proves connectedness of the preimage union before applying the continuous map, rather than reusing the parent's traced-image overlap argument. The explicit image-of-union equivalence and new continuity/product reasoning prevent recall of the parent's final proof from directly solving it.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child uses a genuinely different product map $\$(f(x), g(x))\$$ and proves connectedness of the original union before mapping, then transports it via continuity and an explicit image-iUnion equality. This reasoning is not supplied by recalling the parent's composed trace map proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2188 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem connected_product_image_iUnion.extracted_1_1.{u_3, u_2, u_1} {α : Type u_1} {β : Type u_2} {γ : Type u_3}
  [inst : UniformSpace α] [inst_1 : UniformSpace β] [inst_2 : UniformSpace γ] {A : ℕ → Set α} {f : α → β} {g : α → γ}
  (hA : ∀ (n : ℕ), IsConnected (A n)) (hPred : ∀ (n : ℕ), n ≠ 0 → (A n ∩ ⋃ k, U (λ : k < n), A k).Nonempty)
  (hf : UniformContinuous f) (hg : UniformContinuous g) : IsConnected (U n, (fun x => (f x, g x)) '' A n)
```

Read back by a model that saw only the Lean. Let α , β , and γ be uniform spaces. Let A be a sequence of subsets of α , and let $f : \alpha \rightarrow \beta$ and $g : \alpha \rightarrow \gamma$. Assume that for every natural number n , $A n$ is connected; for every natural number $n \neq 0$, the intersection of $A n$ with the union of all $A k$ for $k < n$ is nonempty; and f and g are uniformly continuous. Then the union over all natural numbers n of the images of $A n$ under the map $x \mapsto (f x, g x)$ is connected.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace
open scoped Topology

theorem connected_product_image_iUnion
  {α β γ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ]
  {A : ℕ → Set α} {f : α → β} {g : α → γ}
  (hA : ∀ n, IsConnected (A n))
  (hPred : ∀ n, n ≠ 0 → (A n ∩ ⋃ k < n, A k).Nonempty)
  (hf : UniformContinuous f) (hg : UniformContinuous g) :
  IsConnected (U n, (fun x => (f x, g x)) '' A n) := by
  classical
  let R : ℕ → ℕ → Prop := fun i j => (A i ∩ A j).Nonempty
  have hReach : ∀ n, Relation.ReflTransGen R 0 n := by
    intro n
    induction n using Nat.strong_induction_on with
    | h n ih =>
      cases n with
      | zero => exact Relation.ReflTransGen.refl
      | succ n =>
        obtain ⟨x, hxN, hxPred⟩ := hPred (n + 1) (by omega)
        rcases Set.mem_iUnion.mp hxPred with ⟨k, hxPred⟩
        rcases Set.mem_iUnion.mp hxPred with ⟨hk, hxk⟩
        have hedge : R k (n + 1) := ⟨x, hxk, hxN⟩
        exact (ih k hk).tail hedge
  have hSym : Symmetric R := by
    intro i j hij
    simpa [R, Set.inter_comm] using hij
  have hChain : ∀ i j : ℕ, Relation.ReflTransGen R i j := by
    intro i j
    exact (Relation.ReflTransGen.symmetric hSym (hReach i)).trans (hReach j)
  have hUnion : IsConnected (U n, A n) := by
    simpa [R] using IsConnected.iUnion_of_reflTransGen (s := A) (H := hA) (K := hChain)
  let T : α → β × γ := fun x => (f x, g x)
  have hT : Continuous T := by
    dsimp [T]
    exact hf.continuous.prodMk hg.continuous
  have hImage : IsConnected (T '' (U n, A n)) :=
```

```

hUnion.image T hT.continuousOn
have hEq : T '' (U n, A n) = U n, T '' A n := by
  ext y
  constructor
  · rintro ⟨x, hx, rfl⟩
    rcases Set.mem_iUnion.mp hx with ⟨n, hxn⟩
    exact Set.mem_iUnion.mpr ⟨n, ⟨x, hxn, rfl⟩⟩
  · intro hy
    rcases Set.mem_iUnion.mp hy with ⟨n, ⟨x, hxn, rfl⟩⟩
    exact ⟨x, Set.mem_iUnion.mpr ⟨n, hxn⟩, rfl⟩
rw [← hEq]
simpa [T] using hImage

```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Munkres_exercise_23_2__me.me.me__1c1dc987

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 1c1dc987ff5b5411

4.88 088. finite_path_connected_union_rooted_relation

The problem

Let a finite family of path-connected subspaces have a distinguished root. If every nonroot member is attached to the root through a finite rooted relation and attached members intersect, prove that the union is path connected.

Lean statement

```
theorem finite_path_connected_union_rooted_relation {X ι : Type*} [TopologicalSpace X] [Fintype ι] (A : ι → Set X) (root : ι) (R : ι → ι
  → Prop) :
  (∀ i, i ≠ root → R i root) →
  (∀ i j, R i j → (A i ∩ A j).Nonempty) →
  (∀ i, IsPathConnected (A i)) →
  IsPathConnected (⋃ i, A i)
```

Parent. Let $A_0, A_1, \dots, A_{n-1}$ be preconnected subspaces of a topological space, with $n > 0$ and A_0 nonempty. If each A_k for $k > 0$ meets the union of its predecessors, prove that their union is connected.

Parent in Lean

```
theorem finite_connected_union_predecessor_preconnected {X : Type*} [TopologicalSpace X]
  (A : ℕ → Set X) (n : ℕ) :
  0 < n →
  (A 0).Nonempty →
  (∀ k < n, IsPreconnected (A k)) →
  (∀ k, 0 < k → k < n →
    (A k ∩ ⋃ j ∈ Finset.range k, A j).Nonempty) →
  IsConnected (⋃ k ∈ Finset.range n, A k)
```

What it contributes. Generalizes the parent's finite predecessor-union induction and attachment-intersection checkpoint, while replacing preconnectedness/connectedness with path connectedness and an arbitrary finite rooted index scheme.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

The child is not a wrapper or decoration: it replaces predecessor attachments by a finite rooted attachment relation and proves path connectedness via finite-set induction and path-connected union. A memorized proof of the parent's Nat-indexed preconnectedness theorem would not directly solve it.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the natural predecessor induction with a finite rooted attachment argument and strengthens connectedness to path connectedness for path-connected pieces. Its proof requires new finite-set induction and path-connected union reasoning, so recalling the parent does not substantially solve it.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces natural-number predecessor induction with finite-type finset induction and strengthens connectedness to path connectedness using path-connected pieces and rooted intersection attachments. Its proof requires a genuinely different union argument and cannot be obtained by recalling the parent's theorem.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry

Check	By	Result
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2852 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	MISMATCH: Statement B omits the finiteness hypothesis on the family/index type.; Statement B requires each nonroot member to relate directly to the root, whereas Statement A allows attachment through a finite rooted relation (potentially via intermediate members).

Goal round-trip: what Lean elaborated

```
theorem finite_path_connected_union_rooted_relation.extracted_1.1.{u_2, u_1} {X : Type u_1} {ι : Type u_2}
[inst : TopologicalSpace X] (A : ι → Set X) (root : ι) (R : ι → ι → Prop) :
(∀ (i : ι), i ≠ root → R i root) →
(∀ (i j : ι), R i j → (A i ∩ A j).Nonempty) →
(∀ (i : ι), IsPathConnected (A i)) → IsPathConnected (U i, A i)
```

Read back by a model that saw only the Lean. Let X be a topological space, let ι be an index type, let A assign to each $i \in \iota$ a subset $A i$ of X , let $\text{root} \in \iota$, and let R be a relation on ι . Assume that for every $i \in \iota$ with $i \neq \text{root}$, $R i \text{root}$ holds; whenever $R i j$ holds, the intersection $A i \cap A j$ is nonempty; and each $A i$ is path-connected. Then the union $\bigcup i, A i$ is path-connected.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace
open scoped Topology

theorem finite_path_connected_union_rooted_relation {X ι : Type*} [TopologicalSpace X] [Fintype ι] (A : ι → Set X) (root : ι) (R : ι → ι
  → Prop) :
  (∀ i, i ≠ root → R i root) →
  (∀ i j, R i j → (A i ∩ A j).Nonempty) →
  (∀ i, IsPathConnected (A i)) →
  IsPathConnected (U i, A i) := by
  intro hroot hattach hpath
  classical
  have aux : ∀ s : Finset ι,
    IsPathConnected (A root ∪ U i ∈ s, A i) := by
    intro s
    induction s using Finset.induction_on with
    | empty =>
      simpa using hpath root
    | @insert a s ha ih =>
      by_cases har : a = root
      · subst a
        have heq : A root ∪ U i ∈ insert root s, A i =
          A root ∪ U i ∈ s, A i := by
          ext x
          simp [ha]
          rw [heq]
          exact ih
      · have hinter : (A a ∩ (A root ∪ U i ∈ s, A i)).Nonempty := by
          rcases hattach a root (hroot a har) with ⟨x, hxa, hxr⟩
          exact ⟨x, hxa, Or.inl hxr⟩
        have hu : IsPathConnected (A a ∪ (A root ∪ U i ∈ s, A i)) :=
          (hpath a).union ih hinter
        have heq : A root ∪ U i ∈ insert a s, A i =
          A a ∪ (A root ∪ U i ∈ s, A i) := by
          ext x
          simp [Finset.mem_insert, ha, or_assoc, or_left_comm, or_comm]
          rw [heq]
```

```

      exact hu
have h := aux Finset.univ
have heq : A root U U i, A i = U i, A i := by
  ext x
  constructor
  · intro hx
    rcases hx with hx | hx
    · exact mem_iUnion.2 ⟨root, hx⟩
    · exact hx
  · intro hx
    exact Or.inr hx
simpa using heq ▸ (by simpa using h)

```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Munkres_exercise_23_2__me.me.me__ba621362

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propept, Classical.choice, Quot.sound, fingerprint ba621362ac60aafd

4.89 089. finite_connected_union_predecessor_preconnected

The problem

Let $A_0, A_1, \dots, A_{n-1}$ be preconnected subspaces of a topological space, with $n > 0$ and A_0 nonempty. If each A_k for $k > 0$ meets the union of its predecessors, prove that their union is connected.

Lean statement

```
theorem finite_connected_union_predecessor_preconnected {X : Type*} [TopologicalSpace X]
  (A : ℕ → Set X) (n : ℕ) :
  0 < n →
  (A 0).Nonempty →
  (∀ k < n, IsPreconnected (A k)) →
  (∀ k, 0 < k → k < n →
    (A k ∩ ⋃ j ∈ Finset.range k, A j).Nonempty) →
  IsConnected (⋃ k ∈ Finset.range n, A k)
```

Parent. Let $A_0, A_1, \dots, A_{N-1}$ be connected subspaces of a topological space, where $N > 0$. If each A_n with $n > 0$ meets the union of the preceding subspaces, prove that their union is connected.

Parent in Lean

```
theorem bounded_finite_predecessor_connected_union {X : Type*} [TopologicalSpace X] {A : ℕ → Set X}
  {N : ℕ} (hN : 0 < N) (hA : ∀ n, IsConnected (A n))
  (hPred : ∀ n, n < N → n ≠ 0 → (A n ∩ ⋃ k < n, A k).Nonempty) :
  IsConnected (⋃ k < N, A k)
```

What it contributes. Retains the finite predecessor-attachment decomposition and induction, while replacing stage connectedness with preconnectedness and adding a nonempty base stage.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child genuinely weakens each finite-stage hypothesis from connectedness to preconnectedness and separately supplies nonemptiness of the base stage. Its proof must establish nonemptiness and use preconnected union reasoning, so recalling the parent's connected-union proof is insufficient.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely weakens each piece from connectedness to preconnectedness and adds a base nonemptiness condition, requiring separate nonemptiness propagation and a preconnected-union argument. The parent theorem cannot be recalled directly because its connectedness hypotheses are unavailable.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely replaces connectedness of each piece with preconnectedness and supplies nonemptiness only at the base stage. The proof must reconstruct connectedness from nonemptiness plus preconnectedness and use the preconnected union lemma, so recalling the parent's connected-union argument is insufficient.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable

Check	By	Result
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2839 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_connected_union_predecessor_preconnected.extracted_1_1.{u_1} {X : Type u_1} [inst : TopologicalSpace X]
  (A : ℕ → Set X) (n : ℕ) :
  0 < n →
  (A 0).Nonempty →
  (∀ k < n, IsPreconnected (A k)) →
  (∀ (k : ℕ), 0 < k → k < n → (A k ∩ ⋃ j ∈ Finset.range k, A j).Nonempty) →
  IsConnected (⋃ k ∈ Finset.range n, A k)
```

Read back by a model that saw only the Lean. *Let X be a topological space, let A be a sequence of subsets of X indexed by the natural numbers, and let n be a natural number. If n is positive, A_0 is nonempty, every A_k with $k < n$ is preconnected, and for every natural number k with $0 < k < n$, the set A_k intersects the union of the sets A_j for $j < k$, then the union of the sets A_k for $k < n$ is connected.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace
open scoped Topology

theorem finite_connected_union_predecessor_preconnected {X : Type*} [TopologicalSpace X]
  (A : ℕ → Set X) (n : ℕ) :
  0 < n →
  (A 0).Nonempty →
  (∀ k < n, IsPreconnected (A k)) →
  (∀ k, 0 < k → k < n →
    (A k ∩ ⋃ j ∈ Finset.range k, A j).Nonempty) →
  IsConnected (⋃ k ∈ Finset.range n, A k) := by
  induction n with
  | zero =>
    intro hn
    omega
  | succ n ih =>
    intro hn hbase hpre hattach
    by_cases hzero : n = 0
    · subst n
      have hne : (⋃ k ∈ Finset.range 1, A k).Nonempty := by
        simpa [Finset.mem_range] using hbase
      have hpc : IsPreconnected (⋃ k ∈ Finset.range 1, A k) := by
        simpa [Finset.mem_range] using hpre 0 (by omega)
      exact ⟨hne, hpc⟩
    · have hnpos : 0 < n := Nat.pos_of_ne_zero hzero
      have hprev : IsConnected (⋃ k ∈ Finset.range n, A k) := by
        apply ih
        · exact hnpos
        · exact hbase
        · intro k hk
          exact hpre k (by omega)
        · intro k hkpos hkl1
          exact hattach k hkpos (by omega)
      have hlast : IsPreconnected (A n) := hpre n (by omega)
      have hinter : ((⋃ k ∈ Finset.range n, A k) ∩ A n).Nonempty := by
        simpa [Set.inter_comm] using hattach n (by omega) (by omega)
      have hunion : IsPreconnected ((⋃ k ∈ Finset.range n, A k) ∪ A n) :=
        IsPreconnected.union' hinter hprev.isPreconnected hlast
      have heq : (⋃ k ∈ Finset.range (Nat.succ n), A k) =
```

```

      (U k ∈ Finset.range n, A k) ∪ A n := by
      simp only [Finset.mem_range]
      exact Set.biUnion_lt_succ A n
    have hfull : ((U k ∈ Finset.range n, A k) ∪ A n).Nonempty := by
      rcases hprev.nonempty with ⟨x, hx⟩
      exact ⟨x, Or.inl hx⟩
    rw [heq]
    exact ⟨hfull, hunion⟩

```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Munkres_exercise_23_2__me.me__3e7b61e1

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 3e7b61e100165ae6

4.90 090. connected_preimage_selected_finite_order_locus

The problem

Let $p : X \rightarrow Y$ be a quotient map with connected fibers. For a finite selected family of continuous real-valued functions f_i, g_i on Y , suppose the set of points satisfying $f_i(y) \leq g_i(y)$ for every selected index i is connected. Prove that its inverse image under p is connected.

Lean statement

```
theorem connected_preimage_selected_finite_order_locus {X Y I : Type*} [TopologicalSpace X] [TopologicalSpace Y] (p : X → Y) (hq :
  Topology.IsQuotientMap p) (s : Finset I) (f g : I → Y → ℝ) (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i)) (hc :
  IsConnected {y : Y | ∀ i ∈ s, f i y ≤ g i y}) (hfib : ∀ y : Y, IsConnected (p ⁻¹' ({y} : Set Y))) : IsConnected (p ⁻¹' {y : Y | ∀ i ∈
  s, f i y ≤ g i y})
```

Parent. Let $p : X \rightarrow Y$ be a quotient map with connected fibers. For finitely many continuous real-valued functions f_i and g_i on Y , suppose the intersection of the loci $\{y | f_i(y) \leq g_i(y)\}$ is connected. Prove that its inverse image under p is connected.

Parent in Lean

```
theorem connected_preimage_finite_intersection_order_locus {X Y I : Type*} [TopologicalSpace X] [TopologicalSpace Y] [Fintype I]
  (p : X → Y) (hq : Topology.IsQuotientMap p)
  (f g : I → Y → ℝ)
  (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i))
  (hc : IsConnected (⋂ i : I, {y : Y | f i y ≤ g i y}))
  (hfib : ∀ y : Y, IsConnected (p ⁻¹' ({y} : Set Y))) :
  IsConnected (p ⁻¹' (⋂ i : I, {y : Y | f i y ≤ g i y}))
```

What it contributes. Retains the quotient-map, connected-fiber, continuous-inequality, closed-locus, and connected-preimage argument, while replacing the full index family by a selected finite subtype and explicitly consuming the point-wise/intersection representation.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the index structure from a globally finite type to an arbitrary finite selected subfamily, requiring a new subtype-indexed intersection representation before the quotient-map argument applies. The parent proof cannot be recalled verbatim because it lacks this representation and does not cover arbitrary I .

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes from an entire fintype-indexed family to an arbitrary finite selected subfamily, requiring a subtype-indexed intersection representation and new equivalence reasoning. A memorized proof of the parent does not directly solve the altered statement.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the index family from an entire finite type to an arbitrary selected finite subset, requiring a subtype intersection representation and transport of connectedness through that representation.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable

Check	By	Result
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2859 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem connected_preimage_selected_finite_order_locus.extracted_1_1.{u_3, u_2, u_1} {X : Type u_1} {Y : Type u_2}
{I : Type u_3} [inst : TopologicalSpace X] [inst_1 : TopologicalSpace Y] (p : X → Y) (hq : Topology.IsQuotientMap p)
(s : Finset I) (f g : I → Y → ℝ) (hf : ∀ (i : I), Continuous (f i)) (hg : ∀ (i : I), Continuous (g i))
(hC : IsConnected {y | ∀ i ∈ s, f i y ≤ g i y}) (hfib : ∀ (y : Y), IsConnected (p ⁻¹' {y})) :
IsConnected (p ⁻¹' {y | ∀ i ∈ s, f i y ≤ g i y})
```

Read back by a model that saw only the Lean. Let X, Y , and I be types, with topological space structures on X and Y . Let $p : X \rightarrow Y$ be a quotient map. Let s be a finite subset of I , and let $f, g : I \rightarrow Y \rightarrow \mathbb{R}$ be families of functions such that, for every $i \in I$, both $f(i)$ and $g(i)$ are continuous. Suppose the subset $\{y \in Y \mid \text{for every } i \in s, f(i)(y) \leq g(i)(y)\}$ is connected, and suppose that for every $y \in Y$, the fiber $p^{-1}(\{y\})$ is connected. Then the preimage under p of this subset, namely $\{x \in X \mid \text{for every } i \in s, f(i)(p(x)) \leq g(i)(p(x))\}$, is connected.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem connected_preimage_selected_finite_order_locus {X Y I : Type*} [TopologicalSpace X] [TopologicalSpace Y] (p : X → Y) (hq :
  Topology.IsQuotientMap p) (s : Finset I) (f g : I → Y → ℝ) (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i)) (hC :
  IsConnected {y : Y | ∀ i ∈ s, f i y ≤ g i y}) (hfib : ∀ y : Y, IsConnected (p ⁻¹' {y} : Set Y)) : IsConnected (p ⁻¹' {y : Y | ∀ i ∈
  s, f i y ≤ g i y}) := by
  have hrepr : {y : Y | ∀ i ∈ s, f i y ≤ g i y} = ⋂ i : {i // i ∈ s}, {y : Y | f i.1 y ≤ g i.1 y} := by
    ext y
    simp only [Set.mem_setOf_eq, Set.mem_iInter]
  constructor
  · intro hy i
    exact hy i.1 i.2
  · intro hy i hi
    exact hy ⟨i, hi⟩
  have hclosed_i : ∀ i : {i // i ∈ s}, IsClosed {y : Y | f i.1 y ≤ g i.1 y} := by
    intro i
    have hcont : Continuous (fun y : Y => f i.1 y - g i.1 y) := (hf i.1).sub (hg i.1)
    have heq : {y : Y | f i.1 y ≤ g i.1 y} = (fun y : Y => f i.1 y - g i.1 y) ⁻¹' Set.Iic 0 := by
      ext y
      simp [sub_nonpos]
    rw [heq]
    exact isClosed_Iic.preimage hcont
  have hclosed : IsClosed (⋂ i : {i // i ∈ s}, {y : Y | f i.1 y ≤ g i.1 y}) := by
    exact isClosed_iInter hclosed_i
  have hc' : IsConnected (⋂ i : {i // i ∈ s}, {y : Y | f i.1 y ≤ g i.1 y}) := by
    rw [← hrepr]
    exact hc
  have hresult : IsConnected (p ⁻¹' (⋂ i : {i // i ∈ s}, {y : Y | f i.1 y ≤ g i.1 y})) := by
    exact hq.isCoinducing.isConnected_preimage_of_isClosed hfib hclosed hc'
  have hfinal : IsConnected (p ⁻¹' {y : Y | ∀ i ∈ s, f i y ≤ g i y}) := by
    rw [hrepr]
    exact hresult
  exact hfinal
```

Operator. mutation / mutation_easy **Lineage depth.** 4 **Certificate.** reproducible

Problem id. Munkres_exercise_23_2__me.me__5097ab74__x__Rudin_exercise_4_3__me.mh.xe.ms.me__d0014abf

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint d0014abfd8565d9b

4.91 091. finite_connected_union_rooted

The problem

Let $A_0, \dots, A_{n-1}$ be connected subspaces of X . Suppose each A_k with $k > 0$ meets an earlier set $A_{r(k)}$, where $r(k) < k$. Show that their union is connected.

Lean statement

```
theorem finite_connected_union_rooted {X : Type*} [TopologicalSpace X]
  (A : ℕ → Set X) (n : ℕ) (r : ℕ → ℕ) :
  0 < n →
  (∀ k < n, IsConnected (A k)) →
  (∀ k, 0 < k → k < n → r k < k ∧ (A k ∩ A (r k)).Nonempty) →
  IsConnected (⋃ k ∈ Finset.range n, A k)
```

Parent. Let $A_0, \dots, A_{n-1}$ be connected subspaces of X , with $n > 0$. Suppose that for every $1 \leq k < n$, the set A_k meets the union of its predecessors. Show that $\bigcup_{k < n} A_k$ is connected.

Parent in Lean

```
theorem finite_connected_union_predecessor {X : Type*} [TopologicalSpace X]
  (A : ℕ → Set X) (n : ℕ) :
  0 < n →
  (∀ k < n, IsConnected (A k)) →
  (∀ k, 0 < k → k < n →
    (A k ∩ ⋃ j ∈ Finset.range k, A j).Nonempty) →
  IsConnected (⋃ k ∈ Finset.range n, A k)
```

What it contributes. Provides the finite-union induction invariant and connected-union assembly pattern; the predecessor-range attachment is replaced by the explicit rooted map r .

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child replaces intersection with the entire predecessor union by an explicit rooted attachment map and must convert the chosen root intersection into membership in the finite union. This is not a forced wrapper or decoration; the new predecessor-map reasoning is absent from the parent proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces intersection with the entire predecessor union by an explicit rooted attachment map and must prove that the chosen root lies in the finite union. This requires a new membership construction and is not recoverable by simply recalling the parent's proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces attachment to the entire predecessor union with an explicit rooted attachment map, and its proof must convert the root intersection into membership in the finite union using the bound $r(k) < k$. This is genuine new reasoning beyond the parent's attachment hypothesis, not a forced wrapper or decoration.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable

Check	By	Result
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2254 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_connected_union_rooted.extracted_1_1.{u_1} {X : Type u_1} [inst : TopologicalSpace X] (A : ℕ → Set X)
  (n : ℕ) (r : ℕ → ℕ) :
  0 < n →
  (∀ k < n, IsConnected (A k)) →
  (∀ (k : ℕ), 0 < k → k < n → r k < k ∧ (A k ∩ A (r k)).Nonempty) → IsConnected (⋃ k ∈ Finset.range n, A k)
```

Read back by a model that saw only the Lean. *Let X be a topological space, let A be a family of subsets of X indexed by the natural numbers, let n be a natural number, and let r be a function from the natural numbers to the natural numbers. If n is positive, every set A_k with $k < n$ is connected, and for every natural number k with $0 < k < n$, one has $r(k) < k$ and the intersection $A_k \cap A_{r(k)}$ is nonempty, then the union of the sets A_k for $k < n$ is connected.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem finite_connected_union_rooted {X : Type*} [TopologicalSpace X]
  (A : ℕ → Set X) (n : ℕ) (r : ℕ → ℕ) :
  0 < n →
  (∀ k < n, IsConnected (A k)) →
  (∀ k, 0 < k → k < n → r k < k ∧ (A k ∩ A (r k)).Nonempty) →
  IsConnected (⋃ k ∈ Finset.range n, A k) := by
  induction n with
  | zero =>
    intro hn
    omega
  | succ n ih =>
    intro hn hconn hattach
    by_cases hzero : n = 0
    · subst n
      simpa [Finset.mem_range] using hconn 0 (by omega)
    · have hnpos : 0 < n := Nat.pos_of_ne_zero hzero
      have hprev : IsConnected (⋃ k ∈ Finset.range n, A k) := by
        apply ih
        · exact hnpos
        · intro k hk
          exact hconn k (by omega)
        · intro k hkpos hkl
          exact hattach k hkpos (by omega)
      have hlast : IsConnected (A n) := hconn n (by omega)
      have hroot : r n < n ∧ (A n ∩ A (r n)).Nonempty :=
        hattach n (by omega) (by omega)
      have hinter : ((⋃ k ∈ Finset.range n, A k) ∩ A n).Nonempty := by
        rcases hroot.2 with ⟨x, hxlast, hxroot⟩
        refine ⟨x, ?_, hxlast⟩
        simp only [Set.mem_iUnion]
        exact ⟨r n, ⟨by simpa [Finset.mem_range] using hroot.1, hxroot⟩⟩
      have hunion : IsConnected ((⋃ k ∈ Finset.range n, A k) ∪ A n) :=
        IsConnected.union hinter hprev hlast
      have heq : (⋃ k ∈ Finset.range (Nat.succ n), A k) =
        (⋃ k ∈ Finset.range n, A k) ∪ A n := by
        simp only [Finset.mem_range]
        exact Set.biUnion_lt_succ A n
      simpa only [heq] using hunion
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Munkres_exercise_23_2__me.me__9f6221c7

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 9f6221c77d66fd83

4.92 092. cumulative_overlap_connected_union

The problem

Let $\{A_n\}$ be a sequence of connected subspaces of a topological space X . Suppose that $A_{n+1} \cap \bigcup_{k \leq n} A_k \neq \emptyset$ for every n . Show that $\bigcup_n A_n$ is connected.

Lean statement

```
theorem cumulative_overlap_connected_union {X : Type*} [TopologicalSpace X] {A : ℕ → Set X}
  (hA : ∀ n, IsConnected (A n))
  (hCum : ∀ n, (⋃ k ≤ n, A k) ∩ A (n + 1) ≠ ∅) :
  IsConnected (⋃ n, A n)
```

Parent. Let $\{A_n\}$ be a sequence of connected subspaces of X , such that $A_n \cap A_{n+1} \neq \emptyset$ for all n . Show that $\bigcup A_n$ is connected.

Parent in Lean

```
theorem Munkres_exercise_23_2 {X : Type*}
  [TopologicalSpace X] {A : ℕ → Set X} (hA : ∀ n, IsConnected (A n))
  (hAn : ∀ n, A n ∩ A (n + 1) ≠ ∅) :
  IsConnected (⋃ n, A n) := by
```

What it contributes. Retains the parent’s connected-union strategy and `IsConnected.iUnion_of_chain` infrastructure, while replacing adjacent-set overlap with an initial-union invariant and cumulative-overlap induction.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely weakens consecutive overlap to cumulative overlap and must inductively prove connectedness of each finite initial union before applying exhaustion. The parent’s adjacency-graph argument does not supply this finite-union induction or its cumulative-to-consecutive reduction.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely replaces pairwise consecutive overlap with cumulative overlap and proves connectedness through the new bounded-union sequence B , an induction, and an exhaustion equality. This proof structure is not supplied by recalling the parent’s direct intersection-chain argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely weakens consecutive overlap to cumulative overlap and proves connectedness by induction on bounded unions, followed by an explicit exhaustion/equality argument. This requires new union decomposition and connected-chain reasoning rather than recalling the parent’s adjacent-intersection proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1942 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem cumulative_overlap_connected_union.extracted_1.1.{u_1} {X : Type u_1} [inst : TopologicalSpace X]
  {A : ℕ → Set X} (hA : ∀ (n : ℕ), IsConnected (A n)) (hCum : ∀ (n : ℕ), (U k, U (_ : k ≤ n), A k) ∩ A (n + 1) ≠ ∅) :
  IsConnected (U n, A n)
```

Read back by a model that saw only the Lean. *Let X be a topological space, and let $(A_n)_{n \in \mathbb{N}}$ be a sequence of connected subsets of X . Suppose that for every $n \in \mathbb{N}$, the union of A_k over all $k \leq n$ has nonempty intersection with A_{n+1} . Then the union $\bigcup_{n \in \mathbb{N}} A_n$ is connected.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem cumulative_overlap_connected_union {X : Type*} [TopologicalSpace X] {A : ℕ → Set X}
  (hA : ∀ n, IsConnected (A n))
  (hCum : ∀ n, (U k ≤ n, A k) ∩ A (n + 1) ≠ ∅) :
  IsConnected (U n, A n) := by
  classical
  let B : ℕ → Set X := fun n => U k ≤ n, A k
  have hStep : ∀ n, B (n + 1) = B n ∪ A (n + 1) := by
    intro n
    ext x
    simp only [B, mem_iUnion, mem_setOf_eq, mem_union]
  constructor
  · rintro ⟨k, hk, hx⟩
    by_cases hkn : k ≤ n
    · exact Or.inl ⟨k, hkn, hx⟩
    · have hkeq : k = n + 1 := by omega
      exact Or.inr (hkeq ▸ hx)
  · rintro (⟨k, hk, hx⟩ | hx)
    · exact ⟨k, Nat.le_trans hk (Nat.le_succ n), hx⟩
    · exact ⟨n + 1, le_rfl, hx⟩
  have hB : ∀ n, IsConnected (B n) := by
    intro n
    induction n with
    | zero =>
      simpa [B] using hA 0
    | succ n ih =>
      rw [hStep n]
      apply IsConnected.union
      · exact (Set.nonempty_iff_ne_empty).2 (hCum n)
      · exact ih
      · exact hA (n + 1)
  have hChain : ∀ n, (B n ∩ B (n + 1)).Nonempty := by
    intro n
    rw [hStep n]
    rcases (Set.nonempty_iff_ne_empty).2 (hCum n) with ⟨x, hxB, hxA⟩
    exact ⟨x, hxB, Or.inr hxA⟩
  have hAll : IsConnected (U n, B n) :=
    IsConnected.iUnion_of_chain hB (by
      intro n
      simpa [Nat.succ_eq_add_one] using hChain n)
  have hEq : (U n, B n) = U n, A n := by
    ext x
    simp only [B, mem_iUnion]
  constructor
  · rintro ⟨n, k, hk, hx⟩
    exact ⟨k, hx⟩
  · rintro ⟨k, hx⟩
    exact ⟨k, k, le_rfl, hx⟩
  rw [hEq] at hAll
```

exact hAll

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_23_2__me__03b7f6c0

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 88ca384e0f975c8c

4.93 093. finite_connected_union_predecessor

The problem

Let $A_0, \dots, A_{n-1}$ be connected subspaces of X , with $n > 0$. Suppose that for every $1 \leq k < n$, the set A_k meets the union of its predecessors. Show that $\bigcup_{k < n} A_k$ is connected.

Lean statement

```
theorem finite_connected_union_predecessor {X : Type*} [TopologicalSpace X]
  (A : ℕ → Set X) (n : ℕ) :
  0 < n →
  (∀ k < n, IsConnected (A k)) →
  (∀ k, 0 < k → k < n →
    (A k ∩ ⋃ j ∈ Finset.range k, A j).Nonempty) →
  IsConnected (⋃ k ∈ Finset.range n, A k)
```

Parent. Let $\{A_n\}$ be a sequence of connected subspaces of X , such that $A_n \cap A_{n+1} \neq \emptyset$ for all n . Show that $\bigcup A_n$ is connected.

Parent in Lean

```
theorem Munkres_exercise_23_2 {X : Type*}
  [TopologicalSpace X] {A : ℕ → Set X} (hA : ∀ n, IsConnected (A n))
  (hAn : ∀ n, A n ∩ A (n + 1) ≠ ∅) :
  IsConnected (⋃ n, A n) := by
```

What it contributes. Generalizes the connected-union argument from consecutive overlaps in an infinite sequence to a finite family whose each noninitial member meets the union of all predecessors.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child replaces an infinite chain with a finite family whose each new set intersects the entire predecessor union, requiring induction and a connected-union argument absent from the parent's consecutive-overlap proof. It is not recoverable by recalling the parent theorem.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the infinite consecutive-overlap argument with a finite induction in which each new set intersects the entire predecessor union. This requires a new connected-union induction and is not obtainable by recalling the parent's proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces consecutive overlaps in an infinite family with bounded attachment to the entire predecessor union and proves the result by finite induction. This requires a genuinely new induction-and-union argument not supplied by the parent's relation-chain proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing

Check	By	Result
corpus_dedup	hash	unique against 2237 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_connected_union_predecessor.extracted_1_1.{u_1} {X : Type u_1} [inst : TopologicalSpace X]
  (A : ℕ → Set X) (n : ℕ) :
  0 < n →
  (∀ k < n, IsConnected (A k)) →
  (∀ (k : ℕ), 0 < k → k < n → (A k ∩ ⋃ j ∈ Finset.range k, A j).Nonempty) →
  IsConnected (⋃ k ∈ Finset.range n, A k)
```

Read back by a model that saw only the Lean. *Let X be a topological space, let A be a sequence of subsets of X indexed by the natural numbers, and let n be a natural number. If n is positive, every set A_k with $k < n$ is connected, and for every natural number k with $0 < k < n$, the set A_k has a nonempty intersection with the union of the sets A_j for $j < k$, then the union of the sets A_k for $k < n$ is connected.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace
open scoped Topology

theorem finite_connected_union_predecessor {X : Type*} [TopologicalSpace X]
  (A : ℕ → Set X) (n : ℕ) :
  0 < n →
  (∀ k < n, IsConnected (A k)) →
  (∀ k, 0 < k → k < n →
    (A k ∩ ⋃ j ∈ Finset.range k, A j).Nonempty) →
  IsConnected (⋃ k ∈ Finset.range n, A k) := by
  induction n with
  | zero =>
    intro hn
    omega
  | succ n ih =>
    intro hn hconn hattach
    by_cases hzero : n = 0
    · subst n
      simpa [Finset.mem_range] using hconn 0 (by omega)
    · have hnpos : 0 < n := Nat.pos_of_ne_zero hzero
      have hprev : IsConnected (⋃ k ∈ Finset.range n, A k) := by
        apply ih
        · exact hnpos
        · intro k hk
          exact hconn k (by omega)
        · intro k hkpos hkl
          exact hattach k hkpos (by omega)
      have hlast : IsConnected (A n) := hconn n (by omega)
      have hinter : ((⋃ k ∈ Finset.range n, A k) ∩ A n).Nonempty := by
        simpa [Set.inter_comm] using hattach n (by omega) (by omega)
      have hunion : IsConnected ((⋃ k ∈ Finset.range n, A k) ∪ A n) :=
        IsConnected.union hinter hprev hlast
      have heq : (⋃ k ∈ Finset.range (Nat.succ n), A k) =
        (⋃ k ∈ Finset.range n, A k) ∪ A n := by
        simp only [Finset.mem_range]
        exact Set.biUnion_lt_succ A n
      rw [heq]
      exact hunion
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_23_2__me__2746d5bb

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 2746d5bb611de8d4

4.94 094. bounded_predecessor_connected_iUnion

The problem

Let (A_n) be connected subspaces of X . Suppose that for every $n > 0$, A_n intersects the union of its predecessors $\bigcup_{k < n} A_k$. Show that $\bigcup_n A_n$ is connected.

Lean statement

```
theorem bounded_predecessor_connected_iUnion {X : Type*} [TopologicalSpace X] {A : ℕ → Set X}
  (hA : ∀ n, IsConnected (A n))
  (hPred : ∀ n, n ≠ 0 → (A n ∩ ⋃ k < n, A k).Nonempty) :
  IsConnected (⋃ n, A n)
```

Parent. Let $\{A_n\}$ be a sequence of connected subspaces of X , such that $A_n \cap A_{n+1} \neq \emptyset$ for all n . Show that $\bigcup A_n$ is connected.

Parent in Lean

```
theorem Munkres_exercise_23.2 {X : Type*}
  [TopologicalSpace X] {A : ℕ → Set X} (hA : ∀ n, IsConnected (A n))
  (hAn : ∀ n, A n ∩ A (n + 1) ≠ ∅) :
  IsConnected (⋃ n, A n) := by
```

What it contributes. Retains the parent's connected indexed-union and overlap-relation framework, while replacing consecutive overlap with cumulative predecessor overlap and supplying a strong-induction bridge.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes consecutive overlap to intersection with the entire bounded predecessor union. Its proof requires strong induction and extracting an arbitrary predecessor index before chaining reachability, which is not supplied by the parent's successor-chain argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes consecutive overlap to intersection with any bounded predecessor union. Its proof requires strong induction to establish reachability and a new graph-connectivity argument, which the parent's adjacent-chain proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely replaces consecutive overlap with bounded-predecessor overlap and proves reachability by strong induction, introducing reasoning absent from the parent's successor-chain argument. The changed hypothesis and proof are mathematically substantive rather than decorative or recall-based.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2142 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_predecessor_connected_iUnion.extracted_1_1.{u_1} {X : Type u_1} [inst : TopologicalSpace X]
  {A : ℕ → Set X} (hA : ∀ (n : ℕ), IsConnected (A n))
  (hPred : ∀ (n : ℕ), n ≠ 0 → (A n ∩ ⋃ k, A k).Nonempty) : IsConnected (⋃ n, A n)
```

Read back by a model that saw only the Lean. *Let X be a topological space and let $(A_n)_{n \in \mathbb{N}}$ be a family of subsets of X . Assume that A_n is connected for every natural number n . Assume also that for every natural number $n \neq 0$, the intersection of A_n with the union of all A_k for natural numbers $k < n$ is nonempty. Then the union of all A_n over $n \in \mathbb{N}$ is connected.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem bounded_predecessor_connected_iUnion {X : Type*} [TopologicalSpace X] {A : ℕ → Set X}
  (hA : ∀ n, IsConnected (A n))
  (hPred : ∀ n, n ≠ 0 → (A n ∩ ⋃ k < n, A k).Nonempty) :
  IsConnected (⋃ n, A n) := by
  classical
  let R : ℕ → ℕ → Prop := fun i j => (A i ∩ A j).Nonempty
  have hReach : ∀ n, Relation.ReflTransGen R 0 n := by
    intro n
    induction n using Nat.strong_induction_on with
    | h n ih =>
      cases n with
      | zero => exact Relation.ReflTransGen.refl
      | succ n =>
        obtain ⟨x, hxN, hxPred⟩ := hPred (n + 1) (by omega)
        rcases Set.mem_iUnion.mp hxPred with ⟨k, hxPred⟩
        rcases Set.mem_iUnion.mp hxPred with ⟨hk, hxk⟩
        have hlt : k < n + 1 := hk
        have hedge : R k (n + 1) := ⟨x, hxk, hxN⟩
        exact (ih k hlt).tail hedge
  have hSym : Symmetric R := by
    intro i j hij
    simpa [R, Set.inter_comm] using hij
  have hChain : ∀ i j : ℕ, Relation.ReflTransGen R i j := by
    intro i j
    exact (Relation.ReflTransGen.symmetric hSym (hReach i)).trans (hReach j)
  have hUnion : IsConnected (⋃ n, A n) := by
    simpa [R] using IsConnected.iUnion_of_reflTransGen (s := A) (H := hA) (K := hChain)
  exact hUnion
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_23_2__me__7edaafef

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 7edaafef58792997

4.95 095. bounded_finite_predecessor_connected_union

The problem

Let $A_0, A_1, \dots, A_{N-1}$ be connected subspaces of a topological space, where $N > 0$. If each A_n with $n > 0$ meets the union of the preceding subspaces, prove that their union is connected.

Lean statement

```
theorem bounded_finite_predecessor_connected_union {X : Type*} [TopologicalSpace X] {A : ℕ → Set X}
  {N : ℕ} (hN : 0 < N) (hA : ∀ n, IsConnected (A n))
  (hPred : ∀ n, n < N → n ≠ 0 → (A n ∩ ⋃ k < n, A k).Nonempty) :
  IsConnected (⋃ k < N, A k)
```

Parent. Let $\{A_n\}$ be a sequence of connected subspaces of X , such that $A_n \cap A_{n+1} \neq \emptyset$ for all n . Show that $\bigcup A_n$ is connected.

Parent in Lean

```
theorem Munkres_exercise_23_2 {X : Type*}
  [TopologicalSpace X] {A : ℕ → Set X} (hA : ∀ n, IsConnected (A n))
  (hAn : ∀ n, A n ∩ A (n + 1) ≠ ∅) :
  IsConnected (⋃ n, A n) := by
```

What it contributes. Supplies the connected-union topology argument and the `IsConnected.union` checkpoint; the mutation replaces consecutive overlap with finite predecessor-union attachment.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child is a genuine finite ordered-attachment generalization: each later set may intersect any earlier union rather than only its immediate predecessor, so the parent's successor-chain argument does not apply. Its induction must establish connectedness of successive finite unions and prove the corresponding union decomposition.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces adjacent overlap on an infinite sequence with predecessor-overlap for a finite ordered family. Its proof requires induction on finite unions and explicitly rewriting the union decomposition, reasoning not supplied by the parent's relation-chain argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely replaces infinite adjacent overlaps with a finite ordered attachment scheme: each noninitial set must meet the entire earlier union. Its induction must establish connectedness incrementally and prove the finite-union decomposition, which the parent's successor-chain argument does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2830 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_finite_predecessor_connected_union.extracted_1.1.{u_1} {X : Type u_1} [inst : TopologicalSpace X]
  {A : ℕ → Set X} {N : ℕ} (hN : 0 < N) (hA : ∀ (n : ℕ), IsConnected (A n))
  (hPred : ∀ n < N, n ≠ 0 → (A n ∩ U k, U (_ : k < n), A k).Nonempty) : IsConnected (U k, U (_ : k < N), A k)
```

Read back by a model that saw only the Lean. *Let X be a topological space, let $(A_n)_{n \in \mathbb{N}}$ be a family of subsets of X , and let $N \in \mathbb{N}$ with $N > 0$. Suppose that every A_n is connected, and that for every $n < N$ with $n \neq 0$, the set A_n intersects the union of all A_k with $k < n$. Then the union of all A_k with $k < N$ is connected.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace
open scoped Topology

theorem bounded_finite_predecessor_connected_union {X : Type*} [TopologicalSpace X] {A : ℕ → Set X}
  {N : ℕ} (hN : 0 < N) (hA : ∀ n, IsConnected (A n))
  (hPred : ∀ n, n < N → n ≠ 0 → (A n ∩ U k < n, A k).Nonempty) :
  IsConnected (U k < N, A k) := by
  classical
  have hUnion : ∀ m, 0 < m → m ≤ N → IsConnected (U k < m, A k) := by
    intro m
    induction m with
    | zero =>
      intro hm hle
      omega
    | succ n ih =>
      intro hm hle
      by_cases hn : n = 0
      · subst n
        simpa using hA 0
      · have hPrev : IsConnected (U k < n, A k) := ih (by omega) (by omega)
        have hOverlap : ((U k < n, A k) ∩ A n).Nonempty := by
          simpa [Set.inter_comm] using hPred n (by omega) hn
        have hStep : IsConnected ((U k < n, A k) ∪ A n) :=
          IsConnected.union hOverlap hPrev (hA n)
        have hEq : (U k < n + 1, A k) = (U k < n, A k) ∪ A n := by
          ext x
          simp only [Set.mem_iUnion, Set.mem_union]
        constructor
        · rintro ⟨k, hk, hx⟩
          by_cases hkn : k < n
          · exact Or.inl ⟨k, hkn, hx⟩
          · have hkeq : k = n := by omega
            exact Or.inr (hkeq ▸ hx)
        · rintro ⟨⟨k, hk, hx⟩ | hx⟩
          · exact ⟨k, by omega, hx⟩
          · exact ⟨n, by omega, hx⟩
      rw [hEq]
      exact hStep
  exact hUnion N hN le_refl
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_23.2__me__9a77ee70

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 9a77ee70d2ecef65

4.96 096. bounded_prefix_connected

The problem

Let X be a topological space and let $\{A_n\}$ be a sequence of connected subspaces of X such that $A_n \cap A_{n+1} \neq \emptyset$ for every n . Show that for every natural number N , the union $A_0 \cup A_1 \cup \dots \cup A_N$ is connected.

Lean statement

```
theorem bounded_prefix_connected {X : Type*}
  [TopologicalSpace X] {A : ℕ → Set X} (hA : ∀ n, IsConnected (A n))
  (hAn : ∀ n, A n ∩ A (n + 1) ≠ ∅) (N : ℕ) :
  IsConnected (⋃ n ∈ Finset.range (N + 1), A n)
```

Parent. Let $\{A_n\}$ be a sequence of connected subspaces of X , such that $A_n \cap A_{n+1} \neq \emptyset$ for all n . Show that $\bigcup A_n$ is connected.

Parent in Lean

```
theorem Munkres_exercise_23_2 {X : Type*}
  [TopologicalSpace X] {A : ℕ → Set X} (hA : ∀ n, IsConnected (A n))
  (hAn : ∀ n, A n ∩ A (n + 1) ≠ ∅) :
  IsConnected (⋃ n, A n) := by
```

What it contributes. Uses the parent's connected-set and consecutive-overlap structure, specialized to an explicit finite prefix and consumed through induction.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves connectedness for each finite prefix by induction, explicitly decomposing the prefix union and establishing overlap with the next set. This bounded exhaustion argument is mathematically distinct from the parent's direct connectedness proof for the full infinite union.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely new finite-prefix connectedness theorem by induction, constructing the prefix-to-next-set overlap and rewriting the finite union at each step. This reasoning is not supplied by the parent's full-union argument and is not a forced wrapper or decoration.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely new bounded-prefix theorem by induction, explicitly decomposing each finite union and establishing overlap with the next set. This reasoning is not supplied by recalling the parent's full-union connectedness proof, and it directly satisfies the planned bounded generalization.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1938 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_prefix_connected.extracted_1_1.{u_1} {X : Type u_1} [inst : TopologicalSpace X] {A : ℕ → Set X}
(hA : ∀ (n : ℕ), IsConnected (A n)) (hAn : ∀ (n : ℕ), A n ∩ A (n + 1) ≠ ∅) (N : ℕ) :
IsConnected (U n ∈ Finset.range (N + 1), A n)
```

Read back by a model that saw only the Lean. *Let X be a topological space and let A be a sequence of subsets of X indexed by the natural numbers. Suppose that each set $A(n)$ is connected, and that for every natural number n , the intersection $A(n) \cap A(n+1)$ is nonempty. Then, for every natural number N , the union of the sets $A(0), A(1), \dots, A(N)$ is connected.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace
open scoped Topology

theorem bounded_prefix_connected {X : Type*}
[TopologicalSpace X] {A : ℕ → Set X} (hA : ∀ n, IsConnected (A n))
(hAn : ∀ n, A n ∩ A (n + 1) ≠ ∅) (N : ℕ) :
IsConnected (U n ∈ Finset.range (N + 1), A n) := by
induction N with
| zero =>
  simpa using hA 0
| succ N ih =>
  change IsConnected (U n ∈ Finset.range (N + 2), A n)
  have hPrefix : A N ⊆ U n ∈ Finset.range (N + 1), A n := by
    intro x hx
    refine Set.mem_iUnion.2 ⟨N, ?_⟩
    refine Set.mem_iUnion.2 ⟨by simp, hx⟩
  have hOverlap :
    ((U n ∈ Finset.range (N + 1), A n) ∩ A (N + 1)).Nonempty := by
    have hAdj : (A N ∩ A (N + 1)).Nonempty :=
      (Set.nonempty_iff_ne_empty).2 (hAn N)
    rcases hAdj with ⟨x, hxN, hxNext⟩
    exact ⟨x, hPrefix hxN, hxNext⟩
  have hUnion :
    (U n ∈ Finset.range (N + 2), A n) =
      (U n ∈ Finset.range (N + 1), A n) ∪ A (N + 1) := by
    ext x
    constructor
    · intro hx
      rcases Set.mem_iUnion.1 hx with ⟨n, hx⟩
      rcases Set.mem_iUnion.1 hx with ⟨hn, hxn⟩
      by_cases h : n = N + 1
      · right
        simpa [h] using hxn
      · left
        refine Set.mem_iUnion.2 ⟨n, ?_⟩
        refine Set.mem_iUnion.2 ⟨?_, hxn⟩
        have hn' : n < N + 2 := Finset.mem_range.1 hn
        exact Finset.mem_range.2 (by omega)
    · intro hx
      rcases hx with hx | hx
      · rcases Set.mem_iUnion.1 hx with ⟨n, hx⟩
        rcases Set.mem_iUnion.1 hx with ⟨hn, hxn⟩
        refine Set.mem_iUnion.2 ⟨n, ?_⟩
        refine Set.mem_iUnion.2 ⟨?_, hxn⟩
        have hn' : n < N + 1 := Finset.mem_range.1 hn
        exact Finset.mem_range.2 (by omega)
      · refine Set.mem_iUnion.2 ⟨N + 1, ?_⟩
        refine Set.mem_iUnion.2 ⟨?_, hx⟩
```

```
exact Finset.mem_range.2 (by omega)
have hConnected := IsConnected.union hOverlap ih (hA (N + 1))
rw [hUnion]
exact hConnected
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_23_2__me__9f0e11a1

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint ed46bedda43616df

4.97 097. two_sided_integer_connected_union

The problem

Let X be a topological space and let $\{B_z\}_{z \in \mathbb{Z}}$ be a family of connected subspaces of X such that $B_z \cap B_{z+1} \neq \emptyset$ for every $z \in \mathbb{Z}$. Show that $\bigcup_{z \in \mathbb{Z}} B_z$ is connected.

Lean statement

```
theorem two_sided_integer_connected_union {X : Type*} [TopologicalSpace X]
  {B : ℤ → Set X} (hB : ∀ z, IsConnected (B z))
  (hAdj : ∀ z : ℤ, (B z ∩ B (z + 1)).Nonempty) :
  IsConnected (⋃ z : ℤ, B z)
```

Parent. Let $\{A_n\}$ be a sequence of connected subspaces of X , such that $A_n \cap A_{n+1} \neq \emptyset$ for all n . Show that $\bigcup A_n$ is connected.

Parent in Lean

```
theorem Munkres_exercise_23.2 {X : Type*}
  [TopologicalSpace X] {A : ℕ → Set X} (hA : ∀ n, IsConnected (A n))
  (hAn : ∀ n, A n ∩ A (n + 1) ≠ ∅) :
  IsConnected (⋃ n, A n) := by
```

What it contributes. Reuse the connected-chain union strategy on each of the two $\mathbb{N}$ -indexed rays, while adding an explicit integer split and zero gluing step.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the one-sided natural chain to a two-sided integer-indexed chain. Its proof requires splitting positive and negative indices into two connected rays, proving their intersection is nonempty, and establishing the union reindexing equality before gluing them.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires a genuinely new two-sided integer reindexing: it decomposes the chain into positive and negative natural rays, proves each ray connected, and glues them across the $-1/0$ intersection. This reasoning is not supplied by the parent's one-sided chain proof and is not a forced-witness or decorative wrapper.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the one-sided natural chain to a two-sided integer-indexed chain, requiring decomposition into positive and negative rays, reindexing, and a nontrivial gluing argument. This reasoning is not supplied by recalling the parent's natural-number union proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1935 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem two_sided_integer_connected_union.extracted_1_1.{u_1} {X : Type u_1} [inst : TopologicalSpace X] {B : ℤ → Set X}
  (hB : ∀ (z : ℤ), IsConnected (B z)) (hAdj : ∀ (z : ℤ), (B z ∩ B (z + 1)).Nonempty) : IsConnected (U z, B z)
```

Read back by a model that saw only the Lean. *Let X be a topological space and let B be a family of subsets of X indexed by the integers. Suppose that for every integer z , the set $B(z)$ is connected, and that for every integer z , the intersection $B(z) \cap B(z + 1)$ is nonempty. Then the union $\bigcup_{z \in \mathbb{Z}} B(z)$ is connected.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace
open scoped Topology

theorem two_sided_integer_connected_union {X : Type*} [TopologicalSpace X]
  {B : ℤ → Set X} (hB : ∀ z, IsConnected (B z))
  (hAdj : ∀ z : ℤ, (B z ∩ B (z + 1)).Nonempty) :
  IsConnected (U z : ℤ, B z) := by
  classical
  let P : ℕ → Set X := fun n => B (n : ℤ)
  let N : ℕ → Set X := fun n => B (-(n : ℤ) - 1)
  have hPconn : ∀ n, IsConnected (P n) := by
    intro n
    simpa [P] using hB (n : ℤ)
  have hNconn : ∀ n, IsConnected (N n) := by
    intro n
    simpa [N] using hB (-(n : ℤ) - 1)
  have hPedge : ∀ n, (P n ∩ P (n + 1)).Nonempty := by
    intro n
    simpa [P] using hAdj (n : ℤ)
  have hNedge : ∀ n, (N n ∩ N (n + 1)).Nonempty := by
    intro n
    have h := hAdj (-(n : ℤ) - 2)
    simpa [N, Set.inter_comm, sub_eq_add_neg, add_assoc, add_comm, add_left_comm] using h
  have hPos : IsConnected (U n : ℕ, P n) := by
    exact IsConnected.iUnion_of_chain hPconn hPedge
  have hNeg : IsConnected (U n : ℕ, N n) := by
    exact IsConnected.iUnion_of_chain hNconn hNedge
  have hGluue : ((U n : ℕ, P n) ∩ (U n : ℕ, N n)).Nonempty := by
    obtain ⟨x, hx⟩ := hAdj (-1)
    rcases hx with ⟨hxneg, hxzero⟩
    refine ⟨x, Set.mem_iUnion.2 ⟨0, ?_⟩, Set.mem_iUnion.2 ⟨0, ?_⟩⟩
    · simpa [P] using hxzero
    · simpa [N] using hxneg
  have hRayUnion : IsConnected ((U n : ℕ, P n) ∪ (U n : ℕ, N n)) := by
    exact IsConnected.union hGluue hPos hNeg
  have hEq : (U z : ℤ, B z) = (U n : ℕ, P n) ∪ (U n : ℕ, N n) := by
    ext x
    constructor
    · intro hx
      rcases Set.mem_iUnion.1 hx with ⟨z, hxz⟩
      cases z with
      | ofNat n =>
        exact Or.inl (Set.mem_iUnion.2 ⟨n, by simpa [P] using hxz⟩)
      | negSucc n =>
        have hz : (Int.negSucc n : ℤ) = -(n : ℤ) - 1 := by omega
        apply Or.inr
        refine Set.mem_iUnion.2 ⟨n, ?_⟩
        change x ∈ B (-(n : ℤ) - 1)
        rw [← hz]
        exact hxz
    · intro hx
```

```

rcases hx with hx | hx
· rcases Set.mem_iUnion.1 hx with ⟨n, hxn⟩
  exact Set.mem_iUnion.2 ⟨(n : ℤ), by simp [P] using hxn⟩
· rcases Set.mem_iUnion.1 hx with ⟨n, hxn⟩
  exact Set.mem_iUnion.2 ⟨-(n : ℤ) - 1, by simp [N] using hxn⟩
rw [hEq]
exact hRayUnion

```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_23_2__me__b8d24dc9

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 813e5feaffbc950b

4.98 098. parameterized_closed_interval_fixed_point

The problem

Let $a, b \in \mathbb{R}$ with $a < b$, and let $f : \mathbb{R} \rightarrow \mathbb{R}$ be continuous and map $[a, b]$ into itself. Show that there exists $t \in [0, 1]$ such that $\frac{f(a+(b-a)t)-a}{b-a} = t$ and $f(a + (b - a)t) = a + (b - a)t$.

Lean statement

```
theorem parameterized_closed_interval_fixed_point (a b : ℝ) (hab : a < b) (f : ℝ → ℝ) (hf : Continuous f) (hmaps : MapsTo f (Set.Icc a b)
  ↪ (Set.Icc a b)) : ∃ t ∈ Set.Icc 0 1, (f (a + (b - a) * t) - a) / (b - a) = t ∧ f (a + (b - a) * t) = a + (b - a) * t
```

Parent. Let $f : X \rightarrow X$ be continuous. Show that if $X = [0, 1]$, there is a point x such that $f(x) = x$. (The point x is called a fixed point of f .)

Parent in Lean

```
theorem Munkres_exercise_24_3a (f : I → I) (hf : Continuous f) :
  ∃ (x : I), f x = x := by
```

What it contributes. Retains the parent's continuous self-map fixed-point structure, while replacing the fixed unit interval with an explicitly parameterized interval and proving the affine transfer obligations.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the fixed-point result to an arbitrary nondegenerate interval and requires constructing and proving properties of the affine transfer map and normalized self-map. The affine algebraic transfer back to a fixed point is new reasoning not supplied by the parent proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the interval and constructs an affine transfer map, then proves that its normalized fixed point yields a fixed point of f . The division by $b-a$ and interval-mapping argument require reasoning absent from the fixed $[0,1]$ parent instance.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the interval and transfers the self-map through the affine map $\varphi(t)=a+(b-a)t$, requiring new continuity, mapping, positivity, and division reasoning. Its proof uses a different real-valued fixed-point construction rather than recalling the subtype proof for $[0,1]$.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1860 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem parameterized_closed_interval_fixed_point.extracted_1_1 (a b : ℝ) (hab : a < b) (f : ℝ → ℝ) (hf : Continuous f)
  (hmaps : MapsTo f (Icc a b) (Icc a b)) :
  ∃ t ∈ Icc 0 1, (f (a + (b - a) * t) - a) / (b - a) = t ∧ f (a + (b - a) * t) = a + (b - a) * t
```

Read back by a model that saw only the Lean. For real numbers a and b with $a < b$, and a continuous function $f: \mathbb{R} \rightarrow \mathbb{R}$ that maps the closed interval $[a, b]$ into itself, there exists $t \in [0, 1]$ such that $(f(a + (b - a)t) - a)/(b - a) = t$ and $f(a + (b - a)t) = a + (b - a)t$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace
open scoped Topology

theorem parameterized_closed_interval_fixed_point (a b : ℝ) (hab : a < b) (f : ℝ → ℝ) (hf : Continuous f) (hmaps : MapsTo f (Set.Icc a b)
  ↪ (Set.Icc a b)) : ∃ t ∈ Set.Icc 0 1, (f (a + (b - a) * t) - a) / (b - a) = t ∧ f (a + (b - a) * t) = a + (b - a) * t := by
  let φ : ℝ → ℝ := fun t => a + (b - a) * t
  let g : ℝ → ℝ := fun t => (f (φ t) - a) / (b - a)
  have hφ_cont : Continuous φ := by
    dsimp [φ]
    fun_prop
  have hg_cont : Continuous g := by
    dsimp [g]
    fun_prop
  have hg_maps : MapsTo g (Set.Icc 0 1) (Set.Icc 0 1) := by
    intro t ht
    have hφ_mem : φ t ∈ Set.Icc a b := by
      dsimp [φ]
      constructor <|> nlinarith [ht.1, ht.2, hab]
    have hf_mem : f (φ t) ∈ Set.Icc a b := hmaps hφ_mem
    constructor
    · apply (le_div_iff₀ (sub_pos.mpr hab)).2
      nlinarith [hf_mem.1]
    · apply (div_le_iff₀ (sub_pos.mpr hab)).2
      nlinarith [hf_mem.2]
  obtain ⟨t, ht, hfixed⟩ := exists_mem_Icc_isFixedPt_of_mapsTo hg_cont.continuousOn (by norm_num) hg_maps
  have htransfer : (f (φ t) - a) / (b - a) = t := by
    simpa [g] using hfixed
  have htransfer' : f (φ t) - a = t * (b - a) := by
    calc
      f (φ t) - a = ((f (φ t) - a) / (b - a)) * (b - a) := by
        field_simp [ne_of_gt (sub_pos.mpr hab)]
      _ = t * (b - a) := by rw [htransfer]
  have hfixed_f : f (φ t) = φ t := by
    dsimp [φ]
    nlinarith [htransfer']
  refine ⟨t, ht, ?_, ?_⟩
  · simpa [φ] using htransfer
  · simpa [φ] using hfixed_f
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_24_3a__me__df176385

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 324ad392b04ea85f

4.99 099. localized_compact_trace_basis

The problem

Let K be a compact metric space, let A be a closed subset containing x , and let U be an open neighborhood of x . Show that some open neighborhood V of x has compact closure contained in U , and that $A \cap \overline{V}$ has a countable topological basis.

Lean statement

```
theorem localized_compact_trace_basis {K : Type*} [MetricSpace K] [CompactSpace K]
  (A : Set K) (hA : IsClosed A) (x : K)
  (U : Set K) (hU : IsOpen U) (hxU : x ∈ U) :
  ∃ (V : Set K) (B : Set (Set {y : K // y ∈ A ∩ closure V})),
    IsOpen V ∧ x ∈ V ∧ IsCompact (closure V) ∧ closure V ⊆ U ∧
    IsClosed (A ∩ closure V) ∧ Set.Countable B ∧
    TopologicalSpace.IsTopologicalBasis B
```

Parent. Show that if X is a Hausdorff space that is locally compact at the point x , then for each neighborhood U of x , there is a neighborhood V of x such that $\overline{V}$ is compact and $\overline{V} \subset U$.

Parent in Lean

```
theorem Munkres_exercise_29_10 {X : Type*}
  [TopologicalSpace X] [T2Space X] (x : X)
  (hx : ∃ U : Set X, x ∈ U ∧ IsOpen U ∧ (∃ K : Set X, U ⊆ K ∧ IsCompact K))
  (U : Set X) (hU : IsOpen U) (hxU : x ∈ U) :
  ∃ (V : Set X), IsOpen V ∧ x ∈ V ∧ IsCompact (closure V) ∧ closure V ⊆ U := by
```

What it contributes. Supplied the compact local-core proof skeleton: choose a neighborhood core, pass to an open interior, control its closure by a compact closed set, and retain containment in U .

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child requires proving compactness of the localized subtype and constructing a countable topological basis for it, obligations absent from the parent's compact-neighborhood construction. Its proof materially consumes the closed intersection and subtype compactness rather than merely reusing the parent's conclusion.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds genuinely new mathematics: it proves the localized intersection $A \cap \text{closure } V$ is closed and compact, then constructs a countable topological basis for its subtype. Its proof does not recall or reuse the parent's separation argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine compactness argument for the localized intersection $A \cap \overline{V}$: it proves closedness, derives compactness of the corresponding subtype, and constructs a countable topological basis. These obligations are absent from and cannot be discharged by recalling the parent's compact-neighborhood proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry

Check	By	Result
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2921 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	MISMATCH: Statement B replaces the conclusion that $A \cap \text{closure}(V)$ has a countable topological basis with the weaker assertion that $A \cap \text{closure}(V)$ is closed; its separate countable basis is for K , not explicitly for the intersection.

Goal round-trip: what Lean elaborated

```
theorem localized_compact_trace_basis.extracted_1.1.{u_1} {K : Type u_1} [inst : MetricSpace K]
  [inst_1 : CompactSpace K] (A : Set K) (hA : IsClosed A) (x : K) (hxA : x ∈ A) (U : Set K) (hU : IsOpen U)
  (hxU : x ∈ U) :
  ∃ V B,
    IsOpen V ∧
    x ∈ V ∧
    IsCompact (closure V) ∧ closure V ⊆ U ∧ IsClosed (A ∩ closure V) ∧ B.Countable ∧ IsTopologicalBasis B
```

Read back by a model that saw only the Lean. *Let K be a compact metric space, let A be a closed subset of K , let $x \in A$, and let U be an open subset of K containing x . Then there exist an open set $V \subseteq K$ and a countable topological basis B for K such that $x \in V$, the closure of V is compact, the closure of V is contained in U , and $A \cap \text{closure}(V)$ is closed.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem localized_compact_trace_basis {K : Type*} [MetricSpace K] [CompactSpace K]
  (A : Set K) (hA : IsClosed A) (x : K)
  (U : Set K) (hU : IsOpen U) (hxU : x ∈ U) :
  ∃ (V : Set K) (B : Set (Set {y : K // y ∈ A ∩ closure V})),
    IsOpen V ∧ x ∈ V ∧ IsCompact (closure V) ∧ closure V ⊆ U ∧
    IsClosed (A ∩ closure V) ∧ Set.Countable B ∧
    TopologicalSpace.IsTopologicalBasis B := by
  classical
  obtain ⟨C, hCcompact, hxC, hCU⟩ := exists_compact_subset hU hxU
  let V : Set K := interior C
  have hVopen : IsOpen V := isOpen_interior
  have hxV : x ∈ V := hxC
  have hclosureV : closure V ⊆ C :=
    closure_minimal interior_subset hCcompact.isClosed
  have hclosureCompact : IsCompact (closure V) :=
    hCcompact.of_isClosed_subset isClosed_closure hclosureV
  have hclosureU : closure V ⊆ U := hclosureV.trans hCU
  have hclosedTrace : IsClosed (A ∩ closure V) :=
    hA.inter isClosed_closure
  have hcompactTrace : IsCompact (A ∩ closure V) :=
    hclosureCompact.of_isClosed_subset hclosedTrace interior_subset_right
  let I : CompactSpace {y : K // y ∈ A ∩ closure V} :=
    isCompact_iff_compactSpace.mp hcompactTrace
  have hcount : (countableBasis {y : K // y ∈ A ∩ closure V}).Countable :=
    countable_countableBasis _
  have hbasis :
    TopologicalSpace.IsTopologicalBasis
      (countableBasis {y : K // y ∈ A ∩ closure V}) :=
    isBasis_countableBasis _
  exact ⟨V, countableBasis _, hVopen, hxV, hclosureCompact, hclosureU,
    hclosedTrace, hcount, hbasis⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_29_10__me__8d1abbd9

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint dc45fa6cf5c8fe6c

4.100 100. localCompactAt_iff_exists_compact_closed_interior

The problem

Show that, in a Hausdorff space, the point x is locally compact exactly when there is a compact closed set K whose interior contains x .

Lean statement

```
theorem localCompactAt_iff_exists_compact_closed_interior {X : Type*}
  [TopologicalSpace X] [T2Space X] (x : X) :
  (∃ U : Set X, x ∈ U ∧ IsOpen U ∧ (∃ K : Set X, U ⊆ K ∧ IsCompact K)) ↔
  ∃ K : Set X, IsCompact K ∧ IsClosed K ∧ x ∈ interior K
```

Parent. Show that if X is a Hausdorff space that is locally compact at the point x , then for each neighborhood U of x , there is a neighborhood V of x such that $\bar{V}$ is compact and $\bar{V} \subset U$.

Parent in Lean

```
theorem Munkres_exercise_29_10 {X : Type*}
  [TopologicalSpace X] [T2Space X] (x : X)
  (hx : ∃ U : Set X, x ∈ U ∧ IsOpen U ∧ (∃ K : Set X, U ⊆ K ∧ IsCompact K))
  (U : Set X) (hU : IsOpen U) (hxU : x ∈ U) :
  ∃ (V : Set X), IsOpen V ∧ x ∈ V ∧ IsCompact (closure V) ∧ closure V ⊆ U := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from producing a relatively compact neighborhood inside an arbitrary open set to a two-way characterization via compact closed sets with x in their interior. Its proof uses interior and neighborhood equivalences in both directions, rather than recalling the parent's separation-and-shrinking argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuine two-way characterization, including the reverse construction using interior K , rather than reproducing the parent's closure-shrinking argument. It is not substantially helped by recalling the parent's proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the parent's compact-closure neighborhood conclusion into a genuine two-way characterization. Its proof uses a distinct direct interior/neighborhood argument in each direction, with compactness supplying closedness, rather than recalling the parent's separation and closure construction.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2916 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem localCompactAt_iff_exists_compact_closed_interior.extracted_1_1.{u_1} {X : Type u_1} [inst : TopologicalSpace X]
[inst_1 : T2Space X] (x : X) :
(∃ U, x ∈ U ∧ IsOpen U ∧ ∃ K, U ⊆ K ∧ IsCompact K) ↔ ∃ K, IsCompact K ∧ IsClosed K ∧ x ∈ interior K
```

Read back by a model that saw only the Lean. *Let X be a topological space satisfying the T_2 (Hausdorff) separation axiom, and let x be a point of X . Then the following are equivalent: (1) there exists a set U such that x belongs to U , U is open, and there exists a compact set K containing U ; (2) there exists a compact, closed set K such that x belongs to the interior of K .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem localCompactAt_iff_exists_compact_closed_interior {X : Type*}
[TopologicalSpace X] [T2Space X] (x : X) :
(∃ U : Set X, x ∈ U ∧ IsOpen U ∧ (∃ K : Set X, U ⊆ K ∧ IsCompact K)) ↔
  ∃ K : Set X, IsCompact K ∧ IsClosed K ∧ x ∈ interior K := by
  constructor
  · rintro ⟨U, hxU, hU, K, hUK, hKcompact⟩
    have hKclosed : IsClosed K := hKcompact.isClosed
    have hUnhds : U ∈ nhds x := hU.mem_nhds hxU
    have hKnhds : K ∈ nhds x := Filter.mem_of_superset hUnhds hUK
    have hxinterior : x ∈ interior K := (mem_interior_iff_mem_nhds).2 hKnhds
    exact ⟨K, hKcompact, hKclosed, hxinterior⟩
  · rintro ⟨K, hKcompact, hKclosed, hxinterior⟩
    have hinterior_open : IsOpen (interior K) := isOpen_interior
    have hinterior_subset : interior K ⊆ K := interior_subset
    exact ⟨interior K, hxinterior, hinterior_open, K, hinterior_subset, hKcompact⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_29_10__me__90c38c4d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 90c38c4d709643ee

4.101 101. compact_set_nested_open_shrink

The problem

Let K be a compact subset of an open set U in a Hausdorff locally compact space. Show that there are open sets V and W such that $K \subseteq V$, $\overline{V} \subseteq W$, $\overline{W}$ is compact, and $\overline{W} \subseteq U$.

Lean statement

```
theorem compact_set_nested_open_shrink {X : Type*} [TopologicalSpace X] [T2Space X] [LocallyCompactSpace X]
  (K U : Set X) (hK : IsCompact K) (hU : IsOpen U) (hKU : K ⊆ U) :
  ∃ V W : Set X, IsOpen V ∧ K ⊆ V ∧ closure V ⊆ W ∧
    IsOpen W ∧ IsCompact (closure W) ∧ closure W ⊆ U
```

Parent. Show that if X is a Hausdorff space that is locally compact at the point x , then for each neighborhood U of x , there is a neighborhood V of x such that $\overline{V}$ is compact and $\overline{V} \subseteq U$.

Parent in Lean

```
theorem Munkres_exercise_29_10 {X : Type*}
  [TopologicalSpace X] [T2Space X] (x : X)
  (hx : ∃ U : Set X, x ∈ U ∧ IsOpen U ∧ (∃ K : Set X, U ⊆ K ∧ IsCompact K))
  (U : Set X) (hU : IsOpen U) (hxU : x ∈ U) :
  ∃ (V : Set X), IsOpen V ∧ x ∈ V ∧ IsCompact (closure V) ∧ closure V ⊆ U := by
```

What it contributes. Retains the parent's compact-closure neighborhood-shrink principle, but upgrades the protected object from a single point to a compact set and makes the second uniform shrink the final obligation.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child requires a genuinely new nested refinement: it first obtains a compact-closure neighborhood W of K inside U , then independently shrinks again to V with $\text{closure } V \subseteq W$. The parent's point-separation argument supplies neither compact-set shrinking nor this second containment obligation.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely adds a nested refinement: after obtaining W with compact closure inside U , it must shrink $\text{closure } V$ again so that $\text{closure } V$ is contained in W . This second application and the resulting two-level inclusion are not supplied by the parent's pointwise construction.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes from a single point to an arbitrary compact set and requires a genuinely nested refinement: first compactly shrinking into U , then shrinking again so that $\text{closure } V$ is contained in W while $\text{closure } W$ remains compact and contained in U . Its proof uses the compact-set shrinking theorem twice and does not recall or reuse the parent's point-separation argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2327 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem compact_set_nested_open_shrink.extracted_1_1.{u_1} {X : Type u_1} [inst : TopologicalSpace X]
  [inst_1 : T2Space X] [inst_2 : LocallyCompactSpace X] (K U : Set X) (hK : IsCompact K) (hU : IsOpen U) (hKU : K ⊆ U) :
  ∃ V W, IsOpen V ∧ K ⊆ V ∧ closure V ⊆ W ∧ IsOpen W ∧ IsCompact (closure W) ∧ closure W ⊆ U
```

Read back by a model that saw only the Lean. *Let X be a locally compact Hausdorff topological space, and let K and U be subsets of X . Suppose that K is compact, U is open, and $K \subseteq U$. Then there exist subsets V and W of X such that V is open, $K \subseteq V$, the closure of V is contained in W , W is open, the closure of W is compact, and the closure of W is contained in U .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem compact_set_nested_open_shrink {X : Type*} [TopologicalSpace X] [T2Space X] [LocallyCompactSpace X]
  (K U : Set X) (hK : IsCompact K) (hU : IsOpen U) (hKU : K ⊆ U) :
  ∃ V W : Set X, IsOpen V ∧ K ⊆ V ∧ closure V ⊆ W ∧
    IsOpen W ∧ IsCompact (closure W) ∧ closure W ⊆ U := by
  have hFirst : ∃ V : Set X, IsOpen V ∧ K ⊆ V ∧ closure V ⊆ U ∧ IsCompact (closure V) := by
    exact exists_open_between_and_isCompact_closure hK hU hKU
  obtain ⟨V, hVopen, hKV, hVsub, hVcompact⟩ := hFirst
  have hSecond : ∃ W : Set X, IsOpen W ∧ closure V ⊆ W ∧ closure W ⊆ U ∧ IsCompact (closure W) := by
    exact exists_open_between_and_isCompact_closure hVcompact hU hVsub
  obtain ⟨W, hWopen, hVW, hWsub, hWcompact⟩ := hSecond
  exact ⟨V, W, hVopen, hKV, hVW, hWopen, hWcompact, hWsub⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_29_10__me__d4a2b15e

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint d4a2b15e8b809b20

4.102 102. prescribed_pair_unit_orthogonal

The problem

Let $x \in \mathbb{R}^n$ and let $i \neq j$ be prescribed coordinates. Prove that there is a unit vector supported only on i and j and orthogonal to x .

Lean statement

```
theorem prescribed_pair_unit_orthogonal
  (n : ℕ)
  (x : EuclideanSpace ℝ (Fin n))
  (i j : Fin n)
  (hij : i ≠ j) :
  ∃ y : EuclideanSpace ℝ (Fin n),
    ‖y‖ = 1 ∧
    ⟨x, y⟩_ℝ = 0 ∧
    ∀ k : Fin n, k ≠ i → k ≠ j → y k = 0
```

Parent. Let $n > 1$, let $x \in \mathbb{R}^n$, and choose distinct coordinates i and j . Prove that there is a nonzero vector supported only on these two coordinates and orthogonal to x .

Parent in Lean

```
theorem prescribed_pair_orthogonal
  (n : ℕ)

  (x : EuclideanSpace ℝ (Fin n))
  (i j : Fin n)
  (hij : i ≠ j) :
  ∃ y : EuclideanSpace ℝ (Fin n),
    y ≠ 0 ∧ ⟨x, y⟩_ℝ = 0 ∧
    ∀ k : Fin n, k ≠ i → k ≠ j → y k = 0
```

What it contributes. Supplies the prescribed-pair witness construction, its coordinate nonzeroness checkpoint, orthogonality calculation, and support argument.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a genuine unit-norm requirement: it constructs the prescribed-pair orthogonal witness, proves its norm is nonzero, and transports orthogonality and support through normalization. This requires reasoning beyond recalling the parent's existential conclusion.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuinely new normalization argument: it must prove the constructed supported orthogonal witness is nonzero, establish its norm is nonzero, and transport orthogonality and support through scalar multiplication. Memorizing the parent only supplies the unnormalized witness, not the unit-norm conclusion.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a substantive normalization argument: it constructs the prescribed-pair orthogonal witness, proves its norm is nonzero, and transports orthogonality and support through scalar multiplication to obtain unit norm. Recalling the parent only supplies the unnormalized witness and does not solve the new claim.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2348 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem prescribed_pair_unit_orthogonal.extracted_1.1 (n : ℕ) (x : EuclideanSpace ℝ (Fin n)) (i j : Fin n)
  (hij : i ≠ j) : ∃ y, ‖y‖ = 1 ∧ ⟨x, y⟩_R = 0 ∧ ∀ (k : Fin n), k ≠ i → k ≠ j → y.ofLp k = 0
```

Read back by a model that saw only the Lean. *For any natural number n , any vector x in the Euclidean space $\mathbb{R}^n$ indexed by $\text{Fin } n$, and any distinct indices $i, j \in \text{Fin } n$, there exists a vector y in the same space such that $\|y\| = 1$, the real inner product of x and y is 0, and for every index $k \in \text{Fin } n$ distinct from both i and j , the k -th coordinate of y is 0.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open scoped InnerProductSpace
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators InnerProductSpace

theorem prescribed_pair_unit_orthogonal
  (n : ℕ)
  (x : EuclideanSpace ℝ (Fin n))
  (i j : Fin n)
  (hij : i ≠ j) :
  ∃ y : EuclideanSpace ℝ (Fin n),
    ‖y‖ = 1 ∧
    ⟨x, y⟩_R = 0 ∧
    ∀ k : Fin n, k ≠ i → k ≠ j → y k = 0 := by
  classical
  have normalize : ∀ z : EuclideanSpace ℝ (Fin n),
    z ≠ 0 →
    ⟨x, z⟩_R = 0 →
    (∀ k : Fin n, k ≠ i → k ≠ j → z k = 0) →
    ∃ y : EuclideanSpace ℝ (Fin n),
      ‖y‖ = 1 ∧
      ⟨x, y⟩_R = 0 ∧
      ∀ k : Fin n, k ≠ i → k ≠ j → y k = 0 := by
    intro z hz horth hsupport
    let y := (‖z‖ : ℝ)⁻¹ • z
    have hn : ‖z‖ ≠ 0 := norm_ne_zero_iff.mpr hz
    have hnorm : ‖y‖ = 1 := by
      simpa [y] using (norm_smul_inv_norm hz)
    have hyorth : ⟨x, y⟩_R = 0 := by
      change ⟨x, (‖z‖ : ℝ)⁻¹ • z⟩_R = 0
      rw [real_inner_smul_right, horth, mul_zero]
    have hysupport : ∀ k : Fin n, k ≠ i → k ≠ j → y k = 0 := by
      intro k hki hkj
      simp [y, hsupport k hki hkj]
    exact ⟨y, hnorm, hyorth, hysupport⟩
  by_cases hxi : x i = 0
  · let z := EuclideanSpace.single i (1 : ℝ)
    have hz : z ≠ 0 := by
      intro hzero
      have hi := congrArg (fun v => v i) hzero
      simp [z] at hi
```

```

have horth :  $\langle x, z \rangle_R = 0 := by
  simp [z, EuclideanSpace.inner_single_right, hxi]
have hsupport :  $\forall k : \text{Fin } n, k \neq i \rightarrow k \neq j \rightarrow z \cdot k = 0 := by
  intro k hki hkj
  simp [z, EuclideanSpace.single_apply, hki]
exact normalize z hz horth hsupport
· let z := EuclideanSpace.single i (x j) - EuclideanSpace.single j (x i)
  have hji :  $j \neq i := \text{Ne.symm } hij$ 
  have hzj :  $z \cdot j = -x \cdot i := by
    simp [z, EuclideanSpace.single_apply, hji]$ 
  have hz :  $z \neq 0 := by
    intro hzero
    have hz' := congrArg (fun v => v j) hzero
    have hzero_i :  $x \cdot i = 0 := by
      simpa [hzj] using hz'$ 
    exact hxi hzero_i$ 
  have horth :  $\langle x, z \rangle_R = 0 := by
    simp [z, inner_sub_right, EuclideanSpace.inner_single_right, mul_comm]$ 
  have hsupport :  $\forall k : \text{Fin } n, k \neq i \rightarrow k \neq j \rightarrow z \cdot k = 0 := by
    intro k hki hkj
    simp [z, EuclideanSpace.single_apply, hki, hkj]$ 
  exact normalize z hz horth hsupport$$ 
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_1_18a__me.me__ecf6bb19

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint ecf6bb190714a02a

4.103 103. prescribed_pair_orthogonal

The problem

Let $n > 1$, let $\mathbf{x} \in \mathbb{R}^n$, and choose distinct coordinates i and j . Prove that there is a nonzero vector supported only on these two coordinates and orthogonal to $\mathbf{x}$.

Lean statement

```
theorem prescribed_pair_orthogonal
  (n : ℕ)

  (x : EuclideanSpace ℝ (Fin n))
  (i j : Fin n)
  (hij : i ≠ j) :
  ∃ y : EuclideanSpace ℝ (Fin n),
    y ≠ 0 ∧ ⟨x, y⟩_R = 0 ∧
    ∀ k : Fin n, k ≠ i → k ≠ j → y k = 0
```

Parent. If $k \geq 2$ and $\mathbf{x} \in \mathbb{R}^k$, prove that there exists $\mathbf{y} \in \mathbb{R}^k$ such that $\mathbf{y} \neq 0$ but $\mathbf{x} \cdot \mathbf{y} = 0$

Parent in Lean

```
theorem Rudin_exercise_1_18a
  (n : ℕ)
  (h : n > 1)
  (x : EuclideanSpace ℝ (Fin n)) -- ℝ^n
  : ∃ (y : EuclideanSpace ℝ (Fin n)), y ≠ 0 ∧ ⟨x, y⟩_R = 0 := by
```

What it contributes. Adapts the parent's two-coordinate orthogonal witness construction, replacing the parent's existentially chosen pair with an arbitrary prescribed distinct pair and adding an explicit zero-coordinate case split plus support conclusion.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child uniformly handles an arbitrary prescribed distinct pair of coordinates, rather than choosing a pair adaptively from $\mathbf{x}$. Its zero/nonzero case split and support verification require reasoning not supplied by the parent's existential construction.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires constructing an orthogonal vector supported on an arbitrary prescribed pair, using the two selected coordinates and a nonzero-coordinate case split. The parent only chooses coordinates existentially and does not supply this uniform prescribed-support argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens the result to every prescribed distinct coordinate pair and requires the orthogonal vector to be supported on that pair. Its proof must analyze the two selected coordinates directly, unlike the parent's adaptive choice of coordinates from a nonzero component.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound

Check	By	Result
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2343 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem prescribed_pair_orthogonal.extracted_1_1 (n : ℕ) (h : n > 1) (x : EuclideanSpace ℝ (Fin n)) (i j : Fin n)
  (hij : i ≠ j) : ∃ y, y ≠ 0 ∧ ⟨x, y⟩_R = 0 ∧ ∀ (k : Fin n), k ≠ i → k ≠ j → y.ofLp k = 0
```

Read back by a model that saw only the Lean. *For every natural number n greater than 1, every vector x in the real Euclidean space indexed by $\text{Fin } n$, and every pair of distinct indices $i, j \in \text{Fin } n$, there exists a nonzero vector y in the same space such that the real inner product $\langle x, y \rangle$ is zero, and for every index $k \in \text{Fin } n$ distinct from both i and j , the k -th coordinate of y is zero.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open scoped InnerProductSpace
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators InnerProductSpace

theorem prescribed_pair_orthogonal
  (n : ℕ)

  (x : EuclideanSpace ℝ (Fin n))
  (i j : Fin n)
  (hij : i ≠ j) :
  ∃ y : EuclideanSpace ℝ (Fin n),
    y ≠ 0 ∧ ⟨x, y⟩_R = 0 ∧
    ∀ k : Fin n, k ≠ i → k ≠ j → y k = 0 := by
classical
by_cases hxi : x i = 0
· by_cases hxj : x j = 0
  · let y := EuclideanSpace.single i (1 : ℝ)
    have hy : y ≠ 0 := by
      intro hz
      have hz' := congrArg (fun z => z i) hz
      simp [y, EuclideanSpace.single_apply] at hz'
    have ho : ⟨x, y⟩_R = 0 := by
      simp [y, EuclideanSpace.inner_single_right, hxi]
    have hs : ∀ k : Fin n, k ≠ i → k ≠ j → y k = 0 := by
      intro k hki hkj
      simp [y, EuclideanSpace.single_apply, hki]
    exact ⟨y, hy, ho, hs⟩
  · let y := EuclideanSpace.single i (x j) - EuclideanSpace.single j (x i)
    have hji : j ≠ i := Ne.symm hij
    have hyj : y j = -x i := by
      simp [y, EuclideanSpace.single_apply, hji]
    have hy : y ≠ 0 := by
```

```

intro hz
have hz' := congrArg (fun z => z j) hz
have : x i = 0 := by
  simp [hyj] using hz'
exact hxi this
have ho :  $\langle x, y \rangle_R = 0$  := by
  simp [y, inner_sub_right, EuclideanSpace.inner_single_right, mul_comm]
have hs :  $\forall k : \text{Fin } n, k \neq i \rightarrow k \neq j \rightarrow y k = 0$  := by
  intro k hki hkj
  simp [y, EuclideanSpace.single_apply, hki, hkj]
exact  $\langle y, hy, ho, hs \rangle$ 

```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_1_18a__me__58f629fc

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint f57f8e5f7e32f73a

4.104 104. irrational_affine_rat

The problem

If q and r are rational numbers with $q \neq 0$ and x is irrational, prove that $xq + r$ is irrational.

Lean statement

```
theorem irrational_affine_rat (x : ℝ) (q r : ℚ) (hq : q ≠ 0) : Irrational x → Irrational (x * q + r)
```

Parent. If r is rational ($r \neq 0$) and x is irrational, prove that rx is irrational.

Parent in Lean

```
theorem Rudin_exercise_1_1b
(x : ℝ)
(y : ℚ)
(h : y ≠ 0)
: (Irrational x) → Irrational (x * y) := by
```

What it contributes. Supplies the nonzero rational scaling checkpoint via `hx.mul_ratCast hq`; the new proof adds a rational translation contradiction.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child genuinely extends irrationality preservation from nonzero rational multiplication to a rational affine map, requiring a new contradiction argument that solves for x via subtraction and division by q . The parent's multiplication lemma alone does not supply translation invariance.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely changes multiplication into a rational affine embedding and proves irrationality by deriving a rational representation of x via the inverse affine transformation. This requires new witness construction and field reasoning rather than recalling the parent's multiplication lemma.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a rational translation and requires an inverse affine argument: from a rational representation of $x*q+r$ it constructs $(s-r)/q$ and derives x is rational. This is not obtained by recalling the parent's direct multiplication lemma.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2015 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem irrational_affine_rat.extracted_1_1 (x : ℝ) (q r : ℚ) (hq : q ≠ 0) :  
  Irrational x → Irrational (x * ↑q + ↑r)
```

Read back by a model that saw only the Lean. *For a real number x and rational numbers q and r , if q is nonzero, then whenever x is irrational, the real number x multiplied by q and then increased by r is irrational.*

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
  
open Topology Filter Real Complex TopologicalSpace Finset  
open scoped BigOperators  
  
theorem irrational_affine_rat (x : ℝ) (q r : ℚ) (hq : q ≠ 0) : Irrational x → Irrational (x * q + r) := by  
  intro hx hrat  
  apply hx  
  rcases hrat with ⟨s, hs⟩  
  refine ⟨(s - r) / q, ?_⟩  
  rw [Rat.cast_div, Rat.cast_sub]  
  field_simp [hq]  
  linarith
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_1_1b__me__ebdca03d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint f712fecdd06d0e5a

4.105 105. rational_affine_action_free

The problem

Let x be irrational, and let r, s be rational numbers and y, z nonzero rational numbers. If $y(x + r) = z(x + s)$, prove that $y = z$ and $r = s$.

Lean statement

```
theorem rational_affine_action_free
(x : ℝ)
(hx : Irrational x)
(r s y z : ℚ)

(hz : z ≠ 0)
(h : y * (x + r) = z * (x + s)) :
y = z ∧ r = s
```

Parent. For a real number x , a rational number r , and a nonzero rational number y , prove that $y(x + r)$ is irrational if and only if x is irrational.

Parent in Lean

```
theorem irrational_rat_translate_mul_iff
(x : ℝ)
(r y : ℚ)
(hy : y ≠ 0) : Irrational (y * (x + r)) ↔ Irrational x
```

What it contributes. Supplies the translation checkpoint via `Irrational.add_ratCast` and the nonzero-scaling checkpoint via `Irrational.mul_ratCast`; these establish irrationality of both affine images before the new parameter-recovery obstruction.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a genuinely different freeness/uniqueness theorem: equality of two rational affine images forces both rational parameters to agree. Its proof requires coefficient comparison and an irrationality contradiction, none of which is supplied by the parent's preservation equivalence.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely new freeness/uniqueness theorem: irrationality forces equality of the rational coefficients, after which nonzero scaling forces equality of translations. The parent's equivalence proof does not supply this coefficient-comparison argument, and the child does not recall or embed the parent proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuine freeness/uniqueness theorem: irrationality forces equality of both rational scale and translation parameters. Its proof requires a new contradiction and rational-expression argument, not recall or reuse of the parent's irrationality equivalence.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable

Check	By	Result
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2766 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem rational_affine_action_free.extracted_1_1 (x : ℝ) (hx : Irrational x) (r s y z : ℚ) (hy : y ≠ 0) (hz : z ≠ 0)
  (h : ↑y * (x + ↑r) = ↑z * (x + ↑s)) : y = z ∧ r = s
```

Read back by a model that saw only the Lean. *Let x be an irrational real number, and let r, s, y , and z be rational numbers with $y \neq 0$ and $z \neq 0$. If $y(x + r) = z(x + s)$, where the rational numbers are viewed as real numbers, then $y = z$ and $r = s$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem rational_affine_action_free
(x : ℝ)
(hx : Irrational x)
(r s y z : ℚ)
(hz : z ≠ 0)
(h : y * (x + r) = z * (x + s)) :
y = z ∧ r = s := by
  have hyz : y = z := by
    by_contra hne
    have hden : y - z ≠ 0 := sub_ne_zero.mpr hne
    have hdenR : (y : ℝ) - (z : ℝ) ≠ 0 := by
      exact_mod_cast hden
    have hlin : ((y : ℝ) - (z : ℝ)) * x =
      (z : ℝ) * (s : ℝ) - (y : ℝ) * (r : ℝ) := by
      linear_combination h
    have hxrat : x =
      ((z : ℝ) * (s : ℝ) - (y : ℝ) * (r : ℝ)) /
      ((y : ℝ) - (z : ℝ)) := by
      apply (eq_div_iff hdenR).2
      nlinarith [hlin]
    have hrat : x = (((z * s - y * r) / (y - z) : ℚ) : ℝ) := by
      calc
        x = ((z : ℝ) * (s : ℝ) - (y : ℝ) * (r : ℝ)) /
          ((y : ℝ) - (z : ℝ)) := hxrat
        _ = (((z * s - y * r) / (y - z) : ℚ) : ℝ) := by
          norm_num [Rat.cast_div]
    apply hx
    exact ⟨(z * s - y * r) / (y - z), hrat.sym⟩
  have hsum : (z : ℝ) * ((r : ℝ) - (s : ℝ)) = 0 := by
    rw [hyz] at h
    nlinarith [h]
  have hzR : (z : ℝ) ≠ 0 := by
    exact_mod_cast hz
  have hrsR : (r : ℝ) - (s : ℝ) = 0 :=
    (mul_eq_zero.mpr hsum).resolve_left hzR
  have hrs : r = s := by
    exact_mod_cast sub_eq_zero.mpr hrsR
  exact ⟨hyz, hrs⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Rudin_exercise_1_1b__mh.me.me__4b96916a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c9cb24bccdc1d4b3

4.106 106. irrational_rat_translate_mul_iff

The problem

For a real number x , a rational number r , and a nonzero rational number y , prove that $y(x + r)$ is irrational if and only if x is irrational.

Lean statement

```
theorem irrational_rat_translate_mul_iff
(x : ℝ)
(r y : ℚ)
(hy : y ≠ 0) : Irrational (y * (x + r)) ↔ Irrational x
```

Parent. If r is a nonzero rational number and x is real, prove that x is irrational if and only if rx is irrational.

Parent in Lean

```
theorem irrational_mul_rat_iff
(x : ℝ)
(y : ℚ)
(h : y ≠ 0) : Irrational (x * y) ↔ Irrational x
```

What it contributes. Uses the parent's nonzero rational scaling checkpoint via `mul_ratCast` and explicitly consumes its inverse in the reverse-recovery direction; rational translation is inserted before scaling.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a symbolic rational translation, requiring irrationality invariance under rational addition and cancellation by the rational multiplier; the parent's scaling equivalence alone does not supply this argument. The proof is not a wrapper, decoration, or recall of the parent.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires a genuinely new argument: irrationality must first be preserved under rational translation and then under nonzero rational scaling. The parent's scaling equivalence alone cannot prove the translated case.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves invariance of irrationality under both rational translation and nonzero rational scaling, requiring additive closure and an inverse-scaling argument absent from the parent. The parent's multiplicative equivalence alone does not substantially help prove the translated expression.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2753 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem irrational_rat_translate_mul_iff.extracted_1_1 (x : ℝ) (r y : ℚ) (hy : y ≠ 0) :  
  Irrational (↑y * (x + ↑r)) ↔ Irrational x
```

Read back by a model that saw only the Lean. For every real number x and rational numbers r and y with $y \neq 0$, the real number $y(x+r)$ is irrational if and only if x is irrational.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open Topology Filter Real Complex TopologicalSpace Finset  
open scoped BigOperators  
  
theorem irrational_rat_translate_mul_iff  
(x : ℝ)  
(r y : ℚ)  
(hy : y ≠ 0) : Irrational (y * (x + r)) ↔ Irrational x := by  
  constructor  
  · intro hxy  
    have hscaled : Irrational ((x + r) * y) := by  
      simpa [mul_comm] using hxy  
    have hinv : Irrational (((x + r) * y) * y⁻¹) := by  
      exact hscaled.mul_ratCast (inv_ne_zero hy)  
    have htranslated := hinv.add_ratCast (q := -r)  
    simpa [mul_assoc, hy] using htranslated  
  · intro hx  
    have htranslated : Irrational (x + r) := by  
      exact hx.add_ratCast (q := r)  
    have hscaled : Irrational ((x + r) * y) := by  
      exact htranslated.mul_ratCast hy  
    simpa [mul_comm] using hscaled
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_1_1b__mh.me__1ceef025

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 1ceef02531900e66

4.107 107. irrational_affine_quotient_rat_iff

The problem

Let x be real, let a and c be nonzero rational numbers, and let b be rational. Prove that $((x/a)+b)c$ is irrational if and only if x is irrational.

Lean statement

```
theorem irrational_affine_quotient_rat_iff
  (x : ℝ) (a b c : ℚ) (ha : a ≠ 0) (hc : c ≠ 0) :
  Irrational (((x / (a : ℝ)) + (b : ℝ)) * (c : ℝ)) ↔ Irrational x
```

Parent. Let x be real and y a nonzero rational number. Prove that x/y is irrational if and only if x is irrational.

Parent in Lean

```
theorem irrational_div_rat_iff
  (x : ℝ) (y : ℚ) (h : y ≠ 0) :
  Irrational (x / (y : ℝ)) ↔ Irrational x
```

What it contributes. Supplies the quotient irrationality normalization, used through `irrational_div_ratCast_iff` for the parameter `a`.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the transformation by adding an arbitrary rational translation and an independent nonzero rational scale. Its proof requires new irrationality-preservation steps for rational addition and multiplication, so recalling the parent's division equivalence alone does not solve it.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires new irrationality-invariance arguments for adding and multiplying by independent rational parameters, beyond the parent's quotient equivalence. Both nonzero hypotheses are materially used, so recalling the parent proof does not solve the child.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds an independent rational translation and scaling, requiring irrationality transport through addition and multiplication by nonzero rational casts. The parent's division-only argument does not supply these checkpoints, and the new parameters are substantively used.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2785 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem irrational_affine_quotient_rat_iff.extracted_1_1 (x : ℝ) (a b c : ℚ) (ha : a ≠ 0) (hc : c ≠ 0) :  
  Irrational ((x / ↑a + ↑b) * ↑c) ↔ Irrational x
```

Read back by a model that saw only the Lean. For every real number x and rational numbers a , b , and c , if $a \neq 0$ and $c \neq 0$, then $(x/a + b)c$ is irrational if and only if x is irrational, where the rational numbers are regarded as real numbers.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
  
open Topology Filter Real Complex TopologicalSpace Finset  
open scoped BigOperators  
  
theorem irrational_affine_quotient_rat_iff  
  (x : ℝ) (a b c : ℚ) (ha : a ≠ 0) (hc : c ≠ 0) :  
    Irrational (((x / (a : ℝ)) + (b : ℝ)) * (c : ℝ)) ↔ Irrational x := by  
  constructor  
  · intro h  
    have htranslated : Irrational ((x / (a : ℝ)) + (b : ℝ)) := by  
      exact h.of_mul_ratCast  
    have hquotient : Irrational (x / (a : ℝ)) := by  
      exact htranslated.of_add_ratCast  
    have hnormalized : a ≠ 0 ∧ Irrational x := by  
      exact irrational_div_ratCast_iff.mp hquotient  
    exact hnormalized.2  
  · intro hx  
    have hquotient : Irrational (x / (a : ℝ)) := by  
      exact irrational_div_ratCast_iff.mpr ⟨ha, hx⟩  
    have htranslated : Irrational ((x / (a : ℝ)) + (b : ℝ)) := by  
      exact hquotient.add_ratCast b  
    have hscaled : Irrational (((x / (a : ℝ)) + (b : ℝ)) * (c : ℝ)) := by  
      exact htranslated.mul_ratCast hc  
    exact hscaled
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Rudin_exercise_1_1b__mh.ms.me__97127643

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 97127643b9327c7c

4.108 108. rational_division_representation

The problem

Let x be real and y a nonzero rational number. Prove that x/y is rational exactly when x is a rational multiple of y .

Lean statement

```
theorem rational_division_representation
  (x : ℝ) (y : ℚ) (h : y ≠ 0) :
  (∃ q : ℚ, x / (y : ℝ) = (q : ℝ)) ↔
  (∃ q : ℚ, x = (q : ℝ) * (y : ℝ))
```

Parent. Let x be real and y a nonzero rational number. Prove that x/y is irrational if and only if x is irrational.

Parent in Lean

```
theorem irrational_div_rat_iff
  (x : ℝ) (y : ℚ) (h : y ≠ 0) :
  Irrational (x / (y : ℝ)) ↔ Irrational x
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from preservation of irrationality to an existential characterization of rational quotients. Its proof requires explicit field cancellation and constructing witnesses, none of which is supplied by the parent's irrationality argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the claim from preservation of irrationality to an existential representation equivalence. Its proof requires clearing a nonzero rational denominator and constructing witnesses in both directions, which the parent's irrationality lemma does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from irrationality preservation to an existential characterization of rational divisibility. Its proof requires introducing a rational witness and using nonzero-field cancellation, which is not supplied by recalling the parent's irrationality equivalence.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2787 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem rational_division_representation.extracted_1_1 (x : ℝ) (y : ℚ) (h : y ≠ 0) :  
  (∃ q, x / ↑y = ↑q) ↔ ∃ q, x = ↑q * ↑y
```

Read back by a model that saw only the Lean. For every real number x and rational number y with $y \neq 0$, the quotient x divided by y is equal to some rational number if and only if there exists a rational number q such that $x = q \cdot y$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open Topology Filter Real Complex TopologicalSpace Finset  
open scoped BigOperators  
  
theorem rational_division_representation  
  (x : ℝ) (y : ℚ) (h : y ≠ 0) :  
    (∃ q : ℚ, x / (y : ℝ) = (q : ℝ)) ↔  
    (∃ q : ℚ, x = (q : ℝ) * (y : ℝ)) := by  
  have hy : (y : ℝ) ≠ 0 := by  
    intro hy0  
    apply h  
    exact_mod_cast hy0  
  constructor  
  · rintro ⟨q, hq⟩  
    refine ⟨q, ?_⟩  
    have hmul : x = (x / (y : ℝ)) * (y : ℝ) := by  
      field_simp [hy]  
    calc  
      x = (x / (y : ℝ)) * (y : ℝ) := hmul  
      _ = (q : ℝ) * (y : ℝ) := by rw [hq]  
  · rintro ⟨q, hq⟩  
    refine ⟨q, ?_⟩  
    calc  
      x / (y : ℝ) = ((q : ℝ) * (y : ℝ)) / (y : ℝ) := by rw [hq]  
      _ = (q : ℝ) := by field_simp [hy]
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Rudin_exercise_1_1b__mh.ms.me__c46ab9d2

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c46ab9d2198a9861

4.109 109. supplied_basis_trace_basis

The problem

Let K be a compact metric space, A a closed subset of K , and B a countable topological basis for K . Show that the traces of the sets in B on A form a countable topological basis for A .

Lean statement

```
theorem supplied_basis_trace_basis {K : Type*} [MetricSpace K] [CompactSpace K]
  {A : Set K} (B : Set (Set K))
  (hBcount : Set.Countable B)
  (hBbasis : TopologicalSpace.IsTopologicalBasis B) :
  Set.Countable
    (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) ∧
    TopologicalSpace.IsTopologicalBasis
      (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B)
```

Parent. Let K be a compact metric space and A a closed subset of K . Construct a countable family of ambient open sets whose traces on A form a topological basis for A .

Parent in Lean

```
theorem closed_subspace_countable_trace_basis {K : Type*} [MetricSpace K] [CompactSpace K] {A : Set K} (hA : IsClosed A) :
  ∃ (B : Set (Set K)), Set.Countable B ∧
    (∀ U ∈ B, IsOpen U) ∧
    TopologicalSpace.IsTopologicalBasis
      (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B)
```

What it contributes. Uses the parent's closed-subspace trace construction, specialized to a fixed supplied ambient basis rather than an existentially chosen family.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a genuine induced-topology result for an arbitrary countable ambient basis, using the basis-induction lemma and image countability; the parent's existential construction and closedness argument do not supply this proof. All explicit child hypotheses are used, and the absence of a closedness hypothesis makes the result strictly more general.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the existential construction into a general theorem about any supplied countable ambient basis, and its proof relies on the induced-topology basis lemma rather than recalling the parent's closed-subspace construction. Compactness and closedness are no longer needed, so this is not decoration or a forced wrapper.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the ambient basis: it proves that any supplied countable topological basis traces to a countable basis on the subtype. The parent only constructs one suitable basis existentially, so recalling its proof does not establish this arbitrary-basis result.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound

Check	By	Result
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2349 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem supplied_basis_trace_basis.extracted_1_1.{u_1} {K : Type u_1} [inst : MetricSpace K] [inst_1 : CompactSpace K]
  {A : Set K} (hA : IsClosed A) (B : Set (Set K)) (hBcount : B.Countable) (hBbasis : IsTopologicalBasis B) :
  ((fun U => Subtype.val ⁻¹' U) '' B).Countable ∧ IsTopologicalBasis ((fun U => Subtype.val ⁻¹' U) '' B)
```

Read back by a model that saw only the Lean. *Let K be a compact metric space, let $A \subseteq K$ be closed, and let B be a countable topological basis for K . Then the family of traces $\{U \cap A \mid U \in B\}$, regarded as subsets of the subspace A , is countable and is a topological basis for A .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem supplied_basis_trace_basis {K : Type*} [MetricSpace K] [CompactSpace K]
  {A : Set K} (B : Set (Set K))
  (hBcount : Set.Countable B)
  (hBbasis : TopologicalSpace.IsTopologicalBasis B) :
  Set.Countable
    (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) ∧
    TopologicalSpace.IsTopologicalBasis
      (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) := by
  have hcount : Set.Countable
    (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) :=
    hBcount.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U)
  have hbasis : TopologicalSpace.IsTopologicalBasis
    (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) :=
    hBbasis.induced (Subtype.val : A → K)
  exact ⟨hcount, hbasis⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_2_25__me.me__86310cc6

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 291534e956a8f28a

4.110 110. compact_closed_refined_trace_family

The problem

Let K be a compact second-countable metric space and let A be a closed subset of K . Show that there is a countable family of open subsets of K whose traces form a basis for A , separate distinct points of A , and admit local refinements whose closures remain inside the original member.

Lean statement

```
theorem compact_closed_refined_trace_family {K : Type*} [MetricSpace K]
  [SecondCountableTopology K] [CompactSpace K] {A : Set K} (hA : IsClosed A) :
  ∃ (B : Set (Set K)), Set.Countable B ∧
    (∀ U ∈ B, IsOpen U) ∧
    TopologicalSpace.IsTopologicalBasis
      (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) ∧
    (∀ x y : A, x ≠ y → ∃ U ∈ B,
      x ∈ (Subtype.val : A → K) ⁻¹' U ∧
      y ∉ (Subtype.val : A → K) ⁻¹' U) ∧
    (∀ U ∈ B, ∀ x : K, x ∈ A → x ∈ U →
      ∃ V ∈ B, x ∈ V ∧ closure (A ∩ V) ⊆ A ∩ U)
```

Parent. Let K be a second-countable metric space and $A \subseteq K$. Show that some countable family of open subsets of K has traces forming a basis for A and separates every two distinct points of A .

Parent in Lean

```
theorem second_countable_subspace_trace_basis {K : Type*} [MetricSpace K]
  [SecondCountableTopology K] {A : Set K} :
  ∃ (B : Set (Set K)), Set.Countable B ∧
    (∀ U ∈ B, IsOpen U) ∧
    TopologicalSpace.IsTopologicalBasis
      (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) ∧
    (∀ x y : A, x ≠ y → ∃ U ∈ B,
      x ∈ (Subtype.val : A → K) ⁻¹' U ∧
      y ∉ (Subtype.val : A → K) ⁻¹' U)
```

What it contributes. Supplied the countable ambient basis, induced trace-basis checkpoint, and point-separation construction; the child adds a consumed closure-refinement obligation.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a genuine closure-refinement property, proved using metric-ball shrinking and closedness of A ; this mathematics is absent from the parent's basis and separation proof. The added compactness assumption is not needed for the refinement itself, but the new conclusion is substantive and cannot be obtained by recalling the parent.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The added relative-closure refinement is a substantive compactness/metric shrinking claim, requiring closedness and a new closure-containment argument beyond the parent's basis and separation proof. Recalling the parent cannot establish this extra conjunct.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuinely new refinement theorem: each basis neighborhood must admit a basis neighborhood whose relative closure in the closed compact subspace remains inside the original neighborhood. Proving this requires metric-ball and closure containment reasoning absent from the parent, so recalling the parent alone does not solve it.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2790 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem compact_closed_refined_trace_family.extracted_1.1.{u_1} {K : Type u_1} [inst : MetricSpace K]
  [inst_1 : SecondCountableTopology K] [inst_2 : CompactSpace K] {A : Set K} (hA : IsClosed A) :
  ∃ B,
    B.Countable ∧
    (∀ U ∈ B, IsOpen U) ∧
    IsTopologicalBasis ((fun U => Subtype.val ⁻¹' U) '' B) ∧
    (∀ (x y : ↑A), x ≠ y → ∃ U ∈ B, x ∈ Subtype.val ⁻¹' U ∧ y ∉ Subtype.val ⁻¹' U) ∧
    ∀ U ∈ B, ∀ x ∈ A, x ∈ U → ∃ V ∈ B, x ∈ V ∧ closure (A ∩ V) ⊆ A ∩ U
```

Read back by a model that saw only the Lean. *Let K be a compact metric space with a second-countable topology, and let A be a closed subset of K . Then there exists a countable collection B of subsets of K such that: every member of B is open in K ; the collection of their preimages under the inclusion map $A \rightarrow K$ is a topological basis for A with its subspace topology; for any two distinct points x and y of A , some $U \in B$ contains x and does not contain y ; and for every $U \in B$ and every $x \in A$ with $x \in U$, there exists $V \in B$ such that $x \in V$ and the closure in K of $A \cap V$ is contained in $A \cap U$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem compact_closed_refined_trace_family {K : Type*} [MetricSpace K]
  [SecondCountableTopology K] [CompactSpace K] {A : Set K} (hA : IsClosed A) :
  ∃ (B : Set (Set K)), Set.Countable B ∧
    (∀ U ∈ B, IsOpen U) ∧
    TopologicalSpace.IsTopologicalBasis
      (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) ∧
    (∀ x y : A, x ≠ y → ∃ U ∈ B,
      x ∈ (Subtype.val : A → K) ⁻¹' U ∧
      y ∉ (Subtype.val : A → K) ⁻¹' U) ∧
    (∀ U ∈ B, ∀ x : K, x ∈ A → x ∈ U →
      ∃ V ∈ B, x ∈ V ∧ closure (A ∩ V) ⊆ A ∩ U) := by
  classical
  obtain ⟨B, hBcount, hBne, hBbasis⟩ :=
    TopologicalSpace.exists_countable_basis (α := K)
  have hBopen : ∀ U ∈ B, IsOpen U := by
    intro U hU
    exact hBbasis.isOpen hU
  have hTraceBasis :
```

```

TopologicalSpace.IsTopologicalBasis
(Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) := by
exact hBbasis.induced (Subtype.val : A → K)
have hSeparate : ∀ x y : A, x ≠ y → ∃ U ∈ B,
  x ∈ (Subtype.val : A → K) ⁻¹' U ∧
  y ∉ (Subtype.val : A → K) ⁻¹' U := by
intro x y hxy
have hpos : 0 < dist (x : K) (y : K) := dist_pos.mpr (by
  intro h
  apply hxy
  exact Subtype.ext h)
have hxball : (x : K) ∈ Metric.ball (x : K) (dist (x : K) (y : K)) := by
  rw [Metric.mem_ball]
  simpa using hpos
have hyball : (y : K) ∉ Metric.ball (x : K) (dist (x : K) (y : K)) := by
  intro hy
  have hlt : dist (x : K) (y : K) < dist (x : K) (y : K) := by
    simpa [Metric.mem_ball, dist_comm] using hy
  exact (lt_irrefl _ hlt)
obtain ⟨U, hU, hxU, hUsub⟩ :=
  hBbasis.exists_subset_of_mem_open hxball Metric.isOpen_ball
refine ⟨U, hU, hxU, ?_⟩
intro hyU
exact hyball (hUsub hyU)
have hAcompact : IsCompact A := hA.isCompact
have hAclosed : IsClosed A := hAcompact.isClosed
have hRefine : ∀ U ∈ B, ∀ x : K, x ∈ A → x ∈ U →
  ∃ V ∈ B, x ∈ V ∧ closure (A ∩ V) ⊆ A ∩ U := by
  intro U hU x hxA hxU
  rcases (Metric.isOpen_iff.mp (hBopen U hU) x hxU) with ⟨ε, hε, hball⟩
  have hxsmall : x ∈ Metric.ball x (ε / 2) := by
    rw [Metric.mem_ball]
    simp [hε]
  obtain ⟨V, hV, hxV, hVsub⟩ :=
    hBbasis.exists_subset_of_mem_open hxsmall Metric.isOpen_ball
  refine ⟨V, hV, hxV, ?_⟩
  have hclosed : IsClosed (A ∩ Metric.closedBall x (ε / 2)) :=
    hAclosed.inter Metric.isClosed_closedBall
  have hsub : A ∩ V ⊆ A ∩ Metric.closedBall x (ε / 2) := by
    intro z hz
    refine ⟨hz.1, ?_⟩
    rw [Metric.mem_closedBall]
    exact le_of_lt (Metric.mem_ball.mp (hVsub hz.2))
  have hcl : closure (A ∩ V) ⊆ A ∩ Metric.closedBall x (ε / 2) :=
    closure_minimal hsub hclosed
  intro z hz
  have hz' := hcl hz
  refine ⟨hz'.1, hball ?_⟩
  rw [Metric.mem_ball]
  calc
    dist z x ≤ ε / 2 := Metric.mem_closedBall.mp hz'.2
    _ < ε := by linarith
exact ⟨B, hBcount, hBopen, hTraceBasis, hSeparate, hRefine⟩

```

Operator. mutation / mutation_easy **Lineage depth.** 3 **Certificate.** reproducible

Problem id. Rudin_exercise_2_25__me.mh.me.me__7c9f9ef2

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 7c9f9ef2278dc87a

4.111 111. second_countable_subspace_trace_basis

The problem

Let K be a second-countable metric space and $A \subseteq K$. Show that some countable family of open subsets of K has traces forming a basis for A and separates every two distinct points of A .

Lean statement

```
theorem second_countable_subspace_trace_basis {K : Type*} [MetricSpace K]
  [SecondCountableTopology K] {A : Set K} :
  ∃ (B : Set (Set K)), Set.Countable B ∧
    (∀ U ∈ B, IsOpen U) ∧
    TopologicalSpace.IsTopologicalBasis
      (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) ∧
    (∀ x y : A, x ≠ y → ∃ U ∈ B,
      x ∈ (Subtype.val : A → K) ⁻¹' U ∧
      y ∉ (Subtype.val : A → K) ⁻¹' U)
```

Parent. Let K be a compact metric space and A a closed subset of K . Show that there is a countable family of ambient open sets whose traces form a basis for A and separate any two distinct points of A .

Parent in Lean

```
theorem closed_subspace_countable_separating_trace_basis {K : Type*} [MetricSpace K] [CompactSpace K] {A : Set K} (hA : IsClosed A) :
  ∃ (B : Set (Set K)), Set.Countable B ∧
    (∀ U ∈ B, IsOpen U) ∧
    TopologicalSpace.IsTopologicalBasis
      (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) ∧
    (∀ x y : A, x ≠ y → ∃ U ∈ B,
      x ∈ (Subtype.val : A → K) ⁻¹' U ∧
      y ∉ (Subtype.val : A → K) ⁻¹' U)
```

What it contributes. Retained the parent's countable ambient-open family, trace-basis construction, and metric-ball separation checkpoint, while replacing compact closed subspaces with arbitrary subspaces of second-countable metric spaces.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes from compact closed subspaces to arbitrary subspaces of a second-countable metric space, and its separation component requires a new positive-distance ball and basis-refinement argument. The parent's compactness-based countable basis construction does not supply this proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes from compact closed subspaces to arbitrary subspaces of a second-countable metric space. Its proof uses a new ambient countable-basis construction and induced-topology argument, while separately deriving metric separation.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely replaces compactness and closedness with ambient second countability, then derives a countable trace basis by inducing an ambient basis and proves separation via metric balls and basis refinement. This reasoning does not rely on recalling the parent's compact-subspace basis construction.

Check	By	Result
statement_type_check	lean	type-checks

Check	By	Result
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2783 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem second_countable_subspace_trace_basis.extracted_1_1.{u_1} {K : Type u_1} [inst : MetricSpace K]
[inst_1 : SecondCountableTopology K] {A : Set K} :
  ∃ B,
    B.Countable ∧
    (∀ U ∈ B, IsOpen U) ∧
    IsTopologicalBasis ((fun U => Subtype.val ⁻¹' U) '' B) ∧
    ∀ (x y : ↑A), x ≠ y → ∃ U ∈ B, x ∈ Subtype.val ⁻¹' U ∧ y ∉ Subtype.val ⁻¹' U
```

Read back by a model that saw only the Lean. *Let K be a metrizable, second-countable topological space, and let A be a subset of K . Then there exists a countable collection B of open subsets of K such that the collection of their preimages under the subtype inclusion map $A \rightarrow K$ forms a topological basis for A with its subspace topology, and such that for any two distinct points x and y of A , there is some $U \in B$ with $x \in U$ (as a point of the subspace A) and $y \notin U$ (as a point of the subspace A).*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem second_countable_subspace_trace_basis {K : Type*} [MetricSpace K]
[SecondCountableTopology K] {A : Set K} :
  ∃ (B : Set (Set K)), Set.Countable B ∧
    (∀ U ∈ B, IsOpen U) ∧
    TopologicalSpace.IsTopologicalBasis
      (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) ∧
    (∀ x y : A, x ≠ y → ∃ U ∈ B,
      x ∈ (Subtype.val : A → K) ⁻¹' U ∧
      y ∉ (Subtype.val : A → K) ⁻¹' U) := by
  classical
  obtain ⟨B, hBcount, hBne, hBbasis⟩ :=
    TopologicalSpace.exists_countable_basis (α := K)
  have hBopen : ∀ U ∈ B, IsOpen U := by
    intro U hU
    exact hBbasis.isOpen hU
  have hTraceBasis :
    TopologicalSpace.IsTopologicalBasis
      (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) := by
    exact hBbasis.induced (Subtype.val : A → K)
  have hSeparate : ∀ x y : A, x ≠ y → ∃ U ∈ B,
    x ∈ (Subtype.val : A → K) ⁻¹' U ∧
    y ∉ (Subtype.val : A → K) ⁻¹' U := by
    intro x y hxy
    have hpos : 0 < dist (x : K) (y : K) := dist_pos.mpr (by
      intro h
      apply hxy
      exact Subtype.ext h)
    have hxball : (x : K) ∈ Metric.ball (x : K) (dist (x : K) (y : K)) := by
      rw [Metric.mem_ball]
    simp using hpos
    have hyball : (y : K) ∉ Metric.ball (x : K) (dist (x : K) (y : K)) := by
      intro hy
```

```

have hlt : dist (x : K) (y : K) < dist (x : K) (y : K) := by
  simpa [Metric.mem_ball, dist_comm] using hy
exact (lt_irrefl _ hlt)
obtain ⟨U, hU, hxU, hUsub⟩ :=
  hBbasis.exists_subset_of_mem_open hxball Metric.isOpen_ball
refine ⟨U, hU, hxU, ?_⟩
intro hyU
exact hyball (hUsub hyU)
exact ⟨B, hBcount, hBopen, hTraceBasis, hSeparate⟩

```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Rudin_exercise_2_25__me.mh.me__129dda3a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 129dda3a21fafe47

4.112 112. closed_subspace_countable_trace_basis_union

The problem

Let K be a compact metric space and A a closed subset of K . Show that there is a countable family of ambient open sets whose traces form a basis for A , and that every open subset of A is the union of the traces contained in it.

Lean statement

```
theorem closed_subspace_countable_trace_basis_union {K : Type*} [MetricSpace K] [CompactSpace K] {A : Set K} (hA : IsClosed A) :
  ∃ (B : Set (Set K)), Set.Countable B ∧
    (∀ U ∈ B, IsOpen U) ∧
    TopologicalSpace.IsTopologicalBasis
      (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) ∧
    (∀ V : Set A, IsOpen V →
      V = ⋃₀ {T : Set A |
        T ∈ Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B ∧ T ⊆ V})
```

Parent. Let K be a compact metric space and A a closed subset of K . Show that there is a countable family of ambient open sets whose traces form a basis for A and separate any two distinct points of A .

Parent in Lean

```
theorem closed_subspace_countable_separating_trace_basis {K : Type*} [MetricSpace K] [CompactSpace K] {A : Set K} (hA : IsClosed A) :
  ∃ (B : Set (Set K)), Set.Countable B ∧
    (∀ U ∈ B, IsOpen U) ∧
    TopologicalSpace.IsTopologicalBasis
      (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) ∧
    (∀ x y : A, x ≠ y → ∃ U ∈ B,
      x ∈ (Subtype.val : A → K) ⁻¹' U ∧
      y ∉ (Subtype.val : A → K) ⁻¹' U)
```

What it contributes. Supplied the compact-subtype countable basis construction, ambient open representatives, and trace-basis identification; the child adds the explicit open-set union representation as the new final obligation.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child replaces the parent's point-separation conclusion with an explicit union representation for every open subset of the subtype. Proving this requires the basis characterization and a new set-theoretic argument, so recalling the parent's final separation proof is insufficient.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a substantive basis characterization: every open subset of the closed subtype is explicitly represented as the union of contained traces. Proving this final obligation requires new membership and subset reasoning beyond the parent's separation argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the parent's point-separation conclusion with an explicit representation of every open subset as a union of contained trace basis elements. Its final obligation uses the basis characterization and requires new membership and union reasoning, so recalling the parent's separation argument is insufficient.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2767 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem closed_subspace_countable_trace_basis_union.extracted_1.1.{u_1} {K : Type u_1} [inst : MetricSpace K]
[inst_1 : CompactSpace K] {A : Set K} (hA : IsClosed A) :
  ∃ B,
    B.Countable ∧
    (∀ U ∈ B, IsOpen U) ∧
    IsTopologicalBasis ((fun U => Subtype.val ⁻¹' U) '' B) ∧
    ∀ (V : Set ↑A), IsOpen V → V = ⋃₀ {T | T ∈ (fun U => Subtype.val ⁻¹' U) '' B ∧ T ⊆ V}
```

Read back by a model that saw only the Lean. *Let K be a compact metric space and let A be a closed subset of K . Then there exists a countable collection B of subsets of K such that every U in B is open in K , the collection of traces $\{U \cap A : U \in B\}$ is a topological basis for the subspace A , and every open subset V of A is the union of all basis elements T from this collection that are contained in V .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem closed_subspace_countable_trace_basis_union {K : Type*} [MetricSpace K] [CompactSpace K] {A : Set K} (hA : IsClosed A) :
  ∃ (B : Set (Set K)), Set.Countable B ∧
    (∀ U ∈ B, IsOpen U) ∧
    TopologicalSpace.IsTopologicalBasis
      (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) ∧
    (∀ V : Set A, IsOpen V →
      V = ⋃₀ {T : Set A |
        T ∈ Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B ∧ T ⊆ V}) := by
  classical
  letI : CompactSpace A := isCompact_iff_compactSpace.mp hA.isCompact
  obtain ⟨C, hCcount, _hCne, hCbasis⟩ :=
    TopologicalSpace.exists_countable_basis (α := A)
  have hRep : ∀ c ∈ C, ∃ U : Set K, IsOpen U ∧
    (Subtype.val : A → K) ⁻¹' U = c := by
    intro c hc
    exact isOpen_induced_iff.mp (hCbasis.isOpen hc)
  choose f hfopen hftrace using hRep
  let g : Set A → Set K := fun c => if hc : c ∈ C then f c hc else ∅
  let B : Set (Set K) := g '' C
  have hBcount : B.Countable := hCcount.image g
  have hBopen : ∀ U ∈ B, IsOpen U := by
    rintro U ⟨c, hc, rfl⟩
    simpa [g, dif_pos hc] using hfopen c hc
  have htrace : Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B = C := by
    apply Set.Subset.antisymm
    · rintro V ⟨U, ⟨c, hc, rfl⟩, rfl⟩
      simpa [g, dif_pos hc, hftrace c hc]
    · intro c hc
      refine ⟨g c, ⟨c, hc, rfl⟩, ?_⟩
      simpa [g, dif_pos hc, hftrace c hc]
  have hBasis : TopologicalSpace.IsTopologicalBasis
    (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) := by
```

```

    rw [htrace]
    exact hCbasis
have hUnion : ∀ V : Set A, IsOpen V →
  V = ⋃ {T : Set A |
    T ∈ Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B ∧ T ⊆ V} := by
  intro V hV
  ext x
  constructor
  · intro hx
    obtain ⟨c, hc, hxc, hsub⟩ := hCbasis.exists_subset_of_mem_open hx hV
    have hcD : c ∈ {T : Set A |
      T ∈ Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B ∧ T ⊆ V} := by
      refine ⟨?_, hsub⟩
      rw [htrace]
      exact hc
    exact Set.mem_sUnion.mpr ⟨c, hcD, hxc⟩
  · intro hx
    obtain ⟨T, hTD, hxT⟩ := Set.mem_sUnion.mp hx
    exact hTD.2 hxT
exact ⟨B, hBcount, hBopen, hBasis, hUnion⟩

```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Rudin_exercise_2_25__me.mh.me__409c12da

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propept, Classical.choice, Quot.sound, fingerprint 409c12da748b86fc

4.113 113. compact_core_countable_trace_basis

The problem

In a compact metric space, every neighborhood of a point contains a compact neighborhood whose relative neighborhoods at that point are refined by traces of a countable ambient basis.

Lean statement

```
theorem compact_core_countable_trace_basis {X : Type*} [MetricSpace X] [CompactSpace X]
  (x : X) (U : Set X) (hU : U ∈ nhds x) :
  ∃ V : Set X, V ∈ nhds x ∧ IsCompact V ∧ V ⊆ U ∧
  ∃ B : Set (Set X), Set.Countable B ∧ TopologicalSpace.IsTopologicalBasis B ∧
  ∀ W ∈ nhds x, ∃ b ∈ B, x ∈ b ∧ b ∩ V ⊆ W ∩ V
```

Parent. Prove that every compact metric space K has a countable base.

Parent in Lean

```
theorem Rudin_exercise_2.25 {K : Type*} [MetricSpace K] [CompactSpace K] :
  ∃ (B : Set (Set K)), Set.Countable B ∧ IsTopologicalBasis B := by
```

What it contributes. Supplies the second-countability/countable-basis checkpoint via `inferInstance` and `TopologicalSpace.exists_countable_basis`.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child adds genuinely new local compactness reasoning: it extracts a smaller closed-ball neighborhood inside U and proves its compactness, then establishes a relative trace refinement for every neighborhood. The parent's global countable-basis existence alone cannot supply these claims.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds genuinely new mathematics: constructing a compact closed-ball neighborhood inside U and proving its relative neighborhood trace property. The parent only supplies a countable topological basis, so recalling its proof does not establish the compact-core construction.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds genuinely new mathematics: it constructs a compact neighborhood contained in an arbitrary neighborhood and proves a relative trace-basis refinement property. The parent's global countable-basis existence supplies only the basis component and does not provide the metric compact-core argument or local containment reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2914 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem compact_core_countable_trace_basis.extracted_1_1.{u_1} {X : Type u_1} [inst : MetricSpace X]
[inst_1 : CompactSpace X] (x : X) (U : Set X) (hU : U ∈ ℳ x) :
  ∃ V ∈ ℳ x,
    IsCompact V ∧ V ⊆ U ∧ ∃ B, B.Countable ∧ IsTopologicalBasis B ∧ ∀ W ∈ ℳ x, ∃ b ∈ B, x ∈ b ∧ b ∩ V ⊆ W ∩ V
```

Read back by a model that saw only the Lean. *Let X be a type equipped with a metric and a compact-space structure, let x be a point of X , and let U be a set containing a neighborhood of x . Then there exists a set V containing a neighborhood of x such that V is compact and V is contained in U , and there exists a countable topological basis B for X such that, for every neighborhood W of x , there is some b in B with $x \in b$ and $b \cap V \subseteq W \cap V$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem compact_core_countable_trace_basis {X : Type*} [MetricSpace X] [CompactSpace X]
(x : X) (U : Set X) (hU : U ∈ nhds x) :
  ∃ V : Set X, V ∈ nhds x ∧ IsCompact V ∧ V ⊆ U ∧
  ∃ B : Set (Set X), Set.Countable B ∧ TopologicalSpace.IsTopologicalBasis B ∧
  ∀ W ∈ nhds x, ∃ b ∈ B, x ∈ b ∧ b ∩ V ⊆ W ∩ V := by
  classical
  rcases Metric.mem_nhds_iff.mp hU with ⟨r, hr, hball⟩
  let V : Set X := Metric.closedBall x (r / 2)
  have hVsub : V ⊆ U := by
    intro y hy
    apply hball
    change dist y x ≤ r / 2 at hy
    change dist y x < r
    linarith
  have hVnhds : V ∈ nhds x := by
    refine Filter.mem_of_superset (show Metric.ball x (r / 2) ∈ nhds x from Metric.ball_mem_nhds x (by linarith)) ?_
    intro y hy
    change dist y x < r / 2 at hy
    change dist y x ≤ r / 2
    exact le_of_lt hy
  have hVcompact : IsCompact V := by
    change IsCompact (Metric.closedBall x (r / 2))
    exact ProperSpace.isCompact_closedBall x (r / 2)
  have hSecond : SecondCountableTopology X := inferInstance
  obtain ⟨B, hBcount, -, hBbasis⟩ := TopologicalSpace.exists_countable_basis (α := X)
  have hTrace : ∀ W ∈ nhds x, ∃ b ∈ B, x ∈ b ∧ b ∩ V ⊆ W ∩ V := by
    intro W hW
    rcases hBbasis.mem_nhds_iff.mp hW with ⟨b, hbB, hxb, hbW⟩
    exact ⟨b, hbB, hxb, fun y hy => ⟨hbW hy.1, hy.2⟩⟩
  exact ⟨V, hVnhds, hVcompact, hVsub, B, hBcount, hBbasis, hTrace⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_2_25__me__71368725

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fin-printer 71368725e7401bae

4.114 114. localized_compact_core_basis

The problem

Let X be a compact metric space, $x \in X$, and U an open neighborhood of x . Show that some compact neighborhood core $K \subseteq U$ has a countable basis whose members are traces of ambient subsets contained in K .

Lean statement

```
theorem localized_compact_core_basis {X : Type*} [MetricSpace X] [CompactSpace X] (x : X) (U : Set X) (hU : IsOpen U) (hxU : x ∈ U) :
  ∃ K : Set X, IsCompact K ∧ x ∈ interior K ∧ K ⊆ U ∧
    ∃ C : Set (Set K), Set.Countable C ∧ TopologicalSpace.IsTopologicalBasis C ∧
      ∃ r : Set K → Set X, ∀ s ∈ C,
        r s ⊆ K ∧ {y : K | (y : X) ∈ r s} = s
```

Parent. Prove that every compact metric space K has a countable base.

Parent in Lean

```
theorem Rudin_exercise_2_25 {K : Type*} [MetricSpace K] [CompactSpace K] :
  ∃ (B : Set (Set K)), Set.Countable B ∧ IsTopologicalBasis B := by
```

What it contributes. Supplied the countable-basis checkpoint, instantiated on the compact-core subtype after consuming the compactness witness.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child requires constructing a compact neighborhood core inside the given open set and then transporting a countable basis from the compact subtype to ambient representatives. The parent only supplies a countable basis for the whole compact space and does not provide either the localized core or the subtype trace argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires a new compact-neighborhood construction inside the open set and an explicit subtype-to-ambient representative argument. The parent's countable-basis result alone does not supply either localization or the representation equality, so recalling the parent is insufficient.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a substantially stronger localized compact-core theorem, including an induced-subtype basis and ambient representatives. The parent only supplies an abstract countable basis for the whole compact metric space and does not provide the compact refinement or localization argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2908 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem localized_compact_core_basis.extracted_1_1.{u_1} {X : Type u_1} [inst : MetricSpace X] [inst_1 : CompactSpace X]
  (x : X) (U : Set X) (hU : IsOpen U) (hxU : x ∈ U) :
  ∃ K,
    IsCompact K ∧
    x ∈ interior K ∧
    K ⊆ U ∧ ∃ C, C.Countable ∧ IsTopologicalBasis C ∧ ∃ r, ∀ s ∈ C, r s ⊆ K ∧ {y | ↑y ∈ r s} = s
```

Read back by a model that saw only the Lean. Let X be a compact metric space, let x be a point of X , and let U be an open subset of X containing x . Then there exist a compact subset K of X and a countable collection C of subsets of the subspace K such that x lies in the interior of K , K is contained in U , C is a topological basis for K , and there exists an assignment r sending each $s \in C$ to a subset $r(s)$ of X with $r(s) \subseteq K$ and such that s is exactly the set of points y of K whose images in X belong to $r(s)$, i.e. $\{y \mid \uparrow y \in r(s)\} = s$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem localized_compact_core_basis {X : Type*} [MetricSpace X] [CompactSpace X] (x : X) (U : Set X) (hU : IsOpen U) (hxU : x ∈ U) :
  ∃ K : Set X, IsCompact K ∧ x ∈ interior K ∧ K ⊆ U ∧
    ∃ C : Set (Set K), Set.Countable C ∧ TopologicalSpace.IsTopologicalBasis C ∧
      ∃ r : Set K → Set X, ∀ s ∈ C,
        r s ⊆ K ∧ {y : K | (y : X) ∈ r s} = s := by
  classical
  obtain ⟨K, hK, hxK, hKU⟩ := exists_compact_subset hU hxU
  let I : CompactSpace K := isCompact_iff_compactSpace.mp hK
  obtain ⟨C, hC, hCempty, hBasis⟩ := ∃ C : Set (Set K), Set.Countable C ∧ ∅ ∉ C ∧ TopologicalSpace.IsTopologicalBasis C :=
    TopologicalSpace.exists_countable_basis K
  refine ⟨K, hK, hxK, hKU, C, hC, hBasis, ?_⟩
  let r : Set K → Set X := fun t => Set.image (fun y : K => (y : X)) t
  refine ⟨r, ?_⟩
  intro s hs
  refine ⟨?, ?_⟩
  · rintro z ⟨y, hy, rfl⟩
    exact y.property
  · ext y
    constructor
    · intro hyPre
      obtain ⟨z, hz, hzy⟩ := hyPre
      have heq : z = y := Subtype.ext hzy
      simpa [heq] using hz
    · intro hyMem
      exact ⟨y, hyMem, rfl⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_2_25__me__c765a259

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint c765a2597f9807ad

4.115 115. closed_subspace_countable_trace_basis

The problem

Let K be a compact metric space and A a closed subset of K . Construct a countable family of ambient open sets whose traces on A form a topological basis for A .

Lean statement

```
theorem closed_subspace_countable_trace_basis {K : Type*} [MetricSpace K] [CompactSpace K] {A : Set K} (hA : IsClosed A) :
  ∃ (B : Set (Set K)), Set.Countable B ∧
    (∀ U ∈ B, IsOpen U) ∧
    TopologicalSpace.IsTopologicalBasis
      (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B)
```

Parent. Prove that every compact metric space K has a countable base.

Parent in Lean

```
theorem Rudin_exercise_2_25 {K : Type*} [MetricSpace K] [CompactSpace K] :
  ∃ (B : Set (Set K)), Set.Countable B ∧ IsTopologicalBasis B := by
```

What it contributes. Supplies the compact-metric countable-basis checkpoint, transferred here to the closed subtype A and then lifted through ambient-open traces.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the parent to a closed subspace: it constructs ambient open representatives and proves their traces form a basis on the subtype. This requires induced-topology reasoning and lifting basis elements, so recalling the parent's ambient-basis proof is insufficient.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the object from an ambient countable basis on K to a countable family of open subsets of K whose traces form a basis on the closed subspace A . Its proof requires compactness and second-countability of A plus representing induced-open sets by ambient opens, which is not supplied by recalling the parent proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the ambient space by a closed subspace and requires constructing countably many ambient open representatives whose traces form a basis. This needs induced-topology reasoning and lifting basis elements, which the parent's direct second-countable-basis packaging does not provide.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2740 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem closed_subspace_countable_trace_basis.extracted_1.1.{u_1} {K : Type u_1} [inst : MetricSpace K]
  [inst.1 : CompactSpace K] {A : Set K} (hA : IsClosed A) :
  ∃ B, B.Countable ∧ (∀ U ∈ B, IsOpen U) ∧ IsTopologicalBasis ((fun U => Subtype.val ⁻¹' U) '' B)
```

Read back by a model that saw only the Lean. *Let K be a compact metric space and let $A \subseteq K$ be closed. Then there exists a countable collection B of open subsets of K such that the collection of traces $\{A \cap U \mid U \in B\}$, viewed as subsets of the subspace A , is a topological basis for A .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem closed_subspace_countable_trace_basis {K : Type*} [MetricSpace K] [CompactSpace K] {A : Set K} (hA : IsClosed A) :
  ∃ (B : Set (Set K)), Set.Countable B ∧
    (∀ U ∈ B, IsOpen U) ∧
    TopologicalSpace.IsTopologicalBasis
      (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) := by
  classical
  let I : CompactSpace A := isCompact_iff_compactSpace.mp hA.isCompact
  have hSecond : SecondCountableTopology A := inferInstance
  obtain ⟨C, hCcount, hCne, hCbasis⟩ :=
    TopologicalSpace.exists_countable_basis (α := A)
  have hRep : ∀ c ∈ C, ∃ U : Set K, IsOpen U ∧ (Subtype.val : A → K) ⁻¹' U = c := by
    intro c hc
    exact isOpen_induced_iff.mp (hCbasis.isOpen hc)
  choose f hfopen hftrace using hRep
  let g : Set A → Set K := fun c => if hc : c ∈ C then f c hc else ∅
  let B : Set (Set K) := g '' C
  have hBcount : B.Countable := hCcount.image g
  have hBopen : ∀ U ∈ B, IsOpen U := by
    rintro U ⟨c, hc, rfl⟩
    simpa [g, dif_pos hc] using hfopen c hc
  have htrace : Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B = C := by
    apply Set.Subset.antisymm
    · rintro V ⟨U, ⟨c, hc, rfl⟩, rfl⟩
      simpa [g, dif_pos hc, hftrace c hc]
    · intro c hc
      refine ⟨g c, ⟨c, hc, rfl⟩, ?_⟩
      simpa [g, dif_pos hc, hftrace c hc]
  refine ⟨B, hBcount, hBopen, ?_⟩
  simpa [htrace] using hCbasis
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_2_25__me__ee014913

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint ee01491334364c9b

4.116 116. uniformContinuousOn_bounded_chain

The problem

Let $(f_k)_{k \geq 0}$ be a sequence of self-maps of a uniform space and let S be a subset of that space. If each f_k is uniformly continuous on the image of S after the preceding k stages, prove that the composite of the first n stages is uniformly continuous on S .

Lean statement

```
theorem uniformContinuousOn_bounded_chain
  {α : Type*} [UniformSpace α]
  {s : Set α} (f : ℕ → α → α) (n : ℕ)
  (hcont : ∀ k, UniformContinuousOn (f k)
    ((Nat.rec (id : α → α) (fun j g => f j ∘ g) k) '' s)) :
  UniformContinuousOn
    (Nat.rec (id : α → α) (fun j g => f j ∘ g) n) s
```

Parent. Let f , g , and h be successive maps, and let S be a subset of the source space. If each map is uniformly continuous on the image relevant to S , prove that $h \circ g \circ f$ is uniformly continuous on S .

Parent in Lean

```
theorem uniformContinuousOn_three_stage_comp
  {α β γ δ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [UniformSpace δ]
  {s : Set α} {f : α → β} {g : β → γ} {h : γ → δ}
  (hf : UniformContinuousOn f s)
  (hg : UniformContinuousOn g (f '' s))
  (hh : UniformContinuousOn h ((g ∘ f) '' s)) :
  UniformContinuousOn (h ∘ g ∘ f) s
```

What it contributes. Generalizes the parent's image-local composition step from three maps to an inductively indexed finite chain, retaining its MapsTo and UniformContinuousOn.comp checkpoints.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the fixed three-stage composition to an arbitrary finite chain and requires an induction proving the successive-image invariant. A solver recalling the parent would still need to construct and manage this recursive invariant.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed three-stage composition to an arbitrary finite chain and requires induction to maintain the successive-image invariant. A memorized proof of the parent does not supply this inductive argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes the fixed three-stage composition to an arbitrary finite chain and requires induction to establish the successive-image invariant. The parent's direct two-step composition proof does not supply this inductive reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry

Check	By	Result
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2900 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem uniformContinuousOn_bounded_chain.extracted_1.1.{u_1} {α : Type u_1} [inst : UniformSpace α] {s : Set α}
(f : ℕ → α → α) (n : ℕ)
(hcont : ∀ (k : ℕ), UniformContinuousOn (f k) (Nat.rec (motive := fun x => α → α) id (fun j g => f j ∘ g) k ' ' s)) :
UniformContinuousOn (Nat.rec (motive := fun x => α → α) id (fun j g => f j ∘ g) n) s
```

Read back by a model that saw only the Lean. *Let α be a type equipped with a uniform space, let $s \subseteq \alpha$, let f be a sequence of functions $f_k : \alpha \rightarrow \alpha$ indexed by natural numbers, and let n be a natural number. Assume that for every natural number k , the function f_k is uniformly continuous on the image of s under the function obtained by recursively composing the functions $f_0, f_1, \dots, f_{k-1}$, starting from the identity function. Then the function obtained by recursively composing the first n functions in this way is uniformly continuous on s .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem uniformContinuousOn_bounded_chain
{α : Type*} [UniformSpace α]
{s : Set α} (f : ℕ → α → α) (n : ℕ)
(hcont : ∀ k, UniformContinuousOn (f k)
  ((Nat.rec (id : α → α) (fun j g => f j ∘ g) k) ' ' s)) :
UniformContinuousOn
  (Nat.rec (id : α → α) (fun j g => f j ∘ g) n) s := by
induction n with
| zero =>
  have hid : UniformContinuousOn (id : α → α) s :=
    uniformContinuous_id.uniformContinuousOn
  simpa using hid
| succ n ih =>
  have hmap : Set.Mapsto
    (Nat.rec (id : α → α) (fun j g => f j ∘ g) n)
    s
    ((Nat.rec (id : α → α) (fun j g => f j ∘ g) n) ' ' s) := by
    intro x hx
    exact ⟨x, hx, rfl⟩
  have hstep : UniformContinuousOn
    (f n ∘ Nat.rec (id : α → α) (fun j g => f j ∘ g) n) s := by
    exact (hcont n).comp ih hmap
  simpa [Function.comp_def] using hstep
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Rudin_exercise_4_12__me.mh.me__47be696d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint 47be696d25904cea

4.117 117. three_stage_uniformContinuousOn_comp

The problem

Let f , g , and h be three successive maps. If f is uniformly continuous, g is uniformly continuous on the image of f , and h is uniformly continuous on the image of $g \circ f$, prove that $h \circ g \circ f$ is uniformly continuous.

Lean statement

```
theorem three_stage_uniformContinuousOn_comp
  {α β γ δ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [UniformSpace δ]
  {f : α → β} {g : β → γ} {h : γ → δ}
  (hf : UniformContinuous f)
  (hg : UniformContinuousOn g (Set.range f))
  (hh : UniformContinuousOn h (Set.range (g ∘ f))) :
  UniformContinuous (h ∘ g ∘ f)
```

Parent. A uniformly continuous function of a uniformly continuous function is uniformly continuous.

Parent in Lean

```
theorem Rudin_exercise_4_12
  {α β γ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ]
  {f : α → β} {g : β → γ}
  (hf : UniformContinuous f) (hg : UniformContinuous g) :
  UniformContinuous (g ∘ f) := by
```

What it contributes. Generalizes binary composition to a three-stage composition while replacing the later global hypotheses with uniform continuity on the successive images.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely specializes global uniform continuity to uniform continuity on the successive ranges, requiring MapsTo obligations and UniformContinuousOn composition through the intermediate image. The parent's direct 'hg.comp hf' argument does not supply this restricted-domain reasoning, and all hypotheses are essential.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely specializes the composition theorem: it derives global uniform continuity from local uniform continuity of the later maps on the successive ranges. This requires range mapping and UniformContinuousOn composition reasoning that the parent's direct global composition lemma does not provide.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely specializes continuity to successive image sets and requires composing UniformContinuousOn hypotheses with explicit range-mapping arguments. The parent's global composition lemma does not supply this restricted-domain reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing

Check	By	Result
corpus_dedup	hash	unique against 2863 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem three_stage_uniformContinuousOn_comp.extracted_1_1.{u_4, u_3, u_2, u_1} {α : Type u_1} {β : Type u_2}
{γ : Type u_3} {δ : Type u_4} [inst : UniformSpace α] [inst_1 : UniformSpace β] [inst_2 : UniformSpace γ]
[inst_3 : UniformSpace δ] {f : α → β} {g : β → γ} {h : γ → δ} (hf : UniformContinuous f)
(hg : UniformContinuousOn g (Set.range f)) (hh : UniformContinuousOn h (Set.range (g ∘ f))) :
UniformContinuous (h ∘ g ∘ f)
```

Read back by a model that saw only the Lean. Let α, β, γ , and δ be uniform spaces, and let $f: \alpha \rightarrow \beta$, $g: \beta \rightarrow \gamma$, and $h: \gamma \rightarrow \delta$. Suppose f is uniformly continuous, g is uniformly continuous on the range of f , and h is uniformly continuous on the range of $g \circ f$. Then the composition $h \circ g \circ f: \alpha \rightarrow \delta$ is uniformly continuous.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem three_stage_uniformContinuousOn_comp
{α β γ δ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [UniformSpace δ]
{f : α → β} {g : β → γ} {h : γ → δ}
(hf : UniformContinuous f)
(hg : UniformContinuousOn g (Set.range f))
(hh : UniformContinuousOn h (Set.range (g ∘ f))) :
UniformContinuous (h ∘ g ∘ f) := by
have hmap_f : Set.MapsTo f Set.univ (Set.range f) := by
  intro x hx
  exact ⟨x, rfl⟩
have hgf : UniformContinuousOn (g ∘ f) Set.univ := by
  exact hg.comp hf.uniformContinuousOn hmap_f
have hmap_gf : Set.MapsTo (g ∘ f) Set.univ (Set.range (g ∘ f)) := by
  intro x hx
  exact ⟨x, rfl⟩
have hfinal_on : UniformContinuousOn (h ∘ (g ∘ f)) Set.univ := by
  exact hh.comp hgf hmap_gf
apply uniformContinuousOn_univ.mp
simpa [Function.comp_def] using hfinal_on
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_4_12__me__a36d04eb

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint a36d04eb5dda70f9

4.118 118. uniformContinuous_paired_two_stage_comp

The problem

If f is uniformly continuous, g is uniformly continuous on a set t containing the range of f , and h is uniformly continuous on the range of $g \circ f$, prove that $x \mapsto (g(f(x)), h(g(f(x))))$ is uniformly continuous.

Lean statement

```
theorem uniformContinuous_paired_two_stage_comp
  {α β γ δ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [UniformSpace δ]
  {f : α → β} {g : β → γ} {h : γ → δ} {t : Set β}
  (hf : UniformContinuous f)
  (hg : UniformContinuousOn g t)
  (hft : Set.range f ⊆ t)
  (hh : UniformContinuousOn h (Set.range (g ∘ f))) :
  UniformContinuous (fun x => (g (f x), h (g (f x))))
```

Parent. If f is uniformly continuous, g is uniformly continuous on a set containing the range of f , and h is uniformly continuous on the range of $g \circ f$, then $h \circ (g \circ f)$ is uniformly continuous.

Parent in Lean

```
theorem uniformContinuous_two_stage_comp_of_on_ambient
  {α β γ δ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [UniformSpace δ]
  {f : α → β} {g : β → γ} {h : γ → δ} {t : Set β}
  (hf : UniformContinuous f)
  (hg : UniformContinuousOn g t)
  (hft : Set.range f ⊆ t)
  (hh : UniformContinuousOn h (Set.range (g ∘ f))) :
  UniformContinuous (h ∘ (g ∘ f))
```

What it contributes. Supplied the ambient-range composition checkpoints for deriving $\text{UniformContinuous } (g \circ f)$ and $\text{UniformContinuous } (h \circ (g \circ f))$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the target to a product-valued uniform-continuity claim. It must derive both intermediate maps and use them together via ‘prodMk’, which the parent’s proof does not establish.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely adds a product-valued uniform-continuity conclusion: both $\text{UniformContinuous } (g \circ f)$ and $\text{UniformContinuous } (h \circ (g \circ f))$ are derived and then combined via prodMk. The parent’s final composition proof alone does not supply this product-map step.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely proves a product-valued uniform-continuity statement: it derives uniform continuity of $g \circ f$, independently derives $h \circ (g \circ f)$, and combines both via prodMk. The parent’s final composition proof does not supply the product-map step or make the first-stage map a separate required component.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry

Check	By	Result
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2020 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem uniformContinuous_paired_two_stage_comp.extracted_1_1.{u_4, u_3, u_2, u_1} {α : Type u_1} {β : Type u_2}
{γ : Type u_3} {δ : Type u_4} [inst : UniformSpace α] [inst_1 : UniformSpace β] [inst_2 : UniformSpace γ]
[inst_3 : UniformSpace δ] {f : α → β} {g : β → γ} {h : γ → δ} {t : Set β} (hf : UniformContinuous f)
(hg : UniformContinuousOn g t) (hft : Set.range f ⊆ t) (hh : UniformContinuousOn h (Set.range (g ∘ f))) :
UniformContinuous fun x => (g (f x), h (g (f x)))
```

Read back by a model that saw only the Lean. Let α, β, γ , and δ be uniform spaces, let $f : \alpha \rightarrow \beta$, $g : \beta \rightarrow \gamma$, and $h : \gamma \rightarrow \delta$. Suppose f is uniformly continuous, g is uniformly continuous on a set $t \subseteq \beta$, the range of f is contained in t , and h is uniformly continuous on the range of $g \circ f$. Then the map from α to $\gamma \times \delta$ defined by $x \mapsto (g(f(x)), h(g(f(x))))$ is uniformly continuous.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem uniformContinuous_paired_two_stage_comp
{α β γ δ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [UniformSpace δ]
{f : α → β} {g : β → γ} {h : γ → δ} {t : Set β}
(hf : UniformContinuous f)
(hg : UniformContinuousOn g t)
(hft : Set.range f ⊆ t)
(hh : UniformContinuousOn h (Set.range (g ∘ f))) :
UniformContinuous (fun x => (g (f x), h (g (f x)))) := by
have hgf_on : UniformContinuousOn (g ∘ f) Set.univ := by
  refine hg.comp hf.uniformContinuousOn ?_
  intro x hx
  exact hft ⟨x, rfl⟩
have hgf : UniformContinuous (g ∘ f) := uniformContinuousOn_univ.mp hgf_on
have hcomp_on : UniformContinuousOn (h ∘ (g ∘ f)) Set.univ := by
  refine hh.comp hgf.uniformContinuousOn ?_
  intro x hx
  exact ⟨x, rfl⟩
have hcomp : UniformContinuous (h ∘ (g ∘ f)) := uniformContinuousOn_univ.mp hcomp_on
have hpair : UniformContinuous (fun x => ((g ∘ f) x, (h ∘ (g ∘ f)) x)) := by
  exact hgf.prodMk hcomp
simpa [Function.comp_def] using hpair
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Rudin_exercise_4_12__mh.me.me__50d0cc9a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint b14ec291665fff5e

4.119 119. iterate_cauchy

The problem

Let (u_i) be a Cauchy net and let f be a uniformly continuous self-map. Show that for every natural number n , the net obtained by applying f n times to each u_i is Cauchy.

Lean statement

```
theorem iterate_cauchy
  {α ι : Type*} [UniformSpace α] [Preorder ι]
  {f : α → α} {u : ι → α}
  (hf : UniformContinuous f) (hu : CauchySeq u) :
  ∀ n : ℕ, CauchySeq ((f^[n]) ∘ u)
```

Parent. Let (u_i) be a Cauchy net indexed by a preordered set. If f and g are uniformly continuous, show that $(g(f(u_i)))$ is Cauchy.

Parent in Lean

```
theorem composite_cauchy_net
  {α β γ ι : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [Preorder ι]
  {f : α → β} {g : β → γ} {u : ι → α}
  (hf : UniformContinuous f) (hg : UniformContinuous g) (hu : CauchySeq u) :
  CauchySeq ((g ∘ f) ∘ u)
```

What it contributes. Reuses the parent's uniform-continuity composition and Cauchy-sequence transport atoms, while adding induction over an arbitrary iterate depth.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the composition depth: it must induct on n , prove uniform continuity of each iterate, and transport the Cauchy property at every stage. The parent's one-step composition proof does not by itself supply this induction.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the iterate depth and proves the result by induction, establishing uniform continuity and Cauchy preservation at every iterate. This requires reasoning beyond the parent's one-step composition argument and is not recoverable by simply recalling the parent proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the iterate depth and requires induction to prove uniform continuity and Cauchy preservation for every iterate. The parent's two-map composition proof does not supply this induction or the iterate-recurrence reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2203 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem iterate_cauchy.extracted_1_1 : ∀ {α : Type u_1} {ι : Type u_2} [inst : UniformSpace α] [inst_1 : Preorder ι]
  {f : α → α} {u : ι → α}, UniformContinuous f → CauchySeq u → ∀ (n : ℕ), CauchySeq (f^[n] ∘ u)
```

Read back by a model that saw only the Lean. *For any type α equipped with a uniform space structure, any type ι equipped with a preorder, any function $f : \alpha \rightarrow \alpha$, and any function $u : \iota \rightarrow \alpha$, if f is uniformly continuous and u is a Cauchy sequence, then for every natural number n , the sequence obtained by applying the n -fold iterate of f to each term of u is also a Cauchy sequence.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem iterate_cauchy
  {α ι : Type*} [UniformSpace α] [Preorder ι]
  {f : α → α} {u : ι → α}
  (hf : UniformContinuous f) (hu : CauchySeq u) :
  ∀ n : ℕ, CauchySeq ((f^[n]) ∘ u) := by
  have hmain : ∀ n : ℕ, UniformContinuous (f^[n]) ∧ CauchySeq ((f^[n]) ∘ u) := by
    intro n
    induction n with
    | zero =>
      constructor
      · simpa using (uniformContinuous_id : UniformContinuous (id : α → α))
      · simpa using hu
    | succ n ih =>
      have hfu : CauchySeq (f ∘ u) := hf.comp_cauchySeq hu
      have hc : CauchySeq ((f^[n]) ∘ (f ∘ u)) :=
        ih.1.comp_cauchySeq hfu
      constructor
      · exact ih.1.comp hf
      · simpa [Function.comp_def, Function.iterate_succ_apply] using hc
  intro n
  exact (hmain n).2
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_4_12__mh.me__3a1560d5

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propept, Quot.sound, fingerprint 3a1560d5a16ba49d

4.120 120. uniformContinuousOn_comp_image

The problem

If f is uniformly continuous on s and g is uniformly continuous on the image of s under f , then $g \circ f$ is uniformly continuous on s .

Lean statement

```
theorem uniformContinuousOn_comp_image
  {α β γ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ]
  {s : Set α} {f : α → β} {g : β → γ}
  (hf : UniformContinuousOn f s)
  (hg : UniformContinuousOn g (f '' s)) :
  UniformContinuousOn (g ∘ f) s
```

Parent. If f is uniformly continuous and g is uniformly continuous on the range of f , then $g \circ f$ is uniformly continuous.

Parent in Lean

```
theorem uniformContinuous_comp_of_on_range
  {α β γ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ]
  {f : α → β} {g : β → γ}
  (hf : UniformContinuous f)
  (hg : UniformContinuousOn g (Set.range f)) :
  UniformContinuous (g ∘ f)
```

What it contributes. Supplied the composition-on-range checkpoint generalized from `Set.range f` to the image $f '' s$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the composition result from global uniform continuity and `Set.range f` to uniform continuity on an arbitrary domain s and the image $f '' s$. Its proof uses the restricted composition theorem and explicitly supplies the image witness $\langle x, hx, rfl \rangle$.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely localizes the composition theorem: it composes uniform continuity on s with continuity of g on the image $f '' s$, and constructs the required image witness $\langle x, hx, rfl \rangle$. The parent's global-domain proof does not supply this restricted-domain reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves the genuinely bounded composition law on an arbitrary set s , using `UniformContinuousOn` composition and the nontrivial image witness $\langle x, hx, rfl \rangle$. The parent only handles global uniform continuity with g restricted to `Set.range f`, so recalling it does not directly solve this local-image theorem.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over proptext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2021 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem uniformContinuousOn_comp_image.extracted_1_1.{u_3, u_2, u_1} {α : Type u_1} {β : Type u_2} {γ : Type u_3}
  [inst : UniformSpace α] [inst_1 : UniformSpace β] [inst_2 : UniformSpace γ] {s : Set α} {f : α → β} {g : β → γ}
  (hf : UniformContinuousOn f s) (hg : UniformContinuousOn g (f '' s)) : UniformContinuousOn (g ∘ f) s
```

Read back by a model that saw only the Lean. *Let α , β , and γ be uniform spaces, let s be a subset of α , let $f : \alpha \rightarrow \beta$, and let $g : \beta \rightarrow \gamma$. If f is uniformly continuous on s and g is uniformly continuous on the image $f(s) = \{f(x) \mid x \in s\}$, then the composition $g \circ f : \alpha \rightarrow \gamma$ is uniformly continuous on s .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem uniformContinuousOn_comp_image
  {α β γ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ]
  {s : Set α} {f : α → β} {g : β → γ}
  (hf : UniformContinuousOn f s)
  (hg : UniformContinuousOn g (f '' s)) :
  UniformContinuousOn (g ∘ f) s := by
  apply hg.comp hf
  intro x hx
  exact ⟨x, hx, rfl⟩
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_4_12__mh.me__c686e5f7

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint 74443254879dfcd9

4.121 121. agreement_iff_three_stage_agreement

The problem

Let three uniformly continuous maps be composed, and let the first map be replaced by another map on a set. If the latter two maps are injective, then the two first maps agree on the set exactly when their three-stage outputs agree there.

Lean statement

```
theorem agreement_iff_three_stage_agreement
  {α β γ δ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [UniformSpace δ]
  {f : α → β} {g : β → γ} {h : γ → δ}

  (hginj : Function.Injective g) (hhinj : Function.Injective h)
  (s : Set α) (m : α → β) :
  (∀ x, x ∈ s → m x = f x) ↔
  (∀ x, x ∈ s → h (g (m x)) = h (g (f x)))
```

Parent. Let three uniformly continuous maps be composed, and let the first map be replaced on a set by another map that agrees with it there. Prove that the resulting three-stage composition is uniformly continuous on that set.

Parent in Lean

```
theorem uniformContinuousOn_three_stage_local_agreement
  {α β γ δ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [UniformSpace δ]
  {f : α → β} {g : β → γ} {h : γ → δ}
  (hf : UniformContinuous f) (hg : UniformContinuous g) (hh : UniformContinuous h)
  (s : Set α) (m : α → β) (hm : ∀ x, x ∈ s → m x = f x) :
  UniformContinuousOn (fun x => h (g (m x))) s
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion to an agreement characterization and requires a genuinely new injectivity-cancellation argument through h and g. Its proof does not recall or reuse the parent's uniform-continuity proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the continuity conclusion into an iff characterization and requires genuinely new cancellation reasoning through injective h and g. Its forward direction is congruence, while the reverse direction cannot be obtained by recalling the parent's continuity proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion to an iff characterization and adds genuinely new injectivity-cancellation reasoning: equality after h and g is cancelled to recover $m x = f x$. This is not supplied by the parent's continuity proof and is not recall or decoration.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing

Check	By	Result
corpus_dedup	hash	unique against 2183 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem agreement_iff_three_stage_agreement.extracted_1_1.{u_4, u_3, u_2, u_1} {α : Type u_1} {β : Type u_2}
{γ : Type u_3} {δ : Type u_4} [inst : UniformSpace α] [inst_1 : UniformSpace β] [inst_2 : UniformSpace γ]
[inst_3 : UniformSpace δ] {f : α → β} {g : β → γ} {h : γ → δ} (hf : UniformContinuous f) (hg : UniformContinuous g)
(hh : UniformContinuous h) (hginj : Function.Injective g) (hhinj : Function.Injective h) (s : Set α) (m : α → β) :
(∀ x ∈ s, m x = f x) ↔ ∀ x ∈ s, h (g (m x)) = h (g (f x))
```

Read back by a model that saw only the Lean. Let α, β, γ , and δ be uniform spaces, let $f : \alpha \rightarrow \beta$, $g : \beta \rightarrow \gamma$, and $h : \gamma \rightarrow \delta$ be uniformly continuous, with g and h injective. For any subset s of α and any function $m : \alpha \rightarrow \beta$, the following are equivalent: for every $x \in s$, $m(x) = f(x)$; and for every $x \in s$, $h(g(m(x))) = h(g(f(x)))$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem agreement_iff_three_stage_agreement
  {α β γ δ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [UniformSpace δ]
  {f : α → β} {g : β → γ} {h : γ → δ}

  (hginj : Function.Injective g) (hhinj : Function.Injective h)
  (s : Set α) (m : α → β) :
  (∀ x, x ∈ s → m x = f x) ↔
    (∀ x, x ∈ s → h (g (m x)) = h (g (f x))) := by
  constructor
  · intro hm x hx
    rw [hm x hx]
  · intro houtputs x hx
    have hmiddle : g (m x) = g (f x) := hhinj (houtputs x hx)
    exact hginj hmiddle
```

Operator. mutation / mutation_easy **Lineage depth.** 3 **Certificate.** reproducible

Problem id. Rudin_exercise_4_12__ms.me.me.me__f7815124

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint ec1531061a228b3a

4.122 122. uniformContinuous_adjacent_stage_pair

The problem

If f , g , and h are uniformly continuous maps between uniform spaces, then the map $x \mapsto (g(f(x)), h(g(f(x))))$ is uniformly continuous.

Lean statement

```
theorem uniformContinuous_adjacent_stage_pair
  {α β γ δ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [UniformSpace δ]
  {f : α → β} {g : β → γ} {h : γ → δ}
  (hf : UniformContinuous f) (hg : UniformContinuous g) (hh : UniformContinuous h) :
  UniformContinuous (fun x => (g (f x), h (g (f x))))
```

Parent. If three maps are uniformly continuous, then their three-stage composition is uniformly continuous.

Parent in Lean

```
theorem uniformContinuous_three_stage
  {α β γ δ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [UniformSpace δ]
  {f : α → β} {g : β → γ} {h : γ → δ}
  (hf : UniformContinuous f) (hg : UniformContinuous g) (hh : UniformContinuous h) :
  UniformContinuous (fun x => h (g (f x)))
```

What it contributes. Uses the parent’s composition checkpoint pattern, extending it so the shared composite serves both product coordinates.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from uniform continuity of the final composite to uniform continuity of a paired function with intermediate and final outputs. Its proof requires the product-preservation lemma ‘prodMk’, so recalling the parent’s composition proof alone does not solve it.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the codomain from a single composition to a product-valued map and requires proving uniform continuity via ‘prodMk’, which the parent’s composition proof does not supply. It is not a forced wrapper, decoration, or repeat of the retained fold-composition theorem.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from continuity of a single final-stage output to continuity of a paired adjacent-stage map. It requires the new product-continuity argument via ‘prodMk’, which the parent proof does not supply, so recalling the parent is insufficient.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2153 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator goal_roundtrip	lean model	statement, axioms and kernel acceptance independently replayed MISMATCH: Statement B gives the codomain as $\beta \times \delta$, but the first component $g(f(x))$ has type γ , so the natural codomain is $\gamma \times \delta$.

Goal round-trip: what Lean elaborated

```
theorem uniformContinuous_adjacent_stage_pair.extracted_1_1.{u_4, u_3, u_2, u_1} {α : Type u_1} {β : Type u_2}
{γ : Type u_3} {δ : Type u_4} [inst : UniformSpace α] [inst_1 : UniformSpace β] [inst_2 : UniformSpace γ]
[inst_3 : UniformSpace δ] {f : α → β} {g : β → γ} {h : γ → δ} (hf : UniformContinuous f) (hg : UniformContinuous g)
(hh : UniformContinuous h) : UniformContinuous fun x => (g (f x), h (g (f x)))
```

Read back by a model that saw only the Lean. *Let α, β, γ , and δ be types equipped with uniform space structures. Let $f : \alpha \rightarrow \beta$, $g : \beta \rightarrow \gamma$, and $h : \gamma \rightarrow \delta$ be uniformly continuous functions. Then the function from α to $\beta \times \delta$ defined by $x \mapsto (g(f(x)), h(g(f(x))))$ is uniformly continuous.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem uniformContinuous_adjacent_stage_pair
{α β γ δ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [UniformSpace δ]
{f : α → β} {g : β → γ} {h : γ → δ}
(hf : UniformContinuous f) (hg : UniformContinuous g) (hh : UniformContinuous h) :
UniformContinuous (fun x => (g (f x), h (g (f x)))) := by
have hfg : UniformContinuous (g ∘ f) := hg.comp hf
have houter : UniformContinuous (h ∘ (g ∘ f)) := hh.comp hfg
have hpair : UniformContinuous (fun x => ((g ∘ f) x, (h ∘ (g ∘ f)) x)) :=
hfg.prodMk houter
simpa [Function.comp_def] using hpair
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Rudin_exercise_4_12__ms.me.me__1381c1ba

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 1381c1bab0e73398

4.123 123. uniformContinuousOn_three_stage_local_agreement

The problem

Let three uniformly continuous maps be composed, and let the first map be replaced on a set by another map that agrees with it there. Prove that the resulting three-stage composition is uniformly continuous on that set.

Lean statement

```
theorem uniformContinuousOn_three_stage_local_agreement
  {α β γ δ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [UniformSpace δ]
  {f : α → β} {g : β → γ} {h : γ → δ}
  (hf : UniformContinuous f) (hg : UniformContinuous g) (hh : UniformContinuous h)
  (s : Set α) (m : α → β) (hm : ∀ x, x ∈ s → m x = f x) :
  UniformContinuousOn (fun x => h (g (m x))) s
```

Parent. If three maps are uniformly continuous, then their three-stage composition is uniformly continuous.

Parent in Lean

```
theorem uniformContinuous_three_stage
  {α β γ δ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [UniformSpace δ]
  {f : α → β} {g : β → γ} {h : γ → δ}
  (hf : UniformContinuous f) (hg : UniformContinuous g) (hh : UniformContinuous h) :
  UniformContinuous (fun x => h (g (f x)))
```

What it contributes. Supplied the three-stage uniform-continuity composition checkpoints; the mutation adds local agreement and UniformContinuousOn.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely changes the claim to uniform continuity on a subset after replacing the first map by an arbitrary map that only agrees there. Its proof must use the agreement hypothesis to establish local continuity of the replacement before composing, which the parent's global composition proof does not provide.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires a genuinely new local-agreement argument: it proves uniform continuity of m only on s via congruence, then composes locally. The parent's global composition proof cannot be recalled to establish this specialized conclusion.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires new local reasoning: it transfers uniform continuity from f to an arbitrary map m using agreement on s , then composes via UniformContinuousOn. The parent's global composition proof does not supply this congruence or restricted-domain argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing

Check	By	Result
corpus_dedup	hash	unique against 2167 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem uniformContinuousOn_three_stage_local_agreement.extracted_1_1.{u_4, u_3, u_2, u_1} {α : Type u_1} {β : Type u_2}
{γ : Type u_3} {δ : Type u_4} [inst : UniformSpace α] [inst_1 : UniformSpace β] [inst_2 : UniformSpace γ]
[inst_3 : UniformSpace δ] {f : α → β} {g : β → γ} {h : γ → δ} (hf : UniformContinuous f) (hg : UniformContinuous g)
(hh : UniformContinuous h) (s : Set α) (m : α → β) (hm : ∀ x ∈ s, m x = f x) :
UniformContinuousOn (fun x => h (g (m x))) s
```

Read back by a model that saw only the Lean. Let α, β, γ , and δ be uniform spaces, let $f : \alpha \rightarrow \beta$, $g : \beta \rightarrow \gamma$, and $h : \gamma \rightarrow \delta$ be uniformly continuous functions, let s be a subset of α , and let $m : \alpha \rightarrow \beta$ satisfy $m(x) = f(x)$ for every $x \in s$. Then the function $x \mapsto h(g(m(x)))$ is uniformly continuous on s .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem uniformContinuousOn_three_stage_local_agreement
{α β γ δ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [UniformSpace δ]
{f : α → β} {g : β → γ} {h : γ → δ}
(hf : UniformContinuous f) (hg : UniformContinuous g) (hh : UniformContinuous h)
(s : Set α) (m : α → β) (hm : ∀ x, x ∈ s → m x = f x) :
UniformContinuousOn (fun x => h (g (m x))) s := by
have hmOn : UniformContinuousOn m s :=
  hf.uniformContinuousOn.congr (fun x hx => (hm x hx).symm)
have hgmOn : UniformContinuousOn (g ∘ m) s :=
  (show UniformContinuousOn g (Set.range m) from hg.uniformContinuousOn).comp hmOn (by
    intro x hx
    exact ⟨x, rfl⟩)
have hfinalOn : UniformContinuousOn (h ∘ (g ∘ m)) s :=
  (show UniformContinuousOn h (Set.range (g ∘ m)) from hh.uniformContinuousOn).comp hgmOn (by
    intro x hx
    exact ⟨x, rfl⟩)
have hrewrite : (fun x => h (g (m x))) = h ∘ (g ∘ m) := by
  funext x
  rfl
rw [hrewrite]
exact hfinalOn
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Rudin_exercise_4_12__ms.me.me__7c32138e

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint 7c32138eabe4522b

4.124 124. uniformContinuous_three_stage

The problem

If three maps are uniformly continuous, then their three-stage composition is uniformly continuous.

Lean statement

```
theorem uniformContinuous_three_stage
  {α β γ δ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [UniformSpace δ]
  {f : α → β} {g : β → γ} {h : γ → δ}
  (hf : UniformContinuous f) (hg : UniformContinuous g) (hh : UniformContinuous h) :
  UniformContinuous (fun x => h (g (f x)))
```

Parent. A uniformly continuous map followed by another uniformly continuous map is uniformly continuous, even when the intermediate map is named separately and agrees pointwise with the first.

Parent in Lean

```
theorem uniformContinuous_named_intermediate
  {α β γ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ]
  {f : α → β} {g : β → γ}
  (hf : UniformContinuous f) (hg : UniformContinuous g)
  (m : α → β) (hm : ∀ x, m x = f x) :
  UniformContinuous (fun x => g (m x))
```

What it contributes. Supplied the uniform-continuity composition principle used to construct the intermediate composite and final composition.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes from two composable uniformly continuous maps to three, requiring a new intermediate composite and a second composition argument. It is not a wrapper, decoration, or recall of the parent's exact proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the two-map composition law to three composable uniformly continuous maps, requiring a new nested composition step involving the final map. It has no forced witness, decoration, or unused hypothesis, and recalling the parent's proof does not directly solve the new theorem.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the composition law from two maps to three: it must first establish uniform continuity of $g \circ f$ and then compose with h . A solver recalling the parent still needs this additional composition step and the new associativity shape.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over proptext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2146 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem uniformContinuous_three_stage.extracted_1_1.{u_4, u_3, u_2, u_1} {α : Type u_1} {β : Type u_2} {γ : Type u_3}
{δ : Type u_4} [inst : UniformSpace α] [inst_1 : UniformSpace β] [inst_2 : UniformSpace γ] [inst_3 : UniformSpace δ]
{f : α → β} {g : β → γ} {h : γ → δ} (hf : UniformContinuous f) (hg : UniformContinuous g) (hh : UniformContinuous h) :
UniformContinuous fun x => h (g (f x))
```

Read back by a model that saw only the Lean. *Let α, β, γ , and δ be types equipped with uniform space structures. Let $f : \alpha \rightarrow \beta$, $g : \beta \rightarrow \gamma$, and $h : \gamma \rightarrow \delta$ be uniformly continuous functions. Then the function from α to δ defined by $x \mapsto h(g(f(x)))$ is uniformly continuous.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem uniformContinuous_three_stage
{α β γ δ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [UniformSpace δ]
{f : α → β} {g : β → γ} {h : γ → δ}
(hf : UniformContinuous f) (hg : UniformContinuous g) (hh : UniformContinuous h) :
UniformContinuous (fun x => h (g (f x))) := by
have hfg : UniformContinuous (g ∘ f) := hg.comp hf
have hfinal : UniformContinuous (h ∘ (g ∘ f)) := hh.comp hfg
have hrewrite : (fun x => h (g (f x))) = h ∘ (g ∘ f) := by
  funext x
  rfl
rw [hrewrite]
exact hfinal
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_4_12__ms.me__adb7544c

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint adb7544cb1cbe0c9

4.125 125. continuous_preimage_finite_common_pointwise_order_closed

The problem

Let $h : \beta \rightarrow X$ be continuous, and let a finite nonempty family of pairs of continuous real-valued functions be given on X . Prove that the inverse image under h of the set where every first function is at most its paired second function is closed in β .

Lean statement

```
theorem continuous_preimage_finite_common_pointwise_order_closed
  {β X ι : Type} [TopologicalSpace β] [TopologicalSpace X]
  (h : β → X) (hh : Continuous h)
  (s : Finset ι) (hs : s.Nonempty)
  (f g : ι → X → ℝ)
  (hf : ∀ i, Continuous (f i))
  (hg : ∀ i, Continuous (g i)) :
  IsClosed (h ⁻¹' {x : X | ∀ i ∈ s, f i x ≤ g i x})
```

Parent. Let X be a metric space and let a finite nonempty family of pairs of continuous real-valued functions be given on X . Prove that the set of points where every first function is at most its paired second function is closed.

Parent in Lean

```
theorem finite_common_pointwise_order_closed
  {α ι : Type} [MetricSpace α]
  (s : Finset ι) (hs : s.Nonempty)
  (f g : ι → α → ℝ)
  (hf : ∀ i, Continuous (f i))
  (hg : ∀ i, Continuous (g i)) :
  IsClosed {x : α | ∀ i ∈ s, f i x ≤ g i x} := by
```

What it contributes. Supplies the finite-induction and intersection argument for common pointwise-order regions; the child adds composition with the continuous map h and proves closedness of the resulting preimage.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the parent to a continuous preimage: it must compose every inequality with h and prove closedness on β , then identify the resulting region with the preimage. The parent's induction alone does not supply the continuity-composition and preimage reasoning.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the result to continuous preimages over arbitrary topological spaces and requires composing every constraint with h before proving closedness. Although the finite-intersection induction is shared, the parent proof does not supply the continuity-composition and preimage reasoning needed here.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the result from metric spaces to arbitrary topological spaces and adds a continuous preimage. Its proof must compose each inequality with the map and establish closedness in the domain, which the parent's proof does not provide.

Check	By	Result
statement_type_check	lean	type-checks

Check	By	Result
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2865 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem continuous_preimage_finite_common_pointwise_order_closed.extracted_1_1 {β X ι : Type}
  [inst : TopologicalSpace β] [inst_1 : TopologicalSpace X] (h : β → X) (hh : Continuous h) (s : Finset ι)
  (hs : s.Nonempty) (f g : ι → X → ℝ) (hf : ∀ (i : ι), Continuous (f i)) (hg : ∀ (i : ι), Continuous (g i)) :
  IsClosed (h ⁻¹' {x | ∀ i ∈ s, f i x ≤ g i x})
```

Read back by a model that saw only the Lean. Let β , X , and ι be types equipped with topological spaces, let $h : \beta \rightarrow X$ be continuous, and let s be a nonempty finite set of indices from ι . Suppose f and g assign to each index $i \in \iota$ continuous real-valued functions on X . Then the preimage under h of the set of points $x \in X$ such that $f(i, x) \leq g(i, x)$ for every $i \in s$ is closed in β .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem continuous_preimage_finite_common_pointwise_order_closed
  {β X ι : Type} [TopologicalSpace β] [TopologicalSpace X]
  (h : β → X) (hh : Continuous h)
  (s : Finset ι) (hs : s.Nonempty)
  (f g : ι → X → ℝ)
  (hf : ∀ i, Continuous (f i))
  (hg : ∀ i, Continuous (g i)) :
  IsClosed (h ⁻¹' {x : X | ∀ i ∈ s, f i x ≤ g i x}) := by
  classical
  induction s using Finset.induction_on with
  | empty =>
    simp at hs
  | @insert i s hi ih =>
    have h_i : IsClosed {y : β | f i (h y) ≤ g i (h y)} := by
      exact isClosed_le ((hf i).comp hh) ((hg i).comp hh)
    have h_s : IsClosed (h ⁻¹' {x : X | ∀ j ∈ s, f j x ≤ g j x}) := by
      rcases s.eq_empty_or_nonempty with rfl | hs'
      · simp
      · exact ih hs'
    have h_inter : IsClosed ({y : β | f i (h y) ≤ g i (h y)} ∩
      (h ⁻¹' {x : X | ∀ j ∈ s, f j x ≤ g j x})) := h_i.inter h_s
    simpa [Finset.mem_insert, and_assoc, and_left_comm, and_comm] using h_inter
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__me.me.me__85d2a477

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 85d2a4777bc98bb7

4.126 126. finite_closed_constraints_compact

The problem

Let K be a compact subset of a metric space, let s be finite, and let each f_i be continuous. If each C_i is closed, prove that the set of points $x \in K$ satisfying $f_i(x) \in C_i$ for every $i \in s$ is compact.

Lean statement

```
theorem finite_closed_constraints_compact
  {α β ι : Type} [MetricSpace α] [MetricSpace β]
  (K : Set α) (hK : IsCompact K) (s : Finset ι)
  (f : ι → α → β) (h : ∀ i, Continuous (f i))
  (C : ι → Set β) (hC : ∀ i, IsClosed (C i)) :
  IsCompact (K ∩ {x : α | ∀ i ∈ s, f i x ∈ C i})
```

Parent. Let X be a metric space, let s be a finite index set, and let $(f_i)_{i \in s}$ be continuous real-valued functions on X . Prove that the common level locus $\{x \in X : f_i(x) = c \text{ for every } i \in s\}$ is closed.

Parent in Lean

```
theorem finite_common_level_locus_closed
  {α ι : Type} [MetricSpace α]
  (s : Finset ι) (f : ι → α → ℝ) (h : ∀ i, Continuous (f i)) (c : ℝ) :
  IsClosed {x : α | ∀ i ∈ s, f i x = c} := by
```

What it contributes. Reused the parent's closed-preimage and finite-intersection checkpoints, replacing singleton equality levels with arbitrary closed target sets and using the resulting closed locus to obtain compactness.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the parent from finite common level sets to arbitrary closed target constraints and adds compactness of their intersection with a compact carrier. Its proof requires a new compact-intersection argument beyond the parent's closedness proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from closedness of a common real level set to compactness of its intersection with an arbitrary compact carrier and finitely many closed target constraints. The proof requires a new compactness-intersection argument and is not recoverable by merely recalling the parent theorem.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from closedness of a finite common level set to compactness of its intersection with an arbitrary compact carrier and arbitrary closed target constraints. Proving compactness requires the additional compact-intersection argument, so recalling the parent does not solve it.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2331 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_closed_constraints_compact.extracted_1_1 {α β ι : Type} [inst : MetricSpace α] [inst_1 : MetricSpace β]
  (K : Set α) (hK : IsCompact K) (s : Finset ι) (f : ι → α → β) (h : ∀ (i : ι), Continuous (f i)) (C : ι → Set β)
  (hC : ∀ (i : ι), IsClosed (C i)) : IsCompact (K ∩ {x | ∀ i ∈ s, f i x ∈ C i})
```

Read back by a model that saw only the Lean. *Let α and β be metric spaces, let K be a compact subset of α , let s be a finite set of indices ι , let f assign to each $i \in \iota$ a continuous function $f_i : \alpha \rightarrow \beta$, and let C_i be a closed subset of β for each $i \in \iota$. Then the set of all $x \in K$ such that $f_i(x) \in C_i$ for every $i \in s$ is compact.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem finite_closed_constraints_compact
  {α β ι : Type} [MetricSpace α] [MetricSpace β]
  (K : Set α) (hK : IsCompact K) (s : Finset ι)
  (f : ι → α → β) (h : ∀ i, Continuous (f i))
  (C : ι → Set β) (hC : ∀ i, IsClosed (C i)) :
  IsCompact (K ∩ {x : α | ∀ i ∈ s, f i x ∈ C i}) := by
  have h_preimage : ∀ i, IsClosed (f i ⁻¹' C i) := by
    intro i
    exact IsClosed.preimage (h i) (hC i)
  have h_constraint : IsClosed {x : α | ∀ i ∈ s, f i x ∈ C i} := by
    have h_inter : IsClosed (⋂ i ∈ s, f i ⁻¹' C i) := by
      exact isClosed_biInter (fun i hi => h_preimage i)
    have h_eq :
      {x : α | ∀ i ∈ s, f i x ∈ C i} =
      ⋂ i ∈ s, f i ⁻¹' C i := by
      ext x
      simp
      rw [h_eq]
      exact h_inter
    exact hK.inter_right h_constraint
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__me__7c585f69

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 7c585f69234fbb03

4.127 127. finite_indexed_closed_target_locus_closed

The problem

Let X be a metric space, let s be a finite index set, let each $f_i : X \rightarrow \mathbb{R}$ be continuous, and let $C_i \subseteq \mathbb{R}$ be closed. Prove that $\{x \in X : f_i(x) \in C_i \text{ for every } i \in s\}$ is closed.

Lean statement

```
theorem finite_indexed_closed_target_locus_closed
  {α 1 : Type} [MetricSpace α]
  (s : Finset 1) (f : 1 → α → ℝ)
  (hcont : ∀ i, Continuous (f i))
  (C : 1 → Set ℝ) (hclosed : ∀ i, IsClosed (C i)) :
  IsClosed {x : α | ∀ i ∈ s, f i x ∈ C i}
```

Parent. Let X be a metric space, let s be a finite index set, and let $(f_i)_{i \in s}$ be continuous real-valued functions on X . Prove that the common level locus $\{x \in X : f_i(x) = c \text{ for every } i \in s\}$ is closed.

Parent in Lean

```
theorem finite_common_level_locus_closed
  {α 1 : Type} [MetricSpace α]
  (s : Finset 1) (f : 1 → α → ℝ) (h : ∀ i, Continuous (f i)) (c : ℝ) :
  IsClosed {x : α | ∀ i ∈ s, f i x = c} := by
```

What it contributes. Reuses the parent’s continuity, closed-preimage, finite-intersection, and set-extensionality checkpoints while replacing the common singleton target with an index-dependent closed set.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes singleton level sets to an indexed family of arbitrary closed target sets. Its proof requires recognizing and applying the general closed-preimage theorem, rather than recalling the parent’s singleton-preimage argument.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed common level to an indexed family of closed target sets, so each constraint requires its own closed-set preimage argument. The parent theorem cannot be recalled to prove arbitrary sets C_i .

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the common singleton target to an indexed family of arbitrary closed sets, requiring the preimage-closedness argument to depend on each target set. This is substantive bounded generalization, not a trivial wrapper or decoration.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2340 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_indexed_closed_target_locus_closed.extracted_1_1 {α ι : Type} [inst : MetricSpace α] (s : Finset ι)
  (f : ι → α → ℝ) (hcont : ∀ (i : ι), Continuous (f i)) (C : ι → Set ℝ) (hclosed : ∀ (i : ι), IsClosed (C i)) :
  IsClosed {x | ∀ i ∈ s, f i x ∈ C i}
```

Read back by a model that saw only the Lean. *Let α be a metric space, let ι be a type, and let s be a finite set of indices in ι . For each $i \in \iota$, let $f_i : \alpha \rightarrow \mathbb{R}$ be continuous, and let $C_i \subseteq \mathbb{R}$ be closed. Then the set of all $x \in \alpha$ such that $f_i(x) \in C_i$ for every $i \in s$ is closed in α .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem finite_indexed_closed_target_locus_closed
  {α ι : Type} [MetricSpace α]
  (s : Finset ι) (f : ι → α → ℝ)
  (hcont : ∀ i, Continuous (f i))
  (C : ι → Set ℝ) (hclosed : ∀ i, IsClosed (C i)) :
  IsClosed {x : α | ∀ i ∈ s, f i x ∈ C i} := by
  have h_pre : ∀ i, IsClosed (f i ⁻¹' C i) := by
    intro i
    exact IsClosed.preimage (hcont i) (hclosed i)
  have h_int : IsClosed (⋂ i ∈ s, f i ⁻¹' C i) := by
    exact isClosed_biInter (fun i hi => h_pre i)
  have h_eq :
    {x : α | ∀ i ∈ s, f i x ∈ C i} =
    ⋂ i ∈ s, f i ⁻¹' C i := by
    ext x
    simp
  rw [h_eq]
  exact h_int
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__me__b11ade90

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint b11ade90a5b15d33

4.128 128. finite_common_pointwise_order_closed

The problem

Let X be a metric space and let a finite nonempty family of pairs of continuous real-valued functions be given on X . Prove that the set of points where every first function is at most its paired second function is closed.

Lean statement

```
theorem finite_common_pointwise_order_closed
  {α ι : Type} [MetricSpace α]
  (s : Finset ι) (hs : s.Nonempty)
  (f g : ι → α → ℝ)
  (hf : ∀ i, Continuous (f i))
  (hg : ∀ i, Continuous (g i)) :
  IsClosed {x : α | ∀ i ∈ s, f i x ≤ g i x} := by
```

Parent. Let f be a continuous real-valued function on a metric space X , and let $a \leq b$. Prove that the set of points x satisfying $a \leq f(x) \leq b$ is closed.

Parent in Lean

```
theorem continuous_interval_preimage_closed
  {α : Type} [MetricSpace α]
  (f : α → ℝ) (h : Continuous f) (a b : ℝ) :
  IsClosed {x : α | a ≤ f x ∧ f x ≤ b} := by
```

What it contributes. Generalizes the parent from constant interval bounds for one continuous function to finite intersections of variable pairwise continuous order loci.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child generalizes a single continuous function's interval preimage to a finite intersection of pointwise inequality loci. Its induction over the finite family and use of 'isClosed_le' require reasoning absent from the parent's interval-preimage proof, so recalling the parent is insufficient.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes a single interval preimage to the common locus of a finite family of pointwise inequalities. Its proof requires induction over the finite index set and repeated intersections of closed sets, which is not supplied by recalling the parent proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes from one interval preimage to the finite intersection of pointwise order loci. Its proof requires finite-set induction and repeated closed-set intersections, reasoning not supplied by the parent's single preimage argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing

Check	By	Result
corpus_dedup	hash	unique against 2846 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_common_pointwise_order_closed.extracted_1_1 {α ι : Type} [inst : MetricSpace α] (s : Finset ι)
  (hs : s.Nonempty) (f g : ι → α → ℝ) (hf : ∀ (i : ι), Continuous (f i)) (hg : ∀ (i : ι), Continuous (g i)) :
  IsClosed {x | ∀ i ∈ s, f i x ≤ g i x}
```

Read back by a model that saw only the Lean. Let α be a metric space, let ι be a type, and let s be a nonempty finite subset of ι . Let f and g be functions from ι to functions $\alpha \rightarrow \mathbb{R}$. Suppose that for every $i \in \iota$, the function $f(i) : \alpha \rightarrow \mathbb{R}$ is continuous, and for every $i \in \iota$, the function $g(i) : \alpha \rightarrow \mathbb{R}$ is continuous. Then the set of all $x \in \alpha$ such that $f(i)(x) \leq g(i)(x)$ for every $i \in s$ is closed in α .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem finite_common_pointwise_order_closed
  {α ι : Type} [MetricSpace α]
  (s : Finset ι) (hs : s.Nonempty)
  (f g : ι → α → ℝ)
  (hf : ∀ i, Continuous (f i))
  (hg : ∀ i, Continuous (g i)) :
  IsClosed {x : α | ∀ i ∈ s, f i x ≤ g i x} := by
  classical
  induction s using Finset.induction_on with
  | empty =>
    simp at hs
  | @insert i s hi ih =>
    have h_i : IsClosed {x : α | f i x ≤ g i x} := by
      exact isClosed_le (hf i) (hg i)
    have h_s : IsClosed {x : α | ∀ j ∈ s, f j x ≤ g j x} := by
      rcases s.eq_empty_or_nonempty with rfl | hs'
      · simp
      · exact ih hs'
    have h_inter := h_i.inter h_s
    simpa [Finset.mem_insert, and_assoc, and_left_comm, and_comm] using h_inter
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__me__c0f67f60

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c0f67f60f7f40288

4.129 129. finite_pointwise_tolerance_locus_closed

The problem

Let X be a metric space, and let $f_1, \dots, f_n$ and $g_1, \dots, g_n$ be continuous real-valued functions on X . Prove that, for any $\varepsilon \in \mathbb{R}$, the set of points where $|f_i(x) - g_i(x)| \leq \varepsilon$ for every i is closed.

Lean statement

```
theorem finite_pointwise_tolerance_locus_closed
  {α : Type} [MetricSpace α]
  (n : ℕ) (f g : Fin n → α → ℝ) (ε : ℝ)
  (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i))
  (z : Set α) (hz : z = {x | ∀ i, |f i x - g i x| ≤ ε}) : IsClosed z
```

Parent. Let X be a metric space, and let $f_1, \dots, f_n$ and $g_1, \dots, g_n$ be continuous real-valued functions on X . Prove that the set of points where $f_i(x) \leq g_i(x)$ for every i is closed.

Parent in Lean

```
theorem finite_pointwise_order_locus_closed
  {α : Type} [MetricSpace α]
  (n : ℕ) (f g : Fin n → α → ℝ)
  (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i))
  (z : Set α) (hz : z = {x | ∀ i, f i x ≤ g i x}) : IsClosed z
```

What it contributes. Supplied the continuous-difference, preimage-of-a-closed-set, and indexed-intersection method; the child expands the order locus to a uniform absolute-error tolerance locus.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child introduces a genuinely new tolerance condition involving absolute values and proves continuity of the composed absolute-value map before taking preimages of a closed ray. The parent's order-locus proof does not provide this absolute-value continuity and preimage argument, so recalling it would not substantially solve the child.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child introduces a symbolic tolerance and absolute-value locus, requiring continuity of the absolute-valued difference and a new preimage argument; recalling the parent's order-locus proof does not directly solve it.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2844 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_pointwise_tolerance_locus_closed.extracted_1_1 {α : Type} [inst : MetricSpace α] (n : ℕ)
  (f g : Fin n → α → ℝ) (ε : ℝ) (hf : ∀ (i : Fin n), Continuous (f i)) (hg : ∀ (i : Fin n), Continuous (g i))
  (z : Set α) (hz : z = {x | ∀ (i : Fin n), |f i x - g i x| ≤ ε}) : IsClosed z
```

Read back by a model that saw only the Lean. Let α be a type equipped with a metric space structure, let n be a natural number, let $f, g : \text{Fin } n \rightarrow \alpha \rightarrow \mathbb{R}$, and let $\varepsilon \in \mathbb{R}$. Assume that for every $i \in \text{Fin } n$, the functions $x \mapsto f i x$ and $x \mapsto g i x$ are continuous. If $z \subseteq \alpha$ is exactly the set $\{x \in \alpha \mid \text{for every } i \in \text{Fin } n, |f i x - g i x| \leq \varepsilon\}$, then z is closed in α .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem finite_pointwise_tolerance_locus_closed
  {α : Type} [MetricSpace α]
  (n : ℕ) (f g : Fin n → α → ℝ) (ε : ℝ)
  (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i))
  (z : Set α) (hz : z = {x | ∀ i, |f i x - g i x| ≤ ε}) : IsClosed z := by
  have hclosed : ∀ i : Fin n, IsClosed {x : α | |f i x - g i x| ≤ ε} := by
    intro i
    let q : α → ℝ := fun x => f i x - g i x
    have hq : Continuous q := (hf i).sub (hg i)
    have habs : Continuous (fun x => |q x|) := hq.abs
    have hpre : IsClosed ((fun x => |q x|)⁻¹' Set.Iic ε) :=
      IsClosed.preimage habs isClosed_Iic
    have heq : {x : α | |f i x - g i x| ≤ ε} =
      (fun x => |q x|)⁻¹' Set.Iic ε := by
      ext x
      simp [q]
    rw [heq]
    exact hpre
  have hcone : IsClosed (⋂ i : Fin n, {x : α | |f i x - g i x| ≤ ε}) :=
    isClosed_iInter hclosed
  have hlocus : z = ⋂ i : Fin n, {x : α | |f i x - g i x| ≤ ε} := by
    rw [hz]
    ext x
    simp
  simpa only [hlocus] using hcone
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__me.mh.me__000e7c9a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 000e7c9a13984189

4.130 130. finite_pointwise_order_locus_as_continuous_zero_set

The problem

Let X be a metric space, and let $f_1, \dots, f_n$ and $g_1, \dots, g_n$ be continuous real-valued functions on X . Show that the locus where $f_i(x) \leq g_i(x)$ for every i is exactly the zero set of a continuous nonnegative function formed by summing the finitely many violations.

Lean statement

```
theorem finite_pointwise_order_locus_as_continuous_zero_set
  {α : Type} [MetricSpace α]
  (n : ℕ) (f g : Fin n → α → ℝ)
  (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i)) :
  ∃ A : α → ℝ,
    (∀ x, A x = (Finset.univ.sum (fun i => max (f i x - g i x) 0))) ∧
    Continuous A ∧
    (∀ x, 0 ≤ A x) ∧
    (∀ x, A x = 0 ↔ ∀ i, f i x ≤ g i x) ∧
    ({x | ∀ i, f i x ≤ g i x} = A⁻¹' ({0} : Set ℝ))
```

Parent. Let X be a metric space, and let $f_1, \dots, f_n$ and $g_1, \dots, g_n$ be continuous real-valued functions on X . Prove that the set of points where $f_i(x) \leq g_i(x)$ for every i is closed.

Parent in Lean

```
theorem finite_pointwise_order_locus_closed
  {α : Type} [MetricSpace α]
  (n : ℕ) (f g : Fin n → α → ℝ)
  (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i))
  (z : Set α) (hz : z = {x | ∀ i, f i x ≤ g i x}) : IsClosed z
```

What it contributes. Provides the finite family of continuous functions and the pointwise order-locus domain; the mutation replaces the closure conclusion with a continuous nonnegative zero-locus representation.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the closed-locus conclusion into an explicit construction of a continuous nonnegative finite-sum aggregate and characterizes its zero set. Its proof uses finite-sum nonnegativity and zero-sum reasoning, not the parent's closed-preimage/intersection argument, so recalling the parent does not substantially help.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from closedness to constructing a specific continuous nonnegative finite sum whose zero set exactly characterizes the order locus. Its proof requires new reasoning about nonnegative sums and zero terms, so recalling the parent's closed-intersection proof would not substantially help.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from closedness to constructing a continuous nonnegative finite-sum aggregate whose zero set exactly characterizes the pointwise order locus. Its proof requires nonnegativity, zero-sum reasoning, and continuity of the max-based aggregate, none of which can be recovered by recalling the parent's preimage/intersection proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2842 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_pointwise_order_locus_as_continuous_zero_set.extracted_1_1 {α : Type} [inst : MetricSpace α] (n : ℕ)
  (f g : Fin n → α → ℝ) (hf : ∀ (i : Fin n), Continuous (f i)) (hg : ∀ (i : Fin n), Continuous (g i)) :
  ∃ A,
    (∀ (x : α), A x = ∑ i, max (f i x - g i x) 0) ∧
    Continuous A ∧
    (∀ (x : α), 0 ≤ A x) ∧
    (∀ (x : α), A x = 0 ↔ ∀ (i : Fin n), f i x ≤ g i x) ∧ {x | ∀ (i : Fin n), f i x ≤ g i x} = A-1{0}
```

Read back by a model that saw only the Lean. Let α be a type equipped with a metric-space structure, let n be a natural number, and let $f, g : \text{Fin } n \rightarrow \alpha \rightarrow \mathbb{R}$ be functions such that, for every $i \in \text{Fin } n$, the functions $x \mapsto f(i, x)$ and $x \mapsto g(i, x)$ are continuous. Then there exists a function $A : \alpha \rightarrow \mathbb{R}$ such that, for every $x \in \alpha$, $A(x) = \sum_{i \in \text{Fin } n} \max(f(i, x) - g(i, x), 0)$; A is continuous; for every $x \in \alpha$, $0 \leq A(x)$; for every $x \in \alpha$, $A(x) = 0$ if and only if, for every $i \in \text{Fin } n$, $f(i, x) \leq g(i, x)$; and the set $\{x \in \alpha \mid \text{for every } i \in \text{Fin } n, f(i, x) \leq g(i, x)\}$ equals the preimage $A^{-1}(\{0\})$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open scoped BigOperators
open Topology Filter Real Complex TopologicalSpace Finset

theorem finite_pointwise_order_locus_as_continuous_zero_set
  {α : Type} [MetricSpace α]
  (n : ℕ) (f g : Fin n → α → ℝ)
  (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i)) :
  ∃ A : α → ℝ,
    (∀ x, A x = (Finset.univ.sum (fun i => max (f i x - g i x) 0))) ∧
    Continuous A ∧
    (∀ x, 0 ≤ A x) ∧
    (∀ x, A x = 0 ↔ ∀ i, f i x ≤ g i x) ∧
    ({x | ∀ i, f i x ≤ g i x} = A-1{0} : Set ℝ) := by
  let A : α → ℝ := fun x => Finset.univ.sum (fun i => max (f i x - g i x) 0)
  have hcont : Continuous A := by
    dsimp [A]
    fun_prop
  have hnonneg : ∀ x, 0 ≤ A x := by
    intro x
    dsimp [A]
    exact Finset.sum_nonneg (fun i hi => le_max_right _ _)
  have hiff : ∀ x, A x = 0 ↔ ∀ i, f i x ≤ g i x := by
    intro x
    constructor
    · intro hi
      have hle : max (f i x - g i x) 0 ≤ A x := by
        dsimp [A]
        exact Finset.single_le_sum
          (fun (j : Fin n) _ => le_max_right (f j x - g j x) 0)
          (Finset.mem_univ i)
      have htermle : max (f i x - g i x) 0 ≤ 0 := by
        simpa [hi] using hle
      have htermzero : max (f i x - g i x) 0 = 0 :=
```

```

    le_antisymm htermle (le_max_right _ _)
  have hdiff : f i x - g i x ≤ 0 := by
    have hleft := le_max_left (f i x - g i x) 0
    rw [htermzero] at hleft
    exact hleft
  linarith
· intro hall
  dsimp [A]
  apply Finset.sum_eq_zero
  intro i hi
  apply max_eq_right
  linarith [hall i]
refine (A, ?_, hcont, hnonneg, hiff, ?_)
· intro x
  rfl
· ext x
  simpa only [Set.mem_setOf_eq, Set.mem_preimage, Set.mem_singleton_iff] using (hiff x).symm

```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__me.mh.me__4d879715

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 4d879715e2757247

4.131 131. continuous_cross_equalizer_closed

The problem

Let f be a continuous real-valued function on a metric space X , and let g be a continuous real-valued function on a metric space Y . Prove that the set of pairs (x, y) satisfying $f(x) = g(y)$ is closed.

Lean statement

```
theorem continuous_cross_equalizer_closed
  {α β : Type} [MetricSpace α] [MetricSpace β]
  (f : α → ℝ) (g : β → ℝ) (hf : Continuous f) (hg : Continuous g) :
  IsClosed {p : α × β | f p.1 = g p.2} := by
```

Parent. Let f and g be continuous real-valued functions on a metric space X . Prove that the set of points x satisfying $f(x) = g(x)$ is closed.

Parent in Lean

```
theorem continuous_equalizer_closed
  {α : Type} [MetricSpace α]
  (f g : α → ℝ) (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x : α | f x = g x} := by
```

What it contributes. Reuses the parent's continuous-difference and closed-zero-preimage method, with product projections added to handle distinct coordinates.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the equalizer from one domain to two independent metric spaces and requires continuity of the composed projections on the product. The parent proof does not supply those projection and product-space arguments directly.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the equalizer from one shared domain to a product of two metric spaces. Its proof must establish continuity after composing each function with the first and second projections, which is mathematics not supplied by the parent proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes from an equalizer on one space to a cross-equalizer on a product of two spaces. Its proof must establish continuity through the first and second projections, which is reasoning not supplied by recalling the parent's proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2856 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem continuous_cross_equalizer_closed.extracted_1_1 {α β : Type} [inst : MetricSpace α] [inst_1 : MetricSpace β]
  (f : α → ℝ) (g : β → ℝ) (hf : Continuous f) (hg : Continuous g) : IsClosed {p | f p.1 = g p.2}
```

Read back by a model that saw only the Lean. *Let α and β be metric spaces, let $f: \alpha \rightarrow \mathbb{R}$ and $g: \beta \rightarrow \mathbb{R}$ be continuous functions. Then the set of pairs $(a,b) \in \alpha \times \beta$ such that $f(a) = g(b)$ is closed in $\alpha \times \beta$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem continuous_cross_equalizer_closed
  {α β : Type} [MetricSpace α] [MetricSpace β]
  (f : α → ℝ) (g : β → ℝ) (hf : Continuous f) (hg : Continuous g) :
  IsClosed {p : α × β | f p.1 = g p.2} := by
  have hfp : Continuous (fun p : α × β => f p.1) := by
    exact hf.comp continuous_fst
  have hgp : Continuous (fun p : α × β => g p.2) := by
    exact hg.comp continuous_snd
  have hdiff : Continuous (fun p : α × β => f p.1 - g p.2) := by
    exact hfp.sub hgp
  have hzero : IsClosed ({0} : Set ℝ) := by
    exact isClosed_singleton
  have hpreimage : IsClosed ((fun p : α × β => f p.1 - g p.2)⁻¹' ({0} : Set ℝ)) := by
    exact IsClosed.preimage hdiff hzero
  have hequalizer : {p : α × β | f p.1 = g p.2} =
    (fun p : α × β => f p.1 - g p.2)⁻¹' ({0} : Set ℝ) := by
    ext p
    simp only [Set.mem_setOf_eq, Set.mem_preimage, Set.mem_singleton_iff]
  constructor
  · intro h
    exact sub_eq_zero.mpr h
  · intro h
    exact sub_eq_zero.mp h
  rw [hequalizer]
  exact hpreimage
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__me.mh.me__d9523552

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint d9523552a0127411

4.132 132. finite_pointwise_strict_order_locus_open

The problem

Let X be a metric space, and let $f_1, \dots, f_n$ and $g_1, \dots, g_n$ be continuous real-valued functions on X . Prove that the set of points where $f_i(x) < g_i(x)$ for every i is open.

Lean statement

```
theorem finite_pointwise_strict_order_locus_open
  {α : Type} [MetricSpace α]
  (n : ℕ) (f g : Fin n → α → ℝ)
  (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i))
  (u : Set α) (hu : u = {x | ∀ i, f i x < g i x}) : IsOpen u
```

Parent. Let X be a metric space, and let $f_1, \dots, f_n$ and $g_1, \dots, g_n$ be continuous real-valued functions on X . Prove that the set of points where $f_i(x) \leq g_i(x)$ for every i is closed.

Parent in Lean

```
theorem finite_pointwise_order_locus_closed
  {α : Type} [MetricSpace α]
  (n : ℕ) (f g : Fin n → α → ℝ)
  (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i))
  (z : Set α) (hz : z = {x | ∀ i, f i x ≤ g i x}) : IsClosed z
```

What it contributes. Supplies the finite-index pointwise order-locus setup and the continuous-difference checkpoint; the mutation changes the closed non-strict locus into an open strict locus.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the mathematical claim from a closed non-strict inequality locus to an open strict inequality locus. Its proof requires the open ray preimage argument and finite indexed intersection, rather than merely wrapping or decorating the parent's conclusion.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the mathematical claim from a finite non-strict-order locus being closed to a finite strict-order locus being open. It requires the open ray and finite intersection arguments, so recalling the parent's closed-set proof does not directly solve it.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely different openness theorem for strict inequalities, requiring open-ray preimages and finite intersections of open sets. The parent's closed non-strict argument does not directly close this goal, despite the analogous proof structure.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2838 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_pointwise_strict_order_locus_open.extracted_1_1 {α : Type} [inst : MetricSpace α] (n : ℕ)
  (f g : Fin n → α → ℝ) (hf : ∀ (i : Fin n), Continuous (f i)) (hg : ∀ (i : Fin n), Continuous (g i)) (u : Set α)
  (hu : u = {x | ∀ (i : Fin n), f i x < g i x}) : IsOpen u
```

Read back by a model that saw only the Lean. Let α be a metric space, let n be a natural number, and let f and g be functions from $\text{Fin } n$ to functions $\alpha \rightarrow \mathbb{R}$. Suppose that for every $i \in \text{Fin } n$, the function $f i$ is continuous and the function $g i$ is continuous. If $u \subseteq \alpha$ satisfies $u = \{x \in \alpha \mid \text{for every } i \in \text{Fin } n, f i(x) < g i(x)\}$, then u is open.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem finite_pointwise_strict_order_locus_open
  {α : Type} [MetricSpace α]
  (n : ℕ) (f g : Fin n → α → ℝ)
  (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i))
  (u : Set α) (hu : u = {x | ∀ i, f i x < g i x}) : IsOpen u := by
  have hopen : ∀ i : Fin n, IsOpen {x : α | f i x < g i x} := by
    intro i
    let q : α → ℝ := fun x => f i x - g i x
    have hq : Continuous q := (hf i).sub (hg i)
    have hpre : IsOpen (q ⁻¹' Set.Iio (0 : ℝ)) :=
      IsOpen.preimage hq isOpen_Iio
    have heq : {x : α | f i x < g i x} = q ⁻¹' Set.Iio (0 : ℝ) := by
      ext x
      change (f i x < g i x) ↔ q x < 0
      constructor <=> intro h <=> dsimp [q] <=> linarith
    rw [heq]
    exact hpre
  have hinter : IsOpen (⋂ i : Fin n, {x : α | f i x < g i x}) :=
    isOpen_iInter_of_finite hopen
  have hu' : u = ⋂ i : Fin n, {x : α | f i x < g i x} := by
    rw [hu]
    ext x
    simp
  rw [hu']
  exact hinter
```

Operator. mutation / mutation_easy **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__me.mh.me__fc50be9d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint fc50be9d98793731

4.133 133. continuous_interval_preimage_closed

The problem

Let f be a continuous real-valued function on a metric space X , and let $a \leq b$. Prove that the set of points x satisfying $a \leq f(x) \leq b$ is closed.

Lean statement

```
theorem continuous_interval_preimage_closed
  {α : Type} [MetricSpace α]
  (f : α → ℝ) (h : Continuous f) (a b : ℝ) :
  IsClosed {x : α | a ≤ f x ∧ f x ≤ b} := by
```

Parent. Let f be a continuous real function on a metric space X . Let $Z(f)$ (the zero set of f) be the set of all $p \in X$ at which $f(p) = 0$. Prove that $Z(f)$ is closed.

Parent in Lean

```
theorem Rudin_exercise_4_3
  {α : Type} [MetricSpace α]
  (f : α → ℝ) (h : Continuous f) (z : Set α) (g : z = f⁻¹' {0})
  : IsClosed z := by
```

What it contributes. Generalizes the parent's continuous-preimage-of-a-closed-set checkpoint from the singleton $\{0\}$ to the closed interval $[a,b]$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the singleton preimage to the preimage of an arbitrary closed interval. Its proof requires establishing closedness of the interval via 'isClosed_Icc', which is not supplied by recalling the parent's singleton argument.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the singleton preimage to the preimage of an arbitrary closed interval. Proving closedness of the interval and identifying the set as its preimage requires reasoning not supplied by the parent's singleton argument.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the closed singleton zero-level set to the inverse image of an arbitrary closed interval. It requires establishing interval closedness and identifying the set as a preimage, so recalling the parent does not directly solve it.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2837 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem continuous_interval_preimage_closed.extracted_1_1 {α : Type} [inst : MetricSpace α] (f : α → ℝ)
  (h : Continuous f) (a b : ℝ) (hab : a ≤ b) : IsClosed {x | a ≤ f x ∧ f x ≤ b}
```

Read back by a model that saw only the Lean. *Let α be a type equipped with a metric-space structure, and let $f : \alpha \rightarrow \mathbb{R}$ be continuous. For real numbers a and b with $a \leq b$, the set of all x in α such that $a \leq f(x)$ and $f(x) \leq b$ is closed in α .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem continuous_interval_preimage_closed
  {α : Type} [MetricSpace α]
  (f : α → ℝ) (h : Continuous f) (a b : ℝ) :
  IsClosed {x : α | a ≤ f x ∧ f x ≤ b} := by
  have h_interval_closed : IsClosed (Set.Icc a b) := by
    exact isClosed_Icc
  have h_preimage_closed : IsClosed (f ⁻¹' Set.Icc a b) := by
    exact IsClosed.preimage h h_interval_closed
  have h_set_eq : {x : α | a ≤ f x ∧ f x ≤ b} = f ⁻¹' Set.Icc a b := by
    ext x
    simp [Set.mem_Icc]
  rw [h_set_eq]
  exact h_preimage_closed
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__me__18b2d420

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 45c40ee9bb3f29e2

4.134 134. finite_common_level_locus_closed

The problem

Let X be a metric space, let s be a finite index set, and let $(f_i)_{i \in s}$ be continuous real-valued functions on X . Prove that the common level locus $\{x \in X : f_i(x) = c \text{ for every } i \in s\}$ is closed.

Lean statement

```
theorem finite_common_level_locus_closed
  {α : Type} [MetricSpace α]
  (s : Finset ι) (f : ι → α → ℝ) (h : ∀ i, Continuous (f i)) (c : ℝ) :
  IsClosed {x : α | ∀ i ∈ s, f i x = c} := by
```

Parent. Let f be a continuous real function on a metric space X . Let $Z(f)$ (the zero set of f) be the set of all $p \in X$ at which $f(p) = 0$. Prove that $Z(f)$ is closed.

Parent in Lean

```
theorem Rudin_exercise_4_3
  {α : Type} [MetricSpace α]
  (f : α → ℝ) (h : Continuous f) (z : Set α) (g : z = f⁻¹' {0})
  : IsClosed z := by
```

What it contributes. Generalizes the parent's continuous preimage-of-a-closed-singleton argument from one zero set to a finite intersection of level sets.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the parent from one zero-level set to the common level locus of a finite family of continuous functions. Its proof requires finite intersections of closed preimages, which is new mathematics not supplied by the parent's single-preimage argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes a single zero-level locus to the common level locus of a finite family, requiring a finite-intersection argument and set extensionality in addition to continuity preimages. Memorizing the parent does not supply the central common-locus reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the parent from one zero-level preimage to the finite intersection of level sets for a family of continuous functions. Its proof requires finite-intersection reasoning and cannot be obtained by simply recalling the parent's singleton-preimage argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2311 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_common_level_locus_closed.extracted_1_1 {α ι : Type} [inst : MetricSpace α] (s : Finset ι)
  (f : ι → α → ℝ) (h : ∀ (i : ι), Continuous (f i)) (c : ℝ) : IsClosed {x | ∀ i ∈ s, f i x = c}
```

Read back by a model that saw only the Lean. *Let α be a metric space, let ι be a type, let s be a finite subset of ι , and let f assign to each $i \in \iota$ a continuous function $f_i : \alpha \rightarrow \mathbb{R}$. For any real number c , the set of points $x \in \alpha$ such that $f_i(x) = c$ for every $i \in s$ is closed in α .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem finite_common_level_locus_closed
  {α ι : Type} [MetricSpace α]
  (s : Finset ι) (f : ι → α → ℝ) (h : ∀ i, Continuous (f i)) (c : ℝ) :
  IsClosed {x : α | ∀ i ∈ s, f i x = c} := by
  have h_level : ∀ i, IsClosed (f i ⁻¹' ({c} : Set ℝ)) := by
    intro i
    exact IsClosed.preimage (h i) (by simp)
  have h_common :
    IsClosed (⋂ i ∈ s, f i ⁻¹' ({c} : Set ℝ)) := by
    exact isClosed_biInter (fun i hi => h_level i)
  have h_eq :
    {x : α | ∀ i ∈ s, f i x = c} =
    ⋂ i ∈ s, f i ⁻¹' ({c} : Set ℝ) := by
    ext x
    simp
  rw [h_eq]
  exact h_common
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__me__517e1096

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 517e109641faffc9

4.135 135. bounded_simultaneous_equalizer_closed

The problem

Let X be a metric space, and let $f_1, \dots, f_n$ and $g_1, \dots, g_n$ be continuous real-valued functions on X . Prove that the set of points where $f_i(x) = g_i(x)$ for every i is closed.

Lean statement

```
theorem bounded_simultaneous_equalizer_closed
  {α : Type} [MetricSpace α]
  (n : ℕ) (f g : Fin n → α → ℝ)
  (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i))
  (z : Set α) (hz : z = {x | ∀ i, f i x = g i x}) : IsClosed z := by
```

Parent. Let f be a continuous real function on a metric space X . Let $Z(f)$ (the zero set of f) be the set of all $p \in X$ at which $f(p) = 0$. Prove that $Z(f)$ is closed.

Parent in Lean

```
theorem Rudin_exercise_4_3
  {α : Type} [MetricSpace α]
  (f : α → ℝ) (h : Continuous f) (z : Set α) (g : z = f⁻¹' {0})
  : IsClosed z := by
```

What it contributes. Used the parent's closed-preimage checkpoint for each continuous difference function, generalized from one zero locus to a bounded simultaneous equalizer.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes from one continuous function's zero locus to the simultaneous equalizer of a finite family. Its proof requires new reasoning via pointwise differences and a finite intersection of closed sets, so recalling the parent's preimage argument alone is insufficient.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes a single zero locus to the simultaneous equalizer locus of a finite family. Its proof requires constructing pairwise closed equalizer sets via continuous differences and combining them with a finite intersection, which the parent proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes a single zero locus to a finite simultaneous equalizer locus. Its proof requires constructing continuous differences and combining their closed equalizer sets via a finite intersection, which the parent proof does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2818 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_simultaneous_equalizer_closed.extracted_1_1 {α : Type} [inst : MetricSpace α] (n : ℕ)
  (f g : Fin n → α → ℝ) (hf : ∀ (i : Fin n), Continuous (f i)) (hg : ∀ (i : Fin n), Continuous (g i)) (z : Set α)
  (hz : z = {x | ∀ (i : Fin n), f i x = g i x}) : IsClosed z
```

Read back by a model that saw only the Lean. *Let α be a metric space, let n be a natural number, and let f and g be families of real-valued functions on α indexed by $\text{Fin } n$. Suppose that for every $i : \text{Fin } n$, $f i$ and $g i$ are continuous. If z is the set of all x in α such that $f i x = g i x$ for every $i : \text{Fin } n$, then z is closed in α .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem bounded_simultaneous_equalizer_closed
  {α : Type} [MetricSpace α]
  (n : ℕ) (f g : Fin n → α → ℝ)
  (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i))
  (z : Set α) (hz : z = {x | ∀ i, f i x = g i x}) : IsClosed z := by
  have hclosed : ∀ i : Fin n, IsClosed {x : α | f i x = g i x} := by
    intro i
    let q : α → ℝ := fun x => f i x - g i x
    have hq : Continuous q := (hf i).sub (hg i)
    have hzero : IsClosed (q ⁻¹' ({0} : Set ℝ)) :=
      IsClosed.preimage hq (by simp)
    have heq : {x : α | f i x = g i x} = q ⁻¹' ({0} : Set ℝ) := by
      ext x
      simp [q, sub_eq_zero]
    rw [heq]
    exact hzero
  have hfinite : IsClosed (⋂ i : Fin n, {x : α | f i x = g i x}) :=
    isClosed_iInter hclosed
  have hz' : z = ⋂ i : Fin n, {x : α | f i x = g i x} := by
    rw [hz]
    ext x
    simp
    rw [hz']
  exact hfinite
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__me__872e7f6f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 872e7f6f479bc1d7

4.136 136. common_nonnegative_locus_closed

The problem

Let X be a metric space and let $(f_i)_{i \in I}$ be a finite family of continuous real-valued functions on X . Prove that the set of points where every $f_i(x) \geq 0$ is closed.

Lean statement

```
theorem common_nonnegative_locus_closed
  {α ι : Type} [MetricSpace α] [Fintype ι]
  (f : ι → α → ℝ) (h : ∀ i, Continuous (f i)) :
  IsClosed {x : α | ∀ i, 0 ≤ f i x} := by
```

Parent. Let X be a metric space and let $(f_i)_{i \in I}$ be a finite family of continuous real-valued functions on X . Prove that the set of points where every f_i vanishes is closed.

Parent in Lean

```
theorem common_zero_locus_closed
  {α ι : Type} [MetricSpace α] [Fintype ι]
  (f : ι → α → ℝ) (h : ∀ i, Continuous (f i)) :
  IsClosed {x : α | ∀ i, f i x = 0} := by
```

What it contributes. Retains the metric-space, finite-family, and continuity setup while replacing the common zero locus with a common inequality locus; its aggregate argument motivates the family-wise closed-set checkpoint.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the zero-locus conclusion to a common nonnegativity locus and proves it via individual preimages of the closed ray $[0, \infty)$ followed by an intersection, rather than recalling the parent's sum-of-squares argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the zero-locus into the common nonnegativity locus and proves it through closed half-line preimages and a finite intersection. This requires a different topological argument and cannot be solved by recalling the parent's sum-of-squares proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely different closed-set claim using individual preimages of the closed ray $[0, \infty)$ and a finite intersection, rather than recalling the parent's sum-of-squares zero-locus argument. All hypotheses and intermediate closedness facts are used in the final proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2154 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem common_nonnegative_locus_closed.extracted_1_1 {α ι : Type} [inst : MetricSpace α] (f : ι → α → ℝ)
  (h : ∀ (i : ι), Continuous (f i)) : IsClosed {x | ∀ (i : ι), 0 ≤ f i x}
```

Read back by a model that saw only the Lean. *Let α and ι be types, let α be equipped with a metric-space structure, and let f be a family of real-valued functions on α indexed by ι . Suppose that for every $i \in \iota$, the function $x \mapsto f(i, x)$ is continuous. Then the set of all $x \in \alpha$ such that $f(i, x) \geq 0$ for every $i \in \iota$ is closed in α .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem common_nonnegative_locus_closed
  {α ι : Type} [MetricSpace α] [Fintype ι]
  (f : ι → α → ℝ) (h : ∀ i, Continuous (f i)) :
  IsClosed {x : α | ∀ i, 0 ≤ f i x} := by
  have h_i : ∀ i, IsClosed {x : α | 0 ≤ f i x} := by
    intro i
    change IsClosed ((f i) ⁻¹' Set.Ici (0 : ℝ))
    exact isClosed_Ici.preimage (h i)
  have h_locus :
    {x : α | ∀ i, 0 ≤ f i x} = ⋂ i, {x : α | 0 ≤ f i x} := by
    ext x
    simp
  rw [h_locus]
  exact isClosed_iInter h_i
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__mh.me__1122ba60

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 1122ba60417540d3

4.137 137. common_zero_locus_closed_normed

The problem

Let X be a metric space and let $(f_i)_{i \in I}$ be a finite family of continuous maps from X into a normed real vector space. Prove that the set of points where every f_i vanishes is closed.

Lean statement

```
theorem common_zero_locus_closed_normed
  {α ι E : Type*} [MetricSpace α] [Fintype ι]
  [NormedAddCommGroup E] [NormedSpace ℝ E]
  (f : ι → α → E) (h : ∀ i, Continuous (f i)) :
  IsClosed {x : α | ∀ i, f i x = 0}
```

Parent. Let X be a metric space and let $(f_i)_{i \in I}$ be a finite family of continuous real-valued functions on X . Prove that the set of points where every f_i vanishes is closed.

Parent in Lean

```
theorem common_zero_locus_closed
  {α ι : Type} [MetricSpace α] [Fintype ι]
  (f : ι → α → ℝ) (h : ∀ i, Continuous (f i)) :
  IsClosed {x : α | ∀ i, f i x = 0} := by
```

What it contributes. Supplied the finite-family simultaneous-zero-locus strategy and the finite sum-of-squares continuity checkpoint; generalized the codomain from $\mathbb{R}$ to a normed real vector space.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the codomain from $\mathbb{R}$ to a real normed vector space and must use norm-square vanishing to recover each zero. The parent's scalar sum-of-squares proof cannot be recalled verbatim to solve this statement.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes from real-valued functions to maps into a real normed vector space and requires the new norm-vanishing argument via 'norm_eq_zero'. Although the proof skeleton is similar, recalling the parent does not supply the codomain-generalized aggregate or its zero characterization.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the codomain from $\mathbb{R}$ to an arbitrary real normed vector space and replaces scalar squares with squared norms. It additionally proves vanishing of each vector from norm-square vanishing, so recalling the parent's scalar proof does not directly solve the new goal.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2161 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	MISMATCH: Statement A assumes the index family is finite, whereas Statement B allows an arbitrary index type ι .

Goal round-trip: what Lean elaborated

```
theorem common_zero_locus_closed_normed.extracted_1_1.{u_3, u_2, u_1} {α : Type u_1} {ι : Type u_2} {E : Type u_3}
  [inst : MetricSpace α] [inst_1 : NormedAddCommGroup E] (f : ι → α → E) (h : ∀ (i : ι), Continuous (f i)) :
  IsClosed {x | ∀ (i : ι), f i x = 0}
```

Read back by a model that saw only the Lean. *Let α be a metric space, E a normed additive commutative group, and ι any type. Let f be a family of functions $f_i : \alpha \rightarrow E$ indexed by ι , and suppose each f_i is continuous. Then the set of all $x \in \alpha$ such that $f_i(x) = 0$ for every $i \in \iota$ is closed in α .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open scoped BigOperators
open Topology Filter Real Complex TopologicalSpace Finset

theorem common_zero_locus_closed_normed
  {α ι E : Type*} [MetricSpace α] [Fintype ι]
  [NormedAddCommGroup E] [NormedSpace ℝ E]
  (f : ι → α → E) (h : ∀ i, Continuous (f i)) :
  IsClosed {x : α | ∀ i, f i x = 0} := by
  have h_aggregate :
    Continuous (fun x : α => ∑ i, ‖f i x‖ ^ 2) := by
    fun_prop
  have h_zero_closed : IsClosed ({0} : Set ℝ) := by
    simp
  have h_aggregate_closed :
    IsClosed ((fun x : α => ∑ i, ‖f i x‖ ^ 2) ^{-1'} ({0} : Set ℝ)) :=
    IsClosed.preimage h_aggregate h_zero_closed
  have h_locus :
    {x : α | ∀ i, f i x = 0} =
      (fun x : α => ∑ i, ‖f i x‖ ^ 2) ^{-1'} ({0} : Set ℝ) := by
    ext x
    simp only [Set.mem_setOf_eq, Set.mem_preimage, Set.mem_singleton_iff]
  constructor
  · intro hx
    simp [hx]
  · intro hx
    have hsq : ∀ i, ‖f i x‖ ^ 2 = 0 := by
      intro i
      have hnonneg : ∀ j ∈ (Finset.univ : Finset ι), 0 ≤ ‖f j x‖ ^ 2 := by
        intro j hj
        positivity
      have hall := (Finset.sum_eq_zero_iff_of_nonneg hnonneg).mp hx
      exact hall i (Finset.mem_univ i)
    intro i
    have hnrm : ‖f i x‖ = 0 := by
      nlinarith [hsq i]
    exact norm_eq_zero.mpr hnrm
  rw [h_locus]
  exact h_aggregate_closed
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__mh.me__4d450bdc

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint 4d450bdc5f309318

4.138 138. finite_zero_constrained_nonneg_closed

The problem

Let X be a metric space, let $f : X \rightarrow \mathbb{R}$ and let $(g_i)_{i \in I} : X \rightarrow \mathbb{R}$ be continuous, where I is finite. Prove that the set of points satisfying $f(x) = 0$ and $g_i(x) \geq 0$ for every $i \in I$ is closed.

Lean statement

```
theorem finite_zero_constrained_nonneg_closed
  {X ι : Type*} [MetricSpace X] [Fintype ι]
  (f : X → ℝ) (g : ι → X → ℝ)
  (hf : Continuous f) (hg : ∀ i, Continuous (g i)) :
  IsClosed {x | f x = 0 ∧ ∀ i, 0 ≤ g i x} := by
```

Parent. Let $f, g : X \rightarrow \mathbb{R}$ be continuous functions on a metric space. Prove that the set of points where f vanishes and g is nonnegative is closed.

Parent in Lean

```
theorem zero_constrained_positivity_closed
  {X : Type*} [MetricSpace X]
  (f g : X → ℝ) (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x | f x = 0 ∧ 0 ≤ g x} := by
```

What it contributes. Supplies the zero-locus checkpoint and the closed comparison/superlevel-set argument, generalized from one inequality to a finite indexed intersection.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes one nonnegativity constraint to an indexed family and requires proving closure via an intersection of closed sets, which the parent's proof does not supply. The finite index assumption is unused because arbitrary intersections of closed sets are closed, making the result stronger rather than a recall or decoration.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes one continuous nonnegativity constraint to a finite family indexed by ι . Its proof requires establishing closedness of an indexed intersection and combining it with the zero locus, which the parent's proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes one nonnegativity constraint to an indexed family and requires proving closedness via an intersection of closed sets, rather than directly reusing the parent's single-constraint argument. The finite-index assumption is unnecessary, but the indexed-family reasoning is substantive and the family hypotheses are used.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing

Check	By	Result
corpus_dedup	hash	unique against 2145 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	MISMATCH: Statement B omits the hypothesis that the index type ι is finite, allowing infinitely many inequalities; arbitrary intersections of closed sets need not be closed.

Goal round-trip: what Lean elaborated

```
theorem finite_zero_constrained_nonneg_closed.extracted_1_1.{u_2, u_1} {X : Type u_1} {ι : Type u_2}
  [inst : MetricSpace X] (f : X → ℝ) (g : ι → X → ℝ) (hf : Continuous f) (hg : ∀ (i : ι), Continuous (g i)) :
  IsClosed {x | f x = 0 ∧ ∀ (i : ι), 0 ≤ g i x}
```

Read back by a model that saw only the Lean. Let X be a metric space, let ι be a type, let $f : X \rightarrow \mathbb{R}$ and $g : \iota \rightarrow X \rightarrow \mathbb{R}$ be functions. Assume that f is continuous and that, for every $i \in \iota$, the function $x \mapsto g i x$ is continuous. Then the set of all $x \in X$ such that $f(x) = 0$ and $g(i, x) \geq 0$ for every $i \in \iota$ is closed in X .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem finite_zero_constrained_nonneg_closed
  {X ι : Type*} [MetricSpace X] [Fintype ι]
  (f : X → ℝ) (g : ι → X → ℝ)
  (hf : Continuous f) (hg : ∀ i, Continuous (g i)) :
  IsClosed {x | f x = 0 ∧ ∀ i, 0 ≤ g i x} := by
  have h_zero_closed : IsClosed {x | f x = 0} := by
    have h_singleton : IsClosed ({0} : Set ℝ) := by
      simp
    exact h_singleton.preimage hf
  have h_ineq_closed : IsClosed (⋂ i : ι, {x | 0 ≤ g i x}) := by
    exact isClosed_iInter (fun i => isClosed_le continuous.const (hg i))
  have h_combined : IsClosed ({x | f x = 0} ∩ (⋂ i : ι, {x | 0 ≤ g i x})) := by
    exact h_zero_closed.inter h_ineq_closed
  have h_set : {x | f x = 0 ∧ ∀ i, 0 ≤ g i x} =
    {x | f x = 0} ∩ (⋂ i : ι, {x | 0 ≤ g i x}) := by
    ext x
    simp
  rw [h_set]
  exact h_combined
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__x__Munkres_exercise_18_8a__xe.me__f4046ed9

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint f4046ed933be35da

4.139 139. bounded_shift_forward_difference

The problem

Suppose f is differentiable for every $x > 0$, and $f'(x) \rightarrow 0$ as $x \rightarrow +\infty$. If h is nonnegative and bounded above by B , prove that $f(x + h) - f(x) \rightarrow 0$ as $x \rightarrow +\infty$.

Lean statement

```
theorem bounded_shift_forward_difference
  {f : ℝ → ℝ} {h B : ℝ}
  (hh0 : 0 ≤ h) (hhB : h ≤ B)
  (hfd : DifferentiableOn ℝ f (Set.Ioi 0))
  (hf : Filter.Tendsto (deriv f) Filter.atTop (ℕ 0)) :
  Filter.Tendsto (fun x => f (x + h) - f x) Filter.atTop (ℕ 0)
```

Parent. Suppose f is defined and differentiable for every $x > 0$, and $f'(x) \rightarrow 0$ as $x \rightarrow +\infty$. Put $g(x) = f(x + 1) - f(x)$. Prove that $g(x) \rightarrow 0$ as $x \rightarrow +\infty$.

Parent in Lean

```
theorem Rudin_exercise_5_5
  {f : ℝ → ℝ}
  (hfd : DifferentiableOn ℝ f (Ioi 0))
  (hf : Tendsto (deriv f) atTop (ℕ 0)) :
  Tendsto (λ x => f (x + 1) - f x) atTop (ℕ 0) := by
```

What it contributes. Reuses the parent's derivative-at-infinity estimate, eventual lower-bound extraction, positivity domain restriction, and mean-value theorem checkpoint, with the unit interval replaced by the bounded interval $[x, x+h]$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the unit shift by a bounded nonnegative shift, requiring a zero-shift case and a new derivative bound scaled by $B+1$. The parent's fixed-shift proof does not directly establish this generalized claim.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely replaces the unit shift with an arbitrary bounded nonnegative shift. It requires new reasoning for the zero-shift case and scales the derivative bound using the symbolic upper bound B , so recalling the parent proof is insufficient.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the unit shift by a bounded nonnegative real h , requiring a zero-shift case and a new derivative bound scaled by $B+1$ before applying the mean value theorem. This is not recoverable by simply recalling the parent's unit-interval proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2763 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_shift_forward_difference.extracted_1_1 {f : ℝ → ℝ} {h B : ℝ} (hh0 : 0 ≤ h) (hhB : h ≤ B)
(hfd : DifferentiableOn ℝ f (Set.Ioi 0)) (hf : Tendsto (deriv f) atTop (ℳ 0)) :
Tendsto (fun x => f (x + h) - f x) atTop (ℳ 0)
```

Read back by a model that saw only the Lean. *Let f be a real-valued function on the real numbers, and let h and B be real numbers. Suppose that $0 \leq h$, $h \leq B$, f is differentiable on the open interval $(0, \infty)$, and the derivative $f'(x)$ tends to 0 as x tends to $+\infty$. Then $f(x + h) - f(x)$ tends to 0 as x tends to $+\infty$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset Set
open scoped BigOperators

theorem bounded_shift_forward_difference
{f : ℝ → ℝ} {h B : ℝ}
(hh0 : 0 ≤ h) (hhB : h ≤ B)
(hfd : DifferentiableOn ℝ f (Ioi 0))
(hf : Tendsto (deriv f) atTop (ℳ 0)) :
Tendsto (fun x => f (x + h) - f x) atTop (ℳ 0) := by
  by_cases hzero : h = 0
  · subst h
    simpa using (tendsto_const_nhds : Tendsto (fun _ : ℝ => (0 : ℝ)) atTop (ℳ 0))
  · have hpos : 0 < h := lt_of_le_of_ne hh0 (Ne.symm hzero)
    refine Metric.tendsto_atTop.2 ?_
    intro ε εpos
    have hB1 : 0 < B + 1 := by linarith
    have hδpos : 0 < ε / (B + 1) := div_pos εpos hB1
    have hderiv_small : ∀ f y : ℝ in atTop, |deriv f y| < ε / (B + 1) := by
      have hball := hf.eventually (Metric.ball_mem_nhds (0 : ℝ) hδpos)
      refine hball.mono ?_
      intro y hy
      simpa [Metric.mem_ball, Real.dist_eq, abs_sub, sub_zero] using hy
    obtain ⟨N, hN⟩ := Filter.eventually_atTop.1 hderiv_small
    refine ⟨max N 1, ?_⟩
    intro x hx
    have hxN : N ≤ x := le_trans (le_max_left N 1) hx
    have hx1 : (1 : ℝ) ≤ x := le_trans (le_max_right N 1) hx
    have hxpos : 0 < x := lt_of_lt_of_le (by norm_num) hx1
    have hxlt : x < x + h := lt_add_of_pos_right x hpos
    have hsubset_cc : Icc x (x + h) ⊆ Ioi (0 : ℝ) := by
      intro y hy
      exact lt_of_lt_of_le hxpos hy.1
    have hsubset_oo : Ioo x (x + h) ⊆ Ioi (0 : ℝ) := by
      intro y hy
      exact lt_trans hxpos hy.1
    have hcont_restrict : ContinuousOn f (Icc x (x + h)) :=
      (hfd.continuousOn).mono hsubset_cc
    have hdiff_restrict : DifferentiableOn ℝ f (Ioo x (x + h)) :=
      hfd.mono hsubset_oo
    obtain ⟨c, hc_mem, hc_deriv⟩ :=
      exists_deriv_eq_slope (f := f) (a := x) (b := x + h)
    (hab := hxlt) (hfc := hcont_restrict) (hfd := hdiff_restrict)
    have hxden : (x + h) - x = h := by ring
    have hc_ge : N ≤ c := le_trans hxN (le_of_lt hc_mem.1)
    have hbound : |deriv f c| < ε / (B + 1) := hN c hc_ge
    have hmul : (B + 1) * |deriv f c| < ε := by
      calc
```

```

(B + 1) * |deriv f c| = |deriv f c| * (B + 1) := by ring
_ < (ε / (B + 1)) * (B + 1) := mul_lt_mul_of_pos_right hbound hB1
_ = ε := by field_simp [ne_of_gt hB1]
have hle : h * |deriv f c| ≤ (B + 1) * |deriv f c| := by
  exact mul_le_mul_of_nonneg_right (by linarith [hhB]) (abs_nonneg _)
have hdiffbound : h * |deriv f c| < ε := lt_of_le_of_lt hle hmul
have hdiff_eq : f (x + h) - f x = h * deriv f c := by
  rw [hc_deriv, hxden]
  field_simp [hzero]
have hgoal : |f (x + h) - f x| < ε := by
  rw [hdiff_eq, abs_mul, abs_of_pos hpos]
  exact hdiffbound
simpa [Real.dist_eq, sub_zero] using hgoal

```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_5_5__me__1e023b0f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 1e023b0fde6347d9

4.140 140. variable_increment_tendsto

The problem

Suppose f is differentiable for every $x > 0$ and $f'(x) \rightarrow L$ as $x \rightarrow +\infty$. For a fixed positive increment a , prove that $f(x + a) - f(x) \rightarrow aL$ as $x \rightarrow +\infty$.

Lean statement

```
theorem variable_increment_tendsto
  {f : ℝ → ℝ} {a L : ℝ}
  (ha : 0 < a)
  (hfd : DifferentiableOn ℝ f (Set.Ioi 0))
  (hf : Tendsto (deriv f) Filter.atTop (ℳ L)) :
  Tendsto (fun x => f (x + a) - f x) Filter.atTop (ℳ (a * L))
```

Parent. Suppose f is defined and differentiable for every $x > 0$, and $f'(x) \rightarrow 0$ as $x \rightarrow +\infty$. Put $g(x) = f(x + 1) - f(x)$. Prove that $g(x) \rightarrow 0$ as $x \rightarrow +\infty$.

Parent in Lean

```
theorem Rudin_exercise_5_5
  {f : ℝ → ℝ}
  (hfd : DifferentiableOn ℝ f (Ioi 0))
  (hf : Tendsto (deriv f) atTop (ℳ 0)) :
  Tendsto (λ x => f (x + 1) - f x) atTop (ℳ 0) := by
```

What it contributes. Supplies the differentiability-on- $(0, \infty)$ setup, atTop derivative convergence strategy, interval restriction, and mean-value-theorem checkpoint; the child carries an arbitrary positive increment and arbitrary derivative limit through the final estimate.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes both the increment and derivative limit, requiring a scaled epsilon bound and a nontrivial mean-value/algebraic argument to obtain the limit aL . The fixed unit-shift, zero-limit parent proof does not directly establish this conclusion.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes both the increment and the derivative limit. Its proof must scale the derivative error by ε/a , use positivity of arbitrary a to apply the mean value theorem on $[x, x+a]$, and algebraically derive convergence to aL ; the parent's unit-zero specialization does not supply these obligations.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2249 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem variable_increment_tendsto.extracted_1_1 {f : ℝ → ℝ} {a L : ℝ} (ha : 0 < a)
(hfd : DifferentiableOn ℝ f (Set.Ioi 0)) (hf : Tendsto (deriv f) atTop (ℳ L)) :
Tendsto (fun x => f (x + a) - f x) atTop (ℳ (a * L))
```

Read back by a model that saw only the Lean. Let f be a real-valued function on the real numbers, and let a and L be real numbers. Assume that a is positive, that f is differentiable on the open interval $(0, \infty)$, and that the derivative of f tends to L as x tends to $+\infty$. Then, as x tends to $+\infty$, the difference $f(x+a) - f(x)$ tends to aL .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset Set
open scoped BigOperators

theorem variable_increment_tendsto
  {f : ℝ → ℝ} {a L : ℝ}
  (ha : 0 < a)
  (hfd : DifferentiableOn ℝ f (Set.Ioi 0))
  (hf : Tendsto (deriv f) Filter.atTop (ℳ L)) :
  Tendsto (fun x => f (x + a) - f x) Filter.atTop (ℳ (a * L)) := by
  refine Metric.tendsto_atTop.2 ?_
  intro ε hε
  have hderiv_small : ∀ε y : ℝ in Filter.atTop, |deriv f y - L| < ε / a := by
    have hball := hf.eventually (Metric.ball_mem_nhds L (div_pos hε ha))
    refine hball.mono ?_
    intro y hy
    simp [Metric.mem_ball, Real.dist_eq] using hy
  obtain ⟨N, hN⟩ := Filter.eventually_atTop.1 hderiv_small
  refine ⟨max N 1, ?_⟩
  intro x hx
  have hxN : N ≤ x := le_trans (le_max_left N 1) hx
  have hx1 : (1 : ℝ) ≤ x := le_trans (le_max_right N 1) hx
  have hxpos : 0 < x := lt_of_lt_of_le (by norm_num) hx1
  have hsubset_cc : Set.Icc x (x + a) ⊆ Set.Ioi (0 : ℝ) := by
    intro y hy
    exact lt_of_lt_of_le hxpos hy.1
  have hsubset_oo : Set.Ioo x (x + a) ⊆ Set.Ioi (0 : ℝ) := by
    intro y hy
    exact lt_trans hxpos hy.1
  have hcont_restrict : ContinuousOn f (Set.Icc x (x + a)) :=
    hfd.continuousOn.mono hsubset_cc
  have hdiff_restrict : DifferentiableOn ℝ f (Set.Ioo x (x + a)) :=
    hfd.mono hsubset_oo
  have hxlt : x < x + a := lt_add_of_pos_right x ha
  obtain ⟨c, hc_mem, hc_deriv⟩ :=
    exists_deriv_eq_slope (f := f) (a := x) (b := x + a)
    (hab := hxlt) (hfc := hcont_restrict) (hfd := hdiff_restrict)
  have hden : (x + a) - x = a := by ring
  have hc_deriv' : deriv f c = (f (x + a) - f x) / a := by
    simp [hden] using hc_deriv
  have hdiff_eq : f (x + a) - f x = a * deriv f c := by
    calc
      f (x + a) - f x = ((f (x + a) - f x) / a) * a := by
        field_simp [ne_of_gt ha]
      _ = deriv f c * a := by rw [← hc_deriv']
      _ = a * deriv f c := by ring
  have hc_ge : N ≤ c := le_trans hxN (le_of_lt hc_mem.1)
  have hbound : |deriv f c - L| < ε / a := hN c hc_ge
  have hgoal : |(f (x + a) - f x) - a * L| < ε := by
    rw [hdiff_eq]
    calc
```

```

|a * deriv f c - a * L| = a * |deriv f c - L| := by
  rw [show a * deriv f c - a * L = a * (deriv f c - L) by ring]
  rw [abs_mul, abs_of_pos ha]
_ < a * (ε / a) := mul_lt_mul_of_pos_left hbound ha
_ = ε := by field_simp [ne_of_gt ha]
have hgoal' : |f (x + a) - (a * L + f x)| < ε := by
  convert hgoal using 1 <;> ring
simpa [Real.dist_eq] using hgoal'

```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_5_5__me__71e4f964

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 71e4f964bc061904

4.141 141. shifted_secant_slope_monotone

The problem

Let f be continuous on $[a, \infty)$, differentiable on (a, ∞) , satisfy $f(a) = 0$, and have an increasing derivative there. Prove that the secant slope $f(x)/(x - a)$ is increasing for $x > a$.

Lean statement

```
theorem shifted_secant_slope_monotone
  {f : ℝ → ℝ} (a : ℝ)
  (hf1 : ContinuousOn f (Set.Ici a))
  (hf2 : DifferentiableOn ℝ f (Set.Ioi a))
  (hf3 : f a = 0)
  (hf4 : MonotoneOn (deriv f) (Set.Ioi a)) :
  MonotoneOn (fun x => f x / (x - a)) (Set.Ioi a)
```

Parent. Suppose (a) f is continuous for $x \geq 0$, (b) $f'(x)$ exists for $x > 0$, (c) $f(0) = 0$, (d) f' is monotonically increasing. Put $g(x) = \frac{f(x)}{x}$ ($x > 0$) and prove that g is monotonically increasing.

Parent in Lean

```
theorem Rudin_exercise_5_6
  {f : ℝ → ℝ}
  (hf1 : ContinuousOn f (Set.Ici 0))
  (hf2 : DifferentiableOn ℝ f (Set.Ioi 0))
  (hf3 : f 0 = 0)
  (hf4 : MonotoneOn (deriv f) (Set.Ioi 0)) :
  MonotoneOn (λ x => f x / x) (Set.Ioi 0) := by
```

What it contributes. Generalizes the parent secant-slope monotonicity argument from basepoint 0 to an arbitrary basepoint a , preserving the convexity and chord-comparison checkpoints while introducing translated domains and denominators.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the basepoint and translates the positive ray, requiring the convexity and secant comparison to be rebuilt with expressions involving $x-a$ and $y-a$. The arbitrary shift is not a decorative wrapper and the proof cannot be applied from the parent without substantive adaptation.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the basepoint: the convexity argument and secant comparison must be translated from 0 to an arbitrary a , with new positivity, affine-combination, and denominator algebra. The parent's proof supplies a strategy but does not directly prove the shifted conclusion by recall.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing

Check	By	Result
corpus_dedup	hash	unique against 2172 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem shifted_secant_slope_monotone.extracted_1_1 {f : ℝ → ℝ} (a : ℝ) (hf1 : ContinuousOn f (Set.Ici a))
(hf2 : DifferentiableOn ℝ f (Set.Ioi a)) (hf3 : f a = 0) (hf4 : MonotoneOn (deriv f) (Set.Ioi a)) :
MonotoneOn (fun x => f x / (x - a)) (Set.Ioi a)
```

Read back by a model that saw only the Lean. *Let $f: \mathbb{R} \rightarrow \mathbb{R}$ and let $a \in \mathbb{R}$. Assume that f is continuous on $[a, \infty)$, differentiable on (a, ∞) , $f(a) = 0$, and the derivative f' is nondecreasing on (a, ∞) . Then the function $x \mapsto f(x)/(x - a)$ is nondecreasing on (a, ∞) .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem shifted_secant_slope_monotone
{f : ℝ → ℝ} (a : ℝ)
(hf1 : ContinuousOn f (Set.Ici a))
(hf2 : DifferentiableOn ℝ f (Set.Ioi a))
(hf3 : f a = 0)
(hf4 : MonotoneOn (deriv f) (Set.Ioi a)) :
MonotoneOn (fun x => f x / (x - a)) (Set.Ioi a) := by
have hfconv : ConvexOn ℝ (Set.Ici a) f := by
  have hdiff : DifferentiableOn ℝ f (interior (Set.Ici a)) := by
    simpa [interior_Ici] using hf2
  have hmono : MonotoneOn (deriv f) (interior (Set.Ici a)) := by
    simpa [interior_Ici] using hf4
  exact hmono.convexOn_of_deriv (convex_Ici a) hf1 hdiff
refine fun x hx y hy hxy => ?_
have hx_pos : a < x := hx
have hy_pos : a < y := hy
have hxden : 0 < x - a := sub_pos.mpr hx_pos
have hyden : 0 < y - a := sub_pos.mpr hy_pos
have hy_mem : y ∈ Set.Ici a := le_of_lt hy_pos
have ha_mem : a ∈ Set.Ici a := by simp
have hy_ne : y - a ≠ 0 := ne_of_gt hyden
set θ : ℝ := (x - a) / (y - a)
have hθ_nonneg : 0 ≤ θ := by
  simpa [θ] using div_nonneg hxden.le hyden.le
have hnum : x - a ≤ y - a := by linarith
have hθ_le_one : θ ≤ 1 := by
  simpa [θ] using div_le_one_of_le hnum hyden.le
have hone_minus_nonneg : 0 ≤ 1 - θ := sub_nonneg.mpr hθ_le_one
have hcomb : θ * y + (1 - θ) * a = x := by
  dsimp [θ]
  field_simp [hy_ne] <|> ring
have hconv_app := hfconv.2 hy_mem ha_mem hθ_nonneg hone_minus_nonneg (by ring)
have hfxle : f x ≤ θ * f y := by
  calc
    f x = f (θ * y + (1 - θ) * a) := by rw [hcomb]
    _ ≤ θ * f y + (1 - θ) * f a := hconv_app
    _ = θ * f y := by rw [hf3]; ring
have hcross : f x * (y - a) ≤ f y * (x - a) := by
  calc
    f x * (y - a) ≤ (θ * f y) * (y - a) :=
      mul_le_mul_of_nonneg_right hfxle hyden.le
    _ = f y * (x - a) := by
      dsimp [θ]
```

```
field_simp [hy_ne] <|> ring
exact (div_le_div_iff hxden hyden).2 hcross
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_5_6__me__a17d08de

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint a17d08de5eac9a69

4.142 142. bounded_coefficient_phase_series_converges

The problem

Let $|z| = 1$. If (b_n) is uniformly bounded by B and (a_n) is dominated by a summable nonnegative sequence (g_n) , prove that the partial sums of $\sum z^{n+1} b_n a_n$ converge.

Lean statement

```
theorem bounded_coefficient_phase_series_converges (z : ℂ) (hz : ‖z‖ = 1) (b : ℕ → ℂ) (B : ℝ) (a : ℕ → ℂ) (g : ℕ → ℝ) (s : ℕ → ℂ) (hb :
  ↪ ∀ n, ‖b n‖ ≤ B) (hdom : ∀ n, ‖a n‖ ≤ g n) (hsummable : Summable g) (h : s = fun n => ∑ i ∈ Finset.range n, z ^ (i + 1) * (b i * a i))
  ↪ : ∃ y, Tendsto s atTop (ℳ y)
```

Parent. Let $|z| = 1$. If the coefficients a_n are dominated in norm by a summable nonnegative sequence g_n , prove that the partial sums of $\sum z^{n+1} a_n$ converge.

Parent in Lean

```
theorem dominated_phase_series_converges (z : ℂ) (hz : ‖z‖ = 1) (a : ℕ → ℂ) (g : ℕ → ℝ) (s : ℕ → ℂ) (hdom : ∀ n, ‖a n‖ ≤ g n) (hsummable
  ↪ : Summable g) (h : s = fun n => ∑ i ∈ Finset.range n, z ^ (i + 1) * a i) : ∃ y, Tendsto s atTop (ℳ y)
```

What it contributes. Retains the parent's unit-phase construction, norm domination, comparison test, and partial-sum convergence argument, while inserting the essential bounded coefficient family and scaled summable majorant.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child genuinely introduces a parameterized bounded coefficient sequence and must establish the new majorant $\|b_n * a_n\| \leq B * g_n$ and summability of the scaled majorant. This requires reasoning beyond recalling the parent's direct domination argument.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The bounded coefficient family introduces genuinely new reasoning: the proof must establish nonnegativity of the majorant, bound the product by $B \cdot g$, and prove that scaled series is summable. The parent's direct norm domination does not supply this argument.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The bounded coefficient family requires new reasoning: deriving nonnegativity of the majorant and proving the product bound $\|b_n a_n\| \leq B g_n$, then establishing summability of the scaled majorant. This is not supplied by recalling the parent's direct domination proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2207 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_coefficient_phase_series_converges.extracted_1_1 (z : ℂ) (hz : ‖z‖ = 1) (b : ℕ → ℂ) (B : ℝ) (hB : 0 ≤ B)
(a : ℕ → ℂ) (g : ℕ → ℝ) (s : ℕ → ℂ) (hb : ∀ (n : ℕ), ‖b n‖ ≤ B) (hdom : ∀ (n : ℕ), ‖a n‖ ≤ g n)
(hsummable : Summable g) (h : s = fun n => ∑ i ∈ range n, z ^ (i + 1) * (b i * a i)) :
  ∃ y, Tendsto s atTop (ℳ y)
```

Read back by a model that saw only the Lean. Let z be a complex number with $\|z\| = 1$, let $b : \mathbb{N} \rightarrow \mathbb{C}$, let B be a real number with $0 \leq B$, let $a : \mathbb{N} \rightarrow \mathbb{C}$, let $g : \mathbb{N} \rightarrow \mathbb{R}$, and let $s : \mathbb{N} \rightarrow \mathbb{C}$. Suppose that for every natural number n , $\|b n\| \leq B$; for every natural number n , $\|a n\| \leq g n$; the series $\sum_{n=0}^{\infty} g n$ is summable; and for every natural number n , $s n = \sum_{i \in \text{range } n} z^{i+1} (b i \cdot a i)$. Then there exists a complex number y such that s tends to y as n tends to infinity.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open scoped BigOperators Topology
open Complex Filter Function Metric Finset

theorem bounded_coefficient_phase_series_converges (z : ℂ) (hz : ‖z‖ = 1) (b : ℕ → ℂ) (B : ℝ) (a : ℕ → ℂ) (g : ℕ → ℝ) (s : ℕ → ℂ)
(hb : ∀ n, ‖b n‖ ≤ B) (hdom : ∀ n, ‖a n‖ ≤ g n) (hsummable : Summable g)
(h : s = fun n => ∑ i ∈ Finset.range n, z ^ (i + 1) * (b i * a i)) :
  ∃ y, Tendsto s atTop (ℳ y) := by
  let f : ℕ → ℂ := fun n => z ^ (n + 1) * (b n * a n)
  have hg : ∀ n, 0 ≤ g n := by
    intro n
    exact le_trans (norm_nonneg (a n)) (hdom n)
  have hmajor : ∀ n, ‖f n‖ ≤ B * g n := by
    intro n
    have hprod : ‖b n‖ * ‖a n‖ ≤ B * g n := by
      nlinarith [hb n, hdom n, hg n, norm_nonneg (b n), norm_nonneg (a n)]
    calc
      ‖f n‖ = ‖b n‖ * ‖a n‖ := by simp [f, norm_mul, norm_pow, hz]
      _ ≤ B * g n := hprod
  have hsum_major : Summable (fun n => B * g n) := by
    simpa [smul_eq_mul] using hsummable.mul_left B
  have hf : Summable f := by
    exact Summable.of_norm_bounded hsum_major hmajor
  have hconv : Tendsto (fun n => ∑ k ∈ Finset.range n, f k) atTop (ℳ (∑' k, f k)) := by
    exact hf.hasSum.tendsto_sum_nat
  refine ⟨∑' k, f k, ?_⟩
  simpa [h, f] using hconv
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Shakarchi_exercise_1_19b__me.me__263cd10d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 09cad9c3a6353e88

4.143 143. summable_majorant_complex_partial_sums

The problem

Let (a_n) be a sequence of complex numbers. If there is a summable nonnegative majorant (g_n) with $\|a_n\| \leq g_n$ for every n , prove that the partial sums of $\sum a_n$ converge.

Lean statement

```
theorem summable_majorant_complex_partial_sums (a : ℕ → ℂ) (g : ℕ → ℝ)
  (hmajorant : Summable g)
  (hdom : ∀ n : ℕ, ‖a n‖ ≤ g n)
  (s : ℕ → ℂ)
  (h : s = (fun n : ℕ => ∑ i ∈ Finset.range n, a i)) :
  ∃ y : ℂ, Tendsto s atTop (ℳ y)
```

Parent. Let (a_n) be a bounded sequence of complex numbers. If the series with terms $\|a_n\|/(n+1)^2$ converges, prove that the partial sums of $\sum a_n/(n+1)^2$ converge.

Parent in Lean

```
theorem bounded_reciprocal_square_convergence (a : ℕ → ℂ) (C : ℝ)
  (hbound : ∀ n : ℕ, ‖a n‖ ≤ C)
  (hbridge : 0 ≤ C → Summable (fun n : ℕ => ‖a n‖ / (n + 1 : ℝ) ^ 2))
  (s : ℕ → ℂ)
  (h : s = (fun n : ℕ => ∑ i ∈ Finset.range n, a i / (i + 1 : ℂ) ^ 2)) :
  ∃ y, Tendsto s atTop (ℳ y)
```

What it contributes. Retains the parent's complex-series-to-partial-sum convergence checkpoint while replacing the reciprocal-square bridge with an arbitrary summable majorant hypothesis.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely replaces the reciprocal-square argument with a general summable majorant and uses norm domination to establish summability of the complex sequence. Its proof requires a different convergence principle and is not recoverable by recalling the parent's proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the reciprocal-square construction with a genuinely general summable-majorant argument and applies a different domination theorem to establish summability of the complex sequence. The parent's denominator-specific norm calculation and bridge hypothesis cannot be recalled to solve it.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the reciprocal-square construction with an arbitrary summable majorant and uses norm domination to prove summability of the complex sequence. This requires a different convergence argument and cannot be solved by recalling the parent's weighted-series proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing

Check	By	Result
corpus_dedup	hash	unique against 2140 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem summable_majorant_complex_partial_sums.extracted_1_1 (a : ℕ → ℂ) (g : ℕ → ℝ) (hmajorant : Summable g)
(hdom : ∀ (n : ℕ), ‖a n‖ ≤ g n) (s : ℕ → ℂ) (h : s = fun n => ∑ i ∈ range n, a i) :
∃ y, Tendsto s atTop (ℳ y)
```

Read back by a model that saw only the Lean. *Let a be a sequence of complex numbers and g a sequence of real numbers. Assume that the series $\sum_n g(n)$ is summable, that for every natural number n , the complex norm $\|a(n)\|$ is at most $g(n)$, and that s is the sequence of complex partial sums defined by $s(n) = \sum_{i < n} a(i)$. Then there exists a complex number y such that $s(n)$ tends to y as n tends to infinity.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open scoped BigOperators Topology
open Complex Filter Function Metric Finset

theorem summable_majorant_complex_partial_sums (a : ℕ → ℂ) (g : ℕ → ℝ)
(hmajorant : Summable g)
(hdom : ∀ n : ℕ, ‖a n‖ ≤ g n)
(s : ℕ → ℂ)
(h : s = (fun n : ℕ => ∑ i ∈ Finset.range n, a i)) :
∃ y : ℂ, Tendsto s atTop (ℳ y) := by
have hsum : Summable a := by
exact Summable.of_norm_bounded hmajorant hdom
refine ⟨∑' n, a n, ?_⟩
have hpartial :
Tendsto (fun n : ℕ => ∑ i ∈ Finset.range n, a i)
atTop (ℳ (∑' n, a n)) :=
hsum.hasSum.tendsto_sum_nat
simpa [h] using hpartial
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Shakarchi_exercise_1_19b__me.me__938439ef

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 938439efcfd451c7

4.144 144. reciprocal_power_partial_sums_converge

The problem

Prove that for every natural number $p \geq 2$, every complex number z with $|z| = 1$, and every sequence s defined by $s_n = \sum_{i=0}^{n-1} \frac{z^{i+1}}{(i+1)^p}$, there exists a complex number y such that s_n converges to y .

Lean statement

```
theorem reciprocal_power_partial_sums_converge (p : ℕ) (hp : 2 ≤ p) (z : ℂ) (hz : ‖z‖ = 1) (s : ℕ → ℂ) (h : s = (fun n => ∑ i ∈ Finset.range n, z ^ (i + 1) / ((i + 1 : ℕ) : ℂ) ^ p)) : ∃ y, Tendsto s atTop (ℳ y)
```

Parent. Prove that the power series $\sum z^n/n^2$ converges at every point of the unit circle.

Parent in Lean

```
theorem Shakarchi_exercise_1_19b (z : ℂ) (hz : ‖z‖ = 1) (s : ℕ → ℂ)
  (h : s = (λ n => ∑ i ∈ (range n), z ^ (i + 1) / (i + 1) ^ 2)) :
  ∃ y, Tendsto s atTop (ℳ y) := by
```

What it contributes. Generalizes the parent's unit-modulus complex partial-sum argument while replacing the fixed square exponent with $p \geq 2$ and explicitly proving the parameterized summability obligation.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the reciprocal-square exponent to an arbitrary natural $p \geq 2$. Its summability argument requires using hp to establish $p > 1$ and applying the p -series theorem at variable exponent p , which the parent's fixed-exponent proof does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the reciprocal-square exponent to an arbitrary natural $p \geq 2$. Its proof must establish p -series summability from hp and transport the resulting norm estimate, which the parent's fixed-exponent argument does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the reciprocal-square exponent to arbitrary $p \geq 2$. Its convergence proof must establish summability for the variable exponent using hp and transport that result through the norm estimate, so recalling the parent's fixed-exponent proof is insufficient.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1934 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem reciprocal_power_partial_sums_converge.extracted_1_1 (p : ℕ) (hp : 2 ≤ p) (z : ℂ) (hz : ‖z‖ = 1) (s : ℕ → ℂ)
  (h : s = fun n => ∑ i ∈ range n, z ^ (i + 1) / ↑(i + 1) ^ p) : ∃ y, Tendsto s atTop (ℳ y)
```

Read back by a model that saw only the Lean. For every natural number p with $2 \leq p$, every complex number z with $\|z\| = 1$, and every sequence $s : \mathbb{N} \rightarrow \mathbb{C}$ satisfying $s(n) = \sum_{i \in \{0, \dots, n-1\}} z^{i+1} / (i+1)^p$ for all $n \in \mathbb{N}$, there exists a complex number y such that s tends to y as n tends to infinity.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Complex Filter Function Metric Finset
open scoped BigOperators Topology

theorem reciprocal_power_partial_sums_converge (p : ℕ) (hp : 2 ≤ p) (z : ℂ) (hz : ‖z‖ = 1) (s : ℕ → ℂ) (h : s = (fun n => ∑ i ∈
  ↪ Finset.range n, z ^ (i + 1) / ((i + 1 : ℕ) : ℂ) ^ p)) : ∃ y, Tendsto s atTop (ℳ y) := by
  classical
  let f : ℕ → ℂ := fun n => z ^ (n + 1) / ((n + 1 : ℕ) : ℂ) ^ p
  have h_norm : (fun n : ℕ => ‖f n‖) = (fun n : ℕ => (1 : ℝ) / ((n + 1 : ℕ) : ℝ) ^ p) := by
    funext n
    have hzpow : ‖z ^ (n + 1)‖ = 1 := by
      simp [norm_pow, hz]
    have hden : ‖((n + 1 : ℕ) : ℂ)‖ = ((n + 1 : ℕ) : ℝ) := by
      simp using (Complex.norm_natCast (n + 1))
    calc
      ‖f n‖ = ‖z ^ (n + 1)‖ / ‖((n + 1 : ℕ) : ℂ) ^ p‖ := by simp [f]
      _ = 1 / ‖((n + 1 : ℕ) : ℂ) ^ p‖ := by rw [hzpow]
      _ = (1 : ℝ) / ((n + 1 : ℕ) : ℝ) ^ p := by rw [norm_pow, hden]
  have h_base : Summable (fun n : ℕ => (1 : ℝ) / (n : ℝ) ^ p) := by
    apply (Real.summable_one_div_nat_pow (p := p)).mpr
  omega
  have h_pSeries : Summable (fun n : ℕ => (1 : ℝ) / ((n + 1 : ℕ) : ℝ) ^ p) := by
    simp [Nat.cast_add, Nat.cast_one] using (summable_nat_add_iff 1).2 h_base
  have h_norm_summable : Summable (fun n : ℕ => ‖f n‖) := by
    rw [h_norm]
    exact h_pSeries
  have hfSummable : Summable f := Summable.of_norm h_norm_summable
  refine ⟨∑' n, f n, ?_⟩
  have hy : HasSum f (∑' n, f n) := hfSummable.hasSum
  have hpartial : Tendsto (fun n => ∑ i ∈ Finset.range n, f i) atTop (ℳ (∑' n, f n)) := by
    exact hy.tendsto_sum_nat
  have htransport : Tendsto s atTop (ℳ (∑' n, f n)) := by
    simp [h, f] using hpartial
  exact htransport
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Shakarchi_exercise_1_19b__me__15f3a09f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c10f3235f84352e8

4.145 145. dominated_phase_series_converges

The problem

Let $|z| = 1$. If the coefficients a_n are dominated in norm by a summable nonnegative sequence g_n , prove that the partial sums of $\sum z^{n+1} a_n$ converge.

Lean statement

```
theorem dominated_phase_series_converges (z : ℂ) (hz : ‖z‖ = 1) (a : ℕ → ℂ) (g : ℕ → ℝ) (s : ℕ → ℂ) (hdom : ∀ n, ‖a n‖ ≤ g n) (hsummable
  ↪ : Summable g) (h : s = fun n => ∑ i ∈ Finset.range n, z ^ (i + 1) * a i) : ∃ y, Tendsto s atTop (ℕ y)
```

Parent. Prove that the power series $\sum z^n/n^2$ converges at every point of the unit circle.

Parent in Lean

```
theorem Shakarchi_exercise_1.19b (z : ℂ) (hz : ‖z‖ = 1) (s : ℕ → ℂ)
  (h : s = (λ n => ∑ i ∈ (range n), z ^ (i + 1) / (i + 1) ^ 2)) :
  ∃ y, Tendsto s atTop (ℕ y) := by
```

What it contributes. Retains the unit-modulus complex phase, partial-sum encoding, and HasSum-based convergence conclusion while replacing reciprocal-square coefficients with a general dominated summable family.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child replaces the fixed coefficients $(n+1)^{-2}$ with an arbitrary complex sequence dominated by a summable real majorant. Its proof requires a new norm-comparison and summability argument, so recalling the parent's p-series proof is insufficient.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed reciprocal-square coefficients to an arbitrary complex sequence dominated by a summable majorant. Its proof requires a norm comparison and the summable comparison theorem, which the parent's proof does not supply.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed $1/(n+1)^2$ coefficients to an arbitrary complex sequence dominated by a summable real majorant. Its proof requires norm domination and a general summability theorem, so recalling the parent's p-series argument does not substantially help.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2172 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem dominated_phase_series_converges.extracted_1_1 (z : ℂ) (hz : ‖z‖ = 1) (a : ℕ → ℂ) (g : ℕ → ℝ) (s : ℕ → ℂ)
  (hdom : ∀ (n : ℕ), ‖a n‖ ≤ g n) (hsummable : Summable g) (h : s = fun n => ∑ i ∈ range n, z ^ (i + 1) * a i) :
  ∃ y, Tendsto s atTop (ℳ y)
```

Read back by a model that saw only the Lean. For a complex number z with norm 1, sequences $a : \mathbb{N} \rightarrow \mathbb{C}$, $g : \mathbb{N} \rightarrow \mathbb{R}$, and $s : \mathbb{N} \rightarrow \mathbb{C}$, suppose that for every natural number n , the norm of $a(n)$ is at most $g(n)$, that the real series $\sum_n g(n)$ is summable, and that for every natural number n , $s(n) = \sum_{i \in \{0, \dots, n-1\}} z^{i+1} a(i)$. Then there exists a complex number y such that $s(n)$ tends to y as n tends to infinity.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open scoped BigOperators Topology
open Complex Filter Function Metric Finset

theorem dominated_phase_series_converges (z : ℂ) (hz : ‖z‖ = 1) (a : ℕ → ℂ) (g : ℕ → ℝ) (s : ℕ → ℂ)
  (hdom : ∀ n, ‖a n‖ ≤ g n) (hsummable : Summable g)
  (h : s = fun n => ∑ i ∈ Finset.range n, z ^ (i + 1) * a i) :
  ∃ y, Tendsto s atTop (ℳ y) := by
  let f : ℕ → ℂ := fun n => z ^ (n + 1) * a n
  have hmajor : ∀ n, ‖f n‖ ≤ g n := by
    intro n
    simpa [f, norm_mul, norm_pow, hz] using hdom n
  have hf : Summable f := by
    exact Summable.of_norm_bounded hsummable hmajor
  have hconv :
    Tendsto (fun n => ∑ k ∈ Finset.range n, f k) atTop (ℳ (∑' k, f k)) := by
    exact hf.hasSum.tendsto_sum_nat
  refine ⟨∑' k, f k, ?_⟩
  simpa [h, f] using hconv
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Shakarchi_exercise_1_19b__me__2f08af31

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 2f08af312633ecb1

4.146 146. phase_weighted_sum_converges

The problem

Let (a_n) be a complex coefficient family and let (c_n) be a summable nonnegative majorant with $\|a_n\| \leq c_n$. Show that for every complex z on the unit circle, the phase-weighted partial sums $\sum_{k < n} a_k z^{k+1}$ converge, with limit $\sum_{n \geq 0} a_n z^{n+1}$.

Lean statement

```
theorem phase_weighted_sum_converges
  (z : ℂ) (hz : ‖z‖ = 1) (a : ℕ → ℂ) (c : ℕ → ℝ)

  (hc : Summable c)
  (hbound : ∀ n, ‖a n‖ ≤ c n)
  (s : ℕ → ℂ)
  (h : s = (fun n => ∑ i ∈ Finset.range n, a i * z ^ (i + 1))) :
  ∃ y, Tendsto s atTop (ℳ y) ∧ y = ∑' n, a n * z ^ (n + 1)
```

Parent. Prove that the power series $\sum z^n/n^2$ converges at every point of the unit circle.

Parent in Lean

```
theorem Shakarchi_exercise_1.19b (z : ℂ) (hz : ‖z‖ = 1) (s : ℕ → ℂ)
  (h : s = (λ n => ∑ i ∈ (range n), z ^ (i + 1) / (i + 1) ^ 2)) :
  ∃ y, Tendsto s atTop (ℳ y) := by
```

What it contributes. Retains the complex phase family, finite partial-sum encoding, and HasSum-to-Tendsto endpoint, while replacing the reciprocal-square coefficients with a parameterized majorant-controlled family.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the parent from the fixed coefficients $1/(n+1)^2$ to an arbitrary coefficient family dominated by an explicitly summable majorant. Its proof requires norm comparison and summability of the weighted series, rather than recalling the parent's p-series argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed coefficients $1/(n+1)^2$ to an arbitrary complex coefficient family dominated by a summable real majorant, and additionally identifies the limit as the infinite sum. Proving summability via norm comparison is new reasoning not supplied by the parent's specific p-series argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child substantially generalizes the parent from the specific coefficients $1/(n+1)^2$ to an arbitrary complex coefficient family dominated by a summable real majorant. Its proof requires a norm comparison and summability argument, rather than recalling the parent's p-series proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing

Check	By	Result
corpus_dedup	hash	unique against 2739 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem phase_weighted_sum_converges.extracted_1.1 (z : ℂ) (hz : ‖z‖ = 1) (a : ℕ → ℂ) (c : ℕ → ℝ)
(hc_nonneg : ∀ (n : ℕ), 0 ≤ c n) (hc : Summable c) (hbound : ∀ (n : ℕ), ‖a n‖ ≤ c n) (s : ℕ → ℂ)
(h : s = fun n => ∑ i ∈ range n, a i * z ^ (i + 1)) :
  ∃ y, Tendsto s atTop (λ y) λ y = ∑' (n : ℕ), a n * z ^ (n + 1)
```

Read back by a model that saw only the Lean. For a complex number z with norm $\|z\| = 1$, sequences $a : \mathbb{N} \rightarrow \mathbb{C}$ and $c : \mathbb{N} \rightarrow \mathbb{R}$ such that $c n \geq 0$ for every n , the series $\sum' n, c n$ is summable, and $\|a n\| \leq c n$ for every n , and for a sequence $s : \mathbb{N} \rightarrow \mathbb{C}$ satisfying $s n = \sum_{i \in \{0, \dots, n-1\}} a i z^{(i+1)}$ for every n , there exists a complex number y such that s tends to y as n tends to infinity and $y = \sum' n, a n z^{(n+1)}$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Complex Filter Function Metric Finset
open scoped BigOperators Topology

theorem phase_weighted_sum_converges
  (z : ℂ) (hz : ‖z‖ = 1) (a : ℕ → ℂ) (c : ℕ → ℝ)
  (hc : Summable c)
  (hbound : ∀ n, ‖a n‖ ≤ c n)
  (s : ℕ → ℂ)
  (h : s = (fun n => ∑ i ∈ Finset.range n, a i * z ^ (i + 1))) :
  ∃ y, Tendsto s atTop (λ y) λ y = ∑' n, a n * z ^ (n + 1) := by
  classical
  let f : ℕ → ℂ := fun n => a n * z ^ (n + 1)
  have h_norm_bound : ∀ n, ‖f n‖ ≤ c n := by
    intro n
    simp [f, norm_mul, norm_pow, hz] using hbound n
  have hfSummable : Summable f := by
    exact Summable.of_norm_bounded hc h_norm_bound
  have hy : HasSum f (∑' n, f n) := hfSummable.hasSum
  refine (∑' n, f n, ?_, ?_)
  · simp [h, f] using hy.tendsto_sum_nat
  · simp [f]
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Shakarchi_exercise_1_19b__me__3eeca7f4

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 20152abf32d46eef

4.147 147. bounded_reciprocal_square_convergence

The problem

Let (a_n) be a bounded sequence of complex numbers. If the series with terms $\|a_n\|/(n+1)^2$ converges, prove that the partial sums of $\sum a_n/(n+1)^2$ converge.

Lean statement

```
theorem bounded_reciprocal_square_convergence (a : ℕ → ℂ) (C : ℝ)
  (hbound : ∀ n : ℕ, ‖a n‖ ≤ C)
  (hbridge : 0 ≤ C → Summable (fun n : ℕ => ‖a n‖ / (n + 1 : ℝ) ^ 2))
  (s : ℕ → ℂ)
  (h : s = (fun n : ℕ => ∑ i ∈ Finset.range n, a i / (i + 1 : ℂ) ^ 2)) :
  ∃ y, Tendsto s atTop (ℳ y)
```

Parent. Prove that the power series $\sum z^n/n^2$ converges at every point of the unit circle.

Parent in Lean

```
theorem Shakarchi_exercise_1_19b (z : ℂ) (hz : ‖z‖ = 1) (s : ℕ → ℂ)
  (h : s = (λ n => ∑ i ∈ (range n), z ^ (i + 1) / (i + 1) ^ 2)) :
  ∃ y, Tendsto s atTop (ℳ y) := by
```

What it contributes. Preserves the parent's reciprocal-square complex partial-sum and HasSum-to-Tendsto structure, while replacing the unit-modulus numerator with a bounded coefficient sequence and an explicit norm-summability bridge.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

The child replaces the fixed unit-modulus coefficient sequence with an arbitrary complex sequence and must establish convergence through absolute summability of its norm terms. Although the bridge hypothesis supplies that summability directly, the parent's power-norm argument cannot simply be recalled to solve the new statement.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely replaces the fixed unit-modulus phase with an arbitrary bounded coefficient sequence and proves convergence by establishing norm-summability from the new bridge hypothesis, then applying absolute convergence. The parent's p-series argument does not supply this reasoning, and the added hypotheses are used materially.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the fixed unit-modulus phase sequence with an arbitrary complex coefficient sequence and proves convergence via a supplied norm-summability bridge, including the nontrivial derivation that the bound constant is nonnegative. The parent's p-series argument cannot be recalled to solve this generalized statement.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing

Check	By	Result
corpus_dedup	hash	unique against 2120 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_reciprocal_square_convergence.extracted_1_1 (a : ℕ → ℂ) (C : ℝ) (hbound : ∀ (n : ℕ), ‖a n‖ ≤ C)
(hbridge : 0 ≤ C → Summable fun n => ‖a n‖ / (↑n + 1) ^ 2) (s : ℕ → ℂ)
(h : s = fun n => ∑ i ∈ range n, a i / (↑i + 1) ^ 2) : ∃ y, Tendsto s atTop (ℕ y)
```

Read back by a model that saw only the Lean. *Let a be a sequence of complex numbers, let C be a real number, and suppose that for every natural number n , the norm of $a(n)$ is at most C . Suppose also that if C is nonnegative, then the series with terms $\|a(n)\|/(n+1)^2$ is summable. Let s be a sequence of complex numbers satisfying, for every natural number n , $s(n) = \sum_{i < n} a(i)/(i+1)^2$. Then there exists a complex number y such that $s(n)$ tends to y as n tends to infinity.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open scoped BigOperators
open Complex Filter Function Metric Finset

theorem bounded_reciprocal_square_convergence (a : ℕ → ℂ) (C : ℝ)
(hbound : ∀ n : ℕ, ‖a n‖ ≤ C)
(hbridge : 0 ≤ C → Summable (fun n : ℕ => ‖a n‖ / (n + 1 : ℝ) ^ 2))
(s : ℕ → ℂ)
(h : s = (fun n : ℕ => ∑ i ∈ Finset.range n, a i / (i + 1 : ℂ) ^ 2)) :
∃ y, Tendsto s atTop (ℕ y) := by
classical
set f : ℕ → ℂ := fun n => a n / (n + 1 : ℂ) ^ 2 with hf
have hC_nonneg : 0 ≤ C :=
le_trans (norm_nonneg (a 0)) (hbound 0)
have hbridge_used :
Summable (fun n : ℕ => ‖a n‖ / (n + 1 : ℝ) ^ 2) :=
hbridge hC_nonneg
have hnorm : ∀ n : ℕ, ‖f n‖ = ‖a n‖ / (n + 1 : ℝ) ^ 2 := by
intro n
have hden : ‖(n + 1 : ℂ) ^ 2‖ = (n + 1 : ℝ) ^ 2 := by
rw [norm_pow]
have hn : ‖(n + 1 : ℂ)‖ = (n + 1 : ℝ) := by
simp [Nat.cast_add, Nat.cast_one] using (Complex.norm_natCast (n + 1))
rw [hn]
calc
‖f n‖ = ‖a n‖ / ‖(n + 1 : ℂ) ^ 2‖ := by
rw [hf, norm_div]
_ = ‖a n‖ / (n + 1 : ℝ) ^ 2 := by rw [hden]
have hnorm_summable : Summable (fun n : ℕ => ‖f n‖) := by
rw [show (fun n : ℕ => ‖f n‖) =
(fun n : ℕ => ‖a n‖ / (n + 1 : ℝ) ^ 2) by
funext n
exact hnorm n]
exact hbridge_used
have hfSummable : Summable f := Summable.of_norm hnorm_summable
refine ⟨∑' n, f n, ?_⟩
have hpartial :
Tendsto (fun n : ℕ => ∑ k ∈ Finset.range n, f k)
atTop (ℕ (∑' n, f n)) :=
hfSummable.hasSum.tendsto_sum_nat
simp [h, f] using hpartial
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Shakarchi_exercise_1_19b__me__8adda5dd

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fin-

gerprint 8adda5dd083d89a9

4.148 148. cubic_denominator_linear_growth_convergence

The problem

Let (a_n) be a complex sequence and suppose there is a real constant C such that $|a_n| \leq C(n+1)$ for every n . Prove that the series $\sum_{n=0}^{\infty} a_n/(n+1)^3$ converges.

Lean statement

```
theorem cubic_denominator_linear_growth_convergence (a : ℕ → ℂ) (C : ℝ)
  (ha : ∀ n : ℕ, ‖a n‖ ≤ C * (n + 1 : ℝ))
  (s : ℕ → ℂ)
  (h : s = (fun n : ℕ => ∑ i ∈ Finset.range n, a i / (i + 1 : ℂ) ^ 3)) :
  ∃ y, Tendsto s atTop (ℳ y)
```

Parent. Prove that the power series $\sum z^n/n^2$ converges at every point of the unit circle.

Parent in Lean

```
theorem Shakarchi_exercise_1_19b (z : ℂ) (hz : ‖z‖ = 1) (s : ℕ → ℂ)
  (h : s = (λ n => ∑ i ∈ (range n), z ^ (i + 1) / (i + 1) ^ 2)) :
  ∃ y, Tendsto s atTop (ℳ y) := by
```

What it contributes. Reuses the parent’s reciprocal-square p-series checkpoint and partial-sum convergence pattern, while replacing the fixed phase numerator with a variable sequence and a cubic-denominator majorant.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the parent from unit-modulus powers and quadratic denominators to arbitrary complex coefficients with linear norm growth and cubic denominators. Its proof requires a new norm comparison against a shifted reciprocal-square majorant and does not reuse the parent’s argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is a genuine bounded generalization: it replaces the unit-modulus numerator with arbitrary coefficients having linear norm growth, then proves convergence via cubic-denominator normalization and a shifted reciprocal-square majorant. This requires comparison and norm-growth reasoning absent from the parent’s fixed reciprocal-square proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the unit-modulus reciprocal-square series with a genuinely different convergence theorem: linearly growing coefficients over cubic denominators are reduced to a shifted reciprocal-square majorant. Its comparison and normalization argument is not supplied by the parent’s fixed-coefficient proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2898 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem cubic_denominator_linear_growth_convergence.extracted_1_1 (a : ℕ → ℂ) (C : ℝ)
  (ha : ∀ (n : ℕ), ‖a n‖ ≤ C * (n + 1)) (s : ℕ → ℂ) (h : s = fun n => ∑ i ∈ range n, a i / (i + 1) ^ 3) :
  ∃ y, Tendsto s atTop (ℕ y)
```

Read back by a model that saw only the Lean. For a sequence $a : \mathbb{N} \rightarrow \mathbb{C}$ and a real constant C , suppose that for every natural number n , the complex norm of $a(n)$ is at most $C(n+1)$. Let $s : \mathbb{N} \rightarrow \mathbb{C}$ be defined by $s(n) = \sum_{i=0}^{n-1} \{a(i)/(i+1)^3\}$. Then there exists a complex number y such that $s(n)$ tends to y as n tends to infinity.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Complex Filter Function Metric Finset
open scoped BigOperators Topology

theorem cubic_denominator_linear_growth_convergence (a : ℕ → ℂ) (C : ℝ)
  (ha : ∀ n : ℕ, ‖a n‖ ≤ C * (n + 1 : ℝ))
  (s : ℕ → ℂ)
  (h : s = (fun n : ℕ => ∑ i ∈ Finset.range n, a i / (i + 1 : ℂ) ^ 3)) :
  ∃ y, Tendsto s atTop (ℕ y) := by
  classical
  let f : ℕ → ℂ := fun n => a n / (n + 1 : ℂ) ^ 3
  have hbase : Summable (fun n : ℕ => (1 : ℝ) / (n + 1 : ℝ) ^ 2) := by
    have hnat : Summable (fun n : ℕ => (1 : ℝ) / (n : ℝ) ^ 2) :=
      (Real.summable_one_div_nat_pow (p := 2)).2 (by norm_num)
    simpa [Nat.cast_add, Nat.cast_one] using (summable_nat_add_iff 1).mpr hnat
  have hmajorant : Summable (fun n : ℕ => C / (n + 1 : ℝ) ^ 2) := by
    simpa [div_eq_mul_inv, mul_comm] using hbase.mul_left C
  have hnorm_bound : ∀ n : ℕ, ‖f n‖ ≤ C / (n + 1 : ℝ) ^ 2 := by
    intro n
    have hden : ‖(n + 1 : ℂ) ^ 3‖ = (n + 1 : ℝ) ^ 3 := by
      rw [norm_pow]
    have hn : ‖(n + 1 : ℂ)‖ = (n + 1 : ℝ) := by
      simpa [Nat.cast_add, Nat.cast_one] using (Complex.norm_natCast (n + 1))
    rw [hn]
  calc
    ‖f n‖ = ‖a n‖ / (n + 1 : ℝ) ^ 3 := by
      simp [f, norm_div, hden]
    _ ≤ (C * (n + 1 : ℝ)) / (n + 1 : ℝ) ^ 3 := by
      exact div_le_div_of_nonneg_right (ha n) (by positivity)
    _ = C / (n + 1 : ℝ) ^ 2 := by
      field_simp
  have hfSummable : Summable f :=
    Summable.of_norm_bounded hmajorant hnorm_bound
  refine (Σ' n, f n, ?_)
  have hpartial :
    Tendsto (fun n : ℕ => ∑ k ∈ Finset.range n, f k)
      atTop (ℕ (Σ' n, f n)) :=
    hfSummable.hasSum.tendsto_sum_nat
  simpa [h, f] using hpartial
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Shakarchi_exercise_1_19b__me__8feb093a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint 8feb093a58671b9d

4.149 149. reciprocal_power_partial_sums_converge

The problem

Let $p \geq 2$. Prove that the power series $\sum z^n/n^p$ converges at every point z on the unit circle.

Lean statement

```
theorem reciprocal_power_partial_sums_converge (z : ℂ) (p : ℕ) (hp : 2 ≤ p) (hz : ‖z‖ = 1)
  (s : ℕ → ℂ)
  (h : s = (fun n => ∑ i ∈ Finset.range n, z ^ (i + 1) / (i + 1 : ℂ) ^ p)) :
  ∃ y, Tendsto s atTop (ℳ y)
```

Parent. Prove that the power series $\sum z^n/n^2$ converges at every point of the unit circle.

Parent in Lean

```
theorem Shakarchi_exercise_1_19b (z : ℂ) (hz : ‖z‖ = 1) (s : ℕ → ℂ)
  (h : s = (λ n => ∑ i ∈ (range n), z ^ (i + 1) / (i + 1) ^ 2)) :
  ∃ y, Tendsto s atTop (ℳ y) := by
```

What it contributes. Preserves the parent's unit-circle norm argument and partial-sum convergence route, while replacing exponent 2 with p and adding reciprocal-square domination.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the reciprocal-square series to every integer exponent $p \geq 2$. Its proof requires a new comparison argument showing $(n+1)^2 \leq (n+1)^p$ and transferring summability, which is not supplied by the parent's fixed-exponent proof.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the reciprocal-power exponent and must prove summability for every integer $p \geq 2$ by comparing the p -series to the square series. This reasoning is not supplied by the parent's fixed-exponent proof, and no hypotheses are redundant.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2116 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem reciprocal_power_partial_sums_converge.extracted_1_1 (z : ℂ) (p : ℕ) (hp : 2 ≤ p) (hz : ‖z‖ = 1) (s : ℕ → ℂ)
  (h : s = fun n => ∑ i ∈ range n, z ^ (i + 1) / (↑i + 1) ^ p) : ∃ y, Tendsto s atTop (ℳ y)
```

Read back by a model that saw only the Lean. For a complex number z and a natural number p with $2 \leq p$, if $\|z\| = 1$, and if $s : \mathbb{N} \rightarrow \mathbb{C}$ satisfies $s(n) = \sum_{i \in \text{range } n} z^{i+1} / ((i : \mathbb{C}) + 1)^p$ for every natural number n , then there exists a complex number y such that s tends to y as n tends to infinity.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Complex Filter Function Metric Finset
open scoped BigOperators Topology

theorem reciprocal_power_partial_sums_converge (z : ℂ) (p : ℕ) (hp : 2 ≤ p) (hz : ‖z‖ = 1)
  (s : ℕ → ℂ)
  (h : s = (fun n => ∑ i ∈ Finset.range n, z ^ (i + 1) / (i + 1 : ℂ) ^ p)) :
  ∃ y, Tendsto s atTop (ℕ y) := by
  classical
  let f : ℕ → ℂ := fun n => z ^ (n + 1) / (n + 1 : ℂ) ^ p
  have h_norm : (fun n : ℕ => ‖f n‖) =
    (fun n : ℕ => (1 : ℝ) / (n + 1 : ℝ) ^ p) := by
    funext n
    have hbase : ‖(n + 1 : ℂ)‖ = (n + 1 : ℝ) := by
      simp using (Complex.norm_natCast (n + 1))
    simp [f, hz, hbase]
  have h_square : Summable (fun n : ℕ => (1 : ℝ) / (n + 1 : ℝ) ^ 2) := by
    have hs : Summable (fun n : ℕ => (1 : ℝ) / (n : ℝ) ^ 2) :=
      (Real.summable_one_div_nat_pow (p := 2)).mpr one_lt_two
    simp [Nat.cast_add, Nat.cast_one] using (summable_nat_add_iff 1).2 hs
  have h_compare : ∀ n : ℕ, 0 ≤ (1 : ℝ) / (n + 1 : ℝ) ^ p ∧
    (1 : ℝ) / (n + 1 : ℝ) ^ p ≤ (1 : ℝ) / (n + 1 : ℝ) ^ 2 := by
    intro n
    let x : ℝ := n + 1
    have hx : 1 ≤ x := by
      dsimp [x]
      exact_mod_cast (Nat.succ_le_succ (Nat.zero_le n))
    have hpow : x ^ 2 ≤ x ^ p := pow_le_pow_right hx hp
    have hpos : 0 < x ^ 2 := pow_pos (by linarith) _
    constructor
    · positivity
    · exact one_div_le_one_div_of_le hpos hpow
  have h_p : Summable (fun n : ℕ => (1 : ℝ) / (n + 1 : ℝ) ^ p) :=
    Summable.of_nonneg_of_le (fun n => (h_compare n).1) (fun n => (h_compare n).2) h_square
  have h_norm_summable : Summable (fun n : ℕ => ‖f n‖) := by
    rw [h_norm]
    exact h_p
  have hf : Summable f := Summable.of_norm h_norm_summable
  refine (∑' n, f n, ?_)
  have hy : HasSum f (∑' n, f n) := hf.hasSum
  have h_partial : Tendsto (fun n => ∑ i ∈ Finset.range n, f i) atTop (ℕ (∑' n, f n)) :=
    hy.tendsto_sum_nat
  simp [h, f] using h_partial
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Shakarchi_exercise_1_19b__me__9361aa77

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint 9361aa775d5ad8a3

4.150 150. scaled_unit_circle_p_series_converges

The problem

Let c be a fixed complex number. Prove that the series obtained by multiplying z^n/n^2 by c converges at every point z on the unit circle.

Lean statement

```
theorem scaled_unit_circle_p_series_converges
  (c z : ℂ) (hz : ‖z‖ = 1) (s : ℕ → ℂ)
  (h : s = fun n => ∑ i ∈ Finset.range n,
    c * (z ^ (i + 1) / (i + 1 : ℂ) ^ 2)) :
  ∃ y, Tendsto s atTop (ℳ y)
```

Parent. Prove that the power series $\sum z^n/n^2$ converges at every point of the unit circle.

Parent in Lean

```
theorem Shakarchi_exercise_1_19b (z : ℂ) (hz : ‖z‖ = 1) (s : ℕ → ℂ)
  (h : s = (λ n => ∑ i ∈ (range n), z ^ (i + 1) / (i + 1) ^ 2)) :
  ∃ y, Tendsto s atTop (ℳ y) := by
```

What it contributes. Uses the parent’s unit-circle norm estimate, shifted p-series summability checkpoint, and partial-sum convergence architecture; mutation adds a fixed scalar multiplier and explicitly consumes the base summability through ‘Summable.mul_left’.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The scalar parameter c is genuinely used through Summable.mul_left, so the parent’s unscaled summability argument does not directly supply the child’s proof. The child also explicitly derives norm summability before transporting it through scalar multiplication and identifying the partial sums.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the series by an arbitrary complex scalar and must establish summability through the new ‘Summable.mul_left c’ step. The parent’s proof does not supply this scalar-dependent argument by recall alone.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child introduces an arbitrary complex scalar and must establish summability of the scaled sequence via ‘Summable.mul_left c’ before identifying its partial sums. This is not solvable by simply recalling the parent’s unscaled convergence proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2932 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem scaled_unit_circle_p_series_converges.extracted_1_1 (c z : ℂ) (hz : ‖z‖ = 1) (s : ℕ → ℂ)
  (h : s = fun n => ∑ i ∈ range n, c * (z ^ (i + 1) / (i + 1) ^ 2)) : ∃ y, Tendsto s atTop (ℕ y)
```

Read back by a model that saw only the Lean. For complex numbers c and z with $\|z\| = 1$, and a sequence $s : \mathbb{N} \rightarrow \mathbb{C}$ satisfying $s(n) = \sum_{i < n} c \cdot (z^{i+1} / (i+1)^2)$ for every natural number n , there exists a complex number y such that $s(n)$ converges to y as n tends to infinity.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open scoped BigOperators Topology
open Complex Filter Function Metric Finset

theorem scaled_unit_circle_p_series_converges
  (c z : ℂ) (hz : ‖z‖ = 1) (s : ℕ → ℂ)
  (h : s = fun n => ∑ i ∈ Finset.range n,
    c * (z ^ (i + 1) / (i + 1 : ℂ) ^ 2)) :
  ∃ y, Tendsto s atTop (ℕ y) := by
  classical
  let f : ℕ → ℂ := fun n => z ^ (n + 1) / (n + 1 : ℂ) ^ 2
  have h_norm :
    (fun n : ℕ => ‖f n‖) =
      fun n : ℕ => (1 : ℝ) / (n + 1 : ℝ) ^ 2 := by
  funext n
  have hzpow : ‖z ^ (n + 1)‖ = 1 := by
    rw [norm_pow, hz]
  norm_num
  have hden : ‖(n + 1 : ℂ)‖ = (n + 1 : ℝ) := by
    simp using (Complex.norm_natCast (n + 1))
  calc
    ‖f n‖ = ‖z ^ (n + 1)‖ / ‖(n + 1 : ℂ) ^ 2‖ := by
      simp [f]
    _ = 1 / ‖(n + 1 : ℂ) ^ 2‖ := by rw [hzpow]
    _ = (1 : ℝ) / (n + 1 : ℝ) ^ 2 := by
      rw [norm_pow, hden]
  have h_pSeries : Summable (fun n : ℕ => (1 : ℝ) / (n + 1 : ℝ) ^ 2) := by
    have h_base :
      Summable (fun n : ℕ => (1 : ℝ) / (n : ℝ) ^ 2) :=
        (Real.summable_one_div_nat_pow (p := 2)).mpr one_lt_two
    simp [Nat.cast_add, Nat.cast_one] using
      (summable_nat_add_iff 1).2 h_base
  have h_norm_summable : Summable f := by
    apply Summable.of_norm
    rw [h_norm]
    exact h_pSeries
  have h_scaled : Summable (fun n : ℕ => c * f n) := by
    exact h_norm_summable.mul_left c
  refine ⟨∑' n, c * f n, ?_⟩
  have hy : HasSum (fun n : ℕ => c * f n) (∑' n, c * f n) :=
    h_scaled.hasSum
  simp [h, f] using hy.tendsto_sum_nat
```

Operator. mutation / mutation_easy **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Shakarchi_exercise_1_19b__me__eb1129bf

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint eb1129bff69b752e

4.151 151. bounded_unit_circle_power_series_hasSum

The problem

Let p be a natural number greater than 1, let z lie on the unit circle, and let (a_n) be a bounded complex sequence. Prove that the series $\sum_{n \geq 0} a_n z^{n+1} / (n+1)^p$ has a sum.

Lean statement

```
theorem bounded_unit_circle_power_series_hasSum
  (z : ℂ) (hz : ‖z‖ = 1) (p : ℕ) (hp : 1 < p)
  (a : ℕ → ℂ) (B : ℝ)
  (hbound : ∀ n : ℕ, ‖a n‖ ≤ B) :
  ∃ y : ℂ, HasSum (fun n : ℕ => a n * z ^ (n + 1) / (n + 1 : ℂ) ^ p) y
```

Parent. Let p be an integer greater than 1. Prove that the power series $\sum z^n / n^p$ has a sum at every point z on the unit circle.

Parent in Lean

```
theorem Shakarchi_exercise_1.19b_mutation (z : ℂ) (hz : ‖z‖ = 1) (p : ℕ) (hp : 1 < p) :
  ∃ y : ℂ, HasSum (fun n : ℕ => z ^ (n + 1) / (n + 1 : ℂ) ^ p) y := by
```

What it contributes. Supplied the unit-circle norm reduction and shifted reciprocal-power summability checkpoint.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the numerator to an arbitrary uniformly bounded complex family and requires a norm-majorant comparison argument. The parent's exact norm computation cannot be recalled to solve this without the new bounded-coefficient reasoning.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the numerator to an arbitrary bounded complex family and must establish a new norm majorization using the bound. The parent proof only handles the constant numerator and cannot be recalled to solve this claim.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the constant numerator to an arbitrary bounded complex coefficient family. Its proof requires a new norm-majorization argument and uses the bound hypothesis essentially, so recalling the parent's norm equality proof is insufficient.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2751 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_unit_circle_power_series_hasSum.extracted_1_1 (z : ℂ) (hz : ‖z‖ = 1) (p : ℕ) (hp : 1 < p) (a : ℕ → ℂ)
  (B : ℝ) (hB : 0 ≤ B) (hbound : ∀ (n : ℕ), ‖a n‖ ≤ B) :
  ∃ y, HasSum (fun n => a n * z ^ (n + 1) / (↑n + 1) ^ p) y
```

Read back by a model that saw only the Lean. For every complex number z with norm $\|z\| = 1$, every natural number p with $1 < p$, every function $a : \mathbb{N} \rightarrow \mathbb{C}$, and every real number $B \geq 0$ such that $\|a(n)\| \leq B$ for all natural numbers n , there exists a complex number y such that the series $\sum_{n=0}^{\infty} a(n) z^{n+1} / (n+1)^p$ converges to y in the sense of `HasSum`.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Complex Filter Function Metric Finset
open scoped BigOperators Topology

theorem bounded_unit_circle_power_series_hasSum
  (z : ℂ) (hz : ‖z‖ = 1) (p : ℕ) (hp : 1 < p)
  (a : ℕ → ℂ) (B : ℝ)
  (hbound : ∀ n : ℕ, ‖a n‖ ≤ B) :
  ∃ y : ℂ, HasSum (fun n : ℕ => a n * z ^ (n + 1) / (n + 1 : ℂ) ^ p) y := by
  classical
  let f : ℕ → ℂ := fun n => a n * z ^ (n + 1) / (n + 1 : ℂ) ^ p
  have h_base : Summable (fun n : ℕ => (1 : ℝ) / (n : ℝ) ^ p) :=
    (Real.summable_one_div_nat_pow (p := p)).mpr hp
  have h_pSeries : Summable (fun n : ℕ => (1 : ℝ) / (n + 1 : ℝ) ^ p) := by
    simpa [Nat.cast_add, Nat.cast_one] using (summable_nat_add_iff 1).2 h_base
  have h_majorant : Summable (fun n : ℕ => B * ((1 : ℝ) / (n + 1 : ℝ) ^ p)) :=
    Summable.mul_left B h_pSeries
  have h_norm_summable : Summable (fun n : ℕ => ‖f n‖) := by
    apply h_majorant.of_nonneg_of_le
    · intro n
      exact norm_nonneg _
    · intro n
      have hzpow : ‖z ^ (n + 1)‖ = 1 := by
        simp [hz, norm_pow]
      have hn1 : ‖(n + 1 : ℂ)‖ = (n + 1 : ℝ) := by
        simpa using (Complex.norm_natCast (n + 1))
      have hden : ‖(n + 1 : ℂ) ^ p‖ = (n + 1 : ℝ) ^ p := by
        rw [norm_pow, hn1]
      calc
        ‖f n‖ = ‖a n‖ / ‖(n + 1 : ℂ) ^ p‖ := by simp [f, hz]
        _ = ‖a n‖ * (1 / (n + 1 : ℝ) ^ p) := by rw [hden]; ring
        _ ≤ B * (1 / (n + 1 : ℝ) ^ p) := by
          exact mul_le_mul_of_nonneg_right (hbound n) (by positivity)
  have hfSummable : Summable f := Summable.of_norm h_norm_summable
  refine (Σ' n, f n, ?_)
  have hsum : HasSum f (Σ' n, f n) := hfSummable.hasSum
  simpa [f] using hsum
```

Operator. mutation / mutation_easy **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Shakarchi_exercise_1_19b__mh.me__b7093508

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 756af0834f5f75eb

4.152 152. finite_field_prod_units_root_characterization

The problem

Let K be a field with finitely many nonzero elements. Prove that the product of all nonzero elements of K is either 1, or else is not 1 and is the unique nonidentity element z satisfying $z^2 = 1$.

Lean statement

```
theorem finite_field_prod_units_root_characterization {K : Type*} [Field K] [Fintype K*] :
  (∏ x : K*, x) = 1 ∨
  ((∏ x : K*, x) ≠ 1 ∧ ∃ z : K*, z ^ 2 = 1 → z = 1 ∨ z = ∏ x : K*, x) := by
```

Parent. Let K be a finite field. Prove that the product of the nonzero elements of K is -1 .

Parent in Lean

```
theorem Artin_exercise_13_6_10 {K : Type*} [Field K] [Fintype K*] :
  (∏ x : K*, x) = -1 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a genuine characterization of all square roots of unity in the unit group, requiring factorization in the field and a characteristic-two case split. The parent's product identity alone does not supply this root analysis.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine root-characterization obligation: it factors z^2-1 in the field and handles the resulting cases, beyond the parent's product identity. Although it reuses the parent lemma for the product, the new universal statement is not supplied by the parent's proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child retains the product identity but additionally proves a complete characterization of square roots of unity in the unit group via factorization and the domain property. That root-classification argument is not supplied by the parent's direct product lemma, so recalling the parent does not solve the child.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1972 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_field_prod_units_root_characterization.extracted_1_1.{u_1} {K : Type u_1} [inst : Field K]
  [inst_1 : Fintype K*] :  $\prod x, x = 1 \vee \prod x, x \neq 1 \wedge \forall (z : K^*), z^2 = 1 \rightarrow z = 1 \vee z = \prod x, x$ 
```

Read back by a model that saw only the Lean. *Let K be a field such that its group of units K^x is finite. Then either the product of all units of K equals 1, or that product is not equal to 1 and every unit z satisfying $z^2 = 1$ is either 1 or equal to the product of all units.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open scoped BigOperators
open Function Fintype Subgroup Ideal Polynomial Submodule Zsqrtd

theorem finite_field_prod_units_root_characterization {K : Type*} [Field K] [Fintype K*] :
  ( $\prod x : K^*, x$ ) = 1  $\vee$ 
  (( $\prod x : K^*, x$ )  $\neq$  1  $\wedge \forall z : K^*, z^2 = 1 \rightarrow z = 1 \vee z = \prod x : K^*, x$ ) := by
  classical
  have hprod : ( $\prod x : K^*, x$ ) = (-1 : K*) :=
    FiniteField.prod_univ_units_id_eq_neg_one (K := K)
  have hroot :  $\forall z : K^*, z^2 = 1 \rightarrow z = 1 \vee z = (-1 : K^*)$  := by
    intro z hz
    have hzK : (z : K) ^ 2 = 1 := by
      simp using congrArg (fun u : K* => (u : K)) hz
    have hfac : ((z : K) - 1) * ((z : K) + 1) = 0 := by
      calc
        ((z : K) - 1) * ((z : K) + 1) = (z : K) ^ 2 - 1 := by ring
        _ = 0 := by rw [hzK]; ring
    rcases mul_eq_zero.mp hfac with hzero | hzero
    · left
      apply Units.ext
      have hval : (z : K) = 1 := by linear_combination hzero
      simp using hval
    · right
      apply Units.ext
      have hval : (z : K) = -1 := by linear_combination hzero
      simp using hval
  rw [hprod]
  by_cases heq : (-1 : K*) = 1
  · exact Or.inl heq
  · right
    constructor
    · exact heq
    · intro z hz
      exact hroot z hz
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Artin_exercise_13_6_10__mh__96bd728d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 7f6c646d04f96f55

4.153 153. even_finite_group_has_fixed_point_free_left_involution

The problem

Prove that every finite group of even order contains an element x of order 2 such that left multiplication by x is a fixed-point-free involution.

Lean statement

```
theorem even_finite_group_has_fixed_point_free_left_involution {G : Type*} [Group G] [Fintype G]
  (hG : Even (Fintype.card G)) :
  ∃ x : G, orderOf x = 2 ∧ ∃ f : G → G,
    f = (fun y => x * y) ∧ Function.Involutive f ∧ ∀ y : G, f y ≠ y
```

Parent. Prove that a group of even order contains an element of order 2.

Parent in Lean

```
theorem Artin_exercise_2_11_3 {G : Type*} [Group G] [Fintype G]
  (hG : Even (card G)) : ∃ x : G, orderOf x = 2 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child uses the parent's order-2 witness but adds genuinely nontrivial mathematics: deriving $x*x = 1$ and $x \neq 1$, then proving left multiplication by x is involutive and has no fixed points via cancellation.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child preserves the order-2 existence but adds a genuinely new construction: left multiplication by that element must be proved involutive and fixed-point free. These obligations require deriving $x^2 = 1$ and $x \neq 1$, reasoning not supplied by the parent's Cauchy argument alone.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds substantive structure: it must construct the left-translation involution and prove both involutivity and absence of fixed points from order two. The parent only supplies the order-two element, so recalling its proof does not discharge the new obligations.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1904 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem even_finite_group_has_fixed_point_free_left_involution.extracted_1.1.{u_1} {G : Type u_1} [inst : Group G]
  [inst.1 : Fintype G] (hG : Even (card G)) :
  ∃ x, orderOf x = 2 ∧ ∃ f, (f = fun y => x * y) ∧ Involutive f ∧ ∀ (y : G), f y ≠ y
```

Read back by a model that saw only the Lean. *For every finite group G of even cardinality, there exists an element x of G whose order is 2 and a function $f: G \rightarrow G$ such that f is the left-multiplication map $f(y) = x \cdot y$, f is involutive ($f(f(y)) = y$ for every $y \in G$), and $f(y) \neq y$ for every $y \in G$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Function Fintype Subgroup Ideal Polynomial Submodule Zsqrtd
open scoped BigOperators

theorem even_finite_group_has_fixed_point_free_left_involution {G : Type*} [Group G] [Fintype G]
  (hG : Even (Fintype.card G)) :
  ∃ x : G, orderOf x = 2 ∧ ∃ f : G → G,
    f = (fun y => x * y) ∧ Function.Involutive f ∧ ∀ y : G, f y ≠ y := by
  obtain ⟨k, hk⟩ := hG
  have h_two_dvd : 2 ∣ Fintype.card G := by
    exact ⟨k, by simpa [two_mul] using hk⟩
  let _ : Fact (Nat.Prime 2) := ⟨Nat.prime_two⟩
  obtain ⟨x, hx⟩ := exists_prime_orderOf_dvd_card (G := G) (p := 2) h_two_dvd
  have hsq : x * x = 1 := by
    have hp := pow_orderOf_eq_one x
    rw [hx] at hp
    simpa [pow_two] using hp
  have hxne : x ≠ 1 := by
    intro h
    subst x
    simpa using hx
  refine ⟨x, hx, (fun y => x * y), rfl, ?_, ?_⟩
  · intro y
    change x * (x * y) = y
    rw [← mul_assoc, hsq, one_mul]
  · intro y h
    have h' : x * y = 1 * y := by
      simpa using h
    exact hxne (mul_right_cancel h')
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Artin_exercise_2_11_3__mh__40815eed

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint d2fbc07787022f25

4.154 154. exists_subgroup_card_two_of_card_dvd_two

The problem

Show that every finite group whose order is divisible by 2 contains a subgroup with exactly 2 elements.

Lean statement

```
theorem exists_subgroup_card_two_of_card_dvd_two {G : Type*} [Group G] [Fintype G]
  (hG : 2 ∣ Fintype.card G) : ∃ H : Subgroup G, Nat.card H = 2
```

Parent. Prove that a finite group whose number of elements is divisible by 2 contains an element of order 2.

Parent in Lean

```
theorem finite_group_order_two_of_card_dvd_two {G : Type*} [Group G] [Fintype G]
  (hG : 2 ∣ Fintype.card G) : ∃ x : G, orderOf x = 2
```

What it contributes. Supplies the divisible-by-two finite-group argument yielding an element of order two; that witness is then consumed to construct the subgroup target.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves genuinely new mathematics: it constructs the subgroup of powers generated by an element of order two and establishes that its cardinality is two. The parent only provides existence of the order-two element and does not supply this subgroup construction.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds genuine subgroup construction: it turns the order-two witness into the subgroup of integer powers and proves its cardinality via the cardinality theorem for cyclic subgroups. The parent's direct existence proof supplies the witness but does not supply this structural argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from finding an element of order two to constructing a subgroup of cardinality two. It must build the cyclic subgroup generated by the witness and prove its cardinality, which is substantive reasoning absent from the parent proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2308 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem exists_subgroup_card_two_of_card_dvd_two.extracted_1_1.{u_1} {G : Type u_1} [inst : Group G]
  [inst_1 : Fintype G] (hG : 2 ∣ card G) : ∃ H, Nat.card ↑H = 2
```

Read back by a model that saw only the Lean. *For a finite group G , if 2 divides the cardinality of G , then there exists a subgroup H of G whose cardinality is 2.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Function Fintype Subgroup Ideal Polynomial Submodule Zsqrtd
open scoped BigOperators

theorem exists_subgroup_card_two_of_card_dvd_two {G : Type*} [Group G] [Fintype G]
  (hG : 2 ∣ Fintype.card G) : ∃ H : Subgroup G, Nat.card H = 2 := by
  let _ : Fact (Nat.Prime 2) := ⟨Nat.prime_two⟩
  obtain ⟨x, hx⟩ := exists_prime_orderOf_dvd_card (G := G) (p := 2) hG
  let H : Subgroup G := Subgroup.zpowers x
  have hcard : Nat.card H = orderOf x := by
    dsimp [H]
    rw [Nat.card_eq_fintype_card, Fintype.card_zpowers]
  have hH : Nat.card H = 2 := by
    calc
      Nat.card H = orderOf x := hcard
      _ = 2 := hx
  exact ⟨H, hH⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Artin_exercise_2_11_3__ms.mh__7b9f6edc

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 7b9f6edc6d4eb13b

4.155 155. subgroup_contains_adjacent_power_words

The problem

Let a and b be elements of a group and let n be a nonnegative integer. A subgroup contains both a and b exactly when it contains both bab^n and bab^{n+1} .

Lean statement

```
theorem subgroup_contains_adjacent_power_words {G : Type*} [Group G]
  (a b : G) (n : ℕ) :
  ∀ H : Subgroup G,
    (a ∈ H ∧ b ∈ H) ↔
      (b * a * b ^ n ∈ H ∧ b * a * b ^ (n + 1) ∈ H)
```

Parent. Let a and b be elements of a group. Show that a subgroup contains both a and b exactly when it contains both bab^2 and bab^3 .

Parent in Lean

```
theorem Artin_exercise_6_8_1_containment {G : Type*} [Group G]
  (a b : G) :
  ∀ H : Subgroup G,
    (a ∈ H ∧ b ∈ H) ↔
      (b * a * b ^ 2 ∈ H ∧ b * a * b ^ 3 ∈ H)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the fixed powers 2 and 3 to arbitrary adjacent exponents n and $n+1$. Its reverse direction requires symbolic power identities and a new algebraic normalization argument, which the parent's fixed-exponent proof does not supply directly.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the two exponents and requires proving the general cancellation identity for adjacent powers, which is not supplied by the parent's fixed-exponent simplifications.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the two adjacent exponents, requiring a general power-cancellation identity for arbitrary n rather than the parent's fixed exponents 2 and 3. The added 'pow_succ' rewrite and algebraic normalization are substantive proof obligations, not decoration or recall.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2302 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem subgroup_contains_adjacent_power_words.extracted_1_1 : ∀ {G : Type u_1} [inst : Group G] (a b : G) (n : ℕ)
  (H : Subgroup G), a ∈ H ∧ b ∈ H ↔ b * a * b ^ n ∈ H ∧ b * a * b ^ (n + 1) ∈ H
```

Read back by a model that saw only the Lean. *For every group G , every $a, b \in G$, every natural number n , and every subgroup H of G , a belongs to H and b belongs to H if and only if both $b * a * b^n$ and $b * a * b^{n+1}$ belong to H .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Function Fintype Subgroup Ideal Polynomial Submodule Zsqrtd
open scoped BigOperators

theorem subgroup_contains_adjacent_power_words {G : Type*} [Group G]
  (a b : G) (n : ℕ) :
  ∀ H : Subgroup G,
    (a ∈ H ∧ b ∈ H) ↔
      (b * a * b ^ n ∈ H ∧ b * a * b ^ (n + 1) ∈ H) := by
  intro H
  constructor
  · rintro ⟨ha, hb⟩
    constructor
    · exact H.mul_mem (H.mul_mem hb ha) (H.pow_mem hb n)
    · exact H.mul_mem (H.mul_mem hb ha) (H.pow_mem hb (n + 1))
  · rintro ⟨h₁, h₂⟩
    have hb_prod :
      (b * a * b ^ n)⁻¹ * (b * a * b ^ (n + 1)) ∈ H :=
      H.mul_mem (H.inv_mem h₁) h₂
    have hb_eq :
      (b * a * b ^ n)⁻¹ * (b * a * b ^ (n + 1)) = b := by
      rw [pow_succ]
      group
    have hb : b ∈ H := by
      exact hb_eq ▸ hb_prod
    have ha_prod :
      (b⁻¹ * (b * a * b ^ n)) * (b ^ n)⁻¹ ∈ H :=
      H.mul_mem
      (H.mul_mem (H.inv_mem hb) h₁)
      (H.inv_mem (H.pow_mem hb n))
    have ha_eq :
      (b⁻¹ * (b * a * b ^ n)) * (b ^ n)⁻¹ = a := by
      group
    have ha : a ∈ H := by
      exact ha_eq ▸ ha_prod
    exact ⟨ha, hb⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Artin_exercise_6_8_1__ms.mh__108154db

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, fingerprint 108154db84973d4c

4.156 156. unit_power_characterization_in_subgroup

The problem

Let g be a unit of exact order p . If a subgroup contains bgb^k and bgb^{k+1} , show that it contains a unit u such that $u^m = 1$ exactly when p divides m .

Lean statement

```
theorem unit_power_characterization_in_subgroup
  (n p k : ℕ) (g b : (ZMod n)*)
  (hg : orderOf g = p) (H : Subgroup (ZMod n)*)
  (h1 : b * g * b ^ k ∈ H) (h2 : b * g * b ^ (k + 1) ∈ H) :
  ∃ u : (ZMod n)*, u ∈ H ∧ ∀ m : ℕ, u ^ m = 1 ↔ p ∣ m
```

Parent. Let p be the exact order of a unit g . If a subgroup contains bgb^k and bgb^{k+1} , show that it contains a cyclic subgroup of cardinality p .

Parent in Lean

```
theorem cyclic_subgroup_of_prime_order_in_subgroup
  (n p k : ℕ) (g b : (ZMod n)*)
  (hg : orderOf g = p) (H : Subgroup (ZMod n)*)
  (h1 : b * g * b ^ k ∈ H) (h2 : b * g * b ^ (k + 1) ∈ H) :
  ∃ K : Subgroup (ZMod n)*, K ≤ H ∧ Nat.card K = p
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from existence of a subgroup of cardinality p to a power-period characterization of an element in H . Although it reuses the parent's derivation that $g \in H$, it must additionally prove the new equivalence using the order-divisibility theorem; the parent's subgroup-cardinality argument does not supply this.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from constructing a subgroup of cardinality p to characterizing the exact powers of an element. After the shared extraction of $g \in H$, it requires the new order-divisibility argument via `orderOf_dvd_iff_pow_eq_one`; the parent's subgroup-cardinality proof does not supply this.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the existential conclusion from finding a subgroup of cardinality p to characterizing all powers of a specific unit by divisibility by p . After reusing the extraction of $g \in H$, it requires the new `orderOf_dvd_iff_pow_eq_one` argument, which the parent's subgroup-cardinality proof does not provide.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2323 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem unit_power_characterization_in_subgroup.extracted_1_1 (n p k : ℕ) (g b : (ZMod n)*) (hg : orderOf g = p)
  (H : Subgroup (ZMod n)*) (h₁ : b * g * b ^ k ∈ H) (h₂ : b * g * b ^ (k + 1) ∈ H) :
  ∃ u ∈ H, ∀ (m : ℕ), u ^ m = 1 ↔ p ∣ m
```

Read back by a model that saw only the Lean. For natural numbers n , p , and k , units g and b of $Z\text{Mod } n$, and a subgroup H of the units of $Z\text{Mod } n$, suppose that the order of g is p , that $b \cdot g \cdot b^k$ belongs to H , and that $b \cdot g \cdot b^{k+1}$ belongs to H . Then there exists an element u in H such that, for every natural number m , $u^m = 1$ if and only if p divides m .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem unit_power_characterization_in_subgroup
  (n p k : ℕ) (g b : (ZMod n)*)
  (hg : orderOf g = p) (H : Subgroup (ZMod n)*)
  (h₁ : b * g * b ^ k ∈ H) (h₂ : b * g * b ^ (k + 1) ∈ H) :
  ∃ u : (ZMod n)*, u ∈ H ∧ ∀ m : ℕ, u ^ m = 1 ↔ p ∣ m := by
  have hbprod :
    (b * g * b ^ k)⁻¹ * (b * g * b ^ (k + 1)) ∈ H :=
    H.mul_mem (H.inv_mem h₁) h₂
  have hb : b ∈ H := by
    convert hbprod using 1 <|> group
  have hbinv : b⁻¹ ∈ H := H.inv_mem hb
  have hbk : (b⁻¹) ^ k ∈ H := H.pow_mem hbinv k
  have hleft : b⁻¹ * (b * g * b ^ k) ∈ H := H.mul_mem hbinv h₁
  have hall : (b⁻¹ * (b * g * b ^ k)) * (b⁻¹) ^ k ∈ H :=
    H.mul_mem hleft hbk
  have hgH : g ∈ H := by
    convert hall using 1 <|> group
  have hpower : ∀ m : ℕ, g ^ m = 1 ↔ p ∣ m := by
    intro m
    rw [← orderOf_dvd_iff_pow_eq_one, hg]
  exact ⟨g, hgH, hpower⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 3 **Certificate.** reproducible

Problem id. Artin_exercise_6_8_1__x__Herstein_exercise_2_4_36__mh.xe.mh.mh__7056edf3

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 7056edf3a6009cab

4.157 157. cyclic_subgroup_of_prime_order_in_subgroup

The problem

Let p be the exact order of a unit g . If a subgroup contains bgb^k and bgb^{k+1} , show that it contains a cyclic subgroup of cardinality p .

Lean statement

```
theorem cyclic_subgroup_of_prime_order_in_subgroup
  (n p k : ℕ) (g b : (ZMod n)*)
  (hg : orderOf g = p) (H : Subgroup (ZMod n)*)
  (h₁ : b * g * b ^ k ∈ H) (h₂ : b * g * b ^ (k + 1) ∈ H) :
  ∃ K : Subgroup (ZMod n)*, K ≤ H ∧ Nat.card K = p
```

Parent. Let p be prime, and let g be a unit of exact order p . Show that any subgroup containing bgb^2 and bgb^3 also contains a unit of exact order p .

Parent in Lean

```
theorem prime_order_unit_forced_into_subgroup
  (n p : ℕ) (g b : (ZMod n)*)
  (hg : orderOf g = p) (H : Subgroup (ZMod n)*)
  (h₁ : b * g * b ^ 2 ∈ H) (h₂ : b * g * b ^ 3 ∈ H) :
  ∃ u : (ZMod n)*, u ∈ H ∧ orderOf u = p
```

What it contributes. Supplied the transformed-word quotient checkpoint that derives $b \in H$ and the subsequent cancellation argument placing g in H ; the child generalizes exponents 2,3 to $k,k+1$ and changes the conclusion to a cyclic subgroup.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the exponents from fixed values to an arbitrary k and additionally constructs the cyclic subgroup generated by g , proving its containment and cardinality. The parent's argument only derives $g \in H$ and does not supply these subgroup-cardinality steps.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The exponent is genuinely generalized from fixed powers to arbitrary k , and the conclusion changes from producing an element of order p to constructing the entire cyclic subgroup with cardinality p . The proof therefore requires new subgroup construction and cardinality reasoning beyond the parent's membership argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The variable exponent k genuinely generalizes the parent's cancellation argument, and the conclusion additionally requires constructing the cyclic subgroup generated by g and proving its cardinality is p . This is not recoverable by recalling the parent's existential witness proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing

Check	By	Result
corpus_dedup	hash	unique against 2306 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	MISMATCH: Statement B restricts the ambient setting to the abelian unit group $(\mathbb{Z}\text{Mod } n)^x$, whereas Statement A is stated for unspecified units and subgroups.; Statement A requires the contained subgroup of cardinality p to be cyclic, while Statement B only requires a subgroup K of cardinality p and does not state cyclicity.

Goal round-trip: what Lean elaborated

```
theorem cyclic_subgroup_of_prime_order_in_subgroup.extracted_1_1 (n p k : ℕ) (g b : (ZMod n)^x) (hg : orderOf g = p)
  (H : Subgroup (ZMod n)^x) (h1 : b * g * b ^ k ∈ H) (h2 : b * g * b ^ (k + 1) ∈ H) : ∃ K ≤ H, Nat.card K = p
```

Read back by a model that saw only the Lean. For natural numbers n , p , and k , and units g and b of $\mathbb{Z}\text{Mod } n$, suppose that the order of g is p . If H is a subgroup of the unit group $(\mathbb{Z}\text{Mod } n)^x$, and both $b g b^k$ and $b g b^{k+1}$ belong to H , then there exists a subgroup K of H whose cardinality is p .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem cyclic_subgroup_of_prime_order_in_subgroup
  (n p k : ℕ) (g b : (ZMod n)^x)
  (hg : orderOf g = p) (H : Subgroup (ZMod n)^x)
  (h1 : b * g * b ^ k ∈ H) (h2 : b * g * b ^ (k + 1) ∈ H) :
  ∃ K : Subgroup (ZMod n)^x, K ≤ H ∧ Nat.card K = p := by
  have hbprod :
    (b * g * b ^ k)^-1 * (b * g * b ^ (k + 1)) ∈ H :=
    H.mul_mem (H.inv_mem h1) h2
  have hb : b ∈ H := by
    convert hbprod using 1 <|> group
  have hbinv : b^-1 ∈ H := H.inv_mem hb
  have hbk : (b^-1) ^ k ∈ H := H.pow_mem hbinv k
  have hleft : b^-1 * (b * g * b ^ k) ∈ H := H.mul_mem hbinv h1
  have hall : (b^-1 * (b * g * b ^ k)) * (b^-1) ^ k ∈ H :=
    H.mul_mem hleft hbk
  have hgH : g ∈ H := by
    convert hall using 1 <|> group
  let K : Subgroup (ZMod n)^x := Subgroup.zpowers g
  have hK : K ≤ H := by
    dsimp [K]
    exact Subgroup.zpowers_le.mpr hgH
  have hcard : Nat.card K = p := by
    dsimp [K]
    rw [Nat.card_zpowers, hg]
  exact ⟨K, hK, hcard⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Artin_exercise_6_8_1__x__Herstein_exercise_2_4_36__mh.xe.mh__2a21cbb7

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 2a21cbb7e5acc33a

4.158 158. additive_inverse_characterization

The problem

In a vector space, show that $v + w = 0$ if and only if $w = -v$.

Lean statement

```
theorem additive_inverse_characterization {F V : Type*} [AddCommGroup V] [Field F]
  [Module F V] : ∀ v w : V, v + w = 0 ↔ w = -v
```

Parent. Prove that $-(-v) = v$ for every $v \in V$.

Parent in Lean

```
theorem Axler_exercise_1_3 {F V : Type*} [AddCommGroup V] [Field F]
  [Module F V] {v : V} : -(-v) = v := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from double negation to an iff characterizing additive inverses. Its proof requires cancellation and additive rearrangement in both directions, which the parent's 'simp' proof does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a double-negation identity into a two-way characterization of additive inverses. Its proof requires cancellation, rearrangement via 'abel', and both implication directions, so recalling the parent does not solve the new obligation.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a direct involution identity into a two-way characterization of additive inverses. Proving the forward implication requires cancellation and additive rearrangement via 'abel', so the parent's 'simp' proof does not supply the main argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propxt, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2776 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem additive_inverse_characterization.extracted_1_1 : ∀ {V : Type u_2} [inst : AddCommGroup V] (v w : V),
  v + w = 0 ↔ w = -v
```

Read back by a model that saw only the Lean. For every type V equipped with an additive commutative group structure

and all elements v, w of V , $v + w = 0$ if and only if $w = -v$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Complex Polynomial LinearMap FiniteDimensional Module Module.End
open scoped BigOperators

theorem additive_inverse_characterization {F V : Type*} [AddCommGroup V] [Field F]
  [Module F V] : ∀ v w : V, v + w = 0 ↔ w = -v := by
  intro v w
  have hdouble : -(-v) = v := by
    simp
  have hcancel : -v + v = 0 := by
    calc
      -v + v = -v + -(-v) := by rw [hdouble]
      _ = 0 := by simp
  constructor
  · intro h
    calc
      w = w + 0 := by simp
      _ = w + (-v + v) := by rw [hcancel]
      _ = (v + w) + (-v) := by abel
      _ = 0 + (-v) := by rw [h]
      _ = -v := by simp
  · intro h
    rw [h]
    simpa [add_comm] using hcancel
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Axler_exercise_1_3__mh__c3a4c52a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axiomspropept, Quot.sound, fingerprint c3a4c52a33e2ccfd

4.159 159. double_neg_smul

The problem

Let F be a field and V an F -vector space. Prove that $(-a)(-v) = av$ for every scalar $a \in F$ and vector $v \in V$.

Lean statement

```
theorem double_neg_smul {F V : Type*} [AddCommGroup V] [Field F] [Module F V]
  (a : F) (v : V) : (-a) • (-v) = a • v := by
```

Parent. Prove that $-(-v) = v$ for every $v \in V$.

Parent in Lean

```
theorem Axler_exercise_1_3 {F V : Type*} [AddCommGroup V] [Field F]
  [Module F V] {v : V} : -(-v) = v := by
```

What it contributes. Supplies the visible double-negation checkpoint, generalized from one vector to every vector in the module.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely extends double-negation to simultaneous negation of a scalar and vector. Proving it requires module-specific identities ('smul_neg' and 'neg_smul') in addition to the parent's 'neg_neg', so recalling the parent proof is insufficient.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes double negation to the interaction of negation with scalar multiplication. Its proof requires the new module identities 'smul_neg' and 'neg_smul', so recalling the parent's 'neg_neg' proof is insufficient.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes double negation to scalar multiplication and requires two new module identities, 'smul_neg' and 'neg_smul', before applying vector double negation. The parent's 'simp' proof does not supply this bilinearity reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2771 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem double_neg_smul.extracted_1_1.{u_2, u_1} {F : Type u_1} {V : Type u_2} [inst : AddCommGroup V]
  [inst_1 : Field F] [inst_2 : Module F V] (a : F) (v : V) : -a • -v = a • v
```

Read back by a model that saw only the Lean. *Let F be a field and V an additive commutative group equipped with an F -module structure. For any scalar $a \in F$ and vector $v \in V$, the scalar multiplication of $-v$ by $-a$ equals the scalar multiplication of v by a : $(-a) \bullet (-v) = a \bullet v$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Complex Polynomial LinearMap FiniteDimensional Module Module.End
open scoped BigOperators

theorem double_neg_smul {F V : Type*} [AddCommGroup V] [Field F] [Module F V]
  (a : F) (v : V) : (-a) • (-v) = a • v := by
  have hneg : ∀ x : V, -(-x) = x := by
    intro x
    exact neg_neg x
  have hsmul_neg : (-a) • (-v) = -((-a) • v) := by
    rw [smul_neg]
  have hnega_smul : (-a) • v = -(a • v) := by
    rw [nega_smul]
  calc
    (-a) • (-v) = -((-a) • v) := hsmul_neg
    _ = -(-(a • v)) := by rw [hnega_smul]
    _ = a • v := hnega (a • v)
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Axler_exercise_1_3__mh__c9459ce0

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axiomsproptext, Quot.sound, fingerprint c9459ce043e9c50c

4.160 160. fixed_point_intersection_is_greatest

The problem

Let $(u_i)_{i \in I}$ be a finite nonempty family of subspaces of V , and let $f : V \rightarrow V$ be linear and preserve every u_i . Prove that the subspace of vectors fixed by f inside their intersection is the greatest subspace contained in every u_i on which f acts as the identity.

Lean statement

```
theorem fixed_point_intersection_is_greatest
  {F V ι : Type*} [AddCommGroup V] [Field F] [Module F V]
  [Fintype ι] [Nonempty ι] (u : ι → Submodule F V)
  (f : V →[F] V)
  (hf : ∀ i : ι, ∀ x : V, x ∈ u i → f x ∈ u i) :
  ∃ U : Submodule F V,
    (∀ x : V, x ∈ U ↔ (∀ i : ι, x ∈ u i) ∧ f x = x) ∧
    (∀ x : V, x ∈ U → f x ∈ U) ∧
    ∀ W : Submodule F V,
      (∀ i : ι, W ≤ u i) →
      (∀ x : V, x ∈ W → f x = x) →
      W ≤ U
```

Parent. Let $(u_i)_{i \in I}$ be a finite nonempty family of subspaces of V , and let $f : V \rightarrow V$ be linear and preserve every u_i . Prove that their intersection is the greatest f -invariant subspace contained in every u_i .

Parent in Lean

```
theorem finite_nonempty_invariant_intersection_is_greatest
  {F V ι : Type*} [AddCommGroup V] [Field F] [Module F V]
  [Fintype ι] [Nonempty ι] (u : ι → Submodule F V)
  (f : V →[F] V)
  (hf : ∀ i : ι, ∀ x : V, x ∈ u i → f x ∈ u i) :
  ∃ U : Submodule F V,
    (∀ x : V, x ∈ U ↔ ∀ i : ι, x ∈ u i) ∧
    (∀ x : V, x ∈ U → f x ∈ U) ∧
    ∀ W : Submodule F V,
      (∀ i : ι, W ≤ u i) →
      (∀ x : V, x ∈ W → f x ∈ W) → W ≤ U
```

What it contributes. Supplied the finite intersection construction, its membership characterization, and the invariance checkpoint used internally to prove the fixed-point submodule is invariant.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the target from the invariant intersection to its fixed-point submodule, requiring construction with the kernel of $f - \text{id}$ and a new greatest-subspace argument under pointwise fixedness. The parent's intersection-and-invariance proof does not supply this reasoning.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely changes the characterized submodule by intersecting the invariant intersection with the fixed-point kernel of $f - \text{id}$. Proving the kernel equivalence and showing this smaller submodule is invariant requires reasoning absent from the parent, so recalling the parent is insufficient.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely characterizes the fixed-point submodule by intersecting the invariant intersection with the kernel of $f - \text{id}$. Proving its invariance and maximality requires fixed-point reasoning absent from the parent proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2245 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem fixed_point_intersection_is_greatest.extracted_1_1.{u_3, u_2, u_1} {F : Type u_1} {V : Type u_2} {ι : Type u_3}
[inst : AddCommGroup V] [inst_1 : Field F] [inst_2 : Module F V] [inst_3 : Nonempty ι] (u : ι → Submodule F V)
(f : V →ι[F] V) (hf : ∀ (i : ι), ∀ x ∈ u i, f x ∈ u i) :
∃ U,
  (∀ (x : V), x ∈ U ↔ (∀ (i : ι), x ∈ u i) ∧ f x = x) ∧
  (∀ x ∈ U, f x ∈ U) ∧ ∀ (W : Submodule F V), (∀ (i : ι), W ≤ u i) → (∀ x ∈ W, f x = x) → W ≤ U
```

Read back by a model that saw only the Lean. Let F be a field, let V be an additive commutative group and an F -module, and let ι be a nonempty type. Given a family $u : \iota \rightarrow \text{Submodule } F V$, a linear map $f : V \rightarrow_{\iota}[F] V$, and the hypothesis that for every $i : \iota$ and every $x \in u i$, one has $f x \in u i$, there exists a submodule U of V such that, for every $x \in V$, $x \in U$ if and only if x belongs to $u i$ for every $i : \iota$ and $f x = x$; moreover, for every $x \in U$, $f x \in U$; and for every submodule W of V , if $W \leq u i$ for every $i : \iota$ and $f x = x$ for every $x \in W$, then $W \leq U$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Complex Polynomial LinearMap FiniteDimensional Module Module.End
open scoped BigOperators

theorem fixed_point_intersection_is_greatest
  {F V ι : Type*} [AddCommGroup V] [Field F] [Module F V]
  [Fintype ι] [Nonempty ι] (u : ι → Submodule F V)
  (f : V →ι[F] V)
  (hf : ∀ i : ι, ∀ x : V, x ∈ u i → f x ∈ u i) :
  ∃ U : Submodule F V,
    (∀ x : V, x ∈ U ↔ (∀ i : ι, x ∈ u i) ∧ f x = x) ∧
    (∀ x : V, x ∈ U → f x ∈ U) ∧
    ∀ W : Submodule F V,
      (∀ i : ι, W ≤ u i) →
      (∀ x : V, x ∈ W → f x = x) →
      W ≤ U := by
  let I : Submodule F V := ⋂ i, u i
  let U : Submodule F V := I ∩ LinearMap.ker (f - (LinearMap.id : V →ι[F] V))
  have hI_mem : ∀ x : V, x ∈ I ↔ ∀ i : ι, x ∈ u i := by
    intro x
    change x ∈ (⋂ i, u i) ↔ ∀ i : ι, x ∈ u i
    exact Submodule.mem_iInf u
  have hI_inv : ∀ x : V, x ∈ I → f x ∈ I := by
    intro x hx
    apply Submodule.mem_iInf u |>.mpr
```

```

intro i
apply hf i x
exact (hI_mem x).mp hx i
have hker :  $\forall x : V, x \in \text{LinearMap.ker } (f - (\text{LinearMap.id} : V \rightarrow_{\mathbb{I}}[F] V)) \leftrightarrow f x = x := \text{by}$ 
  intro x
  constructor
  · intro hx
    change (f - (LinearMap.id :  $V \rightarrow_{\mathbb{I}}[F] V$ )) x = 0 at hx
    change f x - x = 0 at hx
    exact sub_eq_zero.mp hx
  · intro hx
    change (f - (LinearMap.id :  $V \rightarrow_{\mathbb{I}}[F] V$ )) x = 0
    change f x - x = 0
    exact sub_eq_zero.mpr hx
have hU_mem :  $\forall x : V, x \in U \leftrightarrow (\forall i : \mathbb{I}, x \in u i) \wedge f x = x := \text{by}$ 
  intro x
  constructor
  · intro hx
    change  $x \in I \wedge x \in \text{LinearMap.ker } (f - (\text{LinearMap.id} : V \rightarrow_{\mathbb{I}}[F] V))$  at hx
    exact ⟨(hI_mem x).mp hx.1, (hker x).mp hx.2⟩
  · intro hx
    change  $x \in I \wedge x \in \text{LinearMap.ker } (f - (\text{LinearMap.id} : V \rightarrow_{\mathbb{I}}[F] V))$ 
    exact ⟨(hI_mem x).mpr hx.1, (hker x).mpr hx.2⟩
have hU_inv :  $\forall x : V, x \in U \rightarrow f x \in U := \text{by}$ 
  intro x hx
  have hx' := (hU_mem x).mp hx
  apply (hU_mem (f x)).mpr
  constructor
  · exact (hI_mem (f x)).mp (hI_inv x ((hI_mem x).mpr hx'.1))
  · simpa [hx'.2]
have hgreatest :  $\forall W : \text{Submodule } F V,$ 
  ( $\forall i : \mathbb{I}, W \leq u i$ )  $\rightarrow$ 
  ( $\forall x : V, x \in W \rightarrow f x = x$ )  $\rightarrow$ 
   $W \leq U := \text{by}$ 
  intro W hW hfixed x hx
  apply (hU_mem x).mpr
  constructor
  · intro i
    exact hW i hx
  · exact hfixed x hx
exact ⟨U, hU_mem, hU_inv, hgreatest⟩

```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Axler_exercise_1_8__me.mh__3405d747

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 3405d747515afd56

4.161 161. intersection_is_unique_greatest_lower_bound

The problem

Prove that the intersection of any family of subspaces is the unique subspace that is contained in every member and contains every subspace with that property.

Lean statement

```
theorem intersection_is_unique_greatest_lower_bound {F V : Type*} [AddCommGroup V] [Field F] [Module F V] {ι : Type*} (u : ι → Submodule
  F V) :
  ∃! U : Submodule F V,
    (⋂ i : ι, (u i).carrier) = (U : Set V) ∧
    ∀ W : Submodule F V, W ≤ U ↔ ∀ i : ι, W ≤ u i := by
```

Parent. Prove that the intersection of any family of subspaces is the greatest subspace contained in every member of the family.

Parent in Lean

```
theorem intersection_is_greatest_lower_bound {F V : Type*} [AddCommGroup V] [Field F] [Module F V] {ι : Type*} (u : ι → Submodule F V) :
  ∃ U : Submodule F V,
    (⋂ i : ι, (u i).carrier) = (U : Set V) ∧
    ∀ W : Submodule F V, W ≤ U ↔ ∀ i : ι, W ≤ u i := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from existence to a genuine uniqueness statement. Although the parent supplies the candidate intersection submodule, the child requires a new extensionality argument proving any other submodule with the same carrier is equal to it; this is not contained in the parent proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes existence of a greatest lower bound into existence and uniqueness. It must additionally prove that any other candidate has the same carrier and hence is equal as a submodule, which is a genuine obligation absent from the parent.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes existence of a greatest lower bound into existence and uniqueness, requiring a new extensionality argument to show any qualifying submodule has the same carrier. The parent's construction proves existence and the GLB property but does not supply this uniqueness reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2940 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem intersection_is_unique_greatest_lower_bound.extracted_1_1.{u_3, u_2, u_1} {F : Type u_1} {V : Type u_2}
  [inst : AddCommGroup V] [inst_1 : Field F] [inst_2 : Module F V] {ι : Type u_3} (u : ι → Submodule F V) :
  ∃! U, ⋂ i, (u i).carrier = ↑U ∧ ∀ (W : Submodule F V), W ≤ U ↔ ∀ (i : ι), W ≤ u i
```

Read back by a model that saw only the Lean. Let F be a field, let V be an additive commutative group equipped with an F -module structure, let ι be a type, and let u be a family of F -submodules of V indexed by ι . Then there exists a unique F -submodule U of V such that the intersection of the underlying sets of the submodules $u(i)$, over all i , is the underlying set of U , and, for every F -submodule W of V , W is contained in U if and only if W is contained in $u(i)$ for every i .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Complex Polynomial LinearMap FiniteDimensional Module Module.End
open scoped BigOperators

theorem intersection_is_unique_greatest_lower_bound {F V : Type*} [AddCommGroup V] [Field F] [Module F V] {ι : Type*} (u : ι → Submodule
  F V) :
  ∃! U : Submodule F V,
    (⋂ i : ι, (u i).carrier) = (U : Set V) ∧
    ∀ W : Submodule F V, W ≤ U ↔ ∀ i : ι, W ≤ u i := by
  classical
  let U : Submodule F V := ⋂ i, u i
  have hCarrier : (⋂ i : ι, (u i).carrier) = (U : Set V) := by
    dsimp [U]
    exact (Submodule.coe_iInf u).symm
  have hGLB : ∀ W : Submodule F V, W ≤ U ↔ ∀ i : ι, W ≤ u i := by
    dsimp [U]
    intro W
    exact le_iInf_iff
  refine ⟨U, ⟨hCarrier, hGLB⟩, ?_⟩
  intro U' hU'
  have hset : (U : Set V) = (U' : Set V) := by
    rw [← hCarrier, hU'.1]
  apply Submodule.ext
  intro x
  exact (Set.ext_iff.mp hset x).symm
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Axler_exercise_1_8__me.mh__83e0bdc6

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 83e0bdc6510a2780

4.162 162. submodule_intersection_is_greatest

The problem

Prove that for any family of submodules of V , there exists a submodule U whose underlying set is the intersection of the family, and such that a submodule W is contained in every member of the family if and only if W is contained in U .

Lean statement

```
theorem submodule_intersection_is_greatest {F V : Type*} [AddCommGroup V] [Field F]
  [Module F V] {ι : Type*} (u : ι → Submodule F V) :
  ∃ U : Submodule F V,
    (⋂ i, (u i).carrier) = (U : Set V) ∧
    ∀ W : Submodule F V, (∀ i, W ≤ u i) ↔ W ≤ U
```

Parent. Prove that the intersection of any collection of subspaces of V is a subspace of V .

Parent in Lean

```
theorem Axler_exercise_1_8 {F V : Type*} [AddCommGroup V] [Field F]
  [Module F V] {ι : Type*} (u : ι → Submodule F V) :
  ∃ U : Submodule F V, (⋂ i : ι, (u i).carrier) = ↑U := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child strengthens the existential intersection statement into a greatest-submodule characterization, requiring proof of the full universal property via both order directions. The parent's extensional membership proof does not supply this lattice-theoretic uniqueness/greatest argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from mere representability of the set intersection to a greatest-submodule characterization. Its proof requires the additional lattice universal-property arguments 'le_iInf' and 'iInf_le', which are not supplied by the parent's extensionality proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes mere existence of an intersection submodule into a greatest-submodule characterization, requiring the infimum universal property via le_iInf and iInf_le. The parent's carrier equality proof does not supply this order-theoretic equivalence.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1931 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem submodule_intersection_is_greatest.extracted_1_1.{u_3, u_2, u_1} {F : Type u_1} {V : Type u_2}
  [inst : AddCommGroup V] [inst_1 : Field F] [inst_2 : Module F V] {ι : Type u_3} (u : ι → Submodule F V) :
  ∃ U, ⋂ i, (u i).carrier = ↑U ∧ ∀ (W : Submodule F V), (∀ (i : ι), W ≤ u i) ↔ W ≤ U
```

Read back by a model that saw only the Lean. For any family u assigning to each element i of an index type ι a submodule $u(i)$ of the F -module V , there exists a submodule U of V such that the intersection of the underlying sets of all $u(i)$ equals the underlying set of U , and for every submodule W of V , W is contained in every $u(i)$ if and only if W is contained in U .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Complex Polynomial LinearMap FiniteDimensional Module Module.End
open scoped BigOperators

theorem submodule_intersection_is_greatest {F V : Type*} [AddCommGroup V] [Field F]
  [Module F V] {ι : Type*} (u : ι → Submodule F V) :
  ∃ U : Submodule F V,
    (⋂ i, (u i).carrier) = (U : Set V) ∧
    ∀ W : Submodule F V, (∀ i, W ≤ u i) ↔ W ≤ U := by
  classical
  let U : Submodule F V := ⋂ i, u i
  refine ⟨U, ?_, ?_⟩
  · exact (Submodule.coe_iInf u).symm
  · intro W
    constructor
    · intro hW
      exact le_iInf hW
    · intro hW i
      exact le_trans hW (iInf_le u i)
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Axler_exercise_1_8__mh__7a088215

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 597069d000ab8471

4.163 163. genEigenspace_invariant_of_commute

The problem

Every generalized eigenspace of S associated with a scalar $c \in F$ and an order $k \in \mathbb{N} \cup \{\infty\}$ is invariant under T whenever S and T commute.

Lean statement

```
theorem genEigenspace_invariant_of_commute {F V : Type*} [AddCommGroup V] [Field F] [Module F V]
  {S T : Module.End F V} (h_comm : S * T = T * S) :
  ∀ c : F, ∀ k : ℕ∞, Set.MapsTo T (S.genEigenspace c k) (S.genEigenspace c k)
```

Parent. Every eigenspace of S is invariant under T when S and T commute.

Parent in Lean

```
theorem eigenspace_invariant_of_commute {F V : Type*} [AddCommGroup V] [Field F] [Module F V]
  {S T : Module.End F V} (h_comm : S * T = T * S) :
  ∀ c : F, Set.MapsTo T (Module.End.eigenspace S c) (Module.End.eigenspace S c)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes invariant eigenspaces to generalized eigenspaces for every ordinal-valued exponent k , which is a stronger mathematical claim than the parent. Its proof uses a dedicated generalized-eigenspace commutation theorem rather than reusing the parent's ordinary eigenspace argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes invariance from the ordinary eigenspace to every generalized eigenspace and arbitrary ordinal bound k . The parent's eigenvector-level calculation does not establish this broader claim; the child requires a distinct generalized-eigenspace commutation theorem.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens invariance from ordinary eigenspaces to generalized eigenspaces for every exponent k , requiring preservation of kernels of powers of $S - c$. The parent's pointwise eigenvector calculation does not supply this power-commutation argument, and the child uses a distinct generalized-eigenspace lemma.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1941 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem genEigenspace_invariant_of_commute.extracted_1_1 : ∀ {F : Type u_1} {V : Type u_2} [inst : AddCommGroup V]
  [inst_1 : Field F] [inst_2 : Module F V] {S T : End F V},
  S * T = T * S → ∀ (c : F) (k : ℕ), Set.MapsTo ↗T ↑((S.genEigenspace c) k) ↑((S.genEigenspace c) k)
```

Read back by a model that saw only the Lean. For any field F and any additive commutative group V equipped with an F -module structure, and for F -linear endomorphisms S and T of V , if S and T commute ($S \circ T = T \circ S$), then for every scalar $c \in F$ and every $k \in \mathbb{N}_\infty$, the endomorphism T maps the generalized eigenspace of S corresponding to c at level k into itself.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Complex Polynomial LinearMap FiniteDimensional Module Module.End
open scoped BigOperators

theorem genEigenspace_invariant_of_commute {F V : Type*} [AddCommGroup V] [Field F] [Module F V]
  {S T : Module.End F V} (h_comm : S * T = T * S) :
  ∀ c : F, ∀ k : ℕ, Set.MapsTo T (S.genEigenspace c k) (S.genEigenspace c k) := by
  intro c k
  have h_commute : Commute S T := h_comm
  have h_preserved : Set.MapsTo T (S.genEigenspace c k) (S.genEigenspace c k) :=
    Module.End.mapsTo_genEigenspace_of_comm h_commute c k
  exact h_preserved
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Axler_exercise_5_20__mh__80232ce9

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint ec92e5e9d1b2f4a8

4.164 164. normalClosure_eq_intersection_normal_containers

The problem

Let T be a subset of a group. Prove that the normal closure of T is the intersection of all normal subgroups containing T .

Lean statement

```
theorem normalClosure_eq_intersection_normal_containers {G : Type*} [Group G] (T : Set G) :
  Subgroup.normalClosure T =
    ⋂ (N : Subgroup G) (N.Normal) (T ⊆ N), N
```

Parent. Let T be a subset of a group and let K be a normal subgroup. Prove that T is contained in K if and only if every subset of the normal closure of T has its normal closure contained in K .

Parent in Lean

```
theorem subset_iff_forall_normalClosure_subset {G : Type*} [Group G] (T : Set G)
  (K : Subgroup G) (hKNormal : K.Normal) :
  T ⊆ K ↔
    ∀ S : Set G, S ⊆ (Subgroup.normalClosure T) →
      Subgroup.normalClosure S ≤ K
```

What it contributes. Used the parent's normal-closure universal property to establish containment in every normal subgroup containing T .

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child makes a substantive change of shape: it characterizes the normal closure as an intersection over all normal subgroups containing the set. Its proof requires lattice-theoretic intersection bounds and selecting the normal closure among the containers, which the parent's implication proof does not supply directly.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely different characterization: normal closure equals the intersection of all normal subgroups containing the set. Its proof requires lattice infimum bounds and minimality, which the parent's quantified-containment equivalence does not supply directly.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from an implication characterization for one normal subgroup to an equality identifying the normal closure with the intersection of all normal subgroups containing the set. Proving both lattice inequalities requires explicit infimum reasoning and is not obtained by recalling the parent's quantified subset argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2978 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normalClosure_eq_intersection_normal_containers.extracted_1_1.{u_1} {G : Type u_1} [inst : Group G]
  (T : Set G) : normalClosure T =  $\bigcap$  N,  $\bigcap$  ( _ : N.Normal),  $\bigcap$  ( _ : T  $\subseteq$   $\uparrow$ N), N
```

Read back by a model that saw only the Lean. *For any group G and any subset T of G , the normal closure of T is equal to the intersection of all normal subgroups N of G that contain T .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem normalClosure_eq_intersection_normal_containers {G : Type*} [Group G] (T : Set G) :
  Subgroup.normalClosure T =
     $\bigcap$  (N : Subgroup G) ( _ : N.Normal) ( _ : T  $\subseteq$  (N : Set G)), N := by
  let I : Subgroup G :=  $\bigcap$  (N : Subgroup G) ( _ : N.Normal) ( _ : T  $\subseteq$  (N : Set G)), N
  have h_intersection : I =  $\bigcap$  (N : Subgroup G) ( _ : N.Normal) ( _ : T  $\subseteq$  (N : Set G)), N := by
    rfl
  have h_left : Subgroup.normalClosure T  $\leq$  I := by
    rw [h_intersection]
    refine le_iInf (fun N => ?_)
    refine le_iInf (fun hN => ?_)
    refine le_iInf (fun hT => ?_)
    let I : N.Normal := hN
    exact Subgroup.normalClosure_le_normal hT
  have h_right : I  $\leq$  Subgroup.normalClosure T := by
    rw [h_intersection]
    refine iInf_le_of_le (Subgroup.normalClosure T) ?_
    refine iInf_le_of_le (by infer_instance) ?_
    exact iInf_le_of_le Subgroup.subset_normalClosure le_rfl
  exact le_antisymm h_left h_right
```

Operator. mutation / mutation_hard **Lineage depth.** 4 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.me.me.mh__101f67b5

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 101f67b57e008ece

4.165 165. normalClosure_le_iff_subset

The problem

Let S be a subset of a group and let M be a normal subgroup. Prove that the normal closure of S is contained in M if and only if S is contained in M .

Lean statement

```
theorem normalClosure_le_iff_subset {G : Type*} [Group G] (S : Set G)
  (M : Subgroup G) (hMnormal : M.Normal) :
  Subgroup.normalClosure S ≤ M ↔ S ⊆ (M : Set G)
```

Parent. Let S and T be subsets of a group, and let M be a normal subgroup containing the normal closure of T . If S is contained in the normal closure of T , prove that the normal closure of S is contained in M .

Parent in Lean

```
theorem normalClosure_le_of_subset_normalClosure {G : Type*} [Group G] (S T : Set G)
  (M : Subgroup G) (hMnormal : M.Normal)
  (hTM : (Subgroup.normalClosure T : Set G) ⊆ M)
  (hST : S ⊆ (Subgroup.normalClosure T : Set G)) :
  Subgroup.normalClosure S ≤ M
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the one-way containment result into an iff characterization. Its forward direction requires the new argument that generators lie in their normal closure, which the parent proof does not provide; the extra hypotheses involving T are also eliminated.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the one-way transport result into a full iff characterization of normal closure containment. Its forward direction requires the new inclusion of S into its normal closure, so recalling the parent's proof does not supply the argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a one-way containment result into an iff characterization. The reverse direction is new: it requires using that the generating set lies inside its normal closure, which the parent proof does not establish.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2969 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normalClosure_le_iff_subset.extracted_1_1.{u_1} {G : Type u_1} [inst : Group G] (S : Set G) (M : Subgroup G)
(hMnormal : M.Normal) : normalClosure S ≤ M ↔ S ⊆ ↑M
```

Read back by a model that saw only the Lean. For any group G , any subset S of G , and any normal subgroup M of G , the normal closure of S is contained in M if and only if S is contained in M .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem normalClosure_le_iff_subset {G : Type*} [Group G] (S : Set G)
(M : Subgroup G) (hMnormal : M.Normal) :
Subgroup.normalClosure S ≤ M ↔ S ⊆ (M : Set G) := by
letI : M.Normal := hMnormal
have hsubset : S ⊆ (Subgroup.normalClosure S : Set G) :=
Subgroup.subset_normalClosure
have hforward : Subgroup.normalClosure S ≤ M → S ⊆ (M : Set G) := by
intro hclosure x hx
exact hclosure (hsubset hx)
have hbackward : S ⊆ (M : Set G) → Subgroup.normalClosure S ≤ M := by
intro hSM
exact (Subgroup.normalClosure_subset_iff).mp hSM
constructor
· exact hforward
· exact hbackward
```

Operator. mutation / mutation_hard **Lineage depth.** 3 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.me.me.mh__46206500

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 46206500cd88548d

4.166 166. normal_subgroup_le_finset_inf_iff_le_normalCore_inf

The problem

Let G be a group and let Σ be a finite family of subgroups. For every normal subgroup N , prove that N is contained in every member of Σ if and only if N is contained in the normal core of their meet.

Lean statement

```
theorem normal_subgroup_le_finset_inf_iff_le_normalCore_inf {G : Type*} [Group G] (S : Finset (Subgroup G)) :
  ∀ N : Subgroup G, N.Normal →
    ((∀ H ∈ S, N ≤ H) ↔ N ≤ (S.inf (fun H => H)).normalCore)
```

Parent. Let G be a group and let S be a finite family of subgroups, possibly empty. Prove that the normal core of the meet of the subgroups equals the meet of their normal cores.

Parent in Lean

```
theorem normalCore_finset_inf_eq_inf_normalCore {G : Type*} [Group G] (S : Finset (Subgroup G)) :
  (S.inf (fun H => H)).normalCore = S.inf (fun H => H.normalCore)
```

What it contributes. Used the parent's finite-meet and normal-core maximality checkpoints, changing the equality target into a quantified iff.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the equality into a characterization theorem quantified over an arbitrary normal subgroup. Its proof uses normality to pass between containment in the finite meet and containment in its normal core, rather than reproducing the parent's finite-set induction or equality argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the equality into an iff characterizing all normal subgroups contained in every member of the finite family. Its proof uses normality of an arbitrary subgroup and the universal property of the finite infimum, without recalling or applying the parent's finset induction or equality.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes an equality of finite subgroup infima into an iff characterizing all normal subgroups contained in the family. Its proof requires the normal-subgroup-to-normal-core lemma and a two-direction containment argument, none of which is supplied by recalling the parent proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2330 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normal_subgroup_le_finset_inf_iff_le_normalCore_inf.extracted_1_1 : ∀ {G : Type u_1} [inst : Group G]
  (S : Finset (Subgroup G)) (N : Subgroup G), N.Normal → ((∀ H ∈ S, N ≤ H) ↔ N ≤ (S.inf fun H => H).normalCore)
```

Read back by a model that saw only the Lean. *For every group G , every finite set S of subgroups of G , and every subgroup N of G , if N is normal in G , then N is contained in every subgroup H belonging to S if and only if N is contained in the normal core of the infimum of the subgroups in S .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem normal_subgroup_le_finset_inf_iff_le_normalCore_inf {G : Type*} [Group G] (S : Finset (Subgroup G)) :
  ∀ N : Subgroup G, N.Normal →
    ((∀ H ∈ S, N ≤ H) ↔ N ≤ (S.inf (fun H => H)).normalCore) := by
  classical
  intro N hN
  let I : N.Normal := hN
  constructor
  · intro hAll
    have hMeet : N ≤ S.inf (fun H => H) := by
      refine Finset.le_inf ?_
      intro H hH
      exact hAll H hH
    have hCore : N ≤ (S.inf (fun H => H)).normalCore := by
      exact Subgroup.normal_le_normalCore.mpr hMeet
    exact hCore
  · intro hCore H hH
    have hMeet : N ≤ S.inf (fun H => H) := by
      exact hCore.trans (Subgroup.normalCore_le _)
    exact hMeet.trans (Finset.inf_le hH)
```

Operator. mutation / mutation_hard **Lineage depth.** 3 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.me.mh.mh__74053082

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint 740530822b717c0a

4.167 167. normalCore_finset_inf_eq_inf_normalCore

The problem

Let G be a group and let $\mathcal{S}$ be a finite family of subgroups, possibly empty. Prove that the normal core of the meet of the subgroups equals the meet of their normal cores.

Lean statement

```
theorem normalCore_finset_inf_eq_inf_normalCore {G : Type*} [Group G] (S : Finset (Subgroup G)) :
  (S.inf (fun H => H)).normalCore = S.inf (fun H => H.normalCore)
```

Parent. Let G be a group and let $(H_i)_{i \in I}$ be a nonempty indexed family of subgroups. Prove that the normal core of their intersection is the intersection of their normal cores.

Parent in Lean

```
theorem normalCore_iInf_eq_iInf_normalCore {G : Type*} [Group G] {I : Type*} [Nonempty I] (H : I → Subgroup G) :
  (⋂ i, H i).normalCore = ⋂ i, (H i).normalCore
```

What it contributes. Supplied the monotonicity, normality of intersections, and normal-core maximality checkpoints; these were adapted to a finite meet with an explicit empty-family case.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the result to finite subgroup meets, including the empty case. Its induction and empty-meet handling require reasoning absent from the parent's nonempty indexed-intersection proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the result to finite subgroup meets, including the empty meet. Its induction and separate empty-case reasoning are not supplied by the parent's nonempty indexed-inf proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely extends the result to finite subgroup meets that may be empty. Its proof requires an induction over the finset, a separate empty case, and new normality reasoning for the inserted finite infimum; the parent's indexed-infimum argument does not supply this.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2325 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normalCore_finset_inf_eq_inf_normalCore.extracted_1.1.{u_1} {G : Type u_1} [inst : Group G]
  (S : Finset (Subgroup G)) : (S.inf fun H => H).normalCore = S.inf fun H => H.normalCore
```

Read back by a model that saw only the Lean. *For any group G and any finite set S of subgroups of G , the normal core of the infimum of the subgroups in S is equal to the infimum of their normal cores.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem normalCore_finset_inf_eq_inf_normalCore {G : Type*} [Group G] (S : Finset (Subgroup G)) :
  (S.inf (fun H => H)).normalCore = S.inf (fun H => H.normalCore) := by
  classical
  induction S using Finset.induction_on with
  | empty =>
    simp
  | @insert H S hS ih =>
    rw [Finset.inf_insert, Finset.inf_insert]
    apply le_antisymm
    · refine le_inf ?_ ?_
      · exact Subgroup.normalCore_mono inf_le_left
      · exact le_trans (Subgroup.normalCore_mono inf_le_right) ih.le
    · have hS_normal : (S.inf (fun K => K.normalCore)).Normal := by
        rw [Finset.inf_eq_iInf]
        exact Subgroup.normal_iInf_normal (fun K =>
          Subgroup.normal_iInf_normal (fun hK : K ∈ S => K.normalCore_normal))
      let I : (H.normalCore n S.inf (fun K => K.normalCore)).Normal :=
        Subgroup.normal_inf_normal _ _
      apply Subgroup.normal_le_normalCore.mpr
      refine le_inf ?_ ?_
      · exact inf_le_left.trans (Subgroup.normalCore_le H)
      · refine le_trans inf_le_right ?_
        refine Finset.le_inf ?_
        intro K hK
        exact le_trans (Finset.inf_le hK) (Subgroup.normalCore_le K)
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.me.mh__6cd94798

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 6cd947984fdfebd

4.168 168. normal_iInf_is_greatest

The problem

Show that the intersection of any indexed family of normal subgroups is normal and is the greatest subgroup contained in every member of the family, even when the family is empty.

Lean statement

```
theorem normal_iInf_is_greatest {G : Type*} [Group G] (I : Type*) (H : I → Subgroup G) (hH : ∀ i : I, (H i).Normal) : (⋂ i, H i).Normal ∧
  ↪ (∀ i : I, (⋂ j, H j) ≤ H i) ∧ ∀ K : Subgroup G, (∀ i : I, K ≤ H i) → K ≤ (⋂ i, H i)
```

Parent. Prove that the intersection of any indexed family of normal subgroups is closed under conjugation, even when the index set is empty.

Parent in Lean

```
theorem indexed_inf_conjugation_closed {G : Type*} [Group G] (I : Type*) (H : I → Subgroup G) (hH : ∀ i : I, (H i).Normal) : ∀ n, n ∈ (⋂
  ↪ i, H i) → ∀ g : G, g * n * g-1 ∈ (⋂ i, H i)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child strengthens the conclusion from conjugation-closure to a full characterization of the infimum: normality, containment in each family member, and its greatest-subgroup property. The latter two obligations require lattice reasoning ('iInf_le' and 'le_iInf') absent from the parent proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion to a substantive greatest-lower-bound characterization, requiring the new lattice arguments 'iInf_le' and 'le_iInf' in addition to normality. The parent's conjugation-closure proof does not supply these universal containment properties.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine greatest-lower-bound characterization of the indexed infimum: it must prove both containment into every $H i$ and that every subgroup below all $H i$ lies below the infimum. The parent's conjugation argument supplies only normality and cannot establish these lattice obligations.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2963 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normal_iInf_is_greatest.extracted_1.1.{u_2, u_1} {G : Type u_1} [inst : Group G] (I : Type u_2)
  (H : I → Subgroup G) (hH : ∀ (i : I), (H i).Normal) :
  (⋂ i, H i).Normal ∧ (∀ (i : I), ⋂ j, H j ≤ H i) ∧ ∀ (K : Subgroup G), (∀ (i : I), K ≤ H i) → K ≤ ⋂ i, H i
```

Read back by a model that saw only the Lean. *Let G be a group, let I be a type, and let H assign to each element i of I a normal subgroup $H(i)$ of G . Then the infimum $\bigcap_i H(i)$ is normal; for every $i \in I$, $\bigcap_j H(j)$ is contained in $H(i)$; and for every subgroup K of G , if K is contained in $H(i)$ for every $i \in I$, then K is contained in $\bigcap_i H(i)$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem normal_iInf_is_greatest {G : Type*} [Group G] (I : Type*) (H : I → Subgroup G) (hH : ∀ i : I, (H i).Normal) : (⋂ i, H i).Normal ∧
  (∀ i : I, (⋂ j, H j) ≤ H i) ∧ ∀ K : Subgroup G, (∀ i : I, K ≤ H i) → K ≤ ⋂ i, H i := by
  have hNormal : (⋂ i, H i).Normal := Subgroup.normal_iInf_normal hH
  have hContained : ∀ i : I, (⋂ j, H j) ≤ H i := by
    intro i
    exact iInf_le H i
  have hGreatest : ∀ K : Subgroup G, (∀ i : I, K ≤ H i) → K ≤ ⋂ i, H i := by
    intro K hK
    exact le_iInf hK
  exact ⟨hNormal, hContained, hGreatest⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 3 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3.1_22b__me.me.ms.mh__c7c9901b

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c7c9901bbb7e6cba

4.169 169. normalCore_iInf_finet

The problem

Let H_i be a finite nonempty family of subgroups of a group G . Prove that the normal core of their intersection equals the intersection of their normal cores.

Lean statement

```
theorem normalCore_iInf_finet {G 1 : Type*} [Group G] (s : Finset 1) (H : 1 → Subgroup G) :
  (⋂ i ∈ s, (H i).normalCore) = (⋂ i ∈ s, H i).normalCore
```

Parent. Show that the normal core of a subgroup H is the supremum of all normal subgroups of G contained in H .

Parent in Lean

```
theorem normalCore_eq_iSup_child {G : Type*} [Group G] (H : Subgroup G) :
  H.normalCore = ⋃ (N : Subgroup G) (N : Normal N) (N : N ≤ H), N
```

What it contributes. Supplied the normal-core facts that each core is normal and contained in its subgroup, together with the universal property `normal_le_normalCore` used for both inclusions.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a genuinely different lattice identity: normal core commutes with an indexed infimum, requiring normality and universal-property reasoning in the opposite direction from the parent's supremum characterization. It is not recoverable by recalling or directly applying the parent's proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuine family-level lattice identity: normality of an infimum of normal cores and both universal-property inequalities are required. The parent's single-subgroup supremum characterization does not supply this finite-infimum argument by recall.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely different lattice identity: normal core distributes over a finite indexed intersection. Its proof requires establishing normality of the intersection and both lattice inequalities, whereas the parent only characterizes one normal core as a supremum and does not supply this argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2156 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normalCore_iInf_finset.extracted_1_1.{u_2, u_1} {G : Type u_1} {ι : Type u_2} [inst : Group G] (s : Finset ι)
  (hs : s.Nonempty) (H : ι → Subgroup G) : ⋂ i ∈ s, (H i).normalCore = (⋂ i ∈ s, H i).normalCore
```

Read back by a model that saw only the Lean. *Let G be a group, let ι be a type, let s be a nonempty finite subset of ι , and let H assign to each $i \in \iota$ a subgroup $H(i)$ of G . Then the infimum, over all i in s , of the normal cores of $H(i)$ equals the normal core of the infimum, over all i in s , of $H(i)$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem normalCore_iInf_finset {G ι : Type*} [Group G] (s : Finset ι) (H : ι → Subgroup G) :
  (⋂ i ∈ s, (H i).normalCore) = (⋂ i ∈ s, H i).normalCore := by
  have hleft_normal : Normal (⋂ i ∈ s, (H i).normalCore) := by
    constructor
    intro n hn g
    rw [Subgroup.mem_iInf]
    intro i
    rw [Subgroup.mem_iInf]
    intro hi
    apply (H i).normalCore_normal.conj_mem n
    rw [Subgroup.mem_iInf] at hn
    exact (Subgroup.mem_iInf.mp (hn i)) hi
  have hright_normal : Normal (⋂ i ∈ s, H i).normalCore := by
    exact Subgroup.normalCore_normal _
  have hleft_le_right_arg : (⋂ i ∈ s, (H i).normalCore) ≤ (⋂ i ∈ s, H i) := by
    refine le_iInf (fun i => ?_)
    refine le_iInf (fun hi => ?_)
    exact le_trans (iInf_le_of_le i (iInf_le_of_le hi le_refl)) (H i).normalCore_le
  have hright_le_left : (⋂ i ∈ s, H i).normalCore ≤ (⋂ i ∈ s, (H i).normalCore) := by
    refine le_iInf (fun i => ?_)
    refine le_iInf (fun hi => ?_)
    have harg_le_hi : (⋂ i ∈ s, H i) ≤ H i := by
      exact iInf_le_of_le i (iInf_le_of_le hi le_refl)
    letI : Normal (⋂ i ∈ s, H i).normalCore := hright_normal
    exact (Subgroup.normal_le_normalCore).mpr ((Subgroup.normalCore_le _).trans harg_le_hi)
  have hleft_le_right : (⋂ i ∈ s, (H i).normalCore) ≤ (⋂ i ∈ s, H i).normalCore := by
    letI : Normal (⋂ i ∈ s, (H i).normalCore) := hleft_normal
    exact (Subgroup.normal_le_normalCore).mpr hleft_le_right_arg
  exact le_antisymm hleft_le_right hright_le_left
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.mh.mh__e633a98d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 5fa322b68d8c112a

4.170 170. existsUnique_normal_lower_bound

The problem

Let S be a collection of normal subgroups of a group G . Prove that their intersection is the unique greatest normal subgroup contained in every member of S .

Lean statement

```
theorem existsUnique_normal_lower_bound {G : Type*} [Group G] (S : Set (Subgroup G)) (hS : ∀ N ∈ S, N.Normal) : ∃! H : Subgroup G,
  ↪ H.Normal ∧ (∀ N ∈ S, H ≤ N) ∧ (∀ K : Subgroup G, K.Normal → (∀ N ∈ S, K ≤ N) → K ≤ H)
```

Parent. Prove that the intersection of any collection of normal subgroups of a group is a normal subgroup.

Parent in Lean

```
theorem collection_inf_normal {G : Type*} [Group G] (S : Set (Subgroup G)) (hS : ∀ N ∈ S, (N : Subgroup G).Normal) : (sInf S : Subgroup
  ↪ G).Normal
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the obligation from proving normality to proving a uniqueness and universal characterization of the greatest normal lower bound. Its proof requires new order-theoretic arguments using both ‘sInf_le’ and ‘le_sInf’, which the parent proof does not provide.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes normality of the infimum into an exists-unique characterization of the greatest normal lower bound. Beyond the parent’s normality argument, it requires proving the infimum’s lower-bound property and uniqueness via the lattice universal property.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion into an existence-and-uniqueness characterization of the greatest normal lower bound. Beyond the parent’s normality proof, it requires proving the lower-bound property and antisymmetric uniqueness via lattice infimum lemmas.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2826 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem existsUnique_normal_lower_bound.extracted_1.1.{u_1} {G : Type u_1} [inst : Group G] (S : Set (Subgroup G))
  (hS : ∀ N ∈ S, N.Normal) :
```

$$\exists! H, H.\text{Normal} \wedge (\forall N \in S, H \leq N) \wedge \forall (K : \text{Subgroup } G), K.\text{Normal} \rightarrow (\forall N \in S, K \leq N) \rightarrow K \leq H$$

Read back by a model that saw only the Lean. For a group G and a set S of subgroups of G such that every subgroup N in S is normal in G , there exists a unique subgroup H of G such that H is normal in G , H is contained in every subgroup N in S , and for every normal subgroup K of G that is contained in every subgroup N in S , K is contained in H .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem existsUnique_normal_lower_bound {G : Type*} [Group G] (S : Set (Subgroup G))
  (hS : ∀ N ∈ S, N.Normal) :
  ∃! H : Subgroup G,
    H.Normal ∧ (∀ N ∈ S, H ≤ N) ∧
    (∀ K : Subgroup G, K.Normal → (∀ N ∈ S, K ≤ N) → K ≤ H) := by
  have hNormal : (sInf S : Subgroup G).Normal := by
    rw [sInf_eq_iInf]
    apply Subgroup.normal_iInf_normal
    intro N
    apply Subgroup.normal_iInf_normal
    intro hN
    exact hS N hN
  have hLower : ∀ N ∈ S, (sInf S : Subgroup G) ≤ N := by
    intro N hN
    exact sInf_le hN
  have hGreatest : ∀ K : Subgroup G, K.Normal → (∀ N ∈ S, K ≤ N) → K ≤ sInf S := by
    intro K hK hKL
    exact le_sInf hKL
  refine ⟨sInf S, ?_, ?_⟩
  · exact ⟨hNormal, hLower, hGreatest⟩
  · intro K hK
    apply le_antisymm
    · exact hGreatest K hK.1 hK.2.1
    · exact hK.2.2 (sInf S) hNormal hLower
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.mh__7fdfabde

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 7fdfabde60505d2a

4.171 171. normalClosure_normal_and_minimal

The problem

For a subset s of a group, prove that its normal closure is normal and is contained in every normal subgroup containing s .

Lean statement

```
theorem normalClosure_normal_and_minimal {G : Type*} [Group G] (s : Set G) :
  (Subgroup.normalClosure s).Normal ∧
  ∀ N : Subgroup G, N.Normal → s ⊆ N → Subgroup.normalClosure s ≤ N
```

Parent. Prove that the intersection of any collection of normal subgroups of a group is a normal subgroup.

Parent in Lean

```
theorem collection_inf_normal {G : Type*} [Group G] (S : Set (Subgroup G)) (hS : ∀ N ∈ S, (N : Subgroup G).Normal) : (sInf S : Subgroup
  G).Normal
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from normality of an infimum to a conjunction asserting both normality and the universal minimality property of the normal closure. Proving the containment for every normal subgroup containing the generating set requires a new infimum-order argument not supplied by the parent's proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the obligation from normality of an infimum to a normality-and-minimality characterization of the normal closure. Its second conjunct requires proving the universal lower-bound property, which is not supplied by the parent's proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the obligation from proving normality of an infimum to establishing the universal property of the normal closure: both normality and minimality among normal subgroups containing the generators. The parent's normality argument does not supply the new minimality proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2815 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normalClosure_normal_and_minimal.extracted_1_1.{u_1} {G : Type u_1} [inst : Group G] (s : Set G) :  
  (normalClosure s).Normal  $\wedge$   $\forall$  (N : Subgroup G), N.Normal  $\rightarrow$   $s \subseteq N \rightarrow$  normalClosure s  $\leq$  N
```

Read back by a model that saw only the Lean. *Let G be a group and let s be a subset of G . Then the normal closure of s in G is a normal subgroup of G , and for every normal subgroup N of G such that s is contained in N , the normal closure of s is contained in N .*

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open Fintype Subgroup Set Polynomial Ideal  
open scoped BigOperators  
  
theorem normalClosure_normal_and_minimal {G : Type*} [Group G] (s : Set G) :  
  (Subgroup.normalClosure s).Normal  $\wedge$   
     $\forall$  N : Subgroup G, N.Normal  $\rightarrow$   $s \subseteq N \rightarrow$  Subgroup.normalClosure s  $\leq$  N := by  
  rw [Subgroup.normalClosure_eq_iInf]  
  constructor  
  · apply Subgroup.normal_iInf_normal  
    intro N  
    apply Subgroup.normal_iInf_normal  
    intro hN  
    apply Subgroup.normal_iInf_normal  
    intro hs  
    exact hN  
  · intro N hN hs  
    exact iInf_le_of_le N (iInf_le_of_le hN (iInf_le_of_le hs le_refl))
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.mh__aada0d88

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint aada0d8800a170db

4.172 172. normalCore_iInf_fintype

The problem

For a finite nonempty family of subgroups of a group, the normal core of their intersection equals the intersection of their normal cores.

Lean statement

```
theorem normalCore_iInf_fintype {G : Type*} [Group G] {I : Type*} [Fintype I] [Nonempty I]
  (H : I → Subgroup G) :
  (⋂ i, H i).normalCore = ⋂ i, (H i).normalCore
```

Parent. Prove that the intersection of any collection of normal subgroups of a group is a normal subgroup.

Parent in Lean

```
theorem collection_inf_normal {G : Type*} [Group G] (S : Set (Subgroup G)) (hS : ∀ N ∈ S, (N : Subgroup G).Normal) : (sInf S : Subgroup
  ≤ G).Normal
```

What it contributes. Supplies the normality-of-an-infimum checkpoint, used for the finite meet of the family's normal cores.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a new lattice identity: normalCore distributes over an indexed infimum. Its proof requires both inequalities via the normal-core universal property and establishes normality of the infimum, none of which follows by recalling the parent's normality result.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a substantive normal-core lattice identity, requiring two lattice inequalities and normality of the infimum of normal cores; the parent only establishes normality of an arbitrary set infimum. The finite and nonempty assumptions are redundant, but the mathematical conclusion is genuinely different and cannot be obtained by recalling the parent proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely different lattice identity: normal core commutes with a finite indexed infimum. Its proof uses the universal property of normalCore and the fact that an infimum of normal subgroups is normal, which the parent's normality theorem alone does not provide.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2949 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normalCore_iInf_fintype.extracted_1_1.{u_2, u_1} {G : Type u_1} [inst : Group G] {I : Type u_2}
  [inst_1 : Nonempty I] (H : I → Subgroup G) : (⋂ i, H i).normalCore = ⋂ i, (H i).normalCore
```

Read back by a model that saw only the Lean. *Let G be a group, let I be a nonempty type, and let H be a family of subgroups of G indexed by I . Then the normal core of the infimum of the subgroups $H(i)$ is equal to the infimum, over all i in I , of the normal cores of $H(i)$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem normalCore_iInf_fintype {G : Type*} [Group G] {I : Type*} [Fintype I] [Nonempty I]
  (H : I → Subgroup G) :
  (⋂ i, H i).normalCore = ⋂ i, (H i).normalCore := by
  have hFiniteMeetNormal : (⋂ i, (H i).normalCore).Normal := by
    apply Subgroup.normal_iInf_normal
    intro i
    exact (H i).normalCore_normal
  apply le_antisymm
  · refine le_iInf fun i => ?_
    apply Subgroup.normal_le_normalCore.mpr
    exact (Subgroup.normalCore_le (⋂ i, H i)).trans (iInf_le H i)
  · apply Subgroup.normal_le_normalCore.mpr
    refine le_iInf fun i => ?_
    exact (iInf_le (fun j => (H j).normalCore) i).trans (Subgroup.normalCore_le (H i))
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.mh__b696cc8e

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint b696cc8eef7d0af0

4.173 173. normalCore_eq_iSup_child

The problem

Show that the normal core of a subgroup H is the supremum of all normal subgroups of G contained in H .

Lean statement

```
theorem normalCore_eq_iSup_child {G : Type*} [Group G] (H : Subgroup G) :
  H.normalCore = ⋃ (N : Subgroup G) (N : Normal N) (N : N ≤ H), N
```

Parent. For a subgroup H of a group G , prove that its normal core is normal, is contained in H , and contains every normal subgroup of G contained in H .

Parent in Lean

```
theorem normal_core_characterization {G : Type*} [Group G] (H : Subgroup G) :
  Normal H.normalCore ∧ H.normalCore ≤ H ∧
  ∀ N : Subgroup G, Normal N → N ≤ H → N ≤ H.normalCore
```

What it contributes. Supplies the normality and containment checkpoints for $H.normalCore$, which establish the lower bound in the supremum equality.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the characterization into an equality with an indexed supremum, requiring new lattice reasoning in both directions and an explicit supremum construction. The parent's normality, containment, and maximality facts are checkpoints, but its proof alone does not supply the equality.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the characterization from a universal maximality conjunction into an explicit supremum equality. Its proof requires new complete-lattice reasoning in both directions, including constructing the core as a supremum member and bounding every indexed subgroup by the core.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion into an $iSup$ characterization of the normal core. Proving the reverse inclusion requires new supremum-order reasoning, while the forward inclusion uses the parent's normality and containment facts rather than merely replaying its proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2151 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normalCore_eq_iSup_child.extracted_1_1.{u_1} {G : Type u_1} [inst : Group G] (H : Subgroup G) :  
  H.normalCore = ⋃ N, ⋃ (_ : N.Normal), ⋃ (_ : N ≤ H), N
```

Read back by a model that saw only the Lean. For any group G and subgroup H of G , the normal core of H is equal to the supremum of all subgroups N of G such that N is normal in G and N is contained in H .

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open Fintype Subgroup Set Polynomial Ideal  
open scoped BigOperators  
  
theorem normalCore_eq_iSup_child {G : Type*} [Group G] (H : Subgroup G) :  
  H.normalCore = ⋃ (N : Subgroup G) (_ : Normal N) (_ : N ≤ H), N := by  
  have hcore_normal : Normal H.normalCore := by  
    exact H.normalCore_normal  
  have hcore_contained : H.normalCore ≤ H := by  
    exact H.normalCore_le  
  have hcore_le_sup : H.normalCore ≤ ⋃ (N : Subgroup G) (_ : Normal N) (_ : N ≤ H), N := by  
    exact le_iSup_of_le H.normalCore  
      (le_iSup_of_le hcore_normal  
        (le_iSup_of_le hcore_contained le_refl))  
  have hsup_le_core : (⋃ (N : Subgroup G) (_ : Normal N) (_ : N ≤ H), N) ≤ H.normalCore := by  
    refine iSup_le ?_  
    intro N  
    refine iSup_le ?_  
    intro hN  
    refine iSup_le ?_  
    intro hNH  
    letI : Normal N := hN  
    exact Subgroup.normal_le_normalCore.mpr hNH  
  exact le_antisymm hcore_le_sup hsup_le_core
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.mh__cc302730

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint cc30273078e0f798

4.174 174. normal_core_mono_and_contains

The problem

Let $H \leq K$ be subgroups of a group G . Prove that the normal core of H is contained in the normal core of K , and that every normal subgroup contained in H is contained in the normal core of K .

Lean statement

```
theorem normal_core_mono_and_contains {G : Type*} [Group G] (H K : Subgroup G) (hHK : H ≤ K) :
  H.normalCore ≤ K.normalCore ∧
  ∀ N : Subgroup G, Normal N → N ≤ H → N ≤ K.normalCore
```

Parent. For a subgroup H of a group G , prove that its normal core is normal, is contained in H , and contains every normal subgroup of G contained in H .

Parent in Lean

```
theorem normal_core_characterization {G : Type*} [Group G] (H : Subgroup G) :
  Normal H.normalCore ∧ H.normalCore ≤ H ∧
  ∀ N : Subgroup G, Normal N → N ≤ H → N ≤ H.normalCore
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child adds the substantive monotonicity theorem $H.\text{normalCore} \leq K.\text{normalCore}$, which requires a new argument absent from the parent proof. The containment conclusion then combines the parent's normal-core characterization with this new monotonicity result.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds the genuinely new monotonicity claim $H.\text{normalCore} \leq K.\text{normalCore}$, which is not supplied by the parent's characterization proof. The containment claim then uses this new relation, so recalling the parent does not close the whole theorem.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves the genuinely new monotonicity law $H.\text{normalCore} \leq K.\text{normalCore}$ from $H \leq K$, requiring the separate `normalCore_mono` argument. The containment conjunct then transports normal subgroups through this new inclusion, so the parent's characterization proof alone does not close the child.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2127 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normal_core_mono_and_contains.extracted_1_1.{u_1} {G : Type u_1} [inst : Group G] (H K : Subgroup G)
  (hHK : H ≤ K) : H.normalCore ≤ K.normalCore ∧ ∀ (N : Subgroup G), N.Normal → N ≤ H → N ≤ K.normalCore
```

Read back by a model that saw only the Lean. *Let G be a group, and let H and K be subgroups of G . If H is contained in K , then the normal core of H is contained in the normal core of K , and every normal subgroup N of G that is contained in H is contained in the normal core of K .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem normal_core_mono_and_contains {G : Type*} [Group G] (H K : Subgroup G) (hHK : H ≤ K) :
  H.normalCore ≤ K.normalCore ∧
  ∀ N : Subgroup G, Normal N → N ≤ H → N ≤ K.normalCore := by
  have hmono : H.normalCore ≤ K.normalCore := by
    exact Subgroup.normalCore_mono hHK
  have hcontains : ∀ N : Subgroup G, Normal N → N ≤ H → N ≤ K.normalCore := by
    intro N hN hNH
    letI : Normal N := hN
    have hNcore : N ≤ H.normalCore := by
      exact Subgroup.normal_le_normalCore.mpr hNH
    exact hNcore.trans hmono
  exact ⟨hmono, hcontains⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.mh__d953fd9f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propept, Quot.sound, fingerprint d953fd9fcb379eb3

4.175 175. normalCore_closure_properties

The problem

Show that taking the normal core of a subgroup is normal, monotone under inclusion, and idempotent.

Lean statement

```
theorem normalCore_closure_properties {G : Type*} [Group G] :
  ∀ H K : Subgroup G, H ≤ K →
    H.normalCore.Normal ∧ H.normalCore ≤ K.normalCore ∧
    H.normalCore.normalCore = H.normalCore
```

Parent. For a subgroup H of a group G , the normal core of H is normal. Moreover, every normal subgroup N is contained in the normal core exactly when it is contained in H .

Parent in Lean

```
theorem normalCore_greatest_normal_subgroup_reversed {G : Type*} [Group G] (H : Subgroup G) :
  H.normalCore.Normal ∧
  ∀ N : Subgroup G, N.Normal → (N ≤ H.normalCore ↔ N ≤ H)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child proves genuinely different closure properties: monotonicity of the normal core and idempotence of applying the normal core, in addition to normality. These claims are not supplied by the parent's greatest-normal-subgroup equivalence or its closing argument.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child makes a substantive semantic change: it requires monotonicity of normal cores and idempotence of the normal-core operation in addition to normality. The parent's greatest-normal-subgroup equivalence does not directly supply these claims, and the child proof uses distinct library results.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion to monotonicity and idempotence of the normal core, neither of which follows from the parent's greatest-subgroup equivalence proof. Its hypothesis is substantive, and the proof requires distinct normal-core lemmas.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2344 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normalCore_closure_properties.extracted_1_1 : ∀ {G : Type u_1} [inst : Group G] (H K : Subgroup G),  
  H ≤ K → H.normalCore.Normal ∧ H.normalCore ≤ K.normalCore ∧ H.normalCore.normalCore = H.normalCore
```

Read back by a model that saw only the Lean. *For every group G and subgroups H and K of G , if H is contained in K , then the normal core of H is a normal subgroup of G , the normal core of H is contained in the normal core of K , and the normal core of the normal core of H equals the normal core of H .*

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open Fintype Subgroup Set Polynomial Ideal  
open scoped BigOperators  
  
theorem normalCore_closure_properties {G : Type*} [Group G] :  
  ∀ H K : Subgroup G, H ≤ K →  
    H.normalCore.Normal ∧ H.normalCore ≤ K.normalCore ∧  
    H.normalCore.normalCore = H.normalCore := by  
  intro H K hHK  
  have hnormal : H.normalCore.Normal := Subgroup.normalCore_normal H  
  have hmono : H.normalCore ≤ K.normalCore := Subgroup.normalCore_mono hHK  
  have hidem : H.normalCore.normalCore = H.normalCore :=  
    Subgroup.normalCore_idempotent H  
  exact ⟨hnormal, hmono, hidem⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 3 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.ms.ms.mh__e58692b3

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint e58692b36402bfd3

4.176 176. existsUnique_least_normalClosure

The problem

For a subset S of a group, show that there exists a unique least normal subgroup containing S .

Lean statement

```
theorem existsUnique_least_normalClosure {G : Type*} [Group G] (s : Set G) :
  ∃! N : Subgroup G, N.Normal ∧ s ⊆ N ∧
  ∀ M : Subgroup G, M.Normal → s ⊆ M → N ≤ M
```

Parent. For a subset S of a group, show that its normal closure is the least normal subgroup containing S .

Parent in Lean

```
theorem normalClosure_is_least_normal {G : Type*} [Group G] (s : Set G) :
  ∃ N : Subgroup G, N.Normal ∧ s ⊆ N ∧
  ∀ M : Subgroup G, M.Normal → s ⊆ M → N ≤ M
```

What it contributes. Supplies the normal-closure witness, normality, subset containment, and leastness checkpoints; the child adds mutual comparison and uniqueness.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from existence of a least normal subgroup to existence and uniqueness. Its proof adds substantive mutual comparison and antisymmetry reasoning to establish uniqueness, which the parent does not provide.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the existential leastness claim into an existence-and-uniqueness characterization. Its uniqueness proof requires a new mutual-comparison argument using both universal properties and antisymmetry, which is not supplied by the parent's proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens existence of a least normal subgroup to uniqueness. Its proof adds a genuine mutual-inclusion argument using each candidate's universal leastness, which the parent's proof does not provide.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2955 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem existsUnique_least_normalClosure.extracted_1_1.{u_1} {G : Type u_1} [inst : Group G] (s : Set G) :  
  ∃! N, N.Normal ∧ s ⊆ ↑N ∧ ∀ (M : Subgroup G), M.Normal → s ⊆ ↑M → N ≤ M
```

Read back by a model that saw only the Lean. *There exists a unique normal subgroup N of G such that s is contained in N and, for every normal subgroup M of G containing s , N is contained in M .*

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open Fintype Subgroup Set Polynomial Ideal  
open scoped BigOperators  
  
theorem existsUnique_least_normalClosure {G : Type*} [Group G] (s : Set G) :  
  ∃! N : Subgroup G, N.Normal ∧ s ⊆ N ∧  
    ∀ M : Subgroup G, M.Normal → s ⊆ M → N ≤ M := by  
  let N : Subgroup G := Subgroup.normalClosure s  
  have hN : N.Normal := by  
    dsimp [N]  
    exact Subgroup.normalClosure_normal  
  have hsN : s ⊆ N := by  
    dsimp [N]  
    exact Subgroup.subset_normalClosure  
  have hleast : ∀ M : Subgroup G, M.Normal → s ⊆ M → N ≤ M := by  
    intro M hM hsM  
    letI : M.Normal := hM  
    dsimp [N]  
    exact Subgroup.normalClosure_le_normal hsM  
  have huniq : ∀ M : Subgroup G,  
    (M.Normal ∧ s ⊆ M ∧ ∀ K : Subgroup G, K.Normal → s ⊆ K → M ≤ K) → M = N := by  
    intro M hM  
    have hMN : M ≤ N := hM.2.2 N hN hsN  
    have hNM : N ≤ M := hleast M hM.1 hM.2.1  
    exact le_antisymm hMN hNM  
  refine ⟨N, ⟨hN, hsN, hleast⟩, ?_⟩  
  intro M hM  
  exact huniq M hM
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__mh.mh__8e79d468

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint 8e79d4687ae16da7

4.177 177. normalClosure_is_least_normal

The problem

For a subset S of a group, show that its normal closure is the least normal subgroup containing S .

Lean statement

```
theorem normalClosure_is_least_normal {G : Type*} [Group G] (s : Set G) :
  ∃ N : Subgroup G, N.Normal ∧ s ⊆ N ∧
  ∀ M : Subgroup G, M.Normal → s ⊆ M → N ≤ M
```

Parent. Prove that the intersection of an arbitrary nonempty collection of normal subgroups of a group is a normal subgroup (do not assume the collection is countable).

Parent in Lean

```
theorem Dummit_Foote_exercise_3.1.22b {G : Type*} [Group G] (I : Type*) [Nonempty I]
  (H : I → Subgroup G) (hH : ∀ i : I, Normal (H i)) :
  Normal (⋂ (i : I), H i) := by
```

What it contributes. Supplies the central arbitrary-intersection normality checkpoint, applied to the three nested index families in the normal-closure construction.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from normality of an indexed intersection to an existence-and-leastness characterization of the normal closure. Proving minimality for every normal subgroup containing the set requires mathematics not supplied by the parent's normality lemma.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes normality of an arbitrary intersection into the substantive universal-property characterization of the normal closure. Its proof requires constructing the closure, proving containment, and applying the leastness theorem, so recalling the parent's one-lemma proof does not solve it.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the goal from proving normality of an indexed intersection to constructing and characterizing the least normal subgroup containing an arbitrary set. It requires existence, containment, and the minimality argument via 'normalClosure_le_normal', which the parent proof does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2939 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normalClosure_is_least_normal.extracted_1_1.{u_1} {G : Type u_1} [inst : Group G] (s : Set G) :  
  ∃ N, N.Normal ∧ s ⊆ ↑N ∧ ∀ (M : Subgroup G), M.Normal → s ⊆ ↑M → N ≤ M
```

Read back by a model that saw only the Lean. For a group G and a subset s of G , there exists a normal subgroup N of G such that s is contained in N , and for every normal subgroup M of G containing s , N is a subgroup of M .

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
  
open Fintype Subgroup Set Polynomial Ideal  
open scoped BigOperators  
  
theorem normalClosure_is_least_normal {G : Type*} [Group G] (s : Set G) :  
  ∃ N : Subgroup G, N.Normal ∧ s ⊆ N ∧  
    ∀ M : Subgroup G, M.Normal → s ⊆ M → N ≤ M := by  
  let N : Subgroup G := Subgroup.normalClosure s  
  have hN : N.Normal := by  
    dsimp [N]  
    rw [Subgroup.normalClosure_eq_iInf]  
    apply Subgroup.normal_iInf_normal  
    intro H  
    apply Subgroup.normal_iInf_normal  
    intro hH  
    apply Subgroup.normal_iInf_normal  
    intro hs  
    exact hH  
  have hsN : s ⊆ N := by  
    dsimp [N]  
    exact Subgroup.subset_normalClosure  
  refine ⟨N, hN, hsN, ?_⟩  
  intro M hM hsM  
  letI : M.Normal := hM  
  dsimp [N]  
  exact Subgroup.normalClosure_le_normal hsM
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__mh__16e3ee99

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 16e3ee99450f2958

4.178 178. Dummit_Foote_exercise_3_1_22b_mutation

The problem

Prove that the intersection of an arbitrary nonempty collection of normal subgroups is the greatest normal subgroup contained in every member of the collection.

Lean statement

```
theorem Dummit_Foote_exercise_3_1_22b_mutation {G : Type*} [Group G] (I : Type*) [Nonempty I]
  (H : I → Subgroup G) (hH : ∀ i : I, Normal (H i)) :
  IsGreatest {K : Subgroup G | Normal K ∧ ∀ i : I, K ≤ H i} (iInf H)
```

Parent. Prove that the intersection of an arbitrary nonempty collection of normal subgroups of a group is a normal subgroup (do not assume the collection is countable).

Parent in Lean

```
theorem Dummit_Foote_exercise_3_1_22b {G : Type*} [Group G] (I : Type*) [Nonempty I]
  (H : I → Subgroup G) (hH : ∀ i : I, Normal (H i)) :
  Normal (⋂ (i : I), H i) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from mere normality to a greatest-element characterization, requiring a new universal-order argument that every subgroup contained in all $H(i)$ lies below $i\text{Inf } H$. Although it reuses the parent's normality fact, the added obligation is substantive and not supplied by recalling the parent proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from normality to a greatest-element characterization. Besides establishing normality via the parent, it must prove membership and the universal order bound using the lattice properties of infima.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from normality to a greatest-element characterization of the intersection. The parent's normality lemma supplies only one component; proving maximality requires a separate lattice argument using ' le_iInf ', so recalling the parent is insufficient.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2810 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem Dummit_Foote_exercise_3_1_22b_mutation.extracted_1.1.{u_2, u_1} {G : Type u_1} [inst : Group G] (I : Type u_2)
  [inst_1 : Nonempty I] (H : I → Subgroup G) (hH : ∀ (i : I), (H i).Normal) :
  IsGreatest {K | K.Normal ∧ ∀ (i : I), K ≤ H i} (iInf H)
```

Read back by a model that saw only the Lean. Let G be a group, let I be a nonempty type, and let H assign to each $i \in I$ a normal subgroup $H(i)$ of G . Then the infimum $\bigcap_{i \in I} H(i)$ is the greatest normal subgroup K of G such that $K \leq H(i)$ for every $i \in I$; that is, it itself is normal and contained in every $H(i)$, and every normal subgroup contained in every $H(i)$ is contained in $\bigcap_{i \in I} H(i)$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem Dummit_Foote_exercise_3_1_22b_mutation {G : Type*} [Group G] (I : Type*) [Nonempty I]
  (H : I → Subgroup G) (hH : ∀ i : I, Normal (H i)) :
  IsGreatest {K : Subgroup G | Normal K ∧ ∀ i : I, K ≤ H i} (iInf H) := by
  have hnormal : Normal (iInf H) :=
    Subgroup.normal_iInf_normal (fun i => hH i)
  have hlower : ∀ i : I, iInf H ≤ H i := by
    intro i
    exact iInf_le H i
  have hmem : iInf H ∈ {K : Subgroup G | Normal K ∧ ∀ i : I, K ≤ H i} := by
    exact ⟨hnormal, hlower⟩
  have hbound : ∀ {K : Subgroup G},
    K ∈ {K : Subgroup G | Normal K ∧ ∀ i : I, K ≤ H i} → K ≤ iInf H := by
    intro K hK
    exact le_iInf (fun i => hK.2 i)
  exact ⟨hmem, hbound⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__mh__f53b8227

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint f53b8227f02957dc

4.179 179. quotient_is_commutative_of_commutator_le

The problem

Let A be a group and let B be a normal subgroup containing the commutator subgroup of A . Prove that A/B is abelian.

Lean statement

```
theorem quotient_is_commutative_of_commutator_le {A : Type*} [Group A] (B : Subgroup A) [B.Normal] (h : _root_.commutator A ≤ B) :
  ∀ a b : A / B, a * b = b * a
```

Parent. Let A be an abelian group and let B be a subgroup of A . Prove that A/B is abelian.

Parent in Lean

```
theorem Dummit_Foote_exercise_3.1.3a {A : Type*} [CommGroup A] (B : Subgroup A) :
  ∀ a b : A / B, a * b = b * a := by
```

What it contributes. Preserves the parent's quotient-commutativity checkpoint while weakening the ambient hypothesis from an abelian group to an arbitrary group with a commutator-containment condition.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes from an ambient commutative group to an arbitrary group whose normal subgroup contains the commutator subgroup. Proving quotient commutativity now requires the commutator characterization and cannot be obtained by directly applying the parent's 'mul_comm' argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the ambient commutative-group assumption with a general group and requires proving quotient commutativity from the substantive hypothesis that the commutator subgroup lies in the normal subgroup. The parent's direct application of multiplication commutativity cannot supply this quotient-group argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes from an inherently commutative ambient group to an arbitrary group, requiring the nontrivial commutator-subgroup criterion and normality hypothesis to establish quotient commutativity. The parent's direct 'mul_comm' proof cannot supply this argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2934 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quotient_is_commutative_of_commutator_le.extracted_1_1 :  $\forall \{A : \text{Type } u_1\} [\text{inst} : \text{Group } A] (B : \text{Subgroup } A)$   
[inst_1 : B.Normal],  $\_root_.\text{commutator } A \leq B \rightarrow \forall (a \ b : A / B), a * b = b * a$ 
```

Read back by a model that saw only the Lean. *For every group A and every subgroup B of A that is normal, if the commutator subgroup of A is contained in B , then every pair of elements a and b of the quotient group A / B commutes: $a * b = b * a$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem quotient_is_commutative_of_commutator_le {A : Type*} [Group A] (B : Subgroup A) [B.Normal] (h :  $\_root_.\text{commutator } A \leq B$ ) :
   $\forall a \ b : A / B, a * b = b * a$  := by
  have hc : IsMulCommutative (A / B) :=
    (Subgroup.Normal.quotient_commutative_iff_commutator_le).2 h
  intro a b
  exact hc.is_comm.comm a b
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_3a__mh__68f603c7

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 68f603c74754d74f

4.180 180. finite_abelian_group_prime_power_subgroup_quotient

The problem

Let G be a finite abelian group and let p^k divide its order, where p is prime. Show that G has a subgroup of order p^k whose quotient has order $|G|/p^k$.

Lean statement

```
theorem finite_abelian_group_prime_power_subgroup_quotient {G : Type*} [CommGroup G] [Fintype G] {p k : Nat} (hp : Nat.Prime p) (h : p ^ k ∣ Fintype.card G) :
  ∃ (H : Subgroup G) (hH : Fintype H), Nat.card H = p ^ k ∧ Nat.card (G / H) = Fintype.card G / (p ^ k)
```

Parent. Use Cauchy's Theorem and induction to show that a finite abelian group has a subgroup of order n for each positive divisor n of its order.

Parent in Lean

```
theorem Dummit_Foote_exercise_3.4.4 {G : Type*} [CommGroup G] [Fintype G] {n : ℕ}
  (hn : n ∣ (card G)) :
  ∃ (H : Subgroup G) (H_fin : Fintype H), @card H H_fin = n := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child adds a substantive quotient-cardinality conclusion, requiring normality of the subgroup and the finite quotient cardinality formula, which the parent's existence-and-order result does not supply. Although Sylow's theorem supplies the subgroup order in both proofs, the new quotient argument is genuine proof-obligation expansion rather than recall.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the obligation from merely producing a subgroup of order n to producing a prime-power subgroup together with an exact quotient-cardinality formula. Its proof requires normality, quotient finiteness, the subgroup-quotient cardinality identity, and division arithmetic beyond the parent's construction.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a substantive quotient-cardinality theorem, requiring normality, quotient finiteness, and a cardinality product/division argument beyond the parent's subgroup-existence conclusion. Although Sylow supplies the subgroup order, the new quotient equality is not a forced wrapper or trivial conjunction.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2215 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_abelian_group_prime_power_subgroup_quotient.extracted_1_1.{u_1} {G : Type u_1} [inst : CommGroup G]
  [inst.1 : Fintype G] {p k : ℕ} (hp : Nat.Prime p) (h : p ^ k ∣ card G) :
  ∃ H x, Nat.card H = p ^ k ∧ Nat.card (G / H) = card G / p ^ k
```

Read back by a model that saw only the Lean. *Let G be a finite commutative group, let p and k be natural numbers, assume that p is prime and that p^k divides the cardinality of G . Then there exist a subgroup H of G and an element x of G such that the cardinality of H is p^k and the cardinality of the quotient group G/H is $\text{card}(G)$ divided by p^k .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem finite_abelian_group_prime_power_subgroup_quotient {G : Type*} [CommGroup G] [Fintype G] {p k : Nat} (hp : Nat.Prime p) (h : p ^
  k ∣ Fintype.card G) :
  ∃ (H : Subgroup G) (h : Fintype H), Nat.card H = p ^ k ∧ Nat.card (G / H) = Fintype.card G / (p ^ k) := by
  classical
  letI : Fact (Nat.Prime p) := ⟨hp⟩
  have h' : p ^ k ∣ Nat.card G := by
    simpa [Nat.card_eq_fintype_card] using h
  obtain ⟨H, hH⟩ := Sylow.exists_subgroup_card_pow_prime (G := G) p h'
  letI : Fintype H := Fintype.ofFinite H
  have hcard : Nat.card H = p ^ k := hH
  have hcardF : Fintype.card H = p ^ k := by
    simpa [Nat.card_eq_fintype_card] using hcard
  letI : H.Normal := H.normal_of_comm
  letI : Fintype (G / H) := Quotient.fintype (QuotientGroup.leftRel H)
  have hmul : Fintype.card (G / H) * Fintype.card H = Fintype.card G := by
    simpa using (Subgroup.card_eq_card_quotient_mul_card_subgroup H).symm
  have hquot : Nat.card (G / H) = Fintype.card G / (p ^ k) := by
    have hpos : 0 < p ^ k := pow_pos hp.pos k
    have hmul' : Fintype.card (G / H) * (p ^ k) = Fintype.card G := by
      simpa [hcardF] using hmul
    calc
      Nat.card (G / H) = Fintype.card (G / H) := Nat.card_eq_fintype_card (α := G / H)
      _ = Fintype.card (G / H) * (p ^ k) / (p ^ k) := (Nat.mul_div_cancel _ hpos).symm
      _ = Fintype.card G / (p ^ k) := by rw [hmul']
  exact ⟨H, inferInstance, hcard, hquot⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_4_4__mh__174fe9de

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint 174fe9de19a2f61f

4.181 181. unique_normal_sylow_of_card_eq_mul

The problem

Let p be a prime with $25 \leq p \leq 100$. If a finite group has order $24p$, prove that it has a unique normal Sylow p -subgroup.

Lean statement

```
theorem unique_normal_sylow_of_card_eq_mul {G : Type*} [Group G] [Fintype G]
  (p : ℕ) (hp : p.Prime) (hp_bound : 25 ≤ p ∧ p ≤ 100)
  (hG : Nat.card G = 24 * p) :
  ∃ P : Sylow p G, p | Nat.card G ∧ P.Normal ∧ ∀ Q : Sylow p G, Q = P
```

Parent. Prove that a group of order 312 has a normal Sylow p -subgroup for some prime p dividing its order.

Parent in Lean

```
theorem Dummit_Foote_exercise_4_5_14 {G : Type*} [Group G] [Fintype G]
  (hG : card G = 312) :
  ∃ (p : ℕ) (P : Sylow p G), p.Prime ∧ (p | card G) ∧ P.Normal := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the fixed prime and group order, and changes existence of a normal Sylow subgroup into a uniqueness conclusion. Its proof must establish the variable-prime bound argument forcing the Sylow count to equal one, which the parent's fixed-instance proof does not supply.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the specific prime and group order into arbitrary prime p with $25 \leq p \leq 100$, and changes existence of a normal Sylow subgroup into uniqueness of that subgroup. The bound is used essentially to force the Sylow count n dividing 24 and satisfying $n \equiv 1 \pmod p$ to equal 1, yielding a new uniqueness obligation.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the prime and group order, requiring the new bound-based argument that the Sylow count satisfies $n < p$, and it strengthens normality to uniqueness of the Sylow subgroup. The parent's fixed arithmetic for $p = 13$ and $|G| = 312$ cannot be recalled directly to solve this statement.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2239 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem unique_normal_sylow_of_card_eq_mul.extracted_1_1.{u_1} {G : Type u_1} [inst : Group G] (p : ℕ)
  (hp : Nat.Prime p) (hp_bound : 25 ≤ p ∧ p ≤ 100) (hG : Nat.card G = 24 * p) :
  ∃ P, p ∣ Nat.card G ∧ (↑P).Normal ∧ ∀ (Q : Sylow p G), Q = P
```

Read back by a model that saw only the Lean. For a group G and a natural number p , if p is prime, $25 \leq p \leq 100$, and the cardinality of G is $24p$, then there exists a Sylow p -subgroup P of G such that p divides the cardinality of G , the subgroup P is normal in G , and every Sylow p -subgroup Q of G equals P .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem unique_normal_sylow_of_card_eq_mul {G : Type*} [Group G] [Fintype G]
  (p : ℕ) (hp : p.Prime) (hp_bound : 25 ≤ p ∧ p ≤ 100)
  (hG : Nat.card G = 24 * p) :
  ∃ P : Sylow p G, p ∣ Nat.card G ∧ P.Normal ∧ ∀ Q : Sylow p G, Q = P := by
classical
letI : Fact p.Prime := ⟨hp⟩
obtain ⟨P⟩ := (inferInstance : Nonempty (Sylow p G))
set n : ℕ := Nat.card (Sylow p G) with hn
have hmod : n ≡ 1 [MOD p] := by
  simpa [n] using (card_sylow_modEq_one p G)
have hdiv_index : n ∣ P.index := by
  simpa [n] using (Sylow.card_dvd_index (p := p) (G := G) P)
have hindex_dvd : P.index ∣ Nat.card G := by
  simpa [Nat.card_eq_fintype_card] using P.1.index_dvd_card
have hdiv_card : n ∣ Nat.card G := Nat.dvd_trans hdiv_index hindex_dvd
have hdiv_mul : n ∣ p * 24 := by
  rw [hG] at hdiv_card
  simpa [Nat.mul_comm] using hdiv_card
have hnotdiv : ¬p ∣ n := by
  simpa [n] using (not_dvd_card_sylow p G)
have hcoprime : Nat.Coprime n p := by
  simpa [Nat.coprime_comm] using (hp.coprime_iff_not_dvd).2 hnotdiv
have hdiv24 : n ∣ 24 := hcoprime.dvd_of_dvd_mul_left hdiv_mul
have hpos : 0 < n := by
  simpa [n] using (Nat.card_pos : 0 < Nat.card (Sylow p G))
have hn_le : n ≤ 24 := Nat.le_of_dvd (by decide) hdiv24
have hn_lt_p : n < p := by omega
have hmod_eq : n % p = 1 % p := by
  simpa [Nat.ModEq] using hmod
have hn_eq_one : n = 1 := by
  have hn_mod : n % p = n := Nat.mod_eq_of_lt hn_lt_p
  have hmod : 1 % p = 1 := Nat.mod_eq_of_lt (by omega)
  omega
have hsub : Subsingleton (Sylow p G) := by
  exact (Nat.card_eq_one_iff_unique.mp (by simpa [n] using hn_eq_one)).1
have hnormal : P.Normal := by
  letI : Subsingleton (Sylow p G) := hsub
  exact Sylow.normal_of_subsingleton P
have hunique : ∀ Q : Sylow p G, Q = P := by
  intro Q
  exact Subsingleton.elim Q P
refine ⟨P, ?_, hnormal, hunique⟩
rw [hG]
exact ⟨24, by simp [Nat.mul_comm]⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_4_5_14__mh__15ef8ed7

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 15ef8ed7b86c3e91

4.182 182. bounded_normal_sylow

The problem

Let G be a finite group and let p be a prime with $25 \leq p \leq 100$. If $|G| = 24p$, prove that G has a normal Sylow p -subgroup and that p divides $|G|$.

Lean statement

```
theorem bounded_normal_sylow {G : Type*} [Group G] [Fintype G]
  (p : ℕ) (hp : p.Prime) (hp_bound : 25 ≤ p ∧ p ≤ 100)
  (hG : Nat.card G = 24 * p) :
  ∃ P : Sylow p G, p ∣ Nat.card G ∧ P.Normal := by
```

Parent. Prove that a group of order 312 has a normal Sylow p -subgroup for some prime p dividing its order.

Parent in Lean

```
theorem Dummit_Foote_exercise_4_5_14 {G : Type*} [Group G] [Fintype G]
  (hG : card G = 312) :
  ∃ (p : ℕ) (P : Sylow p G), p.Prime ∧ (p ∣ card G) ∧ P.Normal := by
```

What it contributes. Reused the finite-group Sylow existence, Sylow-count congruence, count divisibility through the subgroup index, and coprime cancellation infrastructure while replacing 312 and 13 by the bounded parameter family $24p$ and p .

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the Sylow argument over primes p with $25 \leq p \leq 100$. It derives that the Sylow count divides 24 while being congruent to 1 modulo p , then uses the bound $p > 24$ to force uniqueness and normality; this reasoning is not supplied by the fixed-order parent proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the Sylow-counting argument: it derives $n \mid 24$, uses $p > 24$ to force $n < p$, and combines this with $n \equiv 1 \pmod p$ to prove $n = 1$. The fixed-order parent does not supply this symbolic bounded-family reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes 13 as an arbitrary prime p greater than 24 and replaces fixed arithmetic case-work with a symbolic divisibility and congruence argument showing $n=1$. A memorized proof for order 312 and $p=13$ does not directly establish this family.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1918 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_normal_sylow.extracted_1.1.{u_1} {G : Type u_1} [inst : Group G] (p : ℕ) (hp : Nat.Prime p)
  (hp_bound : 25 ≤ p ∧ p ≤ 100) (hG : Nat.card G = 24 * p) : ∃ P, p | Nat.card G ∧ (↑P).Normal
```

Read back by a model that saw only the Lean. *For a group G , a natural number p that is prime and satisfies $25 \leq p \leq 100$, if the cardinality of G is $24p$, then there exists a Sylow p -subgroup P of G such that p divides the cardinality of G and the subgroup underlying P is normal in G .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem bounded_normal_sylow {G : Type*} [Group G] [Fintype G]
  (p : ℕ) (hp : p.Prime) (hp_bound : 25 ≤ p ∧ p ≤ 100)
  (hG : Nat.card G = 24 * p) :
  ∃ P : Sylow p G, p | Nat.card G ∧ P.Normal := by
  classical
  letI : Fact p.Prime := ⟨hp⟩
  obtain ⟨P⟩ := (inferInstance : Nonempty (Sylow p G))
  set n : ℕ := Nat.card (Sylow p G) with hn
  have hmod : n ≡ 1 [MOD p] := by
    simpa [n] using (card_sylow_modEq_one p G)
  have hdiv_index : n | P.index := by
    simpa [n] using (Sylow.card_dvd_index (p := p) (G := G) P)
  have hindex_dvd : P.index | Nat.card G := by
    simpa [Nat.card_eq_fintype_card] using P.1.index_dvd_card
  have hdiv_card : n | Nat.card G := Nat.dvd_trans hdiv_index hindex_dvd
  have hdiv_mul : n | p * 24 := by
    rw [hG] at hdiv_card
    simpa [Nat.mul_comm] using hdiv_card
  have hnotdiv : ¬p | n := by
    simpa [n] using (not_dvd_card_sylow p G)
  have hcoprime : Nat.Coprime n p := by
    simpa [Nat.coprime_comm] using (hp.coprime_iff_not_dvd).2 hnotdiv
  have hdiv24 : n | 24 := hcoprime.dvd_of_dvd_mul_left hdiv_mul
  have hpos : 0 < n := by
    simpa [n] using (Nat.card_pos : 0 < Nat.card (Sylow p G))
  have hn_le : n ≤ 24 := Nat.le_of_dvd (by decide) hdiv24
  have hn_lt_p : n < p := by omega
  have hmod_eq : n % p = 1 % p := by
    simpa [Nat.ModEq] using hmod
  have hn_eq_one : n = 1 := by
    have hn_mod : n % p = n := Nat.mod_eq_of_lt hn_lt_p
    have hmod : 1 % p = 1 := Nat.mod_eq_of_lt (by omega)
    omega
  have hsub : Subsingleton (Sylow p G) := by
    exact (Nat.card_eq_one_iff_unique.mp (by simpa [n] using hn_eq_one)).1
  have hnormal : P.Normal := by
    letI : Subsingleton (Sylow p G) := hsub
    exact Sylow.normal_of_subsingleton P
  refine ⟨P, ?_, hnormal⟩
  rw [hG]
  exact ⟨24, by simp [Nat.mul_comm]⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_4_5_14__mh__fd07aa1a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint d05b91cd454267e7

4.183 183. exists_element_order_eleven

The problem

Let G be a finite group of order 462. Prove that G contains an element of order 11.

Lean statement

```
theorem exists_element_order_eleven {G : Type*} [Fintype G] [Group G]
  (hG : Fintype.card G = 462) : ∃ x : G, orderOf x = 11
```

Parent. Prove that if $|G| = 462$ then G is not simple.

Parent in Lean

```
theorem Dummit_Foote_exercise_4_5_23 {G : Type*} [Fintype G] [Group G]
  (hG : card G = 462) : ¬ IsSimpleGroup G := by
```

What it contributes. Preserves the finite group and cardinality-462 setting while redirecting the cardinality factorization toward an element-order witness.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from nonsimplicity to existence of an element of order 11 and proves it via a prime-order divisibility theorem, rather than reusing the parent's Sylow-counting argument. The factorization of 462 supplies the required divisibility checkpoint, and no parent conclusion is wrapped or recalled.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from nonsimplicity to existence of an element of order 11 and proves it via prime-order divisibility, using the factorization of 462. The parent's Sylow-subgroup uniqueness and contradiction argument cannot be recalled to solve this statement.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from non-simplicity to explicitly constructing an element of order 11, using divisibility of the group order and a prime-order existence theorem. This is not obtained by simply recalling the parent's contradiction argument or its normal-subgroup conclusion.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2768 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem exists_element_order_eleven.extracted_1_1.{u_1} {G : Type u_1} [inst : Fintype G] [inst_1 : Group G]
(hG : card G = 462) : ∃ x, orderOf x = 11
```

Read back by a model that saw only the Lean. *For any finite group G with exactly 462 elements, there exists an element x in G whose order is 11.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Subgroup Set Polynomial Ideal
open scoped BigOperators

theorem exists_element_order_eleven {G : Type*} [Fintype G] [Group G]
(hG : Fintype.card G = 462) : ∃ x : G, orderOf x = 11 := by
  have h_factor : Fintype.card G = 11 * 42 := by
    calc
      Fintype.card G = 462 := hG
      _ = 11 * 42 := by norm_num
  have h_prime11 : Nat.Prime 11 := by
    norm_num
  letI : Fact (Nat.Prime 11) := ⟨h_prime11⟩
  have h_dvd : 11 ∣ Fintype.card G := by
    rw [h_factor]
    norm_num
  have h_witness : ∃ x : G, orderOf x = 11 :=
    exists_prime_orderOf_dvd_card 11 h_dvd
  exact h_witness
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_4_5_23__mh__fe35117f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint fe35117fe069b9e1

4.184 184. rational_sum_sq_mul

The problem

Prove that if an integer n is the sum of the squares of two rational numbers and an integer m is also the sum of the squares of two rational numbers, then nm is the sum of the squares of two rational numbers.

Lean statement

```
theorem rational_sum_sq_mul {n m : ℤ} {r s u v : ℚ}
  (hn : r ^ 2 + s ^ 2 = n) (hm : u ^ 2 + v ^ 2 = m) :
  ∃ x y : ℚ, x ^ 2 + y ^ 2 = (n * m : ℚ)
```

Parent. Prove that if an integer is the sum of two rational squares, then it is the sum of two integer squares.

Parent in Lean

```
theorem Dummit_Foote_exercise_8_3_4 {n : ℤ} {r s : ℚ}
  (h : r^2 + s^2 = n) :
  ∃ a b : ℤ, a^2 + b^2 = n := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a genuinely different closure theorem: two rational sum-of-two-squares representations combine via the Brahmagupta identity to represent the product. The parent's denominator-clearing and integer-witness argument does not supply this multiplicative construction.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child establishes the symbolic multiplicative closure of rational sums of two squares, using the Brahmagupta–Fibonacci identity and two independent hypotheses. This reasoning is absent from the parent's denominator-clearing argument and cannot be recovered by recalling the parent's proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves the genuinely new Brahmagupta–Fibonacci two-square multiplication identity, combining both hypotheses through a nontrivial ring calculation. The parent only supplies separate representability results and does not provide this multiplicative construction.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1886 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem rational_sum_sq_mul.extracted_1_1 {n m : ℤ} {r s u v : ℚ} (hn : r ^ 2 + s ^ 2 = ↑n) (hm : u ^ 2 + v ^ 2 = ↑m) :  
  ∃ x y, x ^ 2 + y ^ 2 = ↑n * ↑m
```

Read back by a model that saw only the Lean. For integers n and m and rational numbers r, s, u , and v , if $r^2 + s^2 = n$ and $u^2 + v^2 = m$ (viewing n and m as rational numbers), then there exist rational numbers x and y such that $x^2 + y^2 = nm$ (viewing n and m as rational numbers).

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open Fintype Subgroup Set Polynomial Ideal  
open scoped BigOperators  
  
theorem rational_sum_sq_mul {n m : ℤ} {r s u v : ℚ}  
  (hn : r ^ 2 + s ^ 2 = n) (hm : u ^ 2 + v ^ 2 = m) :  
  ∃ x y : ℚ, x ^ 2 + y ^ 2 = (n * m : ℚ) := by  
  refine ⟨r * u - s * v, r * v + s * u, ?_⟩  
  calc  
    (r * u - s * v) ^ 2 + (r * v + s * u) ^ 2 =  
      (r ^ 2 + s ^ 2) * (u ^ 2 + v ^ 2) := by ring  
    _ = (n : ℚ) * (m : ℚ) := by rw [hn, hm]  
    _ = (n * m : ℚ) := by norm_num
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_8_3_4__mh__18a1117f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 23fe6774c7a20af8

4.185 185. unit_cyclic_action_factorization

The problem

Let a , m , and n be natural numbers with $n > 0$. If $a^n \equiv 1 \pmod{m}$ and a is a unit modulo m , show that exponentiation by a defines a multiplicative cyclic action of the n residue classes.

Lean statement

```
theorem unit_cyclic_action_factorization {a n m : ℕ} (hn : 0 < n)
  (hunit : IsUnit (a : ZMod m))
  (hperiod : ((a : ZMod m) ^ n = 1)) :
  ∃ f : Fin n → (ZMod m)×,
    (∀ k : ℕ, f ⟨k % n, Nat.mod_lt _ hn⟩ = hunit.unit ^ k) ∧
    (∀ r s : Fin n,
      f r * f s = f ⟨(r : ℕ) + (s : ℕ) % n, Nat.mod_lt _ hn⟩)
```

Parent. Let a , m , and n be natural numbers with $n > 0$. If $a^n \equiv 1 \pmod{m}$, show that exponentiation by a depends only on the residue of the exponent modulo n , and hence defines a function on the n residue classes.

Parent in Lean

```
theorem residue_power_action {a n m : ℕ} (hn : 0 < n)
  (hperiod : ((a : ZMod m) ^ n = 1)) :
  ∃ f : Fin n → ZMod m,
    (∀ x y : ℕ, x % n = y % n →
      (a : ZMod m) ^ x = (a : ZMod m) ^ y) ∧
    (∀ k : ℕ, f ⟨k % n, Nat.mod_lt _ hn⟩ = (a : ZMod m) ^ k)
```

What it contributes. Used the division-algorithm reduction of exponents modulo n and the visible period equation; upgraded the codomain from residues to units and added multiplicative factorization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child requires genuinely new mathematics: lifting the residue-class element to a unit, transferring the period relation to the unit group, and proving the cyclic multiplicative law. The parent only establishes residue invariance and existence of a function into $\text{ZMod } m$, so recalling its proof does not supply the unit-valued group-action argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely lifts the residue-indexed action from $\text{ZMod } m$ to its unit group and adds a cyclic multiplication law. The parent's reduction argument does not supply the unit lift, the induced period in $(\text{ZMod } m)^\times$, or the group-action identity.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child upgrades the codomain from $\text{ZMod } m$ to its unit group and adds a genuine cyclic-action law for combining residue indices. Its proof requires transporting the period relation through `Units.ext` and proving multiplicative compatibility, which the parent's residue-reduction argument does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound

Check	By	Result
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2319 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem unit_cyclic_action_factorization.extracted_1_1 {a n m : ℕ} (hn : 0 < n) (hunit : IsUnit ↑a)
(hperiod : ↑a ^ n = 1) :
  ∃ f, (∀ (k : ℕ), f (k % n, --) = hunit.unit ^ k) ∧ ∀ (r s : Fin n), f r * f s = f ((↑r + ↑s) % n, --)
```

Read back by a model that saw only the Lean. For natural numbers a , n , and m , if n is positive, the natural number a (viewed in the relevant monoid) is a unit, and the associated unit hunit.unit satisfies $\text{hunit.unit}^n = 1$, then there exists a function f from $\text{Fin } n$ to the monoid of units such that, for every natural number k , f at the residue class $k \bmod n$ equals hunit.unit^k , and for all $r, s : \text{Fin } n$, $f(r) \cdot f(s)$ equals f at the residue class $(r+s) \bmod n$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Set Real Ideal Polynomial
open scoped BigOperators

theorem unit_cyclic_action_factorization {a n m : ℕ} (hn : 0 < n)
(hunit : IsUnit (a : ZMod m))
(hperiod : ((a : ZMod m) ^ n = 1)) :
  ∃ f : Fin n → (ZMod m)×,
    (∀ k : ℕ, f (k % n, Nat.mod_lt _ hn) = hunit.unit ^ k) ∧
    (∀ r s : Fin n,
      f r * f s = f ((r : ℕ) + (s : ℕ)) % n, Nat.mod_lt _ hn)) := by
  have hperiodu : hunit.unit ^ n = 1 := by
    apply Units.ext
    simpa [hunit.unit_spec] using hperiod
  have hreduce : ∀ k : ℕ,
    hunit.unit ^ k = hunit.unit ^ (k % n) := by
    intro k
    have hrepr : k = k % n + n * (k / n) := by
      exact (Nat.mod_add_div k n).symm
    calc
      hunit.unit ^ k = hunit.unit ^ (k % n + n * (k / n)) := by
        exact congrArg (fun t : ℕ => hunit.unit ^ t) hrepr
      _ = hunit.unit ^ (k % n) := by
        rw [pow_add, pow_mul, hperiodu]
      simp
  refine (fun r : Fin n => hunit.unit ^ (r : ℕ), ?_, ?_)
  · intro k
    exact (hreduce k).symm
  · intro r s
    change hunit.unit ^ (r : ℕ) * hunit.unit ^ (s : ℕ) =
      hunit.unit ^ ((r : ℕ) + (s : ℕ)) % n
    calc
      hunit.unit ^ (r : ℕ) * hunit.unit ^ (s : ℕ) =
        hunit.unit ^ ((r : ℕ) + (s : ℕ)) := by rw [← pow_add]
      _ = hunit.unit ^ ((r : ℕ) + (s : ℕ)) % n := hreduce _
```

Operator. mutation / mutation_hard **Lineage depth.** 3 **Certificate.** reproducible

Problem id. Herstein_exercise_2_4_36__me.me.mh.mh__769445da

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 769445da3e5fcf13

4.186 186. residue_power_action

The problem

Let a , m , and n be natural numbers with $n > 0$. If $a^n \equiv 1 \pmod{m}$, show that exponentiation by a depends only on the residue of the exponent modulo n , and hence defines a function on the n residue classes.

Lean statement

```
theorem residue_power_action {a n m : ℕ} (hn : 0 < n)
  (hperiod : ((a : ZMod m) ^ n = 1)) :
  ∃ f : Fin n → ZMod m,
    (∀ x y : ℕ, x % n = y % n →
      (a : ZMod m) ^ x = (a : ZMod m) ^ y) ∧
    (∀ k : ℕ, f (k % n, Nat.mod_lt _ hn) = (a : ZMod m) ^ k)
```

Parent. Let a , n , and m be natural numbers. If a^n is congruent to 1 modulo m , show that a^k is congruent to 1 modulo m for every exponent k divisible by n .

Parent in Lean

```
theorem power_period_of_modulus {a n m : ℕ}
  (hperiod : ((a : ZMod m) ^ n = 1)) :
  ∀ k : ℕ, n ∣ k → ((a : ZMod m) ^ k = 1)
```

What it contributes. Generalizes the parent's divisible-exponent power argument: its period equation and power decomposition become the reduction lemma that supports a residue-indexed function.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes divisibility to a residue-indexed cyclic action: it proves powers depend only on the exponent modulo the period and constructs the corresponding function on 'Fin n'. This requires new remainder decomposition and periodicity reasoning beyond the parent's single divisible-exponent case.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the claim from powers at multiples of n to a well-defined residue-indexed action, requiring division-with-remainder decomposition and reduction of arbitrary exponents. The parent's divisibility argument alone does not establish this remainder-based periodicity or construct the function.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes periodicity from exponents divisible by n to a well-defined action on all residue classes modulo n . Its proof requires division-algorithm decomposition, reduction to the remainder, and construction of a subtype-indexed function, which the parent's divisibility argument does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2313 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	MISMATCH: Statement B replaces the hypothesis $a^n \equiv 1 \pmod m$ with the stronger exact equality $a^n = 1$ and likewise asserts exact equality $a^x = a^y$ rather than congruence modulo m ; it also omits the modulus m from the conclusion.

Goal round-trip: what Lean elaborated

```
theorem residue_power_action.extracted_1_1 {a n m : ℕ} (hn : 0 < n) (hperiod : ↑a ^ n = 1) :
  ∃ f, (∀ (x y : ℕ), x % n = y % n → ↑a ^ x = ↑a ^ y) ∧ ∀ (k : ℕ), f (k % n, --) = ↑a ^ k
```

Read back by a model that saw only the Lean. For natural numbers a , n , and m , if n is positive and the n th power of a (interpreted in the relevant ambient type) equals 1, then there exists a function f on natural numbers less than n such that, for all natural numbers x and y , if x and y have the same remainder modulo n , then $a^x = a^y$, and for every natural number k , f applied to the residue $k \bmod n$ equals a^k .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Set Real Ideal Polynomial
open scoped BigOperators

theorem residue_power_action {a n m : ℕ} (hn : 0 < n)
  (hperiod : ((a : ZMod m) ^ n = 1)) :
  ∃ f : Fin n → ZMod m,
    (∀ x y : ℕ, x % n = y % n →
      (a : ZMod m) ^ x = (a : ZMod m) ^ y) ∧
    (∀ k : ℕ, f (k % n, Nat.mod_lt _ hn) = (a : ZMod m) ^ k) := by
  have hreduce : ∀ k : ℕ,
    (a : ZMod m) ^ k = (a : ZMod m) ^ (k % n) := by
    intro k
    have hrepr : k = k % n + n * (k / n) := by
      exact (Nat.mod_add_div k n).symm
    calc
      (a : ZMod m) ^ k = (a : ZMod m) ^ (k % n + n * (k / n)) := by
        exact congrArg (fun t : ℕ => (a : ZMod m) ^ t) hrepr
      _ = (a : ZMod m) ^ (k % n) := by
        rw [pow_add, pow_mul, hperiod]
        simp
  refine (fun r : Fin n => (a : ZMod m) ^ (r : ℕ), ?_, ?_)
  · intro x y hxy
    calc
      (a : ZMod m) ^ x = (a : ZMod m) ^ (x % n) := hreduce x
      _ = (a : ZMod m) ^ (y % n) := by rw [hxy]
      _ = (a : ZMod m) ^ y := (hreduce y).symm
  · intro k
    exact (hreduce k).symm
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Herstein_exercise_2_4_36__me.me.mh__0604a773

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint 0604a773fdfdc2ec

4.187 187. odd_multiple_period_mod_add_one

The problem

Let $a > 1$ and $n > 0$ be integers. Show that every exponent of the form $n(2q + 1)$ gives $a^{n(2q+1)} \equiv -1 \pmod{a^n + 1}$ for every nonnegative integer q .

Lean statement

```
theorem odd_multiple_period_mod_add_one {a n : ℕ} :
  ∀ q : ℕ, ((a : ℤMod (a ^ n + 1)) ^ (n * (2 * q + 1))) = -1
```

Parent. Let $a > 1$ and $n > 0$ be integers. Show that for every positive exponent k divisible by n , a^k is congruent to 1 modulo $a^n - 1$.

Parent in Lean

```
theorem power_period_mod_sub_one {a n : ℕ} (ha : 1 < a) :
  ∀ k : ℕ, 0 < k → n ∣ k → ((a : ℤMod (a ^ n - 1)) ^ k = 1) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the modulus from $a^n - 1$ to $a^n + 1$ and establishes a sign-changing result: the base power is -1 , so only odd multiples yield the stated value. The parent's argument for a base power equal to 1 does not supply this new odd-exponent reasoning.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes both the modulus and target: it derives $a^n = -1$ modulo $a^n + 1$ and must additionally reason that every odd power of -1 remains -1 . This is not recoverable by recalling the parent's period-to-1 argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the modulus from $a^n - 1$ to $a^n + 1$ and the conclusion from powers divisible by n equaling 1 to odd multiples of n equaling -1 . Its proof requires establishing $a^n = -1$ and handling odd powers of -1 , which the parent's argument does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over proptext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2288 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem odd_multiple_period_mod_add_one.extracted_1.1 : ∀ {a n : ℕ},
  1 < a → 0 < n → ∀ (q : ℕ), ↑a ^ (n * (2 * q + 1)) = -1
```


Read back by a model that saw only the Lean. For all natural numbers a and n , if $1 < a$ and $0 < n$, then for every natural number q , the $(n(2q+1))$ -st power of the cast of a equals -1 .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Set Real Ideal Polynomial
open scoped BigOperators

theorem odd_multiple_period_mod_add_one {a n : ℕ} :
  ∀ q : ℕ, ((a : ZMod (a ^ n + 1)) ^ (n * (2 * q + 1))) = -1 := by
  let m : ℕ := a ^ n + 1
  change ∀ q : ℕ, ((a : ZMod m) ^ (n * (2 * q + 1))) = -1
  have hzero : ((a ^ n + 1 : ℕ) : ZMod m) = 0 := by
    simp only [m] using (ZMod.natCast_self (a ^ n + 1))
  have hsum : ((a ^ n : ℕ) : ZMod m) + 1 = 0 := by
    simp only [Nat.cast_add, Nat.cast_one] using hzero
  have hbase_cast : ((a ^ n : ℕ) : ZMod m) = -1 := by
    calc
      ((a ^ n : ℕ) : ZMod m) = (((a ^ n : ℕ) : ZMod m) + 1) - 1 := by ring
      _ = 0 - 1 := by rw [hsum]
      _ = -1 := by simp
  have hbase : ((a : ZMod m) ^ n) = -1 := by
    simp [Nat.cast_pow] using hbase_cast
  intro q
  have hodd : ((-1 : ZMod m) ^ (2 * q + 1)) = -1 := by
    calc
      (-1 : ZMod m) ^ (2 * q + 1) = ((-1 : ZMod m) ^ 2) ^ q * (-1 : ZMod m) ^ 1 := by
        rw [pow_add, pow_mul]
      _ = -1 := by simp only [pow_one]; simp
  calc
    (a : ZMod m) ^ (n * (2 * q + 1)) = ((a : ZMod m) ^ n) ^ (2 * q + 1) := by
      rw [pow_mul]
    _ = (-1 : ZMod m) ^ (2 * q + 1) := by rw [hbase]
    _ = -1 := hodd
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Herstein_exercise_2_4_36__me.mh__946d4e11

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propept, Quot.sound, fingerprint b2e35bfc0f1aaf9c

4.188 188. coprime_base_pow_sub_one

The problem

For natural numbers a and n , if $a > 1$ and $n > 0$, show that a and $a^n - 1$ are coprime.

Lean statement

```
theorem coprime_base_pow_sub_one {a n : ℕ} (ha : a > 1) (hn : 0 < n) : Nat.Coprime a (a ^ n - 1)
```

Parent. If $a > 1$ is an integer, show that $n \mid \varphi(a^n - 1)$, where ϕ is the Euler ϕ -function.

Parent in Lean

```
theorem Herstein_exercise_2.4_36 {a n : ℕ} (h : a > 1) :  
  n ∣ (a ^ n - 1).totient := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the obligation from a totient divisibility statement to proving coprimality of a with $a^n - 1$. Its proof uses a successor-coprimality argument and divisibility transfer, reasoning not supplied by the parent's cyclic-group totient proof.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes divisibility of the totient into a coprimality claim and proves it via consecutive-number coprimality plus divisibility transfer. This requires genuinely different arithmetic reasoning and cannot be obtained by recalling the parent's cyclic-group argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over proptext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1879 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem coprime_base_pow_sub_one.extracted_1_1 {a n : ℕ} (ha : a > 1) (hn : 0 < n) : a.Coprime (a ^ n - 1)
```

Read back by a model that saw only the Lean. For natural numbers a and n , if $a > 1$ and $n > 0$, then a is coprime to $a^n - 1$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Set Real Ideal Polynomial
open scoped BigOperators

theorem coprime_base_pow_sub_one {a n : ℕ} (ha : a > 1) (hn : 0 < n) : Nat.Coprime a (a ^ n - 1) := by
  have ha_pos : 0 < a := lt_trans Nat.zero_lt_one ha
  have ha_pow : a ∣ a ^ n := dvd_pow_self a (Nat.ne_of_gt hn)
  have hle : 1 ≤ a ^ n := Nat.one_le_pow n a ha_pos
  have hsum : (a ^ n - 1) + 1 = a ^ n := Nat.sub_add_cancel hle
  have hcop_pow : Nat.Coprime (a ^ n - 1) (a ^ n) := by
    have hcop_succ : Nat.Coprime (a ^ n - 1) ((a ^ n - 1) + 1) := by
      rw [Nat.coprime_self_add_right]
      exact Nat.coprime_one_right _
    rw [← hsum]
    exact hcop_succ
  have hcop_a : Nat.Coprime (a ^ n - 1) a := Nat.Coprime.of_dvd_right ha_pow hcop_pow
  exact hcop_a.symm
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Herstein_exercise_2_4_36__mh__41f3065a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint 029c75d57914be73

4.189 189. prime_order_unit_mod_pow_sub_one

The problem

Let $a > 1$, $n > 1$, and let p be a prime divisor of n . If n divides the Euler totient of $a^n - 1$, prove that the group of units modulo $a^n - 1$ contains an element of exact order p .

Lean statement

```
theorem prime_order_unit_mod_pow_sub_one {a n p : ℕ}
  (ha : a > 1) (hn : n > 1) (hp : p.Prime) (hpn : p | n)
  (hφ : n | (a ^ n - 1).totient) :
  ∃ u : (ZMod (a ^ n - 1))^*, orderOf u = p
```

Parent. If $a > 1$ is an integer, show that $n \mid \varphi(a^n - 1)$, where φ is the Euler φ -function.

Parent in Lean

```
theorem Herstein_exercise_2.4.36 {a n : ℕ} (h : a > 1) :
  n | (a ^ n - 1).totient := by
```

What it contributes. Supplies the totient-divisibility checkpoint $n \mid \varphi(a^n - 1)$, consumed via transitivity with $p \mid n$ to obtain the exact-order witness.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes divisibility of the totient into the substantive group-theoretic conclusion that every prime divisor p of n occurs as the exact order of a unit modulo $a^n - 1$. Its proof requires applying the unit-group cardinality theorem and extracting an element of prime order, which the parent's proof does not supply directly.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion to a genuine group-theoretic existence claim: from the totient divisibility it derives that every prime divisor of n divides the unit-group cardinality, then obtains an element of exact order p . This requires reasoning about units and prime orders that is absent from the parent's proof and is not the retained power-period theorem.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes divisibility of the totient into the substantive group-theoretic conclusion that every prime divisor p of n occurs as the exact order of a unit modulo $a^n - 1$. Its proof uses prime divisibility of the unit-group cardinality and a prime-order existence theorem, which the parent's argument does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2276 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem prime_order_unit_mod_pow_sub_one.extracted_1_1 {a n p : ℕ} (ha : a > 1) (hn : n > 1) (hp : Nat.Prime p)
(hpn : p ∣ n) (hφ : n ∣ (a ^ n - 1).totient) : ∃ u, orderOf u = p
```

Read back by a model that saw only the Lean. For natural numbers a , n , and p , if $a > 1$, $n > 1$, p is prime, p divides n , and n divides the Euler totient of $a^n - 1$, then there exists an element u whose order is p .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Set Real Ideal Polynomial
open scoped BigOperators

theorem prime_order_unit_mod_pow_sub_one {a n p : ℕ}
  (ha : a > 1) (hn : n > 1) (hp : p.Prime) (hpn : p ∣ n)
  (hφ : n ∣ (a ^ n - 1).totient) :
  ∃ u : (ZMod (a ^ n - 1))^*, orderOf u = p := by
  have hnpos : 0 < n := by omega
  have hpow : 1 < a ^ n := Nat.one_lt_pow (n := n) (a := a) (by omega) ha
  have hmpos : 0 < a ^ n - 1 := Nat.sub_pos_of_lt hpow
  letI : NeZero (a ^ n - 1) := ⟨Nat.ne_of_gt hmpos⟩
  have hpcard : p ∣ Fintype.card (ZMod (a ^ n - 1))^* := by
    rw [ZMod.card_units_eq_totient]
    exact Nat.dvd_trans hpn hφ
  letI : Fact p.Prime := ⟨hp⟩
  obtain ⟨u, hu⟩ := exists_prime_orderOf_dvd_card p hpcard
  exact ⟨u, hu⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Herstein_exercise_2_4_36__mh__b37ad707

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint b37ad70767ec3371

4.190 190. two_distinct_order_eleven_totient

The problem

Let $a > 1$ be an integer, and suppose that every positive integer n divides $\varphi(a^n - 1)$. If a finite group has order 462, prove that it contains two distinct elements x and y of order 11, each satisfying $11 \mid \varphi(a^{\text{ord}(\cdot)} - 1)$.

Lean statement

```
theorem two_distinct_order_eleven_totient
  {a : ℕ} (ha : a > 1)
  (hHerstein : ∀ n : ℕ, a > 1 → n ∣ (a ^ n - 1).totient)
  {G : Type*} [Fintype G] [Group G]
  (hG : Fintype.card G = 462) :
  ∃ x y : G,
    orderOf x = 11 ∧ orderOf y = 11 ∧ x ≠ y ∧
    11 ∣ (a ^ orderOf x - 1).totient ∧
    11 ∣ (a ^ orderOf y - 1).totient
```

Parent. Let $a > 1$ be an integer, and suppose that $n \mid \varphi(a^n - 1)$ for every positive integer n . If a finite group has order 462, show that it contains an element x of order 11 such that $11 \mid \varphi(a^{\text{ord}(x)} - 1)$.

Parent in Lean

```
theorem order_eleven_totient_bridge {a : ℕ} (ha : a > 1)
  (hHerstein : ∀ n : ℕ, a > 1 → n ∣ (a ^ n - 1).totient)
  {G : Type*} [Fintype G] [Group G]
  (hG : Fintype.card G = 462) :
  ∃ x : G, orderOf x = 11 ∧ 11 ∣ (a ^ orderOf x - 1).totient
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child strengthens existence of one order-11 element to two distinct such elements. Proving $x \neq x^{-1}$ requires a new group-order argument using order 11, which the parent proof does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens existence to two distinct elements of order 11. It requires the new argument that an order-11 element cannot equal its inverse, then separately transports the totient divisibility to the inverse; this is not supplied by the parent's proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine distinctness obligation: it must prove an element of order 11 cannot equal its inverse, using the resulting order divisibility contradiction. This is new mathematics beyond the parent's existence proof, and the Lean proof is complete.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing

Check	By	Result
corpus_dedup	hash	unique against 2792 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem two_distinct_order_eleven_totient.extracted_1.1 {a : ℕ} (ha : a > 1)
  (hHerstein : ∀ (n : ℕ), a > 1 → n ∣ (a ^ n - 1).totient) {G : Type u_1} [inst : Fintype G] [inst_1 : Group G]
  (hG : Fintype.card G = 462) :
  ∃ x y,
    orderOf x = 11 ∧
    orderOf y = 11 ∧ x ≠ y ∧ 11 ∣ (a ^ orderOf x - 1).totient ∧ 11 ∣ (a ^ orderOf y - 1).totient
```

Read back by a model that saw only the Lean. *Let a be a natural number with $a > 1$, and suppose that for every natural number n , if $a > 1$ then n divides the Euler totient of $a^n - 1$. Let G be a finite group with exactly 462 elements. Then there exist distinct elements x and y of G such that both x and y have order 11, and 11 divides the Euler totient of $a^{(\text{orderOf } x)} - 1$ as well as the Euler totient of $a^{(\text{orderOf } y)} - 1$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem two_distinct_order_eleven_totient
  {a : ℕ} (ha : a > 1)
  (hHerstein : ∀ n : ℕ, a > 1 → n ∣ (a ^ n - 1).totient)
  {G : Type*} [Fintype G] [Group G]
  (hG : Fintype.card G = 462) :
  ∃ x y : G,
    orderOf x = 11 ∧ orderOf y = 11 ∧ x ≠ y ∧
    11 ∣ (a ^ orderOf x - 1).totient ∧
    11 ∣ (a ^ orderOf y - 1).totient := by
  letI : Fact (Nat.Prime 11) := ⟨by norm_num⟩
  have hcard : 11 ∣ Fintype.card G := by
    rw [hG]
    norm_num
  obtain (x, hx) := exists_prime_orderOf_dvd_card 11 hcard
  have htot : 11 ∣ (a ^ 11 - 1).totient := hHerstein 11 ha
  have hy : orderOf x⁻¹ = 11 := by
    simpa using hx
  have htot_inv : 11 ∣ (a ^ orderOf x⁻¹ - 1).totient := by
    simpa [hy] using htot
  have hne : x ≠ x⁻¹ := by
    intro heq
    have hmul := congrArg (fun z : G => z * x) heq
    have hpow : x ^ 2 = 1 := by
      simpa [pow_two] using hmul
    have hd : orderOf x ∣ 2 := orderOf_dvd_of_pow_eq_one hpow
    rw [hx] at hd
    norm_num at hd
  have htot_x : 11 ∣ (a ^ orderOf x - 1).totient := by
    simpa [hx] using htot
  exact ⟨x, x⁻¹, hx, hy, hne, htot_x, htot_inv⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Herstein_exercise_2_4_36__x__Dummit_Foote_exercise_4_5_23__mh.xh.mh__9c519466

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 9c5194665bf17a96

4.191 191. mem_zpowers_conjugate_iff

The problem

Assume every subgroup of a group G is normal. Show that an element x belongs to the cyclic subgroup generated by a if and only if its conjugate by b does too.

Lean statement

```
theorem mem_zpowers_conjugate_iff {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b x : G) :
  x ∈ Subgroup.zpowers a ↔ b * x * b⁻¹ ∈ Subgroup.zpowers a
```

Parent. Let every subgroup of a group G be normal. Show that conjugating any integer power of a by b yields another element of the cyclic subgroup generated by a .

Parent in Lean

```
theorem conjugate_zpow_mem_zpowers {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :
  ∀ n : ℤ, b * a ^ n * b⁻¹ ∈ Subgroup.zpowers a
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes a one-way claim about conjugated powers into a bidirectional membership characterization for arbitrary x . Its reverse implication requires a new conjugation-by-inverse argument, which is not supplied by the parent's proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes from conjugates of specific powers a^n to an iff characterization for every element of the subgroup. Its reverse direction requires applying normality to the inverse conjugation and simplifying, which the parent proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes from conjugates of powers of a to an equivalence for every element x , requiring the new reverse-conjugation argument using b^{-1} and group simplification. The parent proof only establishes forward membership for a^n and does not supply this converse.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2316 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem mem_zpowers_conjugate_iff.extracted_1_1.{u_1} {G : Type u_1} [inst : Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b x : G) : x ∈ Subgroup.zpowers a ↔ b * x * b-1 ∈ Subgroup.zpowers a
```

Read back by a model that saw only the Lean. *Let G be a group in which every subgroup is normal. For any elements a, b , and x of G , x belongs to the subgroup of G consisting of the integer powers of a if and only if bxb^{-1} belongs to that same subgroup.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Set Real Ideal Polynomial
open scoped BigOperators

theorem mem_zpowers_conjugate_iff {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b x : G) :
  x ∈ Subgroup.zpowers a ↔ b * x * b-1 ∈ Subgroup.zpowers a := by
  have hnormal : (Subgroup.zpowers a).Normal := hG (Subgroup.zpowers a)
  constructor
  · intro hx
    exact hnormal.conj_mem x hx b
  · intro hx
    have hreverse : b-1 * (b * x * b-1) * (b-1)-1 ∈ Subgroup.zpowers a :=
      hnormal.conj_mem (b * x * b-1) hx b-1
    simpa [mul_assoc] using hreverse
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Herstein_exercise_2_5_23__me.mh__50d566d4

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, fingerprint 50d566d4c0588323

4.192 192. conjugation_bijective_on_zpowers

The problem

Show that conjugation by b restricts to a bijection of the cyclic subgroup generated by a .

Lean statement

```
theorem conjugation_bijective_on_zpowers {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :
  ∃ F : Subgroup.zpowers a → Subgroup.zpowers a,
  Function.Bijective F ∧
  ∀ x : Subgroup.zpowers a, (F x : G) = b * (x : G) * b⁻¹
```

Parent. In a group where every subgroup is normal, conjugating any integer power of an element by another element produces a power of the original element. Express this by exhibiting an integer exponent for each conjugate.

Parent in Lean

```
theorem conjugate_zpow_exists_exp {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :
  ∀ n : ℤ, ∃ k : ℤ, b * a ^ n * b⁻¹ = a ^ k
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a genuinely stronger structural result: conjugation restricts to a bijection on the subgroup of powers, requiring explicit inverse construction and injectivity/surjectivity arguments. The parent's existence-of-an-exponent result supplies at most closure, not the bijection proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from existence of an exponent for each conjugated power to a bijective self-map of the entire subgroup. Proving bijectivity requires constructing and verifying an explicit inverse, a genuinely new obligation absent from the parent's existential membership argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes existence of an exponent for each conjugate power into a bijective self-map of the entire subgroup of integer powers. Proving bijectivity requires constructing and verifying the inverse conjugation map, which is not supplied by the parent's membership-and-extraction argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2314 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	MISMATCH: Statement B adds the hypothesis that every subgroup of G is normal, while Statement A does not state this restriction; without it, conjugation by b need not preserve the cyclic subgroup generated by a .

Goal round-trip: what Lean elaborated

```
theorem conjugation_bijective_on_zpowers.extracted_1_1.{u_1} {G : Type u_1} [inst : Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :
  ∃ F, Function.Bijective F ∧ ∀ (x : ↑(Subgroup.zpowers a)), ↑(F x) = b * ↑x * b-1
```

Read back by a model that saw only the Lean. For a group G such that every subgroup of G is normal, and for any elements $a, b \in G$, there exists a bijective function F from the subgroup of G consisting of the powers of a to itself such that, for every element x of this subgroup, the underlying element of G represented by $F(x)$ equals $b \cdot x \cdot b^{-1}$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Set Real Ideal Polynomial
open scoped BigOperators

theorem conjugation_bijective_on_zpowers {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :
  ∃ F : Subgroup.zpowers a → Subgroup.zpowers a,
  Function.Bijective F ∧
  ∀ x : Subgroup.zpowers a, (F x : G) = b * (x : G) * b-1 := by
  have hnormal : (Subgroup.zpowers a).Normal := hG (Subgroup.zpowers a)
  let F : Subgroup.zpowers a → Subgroup.zpowers a := fun x =>
    (b * (x : G) * b-1, by
      exact hnormal.conj_mem (x : G) x.property b)
  let R : Subgroup.zpowers a → Subgroup.zpowers a := fun x =>
    (b-1 * (x : G) * b, by
      exact hnormal.conj_mem' (x : G) x.property b)
  have hleft : ∀ x : Subgroup.zpowers a, R (F x) = x := by
    intro x
    apply Subtype.ext
    change b-1 * (b * (x : G) * b-1) * b = (x : G)
    group
  have hright : ∀ y : Subgroup.zpowers a, F (R y) = y := by
    intro y
    apply Subtype.ext
    change b * (b-1 * (y : G) * b) * b-1 = (y : G)
    group
  refine ⟨F, ?_, ?_⟩
  · constructor
    · intro x y hxy
      have h := congrArg R hxy
      rw [hleft x, hleft y] at h
      exact h
    · intro y
      exact ⟨R y, hright y⟩
  · intro x
    rfl
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Herstein_exercise_2_5_23__me.ms.mh__6af04906

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, fingerprint 6af049069f0272af

4.193 193. unique_commutation_exponent

The problem

Let G be a group in which every subgroup is normal. Suppose that a has infinite order. Prove that for every $b \in G$, there is a unique integer j such that $ba = a^j b$.

Lean statement

```
theorem unique_commutation_exponent {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal)
  (a : G)
  (hinf : ∀ n : ℤ, a ^ n = 1 → n = 0)
  (b : G) :
  ∃! j : ℤ, b * a = a ^ j * b
```

Parent. Let G be a group such that all subgroups of G are normal in G . If $a, b \in G$, prove that $ba = a^j b$ for some j .

Parent in Lean

```
theorem Herstein_exercise_2_5_23 {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :
  ∃ (j : ℤ), b * a = a ^ j * b := by
```

What it contributes. Supplies the cyclic-subgroup normality and conjugation argument establishing the existence of an exponent.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a genuine uniqueness theorem: the infinite-order hypothesis is used to show that any two exponents satisfying the commutation equation are equal. The parent only supplies existence and does not provide this cancellation-plus-infinite-order argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine uniqueness conclusion: after obtaining existence via the parent's conjugation argument, it uses the infinite-order hypothesis on a to prove any two valid exponents are equal. This additional cancellation and integer-exponent reasoning is not supplied by the parent proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely strengthens existence to uniqueness by using the infinite-order hypothesis to derive $a^j = a^k$ and then $j=k$ via $a^{j-k}=1$. The parent's cyclic-subgroup normality argument supplies only existence and does not determine the exponent.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2197 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem unique_commutation_exponent.extracted_1_1.{u_1} {G : Type u_1} [inst : Group G]
(hG : ∀ (H : Subgroup G), H.Normal) (a : G) (hinf : ∀ (n : ℤ), a ^ n = 1 → n = 0) (b : G) :
∃! j, b * a = a ^ j * b
```

Read back by a model that saw only the Lean. *Let G be a group in which every subgroup is normal. Let a and b be elements of G , and suppose that for every integer n , if $a^n = 1$ then $n = 0$. Then there exists a unique integer j such that $b \cdot a = a^j \cdot b$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Set Real Ideal Polynomial
open scoped BigOperators

theorem unique_commutation_exponent {G : Type*} [Group G]
(hG : ∀ (H : Subgroup G), H.Normal)
(a : G)
(hinf : ∀ n : ℤ, a ^ n = 1 → n = 0)
(b : G) :
∃! j : ℤ, b * a = a ^ j * b := by
classical
have hHnormal : (Subgroup.zpowers a).Normal := hG (Subgroup.zpowers a)
have ha : a ∈ Subgroup.zpowers a := Subgroup.mem_zpowers a
have hconj : b * a * b⁻¹ ∈ Subgroup.zpowers a := by
  simpa using hHnormal.conj_mem a ha b
obtain (j, hj) := Subgroup.mem_zpowers_iff.mp (by
  simpa using hconj)
have hex : b * a = a ^ j * b := by
  calc
    b * a = (b * a) * (b⁻¹ * b) := by simp
    _ = (b * a * b⁻¹) * b := by simp [mul_assoc]
    _ = a ^ j * b := by simp [hj]
refine (j, hex, ?_)
intro k hk
have hpow : a ^ j = a ^ k := by
  exact mul_right_cancel (hex.symm.trans hk)
have hrel : a ^ (j - k) = 1 := by
  rw [zpow_sub, hpow]
  simp
have hzero : j - k = 0 := hinf (j - k) hrel
exact (sub_eq_zero.mp hzero).symm
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Herstein_exercise_2_5_23__mh__85f601c1

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, fingerprint 85f601c1761229c5

4.194 194. Herstein_exercise_2_5_23_mutation

The problem

Let G be a group in which every subgroup is normal. Show that for any $a, b \in G$, there is an integer j such that $ba^n = a^{jn}b$ for every integer n .

Lean statement

```
theorem Herstein_exercise_2_5_23_mutation {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :
  ∃ j : ℤ, ∀ n : ℤ, b * a ^ n = a ^ (j * n) * b
```

Parent. Let G be a group such that all subgroups of G are normal in G . If $a, b \in G$, prove that $ba = a^j b$ for some j .

Parent in Lean

```
theorem Herstein_exercise_2_5_23 {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :
  ∃ (j : ℤ), b * a = a ^ j * b := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child generalizes the single relation to a uniform identity for every integer power n . Proving this requires showing conjugation preserves integer powers and combining exponents, which is new reasoning beyond the parent's base-case rearrangement.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens the single conjugation identity to a uniform statement for every integer power. Proving it requires recognizing conjugation as a group homomorphism and applying preservation of integer powers, which the parent proof does not supply.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens the conclusion from one conjugation identity to a uniform identity for every integer power. Proving it requires recognizing conjugation as a group automorphism and applying that map to arbitrary integer powers, which the parent proof does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2817 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem Herstein_exercise_2_5_23_mutation.extracted_1_1.{u_1} {G : Type u_1} [inst : Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G) : ∃ j, ∀ (n : ℤ), b * a ^ n = a ^ (j * n) * b
```

Read back by a model that saw only the Lean. *Let G be a group in which every subgroup is normal. Then for any elements $a, b \in G$, there exists an integer j such that, for every integer n , $b a^n = a^{jn} b$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Set Real Ideal Polynomial
open scoped BigOperators

theorem Herstein_exercise_2_5_23_mutation {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :
  ∃ j : ℤ, ∀ n : ℤ, b * a ^ n = a ^ (j * n) * b := by
  classical
  have hnormal : (Subgroup.zpowers a).Normal := hG (Subgroup.zpowers a)
  have ha : a ∈ Subgroup.zpowers a := Subgroup.mem_zpowers a
  have hconj : b * a * b⁻¹ ∈ Subgroup.zpowers a := by
    simpa using hnormal.conj_mem a ha b
  obtain ⟨j, hj⟩ := Subgroup.mem_zpowers_iff.mp (by
    simpa using hconj)
  refine ⟨j, ?_⟩
  intro n
  have hpow : b * a ^ n * b⁻¹ = (b * a * b⁻¹) ^ n := by
    change (MulAut.conj b) (a ^ n) = ((MulAut.conj b) a) ^ n
    exact map_zpow (MulAut.conj b) a n
  calc
    b * a ^ n = (b * a ^ n * b⁻¹) * b := by simp [mul_assoc]
    _ = (b * a * b⁻¹) ^ n * b := by rw [hpow]
    _ = (a ^ j) ^ n * b := by rw [hj]
    _ = a ^ (j * n) * b := by simp [zpow_mul]
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Herstein_exercise_2_5_23__mh__ba56f709

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axiomspropext, Quot.sound, fingerprint ba56f7097045d343

4.195 195. anticommutator_commutates_with_int_polynomial

The problem

Let R be a ring with $a^2 = 0$. Show that $ax + xa$ commutes with every polynomial in a with integer coefficients.

Lean statement

```
theorem anticommutator_commutates_with_int_polynomial {R : Type*} [Ring R] (a x : R) (h : a ^ 2 = 0) (p : Polynomial ℤ) :
  (a * x + x * a) * p.sum (fun n c => (c : R) * a ^ n) =
  p.sum (fun n c => (c : R) * a ^ n) * (a * x + x * a)
```

Parent. Let R be a ring with $a^2 = 0$. Show that $ax + xa$ commutes with every polynomial in a with natural-number coefficients.

Parent in Lean

```
theorem anticommutator_commutates_with_nat_polynomial {R : Type*} [Ring R] (a x : R) (h : a ^ 2 = 0) (p : Polynomial ℕ) :
  (a * x + x * a) * p.sum (fun n c => (c : R) * a ^ n) =
  p.sum (fun n c => (c : R) * a ^ n) * (a * x + x * a)
```

What it contributes. Supplied the square-zero calculation, power-commutation induction, natural-coefficient transport, and support-sum evaluation pattern; the child extends the coefficient argument through signed integer decomposition.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes polynomial coefficients from $\mathbb{N}$ to $\mathbb{Z}$. Its proof requires a new signed-integer decomposition and additive-inverse argument to establish centrality of integer casts, which the parent proof does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely extends centrality from natural coefficients to integer coefficients, requiring a new signed-integer decomposition and additive-inverse compatibility argument. This is substantive reasoning absent from the parent and is not the listed repeat.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely extends the result from natural coefficients to integer coefficients. Its proof requires new reasoning for signed coefficients via decomposition into a difference of naturals and additive inverses, which the parent does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propxt, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2894 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem anticommutator_commutes_with_int_polynomial.extracted_1.1.{u_1} {R : Type u_1} [inst : Ring R] (a x : R)
  (h : a ^ 2 = 0) (p : ℤ[X]) :
  ((a * x + x * a) * p.sum fun n c => ↑c * a ^ n) = (p.sum fun n c => ↑c * a ^ n) * (a * x + x * a)
```

Read back by a model that saw only the Lean. Let R be a ring, let a and x be elements of R such that $a^2 = 0$, and let p be a polynomial in $\mathbb{Z}[X]$. Then $(a x + x a)$ commutes with the element of R given by the finite sum over all degrees n of the coefficient of X^n in p , coerced from $\mathbb{Z}$ to R , multiplied by a^n ; that is, $(a x + x a) \cdot \sum_n (p.\text{coeff } X^n \text{ coerced to } R) a^n = (\sum_n (p.\text{coeff } X^n \text{ coerced to } R) a^n) \cdot (a x + x a)$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Set Real Ideal Polynomial
open scoped BigOperators

theorem anticommutator_commutes_with_int_polynomial {R : Type*} [Ring R] (a x : R) (h : a ^ 2 = 0) (p : Polynomial ℤ) :
  (a * x + x * a) * p.sum (fun n c => (c : R) * a ^ n) =
    p.sum (fun n c => (c : R) * a ^ n) * (a * x + x * a) := by
  let c : R := a * x + x * a
  have h_mul_sq : a * a = 0 := by
    simpa [pow_two] using h
  have h_base : a * c = c * a := by
    dsimp [c]
    calc
      a * (a * x + x * a) = (a * a) * x + (a * x) * a := by
        rw [mul_add, ← mul_assoc, ← mul_assoc]
      _ = (a * x) * a + x * (a * a) := by simp [h_mul_sq]
      _ = (a * x + x * a) * a := by rw [add_mul, ← mul_assoc]
  have h_power : ∀ k : ℕ, c * a ^ k = a ^ k * c := by
    intro k
    induction k with
    | zero => simp
    | succ k ih =>
      calc
        c * a ^ (Nat.succ k) = c * (a ^ k * a) := by rw [pow_succ]
        _ = (c * a ^ k) * a := by rw [mul_assoc]
        _ = (a ^ k * c) * a := by rw [ih]
        _ = a ^ k * (c * a) := by rw [mul_assoc]
        _ = a ^ k * (a * c) := by rw [h_base]
        _ = (a ^ k * a) * c := by rw [← mul_assoc]
        _ = a ^ (Nat.succ k) * c := by rw [pow_succ]
  have h_nat_coeff : ∀ n : ℕ, c * (n : R) = (n : R) * c := by
    intro n
    induction n with
    | zero => simp
    | succ n ih =>
      simp [Nat.cast_succ, add_mul, mul_add, ih]
  have h_signed : ∀ z : ℤ, ∃ m n : ℕ, z = (m : ℤ) - n := by
    intro z
    cases z with
    | ofNat n => exact ⟨n, 0, by simp⟩
    | negSucc n => exact ⟨0, n + 1, by omega⟩
  have h_int_coeff : ∀ z : ℤ, c * (z : R) = (z : R) * c := by
    intro z
    rcases h_signed z with ⟨m, n, rfl⟩
    calc
      c * ((m : ℤ) - n : ℤ : R) = c * ((m : R) - (n : R)) := by norm_num
      _ = c * (m : R) - c * (n : R) := by rw [mul_sub]
      _ = (m : R) * c - (n : R) * c := by rw [h_nat_coeff m, h_nat_coeff n]
      _ = ((m : R) - (n : R)) * c := by rw [sub_mul]
      _ = ((m : ℤ) - n : ℤ : R) * c := by norm_num
  have h_coeff : ∀ n : ℕ, c * ((p.coeff n : R) * a ^ n) =
    ((p.coeff n : R) * a ^ n) * c := by
    intro n
    calc
      c * ((p.coeff n : R) * a ^ n) = (c * (p.coeff n : R)) * a ^ n := by rw [mul_assoc]
```

```

_ = ((p.coeff n : R) * c) * a ^ n := by rw [h_int_coeff]
_ = (p.coeff n : R) * (c * a ^ n) := by rw [mul_assoc]
_ = (p.coeff n : R) * (a ^ n * c) := by rw [h_power n]
_ = (p.coeff n : R) * a ^ n * c := by rw [mul_assoc]
have h_eval : c * p.sum (fun n k => (k : R) * a ^ n) =
  p.support.sum (fun n => (p.coeff n : R) * a ^ n * c) := by
  change c * p.support.sum (fun n => (p.coeff n : R) * a ^ n) = _
  rw [Finset.mul_sum]
  apply Finset.sum_congr rfl
  intro n hn
  exact h_coeff n
calc
(a * x + x * a) * p.sum (fun n k => (k : R) * a ^ n) =
  c * p.sum (fun n k => (k : R) * a ^ n) := by rfl
_ = p.support.sum (fun n => (p.coeff n : R) * a ^ n * c) := h_eval
_ = p.sum (fun n k => (k : R) * a ^ n) * c := by
  change _ = p.support.sum (fun n => (p.coeff n : R) * a ^ n) * c
  rw [Finset.sum_mul]
_ = p.sum (fun n k => (k : R) * a ^ n) * (a * x + x * a) := by rfl

```

Operator. mutation / mutation_hard **Lineage depth.** 3 **Certificate.** reproducible

Problem id. Herstein_exercise_4_2_6__mh.me.mh.mh__9b2b60d0

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint 9b2b60d0dc7fa9e9

4.196 196. anticommutator_commutes_with_ordered_product_of_power_sums

The problem

Let R be a ring with $a^2 = 0$. Show that $ax + xa$ commutes with every ordered product whose factors are finite sums of natural powers of a .

Lean statement

```
theorem anticommutator_commutes_with_ordered_product_of_power_sums {R : Type*} [Ring R] (a x : R) (h : a ^ 2 = 0) {ι κ : Type*} (t : List
  <- κ) (s : κ → Finset ι) (f : κ → ι → ℕ) :
  (a * x + x * a) * List.prod (t.map (fun k => (s k).sum (fun i => a ^ f k i))) =
    List.prod (t.map (fun k => (s k).sum (fun i => a ^ f k i))) * (a * x + x * a)
```

Parent. Let R be a ring with $a^2 = 0$. Show that $ax + xa$ commutes with every finite sum of natural powers of a .

Parent in Lean

```
theorem anticommutator_commutes_with_finite_power_sum {R : Type*} [Ring R] (a x : R) (h : a ^ 2 = 0) {ι : Type*} (s : Finset ι) (f : ι →
  <- ℕ) :
  (a * x + x * a) * s.sum (fun i => a ^ f i) = s.sum (fun i => a ^ f i) * (a * x + x * a)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes commutation from one finite power sum to an ordered product of arbitrarily many such sums. Its proof requires a new induction over the list of factors after establishing each factor commutes with the anticommutator, so the parent's proof does not suffice.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a single finite sum into an ordered product of independently indexed finite power sums. Proving commutation with the product requires a new induction over the list and a factor-commutation argument beyond the parent's sum induction.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely expands the claim from one finite sum to an ordered product of many such sums. Its proof requires a new induction showing that a commuting element commutes with the entire product, beyond the parent's finite-sum argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2907 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem anticommutator_commutates_with_ordered_product_of_power_sums.extracted_1_1.{u_3, u_2, u_1} {R : Type u_1}
  [inst : Ring R] (a x : R) (h : a ^ 2 = 0) {ι : Type u_2} {κ : Type u_3} (t : List κ) (s : κ → Finset ι)
  (f : κ → ι → ℕ) :
  (a * x + x * a) * (List.map (fun k => ∑ i ∈ s k, a ^ f k i) t).prod =
  (List.map (fun k => ∑ i ∈ s k, a ^ f k i) t).prod * (a * x + x * a)
```

Read back by a model that saw only the Lean. Let R be a ring, let a and x be elements of R satisfying $a^2 = 0$, let ι and κ be types, let t be a list of elements of κ , let s assign to each $k : \kappa$ a finite subset of ι , and let f assign to each $k : \kappa$ and $i : \iota$ a natural number. Then $(a x + x a)$ multiplied by the ordered product, in the order specified by t , of the finite sums $\sum_{i \in s(k)} a^{f(k,i)}$ equals that ordered product multiplied by $(a x + x a)$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Fintype Set Real Ideal Polynomial
open scoped BigOperators

theorem anticommutator_commutates_with_ordered_product_of_power_sums {R : Type*} [Ring R] (a x : R) (h : a ^ 2 = 0) {ι κ : Type*} (t : List
  κ) (s : κ → Finset ι) (f : κ → ι → ℕ) :
  (a * x + x * a) * List.prod (t.map (fun k => (s k).sum (fun i => a ^ f k i))) =
  List.prod (t.map (fun k => (s k).sum (fun i => a ^ f k i))) * (a * x + x * a) := by
  let c : R := a * x + x * a
  let g : κ → R := fun k => (s k).sum (fun i => a ^ f k i)
  have h_mul_sq : a * a = 0 := by
    simp [pow_two] using h
  have h_base : a * c = c * a := by
    dsimp [c]
    calc
      a * (a * x + x * a) = (a * a) * x + (a * x) * a := by
        rw [mul_add, ← mul_assoc, ← mul_assoc]
      _ = (a * x) * a + x * (a * a) := by
        simp [h_mul_sq]
      _ = (a * x + x * a) * a := by
        rw [add_mul, ← mul_assoc]
  have h_power : ∀ k : ℕ, c * a ^ k = a ^ k * c := by
    intro k
    induction k with
    | zero => simp
    | succ k ih =>
      calc
        c * a ^ (Nat.succ k) = c * (a ^ k * a) := by rw [pow_succ]
        _ = (c * a ^ k) * a := by rw [mul_assoc]
        _ = (a ^ k * c) * a := by rw [ih]
        _ = a ^ k * (c * a) := by rw [mul_assoc]
        _ = a ^ k * (a * c) := by rw [h_base]
        _ = (a ^ k * a) * c := by rw [← mul_assoc]
        _ = a ^ (Nat.succ k) * c := by rw [pow_succ]
  have h_sum : ∀ (k : κ) (u : Finset ι),
    c * u.sum (fun i => a ^ f k i) = u.sum (fun i => a ^ f k i) * c := by
    intro k u
    calc
      c * u.sum (fun i => a ^ f k i) = u.sum (fun i => c * a ^ f k i) := by
        rw [Finset.mul_sum]
      _ = u.sum (fun i => a ^ f k i * c) := by
        apply Finset.sum_congr rfl
        intro i hi
        exact h_power (f k i)
      _ = u.sum (fun i => a ^ f k i) * c := by
        rw [Finset.sum_mul]
  have h_factor : ∀ k : κ, c * g k = g k * c := by
    intro k
    dsimp [g]
    exact h_sum k (s k)
  have h_prod : ∀ l : List κ,
    c * List.prod (l.map g) = List.prod (l.map g) * c := by
```

```

intro l
induction l with
| nil => simp
| cons k l ih =>
  change c * (g k * List.prod (l.map g)) = (g k * List.prod (l.map g)) * c
  calc
  c * (g k * List.prod (l.map g)) = (c * g k) * List.prod (l.map g) := by rw [mul_assoc]
  _ = (g k * c) * List.prod (l.map g) := by rw [h_factor k]
  _ = g k * (c * List.prod (l.map g)) := by rw [mul_assoc]
  _ = g k * (List.prod (l.map g) * c) := by rw [ih]
  _ = (g k * List.prod (l.map g)) * c := by rw [← mul_assoc]
simp [c, g] using h_prod t

```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Herstein_exercise_4_2_6__mh.me.mh__95b28844

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axiomspropext, Quot.sound, fingerprint 95b2884480061167

4.197 197. nilpotent_symmetrized_square

The problem

If $a^2 = 0$ in a ring R , show that $(ax + xa)^2 = (ax)(ax) + (ax)(xa) + (xa)(xa)$.

Lean statement

```
theorem nilpotent_symmetrized_square {R : Type*} [Ring R] (a x : R) (h : a ^ 2 = 0) :
  (a * x + x * a) ^ 2 = (a * x) * (a * x) + (a * x) * (x * a) + (x * a) * (x * a)
```

Parent. If $a^2 = 0$ in R , show that $ax + xa$ commutes with a .

Parent in Lean

```
theorem Herstein_exercise_4_2_6 {R : Type*} [Ring R] (a x : R)
  (h : a ^ 2 = 0) : a * (a * x + x * a) = (a * x + x * a) * a := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from commutation of the symmetrized term with a to an explicit square expansion, requiring a new noncommutative expansion and proof that the cross term $(x*a)*(a*x)$ vanishes from $a^2 = 0$. The parent's proof does not supply this identity directly.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires a new noncommutative expansion and proves the distinct cross term $(x*a)*(a*x)$ vanishes from $a^2 = 0$. The parent only identifies two products involving $a*x + x*a$ and does not supply this square expansion or vanishing-cross-term argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the claim from commutation of the symmetrized product to a square expansion and must additionally prove the distinct cross term $(x*a)*(a*x) = 0$ from $a^2 = 0$. This reasoning is not supplied by the parent's reduction to $a*x*a$, and the proof is complete.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1982 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem nilpotent_symmetrized_square.extracted_1.1.{u_1} {R : Type u_1} [inst : Ring R] (a x : R) (h : a ^ 2 = 0) :
  (a * x + x * a) ^ 2 = a * x * (a * x) + a * x * (x * a) + x * a * (x * a)
```

Read back by a model that saw only the Lean. Let R be a ring, and let a and x be elements of R . If $a^2 = 0$, then $(a x + x a)^2 = a x (a x) + a x (x a) + x a (x a)$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Set Real Ideal Polynomial
open scoped BigOperators

theorem nilpotent_symmetrized_square {R : Type*} [Ring R] (a x : R) (h : a ^ 2 = 0) :
  (a * x + x * a) ^ 2 = (a * x) * (a * x) + (a * x) * (x * a) + (x * a) * (x * a) := by
  have hsq : a * a = 0 := by
    simpa [pow_two] using h
  have hcross : (x * a) * (a * x) = 0 := by
    calc
      (x * a) * (a * x) = x * (a * a) * x := by simp [mul_assoc]
      _ = 0 := by simp [hsq]
  have hexpand :
    (a * x + x * a) ^ 2 =
      (a * x) * (a * x) + (a * x) * (x * a) +
      (x * a) * (a * x) + (x * a) * (x * a) := by
    noncomm_ring
  calc
    (a * x + x * a) ^ 2 =
      (a * x) * (a * x) + (a * x) * (x * a) +
      (x * a) * (a * x) + (x * a) * (x * a) := hexpand
    _ = (a * x) * (a * x) + (a * x) * (x * a) + (x * a) * (x * a) := by
      rw [hcross]
    simp
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Herstein_exercise_4_2_6__mh__1651a8b5

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axiomspropext, Quot.sound, fingerprint 981aaf31b2c9ae13

4.198 198. gaussian_square_dvd_iff_even_parts

The problem

A Gaussian integer is divisible by $(1+i)^2$ exactly when both its real and imaginary parts are even.

Lean statement

```
theorem gaussian_square_dvd_iff_even_parts (b : GaussianInt) : ((1, 1) : GaussianInt) ^ 2 ∣ b ↔ Even b.re ∧ Even b.im
```

Parent. For every Gaussian integer a , show that the quotient of $2a$ by $(1+i)^2$ exists uniquely and equals $a(-i)$.

Parent in Lean

```
theorem gaussian_factorization_unique_scaled (a : GaussianInt) : ∃ q : GaussianInt, ((1, 1) : GaussianInt) ^ 2 * q = 2 * a ∧ ∀ q₁ : GaussianInt, ((1, 1) : GaussianInt) ^ 2 * q₁ = 2 * a → q₁ = a * ⟨0, -1⟩
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes an existential uniqueness result into a divisibility characterization for arbitrary Gaussian integers. Its proof requires decomposing Gaussian components and proving both parity directions, which is not supplied by the parent's factor-and-cancellation argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from a scaled factorization with uniqueness to a divisibility characterization in terms of both Gaussian components being even. Its proof requires a bidirectional coordinate argument and does not reuse the parent's cancellation-based uniqueness reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from existence and uniqueness of a scaled factor to a divisibility characterization in terms of the real and imaginary parts. Its proof requires decomposing Gaussian integers and constructing an explicit quotient from parity witnesses, reasoning not supplied by the parent's cancellation argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2784 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem gaussian_square_dvd_iff_even_parts.extracted_1_1 (b : GaussianInt) : { re := 1, im := 1 } ^ 2 ∣ b ↔ Even b.re ∧ Even b.im
```

Read back by a model that saw only the Lean. For every Gaussian integer b , the square of the Gaussian integer with real part 1 and imaginary part 1 divides b if and only if the real part of b is even and the imaginary part of b is even.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Real
open scoped BigOperators

theorem gaussian_square_dvd_iff_even_parts (b : GaussianInt) :  $\langle 1, 1 \rangle : \text{GaussianInt}$  ^ 2 | b  $\leftrightarrow$  Even b.re  $\wedge$  Even b.im := by
  have hfactor :  $\langle 1, 1 \rangle : \text{GaussianInt}$  ^ 2 = (2 : GaussianInt) *  $\langle 0, 1 \rangle$  := by
    ext <|> norm_num [pow_two]
  constructor
  · intro h
    rcases h with  $\langle q, hq \rangle$ 
    rcases q with  $\langle x, y \rangle$ 
    constructor
    · refine  $\langle -y, ?_ \rangle$ 
      rw [hq, hfactor]
      norm_num
      ring
    · refine  $\langle x, ?_ \rangle$ 
      rw [hq, hfactor]
      norm_num
      ring
  · rintro  $\langle \langle r, hr \rangle, \langle s, hs \rangle \rangle$ 
    refine  $\langle \langle s, -r \rangle, ?_ \rangle$ 
    rw [hfactor]
    ext <|> norm_num [hr, hs] <|> ring
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Ireland_Rosen_exercise_1_31__me.mh.mh__ed2761e4

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint ed2761e4731b3929

4.199 199. gaussian_factorization_unique_scaled

The problem

For every Gaussian integer a , show that the quotient of $2a$ by $(1 + i)^2$ exists uniquely and equals $a(-i)$.

Lean statement

```
theorem gaussian_factorization_unique_scaled (a : GaussianInt) : ∃ q : GaussianInt, ((1, 1) : GaussianInt) ^ 2 * q = 2 * a ∧ ∀ q₁ :  
  ↪ GaussianInt, ((1, 1) : GaussianInt) ^ 2 * q₁ = 2 * a → q₁ = a * ⟨0, -1⟩
```

Parent. Show that the quotient in the factorization of 2 by $(1 + i)^2$ exists, is unique, and equals $-i$.

Parent in Lean

```
theorem gaussian_factorization_unique : ∃ q : GaussianInt, ((1, 1) : GaussianInt) ^ 2 * q = 2 ∧ ∀ q₁ : GaussianInt, ((1, 1) :  
  ↪ GaussianInt) ^ 2 * q₁ = 2 → q₁ = ⟨0, -1⟩ := by
```

What it contributes. Supplied the factorization identity $(1+i)^2(-i)=2$, which is scaled by the arbitrary Gaussian multiplier.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The arbitrary Gaussian integer parameter changes the uniqueness target from a fixed quotient to the law $q = a \cdot (0-i)$. The parent's coordinate-wise omega argument no longer supplies this result; the child requires scaling, commutativity, and cancellation by the nonzero Gaussian factor.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The arbitrary Gaussian integer parameter changes the uniqueness argument: coordinate-wise reasoning for the fixed quotient is replaced by proving the Gaussian factor is nonzero and cancelling it. Although the fixed factorization is reused, the parent proof does not establish this scaled law.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The arbitrary Gaussian integer parameter requires proving the scaled witness equation by distributivity and establishing cancellation from the nonzero Gaussian factor. This is substantive new reasoning beyond the parent's coordinate-wise uniqueness proof, so recalling the parent does not suffice.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2762 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem gaussian_factorization_unique_scaled.extracted_1_1 (a : GaussianInt) :  
  ∃ q,  
    { re := 1, im := 1 } ^ 2 * q = 2 * a ∧  
    ∀ (q₁ : GaussianInt), { re := 1, im := 1 } ^ 2 * q₁ = 2 * a → q₁ = a * { re := 0, im := -1 }
```

Read back by a model that saw only the Lean. For every Gaussian integer a , there exists a Gaussian integer q such that $(1 + i)^2 q = 2a$, and for every Gaussian integer q_1 , if $(1 + i)^2 q_1 = 2a$, then $q_1 = a(0 - i)$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open Real  
open scoped BigOperators  
  
theorem gaussian_factorization_unique_scaled (a : GaussianInt) : ∃ q : GaussianInt, ((1, 1) : GaussianInt) ^ 2 * q = 2 * a ∧ ∀ q₁ :  
  ↪ GaussianInt, ((1, 1) : GaussianInt) ^ 2 * q₁ = 2 * a → q₁ = a * ⟨0, -1⟩ := by  
  have hfactor : ((1, 1) : GaussianInt) ^ 2 * ⟨0, -1⟩ = 2 := by  
    ext <|> simp [pow_two]  
  have hne : ((1, 1) : GaussianInt) ^ 2 ≠ 0 := by  
    intro hz  
    have hzero : (0 : GaussianInt) = 2 := by  
      calc  
        (0 : GaussianInt) = ((1, 1) : GaussianInt) ^ 2 * ⟨0, -1⟩ := by rw [hz]; simp  
        _ = 2 := hfactor  
    norm_num at hzero  
  have hscaled : ((1, 1) : GaussianInt) ^ 2 * (a * ⟨0, -1⟩) = 2 * a := by  
    calc  
      ((1, 1) : GaussianInt) ^ 2 * (a * ⟨0, -1⟩) = (((1, 1) : GaussianInt) ^ 2 * ⟨0, -1⟩) * a := by ring  
      _ = 2 * a := by rw [hfactor]  
  refine ⟨a * ⟨0, -1⟩, hscaled, ?_⟩  
  intro q₁ hq  
  apply mul_left_cancel₀ hne  
  exact hq.trans hscaled.symm
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Ireland_Rosen_exercise_1_31__me.mh__7d67637c

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 7d67637cd579935f

4.200 200. gaussianInt_sq_one_add_i_dvd_imp_two_dvd

The problem

Show that every Gaussian integer divisible by $(1 + i)^2$ is divisible by 2.

Lean statement

```
theorem gaussianInt_sq_one_add_i_dvd_imp_two_dvd : ∀ z : GaussianInt, ((1, 1) : GaussianInt) ^ 2 | z → (2 : GaussianInt) | z
```

Parent. Show that 2 is divisible by $(1 + i)^2$ in $\mathbb{Z}[i]$.

Parent in Lean

```
theorem IrelandRosen_exercise_1_31 : ((1, 1) : GaussianInt) ^ 2 | 2 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a genuine universal divisibility implication: every multiple of $(1+i)^2$ is shown to be a multiple of 2. This requires factoring $(1+i)^2$ as $2 \cdot i$ and composing arbitrary witnesses, beyond the parent's single witness for divisibility by 2.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely universal divisibility-transfer theorem: from $(1+i)^2 \mid z$ it factors $(1+i)^2$ as $2i$ and constructs a quotient witnessing $2 \mid z$. The parent only establishes divisibility for the single element 2 and supplies no argument for arbitrary z .

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes the fixed divisibility witness to an arbitrary Gaussian integer and proves that divisibility by $(1+i)^2$ implies divisibility by 2 via an explicit factorization and distributive rearrangement. The parent's single numerical witness does not supply this universal implication.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2000 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem gaussianInt_sq_one_add_i_dvd_imp_two_dvd.extracted_1_1 : ∀ (z : GaussianInt),
  { re := 1, im := 1 } ^ 2 | z → 2 | z
```

Read back by a model that saw only the Lean. For every Gaussian integer z , if $(1 + i)^2$ divides z , then 2 divides z .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Real
open scoped BigOperators

theorem gaussianInt_sq_one_add_i_dvd_imp_two_dvd :  $\forall z : \text{GaussianInt}, (\langle 1, 1 \rangle : \text{GaussianInt})^2 \mid z \rightarrow (2 : \text{GaussianInt}) \mid z$  := by
  have hfactor :  $(\langle 1, 1 \rangle : \text{GaussianInt})^2 = (2 : \text{GaussianInt}) * \langle 0, 1 \rangle$  := by
    ext <|> simp [pow_two]
  intro z hz
  rcases hz with ⟨q, hq⟩
  refine ⟨⟨0, 1⟩ : GaussianInt⟩ * q, ?_
  calc
    z =  $(\langle 1, 1 \rangle : \text{GaussianInt})^2 * q$  := hq
    _ =  $(2 : \text{GaussianInt}) * \langle 0, 1 \rangle * q$  := by rw [hfactor]
    _ =  $(2 : \text{GaussianInt}) * (\langle 0, 1 \rangle * q)$  := by ring
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Ireland_Rosen_exercise_1_31__mh__13b3d12d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 131d0641f6190be8

4.201 201. gaussianInt_sq_div_two_unique

The problem

Show that there is a unique Gaussian integer q such that $(1 + i)^2 q = 2$.

Lean statement

```
theorem gaussianInt_sq_div_two_unique : ∃! q : GaussianInt, ((1, 1) : GaussianInt) ^ 2 * q = 2
```

Parent. Show that 2 is divisible by $(1 + i)^2$ in $\mathbb{Z}[i]$.

Parent in Lean

```
theorem IrelandRosen_exercise_1_31 : ((1, 1) : GaussianInt) ^ 2 ∣ 2 := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes divisibility into an existence-and-uniqueness statement. Beyond reusing the parent's quotient for existence, it must prove the squared Gaussian integer is nonzero and use cancellation to establish uniqueness.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes divisibility into an existence-and-uniqueness claim. Although the parent proof supplies the witness, the child additionally proves the squared Gaussian integer is nonzero and uses cancellation to establish uniqueness, which the parent does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes existence of a quotient into a genuine uniqueness theorem. Proving uniqueness requires establishing that $(1+i)^2$ is nonzero and applying cancellation, which is reasoning absent from the parent's coordinate computation.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2010 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem gaussianInt_sq_div_two_unique.extracted_1_1 : ∃! q, { re := 1, im := 1 } ^ 2 * q = 2
```

Read back by a model that saw only the Lean. *There exists a unique Gaussian integer q such that $(1 + i)^2 q = 2$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Real
open scoped BigOperators

theorem gaussianInt_sq_div_two_unique :  $\exists! q : \text{GaussianInt}, (\langle 1, 1 \rangle : \text{GaussianInt})^2 * q = 2$  := by
  let q : GaussianInt :=  $\langle 0, -1 \rangle$ 
  have h_parent :  $(\langle 1, 1 \rangle : \text{GaussianInt})^2 * q = 2$  := by
    dsimp [q]
    ext <=> simp [pow_two]
  have h_nonzero :  $(\langle 1, 1 \rangle : \text{GaussianInt})^2 \neq (0 : \text{GaussianInt})$  := by
    intro h
    norm_num [pow_two] at h
    have hr := congrArg (fun x : GaussianInt => x.re) h
    norm_num at hr
  refine ⟨q, h_parent, ?_⟩
  intro y hy
  have h_eq :  $(\langle 1, 1 \rangle : \text{GaussianInt})^2 * y = (\langle 1, 1 \rangle : \text{GaussianInt})^2 * q$  := by
    calc
       $(\langle 1, 1 \rangle : \text{GaussianInt})^2 * y = 2$  := hy
      _ =  $(\langle 1, 1 \rangle : \text{GaussianInt})^2 * q$  := h_parent.symm
  exact mul_left_cancel₀ h_nonzero h_eq
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Ireland_Rosen_exercise_1_31__mh__d5878a1f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 8b5ca32c4cbd8a0d

4.202 202. bounded_family_union_eq_union_interiors

The problem

Let X be a topological space, let $n \in \mathbb{N}$, and let $(A_i)_{i < n}$ be a family of subsets of X . Suppose that for every $i < n$ and every $x \in A_i$, there is an open set U with $x \in U \subseteq A_i$. Prove that $\bigcup_{i < n} \text{int}(A_i) = \bigcup_{i < n} A_i$.

Lean statement

```
theorem bounded_family_union_eq_union_interiors (X : Type*) [TopologicalSpace X] (n : ℕ)
  (A : Fin n → Set X)
  (h : ∀ i x, x ∈ A i → ∃ U : Set X, x ∈ U ∧ IsOpen U ∧ U ⊆ A i) :
  (⋃ i : Fin n, interior (A i)) = ⋃ i : Fin n, A i
```

Parent. A bounded indexed family of locally open sets has an open union.

Parent in Lean

```
theorem bounded_family_union_isOpen (X : Type*) [TopologicalSpace X] (n : ℕ)
  (A : Fin n → Set X)
  (h : ∀ i x, x ∈ A i → ∃ U : Set X, x ∈ U ∧ IsOpen U ∧ U ⊆ A i) :
  IsOpen (⋃ i : Fin n, A i)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the obligation from proving openness of the union to characterizing it as a union of interiors. Its reverse inclusion requires applying the local neighborhood hypothesis to each member and using interior maximality, which the parent's proof does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the obligation from proving openness to characterizing the union as the union of interiors. Its reverse inclusion requires the new interior-maximality argument, which is not supplied by the parent's proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from openness to an equality characterizing the union via interiors. Its reverse inclusion requires proving each locally open neighborhood lies inside interior (using `interior_maximal`), reasoning not supplied by the parent's union-openness proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1970 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_family_union_eq_union_interiors.extracted_1_1.{u_1} (X : Type u_1) [inst : TopologicalSpace X] (n : ℕ)
  (A : Fin n → Set X) (h : ∀ (i : Fin n), ∀ x ∈ A i, ∃ U, x ∈ U ∧ IsOpen U ∧ U ⊆ A i) :
  U i, interior (A i) = U i, A i
```

Read back by a model that saw only the Lean. *Let X be a topological space, let n be a natural number, and let A be a family of subsets of X indexed by the finite type $\text{Fin } n$. Suppose that for every index $i \in \text{Fin } n$ and every point $x \in A i$, there exists a set U such that $x \in U$, U is open in X , and $U \subseteq A i$. Then the union over all $i \in \text{Fin } n$ of the interiors of $A i$ is equal to the union over all $i \in \text{Fin } n$ of $A i$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem bounded_family_union_eq_union_interiors (X : Type*) [TopologicalSpace X] (n : ℕ)
  (A : Fin n → Set X)
  (h : ∀ i x, x ∈ A i → ∃ U : Set X, x ∈ U ∧ IsOpen U ∧ U ⊆ A i) :
  (U i : Fin n, interior (A i)) = U i : Fin n, A i := by
  ext x
  constructor
  · intro hx
    rcases Set.mem_iUnion.mp hx with ⟨i, hxi⟩
    exact Set.mem_iUnion.mpr ⟨i, interior_subset hxi⟩
  · intro hx
    rcases Set.mem_iUnion.mp hx with ⟨i, hxi⟩
    rcases h i x hxi with ⟨U, hxU, hUopen, hUsub⟩
    have hUint : U ⊆ interior (A i) := interior_maximal hUsub hUopen
    have hxinterior : x ∈ interior (A i) := hUint hxU
    exact Set.mem_iUnion.mpr ⟨i, hxinterior⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Munkres_exercise_13_1__me.mh__0e47166e

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint d44f23eb7bec6433

4.203 203. local_open_neighborhoods_eq_sUnion

The problem

Let X be a topological space and let A be a subset of X . Suppose that for each $x \in A$ there is an open set U containing x such that $U \subset A$. Show that A is the union of all open subsets of X contained in A .

Lean statement

```
theorem local_open_neighborhoods_eq_sUnion (X : Type*) [TopologicalSpace X] (A : Set X)
  (h1 : ∀ x ∈ A, ∃ U : Set X, x ∈ U ∧ IsOpen U ∧ U ⊆ A) :
  A = ⋃₀ {U : Set X | IsOpen U ∧ U ⊆ A}
```

Parent. Let X be a topological space; let A be a subset of X . Suppose that for each $x \in A$ there is an open set U containing x such that $U \subset A$. Show that A is open in X .

Parent in Lean

```
theorem Munkres_exercise_13_1 (X : Type*) [TopologicalSpace X] (A : Set X)
  (h1 : ∀ x ∈ A, ∃ U : Set X, x ∈ U ∧ IsOpen U ∧ U ⊆ A) :
  IsOpen A := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from openness to an exact characterization of A as the union of all open subsets contained in A . Its proof requires proving both inclusions and does not merely replay the parent's openness criterion.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from openness to an equality characterizing A as the union of all open subsets contained in A . Its proof requires a new two-sided set-inclusion argument using `sUnion` membership, rather than applying the parent's openness criterion.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the openness conclusion into an extensional characterization of A as the union of all open subsets contained in it. Its proof requires new `sUnion` membership and subset arguments in both directions, which are not supplied by the parent's criterion proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over proptext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1974 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem local_open_neighborhoods_eq_sUnion.extracted_1_1.{u_1} (X : Type u_1) [inst : TopologicalSpace X] (A : Set X)
(h1 : ∀ x ∈ A, ∃ U, x ∈ U ∧ IsOpen U ∧ U ⊆ A) : A = ⋃₀ {U | IsOpen U ∧ U ⊆ A}
```

Read back by a model that saw only the Lean. *Let X be a topological space and let A be a subset of X . Assume that for every $x \in A$, there exists a set U such that $x \in U$, U is open, and $U \subseteq A$. Then A equals the union of all open subsets U of X satisfying $U \subseteq A$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem local_open_neighborhoods_eq_sUnion (X : Type*) [TopologicalSpace X] (A : Set X)
(h1 : ∀ x ∈ A, ∃ U : Set X, x ∈ U ∧ IsOpen U ∧ U ⊆ A) :
A = ⋃₀ {U : Set X | IsOpen U ∧ U ⊆ A} := by
  apply Set.Subset.antisymm
  · intro x hx
    rcases h1 x hx with ⟨U, hxU, hU, hUA⟩
    exact Set.mem_sUnion.2 ⟨U, ⟨hU, hUA⟩, hxU⟩
  · intro x hx
    rcases Set.mem_sUnion.1 hx with ⟨U, hU, hxU⟩
    exact hU.2 hxU
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_13_1__mh__4cf2fc25

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint 198b5089cea48d8c

4.204 204. Munkres_exercise_13_1_mutation

The problem

Let X be a topological space and $A \subseteq X$. Suppose that every point of A lies in an open subset of A . Show that A is the union of a family of open subsets, each containing the point of A indexing it.

Lean statement

```
theorem Munkres_exercise_13_1_mutation (X : Type*) [TopologicalSpace X] (A : Set X)
  (h1 : ∀ x ∈ A, ∃ U : Set X, x ∈ U ∧ IsOpen U ∧ U ⊆ A) :
  ∃ U : A → Set X,
    (∀ x : A, IsOpen (U x) ∧ (x : X) ∈ U x ∧ U x ⊆ A) ∧
    (U x : A, U x) = A
```

Parent. Let X be a topological space; let A be a subset of X . Suppose that for each $x \in A$ there is an open set U containing x such that $U \subset A$. Show that A is open in X .

Parent in Lean

```
theorem Munkres_exercise_13_1 (X : Type*) [TopologicalSpace X] (A : Set X)
  (h1 : ∀ x ∈ A, ∃ U : Set X, x ∈ U ∧ IsOpen U ∧ U ⊆ A) :
  IsOpen A := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a genuine constructive covering theorem: it must choose an open neighborhood for every subtype point and prove that their indexed union is exactly A . The parent's openness criterion does not supply the choice function or the two-sided union equality.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens the openness criterion by constructing a point-indexed family of open neighborhoods and proving their union is exactly A . This requires choice over the subtype and a separate set-extensionality argument for both inclusions, which the parent's criterion application does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child makes a genuine structure expansion: it must choose a neighborhood for every subtype point and prove that their union is exactly A . The parent only establishes openness via the local criterion and does not supply the choice-function or union-equality argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2875 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem Munkres_exercise_13_1_mutation.extracted_1_1.{u_1} (X : Type u_1) [inst : TopologicalSpace X] (A : Set X)
  (h1 : ∀ x ∈ A, ∃ U, x ∈ U ∧ IsOpen U ∧ U ⊆ A) :
  ∃ U, (∀ (x : ↑A), IsOpen (U x) ∧ ↑x ∈ U x ∧ U x ⊆ A) ∧ U x, U x = A
```

Read back by a model that saw only the Lean. Let X be a topological space and let A be a subset of X . Suppose that for every $x \in A$, there exists an open set V such that $x \in V$ and $V \subseteq A$. Then there exists a function U assigning to each point x of the subtype A an open subset $U(x)$ of X such that $x \in U(x)$, $U(x) \subseteq A$, and the union of all the sets $U(x)$, as x ranges over A , is exactly A .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem Munkres_exercise_13_1_mutation (X : Type*) [TopologicalSpace X] (A : Set X)
  (h1 : ∀ x ∈ A, ∃ U : Set X, x ∈ U ∧ IsOpen U ∧ U ⊆ A) :
  ∃ U : A → Set X,
    (∀ x : A, IsOpen (U x) ∧ (x : X) ∈ U x ∧ U x ⊆ A) ∧
    (U x : A, U x) = A := by
  classical
  choose U hmem hopen hsub using fun x : A => h1 x.1 x.2
  refine ⟨U, ?_, ?_⟩
  · intro x
    exact ⟨hopen x, hmem x, hsub x⟩
  · ext y
    constructor
    · intro hy
      rcases Set.mem_iUnion.mp hy with ⟨x, hx⟩
      exact hsub x hx
    · intro hy
      exact Set.mem_iUnion.mpr ⟨⟨y, hy⟩, hmem ⟨y, hy⟩⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_13_1__mh__bde9eb06

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fin-
gerprint bde9eb067942729d

4.205 205. iInf_topology_eq_of_surjective

The problem

For any family of topologies on a space and any surjective reindexing map, the universal topology below every topology in the family equals the universal topology below every reindexed topology.

Lean statement

```
theorem iInf_topology_eq_of_surjective (X I J : Type*) (T : I → TopologicalSpace X) (f : J → I) (hf : Function.Surjective f) :
  (⊓ i, T i) = ⊓ j, T (f j)
```

Parent. Let $\{\mathcal{T}_i\}_{i \in I}$ be a family of topologies on X , and let $f : J \rightarrow I$. Show that every topology on X contained in each $\mathcal{T}_i$ is contained in the largest topology contained in all the $\mathcal{T}_i$, and that this largest topology is contained in the largest topology contained in all the $\mathcal{T}_{f(j)}$.

Parent in Lean

```
theorem reindexed_iInf_topology_comparison (X I J : Type*) (T : I → TopologicalSpace X) (f : J → I) :
  (∀ U : TopologicalSpace X, (∀ i, U ≤ T i) → U ≤ ⊓ i, T i) ∧
  (⊓ i, T i) ≤ ⊓ j, T (f j) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child strengthens the comparison to an equality under surjectivity, requiring a genuinely new reverse inequality by selecting a preimage for each index. The parent supplies only the forward comparison and does not provide this surjectivity-based argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens the comparison to equality under surjectivity and proves a genuinely new reverse inequality by selecting a preimage for each index via hf . The parent's proof supplies only the forward inequality and cannot establish this direction.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens the parent's one-sided comparison to equality under surjectivity, requiring a genuinely new reverse inequality and an index-selection argument via hf . This cannot be obtained by recalling the parent proof, which supplies only the forward direction.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1874 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem iInf_topology_eq_of_surjective.extracted_1_1.{u_3, u_2, u_1} (X : Type u_1) (I : Type u_2) (J : Type u_3)
  (T : I → TopologicalSpace X) (f : J → I) (hf : Function.Surjective f) :  $\prod i, T\ i = \prod j, T\ (f\ j)$ 
```

Read back by a model that saw only the Lean. *Let X be a type, let I and J be types, let T assign a topological space on X to each element of I , and let $f : J \rightarrow I$ be surjective. Then the infimum of the topological spaces $T\ i$ over all $i \in I$ equals the infimum of the topological spaces $T\ (f\ j)$ over all $j \in J$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem iInf_topology_eq_of_surjective (X I J : Type*) (T : I → TopologicalSpace X) (f : J → I) (hf : Function.Surjective f) :
  ( $\prod i, T\ i$ ) =  $\prod j, T\ (f\ j)$  := by
  have h_parent_direction : ( $\prod i, T\ i$ ) ≤  $\prod j, T\ (f\ j)$  := by
    refine le_iInf ?_
    intro j
    exact iInf_le T (f j)
  have h_selected_lower :  $\forall j : J, (\prod j, T\ (f\ j)) \leq T\ (f\ j)$  := by
    intro j
    exact iInf_le (fun j : J => T (f j)) j
  have h_reverse_direction : ( $\prod j, T\ (f\ j)$ ) ≤  $\prod i, T\ i$  := by
    refine le_iInf ?_
    intro i
    obtain ⟨j, rfl⟩ := hf i
    exact h_selected_lower j
  exact le_antisymm h_parent_direction h_reverse_direction
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Munkres_exercise_13_4b2__me.mh__630e48d1

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 9f20fa4ac1aba7ea

4.206 206. smallest_topology_containing_family

The problem

Let $\{\mathcal{T}_\alpha\}$ be a family of topologies on X . Show that there is a unique smallest topology on X containing every $\mathcal{T}_\alpha$.

Lean statement

```
theorem smallest_topology_containing_family (X I : Type*) (τ : I → TopologicalSpace X) :
  ∃! τ' : TopologicalSpace X, (∀ i, τ i ≤ τ') ∧
  ∀ σ : TopologicalSpace X, (∀ i, τ i ≤ σ) → τ' ≤ σ
```

Parent. Let $\mathcal{T}_\alpha$ be a family of topologies on X . Show that there is a unique largest topology on X contained in all the collections $\mathcal{T}_\alpha$.

Parent in Lean

```
theorem Munkres_exercise_13_4b2 (X I : Type*) (T : I → Set (Set X)) (h : ∀ i, is_topology X (T i)) :
  ∃! T', is_topology X T' ∧ (∀ i, T' ⊆ T i) ∧
  ∀ T'', is_topology X T'' → (∀ i, T'' ⊆ T i) → T'' ⊆ T' := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the obligation from an intersection/ greatest lower topology to the least upper topology containing the family. Its proof uses the complete-lattice supremum and a new antisymmetry uniqueness argument, none of which is supplied by the parent's intersection construction.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the construction from the topology of sets common to every family member (an infimum) to the smallest topology containing them (a supremum). Its proof requires lattice supremum and uniqueness reasoning absent from the parent's intersection-based argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes from the greatest topology contained in every member of a family to the least topology containing every member. Its proof requires the complete-lattice supremum universal property, which is absent from the parent's elementwise intersection/topology-axiom argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2226 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem smallest_topology_containing_family.extracted_1_1.{u_2, u_1} (X : Type u_1) (I : Type u_2)
  (τ : I → TopologicalSpace X) :
  ∃! τ', (∀ (i : I), τ i ≤ τ') ∧ ∀ (σ : TopologicalSpace X), (∀ (i : I), τ i ≤ σ) → τ' ≤ σ
```

Read back by a model that saw only the Lean. *For every type X , every type I , and every family of topological spaces τ indexed by I on X , there exists a unique topological space structure τ' on X such that, for every $i \in I$, $\tau i \leq \tau'$, and for every topological space σ on X , if $\tau i \leq \sigma$ for every $i \in I$, then $\tau' \leq \sigma$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem smallest_topology_containing_family (X I : Type*) (τ : I → TopologicalSpace X) :
  ∃! τ' : TopologicalSpace X, (∀ i, τ i ≤ τ') ∧
    ∀ σ : TopologicalSpace X, (∀ i, τ i ≤ σ) → τ' ≤ σ := by
  have h_upper : ∀ i, τ i ≤ ⋃ j, τ j := by
    intro i
    exact le_iSup (fun j => τ j) i
  have h_least : ∀ σ : TopologicalSpace X, (∀ i, τ i ≤ σ) → (⋃ i, τ i) ≤ σ := by
    intro σ hσ
    exact iSup_le hσ
  refine ⟨⋃ i, τ i, ⟨h_upper, h_least⟩, ?_⟩
  intro σ hσ
  apply le_antisymm
  · exact hσ.2 _ h_upper
  · exact h_least σ hσ.1
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_13_4b2__mh__c587ebb9

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c587ebb98b02f88d

4.207 207. isOpen_prod_iff_of_nonempty

The problem

Let U and V be nonempty subsets of topological spaces X and Y . Show that $U \times V$ is open in $X \times Y$ if and only if U and V are both open.

Lean statement

```
theorem isOpen_prod_iff_of_nonempty {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (U : Set X) (V : Set Y) (hU : U.Nonempty) (hV : V.Nonempty) :
  IsOpen (U ×s V) ↔ IsOpen U ∧ IsOpen V
```

Parent. Let X and Y be nonempty topological spaces. Show that the two coordinate projections $X \times Y \rightarrow X$ and $X \times Y \rightarrow Y$ are quotient maps.

Parent in Lean

```
theorem product_projections_are_quotient_maps {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [Nonempty X] [Nonempty Y]
  (π₁ : X × Y → X) (π₂ : X × Y → Y)
  (h₁ : π₁ = Prod.fst) (h₂ : π₂ = Prod.snd) :
  Topology.IsQuotientMap π₁ ∧ Topology.IsQuotientMap π₂
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from quotient-map properties of projections to an iff characterization of openness for nonempty product sets. Its reverse implication is standard, but the forward direction requires new image computations and uses nonemptiness essentially, so the parent's proof does not supply the child's argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from quotient-map properties of projections to an equivalence characterizing openness of a product set. Its reverse direction and image computations using nonempty fibers require mathematics not supplied by the parent's quotient-map proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from quotient-map existence to a characterization of openness of product sets. Its proof requires constructing projection images, using nonemptiness to identify those images with U and V , and proving both directions; the parent's quotient-map proof does not supply this argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2818 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem isOpen_prod_iff_of_nonempty.extracted_1_1.{u_2, u_1} {X : Type u_1} {Y : Type u_2} [inst : TopologicalSpace X]
[inst_1 : TopologicalSpace Y] (U : Set X) (V : Set Y) (hU : U.Nonempty) (hV : V.Nonempty) :
  IsOpen (U ×s V) ↔ IsOpen U ∧ IsOpen V
```

Read back by a model that saw only the Lean. *For topological spaces X and Y , and subsets U of X and V of Y , if U and V are both nonempty, then the product set $U \times V$ is open in the product topology if and only if U is open in X and V is open in Y .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace
open scoped Topology

theorem isOpen_prod_iff_of_nonempty {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
(U : Set X) (V : Set Y) (hU : U.Nonempty) (hV : V.Nonempty) :
  IsOpen (U ×s V) ↔ IsOpen U ∧ IsOpen V := by
  constructor
  · intro hprod
    have hopen1 : IsOpenMap (Prod.fst : X × Y → X) := isOpenMap_fst
    have hopen2 : IsOpenMap (Prod.snd : X × Y → Y) := isOpenMap_snd
    have hUimg : IsOpen (Prod.fst '' (U ×s V)) := hopen1 _ hprod
    have hVimg : IsOpen (Prod.snd '' (U ×s V)) := hopen2 _ hprod
    have hfst_image : Prod.fst '' (U ×s V) = U := by
      ext x
      constructor
      · rintro ⟨p, hp, rfl⟩
        exact hp.1
      · intro hx
        obtain ⟨y, hy⟩ := hV
        exact ⟨(x, y), ⟨hx, hy⟩, rfl⟩
    have hsnd_image : Prod.snd '' (U ×s V) = V := by
      ext y
      constructor
      · rintro ⟨p, hp, rfl⟩
        exact hp.2
      · intro hy
        obtain ⟨x, hx⟩ := hU
        exact ⟨(x, y), ⟨hx, hy⟩, rfl⟩
    have hUopen : IsOpen U := by
      simpa [hfst_image] using hUimg
    have hVopen : IsOpen V := by
      simpa [hsnd_image] using hVimg
    exact ⟨hUopen, hVopen⟩
  · rintro ⟨hUopen, hVopen⟩
    exact hUopen.prod hVopen
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Munkres_exercise_16_4__me.mh__041f8e45

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 041f8e451da480db

4.208 208. continuous_iff_continuous_comp_fst

The problem

Let X , Y , and Z be topological spaces with Y nonempty. A map $g : X \rightarrow Z$ is continuous if and only if $(x, y) \mapsto g(x)$ is continuous on $X \times Y$.

Lean statement

```
theorem continuous_iff_continuous_comp_fst {X Y Z : Type*} [TopologicalSpace X] [TopologicalSpace Y] [TopologicalSpace Z] [Nonempty Y]
  (g : X → Z) :
  Continuous g ↔ Continuous (fun p : X × Y => g p.1)
```

Parent. Let X and Y be nonempty topological spaces. Show that the two coordinate projections $X \times Y \rightarrow X$ and $X \times Y \rightarrow Y$ are quotient maps.

Parent in Lean

```
theorem product_projections_are_quotient_maps {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [Nonempty X] [Nonempty Y]
  (π₁ : X × Y → X) (π₂ : X × Y → Y)
  (h₁ : π₁ = Prod.fst) (h₂ : π₂ = Prod.snd) :
  Topology.IsQuotientMap π₁ ∧ Topology.IsQuotientMap π₂
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from establishing quotient maps to an equivalence of continuity, requiring a new converse argument via the coinducing/quotient-map property of the first projection. The parent's construction of quotient maps does not itself supply this continuity characterization.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from establishing quotient maps to an equivalence of continuity under composition with the first projection. Its reverse direction requires a coinducing/quotient-map argument and continuity characterization not supplied by the parent's proof, so recalling the parent does not substantially solve it.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from proving quotient-map properties to a continuity reflection equivalence. Its reverse direction requires using the quotient/coinducing property of the first projection to recover continuity of g , which is not supplied by merely recalling the parent's conjunction.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2829 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem continuous_iff_continuous_comp_fst.extracted_1.1.{u_3, u_2, u_1} {X : Type u_1} {Y : Type u_2} {Z : Type u_3}
  [inst : TopologicalSpace X] [inst_1 : TopologicalSpace Y] [inst_2 : TopologicalSpace Z] [inst_3 : Nonempty Y]
  (g : X → Z) : Continuous g ↔ Continuous fun p => g p.1
```

Read back by a model that saw only the Lean. Let X , Y , and Z be topological spaces, with Y nonempty, and let $g : X \rightarrow Z$. Then g is continuous if and only if the function from $X \times Y$ to Z defined by $(x, y) \mapsto g(x)$ is continuous.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace
open scoped Topology

theorem continuous_iff_continuous_comp_fst {X Y Z : Type*} [TopologicalSpace X] [TopologicalSpace Y] [TopologicalSpace Z] [Nonempty Y]
  (g : X → Z) :
  Continuous g ↔ Continuous (fun p : X × Y => g p.1) := by
  constructor
  · intro hg
    simpa [Function.comp] using hg.comp (continuous_fst : Continuous (Prod.fst : X × Y → X))
  · intro hg
    have hq : Topology.IsQuotientMap (Prod.fst : X × Y → X) := by
      exact isQuotientMap_fst
    have hcoind : Topology.IsCoinducing (Prod.fst : X × Y → X) := by
      exact (Topology.isQuotientMap_iff (Prod.fst : X × Y → X)).mp hq |>.1
    have hiff : ∀ s : Set X, IsOpen ((Prod.fst : X × Y → X) ⁻¹' s) ↔ IsOpen s := by
      exact Topology.isCoinducing_iff.mp hcoind
    rw [continuous_def]
    intro s hs
    apply (hiff (g ⁻¹' s)).mp
    have hp : IsOpen ((fun p : X × Y => g p.1) ⁻¹' s) :=
      (continuous_def.mp hg) s hs
    simpa using hp
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Munkres_exercise_16_4__me.mh__eb4b00a0

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint eb4b00a0491487ec

4.209 209. isOpenMap_prodMap_general

The problem

Let $f : X \rightarrow Y$ and $g : Z \rightarrow W$ be open maps between topological spaces. Show that the map $X \times Z \rightarrow Y \times W$ given by $(x, z) \mapsto (f(x), g(z))$ is open.

Lean statement

```
theorem isOpenMap_prodMap_general {X Y Z W : Type*} [TopologicalSpace X] [TopologicalSpace Y] [TopologicalSpace Z] [TopologicalSpace W]
  (f : X → Y) (g : Z → W) (hf : IsOpenMap f) (hg : IsOpenMap g) : IsOpenMap (Prod.map f g)
```

Parent. A map $f : X \rightarrow Y$ is said to be an open map if for every open set U of X , the set $f(U)$ is open in Y . Show that $\pi_1 : X \times Y \rightarrow X$ and $\pi_2 : X \times Y \rightarrow Y$ are open maps.

Parent in Lean

```
theorem Munkres_exercise_16_4 {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (π₁ : X × Y → X)
  (π₂ : X × Y → Y)
  (h₁ : π₁ = Prod.fst)
  (h₂ : π₂ = Prod.snd) :
  IsOpenMap π₁ ∧ IsOpenMap π₂ := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child generalizes from the two coordinate projections to arbitrary open maps and proves openness of their product map, requiring the library composition theorem ‘hf.prodMap hg’. The parent’s projection-specific conjunction proof does not supply this generalized argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the special-case projection result with a genuinely general theorem: arbitrary open maps f and g induce an open product map. Its proof requires the library product-map preservation lemma and cannot be obtained from the parent’s projection rewrites alone.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes from the two canonical projections to arbitrary open maps and requires the product-map openness lemma, which the parent’s direct projection proof does not supply. Its proof obligation is not derivable by merely recalling or applying the parent’s conclusion.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1969 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem isOpenMap_prodMap_general.extracted_1_1.{u_4, u_3, u_2, u_1} {X : Type u_1} {Y : Type u_2} {Z : Type u_3}
  {W : Type u_4} [inst : TopologicalSpace X] [inst_1 : TopologicalSpace Y] [inst_2 : TopologicalSpace Z]
  [inst_3 : TopologicalSpace W] (f : X → Y) (g : Z → W) (hf : IsOpenMap f) (hg : IsOpenMap g) :
  IsOpenMap (Prod.map f g)
```

Read back by a model that saw only the Lean. *Let X, Y, Z , and W be topological spaces. If $f : X \rightarrow Y$ and $g : Z \rightarrow W$ are open maps, then the product map $\text{Prod.map } f \, g : X \times Z \rightarrow Y \times W$, defined by $(x, z) \mapsto (f(x), g(z))$, is an open map.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem isOpenMap_prodMap_general {X Y Z W : Type*} [TopologicalSpace X] [TopologicalSpace Y] [TopologicalSpace Z] [TopologicalSpace W]
  (f : X → Y) (g : Z → W) (hf : IsOpenMap f) (hg : IsOpenMap g) : IsOpenMap (Prod.map f g) := by
  have hprod : IsOpenMap (Prod.map f g) := hf.prodMap hg
  exact hprod
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_16_4__mh__198d2479

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 7c4726cf882d08b5

4.210 210. joint_projection_isOpenMap

The problem

Let X and Y be topological spaces, and let $\pi_1 : X \times Y \rightarrow X$ and $\pi_2 : X \times Y \rightarrow Y$ be the coordinate projections. Show that the map $X \times Y \rightarrow X \times Y$ sending (x, y) to $(\pi_1(x, y), \pi_2(x, y))$ is an open map.

Lean statement

```
theorem joint_projection_isOpenMap {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (π₁ : X × Y → X)
  (π₂ : X × Y → Y)
  (h₁ : π₁ = Prod.fst)
  (h₂ : π₂ = Prod.snd) :
  IsOpenMap (fun p : X × Y => (π₁ p, π₂ p))
```

Parent. A map $f : X \rightarrow Y$ is said to be an open map if for every open set U of X , the set $f(U)$ is open in Y . Show that $\pi_1 : X \times Y \rightarrow X$ and $\pi_2 : X \times Y \rightarrow Y$ are open maps.

Parent in Lean

```
theorem Munkres_exercise_16_4 {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (π₁ : X × Y → X)
  (π₂ : X × Y → Y)
  (h₁ : π₁ = Prod.fst)
  (h₂ : π₂ = Prod.snd) :
  IsOpenMap π₁ ∧ IsOpenMap π₂ := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the target from openness of two projections to openness of their jointly paired map, proving that map equals the identity and invoking identity-map openness. The parent's two projection-openness lemmas do not supply this identity argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the target from two separate open-map facts to openness of the joint map, requiring the new observation that pairing the two projections is the identity and then applying `IsOpenMap.id`. The parent's constructor-based projection proof does not supply this identity characterization.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conjunction of separate projection claims into openness of the joint map. Its proof requires establishing that the paired projections equal the identity and invoking openness of the identity, reasoning absent from the parent's separate-library-lemma proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propect, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1945 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem joint_projection_isOpenMap.extracted_1_1.{u_2, u_1} {X : Type u_1} {Y : Type u_2} [inst : TopologicalSpace X]
[inst_1 : TopologicalSpace Y] (π₁ : X × Y → X) (π₂ : X × Y → Y) (h₁ : π₁ = Prod.fst) (h₂ : π₂ = Prod.snd) :
IsOpenMap fun p => (π₁ p, π₂ p)
```

Read back by a model that saw only the Lean. *Let X and Y be topological spaces, and let $\pi_1 : X \times Y \rightarrow X$ and $\pi_2 : X \times Y \rightarrow Y$ be functions such that π_1 is the first projection and π_2 is the second projection. Then the function from $X \times Y$ to $X \times Y$ that sends each p to $(\pi_1(p), \pi_2(p))$ is an open map, meaning it maps every open subset of $X \times Y$ to an open subset of $X \times Y$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace
open scoped Topology

theorem joint_projection_isOpenMap {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
(π₁ : X × Y → X)
(π₂ : X × Y → Y)
(h₁ : π₁ = Prod.fst)
(h₂ : π₂ = Prod.snd) :
IsOpenMap (fun p : X × Y => (π₁ p, π₂ p)) := by
have hjoint : (fun p : X × Y => (π₁ p, π₂ p)) = id := by
  funext p
  simp [h₁, h₂]
rw [hjoint]
exact IsOpenMap.id
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_16_4__mh__226822b9

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propept, Classical.choice, Quot.sound, fingerprint 20f72843a49667ce

4.211 211. coordinate_images_and_local_rectangles

The problem

Let X and Y be topological spaces, and let the maps from $X \times Y$ to X and Y be the coordinate projections. Show that for every open subset U of $X \times Y$, both coordinate images of U are open, and every point of U lies in an open rectangle contained in U .

Lean statement

```
theorem coordinate_images_and_local_rectangles {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (π₁ : X × Y → X)
  (π₂ : X × Y → Y)
  (h₁ : π₁ = Prod.fst)
  (h₂ : π₂ = Prod.snd) :
  ∀ U : Set (X × Y), IsOpen U →
    IsOpen (π₁ '' U) ∧ IsOpen (π₂ '' U) ∧
      ∀ a b, (a, b) ∈ U →
        ∃ u v, IsOpen u ∧ IsOpen v ∧ a ∈ u ∧ b ∈ v ∧ u × v ⊆ U
```

Parent. A map $f : X \rightarrow Y$ is said to be an open map if for every open set U of X , the set $f(U)$ is open in Y . Show that $\pi_1 : X \times Y \rightarrow X$ and $\pi_2 : X \times Y \rightarrow Y$ are open maps.

Parent in Lean

```
theorem Munkres_exercise_16_4 {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (π₁ : X × Y → X)
  (π₂ : X × Y → Y)
  (h₁ : π₁ = Prod.fst)
  (h₂ : π₂ = Prod.snd) :
  IsOpenMap π₁ ∧ IsOpenMap π₂ := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds genuinely new mathematics: it characterizes every open subset locally by product rectangles, in addition to proving openness of both projection images. The parent's projection proof alone cannot supply this local rectangle obligation.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child substantially changes the obligation by characterizing open product neighborhoods at every point, in addition to proving openness of both coordinate images. The local-rectangle existence requires reasoning absent from the parent's projection-openness proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child substantially changes the obligation by adding the local product-neighborhood characterization: every point of an open set must lie in an open rectangle contained in that set. Although the projection-open-map arguments reuse the parent's lemmas, the new rectangle existence requires independent topological reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound

Check	By	Result
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2316 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem coordinate_images_and_local_rectangles.extracted_1_1 : ∀ {X : Type u_1} {Y : Type u_2}
  [inst : TopologicalSpace X] [inst_1 : TopologicalSpace Y] (π₁ : X × Y → X) (π₂ : X × Y → Y),
  π₁ = Prod.fst →
  π₂ = Prod.snd →
  ∀ (U : Set (X × Y)),
  IsOpen U →
  IsOpen (π₁ '' U) ∧
  IsOpen (π₂ '' U) ∧
  ∀ (a : X) (b : Y), (a, b) ∈ U → ∃ u v, IsOpen u ∧ IsOpen v ∧ a ∈ u ∧ b ∈ v ∧ u × v ⊆ U
```

Read back by a model that saw only the Lean. For any topological spaces X and Y , and any functions $\pi_1 : X \times Y \rightarrow X$ and $\pi_2 : X \times Y \rightarrow Y$, if π_1 is the first projection and π_2 is the second projection, then for every open set $U \subseteq X \times Y$, both images $\pi_1(U) \subseteq X$ and $\pi_2(U) \subseteq Y$ are open. Moreover, for every $a \in X$ and $b \in Y$ with $(a, b) \in U$, there exist open sets $u \subseteq X$ and $v \subseteq Y$ such that $a \in u$, $b \in v$, and $u \times v \subseteq U$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem coordinate_images_and_local_rectangles {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (π₁ : X × Y → X)
  (π₂ : X × Y → Y)
  (h₁ : π₁ = Prod.fst)
  (h₂ : π₂ = Prod.snd) :
  ∀ U : Set (X × Y), IsOpen U →
  IsOpen (π₁ '' U) ∧ IsOpen (π₂ '' U) ∧
  ∀ a b, (a, b) ∈ U →
  ∃ u v, IsOpen u ∧ IsOpen v ∧ a ∈ u ∧ b ∈ v ∧ u × v ⊆ U := by
  have hfst : IsOpenMap π₁ := by
    simpa [h₁] using (isOpenMap_fst : IsOpenMap (Prod.fst : X × Y → X))
  have hsnd : IsOpenMap π₂ := by
    simpa [h₂] using (isOpenMap_snd : IsOpenMap (Prod.snd : X × Y → Y))
  intro U hU
  have hlocal : ∀ a b, (a, b) ∈ U →
    ∃ u v, IsOpen u ∧ IsOpen v ∧ a ∈ u ∧ b ∈ v ∧ u × v ⊆ U := by
    intro a b hab
    exact isOpen_prod_iff.mp hU a b hab
  exact ⟨hfst U hU, hsnd U hU, hlocal⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_16_4__mh__51a02d05

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 51a02d05f8b603f8

4.212 212. combined_projection_is_open_bijective

The problem

Suppose two maps from $X \times Y$ are the coordinate projections. Show that the combined map $(x, y) \mapsto (\pi_1(x, y), \pi_2(x, y))$ is an open bijection of $X \times Y$ onto itself.

Lean statement

```
theorem combined_projection_is_open_bijective {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (π₁ : X × Y → X)
  (π₂ : X × Y → Y)
  (h₁ : π₁ = Prod.fst)
  (h₂ : π₂ = Prod.snd) :
  IsOpenMap (fun p : X × Y => (π₁ p, π₂ p)) ∧
  Function.Bijective (fun p : X × Y => (π₁ p, π₂ p))
```

Parent. A map $f : X \rightarrow Y$ is said to be an open map if for every open set U of X , the set $f(U)$ is open in Y . Show that $\pi_1 : X \times Y \rightarrow X$ and $\pi_2 : X \times Y \rightarrow Y$ are open maps.

Parent in Lean

```
theorem Munkres_exercise_16_4 {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (π₁ : X × Y → X)
  (π₂ : X × Y → Y)
  (h₁ : π₁ = Prod.fst)
  (h₂ : π₂ = Prod.snd) :
  IsOpenMap π₁ ∧ IsOpenMap π₂ := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the obligation from openness of each coordinate projection to openness and bijectivity of the combined map. Its proof must identify the combined map with the identity and establish bijectivity via product extensionality, none of which is supplied by the parent proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from separate openness of the coordinate projections to openness and bijectivity of their combined map. Proving that this map is the identity and establishing bijectivity requires reasoning not supplied by the parent's two projection lemmas.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion to the openness and bijectivity of the combined projection map. Proving that this map is the identity and hence bijective requires reasoning absent from the parent's separate openness proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2792 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem combined_projection_is_open_bijective.extracted_1_1.{u_2, u_1} {X : Type u_1} {Y : Type u_2}
  [inst : TopologicalSpace X] [inst_1 : TopologicalSpace Y] (π1 : X × Y → X) (π2 : X × Y → Y) (h1 : π1 = Prod.fst)
  (h2 : π2 = Prod.snd) : (IsOpenMap fun p => (π1 p, π2 p)) ∧ Function.Bijective fun p => (π1 p, π2 p)
```

Read back by a model that saw only the Lean. *Let X and Y be topological spaces, let $\pi_1 : X \times Y \rightarrow X$ and $\pi_2 : X \times Y \rightarrow Y$ be functions such that π_1 is the first projection and π_2 is the second projection. Then the map sending each $p \in X \times Y$ to $(\pi_1(p), \pi_2(p))$ is an open map and is bijective.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace
open scoped Topology

theorem combined_projection_is_open_bijective {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (π1 : X × Y → X)
  (π2 : X × Y → Y)
  (h1 : π1 = Prod.fst)
  (h2 : π2 = Prod.snd) :
  IsOpenMap (fun p : X × Y => (π1 p, π2 p)) ∧
  Function.Bijective (fun p : X × Y => (π1 p, π2 p)) := by
  have hmap : (fun p : X × Y => (π1 p, π2 p)) = id := by
    funext p
    rw [h1, h2]
    rfl
  have hopen : IsOpenMap (fun p : X × Y => (π1 p, π2 p)) := by
    rw [hmap]
    exact IsOpenMap.id
  have hbij : Function.Bijective (fun p : X × Y => (π1 p, π2 p)) := by
    constructor
    · intro a b hab
      apply Prod.ext
      · have h := congrArg Prod.fst hab
        simpa [h1] using h
      · have h := congrArg Prod.snd hab
        simpa [h2] using h
    · intro p
      refine ⟨p, ?_⟩
      simp [h1, h2]
  exact ⟨hopen, hbij⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_16_4__mh__9ac34659

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 9ac346591f88514b

4.213 213. simultaneous_comparison_closed_and_closure_contains

The problem

Let Y be a linearly ordered topological space, let $f_i, g_i : X \rightarrow Y$ be continuous for each index i in a finite nonempty set, and let $h : X \rightarrow X$ be continuous. If $f_i(x) \leq g_i(h(x))$ on a dense set D , show that the set of points satisfying all these inequalities is closed and contains the closure of D .

Lean statement

```
theorem simultaneous_comparison_closed_and_closure_contains {ι X Y : Type*} [Fintype ι] [Nonempty ι]
[TopologicalSpace X] [TopologicalSpace Y] [LinearOrder Y] [OrderTopology Y]
(f g : ι → X → Y) (h : X → X) (hf : ∀ i, Continuous (f i))
(hg : ∀ i, Continuous (g i)) (hh : Continuous h) (D : Set X)
(hineq : ∀ x ∈ D, ∀ i, f i x ≤ g i (h x)) :
IsClosed {x : X | ∀ i, f i x ≤ g i (h x)} ∧
closure D ⊆ {x : X | ∀ i, f i x ≤ g i (h x)}
```

Parent. Let Y be a linearly ordered topological space, let $f_i, g_i : X \rightarrow Y$ be continuous for each index i in a finite nonempty set, and let $h : X \rightarrow X$ be continuous. If $f_i(x) \leq g_i(h(x))$ for every x in a dense subset of X and every i , show that this inequality holds for all $x \in X$ and every i .

Parent in Lean

```
theorem dense_coupled_simultaneous_comparison {ι X Y : Type*} [Fintype ι] [Nonempty ι]
[TopologicalSpace X] [TopologicalSpace Y] [LinearOrder Y] [OrderTopology Y]
(f g : ι → X → Y) (h : X → X) (hf : ∀ i, Continuous (f i))
(hg : ∀ i, Continuous (g i)) (hh : Continuous h) (D : Set X) (hD : Dense D)
(hineq : ∀ x ∈ D, ∀ i, f i x ≤ g i (h x)) :
∀ x, ∀ i, f i x ≤ g i (h x)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the proof obligation from pointwise global comparison to proving that the simultaneous feasible set is closed and contains the closure of D . This requires reasoning about arbitrary intersections of closed sets via `isClosed_iInter`, which the parent's pointwise density argument does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion to a closedness-and-closure characterization, requiring construction of the finite intersection of coordinatewise inequality sets and proving its closedness. It also removes the density hypothesis, so the result is not recoverable by simply recalling the parent's pointwise extension proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the obligation from a pointwise global inequality to a conjunction of two structural claims: closedness of the simultaneous feasible set and containment of the closure of D . Proving closedness requires identifying the set as an indexed intersection and applying a new lattice/topological argument, so the parent's proof is not sufficient by recall alone.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry

Check	By	Result
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2973 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem simultaneous_comparison_closed_and_closure_contains.extracted_1_1.{u_3, u_2, u_1} {ι : Type u_1} {X : Type u_2}
  {Y : Type u_3} [inst : Nonempty ι] [inst_1 : TopologicalSpace X] [inst_2 : TopologicalSpace Y]
  [inst_3 : LinearOrder Y] [inst_4 : OrderTopology Y] (f g : ι → X → Y) (h : X → X) (hf : ∀ (i : ι), Continuous (f i))
  (hg : ∀ (i : ι), Continuous (g i)) (hh : Continuous h) (D : Set X) (hD : Dense D)
  (hineq : ∀ x ∈ D, ∀ (i : ι), f i x ≤ g i (h x)) :
  IsClosed {x | ∀ (i : ι), f i x ≤ g i (h x)} ∧ closure D ⊆ {x | ∀ (i : ι), f i x ≤ g i (h x)}
```

Read back by a model that saw only the Lean. Let ι , X , and Y be types, with ι nonempty; let X and Y be topological spaces, and let Y be a linearly ordered topological space whose topology is the order topology. Let f and g be functions from ι to functions $X \rightarrow Y$, let $h : X \rightarrow X$, and let $D \subseteq X$. Assume that for every $i \in \iota$, $f i$ is continuous; for every $i \in \iota$, $g i$ is continuous; h is continuous; D is dense in X ; and for every $x \in D$ and every $i \in \iota$, $f i x \leq g i (h x)$. Then the set of all $x \in X$ such that, for every $i \in \iota$, $f i x \leq g i (h x)$ is closed, and the closure of D is contained in this set.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem simultaneous_comparison_closed_and_closure_contains {ι X Y : Type*} [Fintype ι] [Nonempty ι]
  [TopologicalSpace X] [TopologicalSpace Y] [LinearOrder Y] [OrderTopology Y]
  (f g : ι → X → Y) (h : X → X) (hf : ∀ i, Continuous (f i))
  (hg : ∀ i, Continuous (g i)) (hh : Continuous h) (D : Set X)
  (hineq : ∀ x ∈ D, ∀ i, f i x ≤ g i (h x)) :
  IsClosed {x : X | ∀ i, f i x ≤ g i (h x)} ∧
  closure D ⊆ {x : X | ∀ i, f i x ≤ g i (h x)} := by
  have hclosed_i : ∀ i, IsClosed {x : X | f i x ≤ g i (h x)} := by
    intro i
    exact isClosed_le (hf i) ((hg i).comp hh)
  have hset : {x : X | ∀ i, f i x ≤ g i (h x)} =
    ⋂ i, {x : X | f i x ≤ g i (h x)} := by
    ext x
    simp
  have hclosed : IsClosed {x : X | ∀ i, f i x ≤ g i (h x)} := by
    rw [hset]
    exact isClosed_iInter hclosed_i
  have hsubset : D ⊆ {x : X | ∀ i, f i x ≤ g i (h x)} := by
    intro x hx i
    exact hineq x hx i
  have hclosure : closure D ⊆ {x : X | ∀ i, f i x ≤ g i (h x)} := by
    exact closure_minimal hsubset hclosed
  exact ⟨hclosed, hclosure⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Munkres_exercise_18_8a__me.me.mh__1a717344

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 7d7aed89fd94f6f3

4.214 214. finite_bounded_infeasible_locus_open

The problem

Let X be a topological space and Y a linearly ordered topological space. If finitely many lower and upper bounds, together with a comparison function, are continuous, show that the set of points violating at least one bound is open.

Lean statement

```
theorem finite_bounded_infeasible_locus_open {ι X Y : Type*} [Fintype ι] [Nonempty ι]
  [TopologicalSpace X] [TopologicalSpace Y] [LinearOrder Y] [OrderTopology Y]
  (l u : ι → X → Y) (h : X → Y)
  (hl : ∀ i, Continuous (l i)) (hu : ∀ i, Continuous (u i))
  (hh : Continuous h) :
  IsOpen {x | ∃ i, h x < l i x ∨ u i x < h x}
```

Parent. Let X be a topological space and Y a linearly ordered topological space. If finitely many lower and upper bounds, together with a comparison function, are continuous, show that the set of points satisfying every lower bound and every upper bound is closed.

Parent in Lean

```
theorem finite_bounded_feasible_locus_closed {ι X Y : Type*} [Fintype ι] [Nonempty ι]
  [TopologicalSpace X] [TopologicalSpace Y] [LinearOrder Y] [OrderTopology Y]
  (l u : ι → X → Y) (h : X → Y)
  (hl : ∀ i, Continuous (l i)) (hu : ∀ i, Continuous (u i))
  (hh : Continuous h) :
  IsClosed {x | ∀ i, l i x ≤ h x ∧ h x ≤ u i x} := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

The child changes the obligation from a closed universal intersection of weak inequalities to an open existential union of strict violations. The parent's 'isClosed_le'/intersection argument cannot be directly applied; the child requires the distinct open-set lemmas and union construction, even though the overall topological pattern is analogous.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the obligation from closedness of a finite intersection of non-strict feasibility constraints to openness of a finite union of strict infeasibility constraints. Its proof requires strict-order openness lemmas, unions, and existential reformulation, so the parent's proof does not supply it.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the obligation from a closed finite intersection of non-strict feasibility constraints to an open finite union of strict infeasibility constraints. It requires dual topological reasoning with strict inequalities, unions, and existential indexing; the parent's closed-set proof does not directly solve it.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable

Check	By	Result
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2947 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_bounded_infeasible_locus_open.extracted_1_1.{u_3, u_2, u_1} {ι : Type u_1} {X : Type u_2} {Y : Type u_3}
  [inst : Nonempty ι] [inst_1 : TopologicalSpace X] [inst_2 : TopologicalSpace Y] [inst_3 : LinearOrder Y]
  [inst_4 : OrderTopology Y] (l u : ι → X → Y) (h : X → Y) (hl : ∀ (i : ι), Continuous (l i))
  (hu : ∀ (i : ι), Continuous (u i)) (hh : Continuous h) : IsOpen {x | ∃ i, h x < l i x ∨ u i x < h x}
```

Read back by a model that saw only the Lean. *Let ι be a nonempty type, let X and Y be topological spaces, and suppose Y is linearly ordered with its order topology. Let $l, u : \iota \rightarrow X \rightarrow Y$ and $h : X \rightarrow Y$. Assume that for every $i \in \iota$, the functions $x \mapsto l(i, x)$ and $x \mapsto u(i, x)$ are continuous, and that h is continuous. Then the set of all $x \in X$ for which there exists $i \in \iota$ such that $h(x) < l(i, x)$ or $u(i, x) < h(x)$ is open in X .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem finite_bounded_infeasible_locus_open {ι X Y : Type*} [Fintype ι] [Nonempty ι]
  [TopologicalSpace X] [TopologicalSpace Y] [LinearOrder Y] [OrderTopology Y]
  (l u : ι → X → Y) (h : X → Y)
  (hl : ∀ i, Continuous (l i)) (hu : ∀ i, Continuous (u i))
  (hh : Continuous h) :
  IsOpen {x | ∃ i, h x < l i x ∨ u i x < h x} := by
  have hlocal : ∀ i : ι, IsOpen {x : X | h x < l i x ∨ u i x < h x} := by
    intro i
    exact (isOpen_lt hh (hl i)).union (isOpen_lt (hu i) hh)
  have hopen : IsOpen (U i : ι, {x : X | h x < l i x ∨ u i x < h x}) :=
    isOpen_iUnion hlocal
  have hset : (U i : ι, {x : X | h x < l i x ∨ u i x < h x}) =
    {x | ∃ i, h x < l i x ∨ u i x < h x} := by
    ext x
    simp
  rw [← hset]
  exact hopen
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Munkres_exercise_18_8a__me.me.mh__74c69977

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 74c69977e7d45292

4.215 215. closed_restricted_eq

The problem

Let X be a topological space and Y an ordered set with the order topology. If $s \subseteq X$ is closed and $f, g : X \rightarrow Y$ are continuous, show that $\{x \in s \mid f(x) = g(x)\}$ is closed in X .

Lean statement

```
theorem closed_restricted_eq {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {s : Set X} {f g : X → Y}
  (hs : IsClosed s) (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x | x ∈ s ∧ f x = g x}
```

Parent. Let X be a topological space and Y an ordered set with the order topology. If $s \subseteq X$ is closed and $f, g : X \rightarrow Y$ are continuous, show that $\{x \in s \mid f(x) \leq g(x)\}$ is closed in X .

Parent in Lean

```
theorem closed_restricted_le {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {s : Set X} {f g : X → Y}
  (hs : IsClosed s) (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x | x ∈ s ∧ f x ≤ g x}
```

What it contributes. Reuses the parent's continuous comparison closedness checkpoint and closed-domain intersection, while replacing the single inequality locus with a two-direction equality characterization.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the coupling from inequality to equality and requires proving equality as the intersection of two closed order loci, followed by antisymmetry. The parent's one-sided 'isClosed_le' argument cannot be recalled to close this goal directly.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the inequality locus to an equality locus, which requires proving equality as the intersection of both inequality loci and applying antisymmetry. The parent proof supplies only one closed inequality and cannot be recalled unchanged.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the inequality locus to an equality locus, requiring a second closed inequality set and a genuine antisymmetry argument. The parent proof only establishes one inequality and cannot be recalled to close the new goal.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2801 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem closed_restricted_eq.extracted_1.1.{u_2, u_1} {X : Type u_1} {Y : Type u_2} [inst : TopologicalSpace X]
[inst_1 : TopologicalSpace Y] [inst_2 : LinearOrder Y] [inst_3 : OrderTopology Y] {s : Set X} {f g : X → Y}
(hs : IsClosed s) (hf : Continuous f) (hg : Continuous g) : IsClosed {x | x ∈ s ∧ f x = g x}
```

Read back by a model that saw only the Lean. *Let X and Y be topological spaces, with Y equipped with a linear order whose order topology is its given topology. For a closed set $s \subseteq X$ and continuous functions $f, g : X \rightarrow Y$, the set $\{x \in X \mid x \in s \text{ and } f(x) = g(x)\}$ is closed in X .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace
open scoped Topology

theorem closed_restricted_eq {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
[LinearOrder Y] [OrderTopology Y] {s : Set X} {f g : X → Y}
(hs : IsClosed s) (hf : Continuous f) (hg : Continuous g) :
IsClosed {x | x ∈ s ∧ f x = g x} := by
have hle : IsClosed {x | f x ≤ g x} := by
  exact isClosed_le hf hg
have hge : IsClosed {x | g x ≤ f x} := by
  exact isClosed_le hg hf
have hcmp : IsClosed ({x | f x ≤ g x} ∩ {x | g x ≤ f x}) := by
  exact hle.inter hge
have heq : {x | f x = g x} = {x | f x ≤ g x} ∩ {x | g x ≤ f x} := by
  ext x
  constructor
  · intro hx
    exact ⟨le_of_eq hx, le_of_eq hx.symm⟩
  · intro hx
    exact le_antisymm hx.1 hx.2
have heqclosed : IsClosed {x | f x = g x} := by
  rw [heq]
  exact hcmp
have hfinal : IsClosed (s ∩ {x | f x = g x}) := by
  exact hs.inter heqclosed
have hset : {x | x ∈ s ∧ f x = g x} = s ∩ {x | f x = g x} := by
  ext x
  rfl
rw [hset]
exact hfinal
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Munkres_exercise_18_8a__me.mh__08974092

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 08974092f919d370

4.216 216. finite_joint_equality_isClosed

The problem

For a finite family of continuous functions from a topological space X into a linearly ordered topological space Y , the set of points where every corresponding pair of functions is equal is closed.

Lean statement

```
theorem finite_joint_equality_isClosed {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {n : ℕ} {f g : Fin n → X → Y}
  (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i)) :
  IsClosed {x | ∀ i, f i x = g i x}
```

Parent. Let X be a topological space and Y a linearly ordered topological space. Let $n \in \mathbb{N}$, and for each $i = 0, \dots, n-1$, let $f_i, g_i : X \rightarrow Y$ be continuous. Show that the set $\{x \in X \mid f_i(x) \leq g_i(x) \text{ for every } i = 0, \dots, n-1\}$ is closed in X .

Parent in Lean

```
theorem finite_joint_comparison_isClosed {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {n : ℕ} {f g : Fin n → X → Y}
  (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i)) :
  IsClosed {x | ∀ i, f i x ≤ g i x}
```

What it contributes. Used the parent's finite-intersection and continuity-based closedness checkpoint for the forward comparison; the reverse comparison and antisymmetry provide the mutation-specific equality argument.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from a simultaneous inequality locus to a simultaneous equality locus. Its proof requires proving both inequality directions for every index, intersecting the resulting closed sets, and recovering equality by antisymmetry; the parent supplies only one direction.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The equality locus requires proving and intersecting two independent finite joint inequality loci, using both $f \leq g$ and $g \leq f$ before applying antisymmetry. The parent's proof supplies only the forward inequality locus and cannot establish closedness of the equality set by itself.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The equality locus requires proving both inequality directions for every index and combining them via antisymmetry; the parent only establishes one-sided comparisons. The child therefore demands genuinely new reasoning beyond the parent's intersection-of-closed-sets argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1956 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_joint_equality_isClosed.extracted_1_1.{u_2, u_1} {X : Type u_1} {Y : Type u_2}
  [inst : TopologicalSpace X] [inst_1 : TopologicalSpace Y] [inst_2 : LinearOrder Y] [inst_3 : OrderTopology Y] {n : ℕ}
  {f g : Fin n → X → Y} (hf : ∀ (i : Fin n), Continuous (f i)) (hg : ∀ (i : Fin n), Continuous (g i)) :
  IsClosed {x | ∀ (i : Fin n), f i x = g i x}
```

Read back by a model that saw only the Lean. *Let X and Y be topological spaces, with Y equipped with a linear order whose topology is the order topology. Let n be a natural number, and let f and g be families of functions from X to Y indexed by $\text{Fin } n$. Assume that for every i in $\text{Fin } n$, $f i$ is continuous and $g i$ is continuous. Then the set of all x in X such that $f i x = g i x$ for every i in $\text{Fin } n$ is closed in X .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem finite_joint_equality_isClosed {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {n : ℕ} {f g : Fin n → X → Y}
  (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i)) :
  IsClosed {x | ∀ i, f i x = g i x} := by
  have hle : ∀ i : Fin n, IsClosed {x | f i x ≤ g i x} := by
    intro i
    exact isClosed_le (hf i) (hg i)
  have hge : ∀ i : Fin n, IsClosed {x | g i x ≤ f i x} := by
    intro i
    exact isClosed_le (hg i) (hf i)
  have hforward : IsClosed (⋂ i : Fin n, {x | f i x ≤ g i x}) := by
    exact isClosed_iInter hle
  have hreverse : IsClosed (⋂ i : Fin n, {x | g i x ≤ f i x}) := by
    exact isClosed_iInter hge
  have hjoint : IsClosed ((⋂ i : Fin n, {x | f i x ≤ g i x}) ⋂
    (⋂ i : Fin n, {x | g i x ≤ f i x})) := by
    exact hforward.inter hreverse
  have hrepr : {x | ∀ i, f i x = g i x} =
    (⋂ i : Fin n, {x | f i x ≤ g i x}) ⋂
    (⋂ i : Fin n, {x | g i x ≤ f i x}) := by
    ext x
    simp only [Set.mem_iInter, Set.mem_setOf_eq, Set.mem_inter_iff]
  constructor
  · intro hx
    constructor
    · intro i
      exact le_of_eq (hx i)
    · intro i
      exact le_of_eq (hx i).symm
  · intro hx i
    exact le_antisymm (hx.1 i) (hx.2 i)
  rw [hrepr]
  exact hjoint
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Munkres_exercise_18_8a__me.mh__8fcf5cd1

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 73ec102f73c680c2

4.217 217. dense_eq_everywhere

The problem

Let X be a topological space and let Y be a linearly ordered set with the order topology. If continuous functions $f, g : X \rightarrow Y$ agree on a dense subset of X , show that they agree everywhere.

Lean statement

```
theorem dense_eq_everywhere {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {f g : X → Y} {s : Set X}
  (hf : Continuous f) (hg : Continuous g) (hs : Dense s)
  (heq : ∀ x ∈ s, f x = g x) :
  ∀ x, f x = g x
```

Parent. Let X be a topological space and let Y be a linearly ordered set with the order topology. If continuous functions $f, g : X \rightarrow Y$ satisfy $f(x) \leq g(x)$ on a dense subset of X , show that $f(x) \leq g(x)$ for every $x \in X$.

Parent in Lean

```
theorem dense_le_everywhere {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {f g : X → Y} {s : Set X}
  (hf : Continuous f) (hg : Continuous g) (hs : Dense s)
  (hle : ∀ x ∈ s, f x ≤ g x) :
  ∀ x, f x ≤ g x
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes a one-sided inequality into an equality, requiring two separate closure arguments and anti-symmetry; the parent proof supplies only one direction. This is a genuine change in proof obligation rather than decoration or recall.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from a one-sided inequality to equality, requiring two separate density-closure arguments: one for $f \leq g$ and a new reversed one for $g \leq f$. The parent's proof supplies only the first direction, so recalling it does not complete the child.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from an inequality to equality, requiring two separate density-closure arguments: one for $f \leq g$ and a new reversed argument for $g \leq f$, followed by antisymmetry. The parent's one-sided proof cannot close the child's goal by recall alone.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2257 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem dense_eq_everywhere.extracted_1_1 : ∀ {X : Type u_1} {Y : Type u_2} [inst : TopologicalSpace X]
  [inst_1 : TopologicalSpace Y] [inst_2 : LinearOrder Y] [inst_3 : OrderTopology Y] {f g : X → Y} {s : Set X},
  Continuous f → Continuous g → Dense s → (∀ x ∈ s, f x = g x) → ∀ (x : X), f x = g x
```

Read back by a model that saw only the Lean. *For any topological spaces X and Y , where Y is linearly ordered and equipped with its order topology, any functions $f, g : X \rightarrow Y$, and any subset s of X , if f and g are continuous, s is dense in X , and $f(x) = g(x)$ for every $x \in s$, then $f(x) = g(x)$ for every $x \in X$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem dense_eq_everywhere {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {f g : X → Y} {s : Set X}
  (hf : Continuous f) (hg : Continuous g) (hs : Dense s)
  (heq : ∀ x ∈ s, f x = g x) :
  ∀ x, f x = g x := by
  have hclosed_le : IsClosed {x : X | f x ≤ g x} := isClosed_le hf hg
  have hsubset_le : s ⊆ {x : X | f x ≤ g x} := by
    intro x hx
    exact le_of_eq (heq x hx)
  have hclosure_le : closure s ⊆ {x : X | f x ≤ g x} := by
    exact closure_minimal hsubset_le hclosed_le
  have hle : ∀ x, f x ≤ g x := by
    intro x
    exact hclosure_le (hs x)
  have hclosed_ge : IsClosed {x : X | g x ≤ f x} := isClosed_le hg hf
  have hsubset_ge : s ⊆ {x : X | g x ≤ f x} := by
    intro x hx
    exact le_of_eq (heq x hx).symm
  have hclosure_ge : closure s ⊆ {x : X | g x ≤ f x} := by
    exact closure_minimal hsubset_ge hclosed_ge
  have hge : ∀ x, g x ≤ f x := by
    intro x
    exact hclosure_ge (hs x)
  intro x
  exact le_antisymm (hle x) (hge x)
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Munkres_exercise_18_8a__mh.mh__180e4a76

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 180e4a765ca2cfc b

4.218 218. closed_order_region_on_closed_set

The problem

Let S be a closed subset of a topological space X , and let $f, g : X \rightarrow Y$ be continuous on S , where Y has the order topology. Show that the points of S where $f(x) \leq g(x)$ form a closed subset of X .

Lean statement

```
theorem closed_order_region_on_closed_set {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {s : Set X} {f g : X → Y}
  (hs : IsClosed s) (hf : ContinuousOn f s) (hg : ContinuousOn g s) :
  IsClosed {x ∈ s | f x ≤ g x}
```

Parent. Let Y be an ordered set in the order topology. Let $f, g : X \rightarrow Y$ be continuous. Show that the set $\{x \mid f(x) \leq g(x)\}$ is closed in X .

Parent in Lean

```
theorem Munkres_exercise_18_8a {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {f g : X → Y}
  (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x | f x ≤ g x} := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the claim from a globally closed order region for continuous functions to the intersection of a closed set with an order region for functions only continuous on that set. The parent's direct 'isClosed_le' proof does not apply; the new argument must use the closedness of S together with restricted continuity.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes from globally continuous functions on all of X to functions continuous only on a closed subset s , proving the order region within s is ambiently closed. The parent's direct 'isClosed_le' argument does not establish this restricted-set result; the new closed-subset reasoning is substantive.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely changes the obligation from global continuity on all of X to continuity only on a closed subset s , while proving the restricted order region is closed. Its proof requires the specialized closed-set/ContinuousOn lemma and cannot be obtained by recalling the parent's 'isClosed_le' proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2132 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem closed_order_region_on_closed_set.extracted_1.1.{u_2, u_1} {X : Type u_1} {Y : Type u_2}
  [inst : TopologicalSpace X] [inst_1 : TopologicalSpace Y] [inst_2 : LinearOrder Y] [inst_3 : OrderTopology Y]
  {s : Set X} {f g : X → Y} (hs : IsClosed s) (hf : ContinuousOn f s) (hg : ContinuousOn g s) :
  IsClosed {x | x ∈ s ∧ f x ≤ g x}
```

Read back by a model that saw only the Lean. *Let X and Y be topological spaces, with Y equipped with a linear order whose topology is the order topology. Let s be a closed subset of X , and let $f, g : X \rightarrow Y$ be continuous on s . Then the subset $\{x \in X \mid x \in s \text{ and } f(x) \leq g(x)\}$ is closed in X .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem closed_order_region_on_closed_set {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {s : Set X} {f g : X → Y}
  (hs : IsClosed s) (hf : ContinuousOn f s) (hg : ContinuousOn g s) :
  IsClosed {x ∈ s | f x ≤ g x} := by
  have hrestricted : IsClosed {x ∈ s | f x ≤ g x} := hs.isClosed_le hf hg
  exact hrestricted
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_18_8a__mh__7798f78c

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 7798f78c1903dfd4

4.219 219. continuous_eq_set_closed

The problem

Let Y be a linearly ordered topological space with its order topology, and let $f, g : X \rightarrow Y$ be continuous. Show that the set $\{x \mid f(x) = g(x)\}$ is closed in X .

Lean statement

```
theorem continuous_eq_set_closed {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {f g : X → Y}
  (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x | f x = g x}
```

Parent. Let Y be an ordered set in the order topology. Let $f, g : X \rightarrow Y$ be continuous. Show that the set $\{x \mid f(x) \leq g(x)\}$ is closed in X .

Parent in Lean

```
theorem Munkres_exercise_18_8a {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {f g : X → Y}
  (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x | f x ≤ g x} := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from a sublevel set to an equalizer and requires representing equality as the intersection of two closed inequality sets. Although it reuses the parent lemma twice, the parent's single application does not prove the new set-identity and intersection argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes closedness of an order region into closedness of an equalizer, requiring two order-region closures, an intersection argument, and conversion via antisymmetry. The parent's single 'isClosed_le' application does not supply this proof structure.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the one-sided order region into an equalizer, requiring both inequalities, intersection closure, and extensional conversion to equality. This reasoning is not supplied by the parent's direct application of 'isClosed_le'.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2800 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem continuous_eq_set_closed.extracted_1_1.{u_2, u_1} {X : Type u_1} {Y : Type u_2} [inst : TopologicalSpace X]
[inst_1 : TopologicalSpace Y] [inst_2 : LinearOrder Y] [inst_3 : OrderTopology Y] {f g : X → Y} (hf : Continuous f)
(hg : Continuous g) : IsClosed {x | f x = g x}
```

Read back by a model that saw only the Lean. *Let X and Y be topological spaces, with Y equipped with a linear order whose order topology is its given topology. If f and g are continuous functions from X to Y , then the set of points x in X such that $f(x) = g(x)$ is closed in X .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem continuous_eq_set_closed {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
[LinearOrder Y] [OrderTopology Y] {f g : X → Y}
(hf : Continuous f) (hg : Continuous g) :
IsClosed {x | f x = g x} := by
have hle : IsClosed {x | f x ≤ g x} := by
  exact isClosed_le hf hg
have hge : IsClosed {x | g x ≤ f x} := by
  exact isClosed_le hg hf
have heq : {x | f x = g x} = {x | f x ≤ g x} ∩ {x | g x ≤ f x} := by
  ext x
  simp only [mem_setOf_eq, mem_inter_iff]
  exact le_antisymm_iff
rw [heq]
exact hle.inter hge
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_18_8a__mh__7cd0cba0

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 7cd0cba08b23dcb1

4.220 220. closed_le_and_open_lt

The problem

Let Y be an ordered set in the order topology, and let $f, g : X \rightarrow Y$ be continuous. Show that $x \mid f(x) \leq g(x)$ is closed and $x \mid f(x) < g(x)$ is open in X .

Lean statement

```
theorem closed_le_and_open_lt {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {f g : X → Y}
  (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x | f x ≤ g x} ∧ IsOpen {x | f x < g x}
```

Parent. Let Y be an ordered set in the order topology. Let $f, g : X \rightarrow Y$ be continuous. Show that the set $\{x \mid f(x) \leq g(x)\}$ is closed in X .

Parent in Lean

```
theorem Munkres_exercise_18_8a {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {f g : X → Y}
  (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x | f x ≤ g x} := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation ACCEPTABLE

model gpt-5.6-luna · saw parents, child

The child adds the genuinely separate obligation that the strict-order locus is open, requiring 'isOpen_lt' in addition to the parent's closedness argument. The conjunction is not a forced or trivial wrapper because its second claim is an independent topological theorem.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds the genuinely new obligation that the strict-inequality set is open, requiring 'isOpen_lt'; the parent proves only closedness of the non-strict inequality set. The conjunction is therefore substantive rather than a restatement or decoration.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds the genuinely distinct obligation that the strict-inequality set is open, requiring the separate theorem 'isOpen_lt'; the parent's 'isClosed_le' argument proves only the first conjunct. This is more than a trivial conjunction wrapper because the added conjunct has substantive topological content and its own proof obligation.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2261 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem closed_le_and_open_lt.extracted_1_1.{u_2, u_1} {X : Type u_1} {Y : Type u_2} [inst : TopologicalSpace X]
[inst_1 : TopologicalSpace Y] [inst_2 : LinearOrder Y] [inst_3 : OrderTopology Y] {f g : X → Y} (hf : Continuous f)
(hg : Continuous g) : IsClosed {x | f x ≤ g x} ∧ IsOpen {x | f x < g x}
```

Read back by a model that saw only the Lean. *Let X and Y be topological spaces, with Y also equipped with a linear order whose topology is the order topology. For functions $f, g : X \rightarrow Y$, if f and g are continuous, then the set $\{x \in X \mid f(x) \leq g(x)\}$ is closed in X , and the set $\{x \in X \mid f(x) < g(x)\}$ is open in X .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem closed_le_and_open_lt {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
[LinearOrder Y] [OrderTopology Y] {f g : X → Y}
(hf : Continuous f) (hg : Continuous g) :
IsClosed {x | f x ≤ g x} ∧ IsOpen {x | f x < g x} := by
have hClosed : IsClosed {x | f x ≤ g x} := by
  simpa using isClosed_le hf hg
have hOpen : IsOpen {x | f x < g x} := by
  simpa using isOpen_lt hf hg
exact ⟨hClosed, hOpen⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_18_8a__mh__b6262118

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint b6262118944a5e2d

4.221 221. equality_and_bound_closed

The problem

Let Y be an ordered set in the order topology, and let $f, g, h : X \rightarrow Y$ be continuous. Show that the set of points where $f = g$ and $g \leq h$ is closed in X .

Lean statement

```
theorem equality_and_bound_closed {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {f g h : X → Y}
  (hf : Continuous f) (hg : Continuous g) (hh : Continuous h) :
  IsClosed {x | f x = g x ∧ g x ≤ h x}
```

Parent. Let Y be an ordered set in the order topology. Let $f, g : X \rightarrow Y$ be continuous. Show that the set $\{x \mid f(x) \leq g(x)\}$ is closed in X .

Parent in Lean

```
theorem Munkres_exercise_18_8a {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  [LinearOrder Y] [OrderTopology Y] {f g : X → Y}
  (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x | f x ≤ g x} := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the obligation from one sublevel set to the closed intersection of an equality locus and a second inequality locus. Proving equality as the intersection of both opposite inequalities, then combining three closed sets, requires reasoning absent from the parent's direct one-lemma proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the obligation from one comparison set to a conjunction combining equality and an independent bound. Proving it requires decomposing equality into two inequalities and taking multiple closed-set intersections, which is not supplied by the parent's single application of `isClosed_le`.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the proof obligation from one comparison set to a conjunction combining equality and an upper bound. Proving equality requires two opposite closed inequalities, then intersecting with the third comparison; the parent's single application of `'isClosed_le'` does not supply this reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2804 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem equality_and_bound_closed.extracted_1.1.{u_2, u_1} {X : Type u_1} {Y : Type u_2} [inst : TopologicalSpace X]
[inst_1 : TopologicalSpace Y] [inst_2 : LinearOrder Y] [inst_3 : OrderTopology Y] {f g h : X → Y} (hf : Continuous f)
(hg : Continuous g) (hh : Continuous h) : IsClosed {x | f x = g x ∧ g x ≤ h x}
```

Read back by a model that saw only the Lean. *Let X and Y be topological spaces, with Y equipped with a linear order whose topology is the order topology. For functions $f, g, h : X \rightarrow Y$, if f, g , and h are continuous, then the subset of X consisting of all points x such that $f(x) = g(x)$ and $g(x) \leq h(x)$ is closed.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem equality_and_bound_closed {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
[LinearOrder Y] [OrderTopology Y] {f g h : X → Y}
(hf : Continuous f) (hg : Continuous g) (hh : Continuous h) :
IsClosed {x | f x = g x ∧ g x ≤ h x} := by
have hfg : IsClosed {x | f x ≤ g x} := isClosed_le hf hg
have hgf : IsClosed {x | g x ≤ f x} := isClosed_le hg hf
have hgh : IsClosed {x | g x ≤ h x} := isClosed_le hg hh
have hpair : IsClosed ({x | f x ≤ g x} ∩ {x | g x ≤ f x}) := hfg.inter hgf
have hcombined : IsClosed (({x | f x ≤ g x} ∩ {x | g x ≤ f x}) ∩ {x | g x ≤ h x}) := hpair.inter hgh
have htarget : {x | f x = g x ∧ g x ≤ h x} = ({x | f x ≤ g x} ∩ {x | g x ≤ f x}) ∩ {x | g x ≤ h x} := by
  ext x
  simp only [Set.mem_setOf_eq, Set.mem_inter_iff]
  constructor
  · rintro ⟨he, hle⟩
    exact ⟨⟨le_of_eq he, le_of_eq he.symm⟩, hle⟩
  · rintro ⟨⟨h₁, h₂⟩, hle⟩
    exact ⟨le_antisymm h₁ h₂, hle⟩
rw [htarget]
exact hcombined
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_18_8a__mh__b6fcc7fc

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint b6fcc7fc87d61208

4.222 222. quotient_image_connectedComponent

The problem

Let $p : X \rightarrow Y$ be a quotient map whose fibers are connected. Show that p maps the connected component of each point onto the connected component of its image.

Lean statement

```
theorem quotient_image_connectedComponent {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (hq : IsQuotientMap p)
  (hX : ∀ y : Y, IsConnected (p ⁻¹' ({y} : Set Y))) (x : X) :
  p '' connectedComponent x = connectedComponent (p x)
```

Parent. Let $p : X \rightarrow Y$ be a quotient map whose fibers are connected. If $C \subseteq Y$ is closed and connected, then $p^{-1}(C)$ is connected.

Parent in Lean

```
theorem connected_preimage_of_closed_connected {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (hq : IsQuotientMap p)
  {C : Set Y} (hCclosed : IsClosed C) (hCconn : IsConnected C)
  (hX : ∀ y : Y, IsConnected (p ⁻¹' ({y} : Set Y))) :
  IsConnected (p ⁻¹' C)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from connectedness of a preimage to an equality of connected components. Its proof requires a component-preimage theorem and separately derives surjectivity from nonempty connected fibers, neither of which is supplied by the parent's argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from connectedness of a preimage of a closed connected set to an equality of images of connected components. Its proof requires component-specific reasoning and surjectivity derived from nonempty connected fibers, none of which is supplied by the parent proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the result from connectedness of a preimage of a closed connected set to an equality characterizing the image of a connected component. Proving this requires the new connected-component preimage theorem and a surjectivity argument from nonempty fibers, neither of which is supplied by the parent's proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2209 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quotient_image_connectedComponent.extracted_1_1.{u_2, u_1} {X : Type u_1} {Y : Type u_2}
  [inst : TopologicalSpace X] [inst_1 : TopologicalSpace Y] (p : X → Y) (hq : IsQuotientMap p)
  (hX : ∀ (y : Y), IsConnected (p ⁻¹ {y})) (x : X) : p '' connectedComponent x = connectedComponent (p x)
```

Read back by a model that saw only the Lean. *Let X and Y be topological spaces, let $p : X \rightarrow Y$ be a quotient map, and suppose that for every $y \in Y$ the fiber $p^{-1}(\{y\})$ is connected. Then, for every $x \in X$, the image under p of the connected component of x is exactly the connected component of $p(x)$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace Topology

theorem quotient_image_connectedComponent {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (hq : IsQuotientMap p)
  (hX : ∀ y : Y, IsConnected (p ⁻¹ ({y} : Set Y))) (x : X) :
  p '' connectedComponent x = connectedComponent (p x) := by
  have hcoind : IsCoinducing p := hq.isCoinducing
  have hpre : p ⁻¹ '' connectedComponent (p x) = connectedComponent x :=
    hcoind.preimage_connectedComponent hX x
  have hsurj : Function.Surjective p := by
    intro y
    rcases (hX y).nonempty with ⟨z, hz⟩
    exact ⟨z, by simpa using hz⟩
  have himage : p '' (p ⁻¹ '' connectedComponent (p x)) = connectedComponent (p x) :=
    image_preimage_eq _ hsurj
  rw [hpre] at himage
  exact himage
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Munkres_exercise_23_11__me.mh__d1579e3d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint d1579e3dca610c95

4.223 223. continuous_strict_gap_side

The problem

Let $p : X \rightarrow Y$ be a map and suppose the lifted subspace $p^{-1}(C)$ is connected. If a continuous function $f : p^{-1}(C) \rightarrow \mathbb{R}$ satisfies $f(x) < a$ or $b < f(x)$ for every x , where $a < b$, prove that all values of f lie strictly on the same side of the gap.

Lean statement

```
theorem continuous_strict_gap_side
  {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (C : Set Y)
  (hD : ConnectedSpace {x : X // p x ∈ C})
  (f : {x : X // p x ∈ C} → ℝ)
  (hf : Continuous f)
  (a b : ℝ) (hab : a < b)
  (hd : ∀ x, f x < a ∨ b < f x) :
  (∀ x, f x < a) ∨ (∀ x, b < f x)
```

Parent. Let $p : X \rightarrow Y$ be a map and let $C \subseteq Y$ be such that the lifted subspace $p^{-1}(C)$ is connected. If $f : p^{-1}(C) \rightarrow \mathbb{R}$ is continuous, $a < b$, and every value of f is at most a or at least b , prove that all values of f lie on the same side of the gap (a, b) .

Parent in Lean

```
theorem continuous_gap_side
  {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (C : Set Y)
  (hD : ConnectedSpace {x : X // p x ∈ C})
  (f : {x : X // p x ∈ C} → ℝ)
  (hf : Continuous f)
  (a b : ℝ) (hab : a < b)
  (hd : ∀ x, f x ≤ a ∨ b ≤ f x) :
  (∀ x, f x ≤ a) ∨ (∀ x, b ≤ f x)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The strict-gap conclusion is not obtainable by simply applying the parent: the proof must rule out values on both sides using the intermediate value property and a midpoint argument. This is a genuinely stronger, different proof obligation with no redundant hypotheses.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The strict inequalities require a genuinely different intermediate-value argument: assuming values occur on both sides forces the continuous image through the forbidden midpoint. The parent's closed-set/clopen proof does not directly supply this strict-gap reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion and hypotheses to strict separation, requiring an intermediate-value argument to rule out simultaneously occurring lower and upper values. The parent's clopen-set proof for non-strict inequalities does not supply this proof obligation.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2859 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem continuous_strict_gap_side.extracted_1_1.{u_2, u_1} {X : Type u_1} {Y : Type u_2} [inst : TopologicalSpace X]
  (p : X → Y) (C : Set Y) (hD : ConnectedSpace { x // p x ∈ C }) (f : { x // p x ∈ C } → ℝ) (hf : Continuous f)
  (a b : ℝ) (hab : a < b) (hd : ∀ (x : { x // p x ∈ C }), f x < a ∨ b < f x) :
  (∀ (x : { x // p x ∈ C }), f x < a) ∨ ∀ (x : { x // p x ∈ C }), b < f x
```

Read back by a model that saw only the Lean. *Let X and Y be types, with X equipped with a topological space structure. Let $p : X \rightarrow Y$ and $C \subseteq Y$, and suppose that the subtype $\{x \in X \mid p(x) \in C\}$ is connected. Let f be a continuous real-valued function on this subtype. For real numbers a and b with $a < b$, assume that for every x in the subtype, either $f(x) < a$ or $b < f(x)$. Then either $f(x) < a$ for every x in the subtype, or $b < f(x)$ for every x in the subtype.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem continuous_strict_gap_side
  {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (C : Set Y)
  (hD : ConnectedSpace { x : X // p x ∈ C })
  (f : { x : X // p x ∈ C } → ℝ)
  (hf : Continuous f)
  (a b : ℝ) (hab : a < b)
  (hd : ∀ x, f x < a ∨ b < f x) :
  (∀ x, f x < a) ∨ (∀ x, b < f x) := by
  have hpre : IsPreconnected (Set.univ : Set { x : X // p x ∈ C }) :=
    (connectedSpace_iff_univ.mp hD).isPreconnected
  by_cases hlow : ∃ x, f x < a
  · by_cases hupp : ∃ y, b < f y
    · rcases hlow with ⟨x, hx⟩
      rcases hupp with ⟨y, hy⟩
      let c : ℝ := (a + b) / 2
      have hxc : f x ≤ c := by
        dsimp [c]
        linarith
      have hcy : c ≤ f y := by
        dsimp [c]
        linarith
      have hinterval :
        Set.Icc (f x) (f y) ⊆
          f '' (Set.univ : Set { x : X // p x ∈ C }) :=
        hpre.intermediate_value (Set.mem_univ x) (Set.mem_univ y) hf.continuousOn
      have hc : c ∈ f '' (Set.univ : Set { x : X // p x ∈ C }) :=
        hinterval ⟨hxc, hcy⟩
      rcases hc with ⟨z, hz, hzc⟩
      rcases hd z with hza | hzb
      · exfalso
        dsimp [c] at hzc
        linarith
      · exfalso
        dsimp [c] at hzc
        linarith
    · left
```

```

      intro x
      rcases hd x with hxa | hxb
      · exact hxa
      · exact False.elim (hupp ⟨x, hxb⟩)
    · right
      intro x
      rcases hd x with hxa | hxb
      · exact False.elim (hlow ⟨x, hxa⟩)
      · exact hxb

```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Munkres_exercise_23_11__me__ee8e2e02__x__Rudin_exercise_4_3__me.xh.mh__1550c1e9

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 1550c1e9931c183b

4.224 224. continuous_gap_image_side

The problem

Let S be a connected subset of the lifted space $p^{-1}(C)$. If a continuous function on S takes every value at most a or at least b , where $a < b$, prove that its image lies entirely on one side of the gap (a, b) .

Lean statement

```
theorem continuous_gap_image_side
  {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (C : Set Y)
  (S : Set {x : X // p x ∈ C})
  (hS : IsConnected S)
  (f : {x : X // p x ∈ C} → ℝ)
  (hf : ContinuousOn f S)
  (a b : ℝ) (hab : a < b)
  (hd : ∀ x ∈ S, f x ≤ a ∨ b ≤ f x) :
  (f '' S ⊆ Set.Iic a) ∨ (f '' S ⊆ Set.Ici b)
```

Parent. Let $p : X \rightarrow Y$ be a map and let $C \subseteq Y$ be such that the lifted subspace $p^{-1}(C)$ is connected. If $f : p^{-1}(C) \rightarrow \mathbb{R}$ is continuous, $a < b$, and every value of f is at most a or at least b , prove that all values of f lie on the same side of the gap (a, b) .

Parent in Lean

```
theorem continuous_gap_side
  {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (C : Set Y)
  (hD : ConnectedSpace {x : X // p x ∈ C})
  (f : {x : X // p x ∈ C} → ℝ)
  (hf : Continuous f)
  (a b : ℝ) (hab : a < b)
  (hd : ∀ x, f x ≤ a ∨ b ≤ f x) :
  (∀ x, f x ≤ a) ∨ (∀ x, b ≤ f x)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely changes the obligation from a global dichotomy on the whole connected subtype to a dichotomy for the image of an arbitrary connected subset. Its proof must handle `ContinuousOn`, relative connectedness, and image witnesses, none of which are supplied by recalling the parent proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the obligation from a global pointwise dichotomy to a dichotomy for the image of an arbitrary connected subset, requiring a new intermediate-value argument with `ContinuousOn`. The parent's clopen-set proof does not supply this image-level reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely changes the obligation to a dichotomy for the image of an arbitrary connected subset under a function continuous only on that subset. Its proof requires an intermediate-value argument to rule out crossing the forbidden gap, which is not supplied by the parent's clopen-subset proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2854 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem continuous_gap_image_side.extracted_1.1.{u_2, u_1} {X : Type u_1} {Y : Type u_2} [inst : TopologicalSpace X]
  (p : X → Y) (C : Set Y) (S : Set { x // p x ∈ C }) (hS : IsConnected S) (f : { x // p x ∈ C } → ℝ)
  (hf : ContinuousOn f S) (a b : ℝ) (hab : a < b) (hd : ∀ x ∈ S, f x ≤ a ∨ b ≤ f x) :
  f '' S ⊆ Set.Iic a ∨ f '' S ⊆ Set.Ici b
```

Read back by a model that saw only the Lean. *Let X be a topological space, let Y be a type, let $p : X \rightarrow Y$, and let $C \subseteq Y$. Consider the subtype $\{x \in X \mid p(x) \in C\}$, let S be a connected subset of this subtype, and let $f : \{x \in X \mid p(x) \in C\} \rightarrow \mathbb{R}$ be continuous on S . If $a, b \in \mathbb{R}$ satisfy $a < b$ and every $x \in S$ satisfies $f(x) \leq a$ or $b \leq f(x)$, then either $f(S) \subseteq (-\infty, a]$ or $f(S) \subseteq [b, \infty)$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem continuous_gap_image_side
  {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (C : Set Y)
  (S : Set { x : X // p x ∈ C })
  (hS : IsConnected S)
  (f : { x : X // p x ∈ C } → ℝ)
  (hf : ContinuousOn f S)
  (a b : ℝ) (hab : a < b)
  (hd : ∀ x ∈ S, f x ≤ a ∨ b ≤ f x) :
  (f '' S ⊆ Set.Iic a) ∨ (f '' S ⊆ Set.Ici b) := by
  let m : ℝ := (a + b) / 2
  have hma : a < m := by
    dsimp [m]
    linarith
  have hmb : m < b := by
    dsimp [m]
    linarith
  by_cases hlow : ∃ x ∈ S, f x ≤ a
  · rcases hlow with ⟨x, hxS, hxa⟩
    left
    intro z hz
    rcases hz with ⟨y, hyS, rfl⟩
    rcases hd y hyS with hya | hyb
    · exact hya
    · exfalso
      have hxy : f x ≤ f y := by
        linarith
      have hmid : m ∈ Set.Icc (f x) (f y) := by
        constructor <|> linarith
      have hmemb : m ∈ f '' S :=
        (hS.isPreconnected.intermediate_value hxS hyS hf) hmid
      rcases hmemb with ⟨w, hwS, hw⟩
      rcases hd w hwS with hwa | hwb
      · linarith
      · linarith
  · right
    intro z hz
    rcases hz with ⟨y, hyS, rfl⟩
```

```
rcases hd y hyS with hya | hyb
• exact False.elim (hlow ⟨y, hyS, hya⟩)
• exact hyb
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Munkres_exercise_23_11__me__ee8e2e02__x__Rudin_exercise_4_3__me.xh.mh__909d624c

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 909d624c0275592a

4.225 225. continuous_gap_intermediate_value

The problem

Let $p : X \rightarrow Y$ be a map whose lifted subspace $p^{-1}(C)$ is connected. If a continuous function on this subspace takes a value at most a and another at least b , where $a < b$, prove that it takes some value in the interval $[a, b]$.

Lean statement

```
theorem continuous_gap_intermediate_value
  {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (C : Set Y)
  (hD : ConnectedSpace {x : X // p x ∈ C})
  (f : {x : X // p x ∈ C} → ℝ)
  (hf : Continuous f)
  (a b : ℝ) (hab : a < b)
  (hlo : ∃ x, f x ≤ a)
  (hhi : ∃ x, b ≤ f x) :
  ∃ x, a ≤ f x ∧ f x ≤ b
```

Parent. Let $p : X \rightarrow Y$ be a map and let $C \subseteq Y$ be such that the lifted subspace $p^{-1}(C)$ is connected. If $f : p^{-1}(C) \rightarrow \mathbb{R}$ is continuous, $a < b$, and every value of f is at most a or at least b , prove that all values of f lie on the same side of the gap (a, b) .

Parent in Lean

```
theorem continuous_gap_side
  {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (C : Set Y)
  (hD : ConnectedSpace {x : X // p x ∈ C})
  (f : {x : X // p x ∈ C} → ℝ)
  (hf : Continuous f)
  (a b : ℝ) (hab : a < b)
  (hd : ∀ x, f x ≤ a ∨ b ≤ f x) :
  (∀ x, f x ≤ a) ∨ (∀ x, b ≤ f x)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child is a genuine intermediate-value theorem: endpoint witnesses on opposite sides force a value in $[a, b]$. Its proof uses connectedness of the domain and continuity via a range lemma, reasoning not supplied by the parent's clopen-set dichotomy argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from a global one-sided dichotomy to an intermediate-value existence claim. Its proof requires the connected-space intermediate value theorem using both low and high witnesses, which the parent's clopen-set argument does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the parent's gap dichotomy with a genuine intermediate-value existence claim. Its proof uses continuity and connectedness to establish that the image contains the endpoint value, reasoning not supplied by the parent's side-classification argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2848 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem continuous_gap_intermediate_value.extracted_1_1.{u_2, u_1} {X : Type u_1} {Y : Type u_2}
  [inst : TopologicalSpace X] (p : X → Y) (C : Set Y) (hD : ConnectedSpace { x // p x ∈ C }) (f : { x // p x ∈ C } → ℝ)
  (hf : Continuous f) (a b : ℝ) (hab : a < b) (hlo : ∃ x, f x ≤ a) (hhi : ∃ x, b ≤ f x) :
  ∃ x, a ≤ f x ∧ f x ≤ b
```

Read back by a model that saw only the Lean. *Let X and Y be types, let X have a topological-space structure, let $p : X \rightarrow Y$, and let C be a subset of Y . Assume that the subtype of elements x of X satisfying $p(x) \in C$ is connected. Let f be a continuous real-valued function on this subtype. For real numbers a and b with $a < b$, suppose there exists an x in the subtype such that $f(x) \leq a$, and there exists an x in the subtype such that $b \leq f(x)$. Then there exists an x in the subtype such that $a \leq f(x)$ and $f(x) \leq b$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem continuous_gap_intermediate_value
  {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (C : Set Y)
  (hD : ConnectedSpace { x : X // p x ∈ C })
  (f : { x : X // p x ∈ C } → ℝ)
  (hf : Continuous f)
  (a b : ℝ) (hab : a < b)
  (hlo : ∃ x, f x ≤ a)
  (hhi : ∃ x, b ≤ f x) :
  ∃ x, a ≤ f x ∧ f x ≤ b := by
  let I : ConnectedSpace { x : X // p x ∈ C } := hD
  have hupper : ∃ x, a ≤ f x := by
    rcases hhi with ⟨x, hx⟩
    exact ⟨x, le_trans (le_of_lt hab) hx⟩
  have hvalue : a ∈ Set.range f := by
    exact mem_range_of_exists_le_of_exists_ge hf hlo hupper
  rcases hvalue with ⟨x, hx⟩
  refine ⟨x, ?, ?⟩
  · exact hx.symm ▸ le_rfl
  · exact hx.symm ▸ le_of_lt hab
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Munkres_exercise_23_11__me__ee8e2e02__x__Rudin_exercise_4_3__me.xh.mh__c65fa6ef

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint c65fa6ef4ca9de06

4.226 226. quotientMap_image_connectedComponent_eq_univ

The problem

Let $p : X \rightarrow Y$ be a quotient map with connected fibers, and suppose Y is connected. Show that the image under p of the connected component containing any $x \in X$ is all of Y .

Lean statement

```
theorem quotientMap_image_connectedComponent_eq_univ {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (hq : IsQuotientMap p)
  (hY : ConnectedSpace Y) (hX : ∀ y : Y, IsConnected (p ⁻¹' {y})) (x : X) :
  p '' connectedComponent x = Set.univ
```

Parent. Let $p : X \rightarrow Y$ be a quotient map. Show that if each set $p^{-1}(\{y\})$ is connected, and if Y is connected, then X is connected.

Parent in Lean

```
theorem Munkres_exercise_23_11 {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (hq : IsQuotientMap p)
  (hY : ConnectedSpace Y) (hX : ∀ y : Y, IsConnected (p ⁻¹' {y})) :
  ConnectedSpace X := by
```

What it contributes. Preserves the parent’s quotient-map, connected-base, and connected-fiber setup; the child consumes the connected-component checkpoint to prove a stronger image-level conclusion.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from global connectedness of X to the substantive characterization that every connected component maps onto all of Y . Its proof requires the quotient-map connected-component image theorem and connectedness of Y , neither of which is supplied by recalling the parent’s proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens connectedness of X to the pointwise characterization $p '' \text{connectedComponent } x = \text{Set.univ}$. Its proof requires the quotient-map image-of-component theorem and connected-component collapse in Y , reasoning not supplied by the parent’s proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from global connectedness of X to a component-image characterization, requiring the distinct mathematical facts that quotient maps preserve connected components and that every component of connected Y is universal. Memorizing the parent’s open-cover proof would not substantially solve this equality.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2181 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem quotientMap_image_connectedComponent_eq_univ.extracted_1_1.{u_2, u_1} {X : Type u_1} {Y : Type u_2}
  [inst : TopologicalSpace X] [inst_1 : TopologicalSpace Y] (p : X → Y) (hq : IsQuotientMap p) (hY : ConnectedSpace Y)
  (hX : ∀ (y : Y), IsConnected (p ⁻¹ {y})) (x : X) : p '' connectedComponent x = univ
```

Read back by a model that saw only the Lean. *Let X and Y be topological spaces, let $p : X \rightarrow Y$ be a quotient map, assume that Y is connected and that for every $y \in Y$ the fiber $p^{-1}(\{y\})$ is connected. Then, for every $x \in X$, the image under p of the connected component of x is all of Y .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace Topology

theorem quotientMap_image_connectedComponent_eq_univ {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (hq : IsQuotientMap p)
  (hY : ConnectedSpace Y) (hX : ∀ y : Y, IsConnected (p ⁻¹ {y})) (x : X) :
  p '' connectedComponent x = Set.univ := by
  have himage : p '' connectedComponent x = connectedComponent (p x) := by
    exact hq.image_connectedComponent hX x
  have hcomponent : connectedComponent (p x) = (Set.univ : Set Y) := by
    exact PreconnectedSpace.connectedComponent_eq_univ (p x)
  exact himage.trans hcomponent
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_23_11__mh__4b02c9a8

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 4b02c9a8e68032af

4.227 227. connected_preimage_of_closed_connected

The problem

Let $p : X \rightarrow Y$ be a quotient map whose fibers are connected. Show that the inverse image of every closed connected subset of Y is connected.

Lean statement

```
theorem connected_preimage_of_closed_connected {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (hq : IsQuotientMap p)
  (hX : ∀ y : Y, IsConnected (p ⁻¹' ({y} : Set Y))) :
  ∀ t : Set Y, IsClosed t → IsConnected t → IsConnected (p ⁻¹' t)
```

Parent. Let $p : X \rightarrow Y$ be a quotient map. Show that if each set $p^{-1}(\{y\})$ is connected, and if Y is connected, then X is connected.

Parent in Lean

```
theorem Munkres_exercise_23_11 {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (hq : IsQuotientMap p)
  (hY : ConnectedSpace Y) (hX : ∀ y : Y, IsConnected (p ⁻¹' {y})) :
  ConnectedSpace X := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child removes the global connectedness hypothesis on Y and proves a parametrized preservation theorem for every closed connected subset t , rather than merely deriving connectedness of X from the special case $t = Y$. Its proof uses the stronger coinducing preimage lemma, so recalling the parent's proof does not solve the new obligation.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from global connectedness of X to connectedness of the preimage of every closed connected subset of Y . Its proof uses the quotient map's coinducing property and a closed-set preimage theorem, which the parent's open-cover argument does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child substantially generalizes connectedness from the whole space to the preimage of every closed connected subset, requiring the quotient-map preimage theorem rather than the parent's global cover argument. The parent's hypotheses on the codomain are removed, and the conclusion is genuinely set-level.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2815 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem connected_preimage_of_closed_connected.extracted_1_1 : ∀ {X : Type u_1} {Y : Type u_2}
  [inst : TopologicalSpace X] [inst_1 : TopologicalSpace Y] (p : X → Y),
  IsQuotientMap p →
  (∀ (y : Y), IsConnected (p-1 {y})) → ∀ (t : Set Y), IsClosed t → IsConnected t → IsConnected (p-1 t)
```

Read back by a model that saw only the Lean. *For any topological spaces X and Y and any function $p : X \rightarrow Y$, if p is a quotient map and the fiber $p^{-1}(\{y\})$ is connected for every $y \in Y$, then for every closed and connected subset t of Y , the preimage $p^{-1}(t)$ is connected.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace Topology

theorem connected_preimage_of_closed_connected {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (hq : IsQuotientMap p)
  (hX : ∀ y : Y, IsConnected (p-1 ({y} : Set Y))) :
  ∀ t : Set Y, IsClosed t → IsConnected t → IsConnected (p-1 t) := by
  intro t ht htc
  have hco : IsCoinducing p := hq.isCoinducing
  have hpre : IsConnected (p-1 t) :=
    hco.isConnected_preimage_of_isClosed hX ht htc
  exact hpre
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_23_11__mh__ddc293ff

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint ddc293ff8c2706b4

4.228 228. connected_bounded_trace_prefix

The problem

Let (A_n) be connected subspaces and let f , g , and h be uniformly continuous. If, for every $n < N$, A_{n+1} intersects $A_0 \cup \dots \cup A_n$, show that $\bigcup_{n=0}^N \{(g(f(x)), h(g(f(x)))) : x \in A_n\}$ is connected.

Lean statement

```
theorem connected_bounded_trace_prefix
  {α γ δ : Type*} [UniformSpace α] [UniformSpace γ] [UniformSpace δ]
  {A : ℕ → Set α} {f : α → γ} {g : γ → γ} {h : γ → δ}
  (N : ℕ)
  (hA : ∀ n, IsConnected (A n))
  (hOverlap : ∀ n, n < N → (A (n + 1) ∩ ⋃ k ≤ n, A k).Nonempty)
  (hf : UniformContinuous f) (hg : UniformContinuous g) (hh : UniformContinuous h) :
  IsConnected (⋃ n ≤ N, (fun x => (g (f x), h (g (f x)))) '' A n)
```

Parent. Let (A_n) be connected subspaces of a uniform space, and suppose each A_n intersects the union of its predecessors. If f , g , and h are uniformly continuous, show that the union of the traced sets $\{(g(f(x)), h(g(f(x)))) : x \in A_n\}$ is connected.

Parent in Lean

```
theorem connected_adjacent_trace_iUnion
  {α γ δ : Type*} [UniformSpace α] [UniformSpace γ] [UniformSpace δ]
  {A : ℕ → Set α} {f : α → γ} {g : γ → γ} {h : γ → δ}
  (hA : ∀ n, IsConnected (A n))
  (hPred : ∀ n, n ≠ 0 → (A n ∩ ⋃ k < n, A k).Nonempty)
  (hf : UniformContinuous f) (hg : UniformContinuous g) (hh : UniformContinuous h) :
  IsConnected (⋃ n, (fun x => (g (f x), h (g (f x)))) '' A n)
```

What it contributes. Retains the parent's connected family, three-stage uniformly continuous trace, and overlap-witness transport, while replacing the terminal predecessor-union conclusion with induction over a bounded prefix.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the infinite-union theorem into a finite-prefix characterization and uses induction to maintain connectedness while adjoining each successive image. Its bounded overlap hypothesis and finite-union equality require reasoning absent from the parent's global reachability argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces the parent's infinite predecessor-connectedness argument with a bounded induction over finite prefixes, using a new union decomposition and a stepwise overlap invariant. This is substantive reasoning that the parent's proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the infinite predecessor-union theorem into a finite-prefix result under only bounded overlap hypotheses. Its induction must establish connectedness of each finite union and prove the prefix decomposition, reasoning not supplied by the parent's infinite reachability argument.

Check	By	Result
statement_type_check	lean	type-checks

Check	By	Result
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2190 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem connected_bounded_trace_prefix.extracted_1_1.{u_3, u_2, u_1} {α : Type u_1} {γ : Type u_2} {δ : Type u_3}
  [inst : UniformSpace α] [inst_1 : UniformSpace γ] [inst_2 : UniformSpace δ] {A : ℕ → Set α} {f : α → γ} {g : γ → γ}
  {h : γ → δ} (N : ℕ) (hA : ∀ (n : ℕ), IsConnected (A n))
  (hOverlap : ∀ n < N, (A (n + 1) ∩ U k ≤ n, A k).Nonempty) (hf : UniformContinuous f)
  (hg : UniformContinuous g) (hh : UniformContinuous h) :
  IsConnected (U n, U (λ n : ℕ, (fun x => (g (f x), h (g (f x))))) '' A n)
```

Read back by a model that saw only the Lean. Let α , γ , and δ be uniformly structured spaces, let A assign to each natural number n a subset $A(n)$ of α , and let $f : \alpha \rightarrow \gamma$, $g : \gamma \rightarrow \gamma$, and $h : \gamma \rightarrow \delta$ be uniformly continuous functions. For a natural number N , suppose that every $A(n)$ is connected, and that for every $n < N$, the set $A(n+1)$ intersects the union of all $A(k)$ with $k \leq n$. Then the union, over all $n \leq N$, of the image of $A(n)$ under the map $x \mapsto (g(f(x)), h(g(f(x))))$ is connected.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace
open scoped Topology

theorem connected_bounded_trace_prefix
  {α γ δ : Type*} [UniformSpace α] [UniformSpace γ] [UniformSpace δ]
  {A : ℕ → Set α} {f : α → γ} {g : γ → γ} {h : γ → δ}
  (N : ℕ)
  (hA : ∀ n, IsConnected (A n))
  (hOverlap : ∀ n, n < N → (A (n + 1) ∩ U k ≤ n, A k).Nonempty)
  (hf : UniformContinuous f) (hg : UniformContinuous g) (hh : UniformContinuous h) :
  IsConnected (U n ≤ N, (fun x => (g (f x), h (g (f x))))) '' A n := by
  classical
  let T : α → γ × δ := fun x => (g (f x), h (g (f x)))
  have hT : Continuous T := by
    dsimp [T]
    exact ((hg.comp hf).prodMk (hh.comp (hg.comp hf))).continuous
  let B : ℕ → Set (γ × δ) := fun n => T '' A n
  have hB : ∀ n, IsConnected (B n) := by
    intro n
    exact (hA n).image T hT.continuousOn
  let U : ℕ → Set (γ × δ) := fun m => U n ≤ m, B n
  induction N with
  | zero =>
    change IsConnected (U 0)
    simpa [U] using hB 0
  | succ N ih =>
    change IsConnected (U (N + 1))
    have hprev : IsConnected (U N) := by
      apply ih
      intro n hn
      exact hOverlap n (by omega)
    have hw : (U N ∩ B (N + 1)).Nonempty := by
      obtain ⟨x, hxnext, hxprev⟩ := hOverlap N (by omega)
      refine ⟨T x, ?, ⟨x, hxnext, rfl⟩⟩
      rcases Set.mem_iUnion.mp hxprev with ⟨k, hxprev⟩
      rcases Set.mem_iUnion.mp hxprev with ⟨hk, hxk⟩
```

```

    exact Set.mem_iUnion.mpr ⟨k, Set.mem_iUnion.mpr ⟨hk, ⟨x, hxk, rfl⟩⟩⟩
have hu : IsConnected (U N ∪ B (N + 1)) := hprev.union hw (hB (N + 1))
have heq : U (N + 1) = U N ∪ B (N + 1) := by
  ext y
  constructor
  · intro hy
    rcases Set.mem_iUnion.mp hy with ⟨n, hy⟩
    rcases Set.mem_iUnion.mp hy with ⟨hn, hyn⟩
    by_cases hnm : n ≤ N
    · exact Or.inl (Set.mem_iUnion.mpr ⟨n, Set.mem_iUnion.mpr ⟨hnm, hyn⟩⟩)
    · have hne : n = N + 1 := by omega
      exact Or.inr (hne ▸ hyn)
  · intro hy
    rcases hy with hy | hy
    · rcases Set.mem_iUnion.mp hy with ⟨n, hy⟩
      rcases Set.mem_iUnion.mp hy with ⟨hn, hyn⟩
      exact Set.mem_iUnion.mpr ⟨n, Set.mem_iUnion.mpr ⟨le_trans hn (Nat.le_succ N), hyn⟩⟩
    · exact Set.mem_iUnion.mpr ⟨N + 1, Set.mem_iUnion.mpr ⟨le_refl _, hy⟩⟩
rw [heq]
exact hu

```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Munkres_exercise_23_2__me.me.mh__5fcff258

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 5fcff258a72c4753

4.229 229. connected_preimage_expanded_chain

The problem

Let $p : X \rightarrow Y$ be a quotient map. Suppose $\{A_n\}$ is a chain of connected subspaces with consecutive overlaps, and let B be connected and intersect every A_n . If $B \cup \bigcup_n A_n$ is closed and every fiber of p is connected, show that $p^{-1}(B \cup \bigcup_n A_n)$ is connected.

Lean statement

```
theorem connected_preimage_expanded_chain {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (hq : Topology.IsQuotientMap p) {A : ℕ → Set Y} {B : Set Y}
  (hA : ∀ n, IsConnected (A n))

  (hB : IsConnected B)
  (hBA : ∀ n, (B ∩ A n).Nonempty)
  (hclosed : IsClosed (B ∪ ⋃ n, A n))
  (hfib : ∀ y : Y, IsConnected (p ⁻¹' ({y} : Set Y))) :
  IsConnected (p ⁻¹' (B ∪ ⋃ n, A n))
```

Parent. Let $p : X \rightarrow Y$ be a quotient map, and let $\{A_n\}$ be connected subspaces of Y with $A_n \cap A_{n+1} \neq \emptyset$ for every n . If their union is closed and every fiber of p is connected, show that $p^{-1}(\bigcup_n A_n)$ is connected.

Parent in Lean

```
theorem connected_preimage_adjacent_union {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (hq : Topology.IsQuotientMap p) {A : ℕ → Set Y}
  (hA : ∀ n, IsConnected (A n))
  (hAn : ∀ n, A n ∩ A (n + 1) ≠ ∅)
  (hclosed : IsClosed (⋃ n, A n))
  (hfib : ∀ y : Y, IsConnected (p ⁻¹' ({y} : Set Y))) :
  IsConnected (p ⁻¹' (⋃ n, A n))
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child removes the parent's adjacent-overlap hypothesis and instead proves connectedness through a new common hub B, constructing the family $C_n = B \cup A_n$ and identifying its union with the target. The parent's direct chain argument is unavailable without hAn, so the proof obligation genuinely changes.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely expands the connected chain by adjoining an arbitrary connected set B and proving each $B \cup A_n$ is connected via intersection hypotheses. It also requires a new set-indexed union equality and connectivity argument, so the parent's proof does not suffice by recall.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces adjacent overlap between successive sets with a genuinely different construction: a connected set B intersects every A_n , making each $B \cup A_n$ connected and providing the chain overlaps. The parent's adjacent-chain argument cannot supply this expanded-union proof, and the child also removes hAn.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry

Check	By	Result
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2863 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem connected_preimage_expanded_chain.extracted_1.1.{u_2, u_1} {X : Type u_1} {Y : Type u_2}
  [inst : TopologicalSpace X] [inst_1 : TopologicalSpace Y] (p : X → Y) (hq : Topology.IsQuotientMap p) {A : ℕ → Set Y}
  (B : Set Y) (hA : ∀ (n : ℕ), IsConnected (A n)) (hAn : ∀ (n : ℕ), (A n ∩ A (n + 1)).Nonempty) (hB : IsConnected B)
  (hBA : ∀ (n : ℕ), (B ∩ A n).Nonempty) (hclosed : IsClosed (B ∪ ⋃ n, A n))
  (hfib : ∀ (y : Y), IsConnected (p-1 {y})) : IsConnected (p-1 (B ∪ ⋃ n, A n))
```

Read back by a model that saw only the Lean. *Let X and Y be topological spaces, let $p : X \rightarrow Y$ be a quotient map, and let $A : \mathbb{N} \rightarrow \text{Set } Y$ be a sequence of subsets of Y . Let $B \subseteq Y$. Assume that each $A\ n$ is connected; for every $n \in \mathbb{N}$, the intersection $A\ n \cap A\ (n + 1)$ is nonempty; B is connected; for every $n \in \mathbb{N}$, the intersection $B \cap A\ n$ is nonempty; $B \cup \bigcup_{n \in \mathbb{N}} A\ n$ is closed in Y ; and for every $y \in Y$, the fiber $p^{-1}(\{y\})$ is connected. Then the preimage $p^{-1}(B \cup \bigcup_{n \in \mathbb{N}} A\ n)$ is connected in X .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace
open scoped Topology

theorem connected_preimage_expanded_chain {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y]
  (p : X → Y) (hq : Topology.IsQuotientMap p) {A : ℕ → Set Y} (B : Set Y)
  (hA : ∀ n, IsConnected (A n))

  (hB : IsConnected B)
  (hBA : ∀ n, (B ∩ A n).Nonempty)
  (hclosed : IsClosed (B ∪ ⋃ n, A n))
  (hfib : ∀ y : Y, IsConnected (p-1 {y})) :
  IsConnected (p-1 (B ∪ ⋃ n, A n)) := by
  let C : ℕ → Set Y := fun n => B ∪ A n
  have hC : ∀ n, IsConnected (C n) := by
    intro n
    exact hB.union (hBA n) (hA n)
  have hBne : B.Nonempty := by
    rcases hBA 0 with ⟨y, hyB, hyA⟩
    exact ⟨y, hyB⟩
  have hAdj : ∀ n, (C n ∩ C (Nat.succ n)).Nonempty := by
    intro n
    rcases hBne with ⟨y, hy⟩
    exact ⟨y, ⟨Or.inl hy, Or.inl hy⟩⟩
  have hbase : IsConnected (⋃ n, C n) := by
    apply IsConnected.iUnion_of_chain hC
    intro n
    exact hAdj n
  have hEq : (⋃ n, C n) = B ∪ ⋃ n, A n := by
    ext y
    constructor
    · intro hy
      rcases Set.mem_iUnion.mp hy with ⟨n, hn⟩
      rcases hn with hn | hn
      · exact Or.inl hn
      · exact Or.inr (Set.mem_iUnion.mpr ⟨n, hn⟩)
    · intro hy
      rcases hy with hy | hy
      · exact Set.mem_iUnion.mpr ⟨0, Or.inl hy⟩
```

```

    · rcases Set.mem_iUnion.mp hy with ⟨n, hn⟩
      exact Set.mem_iUnion.mpr ⟨n, Or.inr hn⟩
have hclosedC : IsClosed (U n, C n) := by
  rw [hEq]
  exact hclosed
have hpre : IsConnected (p ⁻¹' (U n, C n)) := by
  exact hq.isCoinducing.isConnected_preimage_of_isClosed hfib hclosedC hbase
rw [← hEq]
exact hpre

```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Munkres_exercise_23_2__me.mh__467e4adc

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint a2f4814a992e9ee7

4.230 230. connected_iUnion_of_generated_overlap

The problem

Let $\{A_i\}_{i \in I}$ be connected subspaces of a topological space. Suppose a relation on I links any two related indices through a nonempty intersection, and any two indices are connected by a finite chain of such relations. Show that $\bigcup_{i \in I} A_i$ is connected.

Lean statement

```
theorem connected_iUnion_of_generated_overlap {X : Type*} [TopologicalSpace X] {ι : Type*} [Nonempty ι]
  {A : ι → Set X} {R : ι → ι → Prop}
  (hA : ∀ i, IsConnected (A i))
  (hR : ∀ i j, R i j → (A i ∩ A j).Nonempty)
  (hChain : ∀ i j, Relation.ReflTransGen R i j) :
  IsConnected (⋃ i, A i)
```

Parent. Let $\{A_n\}$ be a sequence of connected subspaces of X , such that $A_n \cap A_{n+1} \neq \emptyset$ for all n . Show that $\bigcup A_n$ is connected.

Parent in Lean

```
theorem Munkres_exercise_23_2 {X : Type*}
  [TopologicalSpace X] {A : ℕ → Set X} (hA : ∀ n, IsConnected (A n))
  (hAn : ∀ n, A n ∩ A (n + 1) ≠ ∅) :
  IsConnected (⋃ n, A n) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child generalizes the natural-number chain to an arbitrary nonempty index type and an arbitrary relation whose reflexive-transitive closure connects every pair. Its proof uses a general connected-union theorem and requires transporting reachability through intersection overlap, which the parent's successor-chain argument does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes the consecutive-natural-index argument to an arbitrary nonempty index type and an abstract relation, requiring an induction that transfers relational reachability into overlap reachability. The parent's successor-chain construction cannot prove this theorem directly, so recalling the parent is insufficient.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes consecutive-index overlap to an arbitrary nonempty index type and an arbitrary generating relation. Its proof must transport relation reachability into overlap reachability via induction, which the parent's successor-chain argument does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2839 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	MISMATCH: Statement B assumes the index type is nonempty, while Statement A does not explicitly require the index set to be nonempty.

Goal round-trip: what Lean elaborated

```
theorem connected_iUnion_of_generated_overlap.extracted_1_1.{u_2, u_1} {X : Type u_1} [inst : TopologicalSpace X]
  {ι : Type u_2} [inst_1 : Nonempty ι] {A : ι → Set X} {R : ι → ι → Prop} (hA : ∀ (i : ι), IsConnected (A i))
  (hR : ∀ (i j : ι), R i j → (A i ∩ A j).Nonempty) (hChain : ∀ (i j : ι), Relation.ReflTransGen R i j) :
  IsConnected (U i, A i)
```

Read back by a model that saw only the Lean. *Let X be a topological space, let ι be a nonempty type, let A assign to each $i \in \iota$ a subset $A(i)$ of X , and let R be a binary relation on ι . Assume that each $A(i)$ is connected; whenever $R(i,j)$ holds, the intersection $A(i) \cap A(j)$ is nonempty; and for every $i,j \in \iota$, j is reachable from i by the reflexive transitive closure of R . Then the union $\bigcup \{i \in \iota \mid A(i)\}$ is connected.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem connected_iUnion_of_generated_overlap {X : Type*} [TopologicalSpace X] {ι : Type*} [Nonempty ι]
  {A : ι → Set X} {R : ι → ι → Prop}
  (hA : ∀ i, IsConnected (A i))
  (hR : ∀ i j, R i j → (A i ∩ A j).Nonempty)
  (hChain : ∀ i j, Relation.ReflTransGen R i j) :
  IsConnected (U i, A i) := by
  have hOverlap : ∀ i j, Relation.ReflTransGen (fun i j => (A i ∩ A j).Nonempty) i j := by
    intro i j
    induction hChain i j with
    | refl => exact Relation.ReflTransGen.refl
    | tail hxy hyz ih => exact ih.tail (hR _ _ hyz)
  exact IsConnected.iUnion_of_reflTransGen hA hOverlap
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_23_2__mh__5db0f6a7

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 5db0f6a772de6f26

4.231 231. connected_iUnion_closure

The problem

Let $\{A_n\}$ be a sequence of connected subsets of a topological space X , such that $A_n \cap A_{n+1} \neq \emptyset$ for all n . Show that $\bigcup_n \overline{A_n}$ is connected.

Lean statement

```
theorem connected_iUnion_closure {X : Type*} [TopologicalSpace X] {A : ℕ → Set X} (hA : ∀ n, IsConnected (A n)) (hAn : ∀ n, A n ∩ A (n + 1) ≠ ∅) : IsConnected (⋃ n, closure (A n))
```

Parent. Let $\{A_n\}$ be a sequence of connected subspaces of X , such that $A_n \cap A_{n+1} \neq \emptyset$ for all n . Show that $\bigcup A_n$ is connected.

Parent in Lean

```
theorem Munkres_exercise_23_2 {X : Type*} [TopologicalSpace X] {A : ℕ → Set X} (hA : ∀ n, IsConnected (A n)) (hAn : ∀ n, A n ∩ A (n + 1) ≠ ∅) : IsConnected (⋃ n, A n) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely changes the sets being unioned to their closures, requiring preservation of connectedness under closure and transport of adjacent intersection witnesses into the closures. The parent's union-connectedness proof does not supply these closure-specific steps.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the target to a union of closures and requires new reasoning that closures preserve connectedness and preserve adjacent intersection witnesses via `subset_closure`. Although it reuses the chain argument, the parent proof does not supply these closure-specific obligations.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the union members to their closures, requiring separate reasoning that connectedness is preserved by closure and that the original overlap witnesses remain in both closures. This is not supplied by the parent's chain argument or obtainable by simply recalling its proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1941 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem connected_iUnion_closure.extracted_1_1.{u_1} {X : Type u_1} [inst : TopologicalSpace X] {A : ℕ → Set X}
(hA : ∀ (n : ℕ), IsConnected (A n)) (hAn : ∀ (n : ℕ), A n ∩ A (n + 1) ≠ ∅) : IsConnected (U n, closure (A n))
```

Read back by a model that saw only the Lean. *Let X be a topological space and let $(A_n)_{n \in \mathbb{N}}$ be a sequence of subsets of X . Assume that each A_n is connected and that $A_n \cap A_{n+1}$ is nonempty for every $n \in \mathbb{N}$. Then the union $\bigcup_{n \in \mathbb{N}} \overline{A_n}$ of their closures is connected.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Set
open Filter Set TopologicalSpace
open scoped Topology

theorem connected_iUnion_closure {X : Type*} [TopologicalSpace X] {A : ℕ → Set X}
(hA : ∀ n, IsConnected (A n))
(hAn : ∀ n, A n ∩ A (n + 1) ≠ ∅) :
IsConnected (U n, closure (A n)) := by
have hConnected : ∀ n, IsConnected (closure (A n)) := by
  intro n
  exact (hA n).closure
have hAdjacent : ∀ n, (closure (A n) ∩ closure (A (n + 1))).Nonempty := by
  intro n
  rcases (nonempty_iff_ne_empty.mpr (hAn n)) with ⟨x, hx₁, hx₂⟩
  exact ⟨x, subset_closure hx₁, subset_closure hx₂⟩
have hChain : ∀ n, (closure (A n) ∩ closure (A (Nat.succ n))).Nonempty := by
  intro n
  simpa [Nat.succ_eq_add_one] using hAdjacent n
exact IsConnected.iUnion_of_chain hConnected hChain
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_23_2__mh__65acdd42

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 8c85080e91e6cb12

4.232 232. continuous_fixed_point_on_interval

The problem

Let $f : \mathbb{R} \rightarrow \mathbb{R}$ be continuous, and let $a \leq b$. If $a \leq f(a)$ and $f(b) \leq b$, show that there is some $x \in [a, b]$ such that $f(x) = x$.

Lean statement

```
theorem continuous_fixed_point_on_interval (f : ℝ → ℝ) (hf : Continuous f) (a b : ℝ)
  (hab : a ≤ b) (ha : a ≤ f a) (hb : f b ≤ b) :
  ∃ x ∈ Set.Icc a b, f x = x
```

Parent. Let $f : X \rightarrow X$ be continuous. Show that if $X = [0, 1]$, there is a point x such that $f(x) = x$. (The point x is called a fixed point of f .)

Parent in Lean

```
theorem Munkres_exercise_24_3a (f : I → I) (hf : Continuous f) :
  ∃ (x : I), f x = x := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the fixed-point theorem from self-maps of $[0,1]$ to arbitrary continuous real functions on any interval, with endpoint inequalities replacing the codomain hypothesis. The parent's endpoint argument for a self-map does not directly solve this broader claim.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the fixed-point result from self-maps of $[0,1]$ to arbitrary continuous real functions on $[a,b]$ satisfying endpoint inequalities. Its proof uses a new interval fixed-point theorem and does not reduce to recalling the parent's subtype-valued argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child substantially generalizes the fixed-point result from continuous self-maps of $[0,1]$ to arbitrary continuous real functions on $[a,b]$ satisfying endpoint inequalities. Its proof uses a general interval fixed-point theorem and requires reasoning absent from the parent's subtype-specific endpoint argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2344 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem continuous_fixed_point_on_interval.extracted_1_1 (f : ℝ → ℝ) (hf : Continuous f) (a b : ℝ) (hab : a ≤ b)
(ha : a ≤ f a) (hb : f b ≤ b) : ∃ x ∈ Icc a b, f x = x
```

Read back by a model that saw only the Lean. Let $f: \mathbb{R} \rightarrow \mathbb{R}$ be continuous, and let $a, b \in \mathbb{R}$ satisfy $a \leq b$, $a \leq f(a)$, and $f(b) \leq b$. Then there exists $x \in [a, b]$ such that $f(x)=x$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem continuous_fixed_point_on_interval (f : ℝ → ℝ) (hf : Continuous f) (a b : ℝ)
(hab : a ≤ b) (ha : a ≤ f a) (hb : f b ≤ b) :
  ∃ x ∈ Set.Icc a b, f x = x := by
  have hcont : ContinuousOn f (Set.Icc a b) := hf.continuousOn
  obtain ⟨x, hx, hfixed⟩ :=
    exists_mem_Icc_isFixedPt hcont hab ha hb
  have hxeq : f x = x := hfixed.eq
  exact ⟨x, hx, hxeq⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_24_3a__mh__6885d5d8

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 6885d5d803bd17ef

4.233 233. unique_fixed_point_interval

The problem

Let $f : [0, 1] \rightarrow [0, 1]$ be continuous, and suppose that for distinct $x, y \in [0, 1]$, the distance between $f(x)$ and $f(y)$ is strictly smaller than the distance between x and y . Prove that f has a unique fixed point.

Lean statement

```
theorem unique_fixed_point_interval (f : {x : ℝ // x ∈ Set.Icc 0 1} → {x : ℝ // x ∈ Set.Icc 0 1}) (hf : Continuous f) (hcontract : ∀ x y,
  ↪ x ≠ y → dist (f x) (f y) < dist x y) : ∃! x, f x = x
```

Parent. Let $f : X \rightarrow X$ be continuous. Show that if $X = [0, 1]$, there is a point x such that $f(x) = x$. (The point x is called a fixed point of f .)

Parent in Lean

```
theorem Munkres_exercise_24_3a (f : I → I) (hf : Continuous f) :
  ∃ (x : I), f x = x := by
```

What it contributes. Supplies the continuous auxiliary function $g(x)=f(x)-x$, endpoint sign bounds, and intermediate-value existence argument; the child adds a separate uniqueness contradiction.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child adds a genuine uniqueness argument: strict distance shrinking contradicts two distinct fixed points, while continuity still supplies existence. This is substantive reasoning absent from the parent and is not a forced wrapper or decoration.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens existence to uniqueness by adding a strict distance-shrinking hypothesis and proving that any two fixed points must coincide. The parent's intermediate-value argument supplies existence, but cannot supply the new uniqueness argument.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely strengthens fixed-point existence to uniqueness by adding a strict distance-shrinking hypothesis and proving distinct fixed points impossible. The parent's interval/continuity argument supplies existence but does not supply this uniqueness reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2878 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem unique_fixed_point_interval.extracted_1_1 (f : { x // x ∈ Icc 0 1 } → { x // x ∈ Icc 0 1 }) (hf : Continuous f)
(hcontract : ∀ (x y : { x // x ∈ Icc 0 1 }), x ≠ y → dist (f x) (f y) < dist x y) : ∃! x, f x = x
```

Read back by a model that saw only the Lean. Let f be a function from the real interval $[0,1]$ to itself. Assume that f is continuous and that for any distinct x,y in $[0,1]$, the distance between $f(x)$ and $f(y)$ is strictly less than the distance between x and y . Then there exists exactly one x in $[0,1]$ such that $f(x)=x$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace
open scoped Topology

theorem unique_fixed_point_interval (f : {x : ℝ // x ∈ Set.Icc 0 1} → {x : ℝ // x ∈ Set.Icc 0 1}) (hf : Continuous f) (hcontract : ∀ x y,
  x ≠ y → dist (f x) (f y) < dist x y) : ∃! x, f x = x := by
  classical
  let a : {x : ℝ // x ∈ Set.Icc 0 1} := ⟨0, by exact ⟨le_rfl, zero_le_one⟩⟩
  let b : {x : ℝ // x ∈ Set.Icc 0 1} := ⟨1, by exact ⟨zero_le_one, le_rfl⟩⟩
  let g : {x : ℝ // x ∈ Set.Icc 0 1} → ℝ := fun x => (f x : ℝ) - (x : ℝ)
  have hg_cont : Continuous g := by
    have h_toReal : Continuous fun x : {x : ℝ // x ∈ Set.Icc 0 1} => (f x : ℝ) := continuous_subtype_val.comp hf
    exact h_toReal.sub continuous_subtype_val
  have hga_nonneg : 0 ≤ g a := by
    have hfa_mem : ((f a : ℝ) ∈ Set.Icc 0 1) := (f a).property
    have hfa_lower : 0 ≤ (f a : ℝ) := (Set.mem_Icc.mp hfa_mem).1
    simp [g, a] using hfa_lower
  have hgb_nonpos : g b ≤ 0 := by
    have hfb_mem : ((f b : ℝ) ∈ Set.Icc 0 1) := (f b).property
    have hfb_upper : (f b : ℝ) ≤ 1 := (Set.mem_Icc.mp hfb_mem).2
    have h : (f b : ℝ) - 1 ≤ 0 := sub_nonpos.mpr hfb_upper
    simp [g, b] using h
  have hZeroInRange : (0 : ℝ) ∈ Set.range g := mem_range_of_exists_le_of_exists_ge hg_cont ⟨b, hgb_nonpos⟩ ⟨a, hga_nonneg⟩
  rcases hZeroInRange with ⟨x, hx⟩
  have hRealEq : (f x : ℝ) = (x : ℝ) := by
    have h : (f x : ℝ) - (x : ℝ) = 0 := by simp [g] using hx
    exact sub_eq_zero.mp h
  have hFixed : f x = x := by
    apply Subtype.ext
    simp using hRealEq
  refine ⟨x, hFixed, ?_⟩
  intro y hy
  by_contra hne
  have hstrict := hcontract x y (Ne.symm hne)
  rw [hFixed, hy] at hstrict
  exact (lt_irrefl _ hstrict)
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_24_3a__mh__9559776e

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 9559776e2b164cd0

4.234 234. compact_pair_observable_localization

The problem

Let K be a compact subset of an open set U in a Hausdorff locally compact space, and let $f, g : X \rightarrow \mathbb{R}$ be continuous. Show that there are open sets V, W with $K \subseteq V$, $\overline{V} \subseteq W$, $\overline{W}$ compact, $\overline{W} \subseteq U$, and the set of pairs $(f(x), g(x))$ for $x \in \overline{W}$ compact.

Lean statement

```
theorem compact_pair_observable_localization {X : Type*} [TopologicalSpace X] [T2Space X] [LocallyCompactSpace X]
  (K U : Set X) (f g : X → ℝ) (hK : IsCompact K) (hU : IsOpen U)
  (hKU : K ⊆ U) (hf : Continuous f) (hg : Continuous g) :
  ∃ V W : Set X, IsOpen V ∧ K ⊆ V ∧ closure V ⊆ W ∧
    IsOpen W ∧ IsCompact (closure W) ∧ closure W ⊆ U ∧
    IsCompact ((fun x => (f x, g x)) '' closure W)
```

Parent. Let K be a compact subset of an open set U in a Hausdorff locally compact space, and let $f : X \rightarrow \mathbb{R}$ be continuous. Show that there are open sets V, W with $K \subseteq V$, $\overline{V} \subseteq W$, $\overline{W}$ compact, $\overline{W} \subseteq U$, and $f(\overline{W})$ compact.

Parent in Lean

```
theorem compact_observable_localization {X : Type*} [TopologicalSpace X] [T2Space X] [LocallyCompactSpace X]
  (K U : Set X) (f : X → ℝ) (hK : IsCompact K) (hU : IsOpen U)
  (hKU : K ⊆ U) (hf : Continuous f) :
  ∃ V W : Set X, IsOpen V ∧ K ⊆ V ∧ closure V ⊆ W ∧
    IsOpen W ∧ IsCompact (closure W) ∧ closure W ⊆ U ∧
    IsCompact (f '' closure W)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from compactness of a scalar image to compactness of a paired image and requires proving continuity of the product map $x \mapsto (f x, g x)$. This genuinely adds a new proof obligation not supplied by the parent's localization argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the compact-image obligation from a scalar continuous image to the image of a pair-valued map. Proving continuity of the product map and applying compactness to it requires reasoning absent from the parent's proof, and the new hypothesis is essential.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the compact-image conclusion from a real-valued image to a pair-valued image. Proving compactness now requires constructing continuity of the paired map from both hypotheses, which the parent's proof does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing

Check	By	Result
corpus_dedup	hash	unique against 2357 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem compact_pair_observable_localization.extracted_1_1.{u_1} {X : Type u_1} [inst : TopologicalSpace X]
[inst_1 : T2Space X] [inst_2 : LocallyCompactSpace X] (K U : Set X) (f g : X → ℝ) (hK : IsCompact K) (hU : IsOpen U)
(hKU : K ⊆ U) (hf : Continuous f) (hg : Continuous g) :
  ∃ V W,
    IsOpen V ∧
      K ⊆ V ∧
        closure V ⊆ W ∧
          IsOpen W ∧ IsCompact (closure W) ∧ closure W ⊆ U ∧ IsCompact ((fun x => (f x, g x)) '' closure W)
```

Read back by a model that saw only the Lean. *For every topological space X that is Hausdorff and locally compact, every compact subset K of X , every open subset U of X with $K \subseteq U$, and every pair of continuous functions $f, g : X \rightarrow \mathbb{R}$, there exist subsets $V, W \subseteq X$ such that V and W are open, $K \subseteq V$, the closure of V is contained in W , the closure of W is compact, the closure of W is contained in U , and the image of the closure of W under the map $x \mapsto (f(x), g(x))$ is compact.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace
open scoped Topology

theorem compact_pair_observable_localization {X : Type*} [TopologicalSpace X] [T2Space X] [LocallyCompactSpace X]
(K U : Set X) (f g : X → ℝ) (hK : IsCompact K) (hU : IsOpen U)
(hKU : K ⊆ U) (hf : Continuous f) (hg : Continuous g) :
  ∃ V W : Set X, IsOpen V ∧ K ⊆ V ∧ closure V ⊆ W ∧
    IsOpen W ∧ IsCompact (closure W) ∧ closure W ⊆ U ∧
    IsCompact ((fun x => (f x, g x)) '' closure W) := by
  have hFirst : ∃ V : Set X, IsOpen V ∧ K ⊆ V ∧ closure V ⊆ U ∧ IsCompact (closure V) := by
    exact exists_open_between_and_isCompact_closure hK hU hKU
  obtain ⟨V, hVopen, hKV, hVsub, hVcompact⟩ := hFirst
  have hSecond : ∃ W : Set X, IsOpen W ∧ closure V ⊆ W ∧ closure W ⊆ U ∧ IsCompact (closure W) := by
    exact exists_open_between_and_isCompact_closure hVcompact hU hVsub
  obtain ⟨W, hWopen, hVW, hWsub, hWcompact⟩ := hSecond
  have hPairCont : Continuous (fun x => (f x, g x)) := by
    exact continuous_prodMk.mpr ⟨hf, hg⟩
  have hPairImageCompact : IsCompact ((fun x => (f x, g x)) '' closure W) := by
    exact hWcompact.image hPairCont
  exact ⟨V, W, hVopen, hKV, hVW, hWopen, hWcompact, hWsub, hPairImageCompact⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Munkres_exercise_29_10__me.mh.mh__1f2f481b

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 1f2f481bb6f693db

4.235 235. compact_observable_range_buffer

The problem

Let K be a compact subset of an open set U in a Hausdorff locally compact space, and let $f : X \rightarrow \mathbb{R}$ be continuous. If $f(K) \subset (a, b)$, show that there are open sets V, W with $K \subseteq V$, $\overline{V} \subseteq W$, $\overline{W}$ compact, $\overline{W} \subseteq U$, and $f(\overline{W}) \subset (a, b)$.

Lean statement

```
theorem compact_observable_range_buffer {X : Type*} [TopologicalSpace X] [T2Space X] [LocallyCompactSpace X]
  (K U : Set X) (f : X → ℝ) (a b : ℝ) (hK : IsCompact K) (hU : IsOpen U)
  (hKU : K ⊆ U) (hf : Continuous f) (hbuf : f '' K ⊆ Set.Ioo a b) :
  ∃ V W : Set X, IsOpen V ∧ K ⊆ V ∧ closure V ⊆ W ∧
    IsOpen W ∧ IsCompact (closure W) ∧ closure W ⊆ U ∧
    f '' closure W ⊆ Set.Ioo a b
```

Parent. Let K be a compact subset of an open set U in a Hausdorff locally compact space, and let $f : X \rightarrow \mathbb{R}$ be continuous. Show that there are open sets V, W with $K \subseteq V$, $\overline{V} \subseteq W$, $\overline{W}$ compact, $\overline{W} \subseteq U$, and $f(\overline{W})$ compact.

Parent in Lean

```
theorem compact_observable_localization {X : Type*} [TopologicalSpace X] [T2Space X] [LocallyCompactSpace X]
  (K U : Set X) (f : X → ℝ) (hK : IsCompact K) (hU : IsOpen U)
  (hKU : K ⊆ U) (hf : Continuous f) :
  ∃ V W : Set X, IsOpen V ∧ K ⊆ V ∧ closure V ⊆ W ∧
    IsOpen W ∧ IsCompact (closure W) ∧ closure W ⊆ U ∧
    IsCompact (f '' closure W)
```

What it contributes. Supplied the two-stage nested open localization with compact outer closure; the mutation replaces the parent's decorative compact-image conclusion by localization inside the continuity-preimage range buffer.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child requires genuinely new reasoning: it intersects U with the continuous preimage of the prescribed interval and uses the range-buffer hypothesis to localize K there. The parent's nested-neighborhood and compact-image argument does not establish this stronger range containment.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine range-buffer argument: it intersects U with the preimage of an interval and uses continuity to ensure the entire compact outer closure maps into that interval. This reasoning is absent from the parent's compact-image conclusion and cannot be recovered by recalling the parent's proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine range-buffer specialization: it must intersect the domain with the preimage of the prescribed interval and propagate that constraint through nested compact neighborhoods. The parent's compact-image argument does not supply this preimage and range-containment reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound

Check	By	Result
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2348 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem compact_observable_range_buffer.extracted_1.1.{u_1} {X : Type u_1} [inst : TopologicalSpace X]
[inst_1 : T2Space X] [inst_2 : LocallyCompactSpace X] (K U : Set X) (f : X → ℝ) (a b : ℝ) (hK : IsCompact K)
(hU : IsOpen U) (hKU : K ⊆ U) (hf : Continuous f) (hbuf : f '' K ⊆ Ioo a b) :
  ∃ V W,
    IsOpen V ∧
      K ⊆ V ∧ closure V ⊆ W ∧ IsOpen W ∧ IsCompact (closure W) ∧ closure W ⊆ U ∧ f '' closure W ⊆ Ioo a b
```

Read back by a model that saw only the Lean. *Let X be a topological space that is Hausdorff and locally compact. For sets $K, U \subseteq X$, a function $f : X \rightarrow \mathbb{R}$, and real numbers a, b , suppose K is compact, U is open, $K \subseteq U$, f is continuous, and $f(K) \subseteq (a, b)$. Then there exist sets $V, W \subseteq X$ such that V is open, $K \subseteq V$, the closure of V is contained in W , W is open, the closure of W is compact, the closure of W is contained in U , and $f(\text{closure } W) \subseteq (a, b)$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace
open scoped Topology

theorem compact_observable_range_buffer {X : Type*} [TopologicalSpace X] [T2Space X] [LocallyCompactSpace X]
(K U : Set X) (f : X → ℝ) (a b : ℝ) (hK : IsCompact K) (hU : IsOpen U)
(hKU : K ⊆ U) (hf : Continuous f) (hbuf : f '' K ⊆ Set.Ioo a b) :
  ∃ V W : Set X, IsOpen V ∧ K ⊆ V ∧ closure V ⊆ W ∧
    IsOpen W ∧ IsCompact (closure W) ∧ closure W ⊆ U ∧
      f '' closure W ⊆ Set.Ioo a b := by
  have hDomainOpen : IsOpen (U ∩ f ⁻¹' Set.Ioo a b) := by
    exact hU.inter (isOpen_Ioo.preimage hf)
  have hKDomain : K ⊆ U ∩ f ⁻¹' Set.Ioo a b := by
    intro x hx
    exact ⟨hKU hx, hbuf ⟨x, hx, rfl⟩⟩
  have hFirst : ∃ V : Set X, IsOpen V ∧ K ⊆ V ∧
    closure V ⊆ U ∩ f ⁻¹' Set.Ioo a b ∧ IsCompact (closure V) := by
    exact exists_open_between_and_isCompact_closure hK hDomainOpen hKDomain
  obtain ⟨V, hVopen, hKV, hVsub, hVcompact⟩ := hFirst
  have hSecond : ∃ W : Set X, IsOpen W ∧ closure V ⊆ W ∧
    closure W ⊆ U ∩ f ⁻¹' Set.Ioo a b ∧ IsCompact (closure W) := by
    exact exists_open_between_and_isCompact_closure hVcompact hDomainOpen hVsub
  obtain ⟨W, hWopen, hVW, hWsub, hWcompact⟩ := hSecond
  have hWsubU : closure W ⊆ U := by
    intro x hx
    exact (hWsub hx).1
  have hRange : f '' closure W ⊆ Set.Ioo a b := by
    intro y hy
    rcases hy with ⟨x, hx, rfl⟩
    exact (hWsub hx).2
  exact ⟨V, W, hVopen, hKV, hVW, hWopen, hWcompact, hWsubU, hRange⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Munkres_exercise_29_10__me.mh.mh__1f30fe9a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 1f30fe9a994e1537

4.236 236. compact_open_refinement

The problem

Let K be a compact subset of an open set U in a locally compact regular Hausdorff space. Show that there is an open set V such that $K \subset V$, $\bar{V} \subset U$, and $\bar{V}$ is compact.

Lean statement

```
theorem compact_open_refinement {X : Type*}
  [TopologicalSpace X] [T2Space X] [LocallyCompactSpace X] [RegularSpace X]
  (K U : Set X) (hK : IsCompact K) (hU : IsOpen U) (hKU : K ⊆ U) :
  ∃ V : Set X, IsOpen V ∧ K ⊆ V ∧ closure V ⊆ U ∧ IsCompact (closure V)
```

Parent. Show that if X is a Hausdorff space that is locally compact at the point x , then for each neighborhood U of x , there is a neighborhood V of x such that $\bar{V}$ is compact and $\bar{V} \subset U$.

Parent in Lean

```
theorem Munkres_exercise_29_10 {X : Type*}
  [TopologicalSpace X] [T2Space X] (x : X)
  (hx : ∃ U : Set X, x ∈ U ∧ IsOpen U ∧ (∃ K : Set X, U ⊆ K ∧ IsCompact K))
  (U : Set X) (hU : IsOpen U) (hxU : x ∈ U) :
  ∃ (V : Set X), IsOpen V ∧ x ∈ V ∧ IsCompact (closure V) ∧ closure V ⊆ U := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child replaces refinement around a single point with a simultaneous refinement of an arbitrary compact set K , requiring a compactness-based finite-cover argument and additional regularity structure. Its proof does not reuse the parent's pointwise separation construction and instead establishes a genuinely stronger compact-set theorem.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the pointwise local-compactness conclusion into a compact-set refinement theorem: an arbitrary compact K must be enlarged to an open V whose compact closure remains inside U . This requires a distinct compact-set refinement argument and cannot be obtained by recalling the parent's point-neighborhood construction.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes from a single point to an arbitrary compact set and requires an open neighborhood whose closure remains compact and lies inside the target. This is not recoverable by recalling the parent's pointwise separation-and-compactness proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2932 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem compact_open_refinement.extracted_1_1.{u_1} {X : Type u_1} [inst : TopologicalSpace X] [inst_1 : T2Space X]
[inst_2 : LocallyCompactSpace X] [inst_3 : RegularSpace X] (K U : Set X) (hK : IsCompact K) (hU : IsOpen U)
(hKU : K ⊆ U) : ∃ V, IsOpen V ∧ K ⊆ V ∧ closure V ⊆ U ∧ IsCompact (closure V)
```

Read back by a model that saw only the Lean. *Let X be a topological space that is Hausdorff, locally compact, and regular. For sets $K, U ⊆ X$, suppose K is compact, U is open, and $K ⊆ U$. Then there exists an open set $V ⊆ X$ such that $K ⊆ V$, the closure of V is contained in U , and the closure of V is compact.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Filter Set TopologicalSpace
open scoped Topology

theorem compact_open_refinement {X : Type*}
[TopologicalSpace X] [T2Space X] [LocallyCompactSpace X] [RegularSpace X]
(K U : Set X) (hK : IsCompact K) (hU : IsOpen U) (hKU : K ⊆ U) :
∃ V : Set X, IsOpen V ∧ K ⊆ V ∧ closure V ⊆ U ∧ IsCompact (closure V) := by
have hRefinement := exists_open_between_and_isCompact_closure hK hU hKU
rcases hRefinement with ⟨V, hVopen, hKV, hclosureU, hcompact⟩
have hConclusion :
IsOpen V ∧ K ⊆ V ∧ closure V ⊆ U ∧ IsCompact (closure V) := by
exact ⟨hVopen, hKV, hclosureU, hcompact⟩
exact ⟨V, hConclusion⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_29_10__mh__23d513d8

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 23d513d89cd7c9b1

4.237 237. countable_basis_compact_closure_subset

The problem

Let X be a Hausdorff locally compact space, and let $(B_n)_{n \in \mathbb{N}}$ be a countable open neighborhood basis at x . Show that for every open neighborhood U of x , some B_n has compact closure contained in U .

Lean statement

```
theorem countable_basis_compact_closure_subset {X : Type*} [TopologicalSpace X] [T2Space X] [LocallyCompactSpace X]
  (x : X) (B : ℕ → Set X)
  (hBopen : ∀ n, IsOpen (B n)) (hxB : ∀ n, x ∈ B n)
  (hBbasis : ∀ s ∈ ℳ X, ∃ n, B n ⊆ s)
  (U : Set X) (hU : IsOpen U) (hxU : x ∈ U) :
  ∃ n : ℕ, IsOpen (B n) ∧ x ∈ B n ∧ IsCompact (closure (B n)) ∧ closure (B n) ⊆ U
```

Parent. Show that if X is a Hausdorff space that is locally compact at the point x , then for each neighborhood U of x , there is a neighborhood V of x such that $\bar{V}$ is compact and $\bar{V} \subset U$.

Parent in Lean

```
theorem Munkres_exercise_29_10 {X : Type*}
  [TopologicalSpace X] [T2Space X] (x : X)
  (hx : ∃ U : Set X, x ∈ U ∧ IsOpen U ∧ (∃ K : Set X, U ⊆ K ∧ IsCompact K))
  (U : Set X) (hU : IsOpen U) (hxU : x ∈ U) :
  ∃ (V : Set X), IsOpen V ∧ x ∈ V ∧ IsCompact (closure V) ∧ closure V ⊆ U := by
```

What it contributes. Preserves the Hausdorff/local-compact neighborhood argument while replacing the unrestricted witness by a member of the supplied countable local basis.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child requires a genuinely new basis-refinement argument: after obtaining an open neighborhood with compact closure, one must select an indexed basis element inside it and transfer compactness via closure monotonicity. The parent's direct construction of an open set does not supply this index-selection step, and the local compactness hypothesis replaces the parent's explicit compact-neighborho

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child introduces a countable neighborhood basis and must select an indexed basis element whose closure remains compact and lies in U . This requires a new basis-refinement argument and does not follow by recalling the parent's arbitrary-neighborhood construction.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuinely substantive countable-basis specialization: it must select an indexed basis element contained in an open neighborhood whose compact closure lies in U . The parent's separation-and-finite-subcover argument does not provide this indexed selection, and the child instead uses local compactness plus basis refinement.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound

Check	By	Result
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2892 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem countable_basis_compact_closure_subset.extracted_1.1.{u_1} {X : Type u_1} [inst : TopologicalSpace X]
  [inst_1 : T2Space X] [inst_2 : LocallyCompactSpace X] (x : X) (B : ℕ → Set X) (hBopen : ∀ (n : ℕ), IsOpen (B n))
  (hxB : ∀ (n : ℕ), x ∈ B n) (hBbasis : ∀ s ∈ ℳ x, ∃ n, B n ⊆ s) (U : Set X) (hU : IsOpen U) (hxU : x ∈ U) :
  ∃ n, IsOpen (B n) ∧ x ∈ B n ∧ IsCompact (closure (B n)) ∧ closure (B n) ⊆ U
```

Read back by a model that saw only the Lean. *Let X be a locally compact Hausdorff topological space, let x be a point of X , and let $B : \mathbb{N} \rightarrow \text{Set } X$ be a sequence of subsets of X such that every $B n$ is open, x belongs to every $B n$, and for every neighborhood s of x there exists n with $B n \subseteq s$. Let U be an open subset of X containing x . Then there exists $n \in \mathbb{N}$ such that $B n$ is open, $x \in B n$, the closure of $B n$ is compact, and the closure of $B n$ is contained in U .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Filter Set TopologicalSpace
open scoped Topology

theorem countable_basis_compact_closure_subset {X : Type*} [TopologicalSpace X] [T2Space X] [LocallyCompactSpace X]
  (x : X) (B : ℕ → Set X)
  (hBopen : ∀ n, IsOpen (B n)) (hxB : ∀ n, x ∈ B n)
  (hBbasis : ∀ s ∈ ℳ x, ∃ n, B n ⊆ s)
  (U : Set X) (hU : IsOpen U) (hxU : x ∈ U) :
  ∃ n : ℕ, IsOpen (B n) ∧ x ∈ B n ∧ IsCompact (closure (B n)) ∧ closure (B n) ⊆ U := by
  have hcompact_x : IsCompact ({x} : Set X) := isCompact_singleton
  have hsingletonU : ({x} : Set X) ⊆ U := by
    rintro y rfl
    exact hxU
  obtain ⟨W, hWopen, hsingletonW, hclWsub, hclWcompact⟩ :=
    exists_open_between_and_isCompact_closure hcompact_x hU hsingletonU
  have hxW : x ∈ W := hsingletonW (mem_singleton x)
  have hWnhds : W ∈ ℳ x := hWopen.mem_nhds hxW
  obtain ⟨n, hn⟩ := hBbasis W hWnhds
  have hclsub : closure (B n) ⊆ closure W := closure_mono hn
  have hclU : closure (B n) ⊆ U := hclsub.trans hclWsub
  have hclcompact : IsCompact (closure (B n)) :=
    hclWcompact.of_isClosed_subset isClosed_closure hclsub
  exact ⟨n, hBopen n, hxB n, hclcompact, hclU⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Munkres_exercise_29_10__mh__cb92bce9

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint cb92bce9f8dccfb1

4.238 238. prescribed_pair_exact_support

The problem

Let $x \in \mathbb{R}^n$ and let $i \neq j$ be prescribed coordinates with $x_i \neq 0$ and $x_j \neq 0$. Prove that there is a unit vector orthogonal to x whose support is exactly $\{i, j\}$, with both prescribed coordinates nonzero.

Lean statement

```
theorem prescribed_pair_exact_support
  (n : ℕ)
  (x : EuclideanSpace ℝ (Fin n))
  (i j : Fin n)
  (hi j : i ≠ j)
  (hxi : x i ≠ 0)
  (hxj : x j ≠ 0) :
  ∃ y : EuclideanSpace ℝ (Fin n),
    ‖y‖ = 1 ∧
    ⟨x, y⟩_ℝ = 0 ∧
    {k : Fin n | y k ≠ 0} = ({i, j} : Set (Fin n)) ∧
    y i ≠ 0 ∧
    y j ≠ 0
```

Parent. Let $x \in \mathbb{R}^n$ and let $i \neq j$ be prescribed coordinates. Prove that there is a unit vector supported only on i and j and orthogonal to x .

Parent in Lean

```
theorem prescribed_pair_unit_orthogonal
  (n : ℕ)
  (x : EuclideanSpace ℝ (Fin n))
  (i j : Fin n)
  (hi j : i ≠ j) :
  ∃ y : EuclideanSpace ℝ (Fin n),
    ‖y‖ = 1 ∧
    ⟨x, y⟩_ℝ = 0 ∧
    ∀ k : Fin n, k ≠ i → k ≠ j → y k = 0
```

What it contributes. Reuse the prescribed-pair witness and normalization construction; strengthen the proof with explicit nonvanishing hypotheses and establish exact support plus nonzero values at both selected coordinates.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely specializes to nonzero prescribed coordinates and proves a new exact-support characterization, including both coordinate nonvanishing conditions. The parent supplies normalization and orthogonality, but not the support equality or its required coordinate arguments.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child specializes to nonzero prescribed coordinates and proves the genuinely stronger exact-support characterization, including both coordinates being nonzero. This requires new support and nonvanishing reasoning beyond the parent's merely contained-support construction.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child specializes to the nonzero-coordinate case and strengthens the conclusion to exact support, requiring proofs that both prescribed coordinates remain nonzero and all others vanish. The parent's construction only establishes support containment and allows a zero coordinate, so recalling it does not discharge the new support-equality obligations.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2375 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem prescribed_pair_exact_support.extracted_1_1 (n : ℕ) (x : EuclideanSpace ℝ (Fin n)) (i j : Fin n) (hij : i ≠ j)
  (hxi : x.ofLp i ≠ 0) (hxj : x.ofLp j ≠ 0) :
  ∃ y, ‖y‖ = 1 ∧ ⟨x, y⟩_ℝ = 0 ∧ {k | y.ofLp k ≠ 0} = {i, j} ∧ y.ofLp i ≠ 0 ∧ y.ofLp j ≠ 0
```

Read back by a model that saw only the Lean. *For every natural number n , every vector x in the Euclidean space $\mathbb{R}^n$ indexed by $\text{Fin } n$, and every pair of distinct indices $i, j \in \text{Fin } n$ such that the i -th and j -th coordinates of x are both nonzero, there exists a vector $y \in \mathbb{R}^n$ with Euclidean norm 1, orthogonal to x , whose nonzero coordinates are exactly those indexed by i and j , and whose i -th and j -th coordinates are both nonzero.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open scoped InnerProductSpace
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators InnerProductSpace

theorem prescribed_pair_exact_support
  (n : ℕ)
  (x : EuclideanSpace ℝ (Fin n))
  (i j : Fin n)
  (hij : i ≠ j)
  (hxi : x i ≠ 0)
  (hxj : x j ≠ 0) :
  ∃ y : EuclideanSpace ℝ (Fin n),
    ‖y‖ = 1 ∧
    ⟨x, y⟩_ℝ = 0 ∧
    {k : Fin n | y k ≠ 0} = ({i, j} : Set (Fin n)) ∧
    y i ≠ 0 ∧
    y j ≠ 0 := by
  classical
  let z := EuclideanSpace.single i (x j) - EuclideanSpace.single j (x i)
  have hji : j ≠ i := Ne.symm hij
  have hz_i : z i = x j := by
    simp [z, EuclideanSpace.single_apply, hij, hji]
  have hz_j : z j = -x i := by
    simp [z, EuclideanSpace.single_apply, hij, hji]
  have hz : z ≠ 0 := by
    intro hzero
    have hzero_j := congrArg (fun v => v j) hzero
```

```

have : x i = 0 := by
  simpa [hz_j] using hzero_j
exact hxi this
have hnormz : ‖z‖ ≠ 0 := norm.ne_zero_iff.mpr hz
let y := (‖z‖ : ℝ)⁻¹ • z
have hnorm : ‖y‖ = 1 := by
  simpa [y] using (norm.smul_inv_norm hz)
have horthz : ⟨x, z⟩_R = 0 := by
  simp [z, inner_sub_right, EuclideanSpace.inner_single_right, mul_comm]
have horth : ⟨x, y⟩_R = 0 := by
  change ⟨x, (‖z‖ : ℝ)⁻¹ • z⟩_R = 0
  rw [real_inner_smul_right, horthz, mul_zero]
have hy_i : y i ≠ 0 := by
  simp [y, hz_i, hnormz, hxj]
have hy_j : y j ≠ 0 := by
  simp [y, hz_j, hnormz, hxi]
have hsupport : {k : Fin n | y k ≠ 0} = ({i, j} : Set (Fin n)) := by
  ext k
  constructor
  · intro hk
    change y k ≠ 0 at hk
    by_cases hki : k = i
    · simp [hki]
    · by_cases hkj : k = j
      · simp [hkj]
      · have hyzero : y k = 0 := by
          simp [y, z, EuclideanSpace.single_apply, hki, hkj]
          exact False.elim (hk hyzero)
  · intro hk
    simp only [Set.mem_insert_iff, Set.mem_singleton_iff] at hk
    rcases hk with rfl | rfl
    · exact hy_i
    · exact hy_j
exact ⟨y, hnorm, horth, hsupport, hy_i, hy_j⟩

```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Rudin_exercise_1_18a__me.me.mh__9c4a022c

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 9c4a022cb361de77

4.239 239. irrational_quotient_injective

The problem

For an irrational real number x , division by nonzero rational numbers is injective: if $x/y = x/z$, then $y = z$.

Lean statement

```
theorem irrational_quotient_injective
  (x : ℝ) (hx : Irrational x) :
  Function.Injective (fun y : {q : ℚ // q ≠ 0} => x / (y.1 : ℝ))
```

Parent. Let x be real and y a nonzero rational number. Prove that x/y is irrational if and only if x is irrational.

Parent in Lean

```
theorem irrational_div_rat_iff
  (x : ℝ) (y : ℚ) (h : y ≠ 0) :
  Irrational (x / (y : ℝ)) ↔ Irrational x
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from irrationality preservation to injectivity over nonzero rational divisors. Its proof requires cross-multiplication and cancellation using irrationality, which is not supplied by the parent's equivalence argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes an irrationality equivalence into an injectivity statement and requires a new cancellation argument: cross-multiplication, zero-product reasoning, and use of irrationality to show the denominator difference vanishes. The parent's rational-scaling equivalence does not directly supply this proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the equivalence into an injectivity theorem for division by nonzero rationals. Its proof requires cross-multiplication and cancellation using the irrational numerator's nonzeroness, which is not supplied by the parent's irrationality-transport argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2792 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	MISMATCH: Statement A quantifies over nonzero rational numbers, while Statement B quantifies over natural numbers, omitting noninteger and negative rationals and including 0, where division is undefined.

Goal round-trip: what Lean elaborated

```
theorem irrational_quotient_injective.extracted_1_1 (x : ℝ) (hx : Irrational x) :  
  Function.Injective fun y => x / ↑↑y
```

Read back by a model that saw only the Lean. For every irrational real number x , the function that maps each natural number y to x divided by y , viewed as a real number, is injective; that is, whenever x divided by y equals x divided by z , we have $y = z$.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open Topology Filter Real Complex TopologicalSpace Finset  
open scoped BigOperators  
  
theorem irrational_quotient_injective  
  (x : ℝ) (hx : Irrational x) :  
    Function.Injective (fun y : {q : ℚ // q ≠ 0} => x / (y.1 : ℝ)) := by  
  intro y z hEq  
  apply Subtype.ext  
  have hy : (y.1 : ℝ) ≠ 0 := by  
    exact_mod_cast y.property  
  have hz : (z.1 : ℝ) ≠ 0 := by  
    exact_mod_cast z.property  
  have hcross : x * (z.1 : ℝ) = x * (y.1 : ℝ) :=  
    (div_eq_div_iff hy hz).mp hEq  
  have hprod : x * ((z.1 : ℝ) - (y.1 : ℝ)) = 0 := by  
    nlinarith [hcross]  
  have hdiff : (z.1 : ℝ) - (y.1 : ℝ) = 0 :=  
    (mul_eq_zero.mp hprod).resolve_left hx.ne_zero  
  have hcast : (z.1 : ℝ) = (y.1 : ℝ) :=  
    sub_eq_zero.mp hdiff  
  exact_mod_cast hcast.symm
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Rudin_exercise_1_1b__mh.ms.mh__10b2735a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 10b2735aeff611d2

4.240 240. irrational_mul_ratCast_iff_of_irrational

The problem

Let x be irrational and r rational. Prove that rx is irrational if and only if $r \neq 0$.

Lean statement

```
theorem irrational_mul_ratCast_iff_of_irrational (x : ℝ) (r : ℚ) (hx : Irrational x) : Irrational (x * r) ↔ r ≠ 0
```

Parent. If r is rational ($r \neq 0$) and x is irrational, prove that rx is irrational.

Parent in Lean

```
theorem Rudin_exercise_1_1b
(x : ℝ)
(y : ℚ)
(h : y ≠ 0)
: (Irrational x) -> Irrational (x * y) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the one-way preservation result into an iff and requires a new converse argument: if the rational multiplier is zero, the product is rational, contradicting its irrationality. The parent's 'mul_ratCast' application supplies only the forward direction.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the implication into an iff and adds the substantive converse that irrationality of the product forces the rational multiplier to be nonzero, proved by contradiction using irrationality of zero. Although one direction reuses the parent lemma, the new converse requires reasoning absent from the parent proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the implication into an iff and adds the converse argument that a zero rational multiplier makes the product rational, which is not supplied by the parent's forward lemma. The proof therefore requires genuinely new reasoning beyond recalling the parent.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2140 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem irrational_mul_ratCast_iff_of_irrational.extracted_1_1 (x : ℝ) (r : ℚ) (hx : Irrational x) :
  Irrational (x * ↑r) ↔ r ≠ 0
```

Read back by a model that saw only the Lean. For a real number x that is irrational and a rational number r , the product of x and the real-number cast of r is irrational if and only if r is nonzero.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem irrational_mul_ratCast_iff_of_irrational (x : ℝ) (r : ℚ) (hx : Irrational x) : Irrational (x * r) ↔ r ≠ 0 := by
  constructor
  · intro hprod hr
    subst r
    have hzero : Irrational (0 : ℝ) := by
      simp using hprod
    norm_num at hzero
  · intro hr
    exact hx.mul_ratCast hr
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_1_1b__mh__4decaa2e

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 4decaa2ed2957406

4.241 241. irrational_add_rational

The problem

If x is irrational and r is rational, prove that $x + r$ is irrational.

Lean statement

```
theorem irrational_add_rational
(x : ℝ)
(r : ℚ)
: Irrational x → Irrational (x + r) := by
```

Parent. If r is rational ($r \neq 0$) and x is irrational, prove that rx is irrational.

Parent in Lean

```
theorem Rudin_exercise_1_1b
(x : ℝ)
(y : ℚ)
(h : y ≠ 0)
: (Irrational x) → Irrational (x * y) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes nonzero rational scaling into rational translation, requiring a different irrationality-preservation theorem; the parent's 'mul_ratCast' argument cannot prove the additive conclusion. The proof is complete and uses no redundant hypotheses.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes nonzero rational scaling to rational translation, requiring a different irrationality theorem and eliminating the parent's nonzero hypothesis. The parent's proof and closing lemma do not solve this additive claim.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes rational scaling to rational translation, so the parent's nonzero-multiplier argument and 'mul_ratCast' lemma no longer apply. Proving irrationality is preserved under addition requires a distinct mathematical lemma and proof obligation.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2746 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem irrational_add_rational.extracted_1_1 (x : ℝ) (r : ℚ) : Irrational x → Irrational (x + ↑r)
```

Read back by a model that saw only the Lean. *For every real number x and rational number r , if x is irrational, then x plus the real number represented by r is irrational.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem irrational_add_rational
(x : ℝ)
(r : ℚ)
: Irrational x → Irrational (x + r) := by
  intro hx
  exact hx.add_ratCast r
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_1_1b__mh__69129ea3

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 69129ea3cd3a3493

4.242 242. irrational_inv_of_irrational

The problem

If x is a nonzero irrational real number, prove that its reciprocal x^{-1} is irrational.

Lean statement

```
theorem irrational_inv_of_irrational {x : ℝ} (hx : Irrational x) : Irrational x⁻¹
```

Parent. If r is rational ($r \neq 0$) and x is irrational, prove that rx is irrational.

Parent in Lean

```
theorem Rudin_exercise_1_1b
(x : ℝ)
(y : ℚ)
(h : y ≠ 0)
: (Irrational x) -> Irrational (x * y) := by
```

What it contributes. Transferred the parent’s irrationality-by-contradiction structure, while replacing rational multiplication with reciprocal rationality and explicitly using the nonzero hypothesis.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a genuinely different specialization: inversion preserves irrationality, relying on inverse-specific reasoning and the fact that irrational reals are nonzero. The parent’s rational-scaling lemma does not directly supply this conclusion.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes multiplication by a nonzero rational into inversion, relying on a distinct irrational-inverse theorem. The parent’s ‘mul_ratCast’ argument cannot establish irrationality of the reciprocal.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely different preservation result: inversion of an irrational real remains irrational. The parent’s rational-scaling lemma and proof do not supply this claim, and the irrationality hypothesis itself supplies the needed nonzero fact.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2933 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem irrational_inv_of_irrational.extracted_1_1 {x : ℝ} (hx : Irrational x) (hx0 : x ≠ 0) : Irrational x-1
```

Read back by a model that saw only the Lean. *For a real number x , if x is irrational and $x \neq 0$, then its reciprocal $1/x$ is irrational.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem irrational_inv_of_irrational {x : ℝ} (hx : Irrational x) : Irrational x-1 := by
  exact hx.inv
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_1_1b__mh__9f344909

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 74e5ab2986fb0495

4.243 243. affine_irrational

The problem

If x is irrational, a and b are rational, and $a \neq 0$, prove that $ax + b$ is irrational.

Lean statement

```
theorem affine_irrational (x : ℝ) (a b : ℚ) (ha : a ≠ 0) : Irrational x → Irrational ((a : ℝ) * x + b)
```

Parent. If r is rational ($r \neq 0$) and x is irrational, prove that rx is irrational.

Parent in Lean

```
theorem Rudin_exercise_1.1b
(x : ℝ)
(y : ℚ)
(h : y ≠ 0)
: (Irrational x) → Irrational (x * y) := by
```

What it contributes. Preserves the parent's real irrational input and nonzero rational multiplier domain while extending the transformation with a rational translation; the parent's contradiction idea is realized through explicit subtraction and division.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a rational translation and proves irrationality by recovering x through the nonzero affine coefficient, then deriving a rational representation of x for contradiction. This reasoning is not supplied by the parent's multiplicative lemma and is distinct from the already-kept scaling and inversion results.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely adds a rational translation to the nonzero rational scaling. Its proof must derive a rational preimage under the affine map, using subtraction and division, which the parent's multiplication-only lemma does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves irrationality under a nondegenerate rational affine transformation, requiring both translation handling and inversion of the affine map; the parent's multiplicative scaling lemma alone does not supply this argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2940 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem affine_irrational.extracted_1_1 (x : ℝ) (a b : ℚ) (ha : a ≠ 0) :  
  Irrational x → Irrational (↑a * x + ↑b)
```

Read back by a model that saw only the Lean. *For a real number x and rational numbers a and b with $a \neq 0$, if x is irrational, then $a x + b$ is irrational.*

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open Topology Filter Real Complex TopologicalSpace Finset  
open scoped BigOperators  
  
theorem affine_irrational (x : ℝ) (a b : ℚ) (ha : a ≠ 0) : Irrational x → Irrational ((a : ℝ) * x + b) := by  
  intro hx hbad  
  obtain ⟨q, hq⟩ := hbad  
  have haR : (a : ℝ) ≠ 0 := by  
    exact_mod_cast ha  
  have hsub : (q : ℝ) - b = (a : ℝ) * x := by  
    linarith [hq]  
  have hxa : x = ((q : ℝ) - b) / a := by  
    apply (eq_div_iff haR).2  
    calc  
      x * (a : ℝ) = (a : ℝ) * x := by ring  
      _ = (q : ℝ) - b := hsub.symm  
  apply hx  
  refine ⟨(q - b) / a, ?_⟩  
  rw [hxa]  
  push_cast  
  rfl
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_1_1b__mh__a86cb245

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint a86cb2458ef9e167

4.244 244. irrational_mul_ratCast_characterization

The problem

For a real number x and a rational number y , prove that xy is irrational if and only if $y \neq 0$ and x is irrational.

Lean statement

```
theorem irrational_mul_ratCast_characterization (x : ℝ) (y : ℚ) : Irrational (x * y) ↔ y ≠ 0 ∧ Irrational x
```

Parent. If r is rational ($r \neq 0$) and x is irrational, prove that rx is irrational.

Parent in Lean

```
theorem Rudin_exercise_1_1b
(x : ℝ)
(y : ℚ)
(h : y ≠ 0)
: (Irrational x) -> Irrational (x * y) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child proves a genuine two-way characterization: from $\text{Irrational } (x * y)$ it must derive both $y \neq 0$ and $\text{Irrational } x$, using cancellation and contradiction. The parent only supplies the forward scaling implication, so memorizing its proof does not solve the new direction.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine converse obligation: from irrationality of $x*y$ it derives both $y \neq 0$ and $\text{Irrational } x$, yielding an iff characterization. This is not supplied by the parent's one-way scaling proof, despite reusing that proof for the forward direction.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a one-way preservation result into a full characterization. Its reverse direction requires proving both that the rational multiplier is nonzero and that irrationality of x follows by cancellation, obligations absent from the parent's forward-only proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2001 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem irrational_mul_ratCast_characterization.extracted_1_1 (x : ℝ) (y : ℚ) :  
  Irrational (x * ↑y) ↔ y ≠ 0 ∧ Irrational x
```

Read back by a model that saw only the Lean. For a real number x and a rational number y , the product of x and the real-number cast of y is irrational if and only if y is nonzero and x is irrational.

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
open Topology Filter Real Complex TopologicalSpace Finset  
open scoped BigOperators  
  
theorem irrational_mul_ratCast_characterization (x : ℝ) (y : ℚ) :  
  Irrational (x * y) ↔ y ≠ 0 ∧ Irrational x := by  
  constructor  
  · intro hxy  
    have hx : Irrational x := hxy.of_mul_ratCast  
    have hy : y ≠ 0 := by  
      intro hy0  
      subst hy0  
      simpa using hxy  
    exact ⟨hy, hx⟩  
  · intro h  
    have hscaled : Irrational (x * y) := h.2.mul_ratCast h.1  
    exact hscaled
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_1_1b__mh__ccdca90e

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 79d3091bb0fb36b1

4.245 245. countable_trace_basis_countable_union

The problem

Let K be a compact metric space, A a subset of K , and B a countable topological basis for K . Show that the traces of the sets in B form a countable basis for A , and that every open subset of A is a countable union of such traces.

Lean statement

```
theorem countable_trace_basis_countable_union {K : Type*} [MetricSpace K] [CompactSpace K]
  {A : Set K} (B : Set (Set K))
  (hBcount : Set.Countable B)
  (hBbasis : TopologicalSpace.IsTopologicalBasis B) :
  Set.Countable
    (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) ∧
  TopologicalSpace.IsTopologicalBasis
    (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) ∧
  ∀ V : Set A, IsOpen V →
    ∃ s ⊆ Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B,
      s.Countable ∧ V = ⋃ U ∈ s, U
```

Parent. Let K be a compact metric space, A a closed subset of K , and B a countable topological basis for K . Show that the traces of the sets in B on A form a countable topological basis for A .

Parent in Lean

```
theorem supplied_basis_trace_basis {K : Type*} [MetricSpace K] [CompactSpace K]
  {A : Set K} (B : Set (Set K))
  (hBcount : Set.Countable B)
  (hBbasis : TopologicalSpace.IsTopologicalBasis B) :
  Set.Countable
    (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) ∧
  TopologicalSpace.IsTopologicalBasis
    (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a substantive countable-union representation for every open subset of the subtype. Proving it requires deriving second countability and applying the basis union theorem, which is not supplied by the parent's proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a substantive open-set decomposition claim: every open subset of the subspace is a countable union of traced basis elements. Proving it requires deriving second countability and applying a countable-basis union theorem, which is not supplied by the parent's proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a substantive theorem: every open subset of the subspace is a countable union of members of the traced basis. Proving this requires deriving second countability and applying a countable-basis union lemma, which is not supplied by the parent's countability-and-basis proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2367 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem countable_trace_basis_countable_union.extracted_1_1.{u_1} {K : Type u_1} [inst : MetricSpace K]
[inst_1 : CompactSpace K] {A : Set K} {B : Set (Set K)} (hBcount : B.Countable) (hBbasis : IsTopologicalBasis B) :
((fun U => Subtype.val ^-1' U) '' B).Countable ∧
IsTopologicalBasis ((fun U => Subtype.val ^-1' U) '' B) ∧
∀ (V : Set ↑A), IsOpen V → ∃ s ⊆ (fun U => Subtype.val ^-1' U) '' B, s.Countable ∧ V = ⋃ U ∈ s, U
```

Read back by a model that saw only the Lean. Let K be a compact metric space, let A be a subset of K , and let B be a countable topological basis for K . Then the collection of traces of the sets in B on the subtype of points of A , namely $\{x : K \mid x \in A\}$ with each trace given by $\{x \in A \mid x \in U\}$ for $U \in B$, is countable and is a topological basis for the subtype. Moreover, every open set V in this subtype is the union of some countable subcollection s of these traces: there exists a countable set s contained in the collection of traces such that $V = \bigcup \{U \in s\} U$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem countable_trace_basis_countable_union {K : Type*} [MetricSpace K] [CompactSpace K]
{A : Set K} {B : Set (Set K)}
(hBcount : Set.Countable B)
(hBbasis : TopologicalSpace.IsTopologicalBasis B) :
Set.Countable
(Set.image (fun U : Set K => (Subtype.val : A → K) ^-1' U) B) ∧
TopologicalSpace.IsTopologicalBasis
(Set.image (fun U : Set K => (Subtype.val : A → K) ^-1' U) B) ∧
∀ V : Set A, IsOpen V →
∃ s ⊆ Set.image (fun U : Set K => (Subtype.val : A → K) ^-1' U) B,
s.Countable ∧ V = ⋃ U ∈ s, U := by
let I : SecondCountableTopology K := hBbasis.secondCountableTopology hBcount
have hcount : Set.Countable
(Set.image (fun U : Set K => (Subtype.val : A → K) ^-1' U) B) :=
hBcount.image (fun U : Set K => (Subtype.val : A → K) ^-1' U)
have hbasis : TopologicalSpace.IsTopologicalBasis
(Set.image (fun U : Set K => (Subtype.val : A → K) ^-1' U) B) :=
hBbasis.induced (Subtype.val : A → K)
have hunion : ∀ V : Set A, IsOpen V →
∃ s ⊆ Set.image (fun U : Set K => (Subtype.val : A → K) ^-1' U) B,
s.Countable ∧ V = ⋃ U ∈ s, U := by
intro V hV
exact hbasis.exists_countable_biUnion_of_isOpen hV
exact ⟨hcount, hbasis, hunion⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Rudin_exercise_2_25__me.me.mh__75316feb

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 75316febde0967fe

4.246 246. arbitrary_subspace_countable_trace_basis

The problem

Let K be a compact metric space and let A be any subset of K . Construct a countable family of ambient open sets whose traces on A form a topological basis for A .

Lean statement

```
theorem arbitrary_subspace_countable_trace_basis {K : Type*} [MetricSpace K] [CompactSpace K] (A : Set K) :
  ∃ (B : Set (Set K)), Set.Countable B ∧
    (∀ U ∈ B, IsOpen U) ∧
    TopologicalSpace.IsTopologicalBasis
      (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B)
```

Parent. Let K be a compact metric space and A a closed subset of K . Construct a countable family of ambient open sets whose traces on A form a topological basis for A .

Parent in Lean

```
theorem closed_subspace_countable_trace_basis {K : Type*} [MetricSpace K] [CompactSpace K] {A : Set K} (hA : IsClosed A) :
  ∃ (B : Set (Set K)), Set.Countable B ∧
    (∀ U ∈ B, IsOpen U) ∧
    TopologicalSpace.IsTopologicalBasis
      (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B)
```

What it contributes. Preserves the compact metric ambient space and trace-basis conclusion, while replacing closed-subspace inheritance with the direct ambient countable-basis construction.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely removes closedness and changes the construction: it uses a countable ambient basis of K and proves its traces form a basis on arbitrary A via the induced-topology theorem. The parent's compact-subtype basis and ambient lifting argument does not directly supply this proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child removes the closedness hypothesis and proves the stronger result for every subspace by using a countable ambient basis and the induced-topology basis theorem. This requires a genuinely different argument from the parent's compactness-based basis construction.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely removes the closedness hypothesis and proves the result by tracing a countable ambient basis of the compact metric space, rather than using compactness and a basis of the closed subspace. This requires a materially different argument and is not recoverable by recalling the parent's proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2353 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem arbitrary_subspace_countable_trace_basis.extracted_1.1.{u_1} {K : Type u_1} [inst : MetricSpace K]
  [inst_1 : CompactSpace K] (A : Set K) :
  ∃ B, B.Countable ∧ (∀ U ∈ B, IsOpen U) ∧ IsTopologicalBasis ((fun U => Subtype.val ⁻¹' U) '' B)
```

Read back by a model that saw only the Lean. *For every subset A of a compact metric space K , there exists a countable family B of open subsets of K such that the family of their traces on the subtype A , namely $\{x \in A \mid x \in U\}$ for $U \in B$, is a topological basis for A with its subspace topology.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem arbitrary_subspace_countable_trace_basis {K : Type*} [MetricSpace K] [CompactSpace K] (A : Set K) :
  ∃ (B : Set (Set K)), Set.Countable B ∧
    (∀ U ∈ B, IsOpen U) ∧
    TopologicalSpace.IsTopologicalBasis
      (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) := by
  classical
  have hSecond : SecondCountableTopology K := inferInstance
  have hCount : (countableBasis K).Countable := countable_countableBasis K
  have hOpen : ∀ U ∈ countableBasis K, IsOpen U := by
    intro U hU
    exact isOpen_of_mem_countableBasis hU
  have hTrace : TopologicalSpace.IsTopologicalBasis
    (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) (countableBasis K)) := by
    simpa only using
      (TopologicalSpace.IsTopologicalBasis.induced (Subtype.val : A → K)
        (isBasis_countableBasis K))
  exact ⟨countableBasis K, hCount, hOpen, hTrace⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_2_25__me.mh__1c9af0d8

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 1c9af0d8112d4ffa

4.247 247. closed_subspace_countable_separating_trace_basis

The problem

Let K be a compact metric space and A a closed subset of K . Show that there is a countable family of ambient open sets whose traces form a basis for A and separate any two distinct points of A .

Lean statement

```
theorem closed_subspace_countable_separating_trace_basis {K : Type*} [MetricSpace K] [CompactSpace K] {A : Set K} (hA : IsClosed A) :
  ∃ (B : Set (Set K)), Set.Countable B ∧
    (∀ U ∈ B, IsOpen U) ∧
    TopologicalSpace.IsTopologicalBasis
      (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) ∧
    (∀ x y : A, x ≠ y → ∃ U ∈ B,
      x ∈ (Subtype.val : A → K) ⁻¹' U ∧
      y ∉ (Subtype.val : A → K) ⁻¹' U)
```

Parent. Let K be a compact metric space and A a closed subset of K . Construct a countable family of ambient open sets whose traces on A form a topological basis for A .

Parent in Lean

```
theorem closed_subspace_countable_trace_basis {K : Type*} [MetricSpace K] [CompactSpace K] {A : Set K} (hA : IsClosed A) :
  ∃ (B : Set (Set K)), Set.Countable B ∧
    (∀ U ∈ B, IsOpen U) ∧
    TopologicalSpace.IsTopologicalBasis
      (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B)
```

What it contributes. Supplied the compact-subtype countable basis, ambient open representatives, countability argument, and trace-basis construction; the mutation adds point separation via metric balls.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The added point-separation conclusion is not a trivial conjunction: it requires a new metric argument using positive distance, open balls, and basis refinement to construct a separating trace. The parent proof only constructs the countable trace basis and does not supply this reasoning.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine countable separation property for distinct points, requiring a new metric-ball and basis-refinement argument absent from the parent proof. The existing basis construction alone does not establish this claim.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuine point-separation theorem: it uses positive distance, an open metric ball, and basis refinement to find a basis element containing one point but not the other. The parent's countable trace-basis construction alone does not supply this argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable

Check	By	Result
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2745 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem closed_subspace_countable_separating_trace_basis.extracted_1.1.{u_1} {K : Type u_1} [inst : MetricSpace K]
[inst_1 : CompactSpace K] {A : Set K} (hA : IsClosed A) :
  ∃ B,
    B.Countable ∧
    (∀ U ∈ B, IsOpen U) ∧
    IsTopologicalBasis ((fun U => Subtype.val ⁻¹' U) '' B) ∧
    ∀ (x y : ↑A), x ≠ y → ∃ U ∈ B, x ∈ Subtype.val ⁻¹' U ∧ y ∉ Subtype.val ⁻¹' U
```

Read back by a model that saw only the Lean. *Let K be a metrizable compact topological space, and let A be a closed subset of K . Then there exists a countable collection B of subsets of K such that every U in B is open in K , the collection of subsets of A given by the preimages of the members of B under the inclusion map $A \rightarrow K$ is a topological basis for A with its subspace topology, and for any two distinct points x and y of A , there is some U in B such that x belongs to the preimage of U in A while y does not.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem closed_subspace_countable_separating_trace_basis {K : Type*} [MetricSpace K] [CompactSpace K] {A : Set K} (hA : IsClosed A) :
  ∃ (B : Set (Set K)), Set.Countable B ∧
    (∀ U ∈ B, IsOpen U) ∧
    TopologicalSpace.IsTopologicalBasis
      (Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B) ∧
    (∀ x y : A, x ≠ y → ∃ U ∈ B,
      x ∈ (Subtype.val : A → K) ⁻¹' U ∧
      y ∉ (Subtype.val : A → K) ⁻¹' U) := by
  classical
  letI : CompactSpace A := isCompact_iff_compactSpace.mp hA.isCompact
  obtain ⟨C, hCcount, hCne, hCbasis⟩ :=
    TopologicalSpace.exists_countable_basis (α := A)
  have hRep : ∀ c ∈ C, ∃ U : Set K, IsOpen U ∧
    (Subtype.val : A → K) ⁻¹' U = c := by
    intro c hc
    exact isOpen_induced_iff.mp (hCbasis.isOpen hc)
  choose f hfopen hftrace using hRep
  let g : Set A → Set K := fun c => if hc : c ∈ C then f c hc else ∅
  let B : Set (Set K) := g '' C
  have hBcount : B.Countable := hCcount.image g
  have hBopen : ∀ U ∈ B, IsOpen U := by
    rintro U ⟨c, hc, rfl⟩
    simpa [g, dif_pos hc] using hfopen c hc
  have htrace : Set.image (fun U : Set K => (Subtype.val : A → K) ⁻¹' U) B = C := by
    apply Set.Subset.antisymm
    · rintro V ⟨U, ⟨c, hc, rfl⟩, rfl⟩
      simpa [g, dif_pos hc, hftrace c hc]
    · intro c hc
      refine ⟨g c, ⟨c, hc, rfl⟩, ?_⟩
      simpa [g, dif_pos hc, hftrace c hc]
  have hSeparate : ∀ x y : A, x ≠ y → ∃ U ∈ B,
    x ∈ (Subtype.val : A → K) ⁻¹' U ∧
    y ∉ (Subtype.val : A → K) ⁻¹' U := by
    intro x y hxy
    have hxypos : 0 < dist x y := dist_pos.mpr hxy
```



```

have hxball : x ∈ Metric.ball x (dist x y) := by
  rw [Metric.mem_ball]
  simpa using hxypos
have hyball : y ∉ Metric.ball x (dist x y) := by
  intro hy
  have hy' : dist x y < dist x y := by
    simpa [Metric.mem_ball, dist_comm] using hy
  exact (lt_irrefl _ hy')
have hopen : IsOpen (Metric.ball x (dist x y)) := Metric.isOpen_ball
obtain ⟨c, hc, hxc, hsub⟩ := hCbasis.exists_subset_of_mem_open hxball hopen
have hg : g c = f c hc := by
  simp [g, dif_pos hc]
refine ⟨f c hc, ⟨c, hc, hg⟩, ?_, ?_⟩
· rw [hftrace c hc]
  exact hxc
· intro hy
  have hyc : y ∈ c := by
    rw [← hftrace c hc]
    exact hy
  exact hyball (hsub hyc)
refine ⟨B, hBcount, hBopen, ?_, hSeparate⟩
rw [htrace]
exact hCbasis

```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_2_25__me.mh__f713c059

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint f713c059eca8929b

4.248 248. proper_pseudoMetricSpace_has_countable_basis

The problem

Prove that every proper pseudo-metric space has a countable base.

Lean statement

```
theorem proper_pseudoMetricSpace_has_countable_basis {K : Type*} [PseudoMetricSpace K] [ProperSpace K] :
  ∃ (B : Set (Set K)), Set.Countable B ∧ IsTopologicalBasis B := by
```

Parent. Prove that every compact metric space K has a countable base.

Parent in Lean

```
theorem Rudin_exercise_2_25 {K : Type*} [MetricSpace K] [CompactSpace K] :
  ∃ (B : Set (Set K)), Set.Countable B ∧ IsTopologicalBasis B := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the parent from compact metric spaces to proper pseudometric spaces. Its proof requires the distinct theorem that proper spaces are second-countable, rather than merely reusing compactness-based inference.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child substantially generalizes the result from compact metric spaces to proper pseudometric spaces, so the parent's compactness-based instance argument does not apply. Its proof requires invoking the distinct theorem that proper spaces are second-countable and constructing the canonical countable basis.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes compact metric spaces to proper pseudo-metric spaces and must invoke the distinct theorem that proper spaces are second-countable. The parent's compactness-based instance argument does not supply this result.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2950 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem proper_pseudoMetricSpace_has_countable_basis.extracted_1_1.{u_1} {K : Type u_1} [inst : PseudoMetricSpace K]
  [inst_1 : ProperSpace K] : ∃ B, B.Countable ∧ IsTopologicalBasis B
```

Read back by a model that saw only the Lean. For every proper pseudometric space K , there exists a countable collection

B of subsets of K such that B is a topological basis for K .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem proper_pseudoMetricSpace_has_countable_basis {K : Type*} [PseudoMetricSpace K] [ProperSpace K] :
  ∃ (B : Set (Set K)), Set.Countable B ∧ IsTopologicalBasis B := by
  let I : SecondCountableTopology K := secondCountable_of_proper
  have hCount : (TopologicalSpace.countableBasis K).Countable :=
    TopologicalSpace.countable_countableBasis K
  have hBasis : IsTopologicalBasis (TopologicalSpace.countableBasis K) :=
    TopologicalSpace.isBasis_countableBasis K
  exact ⟨TopologicalSpace.countableBasis K, hCount, hBasis⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_2_25__mh__8949e07b

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 8949e07b76d99ef5

4.249 249. compact_metric_countable_dense_basis

The problem

Prove that every compact metric space has a countable dense subset and a countable base whose every nonempty member meets that dense subset.

Lean statement

```
theorem compact_metric_countable_dense_basis {K : Type*} [MetricSpace K] [CompactSpace K] :
  ∃ D : Set K, D.Countable ∧ Dense D ∧
  ∃ B : Set (Set K), B.Countable ∧ IsTopologicalBasis B ∧
  ∀ U ∈ B, U.Nonempty → (D ∩ U).Nonempty := by
```

Parent. Prove that every compact metric space K has a countable base.

Parent in Lean

```
theorem Rudin_exercise_2_25 {K : Type*} [MetricSpace K] [CompactSpace K] :
  ∃ (B : Set (Set K)), Set.Countable B ∧ IsTopologicalBasis B := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a substantive countable dense subset and proves that every nonempty basis element meets it, requiring separability and a density–openness argument absent from the parent. The parent’s basis result is reused only as one component of a genuinely expanded conclusion.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuinely new existence requirement: a countable dense subset and a proof that it meets every nonempty basis element. This requires separability and the dense-set/open-set intersection argument, which are absent from the parent’s basis-extraction proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a genuinely new existence-and-incidence claim: it must construct a countable dense set and prove every nonempty basis element intersects it. The parent only supplies countable-basis existence, so its proof does not discharge the new density argument.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2926 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem compact_metric_countable_dense_basis.extracted_1_1.{u_1} {K : Type u_1} [inst : MetricSpace K]
  [inst_1 : CompactSpace K] :
  ∃ D, D.Countable ∧ Dense D ∧ ∃ B, B.Countable ∧ IsTopologicalBasis B ∧ ∀ U ∈ B, U.Nonempty → (D ∩ U).Nonempty
```

Read back by a model that saw only the Lean. *For every compact metric space K , there exists a countable dense subset D of K and a countable collection B of subsets of K that is a topological basis, such that every nonempty U in B has nonempty intersection with D .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem compact_metric_countable_dense_basis {K : Type*} [MetricSpace K] [CompactSpace K] :
  ∃ D : Set K, D.Countable ∧ Dense D ∧
    ∃ B : Set (Set K), B.Countable ∧ IsTopologicalBasis B ∧
      ∀ U ∈ B, U.Nonempty → (D ∩ U).Nonempty := by
  classical
  have hSecond : SecondCountableTopology K := inferInstance
  letI : SecondCountableTopology K := hSecond
  have hSep : SeparableSpace K := inferInstance
  letI : SeparableSpace K := hSep
  obtain ⟨D, hDcount, hDdense⟩ := TopologicalSpace.exists_countable_dense K
  obtain ⟨B, hBcount, -, hBbasis⟩ :=
    TopologicalSpace.exists_countable_basis (α := K)
  have hIncidence : ∀ U ∈ B, U.Nonempty → (D ∩ U).Nonempty := by
    intro U hU hUne
    have hMeet : (U ∩ D).Nonempty :=
      hDdense.inter_open_nonempty U (hBbasis.isOpen hU) hUne
    simpa [Set.inter_comm] using hMeet
  exact ⟨D, hDcount, hDdense, B, hBcount, hBbasis, hIncidence⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_2_25__mh__b74d259e

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propept, Classical.choice, Quot.sound, fingerprint b74d259e7d109c53

4.250 250. three_disjoint_perfect_subsets

The problem

Let A be an uncountable closed subset of a separable metric space. Prove that A contains three pairwise disjoint nonempty perfect subsets.

Lean statement

```
theorem three_disjoint_perfect_subsets (X : Type*) [MetricSpace X] [SeparableSpace X]
  (A : Set X) (hA : IsClosed A) (hunc : ¬ Set.Countable A) :
  ∃ F : Fin 3 → Set X,
    (∀ i, F i ⊆ A ∧ Perfect (F i) ∧ (F i).Nonempty) ∧
    Pairwise (fun i j => Disjoint (F i) (F j))
```

Parent. Let A be an uncountable closed subset of a separable metric space. Prove that A contains an uncountable perfect subset.

Parent in Lean

```
theorem uncountable_perfect_subset (X : Type*) [MetricSpace X] [SeparableSpace X]
  (A : Set X) (hA : IsClosed A) (hunc : ¬ Set.Countable A) :
  ∃ P : Set X, P ⊆ A ∧ Perfect P ∧ ¬ Set.Countable P
```

What it contributes. Supplied the initial nonempty perfect subset of the closed uncountable set A ; repeated perfect-set splitting produces the $\text{Fin } 3$ family.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child substantially strengthens the parent from one uncountable perfect subset to three nonempty, pairwise disjoint perfect subsets. Its proof requires iterative small-diameter splitting and new disjointness arguments, so recalling the parent does not supply the result.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely strengthens the parent from one uncountable perfect subset to three pairwise disjoint nonempty perfect subsets contained in the closed set. Its proof requires iterative perfect-set splitting and a new finite-index disjointness argument, which the parent proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires constructing three pairwise disjoint nonempty perfect subsets via repeated perfect-set splitting, containment propagation, and finite-index pairwise reasoning. The parent only supplies existence of one uncountable perfect subset and does not provide this disjoint-family construction.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2800 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem three_disjoint_perfect_subsets.extracted_1_1.{u_1} (X : Type u_1) [inst : MetricSpace X]
  [inst_1 : SeparableSpace X] (A : Set X) (hA : IsClosed A) (hunc : ¬A.Countable) :
  ∃ F, (∀ (i : Fin 3), F i ⊆ A ∧ Perfect (F i) ∧ (F i).Nonempty) ∧ Pairwise fun i j => Disjoint (F i) (F j)
```

Read back by a model that saw only the Lean. *For every separable metric space X and every closed, uncountable subset A of X , there exists a family of three subsets $F(0)$, $F(1)$, and $F(2)$ of X such that each $F(i)$ is contained in A , is perfect, and is nonempty, and any two subsets $F(i)$ and $F(j)$ with $i \neq j$ are disjoint.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem three_disjoint_perfect_subsets (X : Type*) [MetricSpace X] [SeparableSpace X]
  (A : Set X) (hA : IsClosed A) (hunc : ¬ Set.Countable A) :
  ∃ F : Fin 3 → Set X,
    (∀ i, F i ⊆ A ∧ Perfect (F i) ∧ (F i).Nonempty) ∧
    Pairwise (fun i j => Disjoint (F i) (F j)) := by
  classical
  obtain ⟨P, hPperf, hPne, hPsub⟩ :=
    exists_perfect_nonempty_of_isClosed_of_not_countable (C := A) hA hunc
  obtain ⟨P0, P1, h0, h1, hd01⟩ :=
    Perfect.small_diam_splitting hPperf hPne (ε := 1) (by norm_num)
  obtain ⟨P10, P11, h10, h11, hd1011⟩ :=
    Perfect.small_diam_splitting h1.1 h1.2.1 (ε := 1) (by norm_num)
  have h0sub : P0 ⊆ A := h0.2.2.1.trans hPsub
  have h10sub : P10 ⊆ A := (h10.2.2.1.trans h1.2.2.1).trans hPsub
  have h11sub : P11 ⊆ A := (h11.2.2.1.trans h1.2.2.1).trans hPsub
  have hd010 : Disjoint P0 P10 := by
    refine Set.disjoint_left.2 ?_
    intro x hx0 hx10
    exact Set.disjoint_left.1 hd01 hx0 (h10.2.2.1 hx10)
  have hd011 : Disjoint P0 P11 := by
    refine Set.disjoint_left.2 ?_
    intro x hx0 hx11
    exact Set.disjoint_left.1 hd01 hx0 (h11.2.2.1 hx11)
  have hd100 : Disjoint P10 P0 := hd010.symm
  have hd110 : Disjoint P11 P0 := hd011.symm
  have hd110 : Disjoint P11 P0 := hd011.symm
  let F : Fin 3 → Set X := fun i =>
    if i = 0 then P0 else if i = 1 then P10 else P11
  have hFpair : Pairwise (fun i j => Disjoint (F i) (F j)) := by
    intro i j hij
    fin_cases i <|> fin_cases j <|> simp_all [F]
  refine ⟨F, ?_, hFpair⟩
  intro i
  fin_cases i <|> simp_all [F]
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_2_28__mh__00c8e1e0

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 00c8e1e07a25fec4

4.251 251. Rudin_exercise_2_28_mutation

The problem

Prove that every closed set in a separable metric space is either at most countable or contains a nonempty perfect subset.

Lean statement

```
theorem Rudin_exercise_2_28_mutation (X : Type*) [MetricSpace X] [SeparableSpace X]
  (A : Set X) (hA : IsClosed A) :
  A.Countable ∨ ∃ P : Set X, Perfect P ∧ P.Nonempty ∧ P ⊆ A
```

Parent. Prove that every closed set in a separable metric space is the union of a (possibly empty) perfect set and a set which is at most countable.

Parent in Lean

```
theorem Rudin_exercise_2_28 (X : Type*) [MetricSpace X] [SeparableSpace X]
  (A : Set X) (hA : IsClosed A) :
  ∃ P₁ P₂ : Set X, A = P₁ ∪ P₂ ∧ Perfect P₁ ∧ Set.Countable P₂ := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the decomposition conclusion into a genuine countability-versus-perfect-subset dichotomy. Beyond invoking the parent theorem, its proof requires a nontrivial case split and derives that the countable component covers A when the perfect component is empty.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the decomposition conclusion into a genuine dichotomy: either the closed set is countable or it contains a nonempty perfect subset. Proving this requires a new nonemptiness case split and deriving countability when the perfect component is empty, which the parent's direct repackaging proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the decomposition conclusion into a substantive dichotomy: either the entire closed set is countable or it contains a nonempty perfect subset. Proving this requires a nontrivial case split on the perfect component and subset/countability reasoning not present in the parent's proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2785 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem Rudin_exercise_2_28_mutation.extracted_1_1.{u_1} (X : Type u_1) [inst : MetricSpace X]
  [inst_1 : SeparableSpace X] (A : Set X) (hA : IsClosed A) : A.Countable ∨ ∃ P, Perfect P ∧ P.Nonempty ∧ P ⊆ A
```

Read back by a model that saw only the Lean. *Let X be a separable metric space, and let A be a closed subset of X . Then either A is countable, or there exists a nonempty perfect subset P of X such that P is contained in A .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem Rudin_exercise_2_28_mutation (X : Type*) [MetricSpace X] [SeparableSpace X]
  (A : Set X) (hA : IsClosed A) :
  A.Countable ∨ ∃ P : Set X, Perfect P ∧ P.Nonempty ∧ P ⊆ A := by
  classical
  obtain ⟨C, P, hC, hP, hdecomp⟩ :=
    exists_countable_union_perfect_of_isClosed (α := X) (C := A) hA
  by_cases hPne : P.Nonempty
  · right
    refine ⟨P, hP, hPne, ?_⟩
    intro x hx
    rw [hdecomp]
    exact Or.inr hx
  · left
    have hPsub : P ⊆ C := by
      intro x hx
      exact False.elim (hPne ⟨x, hx⟩)
    have hAsub : A ⊆ C := by
      intro x hx
      rw [hdecomp] at hx
      rcases hx with hx | hx
      · exact hx
      · exact hPsub hx
    exact hC.mono hAsub
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_2_28__mh__0657fa6f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 0657fa6f251f1874

4.252 252. exists_nonempty_perfect_subset_of_isClosed_of_not_countable

The problem

Let A be an uncountable closed subset of a separable metric space. Prove that A contains a nonempty perfect subset.

Lean statement

```
theorem exists_nonempty_perfect_subset_of_isClosed_of_not_countable (X : Type*) [MetricSpace X] [SeparableSpace X]
  (A : Set X) (hA : IsClosed A) (hunc : ¬ Set.Countable A) :
  ∃ P : Set X, Perfect P ∧ P.Nonempty ∧ P ⊆ A
```

Parent. Prove that every closed set in a separable metric space is the union of a (possibly empty) perfect set and a set which is at most countable.

Parent in Lean

```
theorem Rudin_exercise_2_28 (X : Type*) [MetricSpace X] [SeparableSpace X]
  (A : Set X) (hA : IsClosed A) :
  ∃ P₁ P₂ : Set X, A = P₁ ∪ P₂ ∧ Perfect P₁ ∧ Set.Countable P₂ := by
```

What it contributes. Uses the parent decomposition checkpoint: a closed set is the union of a countable set and a perfect set; the child additionally consumes uncountability to force the perfect component nonempty.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child derives a nonempty perfect component from the decomposition only after a new contradiction argument: if the perfect part were empty, the closed set would be countable, contradicting the hypothesis. This nonemptiness reasoning is absent from the parent's proof and is not a forced wrapper or trivial conjunction.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the decomposition conclusion into the substantive claim that an uncountable closed set contains a nonempty perfect subset. It requires a new contradiction argument showing the perfect component cannot be empty, plus an inclusion proof; recalling the parent's decomposition proof is insufficient.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the decomposition conclusion into an existence theorem for a nonempty perfect subset and must use uncountability to prove the perfect component is nonempty. This contradiction and subset argument are substantive reasoning absent from the parent's mere repackaging proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2794 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem exists_nonempty_perfect_subset_of_isClosed_of_not_countable.extracted_1.1.{u_1} (X : Type u_1)
  [inst : MetricSpace X] [inst_1 : SeparableSpace X] (A : Set X) (hA : IsClosed A) (hunc : ¬A.Countable) :
  ∃ P, Perfect P ∧ P.Nonempty ∧ P ⊆ A
```

Read back by a model that saw only the Lean. For a separable metric space X and a closed, uncountable subset A of X , there exists a nonempty perfect subset P of X such that P is contained in A .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem exists_nonempty_perfect_subset_of_isClosed_of_not_countable (X : Type*) [MetricSpace X] [SeparableSpace X]
  (A : Set X) (hA : IsClosed A) (hunc : ¬ Set.Countable A) :
  ∃ P : Set X, Perfect P ∧ P.Nonempty ∧ P ⊆ A := by
  classical
  obtain ⟨C, P, hC, hP, hdecomp⟩ :=
    exists_countable_union_perfect_of_isClosed (α := X) (C := A) hA
  have hPsub : P ⊆ A := by
    intro x hx
    rw [hdecomp]
    exact Or.inr hx
  have hPne : P.Nonempty := by
    by_contra hPne
    have hPempty : P = ∅ := by
      ext x
      constructor
      · intro hx
        exact (hPne ⟨x, hx⟩).elim
      · intro hx
        exact False.elim (by simpa using hx)
    have hAcount : A.Countable := by
      rw [hdecomp, hPempty]
      simpa using hC
    exact hunc hAcount
  exact ⟨P, hP, hPne, hPsub⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_2_28__mh__87abe271

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 87abe271b87ea292

4.253 253. closed_set_countable_or_contains_perfect

The problem

Prove the Cantor–Bendixson alternative: every closed subset of a separable metric space is either at most countable or contains a nonempty perfect subset.

Lean statement

```
theorem closed_set_countable_or_contains_perfect (X : Type*) [MetricSpace X] [SeparableSpace X]
  (A : Set X) (hA : IsClosed A) :
  Set.Countable A ∨ ∃ P : Set X, P.Nonempty ∧ Perfect P ∧ P ⊆ A := by
```

Parent. Prove that every closed set in a separable metric space is the union of a (possibly empty) perfect set and a set which is at most countable.

Parent in Lean

```
theorem Rudin_exercise_2_28 (X : Type*) [MetricSpace X] [SeparableSpace X]
  (A : Set X) (hA : IsClosed A) :
  ∃ P₁ P₂ : Set X, A = P₁ ∪ P₂ ∧ Perfect P₁ ∧ Set.Countable P₂ := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the decomposition conclusion into a countable-or-nonempty-perfect-subset dichotomy. Beyond obtaining the Cantor–Bendixson decomposition, it requires a new nonemptiness case split and a separate countability argument for the empty perfect component.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the decomposition conclusion into a genuine countability-versus-perfect-subset dichotomy. Its proof requires a nonemptiness case split, subset extraction, and deriving countability when the perfect component is empty; this reasoning is not supplied by the parent's direct repackaging proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the decomposition existence claim into a countability-versus-perfect-subset dichotomy. It requires a new case split on whether the perfect component is nonempty, plus deriving countability of the closed set when that component is empty; the parent's repackaging proof does not supply this reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2195 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	MISMATCH: Statement B says A is countable, which may mean countably infinite, whereas Statement A allows A to be finite or countably infinite (at most countable).

Goal round-trip: what Lean elaborated

```
theorem closed_set_countable_or_contains_perfect.extracted_1_1.{u_1} (X : Type u_1) [inst : MetricSpace X]
  [inst_1 : SeparableSpace X] (A : Set X) (hA : IsClosed A) : A.Countable ∨ ∃ P, P.Nonempty ∧ Perfect P ∧ P ⊆ A
```

Read back by a model that saw only the Lean. *Let X be a separable metric space, let A be a closed subset of X . Then either A is countable, or there exists a nonempty perfect subset P of X such that P is contained in A .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem closed_set_countable_or_contains_perfect (X : Type*) [MetricSpace X] [SeparableSpace X]
  (A : Set X) (hA : IsClosed A) :
  Set.Countable A ∨ ∃ P : Set X, P.Nonempty ∧ Perfect P ∧ P ⊆ A := by
  classical
  obtain ⟨V, D, hV, hD, hdecomp⟩ :=
    exists_countable_union_perfect_of_isClosed (α := X) (C := A) hA
  by_cases hDne : D.Nonempty
  · right
    have hsubset : D ⊆ A := by
      intro x hx
      rw [hdecomp]
      exact Or.inr hx
    exact ⟨D, hDne, hD, hsubset⟩
  · left
    have hDempty : D = ∅ := Set.not_nonempty_iff_eq_empty.mp hDne
    have hAcount : Set.Countable (V ∪ D) := by
      simpa [hDempty] using hV
    simpa [hdecomp] using hAcount
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_2_28__mh__b4fbdc16

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint b4fbdc163b4ce6c9

4.254 254. closed_set_disjoint_perfect_countable

The problem

Prove that every closed set in a separable metric space is the disjoint union of a perfect set and a countable set, with both pieces contained in the original set.

Lean statement

```
theorem closed_set_disjoint_perfect_countable (X : Type*) [MetricSpace X] [SeparableSpace X]
  (A : Set X) (hA : IsClosed A) :
  ∃ P C : Set X, A = P ∪ C ∧ Disjoint P C ∧ Perfect P ∧ Set.Countable C ∧ P ⊆ A ∧ C ⊆ A
```

Parent. Prove that every closed set in a separable metric space is the union of a (possibly empty) perfect set and a set which is at most countable.

Parent in Lean

```
theorem Rudin_exercise_2_28 (X : Type*) [MetricSpace X] [SeparableSpace X]
  (A : Set X) (hA : IsClosed A) :
  ∃ P₁ P₂ : Set X, A = P₁ ∪ P₂ ∧ Perfect P₁ ∧ Set.Countable P₂ := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the decomposition by requiring the perfect and countable pieces to be disjoint and both contained in A. Its proof must construct a new remainder via set difference and establish disjointness, inclusions, and preservation of the decomposition; this reasoning is not supplied by the parent.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens the decomposition by requiring the perfect and countable pieces to be disjoint, while preserving coverage and adding subset obligations. Its proof must construct a modified countable component via set difference and establish the new decomposition and disjointness, which the parent's direct theorem application does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child strengthens the decomposition to a disjoint perfect/countable partition and explicitly proves both components are subsets of A. This requires constructing the remainder as a set difference and establishing disjointness, which the parent's direct decomposition proof does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2247 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem closed_set_disjoint_perfect_countable.extracted_1.1.{u_1} (X : Type u_1) [inst : MetricSpace X]
  [inst.1 : SeparableSpace X] (A : Set X) (hA : IsClosed A) :
  ∃ P C, A = P ∪ C ∧ Disjoint P C ∧ Perfect P ∧ C.Countable ∧ P ⊆ A ∧ C ⊆ A
```

Read back by a model that saw only the Lean. *For any separable metric space X , any subset A of X that is closed can be written as $A = P \cup C$ for some subsets P and C of X such that P and C are disjoint, P is perfect, C is countable, $P \subseteq A$, and $C \subseteq A$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem closed_set_disjoint_perfect_countable (X : Type*) [MetricSpace X] [SeparableSpace X]
  (A : Set X) (hA : IsClosed A) :
  ∃ P C : Set X, A = P ∪ C ∧ Disjoint P C ∧ Perfect P ∧ Set.Countable C ∧ P ⊆ A ∧ C ⊆ A := by
  classical
  obtain ⟨V, D, hVcount, hDperfect, hdecomp⟩ :=
    exists_countable_union_perfect_of_isClosed (α := X) (C := A) hA
  have hDsub : D ⊆ A := by
    intro x hx
    rw [hdecomp]
    exact Or.inr hx
  have hVsub : V ⊆ A := by
    intro x hx
    rw [hdecomp]
    exact Or.inl hx
  let C : Set X := V \ D
  have hnewdecomp : A = D ∪ C := by
    ext x
    constructor
    · intro hx
      rw [hdecomp] at hx
      rcases hx with hxV | hxD
      · by_cases hxD' : x ∈ D
        · exact Or.inl hxD'
        · exact Or.inr ⟨hxV, hxD'⟩
      · exact Or.inl hxD
    · intro hx
      rw [hdecomp]
      rcases hx with hxD | hxC
      · exact Or.inr hxD
      · exact Or.inl hxC.1
  have hdisj : Disjoint D C := by
    refine Set.disjoint_left.2 ?_
    intro x hxD hxC
    exact hxC.2 hxD
  have hCcount : Set.Countable C := by
    exact hVcount.mono (by intro x hx; exact hx.1)
  have hCsub : C ⊆ A := by
    intro x hx
    exact hVsub hx.1
  exact ⟨D, C, hnewdecomp, hdisj, hDperfect, hCcount, hDsub, hCsub⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_2_28__mh__c14d2d0c

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint c14d2d0c752f1883

4.255 255. closed_compact_perfect_countable_with_dense_perfect_part

The problem

Let A be a closed subset of a compact metric space. Show that A is the union of a perfect set P and a countable set, and that the subspace P has a countable dense subset.

Lean statement

```
theorem closed_compact_perfect_countable_with_dense_perfect_part
  {K : Type*} [MetricSpace K] [CompactSpace K]
  (A : Set K) (hA : IsClosed A) :
  ∃ P C : Set K, A = P ∪ C ∧ Perfect P ∧ C.Countable ∧
    ∃ s : Set {x // x ∈ P}, s.Countable ∧ Dense s
```

Parent. Let A be a closed subset of a compact metric space. Show that A is the union of a perfect set P and a countable set, and that P has a countable basis obtained by restricting a countable basis of the ambient space.

Parent in Lean

```
theorem closed_compact_perfect_countable_with_restricted_basis
  {K : Type*} [MetricSpace K] [CompactSpace K]
  (A : Set K) (hA : IsClosed A) :
  ∃ P C : Set K, ∃ B : Set (Set K), ∃ R : Set (Set {x // x ∈ P}),
    A = P ∪ C ∧ Perfect P ∧ C.Countable ∧
    B.Countable ∧ TopologicalSpace.IsTopologicalBasis B ∧
    R = Set.preimage (Subtype.val : {x // x ∈ P} → K) '' B ∧
    R.Countable ∧ TopologicalSpace.IsTopologicalBasis R
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the obligation from constructing a global basis and its restricted image to proving that the perfect subtype has a countable dense subset. Its proof requires compactness of the perfect part and separability of the subtype, which the parent's basis-construction proof does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the proof obligation from constructing a restricted countable topological basis to proving existence of a countable dense subset of the perfect part. This requires the new separability argument and 'exists_countable_dense', which the parent's basis-construction proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from constructing a countable topological basis and its restricted image to constructing a countable dense subset of the perfect part. Its proof requires a new compact-subtype separability and dense-set argument that the parent's basis construction does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2264 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	MISMATCH: Statement B requires a countable dense subset of K , whereas Statement A requires a countable dense subset of the subspace P ; B omits the latter requirement.

Goal round-trip: what Lean elaborated

```
theorem closed_compact_perfect_countable_with_dense_perfect_part.extracted_1_1.{u_1} {K : Type u_1}
  [inst : MetricSpace K] [inst_1 : CompactSpace K] (A : Set K) (hA : IsClosed A) :
  ∃ P C, A = P ∪ C ∧ Perfect P ∧ C.Countable ∧ ∃ s, s.Countable ∧ Dense s
```

Read back by a model that saw only the Lean. *Let K be a compact metric space, and let A be a closed subset of K . Then there exist subsets P and C of K such that $A = P \cup C$, P is perfect, C is countable, and there exists a countable dense subset s of K .*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem closed_compact_perfect_countable_with_dense_perfect_part
  {K : Type*} [MetricSpace K] [CompactSpace K]
  (A : Set K) (hA : IsClosed A) :
  ∃ P C : Set K, A = P ∪ C ∧ Perfect P ∧ C.Countable ∧
  ∃ s : Set {x // x ∈ P}, s.Countable ∧ Dense s := by
  classical
  have hSecond : SecondCountableTopology K := inferInstance
  let I : SecondCountableTopology K := hSecond
  obtain ⟨C, P, hCcount, hP, hdecomp⟩ :=
    exists_countable_union_perfect_of_isClosed (α := K) (C := A) hA
  have hCompactP : IsCompact P := hP.closed.isCompact
  have hCompactSubtype : CompactSpace {x // x ∈ P} :=
    isCompact_iff_compactSpace.mp hCompactP
  let I := hCompactSubtype
  have hSeparable : TopologicalSpace.SeparableSpace {x // x ∈ P} := inferInstance
  obtain ⟨s, hs, hd⟩ :=
    TopologicalSpace.exists_countable_dense {x // x ∈ P}
  have hDecomp : A = P ∪ C := hdecomp.trans (Set.union_comm C P)
  have hDensePart : ∃ s : Set {x // x ∈ P}, s.Countable ∧ Dense s :=
    ⟨s, hs, hd⟩
  exact ⟨P, C, hDecomp, hP, hCcount, hDensePart⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_2_28__x__Rudin_exercise_2_25__xe.mh__6cee1840

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 6cee184021acdc6

4.256 256. uniformContinuousOn_typed_bounded_chain

The problem

Let $(X_k)_{k \geq 0}$ be a sequence of uniform spaces and let $f_k : X_k \rightarrow X_{k+1}$. If the partial composites $F_k : X_0 \rightarrow X_k$ begin with the identity and satisfy $F_{k+1} = f_k \circ F_k$, and each f_k is uniformly continuous on $F_k(S)$, prove that F_n is uniformly continuous on S .

Lean statement

```
theorem uniformContinuousOn_typed_bounded_chain
  {X : ℕ → Type*} [∀ k, UniformSpace (X k)]
  {s : Set (X 0)}
  (f : ∀ k : ℕ, X k → X (k + 1))
  (F : ∀ k : ℕ, X 0 → X k)
  (n : ℕ)
  (hF0 : F 0 = id)
  (hFsucc : ∀ k : ℕ, F (k + 1) = f k ∘ F k)
  (hcont : ∀ k : ℕ, UniformContinuousOn (f k) (F k '' s)) :
  UniformContinuousOn (F n) s
```

Parent. Let $(f_k)_{k \geq 0}$ be a sequence of self-maps of a uniform space and let S be a subset of that space. If each f_k is uniformly continuous on the image of S after the preceding k stages, prove that the composite of the first n stages is uniformly continuous on S .

Parent in Lean

```
theorem uniformContinuousOn_bounded_chain
  {α : Type*} [UniformSpace α]
  {s : Set α} (f : ℕ → α → α) (n : ℕ)
  (hcont : ∀ k, UniformContinuousOn (f k)
    ((Nat.rec (id : α → α) (fun j g => f j ∘ g) k) '' s)) :
  UniformContinuousOn
    (Nat.rec (id : α → α) (fun j g => f j ∘ g) n) s
```

What it contributes. Supplied the finite-stage induction and local composition pattern; this version generalizes it to heterogeneous spaces $X k$ with explicit typed image transport.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes from self-maps on one uniform space to a dependent typed chain with different spaces and explicitly propagated composites. The parent's definitional recursion and simplification no longer apply; the proof must use the supplied recurrence and typed composition across spaces.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the composition law to a dependent sequence of distinct uniform spaces and arbitrary recursively specified maps, so the parent's concrete self-map recursion cannot be recalled verbatim. The proof must use the typed recurrence for F and propagate image domains through the heterogeneous chain.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the composition law from self-maps on one uniform space to a dependent chain of distinct uniform spaces. The parent's proof does not directly apply because composition and image domains must be type-propagated through the indexed family.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2915 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem uniformContinuousOn_typed_bounded_chain.extracted_1_1.{u_1} {X : ℕ → Type u_1}
  [inst : (k : ℕ) → UniformSpace (X k)] {s : Set (X 0)} (f : (k : ℕ) → X k → X (k + 1)) (F : (k : ℕ) → X 0 → X k)
  (n : ℕ) (hF0 : F 0 = id) (hFsucc : ∀ (k : ℕ), F (k + 1) = f k ∘ F k)
  (hcont : ∀ (k : ℕ), UniformContinuousOn (f k) (F k '' s)) : UniformContinuousOn (F n) s
```

Read back by a model that saw only the Lean. Let $(X_k)_{k \in \mathbb{N}}$ be a family of types, each equipped with a uniform space, and let s be a subset of X_0 . Suppose $f_k : X_k \rightarrow X_{k+1}$ and $F_k : X_0 \rightarrow X_k$ are families of functions, and let $n \in \mathbb{N}$. If F_0 is the identity function on X_0 , if for every $k \in \mathbb{N}$ we have $F_{k+1} = f_k \circ F_k$, and if for every $k \in \mathbb{N}$ the function f_k is uniformly continuous on the image $F_k(s)$, then F_n is uniformly continuous on s .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem uniformContinuousOn_typed_bounded_chain
  {X : ℕ → Type*} [∀ k, UniformSpace (X k)]
  {s : Set (X 0)}
  (f : ∀ k : ℕ, X k → X (k + 1))
  (F : ∀ k : ℕ, X 0 → X k)
  (n : ℕ)
  (hF0 : F 0 = id)
  (hFsucc : ∀ k : ℕ, F (k + 1) = f k ∘ F k)
  (hcont : ∀ k : ℕ, UniformContinuousOn (f k) (F k '' s)) :
  UniformContinuousOn (F n) s := by
  induction n with
  | zero =>
    rw [hF0]
    exact uniformContinuous_id.uniformContinuousOn
  | succ n ih =>
    have hmap : Set.MapsTo (F n) s (F n '' s) := by
      intro x hx
      exact (x, hx, rfl)
    have hstep : UniformContinuousOn (f n ∘ F n) s := by
      exact (hcont n).comp ih hmap
    rw [hFsucc n]
    exact hstep
```

Operator. mutation / mutation_hard **Lineage depth.** 3 **Certificate.** reproducible

Problem id. Rudin_exercise_4_12__me.mh.me.mh__7c97e722

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint 7c97e722db1740e3

4.257 257. graph_cauchy_net

The problem

Let (u_i) be a Cauchy net indexed by a preordered set, and let f and g be uniformly continuous. Show that the net $(u_i, g(f(u_i)))$ is Cauchy.

Lean statement

```
theorem graph_cauchy_net
  {α β γ ι : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [Preorder ι]
  {f : α → β} {g : β → γ} {u : ι → α}
  (hf : UniformContinuous f) (hg : UniformContinuous g) (hu : CauchySeq u) :
  CauchySeq (fun i => (u i, g (f (u i))))
```

Parent. Let (u_i) be a Cauchy net indexed by a preordered set. If f and g are uniformly continuous, show that $(g(f(u_i)))$ is Cauchy.

Parent in Lean

```
theorem composite_cauchy_net
  {α β γ ι : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [Preorder ι]
  {f : α → β} {g : β → γ} {u : ι → α}
  (hf : UniformContinuous f) (hg : UniformContinuous g) (hu : CauchySeq u) :
  CauchySeq ((g ∘ f) ∘ u)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the target to Cauchy convergence of a graph-valued sequence in a product space. Its proof requires constructing the uniformly continuous map $x \mapsto (x, g(f x))$ and transporting Cauchy-ness through it, which is additional reasoning beyond the parent's composition argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the target from preservation of Cauchy-ness under composition to Cauchy-ness of a paired graph sequence. It requires the new product-uniformity argument via 'uniformContinuous_id.prodMk', which the parent proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely stronger joint Cauchy claim for the graph-valued sequence, requiring construction of a uniform-continuous map into a product space. The parent only establishes Cauchy preservation under composition and does not supply this product-valued obligation.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2189 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem graph_cauchy_net.extracted_1_1.{u_4, u_3, u_2, u_1} {α : Type u_1} {β : Type u_2} {γ : Type u_3} {ι : Type u_4}
  [inst : UniformSpace α] [inst_1 : UniformSpace β] [inst_2 : UniformSpace γ] [inst_3 : Preorder ι] {f : α → β}
  {g : β → γ} {u : ι → α} (hf : UniformContinuous f) (hg : UniformContinuous g) (hu : CauchySeq u) :
  CauchySeq fun i => (u i, g (f (u i)))
```

Read back by a model that saw only the Lean. *Let α , β , and γ be types equipped with uniform space structures, and let ι be a preorder. Let $f : \alpha \rightarrow \beta$ and $g : \beta \rightarrow \gamma$ be functions, and let $u : \iota \rightarrow \alpha$ be a Cauchy sequence. If f and g are uniformly continuous, then the sequence $i \mapsto (u(i), g(f(u(i))))$ in the product space $\alpha \times \gamma$ is Cauchy.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem graph_cauchy_net
  {α β γ ι : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [Preorder ι]
  {f : α → β} {g : β → γ} {u : ι → α}
  (hf : UniformContinuous f) (hg : UniformContinuous g) (hu : CauchySeq u) :
  CauchySeq (fun i => (u i, g (f (u i)))) := by
  have hgf : UniformContinuous (g ∘ f) := hg.comp hf
  have hgraph : UniformContinuous (fun x => (x, g (f x))) :=
    uniformContinuous_id.prodMk hgf
  have htransport : CauchySeq ((fun x => (x, g (f x))) ∘ u) :=
    hgraph.comp_cauchySeq hu
  simpa [Function.comp_def] using htransport
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_4_12__mh.mh__2c31f62c

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 2c31f62c94f7d586

4.258 258. uniformContinuous_comp_of_on_range

The problem

If f is uniformly continuous and g is uniformly continuous on the range of f , then $g \circ f$ is uniformly continuous.

Lean statement

```
theorem uniformContinuous_comp_of_on_range
  {α β γ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ]
  {f : α → β} {g : β → γ}
  (hf : UniformContinuous f)
  (hg : UniformContinuousOn g (Set.range f)) :
  UniformContinuous (g ∘ f)
```

Parent. A uniformly continuous function of a uniformly continuous function is uniformly continuous.

Parent in Lean

```
theorem Rudin_exercise_4_12
  {α β γ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ]
  {f : α → β} {g : β → γ}
  (hf : UniformContinuous f) (hg : UniformContinuous g) :
  UniformContinuous (g ∘ f) := by
```

What it contributes. Preserves the composition checkpoint while weakening the outer-function hypothesis from global uniform continuity to uniform continuity on the range of the inner function.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely weakens the outer function's hypothesis from global uniform continuity to uniform continuity on the range of f . Its proof must establish that f maps into this range and combine that restricted continuity with hf , so the parent's direct composition proof does not suffice.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely weakens the outer-function hypothesis from global uniform continuity to uniform continuity on the range of f . Its proof must use the on-range composition lemma and establish range membership, which the parent's direct ' $hg.comp hf$ ' argument cannot supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely weakens the outer hypothesis from global uniform continuity to uniform continuity on the range of f . Proving the composition requires restricting f to its uniform-continuous-on-univ form and explicitly establishing that $f x$ lies in $\text{Set.range } f$, so the parent's direct composition proof does not suffice.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2007 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem uniformContinuous_comp_of_on_range.extracted_1_1.{u_3, u_2, u_1} {α : Type u_1} {β : Type u_2} {γ : Type u_3}
  [inst_1 : UniformSpace α] [inst_2 : UniformSpace β] [inst_3 : UniformSpace γ] {f : α → β} {g : β → γ}
  (hf : UniformContinuous f) (hg : UniformContinuousOn g (Set.range f)) : UniformContinuous (g ∘ f)
```

Read back by a model that saw only the Lean. *Let α , β , and γ be types equipped with uniform spaces. Let $f : \alpha \rightarrow \beta$ and $g : \beta \rightarrow \gamma$. If f is uniformly continuous and g is uniformly continuous on the range of f , then the composition $g \circ f : \alpha \rightarrow \gamma$ is uniformly continuous.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem uniformContinuous_comp_of_on_range
  {α β γ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ]
  {f : α → β} {g : β → γ}
  (hf : UniformContinuous f)
  (hg : UniformContinuousOn g (Set.range f)) :
  UniformContinuous (g ∘ f) := by
  apply uniformContinuousOn_univ.mp
  refine hg.comp hf.uniformContinuousOn ?_
  intro x hx
  exact ⟨x, rfl⟩
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_4_12__mh__bb8290d4

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 1565e00b8634bb60

4.259 259. composite_cauchy_net

The problem

Let (u_i) be a Cauchy net indexed by a preordered set. If f and g are uniformly continuous, show that $(g(f(u_i)))$ is Cauchy.

Lean statement

```
theorem composite_cauchy_net
  {α β γ ι : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [Preorder ι]
  {f : α → β} {g : β → γ} {u : ι → α}
  (hf : UniformContinuous f) (hg : UniformContinuous g) (hu : CauchySeq u) :
  CauchySeq ((g ∘ f) ∘ u)
```

Parent. A uniformly continuous function of a uniformly continuous function is uniformly continuous.

Parent in Lean

```
theorem Rudin_exercise_4_12
  {α β γ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ]
  {f : α → β} {g : β → γ}
  (hf : UniformContinuous f) (hg : UniformContinuous g) :
  UniformContinuous (g ∘ f) := by
```

What it contributes. Supplies the intermediate checkpoint that the composite $g \circ f$ is uniformly continuous.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child requires an additional mathematical step: transporting a Cauchy net through the uniformly continuous composite using ‘comp_cauchySeq’. The parent’s composition proof alone does not establish Cauchy preservation, so recalling it is insufficient.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from uniform continuity of a composite to preservation of a Cauchy sequence under that composite. It requires both composing uniform-continuity proofs and applying the separate Cauchy-transport lemma, which the parent’s proof does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires the additional mathematical step that a uniformly continuous map sends a Cauchy sequence to a Cauchy sequence, after composing the maps. Recalling the parent’s composition proof supplies only the intermediate uniform-continuity fact and does not solve the new Cauchy conclusion.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2183 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem composite_cauchy_net.extracted_1_1.{u_4, u_3, u_2, u_1} {α : Type u_1} {β : Type u_2} {γ : Type u_3}
  {ι : Type u_4} [inst : UniformSpace α] [inst_1 : UniformSpace β] [inst_2 : UniformSpace γ] [inst_3 : Preorder ι]
  {f : α → β} {g : β → γ} {u : ι → α} (hf : UniformContinuous f) (hg : UniformContinuous g) (hu : CauchySeq u) :
  CauchySeq ((g ∘ f) ∘ u)
```

Read back by a model that saw only the Lean. Let α , β , and γ be types equipped with uniform spaces, and let ι be a type equipped with a preorder. For functions $f : \alpha \rightarrow \beta$, $g : \beta \rightarrow \gamma$, and $u : \iota \rightarrow \alpha$, assume that f and g are uniformly continuous and that u is a Cauchy sequence (net). Then the net obtained by applying g after f to u , namely $\iota \mapsto g(f(u(i)))$, is a Cauchy sequence (net) in γ .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem composite_cauchy_net
  {α β γ ι : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [Preorder ι]
  {f : α → β} {g : β → γ} {u : ι → α}
  (hf : UniformContinuous f) (hg : UniformContinuous g) (hu : CauchySeq u) :
  CauchySeq ((g ∘ f) ∘ u) := by
  have hcomp : UniformContinuous (g ∘ f) := hg.comp hf
  have htransport : CauchySeq ((g ∘ f) ∘ u) := hcomp.comp_cauchySeq hu
  exact htransport
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_4_12__mh__fcde3616

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint fcde3616b61ab92a

4.260 260. uniformContinuousOn_composed_output_pair

The problem

Let uniformly continuous maps be composed, and replace the first map by a map agreeing with it on a set. Prove that the ordered pair of the intermediate and final outputs is uniformly continuous on that set.

Lean statement

```
theorem uniformContinuousOn_composed_output_pair
  {α β γ δ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [UniformSpace δ]
  {f : α → β} {g : β → γ} {h : γ → δ}
  (hf : UniformContinuous f) (hg : UniformContinuous g) (hh : UniformContinuous h)
  (s : Set α) (m : α → β) (hm : ∀ x, x ∈ s → m x = f x) :
  UniformContinuousOn (fun x => (g (m x), h (g (m x)))) s
```

Parent. Let three uniformly continuous maps be composed, and let the first map be replaced on a set by another map that agrees with it there. Prove that the resulting three-stage composition is uniformly continuous on that set.

Parent in Lean

```
theorem uniformContinuousOn_three_stage_local_agreement
  {α β γ δ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [UniformSpace δ]
  {f : α → β} {g : β → γ} {h : γ → δ}
  (hf : UniformContinuous f) (hg : UniformContinuous g) (hh : UniformContinuous h)
  (s : Set α) (m : α → β) (hm : ∀ x, x ∈ s → m x = f x) :
  UniformContinuousOn (fun x => h (g (m x))) s
```

What it contributes. Reuses the parent’s local-agreement transfer and successive range-based composition checkpoints, with the final obligation strengthened to uniform continuity of an output pair.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the codomain and conclusion to uniform continuity of a paired intermediate/final output. The parent’s composition proof does not establish this; the child requires constructing a product-valued uniformly continuous map and composing it locally.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the codomain from a single composed output to a paired output and requires proving uniform continuity of the product map, using a new product-composition argument absent from the parent proof. This is a genuine change of claim rather than decoration or recall.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the codomain from a single composed output to a paired output and requires proving uniform continuity of both components via a product construction. The parent’s composition argument alone does not establish this pair-valued conclusion, so recalling the parent is insufficient.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable

Check	By	Result
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2178 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem uniformContinuousOn_composed_output_pair.extracted_1_1.{u_4, u_3, u_2, u_1} {α : Type u_1} {β : Type u_2}
  {γ : Type u_3} {δ : Type u_4} [inst : UniformSpace α] [inst_1 : UniformSpace β] [inst_2 : UniformSpace γ]
  [inst_3 : UniformSpace δ] {f : α → β} {g : β → γ} {h : γ → δ} (hf : UniformContinuous f) (hg : UniformContinuous g)
  (hh : UniformContinuous h) (s : Set α) (m : α → β) (hm : ∀ x ∈ s, m x = f x) :
  UniformContinuousOn (fun x => (g (m x), h (g (m x)))) s
```

Read back by a model that saw only the Lean. Let α, β, γ , and δ be types equipped with uniform space structures. Let $f : \alpha \rightarrow \beta$, $g : \beta \rightarrow \gamma$, and $h : \gamma \rightarrow \delta$ be uniformly continuous functions. Let s be a subset of α , and let $m : \alpha \rightarrow \beta$ be a function such that for every $x \in s$, $m(x) = f(x)$. Then the function $x \mapsto (g(m(x)), h(g(m(x))))$ from α to $\gamma \times \delta$ is uniformly continuous on s .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem uniformContinuousOn_composed_output_pair
  {α β γ δ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [UniformSpace δ]
  {f : α → β} {g : β → γ} {h : γ → δ}
  (hf : UniformContinuous f) (hg : UniformContinuous g) (hh : UniformContinuous h)
  (s : Set α) (m : α → β) (hm : ∀ x, x ∈ s → m x = f x) :
  UniformContinuousOn (fun x => (g (m x), h (g (m x)))) s := by
  have hmOn : UniformContinuousOn m s :=
    hf.uniformContinuousOn.congr (fun x hx => (hm x hx).symm)
  have hhg : UniformContinuous (h ∘ g) := hh.comp hg
  have hgh : UniformContinuous (fun y : β => (g y, (h ∘ g) y)) :=
    hg.prodMk hhg
  have hcomp : UniformContinuousOn ((fun y : β => (g y, (h ∘ g) y)) ∘ m) s :=
    hgh.comp_uniformContinuousOn hmOn
  simpa [Function.comp_def] using hcomp
```

Operator. mutation / mutation_hard **Lineage depth.** 3 **Certificate.** reproducible

Problem id. Rudin_exercise_4_12__ms.me.me.mh__51bf287b

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 51bf287bbfed901e

4.261 261. uniformContinuous_pair_with_three_stage

The problem

If three maps between uniform spaces are uniformly continuous, and another uniformly continuous map has the same domain and codomain as their three-stage composition, then the paired map formed from these two outputs is uniformly continuous.

Lean statement

```
theorem uniformContinuous_pair_with_three_stage
  {α β γ δ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [UniformSpace δ]
  {f : α → β} {g : β → γ} {h : γ → δ} {q : α → δ}
  (hf : UniformContinuous f) (hg : UniformContinuous g) (hh : UniformContinuous h)
  (hq : UniformContinuous q) :
  UniformContinuous (fun x => (q x, h (g (f x))))
```

Parent. If three maps are uniformly continuous, then their three-stage composition is uniformly continuous.

Parent in Lean

```
theorem uniformContinuous_three_stage
  {α β γ δ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [UniformSpace δ]
  {f : α → β} {g : β → γ} {h : γ → δ}
  (hf : UniformContinuous f) (hg : UniformContinuous g) (hh : UniformContinuous h) :
  UniformContinuous (fun x => h (g (f x)))
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The conclusion changes from continuity of a single three-stage composition to continuity of a paired map with an independent uniformly continuous component. The proof must construct continuity into a product via ‘prodMk’, an obligation absent from the parent’s composition-only proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the obligation from continuity of a single composite to continuity of a product-valued map, requiring the separate product-continuity construction via ‘prodMk’ and an additional branch ‘q’; the parent’s composition proof alone does not establish this.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the obligation from uniform continuity of a single three-stage composition to uniform continuity of a product-valued map. Proving it requires the additional product-continuity construction for the independent map q, which the parent proof does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2160 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem uniformContinuous_pair_with_three_stage.extracted_1_1.{u_4, u_3, u_2, u_1} {α : Type u_1} {β : Type u_2}
{γ : Type u_3} {δ : Type u_4} [inst : UniformSpace α] [inst_1 : UniformSpace β] [inst_2 : UniformSpace γ]
[inst_3 : UniformSpace δ] {f : α → β} {g : β → γ} {h : γ → δ} {q : α → δ} (hf : UniformContinuous f)
(hg : UniformContinuous g) (hh : UniformContinuous h) (hq : UniformContinuous q) :
UniformContinuous fun x => (q x, h (g (f x)))
```

Read back by a model that saw only the Lean. Let α, β, γ , and δ be types equipped with uniform spaces. Let $f : \alpha \rightarrow \beta$, $g : \beta \rightarrow \gamma$, $h : \gamma \rightarrow \delta$, and $q : \alpha \rightarrow \delta$. Assume that f, g, h , and q are uniformly continuous. Then the function from α to $\delta \times \delta$ defined by $x \mapsto (q(x), h(g(f(x))))$ is uniformly continuous.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem uniformContinuous_pair_with_three_stage
{α β γ δ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ] [UniformSpace δ]
{f : α → β} {g : β → γ} {h : γ → δ} {q : α → δ}
(hf : UniformContinuous f) (hg : UniformContinuous g) (hh : UniformContinuous h)
(hq : UniformContinuous q) :
UniformContinuous (fun x => (q x, h (g (f x)))) := by
exact hq.prodMk (hh.comp (hg.comp hf))
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Rudin_exercise_4_12__ms.me.mh__a2494e74

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint a2494e744469d309

4.262 262. uniformContinuous_paired_composite

The problem

If $f : \alpha \rightarrow \beta$ and $g : \beta \rightarrow \gamma$ are uniformly continuous, then the map $x \mapsto (f(x), g(f(x)))$ into $\beta \times \gamma$ is uniformly continuous.

Lean statement

```
theorem uniformContinuous_paired_composite
  {α β γ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ]
  {f : α → β} {g : β → γ}
  (hf : UniformContinuous f) (hg : UniformContinuous g) :
  UniformContinuous (fun x => (f x, g (f x)))
```

Parent. A uniformly continuous map followed by another uniformly continuous map is uniformly continuous, even when the intermediate map is named separately and agrees pointwise with the first.

Parent in Lean

```
theorem uniformContinuous_named_intermediate
  {α β γ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ]
  {f : α → β} {g : β → γ}
  (hf : UniformContinuous f) (hg : UniformContinuous g)
  (m : α → β) (hm : ∀ x, m x = f x) :
  UniformContinuous (fun x => g (m x))
```

What it contributes. Supplied the composition checkpoint `hcomp : UniformContinuous (g ∘ f)`.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the codomain and conclusion to a product-valued map, requiring the product uniform-continuity construction ‘`hf.prodMk hcomp`’ in addition to composition. The parent proves only continuity of the composite and cannot directly supply this paired-map result.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from continuity of a single composite to continuity of a product-valued map, requiring the product construction and continuity of both components. The parent’s proof of the composite alone does not supply the paired-map argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from continuity of a scalar composite to uniform continuity of a product-valued map. Although it reuses the composition argument, it additionally requires constructing continuity of the paired map via the product lemma, which the parent’s proof does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2147 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem uniformContinuous_paired_composite.extracted_1_1.{u_3, u_2, u_1} {α : Type u_1} {β : Type u_2} {γ : Type u_3}
  [inst : UniformSpace α] [inst_1 : UniformSpace β] [inst_2 : UniformSpace γ] {f : α → β} {g : β → γ}
  (hf : UniformContinuous f) (hg : UniformContinuous g) : UniformContinuous fun x => (f x, g (f x))
```

Read back by a model that saw only the Lean. *Let α , β , and γ be types equipped with uniform space structures. If $f : \alpha \rightarrow \beta$ and $g : \beta \rightarrow \gamma$ are uniformly continuous, then the map from α to $\beta \times \gamma$ defined by $x \mapsto (f(x), g(f(x)))$ is uniformly continuous.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem uniformContinuous_paired_composite
  {α β γ : Type*} [UniformSpace α] [UniformSpace β] [UniformSpace γ]
  {f : α → β} {g : β → γ}
  (hf : UniformContinuous f) (hg : UniformContinuous g) :
  UniformContinuous (fun x => (f x, g (f x))) := by
  have hcomp : UniformContinuous (g ∘ f) := hg.comp hf
  simpa [Function.comp_apply] using hf.prodMk hcomp
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_4_12__ms.mh__ab5b4a93

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint ab5b4a93f6986564

4.263 263. finite_existential_pointwise_order_closed

The problem

Let X be a metric space and let a finite family of pairs of continuous real-valued functions be given on X . Prove that the set of points where at least one first function is at most its paired second function is closed.

Lean statement

```
theorem finite_existential_pointwise_order_closed
  {α ι : Type} [MetricSpace α]
  (s : Finset ι)
  (f g : ι → α → ℝ)
  (hf : ∀ i, Continuous (f i))
  (hg : ∀ i, Continuous (g i)) :
  IsClosed {x : α | ∃ i ∈ s, f i x ≤ g i x} := by
```

Parent. Let X be a metric space and let a finite nonempty family of pairs of continuous real-valued functions be given on X . Prove that the set of points where every first function is at most its paired second function is closed.

Parent in Lean

```
theorem finite_common_pointwise_order_closed
  {α ι : Type} [MetricSpace α]
  (s : Finset ι) (hs : s.Nonempty)
  (f g : ι → α → ℝ)
  (hf : ∀ i, Continuous (f i))
  (hg : ∀ i, Continuous (g i)) :
  IsClosed {x : α | ∀ i ∈ s, f i x ≤ g i x} := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the proof obligation from a finite intersection (universal condition) to a finite union (existential condition), requiring union closure and a different induction structure. It also correctly removes the parent's nonemptiness hypothesis because the empty union is closed.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the finite intersection of closed inequality loci into a finite union, requiring closure under finite unions and an existential disjunction rather than the parent's intersection argument. A memorized proof of the parent does not directly establish this differently shaped conclusion.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a finite intersection of closed order sets into a finite union, requiring a different induction normalization and the closed-union argument. The parent's intersection proof does not directly supply this obligation, and the child has no redundant hypotheses.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable

Check	By	Result
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2855 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_existential_pointwise_order_closed.extracted_1_1 {α ι : Type} [inst : MetricSpace α] (s : Finset ι)
  (f g : ι → α → ℝ) (hf : ∀ (i : ι), Continuous (f i)) (hg : ∀ (i : ι), Continuous (g i)) :
  IsClosed {x | ∃ i ∈ s, f i x ≤ g i x}
```

Read back by a model that saw only the Lean. Let α be a metric space, let ι be a type, and let s be a finite set of indices in ι . Let f and g be functions assigning to each $i \in \iota$ real-valued functions on α . Assume that for every $i \in \iota$, the functions $f(i)$ and $g(i)$ are continuous. Then the set of points $x \in \alpha$ for which there exists an index $i \in s$ such that $f(i)(x) \leq g(i)(x)$ is closed in α .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem finite_existential_pointwise_order_closed
  {α ι : Type} [MetricSpace α]
  (s : Finset ι)
  (f g : ι → α → ℝ)
  (hf : ∀ i, Continuous (f i))
  (hg : ∀ i, Continuous (g i)) :
  IsClosed {x : α | ∃ i ∈ s, f i x ≤ g i x} := by
  classical
  induction s using Finset.induction_on with
  | empty =>
    simp
  | @insert i s hi ih =>
    have h_i : IsClosed {x : α | f i x ≤ g i x} := by
      exact isClosed_le (hf i) (hg i)
    have h_s : IsClosed {x : α | ∃ j ∈ s, f j x ≤ g j x} := by
      exact ih
    have h_union : IsClosed ({x : α | f i x ≤ g i x} ∪ {x : α | ∃ j ∈ s, f j x ≤ g j x}) := by
      exact h_i.union h_s
    simpa [Finset.mem_insert, or_assoc] using h_union
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__me.me.mh__429658dc

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 429658dc689b33fb

4.264 264. finite_common_strict_order_closure_subset

The problem

Let X be a metric space and let a finite nonempty family of pairs of continuous real-valued functions be given on X . Show that the closure of the set where every first function is strictly less than its paired second function is contained in the set where every first function is at most its paired second function.

Lean statement

```
theorem finite_common_strict_order_closure_subset
  {α ι : Type} [MetricSpace α]
  (s : Finset ι)
  (f g : ι → α → ℝ)
  (hf : ∀ i, Continuous (f i))
  (hg : ∀ i, Continuous (g i)) :
  closure {x : α | ∀ i ∈ s, f i x < g i x} ⊆
  {x : α | ∀ i ∈ s, f i x ≤ g i x}
```

Parent. Let X be a metric space and let a finite nonempty family of pairs of continuous real-valued functions be given on X . Prove that the set of points where every first function is at most its paired second function is closed.

Parent in Lean

```
theorem finite_common_pointwise_order_closed
  {α ι : Type} [MetricSpace α]
  (s : Finset ι) (hs : s.Nonempty)
  (f g : ι → α → ℝ)
  (hf : ∀ i, Continuous (f i))
  (hg : ∀ i, Continuous (g i)) :
  IsClosed {x : α | ∀ i ∈ s, f i x ≤ g i x} := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the claim from closedness of a finite intersection of non-strict inequalities to a closure-containment theorem for strict inequalities. Its proof requires the new topological fact that the closure of a strict real inequality is contained in the corresponding non-strict inequality, which the parent's finite-intersection induction does not provide.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the problem from proving a finite intersection of weak-order sets is closed to proving that the closure of a strict-order feasible set lies inside the weak-order feasible set. Its proof uses closure monotonicity and the strict-to-nonstrict closure lemma, with no reliance on the parent's induction argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the result from closedness of a finite intersection of weak inequalities to a closure-containment theorem for strict inequalities. Its proof requires monotonicity of closure and the strict-to-weak closure lemma, so the parent's finite induction and intersection argument do not supply it.

Check	By	Result
statement_type_check	lean	type-checks

Check	By	Result
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2887 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_common_strict_order_closure_subset.extracted_1_1 {α ι : Type} [inst : MetricSpace α] (s : Finset ι)
  (hs : s.Nonempty) (f g : ι → α → ℝ) (hf : ∀ (i : ι), Continuous (f i)) (hg : ∀ (i : ι), Continuous (g i)) :
  closure {x | ∀ i ∈ s, f i x < g i x} ⊆ {x | ∀ i ∈ s, f i x ≤ g i x}
```

Read back by a model that saw only the Lean. Let α and ι be types, with α equipped with a metric, and let s be a nonempty finite set of indices in ι . Let f and g be functions assigning to each index i functions from α to $\mathbb{R}$. Assume that for every i , both f_i and g_i are continuous. Then every point in the closure of the set of points $x \in \alpha$ such that $f_i(x) < g_i(x)$ for every $i \in s$ also satisfies $f_i(x) \leq g_i(x)$ for every $i \in s$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem finite_common_strict_order_closure_subset
  {α ι : Type} [MetricSpace α]
  (s : Finset ι)
  (f g : ι → α → ℝ)
  (hf : ∀ i, Continuous (f i))
  (hg : ∀ i, Continuous (g i)) :
  closure {x : α | ∀ i ∈ s, f i x < g i x} ⊆
    {x : α | ∀ i ∈ s, f i x ≤ g i x} := by
  intro x hx i hi
  have hsub : {y : α | ∀ j ∈ s, f j y < g j y} ⊆
    {y : α | f i y < g i y} := by
    intro y hy
    exact hy i hi
  have hxi : x ∈ closure {y : α | f i y < g i y} :=
    closure_mono hsub hx
  exact (closure_lt_subset_le (hf i) (hg i)) hxi
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__me.me.mh__a5ad1af2

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 08ddd17158cbaa74

4.265 265. continuous_finite_minimum_of_differences

The problem

Let X be a metric space and let a finite nonempty family of pairs of continuous real-valued functions be given on X . Prove that the pointwise minimum of the differences between each second function and its paired first function is continuous.

Lean statement

```
theorem continuous_finite_minimum_of_differences
  {α ι : Type} [MetricSpace α]
  (s : Finset ι) (hs : s.Nonempty)
  (f g : ι → α → ℝ)
  (hf : ∀ i, Continuous (f i))
  (hg : ∀ i, Continuous (g i)) :
  Continuous (fun x => s.inf' hs (fun i => g i x - f i x))
```

Parent. Let X be a metric space and let a finite nonempty family of pairs of continuous real-valued functions be given on X . Prove that the set of points where every first function is at most its paired second function is closed.

Parent in Lean

```
theorem finite_common_pointwise_order_closed
  {α ι : Type} [MetricSpace α]
  (s : Finset ι) (hs : s.Nonempty)
  (f g : ι → α → ℝ)
  (hf : ∀ i, Continuous (f i))
  (hg : ∀ i, Continuous (g i)) :
  IsClosed {x : α | ∀ i ∈ s, f i x ≤ g i x} := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from closedness of a common pointwise-order set to continuity of a finite pointwise minimum of differences. Its proof requires continuity of subtraction and a finite-infimum continuity argument, neither supplied by the parent's closed-set induction.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from closedness of a finite intersection of order constraints to continuity of a finite minimum of difference functions. Its proof requires continuity under subtraction and the finite infimum continuity theorem, which the parent's induction proof does not provide.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a closed-set claim into continuity of a finite infimum of difference functions. Its proof requires subtraction continuity and a finite-infimum continuity theorem, which the parent's induction-and-intersection argument does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable

Check	By	Result
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2865 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem continuous_finite_minimum_of_differences.extracted_1_1 {α ι : Type} [inst : MetricSpace α] (s : Finset ι)
  (hs : s.Nonempty) (f g : ι → α → ℝ) (hf : ∀ (i : ι), Continuous (f i)) (hg : ∀ (i : ι), Continuous (g i)) :
  Continuous fun x => s.inf' hs fun i => g i x - f i x
```

Read back by a model that saw only the Lean. *Let α be a metric space and ι be a type. For a nonempty finite set s of indices in ι , let f and g assign to each index i continuous real-valued functions $f(i)$, $g(i)$ on α . Then the function sending $x \in \alpha$ to the minimum, over $i \in s$, of $g(i)(x) - f(i)(x)$ is continuous.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem continuous_finite_minimum_of_differences
  {α ι : Type} [MetricSpace α]
  (s : Finset ι) (hs : s.Nonempty)
  (f g : ι → α → ℝ)
  (hf : ∀ i, Continuous (f i))
  (hg : ∀ i, Continuous (g i)) :
  Continuous (fun x => s.inf' hs (fun i => g i x - f i x)) := by
  have hdiff : ∀ i, Continuous (fun x => g i x - f i x) := by
    intro i
    exact (hg i).sub (hf i)
  have haggregate : Continuous (fun x => s.inf' hs (fun i => g i x - f i x)) := by
    exact Continuous.finset_inf'_apply hs (fun i hi => hdiff i)
  exact haggregate
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__me.me.mh__b39ad3be

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint b39ad3be276287ba

4.266 266. finite_sum_order_closed

The problem

Let X be a metric space and let a finite nonempty family of pairs of continuous real-valued functions be given on X . Prove that the set of points where the sum of the first functions is at most the sum of the paired second functions is closed.

Lean statement

```
theorem finite_sum_order_closed
  {α ι : Type} [MetricSpace α]
  (s : Finset ι)
  (f g : ι → α → ℝ)
  (hf : ∀ i, Continuous (f i))
  (hg : ∀ i, Continuous (g i)) :
  IsClosed {x : α | (∑ i ∈ s, f i x) ≤ ∑ i ∈ s, g i x}
```

Parent. Let X be a metric space and let a finite nonempty family of pairs of continuous real-valued functions be given on X . Prove that the set of points where every first function is at most its paired second function is closed.

Parent in Lean

```
theorem finite_common_pointwise_order_closed
  {α ι : Type} [MetricSpace α]
  (s : Finset ι) (hs : s.Nonempty)
  (f g : ι → α → ℝ)
  (hf : ∀ i, Continuous (f i))
  (hg : ∀ i, Continuous (g i)) :
  IsClosed {x : α | ∀ i ∈ s, f i x ≤ g i x} := by
```

What it contributes. Preserves the metric-space, finite-index-family, and continuous-function setting while changing the final obligation from an intersection of pointwise inequalities to an inequality between finite aggregates.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child replaces a finite conjunction of pointwise inequalities with a single inequality between finite sums. Its proof requires establishing continuity of indexed finite sums, which is not supplied by the parent's induction-and-intersection argument, and the nonemptiness hypothesis is genuinely unnecessary.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the mathematics from a finite intersection of pointwise inequalities to a single inequality between finite sums. Its proof requires establishing continuity of the aggregate sums and applying closedness of an order relation, so the parent's induction argument does not supply it.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the pointwise finite conjunction into a closed sublevel set for finite sums, requiring continuity of finite sums and an aggregate order argument; the parent's induction-and-intersection proof does not supply this reasoning.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry

Check	By	Result
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2852 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_sum_order_closed.extracted_1.1 {α ι : Type} [inst : MetricSpace α] (s : Finset ι) (hs : s.Nonempty)
  (f g : ι → α → ℝ) (hf : ∀ (i : ι), Continuous (f i)) (hg : ∀ (i : ι), Continuous (g i)) :
  IsClosed {x | ∑ i ∈ s, f i x ≤ ∑ i ∈ s, g i x}
```

Read back by a model that saw only the Lean. Let α and ι be types, with α equipped with a metric-space structure. Let s be a nonempty finite set of elements of ι , and let f and g be functions assigning to each $i : \iota$ continuous real-valued functions on α . Then the set of all $x \in \alpha$ such that the finite sum $\sum_{i \in s} f(i)(x)$ is less than or equal to $\sum_{i \in s} g(i)(x)$ is closed in α .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open scoped BigOperators
open Topology Filter Real Complex TopologicalSpace Finset

theorem finite_sum_order_closed
  {α ι : Type} [MetricSpace α]
  (s : Finset ι)
  (f g : ι → α → ℝ)
  (hf : ∀ i, Continuous (f i))
  (hg : ∀ i, Continuous (g i)) :
  IsClosed {x : α | (∑ i ∈ s, f i x) ≤ ∑ i ∈ s, g i x} := by
  classical
  have hfs : Continuous (fun x : α => ∑ i ∈ s, f i x) := by
    exact continuous_finset_sum s (fun i hi => hf i)
  have hgs : Continuous (fun x : α => ∑ i ∈ s, g i x) := by
    exact continuous_finset_sum s (fun i hi => hg i)
  have h_order : IsClosed {x : α | (∑ i ∈ s, f i x) ≤ ∑ i ∈ s, g i x} := by
    exact isClosed_le hfs hgs
  exact h_order
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__me.me.mh__bda8ceec

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 239e307b69e11bb5

4.267 267. continuous_equalizer_closed_hausdorff

The problem

Let X be a topological space and Y a Hausdorff space. If $f, g : X \rightarrow Y$ are continuous, prove that the set of points x satisfying $f(x) = g(x)$ is closed.

Lean statement

```
theorem continuous_equalizer_closed_hausdorff
  {α β : Type} [TopologicalSpace α] [TopologicalSpace β] [T2Space β]
  (f g : α → β) (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x : α | f x = g x} := by
```

Parent. Let f and g be continuous real-valued functions on a metric space X . Prove that the set of points x satisfying $f(x) = g(x)$ is closed.

Parent in Lean

```
theorem continuous_equalizer_closed
  {α : Type} [MetricSpace α]
  (f g : α → ℝ) (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x : α | f x = g x} := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the result from metric-space real-valued functions to continuous maps into an arbitrary Hausdorff topological space. Its proof relies on the general closed-equality theorem, which is not supplied by the parent's real-valued difference/preimage argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the result from metric spaces with real-valued functions to arbitrary topological domains and Hausdorff codomains. The parent's subtraction-and-zero-set argument no longer applies; closure must instead follow from the Hausdorff diagonal/equalizer theorem.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the result from real-valued functions on metric spaces to functions into any Hausdorff topological space. The parent's subtraction-and-zero-preimage argument no longer applies; the child requires the distinct Hausdorff equalizer-closedness theorem.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2866 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem continuous_equalizer_closed_hausdorff.extracted_1_1 {α β : Type} [inst : TopologicalSpace α]
  [inst_1 : TopologicalSpace β] [inst_2 : T2Space β] (f g : α → β) (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x | f x = g x}
```

Read back by a model that saw only the Lean. Let α and β be topological spaces, with β Hausdorff. If $f, g : \alpha \rightarrow \beta$ are continuous, then the set $\{x \in \alpha \mid f(x) = g(x)\}$ is closed in α .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem continuous_equalizer_closed_hausdorff
  {α β : Type} [TopologicalSpace α] [TopologicalSpace β] [T2Space β]
  (f g : α → β) (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x : α | f x = g x} := by
  have hclosed : IsClosed {x : α | f x = g x} := by
    exact isClosed_eq hf hg
  exact hclosed
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__me.mh.mh__7049d394

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 7049d394becbadf3

4.268 268. continuous_translated_interval_band_closed

The problem

Let f be a continuous real-valued function on a metric space X , and let $a \leq b$. Prove that the set of pairs (x, t) satisfying $a \leq f(x) - t \leq b$ is closed in $X \times \mathbb{R}$.

Lean statement

```
theorem continuous_translated_interval_band_closed
  {α : Type} [MetricSpace α]
  (f : α → ℝ) (h : Continuous f) (a b : ℝ) :
  IsClosed {p : α × ℝ | a ≤ f p.1 - p.2 ∧ f p.1 - p.2 ≤ b} := by
```

Parent. Let f be a continuous real-valued function on a metric space X , and let $a \leq b$. Prove that the set of points x satisfying $a \leq f(x) \leq b$ is closed.

Parent in Lean

```
theorem continuous_interval_preimage_closed
  {α : Type} [MetricSpace α]
  (f : α → ℝ) (h : Continuous f) (a b : ℝ) :
  IsClosed {x : α | a ≤ f x ∧ f x ≤ b} := by
```

What it contributes. Used the parent's closed-interval and continuous-preimage checkpoints, with the new product-coordinate difference map supplying the mutation.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the domain to a product and proves continuity of the coupled map $(x, t) \mapsto f(x) - t$, requiring product-space projection and subtraction reasoning absent from the parent. It is a genuine translated-band generalization, not a decorative restatement.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child requires proving continuity of the coupled map $(x, t) \mapsto f(x) - t$ on a product space, then taking its closed interval preimage. The parent only handles the preimage of an interval under f , so its proof does not supply the essential product-continuity argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves closure of a genuinely new subset of the product space: it must establish continuity of the coupled map $(x, t) \mapsto f(x) - t$ using projections and subtraction before applying the closed-interval preimage argument. The parent's one-variable continuity proof does not supply this coupling step.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2858 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem continuous_translated_interval_band_closed.extracted_1.1 {α : Type} [inst : MetricSpace α] (f : α → ℝ)
  (h : Continuous f) (a b : ℝ) : IsClosed {p | a ≤ f p.1 - p.2 ∧ f p.1 - p.2 ≤ b}
```

Read back by a model that saw only the Lean. *Let α be a type equipped with a metric, let $f : \alpha \rightarrow \mathbb{R}$ be continuous, and let $a, b \in \mathbb{R}$. Then the set of pairs $(p_1, p_2) \in \alpha \times \mathbb{R}$ such that $a \leq f(p_1) - p_2$ and $f(p_1) - p_2 \leq b$ is closed.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem continuous_translated_interval_band_closed
  {α : Type} [MetricSpace α]
  (f : α → ℝ) (h : Continuous f) (a b : ℝ) :
  IsClosed {p : α × ℝ | a ≤ f p.1 - p.2 ∧ f p.1 - p.2 ≤ b} := by
  have h_difference_continuous : Continuous (fun p : α × ℝ => f p.1 - p.2) := by
    exact (h.comp continuous_fst).sub continuous_snd
  have h_interval_closed : IsClosed (Set.Icc a b) := by
    exact isClosed_Icc
  have h_band_closed : IsClosed ((fun p : α × ℝ => f p.1 - p.2) ⁻¹' Set.Icc a b) := by
    exact h_interval_closed.preimage h_difference_continuous
  have h_set_eq :
    {p : α × ℝ | a ≤ f p.1 - p.2 ∧ f p.1 - p.2 ≤ b} =
    (fun p : α × ℝ => f p.1 - p.2) ⁻¹' Set.Icc a b := by
    ext p
    simp [Set.mem_Icc]
  rw [h_set_eq]
  exact h_band_closed
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__me.mh__08cbac12

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 08cbac121aaa9276

4.269 269. compact_level_set

The problem

Let K be a compact subset of a metric space X , and let $f : X \rightarrow \mathbb{R}$ be continuous. Prove that the set of points in K where $f(x) = c$ is compact for every $c \in \mathbb{R}$.

Lean statement

```
theorem compact_level_set
  {α : Type} [MetricSpace α]
  (K : Set α) (hK : IsCompact K)
  (f : α → ℝ) (h : Continuous f) (c : ℝ) (z : Set α)
  (g : z = K ∩ f ⁻¹' ({c} : Set ℝ)) : IsCompact z
```

Parent. Let f be a continuous real-valued function on a metric space X , and let $c \in \mathbb{R}$. Prove that the level set $\{x \in X \mid f(x) = c\}$ is closed.

Parent in Lean

```
theorem continuous_level_set_closed
  {α : Type} [MetricSpace α]
  (f : α → ℝ) (h : Continuous f) (c : ℝ) (z : Set α)
  (g : z = f ⁻¹' ({c} : Set ℝ)) : IsClosed z := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child strengthens closedness of a level set to compactness of its intersection with an arbitrary compact set. Its proof still reuses the level-set argument, but additionally requires proving the intersection is closed and invoking compactness inheritance, which the parent does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the obligation from closedness to compactness and adds the substantive argument that the level set is a closed subset of the compact set K . The parent proof only establishes closedness of the level set and does not supply the compact-subset reasoning.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes closedness of a level set into compactness of its intersection with a compact set. The parent's continuity argument supplies only closedness; the new proof additionally requires showing the intersection is a closed subset of K and invoking compactness inheritance.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2837 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem compact_level_set.extracted_1_1 {α : Type} [inst : MetricSpace α] (K : Set α) (hK : IsCompact K) (f : α → ℝ)
(h : Continuous f) (c : ℝ) (z : Set α) (g : z = K ∩ f-1 {c}) : IsCompact z
```

Read back by a model that saw only the Lean. *Let α be a type equipped with a metric space structure, let K be a compact subset of α , and let $f : \alpha \rightarrow \mathbb{R}$ be continuous. For any real number c and subset z of α , if $z = K \cap \{x \in \alpha \mid f(x) = c\}$, then z is compact.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem compact_level_set
  {α : Type} [MetricSpace α]
  (K : Set α) (hK : IsCompact K)
  (f : α → ℝ) (h : Continuous f) (c : ℝ) (z : Set α)
  (g : z = K ∩ f-1 ({c} : Set ℝ)) : IsCompact z := by
  have h_shift : Continuous (fun x => f x - c) := h.sub continuous_const
  have h_zero_closed : IsClosed ({0} : Set ℝ) := by
    simp
  have h_shift_preimage :
    IsClosed ((fun x => f x - c)-1 ({0} : Set ℝ)) :=
    IsClosed.preimage h_shift h_zero_closed
  have h_level_eq :
    (fun x => f x - c)-1 ({0} : Set ℝ) = f-1 ({c} : Set ℝ) := by
    ext x
    simp [sub_eq_zero]
  have h_preimage_closed : IsClosed (f-1 ({c} : Set ℝ)) := by
    rw [← h_level_eq]
    exact h_shift_preimage
  have h_target_closed : IsClosed (K ∩ f-1 ({c} : Set ℝ)) := by
    exact hK.isClosed.inter h_preimage_closed
  rw [g]
  apply hK.of_isClosed_subset h_target_closed
  intro x hx
  exact hx.1
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__me.mh__62b4c13e

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 62b4c13e914dfe94

4.270 270. finite_pointwise_order_locus_closed

The problem

Let X be a metric space, and let $f_1, \dots, f_n$ and $g_1, \dots, g_n$ be continuous real-valued functions on X . Prove that the set of points where $f_i(x) \leq g_i(x)$ for every i is closed.

Lean statement

```
theorem finite_pointwise_order_locus_closed
  {α : Type} [MetricSpace α]
  (n : ℕ) (f g : Fin n → α → ℝ)
  (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i))
  (z : Set α) (hz : z = {x | ∀ i, f i x ≤ g i x}) : IsClosed z
```

Parent. Let X be a metric space, and let $f_1, \dots, f_n$ and $g_1, \dots, g_n$ be continuous real-valued functions on X . Prove that the set of points where $f_i(x) = g_i(x)$ for every i is closed.

Parent in Lean

```
theorem bounded_simultaneous_equalizer_closed
  {α : Type} [MetricSpace α]
  (n : ℕ) (f g : Fin n → α → ℝ)
  (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i))
  (z : Set α) (hz : z = {x | ∀ i, f i x = g i x}) : IsClosed z := by
```

What it contributes. Retained the metric-space setting, finite indexing, continuity-of-differences argument, and finite intersection checkpoint; replaced equality with membership in the closed nonpositive ray.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child replaces equality loci with pointwise inequality loci. The parent's zero-set argument no longer applies; the proof must use the closed ray $(-\infty, 0]$ and establish the inequality via subtraction, then intersect the finite family.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes equality loci into pointwise inequality loci, requiring a new characterization as preimages of the closed ray $(-\infty, 0]$ and an arithmetic equivalence proved by linear reasoning. The parent's zero-set argument does not supply this proof.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child replaces equality loci with order loci, requiring the distinct fact that the closed ray $(-\infty, 0]$ is closed and a nontrivial equivalence between $f_i x \leq g_i x$ and $f_i x - g_i x \leq 0$. The parent's zero-set argument cannot be recalled to prove this conclusion.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2829 statements

Check	By	Result
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_pointwise_order_locus_closed.extracted_1_1 {α : Type} [inst : MetricSpace α] (n : ℕ)
  (f g : Fin n → α → ℝ) (hf : ∀ (i : Fin n), Continuous (f i)) (hg : ∀ (i : Fin n), Continuous (g i)) (z : Set α)
  (hz : z = {x | ∀ (i : Fin n), f i x ≤ g i x}) : IsClosed z
```

Read back by a model that saw only the Lean. *Let α be a metric space, let n be a natural number, and let f and g be functions from $\text{Fin } n$ to functions $\alpha \rightarrow \mathbb{R}$. Suppose that for every $i \in \text{Fin } n$, the function $x \mapsto f(i)(x)$ is continuous, and for every $i \in \text{Fin } n$, the function $x \mapsto g(i)(x)$ is continuous. If $z \subseteq \alpha$ satisfies $z = \{x \in \alpha \mid \text{for every } i \in \text{Fin } n, f(i)(x) \leq g(i)(x)\}$, then z is closed.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem finite_pointwise_order_locus_closed
  {α : Type} [MetricSpace α]
  (n : ℕ) (f g : Fin n → α → ℝ)
  (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i))
  (z : Set α) (hz : z = {x | ∀ i, f i x ≤ g i x}) : IsClosed z := by
  have hclosed : ∀ i : Fin n, IsClosed {x : α | f i x ≤ g i x} := by
    intro i
    let q : α → ℝ := fun x => f i x - g i x
    have hq : Continuous q := (hf i).sub (hg i)
    have hpre : IsClosed (q⁻¹' Set.Iic (0 : ℝ)) :=
      IsClosed.preimage hq isClosed_Iic
    have heq : {x : α | f i x ≤ g i x} = q⁻¹' Set.Iic (0 : ℝ) := by
      ext x
      change (f i x ≤ g i x) ↔ q x ≤ 0
      constructor <=> intro h <=> dsimp [q] <=> linarith
    rw [heq]
    exact hpre
  have hcne : IsClosed (⋂ i : Fin n, {x : α | f i x ≤ g i x}) :=
    isClosed_iInter hclosed
  have hz' : z = ⋂ i : Fin n, {x : α | f i x ≤ g i x} := by
    rw [hz]
    ext x
    simp
    rw [hz']
    exact hcne
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__me.mh__69f231de

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 69f231defa9ab9cd

4.271 271. finite_common_equalizer_closed

The problem

Let X be a metric space, let s be a finite index set, and let (f_i) and (g_i) be continuous real-valued functions on X . Prove that the set of points where $f_i(x) = g_i(x)$ for every $i \in s$ is closed.

Lean statement

```
theorem finite_common_equalizer_closed
  {α 1 : Type} [MetricSpace α]
  (s : Finset 1) (f g : 1 → α → ℝ)
  (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i)) :
  IsClosed {x : α | ∀ i ∈ s, f i x = g i x} := by
```

Parent. Let X be a metric space, let s be a finite index set, and let $(f_i)_{i \in s}$ be continuous real-valued functions on X . Prove that the common level locus $\{x \in X : f_i(x) = c \text{ for every } i \in s\}$ is closed.

Parent in Lean

```
theorem finite_common_level_locus_closed
  {α 1 : Type} [MetricSpace α]
  (s : Finset 1) (f : 1 → α → ℝ) (h : ∀ i, Continuous (f i)) (c : ℝ) :
  IsClosed {x : α | ∀ i ∈ s, f i x = c} := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child replaces fixed-level sets with equalizer sets for two independently varying continuous families. Proving each equalizer is closed requires a distinct closure argument via 'isClosed_eq'; the parent's singleton-preimage proof does not directly supply this step.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes a common level-set problem into a finite equalizer problem for two independently continuous function families. Its proof requires the distinct closedness theorem for equality sets, so recalling the parent's singleton-preimage argument does not directly solve it.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child generalizes a common level set to a finite intersection of equalizers between two independently varying continuous function families. The parent's singleton-preimage argument does not prove this; the child requires the distinct closed-equalizer lemma for each index.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2364 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_common_equalizer_closed.extracted_1_1 {α ι : Type} [inst : MetricSpace α] (s : Finset ι)
  (f g : ι → α → ℝ) (hf : ∀ (i : ι), Continuous (f i)) (hg : ∀ (i : ι), Continuous (g i)) :
  IsClosed {x | ∀ i ∈ s, f i x = g i x}
```

Read back by a model that saw only the Lean. Let α be a metric space and let ι be a type. For a finite set s of elements of ι , let f and g assign to each $i \in \iota$ real-valued functions on α . Assume that for every $i \in \iota$, the functions $f(i)$ and $g(i)$ are continuous. Then the set of all $x \in \alpha$ such that $f(i)(x) = g(i)(x)$ for every $i \in s$ is closed in α .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem finite_common_equalizer_closed
  {α ι : Type} [MetricSpace α]
  (s : Finset ι) (f g : ι → α → ℝ)
  (hf : ∀ i, Continuous (f i)) (hg : ∀ i, Continuous (g i)) :
  IsClosed {x : α | ∀ i ∈ s, f i x = g i x} := by
  have h_equal : ∀ i, IsClosed {x : α | f i x = g i x} := by
    intro i
    exact isClosed_eq (hf i) (hg i)
  have h_common : IsClosed (⋂ i ∈ s, {x : α | f i x = g i x}) := by
    exact isClosed_biInter (fun i hi => h_equal i)
  have h_repr :
    {x : α | ∀ i ∈ s, f i x = g i x} =
      ⋂ i ∈ s, {x : α | f i x = g i x} := by
    ext x
    simp
  rw [h_repr]
  exact h_common
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__me.mh__a08deb7d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint a08deb7d15400b3e

4.272 272. finite_common_zero_locus_sum_squares

The problem

Let X be a metric space and let $(f_i)_{i \in s}$ be finitely many continuous real-valued functions on X . Prove that their common zero locus is closed and is exactly the zero set of $\sum_{i \in s} f_i(x)^2$.

Lean statement

```
theorem finite_common_zero_locus_sum_squares
  {α 1 : Type} [MetricSpace α]
  (s : Finset 1) (f : 1 → α → ℝ) (h : ∀ i, Continuous (f i)) :
  IsClosed {x : α | ∀ i ∈ s, f i x = 0} ∧
  {x : α | ∀ i ∈ s, f i x = 0} =
  {x : α | ∑ i ∈ s, (f i x)^2 = 0}
```

Parent. Let X be a metric space, let s be a finite index set, and let $(f_i)_{i \in s}$ be continuous real-valued functions on X . Prove that the common level locus $\{x \in X : f_i(x) = c \text{ for every } i \in s\}$ is closed.

Parent in Lean

```
theorem finite_common_level_locus_closed
  {α 1 : Type} [MetricSpace α]
  (s : Finset 1) (f : 1 → α → ℝ) (h : ∀ i, Continuous (f i)) (c : ℝ) :
  IsClosed {x : α | ∀ i ∈ s, f i x = c} := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child adds a genuine algebraic characterization: it proves that the common zero locus equals the zero set of a finite sum of squares, requiring nonnegativity and a sum-of-squares zero argument beyond the parent's intersection-of-closed-preimages proof. The appended equality is substantive rather than a trivial conjunction or restatement.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a substantive characterization: the common zero locus equals the zero set of a finite sum of squares. Proving the reverse inclusion requires nontrivial sum-of-squares reasoning unavailable from the parent's closedness argument.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child adds a substantive characterization: the common zero locus equals the zero set of a finite sum of squares. Proving the converse requires a nontrivial sum-of-squares argument absent from the parent's closure proof, while the continuity hypotheses are still used for the closure conjunct.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2352 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem finite_common_zero_locus_sum_squares.extracted_1_1 {α ι : Type} [inst : MetricSpace α] (s : Finset ι)
  (f : ι → α → ℝ) (h : ∀ (i : ι), Continuous (f i)) :
  IsClosed {x | ∀ i ∈ s, f i x = 0} ∧ {x | ∀ i ∈ s, f i x = 0} = {x | ∑ i ∈ s, f i x ^ 2 = 0}
```

Read back by a model that saw only the Lean. Let α and ι be types, let α be equipped with a metric-space structure, let s be a finite subset of ι , and let f assign to each $i \in \iota$ a real-valued function on α . Assume that for every $i \in \iota$, the function $f(i)$ is continuous. Then the set of all $x \in \alpha$ such that $f(i)(x) = 0$ for every $i \in s$ is closed, and this set is exactly the set of all $x \in \alpha$ such that the finite sum $\sum_{i \in s} (f(i)(x))^2$ equals 0.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset
open scoped BigOperators

theorem finite_common_zero_locus_sum_squares
  {α ι : Type} [MetricSpace α]
  (s : Finset ι) (f : ι → α → ℝ) (h : ∀ i, Continuous (f i)) :
  IsClosed {x : α | ∀ i ∈ s, f i x = 0} ∧
  {x : α | ∀ i ∈ s, f i x = 0} =
  {x : α | ∑ i ∈ s, (f i x)^2 = 0} := by
  have h_level : ∀ i, IsClosed (f i ⁻¹' ({0} : Set ℝ)) := by
    intro i
    exact IsClosed.preimage (h i) (by simp)
  have h_common :
    IsClosed (⋂ i ∈ s, f i ⁻¹' ({0} : Set ℝ)) := by
    exact isClosed_biInter (fun i hi => h_level i)
  have h_common_eq :
    {x : α | ∀ i ∈ s, f i x = 0} = ⋂ i ∈ s, f i ⁻¹' ({0} : Set ℝ) := by
    ext x
    simp
  have h_closed : IsClosed {x : α | ∀ i ∈ s, f i x = 0} := by
    rw [h_common_eq]
    exact h_common
  have h_sum_cont : Continuous (fun x : α => ∑ i ∈ s, (f i x)^2) := by
    fun_prop
  constructor
  · exact h_closed
  · ext x
    constructor
    · intro hx
      simp only [Set.mem_setOf_eq] at hx ⊢
      apply Finset.sum_eq_zero
      intro i hi
      rw [hx i hi]
      simp
    · intro hx
      simp only [Set.mem_setOf_eq] at hx ⊢
      have hs : ∑ i ∈ s, f i x * f i x = 0 := by
        simpa [pow_two] using hx
      exact (Finset.sum_mul_self_eq_zero_iff s (fun i => f i x)).mp hs
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__me.mh__a6c49faf

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint a6c49fafba296734

4.273 273. constrained_common_zero_locus_closed

The problem

Let X be a metric space, let $(f_i)_{i \in I}$ be a family of continuous real-valued functions, and let $g : X \rightarrow \mathbb{R}$ be continuous. Prove that the set of points where every f_i vanishes and g is nonnegative is closed.

Lean statement

```
theorem constrained_common_zero_locus_closed
  {α ι : Type} [MetricSpace α]
  (f : ι → α → ℝ) (h : ∀ i, Continuous (f i))
  (g : α → ℝ) (hg : Continuous g) :
  IsClosed {x : α | (∀ i, f i x = 0) ∧ 0 ≤ g x}
```

Parent. Let X be a metric space and let $(f_i)_{i \in I}$ be a finite family of continuous real-valued functions on X . Prove that the set of points where every f_i vanishes is closed.

Parent in Lean

```
theorem common_zero_locus_closed
  {α ι : Type} [MetricSpace α] [Fintype ι]
  (f : ι → α → ℝ) (h : ∀ i, Continuous (f i)) :
  IsClosed {x : α | ∀ i, f i x = 0} := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child removes the finite-index assumption, requiring closure under an arbitrary intersection, and adds a continuous nonnegativity constraint requiring a separate preimage/intersection argument. The parent's finite sum-of-squares proof does not supply either obligation.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child removes the finite-index assumption and adds a genuinely relevant closed inequality constraint. Its proof must establish arbitrary intersection closure and preimage closure for the constraint, so the parent's finite sum-of-squares argument cannot be recalled to solve it.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

Removing the finite-index hypothesis breaks the parent's finite sum-of-squares argument, and the child must instead prove closure via arbitrary intersections. It also genuinely adds a continuous inequality constraint and proves the resulting intersection is closed.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2146 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem constrained_common_zero_locus_closed.extracted_1_1 {α ι : Type} [inst : MetricSpace α] (f : ι → α → ℝ)
(h : ∀ (i : ι), Continuous (f i)) (g : α → ℝ) (hg : Continuous g) :
IsClosed {x | (∀ (i : ι), f i x = 0) ∧ 0 ≤ g x}
```

Read back by a model that saw only the Lean. Let α and ι be types, let α be equipped with a metric-space structure, and let $f : \iota \rightarrow \alpha \rightarrow \mathbb{R}$ be such that for every $i \in \iota$, the function $x \mapsto f(i, x)$ is continuous. Let $g : \alpha \rightarrow \mathbb{R}$ be continuous. Then the set of all $x \in \alpha$ such that $f(i, x) = 0$ for every $i \in \iota$ and $0 \leq g(x)$ is closed in α .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open scoped BigOperators
open Topology Filter Real Complex TopologicalSpace Finset

theorem constrained_common_zero_locus_closed
{α ι : Type} [MetricSpace α]
(f : ι → α → ℝ) (h : ∀ i, Continuous (f i))
(g : α → ℝ) (hg : Continuous g) :
IsClosed {x : α | (∀ i, f i x = 0) ∧ 0 ≤ g x} := by
have hzero : IsClosed {x : α | ∀ i, f i x = 0} := by
  have hi : IsClosed (⋂ i, (f i)⁻¹' ({0} : Set ℝ)) := by
    exact isClosed_iInter (fun i => (isClosed_singleton : IsClosed ({0} : Set ℝ)).preimage (h i))
  have heq : {x : α | ∀ i, f i x = 0} = ⋂ i, (f i)⁻¹' ({0} : Set ℝ) := by
    ext x
    simp
  rw [heq]
  exact hi
have hconstraint : IsClosed {x : α | 0 ≤ g x} := by
  have hi : IsClosed (Set.Ici (0 : ℝ)) := isClosed_Ici
  have hp := hi.preimage hg
  simpa only [Set.mem_preimage, Set.mem_Ici, Set.mem_setOf_eq] using hp
have hinter : IsClosed ({x : α | ∀ i, f i x = 0} ∩ {x : α | 0 ≤ g x}) :=
  hzero.inter hconstraint
convert hinter using 1
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__mh.mh__3ba4bcc3

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 3ba4bcc389bdec88

4.274 274. zero_constrained_positivity_compact

The problem

Let X be a compact metric space, and let $f, g : X \rightarrow \mathbb{R}$ be continuous. Prove that the set of points where f vanishes and g is nonnegative is compact.

Lean statement

```
theorem zero_constrained_positivity_compact
  {X : Type*} [MetricSpace X] [CompactSpace X]
  (f g : X → ℝ) (hf : Continuous f) (hg : Continuous g) :
  IsCompact {x | f x = 0 ∧ 0 ≤ g x} := by
```

Parent. Let $f, g : X \rightarrow \mathbb{R}$ be continuous functions on a metric space. Prove that the set of points where f vanishes and g is nonnegative is closed.

Parent in Lean

```
theorem zero_constrained_positivity_closed
  {X : Type*} [MetricSpace X]
  (f g : X → ℝ) (hf : Continuous f) (hg : Continuous g) :
  IsClosed {x | f x = 0 ∧ 0 ≤ g x} := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes the conclusion from closedness to compactness and requires the new argument that a closed subset of a compact space is compact. Although it reuses the parent's closedness proof, that proof alone does not establish the child's compactness claim.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes closedness to compactness and adds a genuine CompactSpace hypothesis. Its proof must establish the constrained set is closed and then derive compactness as a closed subset of the compact universe, which the parent's argument does not supply.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from closedness to compactness and uses the new compactness hypothesis together with a closed-subset argument. This requires reasoning not supplied by the parent's closedness proof and is not a recall or decorative extension.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2155 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem zero_constrained_positivity_compact.extracted_1_1.{u_1} {X : Type u_1} [inst : MetricSpace X]
  [inst.1 : CompactSpace X] (f g : X → ℝ) (hf : Continuous f) (hg : Continuous g) :
  IsCompact {x | f x = 0 ∧ 0 ≤ g x}
```

Read back by a model that saw only the Lean. *Let X be a compact metric space, and let $f, g : X \rightarrow \mathbb{R}$ be continuous functions. Then the set of all $x \in X$ such that $f(x) = 0$ and $0 \leq g(x)$ is compact.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

theorem zero_constrained_positivity_compact
  {X : Type*} [MetricSpace X] [CompactSpace X]
  (f g : X → ℝ) (hf : Continuous f) (hg : Continuous g) :
  IsCompact {x | f x = 0 ∧ 0 ≤ g x} := by
  have h_zero_closed : IsClosed {x | f x = 0} := by
    have h_singleton : IsClosed ({0} : Set ℝ) := by
      simp
    exact h_singleton.preimage hf
  have h_constrained_closed :
    IsClosed ({x ∈ {x | f x = 0} | (fun _ : X => 0) x ≤ g x}) := by
    exact h_zero_closed.isClosed_le continuous_const.continuousOn hg.continuousOn
  have h_compact_univ : IsCompact (Set.univ : Set X) := by
    exact isCompact_univ
  have h_subset : {x | f x = 0 ∧ 0 ≤ g x} ⊆ (Set.univ : Set X) := by
    intro x hx
    simp
  exact h_compact_univ.of_isClosed_subset h_constrained_closed h_subset
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_4_3__x__Munkres_exercise_18_8a__xe.mh__324f6568

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propect, Classical.choice, Quot.sound, fingerprint 324f656808cc9a8b

4.275 275. tendsto_increment_of_tendsto_deriv

The problem

Let $f : \mathbb{R} \rightarrow \mathbb{R}$ be differentiable on $(0, \infty)$, and suppose f' tends to L at $+\infty$. Then for every $h > 0$, the increment $f(x+h) - f(x)$ tends to hL as x tends to $+\infty$.

Lean statement

```
theorem tendsto_increment_of_tendsto_deriv
  {f : ℝ → ℝ} {h L : ℝ}
  (hh : 0 < h)
  (hfd : DifferentiableOn ℝ f (Set.Ioi 0))
  (hf : Filter.Tendsto (deriv f) Filter.atTop (ℳ L)) :
  Filter.Tendsto (fun x => f (x + h) - f x) Filter.atTop (ℳ (h * L))
```

Parent. Suppose f is defined and differentiable for every $x > 0$, and $f'(x) \rightarrow 0$ as $x \rightarrow +\infty$. Put $g(x) = f(x+1) - f(x)$. Prove that $g(x) \rightarrow 0$ as $x \rightarrow +\infty$.

Parent in Lean

```
theorem Rudin_exercise_5_5
  {f : ℝ → ℝ}
  (hfd : DifferentiableOn ℝ f (Ioi 0))
  (hf : Tendsto (deriv f) atTop (ℳ 0)) :
  Tendsto (λ x => f (x + 1) - f x) atTop (ℳ 0) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes both the increment length h and derivative limit L , proving the new limit $h * L$ via a scaled mean-value-theorem estimate. The parent only handles $h = 1$ and $L = 0$, so its proof does not supply this argument.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes both the increment length and the derivative limit, requiring a centered derivative estimate and scaling by h to obtain the limit $h * L$. The parent only handles unit increments and limit zero, so its proof does not directly supply this obligation.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes both the increment length and derivative limit, requiring a scaled error estimate and algebra showing the increment approaches $h * L$. The parent only handles the fixed case $h = 1$ and $L = 0$, so its proof does not supply the generalized conclusion.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 1949 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed

Check	By	Result
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem tendsto_increment_of_tendsto_deriv.extracted_1_1 {f : ℝ → ℝ} {h L : ℝ} (hh : 0 < h)
(hfd : DifferentiableOn ℝ f (Set.Ioi 0)) (hf : Tendsto (deriv f) atTop (ℳ L)) :
Tendsto (fun x => f (x + h) - f x) atTop (ℳ (h * L))
```

Read back by a model that saw only the Lean. Let $f: \mathbb{R} \rightarrow \mathbb{R}$ and let $h, L \in \mathbb{R}$. Assume $h > 0$, f is differentiable on the open interval $(0, \infty)$, and the derivative $f'(x)$ tends to L as x tends to $+\infty$. Then $f(x+h) - f(x)$ tends to hL as x tends to $+\infty$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset Set
open scoped BigOperators

theorem tendsto_increment_of_tendsto_deriv
{f : ℝ → ℝ} {h L : ℝ}
(hh : 0 < h)
(hfd : DifferentiableOn ℝ f (Set.Ioi 0))
(hf : Filter.Tendsto (deriv f) Filter.atTop (ℳ L)) :
Filter.Tendsto (fun x => f (x + h) - f x) Filter.atTop (ℳ (h * L)) := by
  refine Metric.tendsto_atTop.2 ?_
  intro ε hε
  have hderiv_small : ∀f y : ℝ in Filter.atTop, |deriv f y - L| < ε / h := by
    have hb := hf.eventually (Metric.ball_mem_nhds L (div_pos hε hh))
    refine hb.mono ?_
    intro y hy
    simpa [Metric.mem_ball, Real.dist_eq] using hy
  obtain (N, hN) := Filter.eventually_atTop.1 hderiv_small
  refine (max N 1, ?_)
  intro x hx
  have hxN : N ≤ x := le_trans (le_max_left N 1) hx
  have hx1 : (1 : ℝ) ≤ x := le_trans (le_max_right N 1) hx
  have hxpos : 0 < x := lt_of_lt_of_le (by norm_num) hx1
  have hcc : Set.Icc x (x + h) ⊆ Set.Ioi (0 : ℝ) := by
    intro y hy
    exact lt_of_lt_of_le hxpos hy.1
  have hoo : Set.Ioo x (x + h) ⊆ Set.Ioi (0 : ℝ) := by
    intro y hy
    exact lt_trans hxpos hy.1
  have hcont : ContinuousOn f (Set.Icc x (x + h)) := hfd.continuousOn.mono hcc
  have hdiff : DifferentiableOn ℝ f (Set.Ioo x (x + h)) := hfd.mono hoo
  have hxlt : x < x + h := lt_add_of_pos_right x hx
  obtain (c, hc, hcderiv) := exists_deriv_eq_slope (f := f) (a := x) (b := x + h)
  (hab := hxlt) (hfc := hcont) (hfd := hdiff)
  have hinc : f (x + h) - f x = h * deriv f c := by
    have hden : (x + h) - x = h := by ring
    have heq : deriv f c = (f (x + h) - f x) / h := by
      simpa [hden] using hcderiv
    calc
      f (x + h) - f x = ((f (x + h) - f x) / h) * h := by field_simp
      _ = deriv f c * h := by rw [heq]
      _ = h * deriv f c := by ring
  have hcge : N ≤ c := le_trans hxN (le_of_lt hc.1)
  have hbound : |deriv f c - L| < ε / h := hN c hcge
  have hscaled : h * |deriv f c - L| < h * (ε / h) :=
    mul_lt_mul_of_pos_left hbound hh
  have hquot : h * (ε / h) = ε := by field_simp
  rw [hquot] at hscaled
  rw [Real.dist_eq, hinc, ← mul_sub, abs_mul, abs_of_pos hh]
  exact hscaled
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_5_5__mh__0c421778

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 484916ad7b404fef

4.276 276. bounded_shift_difference_tendsto

The problem

Suppose f is differentiable for every $x > 0$, and $f'(x) \rightarrow 0$ as $x \rightarrow +\infty$. For every $B > 0$ and every fixed shift h with $0 < h \leq B$, prove that $f(x+h) - f(x) \rightarrow 0$ as $x \rightarrow +\infty$.

Lean statement

```
theorem bounded_shift_difference_tendsto
  {f : ℝ → ℝ} {B : ℝ}
  (hB : 0 < B)
  (hfd : DifferentiableOn ℝ f (Set.Ioi 0))
  (hf : Tendsto (deriv f) atTop (ℕ 0)) :
  ∀ h : ℝ, h ∈ Set.Ioc 0 B →
    Tendsto (fun x => f (x + h) - f x) atTop (ℕ 0)
```

Parent. Suppose f is defined and differentiable for every $x > 0$, and $f'(x) \rightarrow 0$ as $x \rightarrow +\infty$. Put $g(x) = f(x+1) - f(x)$. Prove that $g(x) \rightarrow 0$ as $x \rightarrow +\infty$.

Parent in Lean

```
theorem Rudin_exercise_5_5
  {f : ℝ → ℝ}
  (hfd : DifferentiableOn ℝ f (Ioi 0))
  (hf : Tendsto (deriv f) atTop (ℕ 0)) :
  Tendsto (λ x => f (x + 1) - f x) atTop (ℕ 0) := by
```

What it contributes. Generalizes the parent's mean-value-theorem argument from unit shift to a parameter h in the bounded interval $(0,B]$, with the derivative estimate scaled by ε/B .

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely generalizes the unit shift to an arbitrary positive bounded shift and introduces new scaling reasoning via ε/B and $h \leq B$. Its proof must control the mean-value slope over an interval of symbolic length h , so recalling the parent's unit-interval argument is insufficient.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the unit shift to an arbitrary positive bounded shift and must scale the derivative bound by ε/B before applying the mean value theorem. The parent's fixed-unit argument does not supply this parameterized estimate.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the increment by an arbitrary positive bounded shift and must scale the derivative tolerance by ε/B , then use $h \leq B$ to recover the ε -bound. The parent's unit-increment argument does not supply this uniform-in- h estimate.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing

Check	By	Result
corpus_dedup	hash	unique against 2137 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_shift_difference_tendsto.extracted_1_1 : ∀ {f : ℝ → ℝ} {B : ℝ},
  0 < B →
  DifferentiableOn ℝ f (Set.Ioi 0) →
  Tendsto (deriv f) atTop (ℕ 0) → ∀ h ∈ Set.Ioc 0 B, Tendsto (fun x => f (x + h) - f x) atTop (ℕ 0)
```

Read back by a model that saw only the Lean. For every function $f: \mathbb{R} \rightarrow \mathbb{R}$ and every real number B , if $B > 0$, f is differentiable on $(0, \infty)$, and the derivative f' tends to 0 as x tends to $+\infty$, then for every $h \in (0, B]$, the function $x \mapsto f(x+h) - f(x)$ tends to 0 as x tends to $+\infty$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset Set
open scoped BigOperators

theorem bounded_shift_difference_tendsto
  {f : ℝ → ℝ} {B : ℝ}
  (hB : 0 < B)
  (hfd : DifferentiableOn ℝ f (Set.Ioi 0))
  (hf : Tendsto (deriv f) atTop (ℕ 0)) :
  ∀ h : ℝ, h ∈ Set.Ioc 0 B →
    Tendsto (fun x => f (x + h) - f x) atTop (ℕ 0) := by
  intro h hh
  have hhpos : 0 < h := hh.1
  refine Metric.tendsto_atTop.2 ?_
  intro ε εpos
  have hεB : 0 < ε / B := div_pos εpos hB
  have hderiv_small : ∀f y : ℝ in atTop, |deriv f y| < ε / B := by
    have hball := hf.eventually (Metric.ball_mem_nhds (0 : ℝ) hεB)
    refine hball.mono ?_
    intro y hy
    simp [Metric.mem_ball, Real.dist_eq, abs_sub, sub_zero] using hy
  obtain ⟨N, hN⟩ := Filter.eventually_atTop.1 hderiv_small
  refine ⟨max N 1, ?_⟩
  intro x hx
  have hxN : N ≤ x := le_trans (le_max_left _ _) hx
  have hx1 : (1 : ℝ) ≤ x := le_trans (le_max_right _ _) hx
  have hxpos : 0 < x := lt_of_lt_of_le (by norm_num) hx1
  have hsubset_cc : Set.Icc x (x + h) ⊆ Set.Ioi (0 : ℝ) := by
    intro y hy
    exact lt_of_lt_of_le hxpos hy.1
  have hsubset_oo : Set.Ioo x (x + h) ⊆ Set.Ioi (0 : ℝ) := by
    intro y hy
    exact lt_trans hxpos hy.1
  have hcont_restrict : ContinuousOn f (Set.Icc x (x + h)) :=
    hfd.continuousOn.mono hsubset_cc
  have hdiff_restrict : DifferentiableOn ℝ f (Set.Ioo x (x + h)) :=
    hfd.mono hsubset_oo
  have hxlt : x < x + h := lt_add_of_pos_right x hhpos
  obtain ⟨c, hc_mem, hc_deriv⟩ :=
    exists_deriv_eq_slope (f := f) (a := x) (b := x + h)
    (hab := hxlt) (hfc := hcont_restrict) (hfd := hdiff_restrict)
  have hxden : (x + h) - x = h := by ring
  have hc_deriv' : deriv f c = (f (x + h) - f x) / h := by
    simp [hxden] using hc_deriv
  have hdiff_eq : deriv f c * h = f (x + h) - f x := by
    rw [hc_deriv']
```

```

    field_simp [ne_of_gt hhpos]
have hc_ge : N ≤ c := le_trans hxN (le_of_lt hc_mem.1)
have hbound : |deriv f c| < ε / B := hN c hc_ge
have hprod : |deriv f c| * h < ε := by
  calc
    |deriv f c| * h < (ε / B) * h := mul_lt_mul_of_pos_right hbound hhpos
    _ ≤ (ε / B) * B := mul_le_mul_of_nonneg_left hh.2 (le_of_lt hεB)
    _ = ε := by field_simp [ne_of_gt hB]
have hgoal : |f (x + h) - f x| < ε := by
  rw [← hdiff_eq, abs_mul]
  simpa [abs_of_pos hhpos] using hprod
  simpa [Real.dist_eq, sub_zero] using hgoal

```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_5_5__mh__3b64044d

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 3b64044d16efdd86

4.277 277. bounded_shift_difference

The problem

Let f be differentiable on the positive reals and suppose $f'(x) \rightarrow 0$ as $x \rightarrow +\infty$. For every $B \geq 0$, prove that $f(x+h) - f(x) \rightarrow 0$ uniformly for $0 \leq h \leq B$.

Lean statement

```
theorem bounded_shift_difference
  {f : ℝ → ℝ}
  (hfd : DifferentiableOn ℝ f (Set.Ioi 0))
  (hf : Tendsto (deriv f) Filter.atTop (ℕ 0)) :
  ∀ B : ℝ, 0 ≤ B →
    ∀ ε : ℝ, 0 < ε →
      ∃ N : ℝ, ∀ x : ℝ, N ≤ x → ∀ h ∈ Set.Icc 0 B,
        dist (f (x + h) - f x) 0 < ε
```

Parent. Suppose f is defined and differentiable for every $x > 0$, and $f'(x) \rightarrow 0$ as $x \rightarrow +\infty$. Put $g(x) = f(x+1) - f(x)$. Prove that $g(x) \rightarrow 0$ as $x \rightarrow +\infty$.

Parent in Lean

```
theorem Rudin_exercise_5_5
  {f : ℝ → ℝ}
  (hfd : DifferentiableOn ℝ f (Ioi 0))
  (hf : Tendsto (deriv f) atTop (ℕ 0)) :
  Tendsto (λ x => f (x + 1) - f x) atTop (ℕ 0) := by
```

What it contributes. Reuses the parent's derivative-at-infinity estimate, positive-domain restriction, and mean-value-theorem argument, while replacing the fixed unit interval by a quantified bounded interval and explicitly handling $h=0$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the increment by an arbitrary bounded shift and proves uniform convergence over $h \in [0, B]$. The parent's fixed-unit mean-value argument does not supply the epsilon scaling, interval control, or separate zero-shift case needed here.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child proves a genuinely uniform bounded-shift estimate, handling every $h \in [0, B]$ with a derivative bound scaled by $1+B$ and a separate zero-shift case. This requires new reasoning beyond the parent's fixed unit increment and is not merely the kept fixed-parameter theorem.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely upgrades a fixed unit increment to a uniform estimate over every shift h in a bounded interval, requiring a new zero-shift case and scaling the derivative bound by $1+B$. This uniform bounded-shift argument is not supplied by the parent's single-interval proof or by the fixed-parameter limit statement.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable

Check	By	Result
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2217 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem bounded_shift_difference.extracted_1_1 : ∀ {f : ℝ → ℝ},
  DifferentiableOn ℝ f (Set.Ioi 0) →
  Tendsto (deriv f) atTop (ℕ 0) →
  ∀ (B : ℝ),
    0 ≤ B → ∀ (ε : ℝ), 0 < ε → ∃ N, ∀ (x : ℝ), N ≤ x → ∀ h ∈ Set.Icc 0 B, dist (f (x + h) - f x) 0 < ε
```

Read back by a model that saw only the Lean. For every function $f: \mathbb{R} \rightarrow \mathbb{R}$ that is differentiable on the open interval $(0, \infty)$ and whose derivative tends to 0 as x tends to $+\infty$, for every real number $B \geq 0$ and every $\varepsilon > 0$, there exists a real number N such that, for every real $x \geq N$ and every $h \in [0, B]$, the distance between $f(x + h) - f(x)$ and 0 is less than ε .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset Set
open scoped BigOperators

theorem bounded_shift_difference
  {f : ℝ → ℝ}
  (hfd : DifferentiableOn ℝ f (Set.Ioi 0))
  (hf : Tendsto (deriv f) Filter.atTop (ℕ 0)) :
  ∀ B : ℝ, 0 ≤ B →
  ∀ ε : ℝ, 0 < ε →
    ∃ N : ℝ, ∀ x : ℝ, N ≤ x → ∀ h ∈ Set.Icc 0 B,
      dist (f (x + h) - f x) 0 < ε := by
  intro B hB ε hε
  have hden : 0 < 1 + B := by linarith
  have hsmall : 0 < ε / (1 + B) := div_pos hε hden
  have hderiv_small : ∀f y : ℝ in Filter.atTop, |deriv f y| < ε / (1 + B) := by
    have hball := hf.eventually (Metric.ball_mem_nhds (0 : ℝ) hsmall)
    refine hball.mono ?_
    intro y hy
    simp [Metric.mem_ball, Real.dist_eq, abs_sub, sub_zero] using hy
  obtain ⟨N, hN⟩ := Filter.eventually_atTop.1 hderiv_small
  refine ⟨max N (1 + B), ?_⟩
  intro x hx h hmem
  have hxN : N ≤ x := le_trans (le_max_left N (1 + B)) hx
  have hx1B : 1 + B ≤ x := le_trans (le_max_right N (1 + B)) hx
  have hxpos : 0 < x := by linarith
  rcases eq_or_ne h 0 with rfl | hne
  · simp using hε
  have hpos : 0 < h := lt_of_le_of_ne hmem.1 (Ne.symm hne)
  have hsubset_cc : Set.Icc x (x + h) ⊆ Set.Ioi (0 : ℝ) := by
    intro y hy
    exact lt_of_lt_of_le hxpos hy.1
  have hsubset_oo : Set.Ioo x (x + h) ⊆ Set.Ioi (0 : ℝ) := by
    intro y hy
    exact lt_trans hxpos hy.1
  have hcont_restrict : ContinuousOn f (Set.Icc x (x + h)) :=
    hfd.continuousOn.mono hsubset_cc
  have hdiff_restrict : DifferentiableOn ℝ f (Set.Ioo x (x + h)) :=
    hfd.mono hsubset_oo
  have hxlt : x < x + h := lt_add_of_pos_right x hpos
  obtain ⟨c, hc_mem, hc_deriv⟩ :=
    exists_deriv_eq_slope (f := f) (a := x) (b := x + h)
  (hab := hxlt) (hfc := hcont_restrict) (hfd := hdiff_restrict)
```

```

have hdiff_eq : deriv f c = (f (x + h) - f x) / h := by
  simpa [sub_eq_add_neg] using hc_deriv
have hinc : f (x + h) - f x = h * deriv f c := by
  rw [hdiff_eq]
  field_simp
have hc_ge : N ≤ c := le_trans hxN (le_of_lt hc_mem.1)
have hbound : |deriv f c| < ε / (1 + B) := hN c hc_ge
have hprod : h * |deriv f c| ≤ B * (ε / (1 + B)) := by
  exact mul_le_mul hmem.2 (le_of_lt hbound) (abs_nonneg _) hB
have hBlt : B < 1 + B := by linarith
have hstrict : B * (ε / (1 + B)) < ε := by
  calc
    B * (ε / (1 + B)) < (1 + B) * (ε / (1 + B)) :=
      mul_lt_mul_of_pos_right hBlt hsmall
    _ = ε := by field_simp
have habs : |f (x + h) - f x| = h * |deriv f c| := by
  rw [hinc, abs_mul, abs_of_pos hpos]
have hgoal : |f (x + h) - f x| < ε := by
  calc
    |f (x + h) - f x| = h * |deriv f c| := habs
    _ ≤ B * (ε / (1 + B)) := hprod
    _ < ε := hstrict
simpa [Real.dist_eq, sub_zero] using hgoal

```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Rudin_exercise_5_5__mh__46380a86

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint 46380a86008e6a6f

4.278 278. Rudin_exercise_5_5_unit_oscillation

The problem

Suppose f is differentiable for every positive input and $f'(x) \rightarrow 0$ as $x \rightarrow +\infty$. Show that for every $\varepsilon > 0$, there is N such that whenever $x, y \geq N$ and $|x - y| \leq 1$, one has $|f(x) - f(y)| < \varepsilon$.

Lean statement

```
theorem Rudin_exercise_5_5_unit_oscillation
  {f : ℝ → ℝ}
  (hfd : DifferentiableOn ℝ f (Set.Ioi 0))
  (hf : Tendsto (deriv f) atTop (ℕ 0)) :
  ∀ ε : ℝ, 0 < ε → ∃ N : ℝ, ∀ x y : ℝ, N ≤ x → N ≤ y → |x - y| ≤ 1 → |f x - f y| < ε
```

Parent. Suppose a response curve is differentiable for every positive input and its derivative tends to zero as the input grows without bound. Show that for every tolerance $\varepsilon > 0$, the change in response between inputs one unit apart is eventually smaller than ε .

Parent in Lean

```
theorem Rudin_exercise_5_5_eventual
  {f : ℝ → ℝ}
  (hfd : DifferentiableOn ℝ f (Ioi 0))
  (hf : Tendsto (deriv f) atTop (ℕ 0)) :
  ∀ ε : ℝ, 0 < ε → ∃ N : ℝ, ∀ x : ℝ, N ≤ x → |f (x + 1) - f x| < ε
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child strengthens the fixed unit-step estimate to a uniform bound for every pair of sufficiently large points with distance at most 1. The parent's proof of the specific increment does not supply this two-variable oscillation claim; the child needs a new ordering and arbitrary-interval mean-value argument.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely changes the conclusion from a fixed unit increment to uniform oscillation over every pair of sufficiently large points within distance 1. Its proof must handle arbitrary interval orientation and variable interval lengths, with a new mean-value estimate using the distance bound; the parent's fixed-step proof does not directly supply this.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the fixed unit increment into a uniform oscillation bound for every pair of sufficiently large points at distance at most one. Its proof must handle arbitrary interval lengths, order both endpoints, and combine the mean value theorem with the distance bound, which the parent's fixed-step argument does not supply.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2830 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone

Check	By	Result
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem Rudin_exercise_5_5_unit_oscillation.extracted_1_1 : ∀ {f : ℝ → ℝ},
  DifferentiableOn ℝ f (Set.Ioi 0) →
  Tendsto (deriv f) atTop (ℕ 0) →
  ∀ (ε : ℝ), 0 < ε → ∃ N, ∀ (x y : ℝ), N ≤ x → N ≤ y → |x - y| ≤ 1 → |f x - f y| < ε
```

Read back by a model that saw only the Lean. For every function f from $\mathbb{R}$ to $\mathbb{R}$, if f is differentiable on $(0, \infty)$ and its derivative tends to 0 as x tends to $+\infty$, then for every $\varepsilon > 0$ there exists a real number N such that for all real numbers x and y satisfying $N \leq x$, $N \leq y$, and $|x - y| \leq 1$, we have $|f(x) - f(y)| < \varepsilon$.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Topology Filter Real Complex TopologicalSpace Finset Set
open scoped BigOperators

theorem Rudin_exercise_5_5_unit_oscillation
  {f : ℝ → ℝ}
  (hfd : DifferentiableOn ℝ f (Set.Ioi 0))
  (hf : Tendsto (deriv f) atTop (ℕ 0)) :
  ∀ ε : ℝ, 0 < ε → ∃ N : ℝ, ∀ x y : ℝ, N ≤ x → N ≤ y → |x - y| ≤ 1 → |f x - f y| < ε := by
  intro ε hε
  have hsmall : ∀ z : ℝ in atTop, |deriv f z| < ε := by
    have hball := hf.eventually (Metric.ball_mem_nhds (0 : ℝ) hε)
    filter_upwards [hball] with z hz
    simpa [Metric.mem_ball, Real.dist_eq, abs_sub, sub_zero] using hz
  obtain ⟨N, hN⟩ := Filter.eventually_atTop.1 hsmall
  refine ⟨max N 1, ?_⟩
  intro x y hx hy hxy
  have hforward : ∀ a b : ℝ, max N 1 ≤ a → max N 1 ≤ b → a ≤ b → |b - a| ≤ 1 → |f b - f a| < ε := by
    intro a b ha hb hab hd
    by_cases heq : a = b
    · simpa [heq] using hε
    have hlt : a < b := lt_of_le_of_ne hab heq
    have ha1 : (1 : ℝ) ≤ a := le_of_max_le_right ha
    have hapos : 0 < a := lt_of_lt_of_le zero_lt_one ha1
    have hcc : Set.Icc a b ⊆ Set.Ioi (0 : ℝ) := by
      intro z hz
      exact lt_of_lt_of_le hapos hz.1
    have hoo : Set.Ioo a b ⊆ Set.Ioi (0 : ℝ) := by
      intro z hz
      exact lt_trans hapos hz.1
    have hcont : ContinuousOn f (Set.Icc a b) := hfd.continuousOn.mono hcc
    have hdiff : DifferentiableOn ℝ f (Set.Ioo a b) := hfd.mono hoo
    obtain ⟨c, hc, hcs⟩ := exists_deriv_eq_slope (f := f) (a := a) (b := b)
    (hab := hlt) (hfc := hcont) (hfd := hdiff)
    have hcN : N ≤ c := le_trans (le_trans (le_max_left N 1) ha) (le_of_lt hc.1)
    have hcbound : |deriv f c| < ε := hN c hcN
    have hdiff_eq : f b - f a = deriv f c * (b - a) := by
      rw [hcs]
      field_simp
    have habs : |f b - f a| = |deriv f c| * |b - a| := by
      rw [hdiff_eq, abs_mul]
    have hdp : 0 < |b - a| := abs_pos.mpr (sub_ne_zero.mpr (ne_of_gt hlt))
    have hmul : |deriv f c| * |b - a| < ε * |b - a| :=
      mul_lt_mul_of_pos_right hcbound hdp
    have hdist : |b - a| ≤ 1 := hd
    rw [habs]
    nlinarith
  rcases le_total x y with hxy' | hxy'
```

```
• have h := hforward x y hx hy hxy' (by simpa [abs_sub_comm] using hxy)
  simpa [abs_sub_comm] using h
• exact hforward y x hy hx hxy' hxy
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Rudin_exercise_5_5__ms.mh__9761fa5f

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 9761fa5f46749496

4.279 279. weighted_summable_majorant_complex_partial_sums

The problem

Let (a_n) be a sequence of complex numbers dominated in norm by a summable real sequence (g_n) . Show that for any complex weights (c_n) with $|c_n| \leq 1$, the weighted partial sums $\sum_{k < n} c_k a_k$ converge.

Lean statement

```
theorem weighted_summable_majorant_complex_partial_sums (a : ℕ → ℂ) (g : ℕ → ℝ)
  (hmajorant : Summable g)
  (hdom : ∀ n : ℕ, ‖a n‖ ≤ g n)
  (c : ℕ → ℂ)
  (hc : ∀ n : ℕ, ‖c n‖ ≤ 1)
  (s : ℕ → ℂ)
  (h : s = (fun n : ℕ => ∑ i ∈ Finset.range n, c i * a i)) :
  ∃ y : ℂ, Tendsto s atTop (ℕ y)
```

Parent. Let (a_n) be a sequence of complex numbers. If there is a summable nonnegative majorant (g_n) with $\|a_n\| \leq g_n$ for every n , prove that the partial sums of $\sum a_n$ converge.

Parent in Lean

```
theorem summable_majorant_complex_partial_sums (a : ℕ → ℂ) (g : ℕ → ℝ)
  (hmajorant : Summable g)
  (hdom : ∀ n : ℕ, ‖a n‖ ≤ g n)
  (s : ℕ → ℂ)
  (h : s = (fun n : ℕ => ∑ i ∈ Finset.range n, a i)) :
  ∃ y : ℂ, Tendsto s atTop (ℕ y)
```

What it contributes. Preserves the parent's majorant-to-summability and partial-sum convergence checkpoints, while applying them to the new weighted sequence $c_n a_n$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child genuinely parameterizes the summands by arbitrary complex weights and requires new reasoning to establish that the weighted terms remain dominated by the summable majorant. The parent's convergence argument alone does not provide this weighted domination step.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the summands by an arbitrary bounded complex coefficient sequence and requires proving a new norm domination before establishing summability. The parent's direct majorant argument does not supply this weighted domination step.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes each summand by an arbitrary complex weight sequence and must prove the new norm domination using multiplicativity and the bound on c . This reasoning is not supplied by the parent proof, which can only directly establish summability of a .

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry

Check	By	Result
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2153 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem weighted_summable_majorant_complex_partial_sums.extracted_1_1 (a : ℕ → ℂ) (g : ℕ → ℝ) (hmajorant : Summable g)
(hdom : ∀ (n : ℕ), ‖a n‖ ≤ g n) (c : ℕ → ℂ) (hc : ∀ (n : ℕ), ‖c n‖ ≤ 1) (s : ℕ → ℂ)
(h : s = fun n => ∑ i ∈ range n, c i * a i) : ∃ y, Tendsto s atTop (ℳ y)
```

Read back by a model that saw only the Lean. For a sequence $a : \mathbb{N} \rightarrow \mathbb{C}$, a sequence $g : \mathbb{N} \rightarrow \mathbb{R}$ with summable series, and a sequence $c : \mathbb{N} \rightarrow \mathbb{C}$ satisfying $\|a(n)\| \leq g(n)$ and $\|c(n)\| \leq 1$ for every n , if $s : \mathbb{N} \rightarrow \mathbb{C}$ is defined by $s(n) = \sum_{\{i < n\}} c(i) a(i)$, then there exists $y \in \mathbb{C}$ such that $s(n)$ tends to y as n tends to infinity.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open scoped BigOperators Topology
open Complex Filter Function Metric Finset

theorem weighted_summable_majorant_complex_partial_sums (a : ℕ → ℂ) (g : ℕ → ℝ)
(hmajorant : Summable g)
(hdom : ∀ n : ℕ, ‖a n‖ ≤ g n)
(c : ℕ → ℂ)
(hc : ∀ n : ℕ, ‖c n‖ ≤ 1)
(s : ℕ → ℂ)
(h : s = (fun n : ℕ => ∑ i ∈ Finset.range n, c i * a i)) :
∃ y : ℂ, Tendsto s atTop (ℳ y) := by
have hweighted : ∀ n : ℕ, ‖c n * a n‖ ≤ g n := by
intro n
calc
‖c n * a n‖ = ‖c n‖ * ‖a n‖ := norm_mul _ _
_ ≤ 1 * ‖a n‖ := mul_le_mul_of_nonneg_right (hc n) (norm_nonneg _)
_ = ‖a n‖ := one_mul _
_ ≤ g n := hdom n
have hsum : Summable (fun n : ℕ => c n * a n) := by
exact Summable.of_norm_bounded hmajorant hweighted
have hpartial :
Tendsto (fun n : ℕ => ∑ i ∈ Finset.range n, c i * a i)
atTop (ℳ (∑' n, c n * a n)) :=
hsum.hasSum.tendsto_sum_nat
refine (∑' n, c n * a n, ?_)
simp [h] using hpartial
```

Operator. mutation / mutation_hard **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Shakarchi_exercise_1_19b__me.me.mh__4ce18d21

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 4ce18d21a38cae68

4.280 280. dominated_phase_series_converges_bounded

The problem

Let z be a complex number with $|z| \leq 1$. If the coefficients a_n are dominated in norm by a summable nonnegative sequence g_n , prove that the partial sums of $\sum z^{n+1} a_n$ converge.

Lean statement

```
theorem dominated_phase_series_converges_bounded (z : ℂ) (hz : ‖z‖ ≤ 1) (a : ℕ → ℂ) (g : ℕ → ℝ) (s : ℕ → ℂ) (hdom : ∀ n, ‖a n‖ ≤ g n)
  (hsummable : Summable g) (h : s = fun n => ∑ i ∈ Finset.range n, z ^ (i + 1) * a i) : ∃ y, Tendsto s atTop (ℳ y)
```

Parent. Let $|z| = 1$. If the coefficients a_n are dominated in norm by a summable nonnegative sequence g_n , prove that the partial sums of $\sum z^{n+1} a_n$ converge.

Parent in Lean

```
theorem dominated_phase_series_converges (z : ℂ) (hz : ‖z‖ = 1) (a : ℕ → ℂ) (g : ℕ → ℝ) (s : ℕ → ℂ) (hdom : ∀ n, ‖a n‖ ≤ g n) (hsummable
  : Summable g) (h : s = fun n => ∑ i ∈ Finset.range n, z ^ (i + 1) * a i) : ∃ y, Tendsto s atTop (ℳ y)
```

What it contributes. Retained the parent's phased-coefficient and summable-majorant framework, while replacing the unit-modulus step with an inductive bound for powers under $\|z\| \leq 1$.

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation STRONG

model gpt-5.6-luna · saw parents, child

The child genuinely weakens $\|z\| = 1$ to $\|z\| \leq 1$. Its proof must establish the new bound $\|z^{(n+1)}\| \leq 1$ by induction before applying domination, which is reasoning absent from the parent proof.

Re-judged, first pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The weakened hypothesis $\|z\| \leq 1$ requires a new induction proving $\|z^{(n+1)}\| \leq 1$ before establishing domination; the parent's direct norm-equality simplification no longer applies. The convergence argument therefore depends on genuinely new mathematics, and the statement accurately matches the Lean theorem.

Re-judged, second pass STRONG

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely weakens the phase assumption from $\|z\| = 1$ to $\|z\| \leq 1$ and supplies a new inductive argument establishing $\|z\|^k \leq 1$. This produces the domination needed for summability, so the parent's proof cannot simply be recalled.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2197 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem dominated_phase_series_converges_bounded.extracted_1_1 (z : ℂ) (hz : ‖z‖ ≤ 1) (a : ℕ → ℂ) (g : ℕ → ℝ)
  (s : ℕ → ℂ) (hdom : ∀ (n : ℕ), ‖a n‖ ≤ g n) (hsummable : Summable g)
  (h : s = fun n => ∑ i ∈ range n, z ^ (i + 1) * a i) : ∃ y, Tendsto s atTop (ℳ y)
```

Read back by a model that saw only the Lean. For a complex number z with norm at most 1, functions $a : \mathbb{N} \rightarrow \mathbb{C}$ and $g : \mathbb{N} \rightarrow \mathbb{R}$ satisfying $\|a(n)\| \leq g(n)$ for every natural number n , and such that the series $\sum n, g(n)$ is summable, suppose that $s : \mathbb{N} \rightarrow \mathbb{C}$ is defined by $s(n) = \sum_{i=0}^{n-1} z^{i+1} a(i)$. Then there exists a complex number y such that the sequence $s(n)$ converges to y as n tends to infinity.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open scoped BigOperators
open Complex Filter Function Metric Finset

theorem dominated_phase_series_converges_bounded (z : ℂ) (hz : ‖z‖ ≤ 1) (a : ℕ → ℂ) (g : ℕ → ℝ) (s : ℕ → ℂ)
  (hdom : ∀ n, ‖a n‖ ≤ g n) (hsummable : Summable g)
  (h : s = fun n => ∑ i ∈ Finset.range n, z ^ (i + 1) * a i) :
  ∃ y, Tendsto s atTop (ℳ y) := by
  let f : ℕ → ℂ := fun n => z ^ (n + 1) * a n
  have hpow : ∀ k : ℕ, ‖z‖ ^ k ≤ 1 := by
    intro k
    induction k with
    | zero => simp
    | succ k ih =>
      rw [pow_succ]
      calc
        ‖z‖ ^ k * ‖z‖ ≤ 1 * ‖z‖ :=
          mul_le_mul_of_nonneg_right ih (norm_nonneg z)
        _ ≤ 1 := by simpa using hz
  have hpower : ∀ n : ℕ, ‖z‖ ^ (n + 1) ≤ 1 := by
    intro n
    rw [norm_pow]
    exact hpow (n + 1)
  have hmajor : ∀ n, ‖f n‖ ≤ g n := by
    intro n
    calc
      ‖f n‖ = ‖z‖ ^ (n + 1) * ‖a n‖ := by simp [f, norm_mul]
      _ ≤ 1 * ‖a n‖ := mul_le_mul_of_nonneg_right (hpower n) (norm_nonneg (a n))
      _ = ‖a n‖ := by simp
      _ ≤ g n := hdom n
  have hf : Summable f := by
    exact Summable.of_norm_bounded hsummable hmajor
  have hsum : HasSum f (∑' k, f k) := hf.hasSum
  have hconv :
    Tendsto (fun n => ∑ k ∈ Finset.range n, f k) atTop (ℳ (∑' k, f k)) := by
    simpa using hsum.tendsto_sum_nat
  refine (∑' k, f k, ?_)
  simpa [h, f] using hconv
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Shakarchi_exercise_1_19b__me.mh__7d0d3155

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propxt, Classical.choice, Quot.sound, fingerprint 7d0d315577046b18

4.281 281. dominated_phase_series_has_unique_limit

The problem

Let $|z| = 1$. If the coefficients a_n are dominated in norm by a summable nonnegative sequence g_n , then the partial sums of $\sum z^{n+1} a_n$ converge to a unique limit.

Lean statement

```
theorem dominated_phase_series_has_unique_limit (z : ℂ) (hz : ‖z‖ = 1) (a : ℕ → ℂ) (g : ℕ → ℝ) (s : ℕ → ℂ) (hdom : ∀ n, ‖a n‖ ≤ g n)
  (hsummable : Summable g) (h : s = fun n => ∑ i ∈ Finset.range n, z ^ (i + 1) * a i) : ∃! y, Tendsto s atTop (ℕ y)
```

Parent. Let $|z| = 1$. If the coefficients a_n are dominated in norm by a summable nonnegative sequence g_n , prove that the partial sums of $\sum z^{n+1} a_n$ converge.

Parent in Lean

```
theorem dominated_phase_series_converges (z : ℂ) (hz : ‖z‖ = 1) (a : ℕ → ℂ) (g : ℕ → ℝ) (s : ℕ → ℂ) (hdom : ∀ n, ‖a n‖ ≤ g n) (hsummable
  : Summable g) (h : s = fun n => ∑ i ∈ Finset.range n, z ^ (i + 1) * a i) : ∃ y, Tendsto s atTop (ℕ y)
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes existence of a limit into existence and uniqueness of the limit. Its uniqueness branch requires the separate Hausdorff-limit argument via ‘tendsto_nhds_unique’, which the parent’s proof does not supply.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes existence of a limit into uniqueness of the limit, requiring a genuinely new Hausdorff-space uniqueness argument via ‘tendsto_nhds_unique’; the witness is not forced by a wrapper hypothesis. The parent’s convergence construction supplies existence but does not prove the added uniqueness obligation.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes the conclusion from existence to a genuine uniqueness statement. After establishing convergence, it must additionally use Hausdorff-limit uniqueness to prove that every possible limit equals the constructed one; this is not supplied by the parent’s proof.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2178 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem dominated_phase_series_has_unique_limit.extracted_1_1 (z : ℂ) (hz : ‖z‖ = 1) (a : ℕ → ℂ) (g : ℕ → ℝ) (s : ℕ → ℂ)
  (hdom : ∀ (n : ℕ), ‖a n‖ ≤ g n) (hsummable : Summable g) (h : s = fun n => ∑ i ∈ range n, z ^ (i + 1) * a i) :
```


$\exists! y, \text{Tendsto } s \text{ atTop } (\mathcal{U} \ y)$

Read back by a model that saw only the Lean. For a complex number z with norm 1, sequences $a : \mathbb{N} \rightarrow \mathbb{C}$, $g : \mathbb{N} \rightarrow \mathbb{R}$, and $s : \mathbb{N} \rightarrow \mathbb{C}$, suppose that for every natural number n , $\|a \ n\| \leq g \ n$, that the real series $\sum_n g \ n$ is summable, and that for every n , $s \ n = \sum_{i < n} z^{i+1} a \ i$. Then there exists a unique complex number y such that the sequence s converges to y as n tends to infinity.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open scoped BigOperators Topology
open Complex Filter Function Metric Finset

theorem dominated_phase_series_has_unique_limit (z : ℂ) (hz : ‖z‖ = 1) (a : ℕ → ℂ) (g : ℕ → ℝ) (s : ℕ → ℂ)
  (hdom : ∀ n, ‖a n‖ ≤ g n) (hsummable : Summable g)
  (h : s = fun n => ∑ i ∈ Finset.range n, z ^ (i + 1) * a i) :
  ∃! y, Tendsto s atTop (ℳ y) := by
  let f : ℕ → ℂ := fun n => z ^ (n + 1) * a n
  have hmajor : ∀ n, ‖f n‖ ≤ g n := by
    intro n
    simp [f, norm_mul, norm_pow, hz] using hdom n
  have hf : Summable f := by
    exact Summable.of_norm_bounded hsummable hmajor
  have hconv :
    Tendsto (fun n => ∑ k ∈ Finset.range n, f k) atTop (ℳ (∑' k, f k)) := by
    exact hf.hasSum.tendsto_sum_nat
  have hsconv : Tendsto s atTop (ℳ (∑' k, f k)) := by
    simp [h, f] using hconv
  refine ⟨∑' k, f k, hsconv, ?_⟩
  intro y hy
  exact (tendsto_nhds_unique hsconv hy).symm
```

Operator. mutation / mutation_hard **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Shakarchi_exercise_1_19b__me.mh__97dfff79

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 97dfff79790543fe

4.282 282. Shakarchi_exercise_1_19b_unique

The problem

Prove that the power series $\sum z^n/n^2$ converges at every point of the unit circle, and that its limit is unique.

Lean statement

```
theorem Shakarchi_exercise_1_19b_unique (z : ℂ) (hz : ‖z‖ = 1) (s : ℕ → ℂ)
  (h : s = (fun n => ∑ i ∈ Finset.range n, z ^ (i + 1) / (i + 1) ^ 2)) :
  ∃! y, Tendsto s atTop (ℳ y)
```

Parent. Prove that the power series $\sum z^n/n^2$ converges at every point of the unit circle.

Parent in Lean

```
theorem Shakarchi_exercise_1_19b (z : ℂ) (hz : ‖z‖ = 1) (s : ℕ → ℂ)
  (h : s = (λ n => ∑ i ∈ (range n), z ^ (i + 1) / (i + 1) ^ 2)) :
  ∃ y, Tendsto s atTop (ℳ y) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child changes existence of a limit into existence and uniqueness of the limit. Although the convergence argument is inherited, the added obligation requires a separate Hausdorff-topology uniqueness argument, so recalling the parent alone does not complete the proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes existence of a limit into existence and uniqueness, requiring the new Hausdorff-limit argument via ‘tendsto_nhds_unique’; the parent’s convergence proof alone cannot discharge that obligation.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child changes existence of a limit into existence and uniqueness of the limit. The parent’s summability argument supplies existence, but the new uniqueness obligation requires a separate Hausdorff-limit argument and is not a forced-witness wrapper.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2901 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem Shakarchi_exercise_1_19b_unique.extracted_1_1 (z : ℂ) (hz : ‖z‖ = 1) (s : ℕ → ℂ)
  (h : s = fun n => ∑ i ∈ range n, z ^ (i + 1) / (i + 1) ^ 2) : ∃! y, Tendsto s atTop (ℕ y)
```

Read back by a model that saw only the Lean. For a complex number z with norm 1, and a sequence s of complex numbers defined by $s(n) =$ the sum over all natural numbers $i < n$ of z^{i+1} divided by $(i+1)^2$, there exists a unique complex number y such that $s(n)$ converges to y as n tends to infinity.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Complex Filter Function Metric Finset
open scoped BigOperators Topology

theorem Shakarchi_exercise_1_19b_unique (z : ℂ) (hz : ‖z‖ = 1) (s : ℕ → ℂ)
  (h : s = (fun n => ∑ i ∈ Finset.range n, z ^ (i + 1) / (i + 1) ^ 2)) :
  ∃! y, Tendsto s atTop (ℕ y) := by
  classical
  set f : ℕ → ℂ := fun n => z ^ (n + 1) / (n + 1 : ℂ) ^ 2 with hfdef
  have h_norm :
    (fun n : ℕ => ‖f n‖) =
      (fun n : ℕ => (1 : ℝ) / (n + 1 : ℝ) ^ 2) := by
    funext n
    have hzpow : ‖z ^ (n + 1)‖ = 1 := by
      simp [hz, Complex.norm_pow z (n + 1)]
    have hdenpow :
      ‖(n + 1 : ℂ) ^ 2‖ = (n + 1 : ℝ) ^ 2 := by
      have hden : ‖(n + 1 : ℂ)‖ = (n + 1 : ℝ) := by
        simp [Nat.cast_add, Nat.cast_one] using (Complex.norm_natCast (n + 1))
      simp [pow_two, hden]
    calc
      ‖f n‖ = ‖z ^ (n + 1)‖ / ‖(n + 1 : ℂ) ^ 2‖ := by simp [f]
      _ = 1 / ‖(n + 1 : ℂ) ^ 2‖ := by simp [hzpow]
      _ = (1 : ℝ) / (n + 1 : ℝ) ^ 2 := by simp [hdenpow]
  have h_pSeries :
    Summable (fun n : ℕ => (1 : ℝ) / (n + 1 : ℝ) ^ 2) := by
  have h_base :
    Summable (fun n : ℕ => (1 : ℝ) / (n : ℝ) ^ 2) :=
      (Real.summable_one_div_nat_pow (p := 2)).mpr one_lt_two
  simp [Nat.cast_add, Nat.cast_one] using (summable_nat_add_iff 1).2 h_base
  have h_norm_summable : Summable (fun n : ℕ => ‖f n‖) := by
    simp [h_norm] using h_pSeries
  have hfSummable : Summable f := Summable.of_norm h_norm_summable
  refine (∑' n, f n, ?_, ?_)
  · have hy : HasSum f (∑' n, f n) := hfSummable.hasSum
    simp [h, f] using hy.tendsto_sum_nat
  · intro y hy
    have hlim : Tendsto s atTop (ℕ (∑' n, f n)) := by
      have hsum : HasSum f (∑' n, f n) := hfSummable.hasSum
      simp [h, f] using hsum.tendsto_sum_nat
    exact (tendsto_nhds_unique hlim hy).symm
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Shakarchi_exercise_1_19b__mh__02a0eb22

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 02a0eb2227165634

4.283 283. Shakarchi_exercise_1_19b_mutation

The problem

Prove that for every complex number z on the unit circle and every integer $p > 1$, the power series $\sum z^n/n^p$ converges.

Lean statement

```
theorem Shakarchi_exercise_1_19b_mutation (z : ℂ) (p : ℕ) (hp : 1 < p) (hz : ‖z‖ = 1) (s : ℕ → ℂ)
  (h : s = (fun n => ∑ i ∈ Finset.range n, z ^ (i + 1) / (i + 1 : ℂ) ^ p)) :
  ∃ y, Tendsto s atTop (ℕ y)
```

Parent. Prove that the power series $\sum z^n/n^2$ converges at every point of the unit circle.

Parent in Lean

```
theorem Shakarchi_exercise_1_19b (z : ℂ) (hz : ‖z‖ = 1) (s : ℕ → ℂ)
  (h : s = (λ n => ∑ i ∈ (range n), z ^ (i + 1) / (i + 1 : ℂ) ^ 2)) :
  ∃ y, Tendsto s atTop (ℕ y) := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The exponent 2 is genuinely parameterized to an arbitrary natural number $p > 1$. The proof must establish p -series summability using the new hypothesis hp , which is not supplied by the parent proof.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely generalizes the denominator exponent from fixed 2 to arbitrary $p > 1$, requiring a new p -series summability argument and carrying that parameter through the norm calculation. The parent proof does not establish this family, so recalling it is insufficient.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child genuinely parameterizes the denominator exponent from 2 to an arbitrary natural p with $1 < p$. Its proof requires the generalized p -series summability theorem and uses the new hypothesis hp , so recalling the parent's fixed-exponent proof is insufficient.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2930 statements
redundancy	lean	no hypothesis Lean could drop, no parent it could prove alone
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem Shakarchi_exercise_1_19b_mutation.extracted_1_1 (z : ℂ) (p : ℕ) (hp : 1 < p) (hz : ‖z‖ = 1) (s : ℕ → ℂ)
  (h : s = fun n => ∑ i ∈ range n, z ^ (i + 1) / (i + 1 : ℂ) ^ p) : ∃ y, Tendsto s atTop (ℕ y)
```

Read back by a model that saw only the Lean. Let z be a complex number, let p be a natural number with $1 < p$, and suppose that the norm of z is 1. Let s be a sequence of complex numbers satisfying, for every natural number n , $s(n) = \sum_{i=0}^{n-1} z^{i+1} / (i+1)^p$. Then there exists a complex number y such that $s(n)$ converges to y as n tends to infinity.

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false

open Complex Filter Function Metric Finset
open scoped BigOperators Topology

theorem Shakarchi_exercise_1_19b_mutation (z : ℂ) (p : ℕ) (hp : 1 < p) (hz : ‖z‖ = 1) (s : ℕ → ℂ)
  (h : s = (fun n => ∑ i ∈ Finset.range n, z ^ (i + 1) / (i + 1 : ℂ) ^ p)) :
  ∃ y, Tendsto s atTop (ℳ y) := by
  classical
  set f : ℕ → ℂ := fun n => z ^ (n + 1) / (n + 1 : ℂ) ^ p with hfdef
  have h_norm :
    (fun n : ℕ => ‖f n‖) =
      fun n : ℕ => (1 : ℝ) / (n + 1 : ℝ) ^ p := by
  funext n
  have hzpow : ‖z ^ (n + 1)‖ = 1 := by
    simp [hz]
  have hdenpow :
    ‖(n + 1 : ℂ) ^ p‖ = (n + 1 : ℝ) ^ p := by
  have hden : ‖(n + 1 : ℂ)‖ = (n + 1 : ℝ) := by
    simp [Nat.cast_add, Nat.cast_one] using (Complex.norm_natCast (n + 1))
    simp [hden]
  calc
    ‖f n‖ = ‖z ^ (n + 1)‖ / ‖(n + 1 : ℂ) ^ p‖ := by simp [f]
    _ = 1 / ‖(n + 1 : ℂ) ^ p‖ := by rw [hzpow]
    _ = (1 : ℝ) / (n + 1 : ℝ) ^ p := by rw [hdenpow]
  have h_pSeries :
    Summable fun n : ℕ => (1 : ℝ) / (n + 1 : ℝ) ^ p := by
  have h_base :
    Summable fun n : ℕ => (1 : ℝ) / (n : ℝ) ^ p :=
      (Real.summable_one_div_nat_pow (p := p)).mpr hp
  simp [Nat.cast_add, Nat.cast_one] using (summable_nat_add_iff 1).2 h_base
  have h_norm_summable : Summable fun n : ℕ => ‖f n‖ := by
    rw [h_norm]
    exact h_pSeries
  have hfSummable : Summable f := Summable.of_norm h_norm_summable
  refine (⟨∑' n, f n, ?_⟩)
  have hy : HasSum f (∑' n, f n) := hfSummable.hasSum
  have hconv : Tendsto (fun n => ∑ i ∈ Finset.range n, f i) atTop (ℳ (∑' n, f n)) :=
    hy.tendsto_sum_nat
  simp [h, f] using hconv
```

Operator. mutation / mutation_hard **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Shakarchi_exercise_1_19b__mh__0fb1e4c0

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint 0fb1e4c08ebaa52d

4.284 284. Artin_exercise_6_8_1_containment

The problem

Let a and b be elements of a group. Show that a subgroup contains both a and b exactly when it contains both bab^2 and bab^3 .

Lean statement

```
theorem Artin_exercise_6_8_1_containment {G : Type*} [Group G]
  (a b : G) :
  ∀ H : Subgroup G,
    (a ∈ H ∧ b ∈ H) ↔
      (b * a * b ^ 2 ∈ H ∧ b * a * b ^ 3 ∈ H)
```

Parent. Prove that two elements a, b of a group generate the same subgroup as bab^2, bab^3 .

Parent in Lean

```
theorem Artin_exercise_6_8_1 {G : Type*} [Group G]
  (a b : G) : closure ({a, b} : Set G) = Subgroup.closure {b*a*b^2, b*a*b^3} := by
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child is equivalent to the parent: equality of the two subgroup closures holds exactly when every subgroup contains the original generators iff it contains the transformed generators. The universal subgroup-membership characterization substantially changes the surface while preserving the mathematics.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is equivalent to the parent: two elements lie in the same generated subgroup exactly when every subgroup contains one generating pair iff it contains the Artin-transformed pair. It uses a genuine universal-subgroup characterization rather than a cosmetic restatement.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is exactly equivalent to the parent: equality of the two subgroup closures is characterized by every subgroup containing a and b exactly when it contains the two transformed generators. No hypotheses or scope are added or removed, and the universal-subgroup membership formulation is a substantive surface re-encoding.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2285 statements
hypothesis_preservation	parser	every hypothesis the parent had is still there
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem Artin_exercise_6_8_1_containment.extracted_1_1 : ∀ {G : Type u_1} [inst : Group G] (a b : G) (H : Subgroup G),  
  a ∈ H ∧ b ∈ H ↔ b * a * b ^ 2 ∈ H ∧ b * a * b ^ 3 ∈ H
```

Read back by a model that saw only the Lean. For every group G , all elements a and b of G , and every subgroup H of G , a belongs to H and b belongs to H if and only if $b \cdot a \cdot b^2$ belongs to H and $b \cdot a \cdot b^3$ belongs to H .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Function Fintype Subgroup Ideal Polynomial Submodule Zsqrtd
open scoped BigOperators

theorem Artin_exercise_6_8_1_containment {G : Type*} [Group G]
  (a b : G) :
  ∀ H : Subgroup G,
    (a ∈ H ∧ b ∈ H) ↔
      (b * a * b ^ 2 ∈ H ∧ b * a * b ^ 3 ∈ H) := by
  intro H
  constructor
  · rintro ⟨ha, hb⟩
    constructor
    · exact H.mul_mem (H.mul_mem hb ha) (H.pow_mem hb 2)
    · exact H.mul_mem (H.mul_mem hb ha) (H.pow_mem hb 3)
  · rintro ⟨h₁, h₂⟩
    have hb_prod :
      (b * a * b ^ 2)⁻¹ * (b * a * b ^ 3) ∈ H :=
      H.mul_mem (H.inv_mem h₁) h₂
    have hb : b ∈ H := by
      simpa [pow_two, pow_succ, mul_assoc] using hb_prod
    have ha_prod :
      (b⁻¹ * (b * a * b ^ 2)) * (b⁻¹ * b⁻¹) ∈ H :=
      H.mul_mem (H.mul_mem (H.inv_mem hb) h₁)
        (H.mul_mem (H.inv_mem hb) (H.inv_mem hb))
    have ha : a ∈ H := by
      simpa [pow_two, mul_assoc] using ha_prod
    exact ⟨ha, hb⟩
```

Operator. mutation / mutation_silent **Lineage depth.** 0 **Certificate.** reproducible

Problem id. Artin_exercise_6_8_1__ms__132343b1

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, fingerprint 132343b19aa72acf

4.285 285. normalCore_greatest_normal_subgroup_order

The problem

For a subgroup H of a group G , show that its normal core is normal and that, for every normal subgroup N , N is contained in H exactly when it is contained in the normal core of H .

Lean statement

```
theorem normalCore_greatest_normal_subgroup_order {G : Type*} [Group G] (H : Subgroup G) :
  H.normalCore.Normal ∧
  ∀ N : Subgroup G, N.Normal → (N ≤ H ↔ N ≤ H.normalCore)
```

Parent. For a subgroup H of a group G , its normal core is the largest normal subgroup contained in H : it is normal, lies in H , and contains every normal subgroup of G that lies in H .

Parent in Lean

```
theorem normalCore_greatest_normal_subgroup {G : Type*} [Group G] (H : Subgroup G) :
  H.normalCore.Normal ∧ H.normalCore ≤ H ∧
  ∀ N : Subgroup G, N.Normal → N ≤ H → N ≤ H.normalCore
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child is equivalent to the parent: taking $N = H.\text{normalCore}$ in the biconditional recovers $H.\text{normalCore} \leq H$, while the parent gives both directions for every normal N . The biconditional is a genuine order-theoretic re-encoding, not a rename or definitional decoration.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is equivalent to the parent: instantiating its biconditional at $N = H.\text{normalCore}$ recovers $H.\text{normalCore} \leq H$, while the parent supplies the reverse implication for every normal N . The universal order characterization is a genuine surface re-encoding of greatest normal-subgroup maximality.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is equivalent to the parent: its biconditional gives both the maximality direction and $\$N \leq H.\text{normalCore} \rightarrow N \leq H\$$; instantiating it with $\$N = H.\text{normalCore}\$$ recovers $\$H.\text{normalCore} \leq H\$$. The order-theoretic reformulation is a substantive surface change requiring this equivalence to recognize.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2317 statements
hypothesis_preservation	parser	every hypothesis the parent had is still there
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem normalCore_greatest_normal_subgroup_order.extracted_1_1.{u_1} {G : Type u_1} [inst : Group G] (H : Subgroup G) :  
  H.normalCore.Normal  $\wedge$   $\forall$  (N : Subgroup G), N.Normal  $\rightarrow$  (N  $\leq$  H  $\leftrightarrow$  N  $\leq$  H.normalCore)
```

Read back by a model that saw only the Lean. *Let G be a group and H a subgroup of G . Then the normal core of H is a normal subgroup of G , and for every normal subgroup N of G , N is contained in H if and only if N is contained in the normal core of H .*

Lean certificate

```
import Mathlib  
import Aesop  
set_option maxHeartbeats 2000000  
set_option autoImplicit false  
  
open Fintype Subgroup Set Polynomial Ideal  
open scoped BigOperators  
  
theorem normalCore_greatest_normal_subgroup_order {G : Type*} [Group G] (H : Subgroup G) :  
  H.normalCore.Normal  $\wedge$   
     $\forall$  N : Subgroup G, N.Normal  $\rightarrow$  (N  $\leq$  H  $\leftrightarrow$  N  $\leq$  H.normalCore) := by  
  refine (Subgroup.normalCore_normal H, ?_)  
  intro N hN  
  letI : N.Normal := hN  
  exact (Subgroup.normal_le_normalCore (H := H) (N := N)).symm
```

Operator. mutation / mutation_silent **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Dummit_Foote_exercise_3_1_22b__me.ms__d6f85b6a

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Quot.sound, fingerprint d6f85b6aaa5c648d

4.286 286. cyclic_subgroup_card_prime_units

The problem

Let $a > 1$, $n > 0$, and let p be prime. If p divides the number of invertible residue classes modulo $a^n - 1$, prove that these units contain a cyclic subgroup with p elements.

Lean statement

```
theorem cyclic_subgroup_card_prime_units {a n p : ℕ}
  (ha : a > 1) (hn : n > 0) (hp : p.Prime)
  (hcard : p ∣ Nat.card ((ZMod (a ^ n - 1))^x)) :
  ∃ K : Subgroup (ZMod (a ^ n - 1))^x, Nat.card K = p
```

Parent. Let $a > 1$, $n > 0$, and let p be prime. If p divides the Euler totient of $a^n - 1$, prove that the group of units modulo $a^n - 1$ contains a cyclic subgroup of cardinality p .

Parent in Lean

```
theorem cyclic_subgroup_card_prime {a n p : ℕ}
  (ha : a > 1) (hn : n > 0) (hp : p.Prime)
  (hφ : p ∣ (a ^ n - 1).totient) :
  ∃ K : Subgroup (ZMod (a ^ n - 1))^x, Nat.card K = p
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child preserves exactly the same existence theorem: under the existing positivity assumptions, divisibility of the unit group's cardinality is equivalent to divisibility of the totient. Replacing the totient hypothesis with the explicit cardinality of the unit group is a genuine mathematical surface re-encoding.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child preserves all hypotheses and replaces divisibility by the totient with the equivalent divisibility of the unit group's cardinality; the assumptions ensure the required nonzero modulus. This is a genuine surface change requiring recognition of the cardinality–totient equivalence, not mere decoration.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is equivalent under the preserved hypotheses: for the positive modulus ensured by $a > 1$ and $n > 0$, the cardinality of the unit group modulo $a^n - 1$ equals its Euler totient. Replacing totient divisibility with unit-group cardinality is a substantive surface re-encoding requiring the cardinality theorem.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext, Classical.choice, Quot.sound
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2312 statements
hypothesis_preservation	parser	changed: added #6# : #2# Nat.card((ZMod(#0#^#1#-1))^x)
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem cyclic_subgroup_card_prime_units.extracted_1_1 {a n p : ℕ} (ha : a > 1) (hn : n > 0) (hp : Nat.Prime p)
(hcard : p ∣ Nat.card (ZMod (a ^ n - 1))) : ∃ K, Nat.card K = p
```

Read back by a model that saw only the Lean. For natural numbers a , n , and p , if $a > 1$, $n > 0$, p is prime, and p divides the cardinality of the group of units of $Z\text{Mod}(a^n - 1)$, then there exists a subgroup K of that unit group whose cardinality is p .

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Set Real Ideal Polynomial
open scoped BigOperators

theorem cyclic_subgroup_card_prime_units {a n p : ℕ}
  (ha : a > 1) (hn : n > 0) (hp : p.Prime)
  (hcard : p ∣ Nat.card ((ZMod (a ^ n - 1)))^x) :
  ∃ K : Subgroup (ZMod (a ^ n - 1))^x, Nat.card K = p := by
  have hpow : 1 < a ^ n := Nat.one_lt_pow (n := n) (a := a) (by omega) ha
  have hnpos : 0 < a ^ n - 1 := Nat.sub_pos_of_lt hpow
  letI : NeZero (a ^ n - 1) := ⟨Nat.ne_of_gt hnpos⟩
  have hpcard : p ∣ Fintype.card (ZMod (a ^ n - 1))^x := by
    simpa only [Nat.card_eq_fintype_card] using hcard
  letI : Fact p.Prime := ⟨hp⟩
  obtain ⟨u, hu⟩ := exists_prime_orderOf_dvd_card p hpcard
  refine ⟨Subgroup.zpowers u, ?_⟩
  rw [Nat.card_zpowers, hu]
```

Operator. mutation / mutation_silent **Lineage depth.** 2 **Certificate.** reproducible

Problem id. Herstein_exercise_2_4_36__mh.mh.ms__f86c15a2

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, Classical.choice, Quot.sound, fingerprint f86c15a23843e442

4.287 287. conjugate_zpow_exists_exp

The problem

In a group where every subgroup is normal, conjugating any integer power of an element by another element produces a power of the original element. Express this by exhibiting an integer exponent for each conjugate.

Lean statement

```
theorem conjugate_zpow_exists_exp {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :
  ∀ n : ℤ, ∃ k : ℤ, b * a ^ n * b⁻¹ = a ^ k
```

Parent. Let every subgroup of a group G be normal. Show that conjugating any integer power of a by b yields another element of the cyclic subgroup generated by a .

Parent in Lean

```
theorem conjugate_zpow_mem_zpowers {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :
  ∀ n : ℤ, b * a ^ n * b⁻¹ ∈ Subgroup.zpowers a
```

Review

Every judgement this row received. Admission required strong from both passes of the current judge; the generation-time judge saw neither the plan, nor the siblings already kept from these parents, nor which tier the slot asked for, and is recorded rather than counted.

At generation **STRONG**

model gpt-5.6-luna · saw parents, child

The child is mathematically equivalent to the parent because membership in ‘Subgroup.zpowers a’ is exactly the existence of an integer exponent k with the element equal to ‘ $a ^ k$ ’. It preserves all hypotheses and genuinely re-encodes membership as an explicit existential equality.

Re-judged, first pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is exactly equivalent to the parent via ‘ $x \in \text{Subgroup.zpowers } a \leftrightarrow \exists k : \mathbb{Z}, a ^ k = x$ ’; it preserves all hypotheses and quantifiers while replacing subgroup membership with an explicit exponent certificate. This is a substantive Lean-surface re-encoding, not decoration or an alias.

Re-judged, second pass **STRONG**

model gpt-5.6-luna · saw parents, child, plan, kept siblings, operator tier, probe results

The child is equivalent to the parent because membership in ‘Subgroup.zpowers a’ is exactly the existence of an integer k with the element equal to ‘ $a ^ k$ ’. It preserves all hypotheses and changes the surface from subgroup membership to an explicit exponent certificate.

Check	By	Result
statement_type_check	lean	type-checks
proof_accepted	lean	accepted, no sorry
axiom_audit	lean	closed over propext
vacuity	lean	hypotheses are satisfiable
dead_hypotheses	lean	every hypothesis is load-bearing
corpus_dedup	hash	unique against 2291 statements
hypothesis_preservation	parser	every hypothesis the parent had is still there
comparator	lean	statement, axioms and kernel acceptance independently replayed
goal_roundtrip	model	elaborated goal reads back as the stated problem

Goal round-trip: what Lean elaborated

```
theorem conjugate_zpow_exists_exp.extracted_1_1 : ∀ {G : Type u_1} [inst : Group G],  
  (∀ (H : Subgroup G), H.Normal) → ∀ (a b : G) (n : ℤ), ∃ k, b * a ^ n * b-1 = a ^ k
```

Read back by a model that saw only the Lean. *For every group G in which every subgroup is normal, for all elements $a, b \in G$ and every integer n , there exists an integer k such that $b a^n b^{-1} = a^k$.*

Lean certificate

```
import Mathlib
import Aesop
set_option maxHeartbeats 2000000
set_option autoImplicit false
open Fintype Set Real Ideal Polynomial
open scoped BigOperators

theorem conjugate_zpow_exists_exp {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G) :
  ∀ n : ℤ, ∃ k : ℤ, b * a ^ n * b-1 = a ^ k := by
  intro n
  have hHnormal : (Subgroup.zpowers a).Normal := hG (Subgroup.zpowers a)
  have han : a ^ n ∈ Subgroup.zpowers a := Subgroup.zpow_mem_zpowers a n
  have hconj : b * a ^ n * b-1 ∈ Subgroup.zpowers a := by
    exact hHnormal.conj_mem (a ^ n) han b
  rw [Subgroup.mem_zpowers_iff] at hconj
  rcases hconj with ⟨k, hk⟩
  exact ⟨k, hk.symm⟩

theorem conjugate_zpow_exists_exp_forward {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G)
  (h : ∀ n : ℤ, b * a ^ n * b-1 ∈ Subgroup.zpowers a) :
  ∀ n : ℤ, ∃ k : ℤ, b * a ^ n * b-1 = a ^ k := by
  intro n
  have hn := h n
  rw [Subgroup.mem_zpowers_iff] at hn
  rcases hn with ⟨k, hk⟩
  exact ⟨k, hk.symm⟩

theorem conjugate_zpow_exists_exp_backward {G : Type*} [Group G]
  (hG : ∀ (H : Subgroup G), H.Normal) (a b : G)
  (h : ∀ n : ℤ, ∃ k : ℤ, b * a ^ n * b-1 = a ^ k) :
  ∀ n : ℤ, b * a ^ n * b-1 ∈ Subgroup.zpowers a := by
  intro n
  rw [Subgroup.mem_zpowers_iff]
  rcases h n with ⟨k, hk⟩
  exact ⟨k, hk.symm⟩
```

Operator. mutation / mutation_silent **Lineage depth.** 1 **Certificate.** reproducible

Problem id. Herstein_exercise_2_5_23__me.ms__76665334

toolchain leanprover/lean4:v4.30.0-rc2, mathlib 0fb204502963, axioms propext, fingerprint 766653349f82aaa0